

Proprietary Notice

This document is protected by copyright and other related rights and the use or implementation of the information contained in this document may be protected by one or more patents or pending patent applications. No part of this document may be reproduced in any form by any means without the express prior written permission of Arm Limited ("Arm"). **No license, express or implied, by estoppel or otherwise to any intellectual property rights is granted by this document unless specifically stated.**

Your access to the information in this document is conditional upon your acceptance that you will not use or permit others to use the information for the purposes of determining whether the subject matter of this document infringes any third party patents.

The content of this document is informational only. Any solutions presented herein are subject to changing conditions, information, scope, and data. This document was produced using reasonable efforts based on information available as of the date of issue of this document. The scope of information in this document may exceed that which Arm is required to provide, and such additional information is merely intended to further assist the recipient and does not represent Arm's view of the scope of its obligations. You acknowledge and agree that you possess the necessary expertise in system security and functional safety and that you shall be solely responsible for compliance with all legal, regulatory, safety and security related requirements concerning your products, notwithstanding any information or support that may be provided by Arm herein. In addition, you are responsible for any applications which are used in conjunction with any Arm technology described in this document, and to minimize risks, adequate design and operating safeguards should be provided for by you.

This document may include technical inaccuracies or typographical errors. THIS DOCUMENT IS PROVIDED "AS IS". ARM PROVIDES NO REPRESENTATIONS AND NO WARRANTIES, EXPRESS, IMPLIED OR STATUTORY, INCLUDING, WITHOUT LIMITATION, THE IMPLIED WARRANTIES OF MERCHANTABILITY, SATISFACTORY QUALITY, NON-INFRINGEMENT OR FITNESS FOR A PARTICULAR PURPOSE WITH RESPECT TO THE DOCUMENT. For the avoidance of doubt, Arm makes no representation with respect to, and has undertaken no analysis to identify or understand the scope and content of, any patents, copyrights, trade secrets, trademarks, or other rights. TO THE EXTENT NOT PROHIBITED BY LAW, IN NO EVENT WILL ARM BE LIABLE FOR ANY DAMAGES, INCLUDING WITHOUT LIMITATION ANY DIRECT, INDIRECT, SPECIAL, INCIDENTAL, PUNITIVE, OR CONSEQUENTIAL DAMAGES, HOWEVER CAUSED AND REGARDLESS OF THE THEORY OF LIABILITY, ARISING OUT OF ANY USE OF THIS DOCUMENT, EVEN IF ARM HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

Reference by Arm to any third party's products or services within this document is not an express or implied approval or endorsement of the use thereof.

This document consists solely of commercial items. You shall be responsible for ensuring that any permitted use, duplication, or disclosure of this document complies fully with any relevant export laws and regulations to assure that this document or any portion thereof is not exported, directly or indirectly, in violation of such export laws. Use of the word "partner" in reference to Arm's customers is not intended to create or refer to any partnership relationship with any other company. Arm may make changes to this document at any time and without notice.

This document may be translated into other languages for convenience, and you agree that if there is any conflict between the English version of this document and any translation, the terms of the English version of the Agreement shall prevail.

The validity, construction and performance of this notice shall be governed by English Law.

The Arm corporate logo and words marked with ® or ™ are registered trademarks or trademarks of Arm Limited (or its affiliates) in the US and/or elsewhere. Please follow Arm's trademark usage guidelines at <http://www.arm.com/company/policies/trademarks>. All rights reserved. Other brands and names mentioned in this document may be the trademarks of their respective owners.

Copyright © 2010-2025 Arm Limited (or its affiliates). All rights reserved.

Arm Limited. Company 02557590 registered in England.

110 Fulbourn Road, Cambridge, England CB1 9NJ.

LES-PRE-20349

8 March 2024

2025-12_rel2025-09_diff_tag 2025-12-102025-11-28 16:4129:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

A64 -- Base Instructions (alphabetic order)

ABS: Absolute value.

ADC: Add with carry.

ADCS: Add with carry, setting flags.

ADD (extended register): Add extended and scaled register.

ADD (immediate): Add immediate value.

ADD (shifted register): Add optionally-shifted register.

ADDG: Add with tag.

ADDPT: Add checked pointer.

ADDS (extended register): Add extended and scaled register, setting flags.

ADDS (immediate): Add immediate value, setting flags.

ADDS (shifted register): Add optionally-shifted register, setting flags.

ADR: Form PC-relative address.

ADRP: Form PC-relative address to 4KB page.

AND (immediate): Bitwise AND (immediate).

AND (shifted register): Bitwise AND (shifted register).

ANDS (immediate): Bitwise AND (immediate), setting flags.

ANDS (shifted register): Bitwise AND (shifted register), setting flags.

APAS: Associate physical address space: an alias of SYS.

ASR (immediate): Arithmetic shift right (immediate): an alias of SBFM.

ASR (register): Arithmetic shift right (register): an alias of ASRV.

ASRV: Arithmetic shift right variable.

AT: Address translate: an alias of SYS.

AUTDA, AUTDZA: Authenticate data address, using key A.

AUTDB, AUTDZB: Authenticate data address, using key B.

AUTIA, AUTIA1716, AUTIASP, AUTIAZ, AUTIZA: Authenticate instruction address, using key A.

AUTIA171615: Authenticate instruction address, using key A.

AUTIASPPC: Authenticate return address using key A, using an immediate offset.

AUTIASPPCR: Authenticate return address using key A, using a register.

AUTIB, AUTIB1716, AUTIBSP, AUTIBZ, AUTIZB: Authenticate instruction address, using key B.

AUTIB171615: Authenticate instruction address, using key B.

AUTIBSPPC: Authenticate return address using key B, using an immediate offset.

AUTIBSPPCR: Authenticate return address using key B, using a register.

AXFLAG: Convert floating-point condition flags from Arm to external format.

B: Branch.

B.cond: Branch conditionally.

BC.cond: Branch consistent conditionally.

BFC: Bitfield clear: an alias of BFM.

BFI: Bitfield insert: an alias of BFM.

BFM: Bitfield move.

BFXIL: Bitfield extract and insert at low end: an alias of BFM.

BIC (shifted register): Bitwise bit clear (shifted register).

BICS (shifted register): Bitwise bit clear (shifted register), setting flags.

BL: Branch with link.

BLR: Branch with link to register.

BLRAA, BLRAAZ, BLRAB, BLRABZ: Branch with link to register, with pointer authentication.

BR: Branch to register.

BRAA, BRAAZ, BRAB, BRABZ: Branch to register, with pointer authentication.

BRB: Branch record buffer: an alias of SYS.

BRK: Breakpoint instruction.

BTI: Branch target identification.

[CAS, CASA, CASAL, CASL](#): Compare and swap word or doubleword in memory.

[CASB, CASAB, CASALB, CASLB](#): Compare and swap byte in memory.

[CASH, CASHA, CASALH, CASLH](#): Compare and swap halfword in memory.

[CASP, CASPA, CASPAL, CASPL](#): Compare and swap pair of words or doublewords in memory.

[CASPT, CASPAT, CASPALT, CASPLT](#): Compare and swap pair unprivileged.

[CAST, CASAT, CASALT, CASLT](#): Compare and swap unprivileged.

CB<cc> (immediate): Compare register with immediate and branch.

CB<cc> (register): Compare registers and branch.

CBB<cc>: Compare bytes and branch.

CBBLE: Compare signed less than or equal to bytes and branch: an alias of CBB<cc>.

CBBLO: Compare unsigned lower than bytes and branch: an alias of CBB<cc>.

CBBLS: Compare unsigned lower than or same as bytes and branch: an alias of CBB<cc>.

CBBLT: Compare signed less than bytes and branch: an alias of CBB<cc>.

CBGE (immediate): Compare signed greater than or equal to immediate and branch: an alias of CB<cc> (immediate).

CBH<cc>: Compare halfwords and branch.

CBHLE: Compare signed less than or equal to halfwords and branch: an alias of CBH<cc>.

CBHLO: Compare unsigned lower than halfwords and branch: an alias of CBH<cc>.

CBHLS: Compare unsigned lower than or same as halfwords and branch: an alias of CBH<cc>.

CBHLT: Compare signed less than halfwords and branch: an alias of CBH<cc>.

CBHS (immediate): Compare unsigned higher than or same as immediate and branch: an alias of CB<cc> (immediate).

CBLE (immediate): Compare signed less than or equal to immediate and branch: an alias of CB<cc> (immediate).

CBLE (register): Compare signed less than or equal to register and branch: an alias of CB<cc> (register).

CBLO (register): Compare unsigned lower than register and branch: an alias of CB<cc> (register).

CBLS (immediate): Compare unsigned lower than or same as immediate and branch: an alias of CB<cc> (immediate).

CBLS (register): Compare unsigned lower than or same as register and branch: an alias of CB<cc> (register).

CBLT (register): Compare signed less than register and branch: an alias of CB<cc> (register).

CBNZ: Compare and branch on nonzero.

CBZ: Compare and branch on zero.

CCMN (immediate): Conditional compare negative (immediate).

CCMN (register): Conditional compare negative (register).

CCMP (immediate): Conditional compare (immediate).

CCMP (register): Conditional compare (register).

CFINV: Invert carry flag.

CFP: Control flow prediction restriction by context: an alias of SYS.

CHKFEAT: Check feature status.

CINC: Conditional increment: an alias of CSINC.

CINV: Conditional invert: an alias of CSINV.

CLRBHB: Clear branch history.

CLREX: Clear exclusive.

CLS: Count leading sign bits.

CLZ: Count leading zeros.

CMN (extended register): Compare negative (extended register): an alias of ADDS (extended register).

CMN (immediate): Compare negative (immediate): an alias of ADDS (immediate).

CMN (shifted register): Compare negative (shifted register): an alias of ADDS (shifted register).

CMP (extended register): Compare (extended register): an alias of SUBS (extended register).

CMP (immediate): Compare (immediate): an alias of SUBS (immediate).

CMP (shifted register): Compare (shifted register): an alias of SUBS (shifted register).

CMPP: Compare with tag: an alias of SUBPS.

CNEG: Conditional negate: an alias of CSNEG.

CNT: Count bits.

COSP: Clear other speculative prediction restriction by context: an alias of SYS.

CPP: Cache prefetch prediction restriction by context: an alias of SYS.

CPYFP, CPYFM, CPYFE: Memory copy forward-only.

CPYFPN, CPYFMN, CPYFEN: Memory copy forward-only, reads and writes non-temporal.

CPYFPRN, CPYFMRN, CPYFERN: Memory copy forward-only, reads non-temporal.

CPYFPRT, CPYFMRT, CPYFERT: Memory copy forward-only, reads unprivileged.

CPYFPRTN, CPYFMRTN, CPYFERTN: Memory copy forward-only, reads unprivileged, reads and writes non-temporal.

CPYFPRTRN, CPYFMRTRN, CPYFERTRN: Memory copy forward-only, reads unprivileged and non-temporal.

CPYFPRTWN, CPYFMRTWN, CPYFERTWN: Memory copy forward-only, reads unprivileged, writes non-temporal.

CPYFPT, CPYFMT, CPYFET: Memory copy forward-only, reads and writes unprivileged.

CPYFPTN, CPYFMTN, CPYFETN: Memory copy forward-only, reads and writes unprivileged and non-temporal.

CPYFPTRN, CPYFMTRN, CPYFETRN: Memory copy forward-only, reads and writes unprivileged, reads non-temporal.

CPYFPTWN, CPYFMTWN, CPYFETWN: Memory copy forward-only, reads and writes unprivileged, writes non-temporal.

CPYFPWN, CPYFMWN, CPYFEWN: Memory copy forward-only, writes non-temporal.

CPYFPWT, CPYFMWT, CPYFEWT: Memory copy forward-only, writes unprivileged.

CPYFPWTN, CPYFMWTN, CPYFEWTN: Memory copy forward-only, writes unprivileged, reads and writes non-temporal.

CPYFPWTRN, CPYFMWTRN, CPYFEWTRN: Memory copy forward-only, writes unprivileged, reads non-temporal.

CPYFPWTWN, CPYFMWTWN, CPYFEWTWN: Memory copy forward-only, writes unprivileged and non-temporal.

[CPYP, CPYM, CPYE](#): Memory copy.

[CPYPN, CPYMN, CPYEN](#): Memory copy, reads and writes non-temporal.

[CPYPRN, CPYMRN, CPYERN](#): Memory copy, reads non-temporal.

[CPYPRT, CPYMRT, CPYERT](#): Memory copy, reads unprivileged.

[CPYPRTN, CPYMRTN, CPYERTN](#): Memory copy, reads unprivileged, reads and writes non-temporal.

[CPYPRTRN, CPYMRTRN, CPYERTRN](#): Memory copy, reads unprivileged and non-temporal.

[CPYPRTWN, CPYMRTWN, CPYERTWN](#): Memory copy, reads unprivileged, writes non-temporal.

[CPYPT, CPYMT, CPYET](#): Memory copy, reads and writes unprivileged.

[CPYPTN, CPYMTN, CPYETN](#): Memory copy, reads and writes unprivileged and non-temporal.

[CPYPTRN, CPYMRTRN, CPYETRN](#): Memory copy, reads and writes unprivileged, reads non-temporal.

[CPYPTWN, CPYMTWN, CPYETWN](#): Memory copy, reads and writes unprivileged, writes non-temporal.

[CPYPWN, CPYMWN, CPYEWN](#): Memory copy, writes non-temporal.

[CPYPWT, CPYMWI, CPYEWI](#): Memory copy, writes unprivileged.

[CPYPWTN, CPYMWTN, CPYEWTN](#): Memory copy, writes unprivileged, reads and writes non-temporal.

[CPYPWTRN, CPYMWTRN, CPYEWTRN](#): Memory copy, writes unprivileged, reads non-temporal.

[CPYPWTWN, CPYMWWTWN, CPYEWWTWN](#): Memory copy, writes unprivileged and non-temporal.

CRC32B, CRC32H, CRC32W, CRC32X: CRC32 checksum.

CRC32CB, CRC32CH, CRC32CW, CRC32CX: CRC32C checksum.

CSDB: Consumption of speculative data barrier.

CSEL: Conditional select.

CSET: Conditional set: an alias of CSINC.

CSETM: Conditional set mask: an alias of CSINV.

CSINC: Conditional select increment.

CSINV: Conditional select invert.

CSNEG: Conditional select negation.

CTZ: Count trailing zeros.

DC: Data cache operation: an alias of SYS.

DCPS1: Debug change PE state to EL1.

DCPS2: Debug change PE state to EL2.

DCPS3: Debug change PE state to EL3.

DGH: Data gathering hint.

DMB: Data memory barrier.

DRPS: Debug restore PE state.

DSB: Data synchronization barrier.

DVP: Data value prediction restriction by context: an alias of SYS.

EON (shifted register): Bitwise exclusive-OR NOT (shifted register).

EOR (immediate): Bitwise exclusive-OR (immediate).

EOR (shifted register): Bitwise exclusive-OR (shifted register).

ERET: Exception return.

ERETAA, ERETAB: Exception return, with pointer authentication.

ESB: Error synchronization barrier.

EXTR: Extract register.

GCSB: Guarded Control Stack barrier.

GCSPOPCX: Guarded Control Stack pop and compare exception return record: an alias of SYS.

GCSPOPM: Guarded Control Stack pop: an alias of SYSL.

GCSPOPX: Guarded Control Stack pop exception return record: an alias of SYS.

GCSPUSHM: Guarded Control Stack push: an alias of SYS.

GCSPUSHX: Guarded Control Stack push exception return record: an alias of SYS.

GCSSS1: Guarded Control Stack switch stack 1: an alias of SYS.

GCSSS2: Guarded Control Stack switch stack 2: an alias of SYSL.

GCSSTR: Guarded Control Stack store register.

GCSSTTR: Guarded Control Stack store register (unprivileged).

GIC: Generic Interrupt Controller: an alias of SYS.

GICR: Generic Interrupt Controller with result: an alias of SYSL.

GMI: Tag mask insert.

GSB: Generic Interrupt Controller synchronization barrier: an alias of SYS.

HINT: Hint instruction.

HLT: Halt instruction.

HVC: Hypervisor call.

IC: Instruction cache operation: an alias of SYS.

IRG: Insert random tag.

ISB: Instruction synchronization barrier.

LD64B: Single-copy atomic 64-byte Load.

LDADD, LDADDA, LDADDAL, LDADDL: Atomic add on word or doubleword.

LDADDB, LDADDAB, LDADDALB, LDADDLB: Atomic add on byte.

LDADDH, LDADDAH, LDADDALH, LDADDLH: Atomic add on halfword.

LDAP: Load-acquire pair of registers.

LDAPP: Load-acquire RCpc pair of registers.

LDAPR: Load-acquire RCpc register.

LDAPRB: Load-acquire RCpc register byte.

LDAPRH: Load-acquire RCpc register halfword.

LDAPUR: Load-acquire RCpc register (unscaled).

LDAPURB: Load-acquire RCpc register byte (unscaled).

LDAPURH: Load-acquire RCpc register halfword (unscaled).

LDAPURSB: Load-acquire RCpc register signed byte (unscaled).

LDAPURSH: Load-acquire RCpc register signed halfword (unscaled).

LDAPURSW: Load-acquire RCpc register signed word (unscaled).

LDAR: Load-acquire register.

LDARB: Load-acquire register byte.

LDARH: Load-acquire register halfword.

LDATXR: Load-acquire unprivileged exclusive register.

LDAXP: Load-acquire exclusive pair of registers.

LDAXR: Load-acquire exclusive register.

LDAXRB: Load-acquire exclusive register byte.

LDAXRH: Load-acquire exclusive register halfword.

LDCLR, LDCLRA, LDCLRAL, LDCLRL: Atomic bit clear on word or doubleword.

LDCLRB, LDCLRAB, LDCLRALB, LDCLRLB: Atomic bit clear on byte.

LDCLRH, LDCLRAH, LDCLRALH, LDCLRLH: Atomic bit clear on halfword.

LDCLRP, LDCLRPA, LDCLRPAL, LDCLRPL: Atomic bit clear on quadword.

LDEOR, LDEORA, LDEORAL, LDEORL: Atomic exclusive-OR on word or doubleword.

LDEORB, LDEORAB, LDEORALB, LDEORLB: Atomic exclusive-OR on byte.

LDEORH, LDEORAH, LDEORALH, LDEORLH: Atomic exclusive-OR on halfword.

LDG: Load Allocation Tag.

LDGM: Load tag multiple.

LDIAPP: Load-Acquire RCpc ordered pair of registers.

LDLAR: Load LOAcquire register.

LDLARB: Load LOAcquire register byte.

LDLARH: Load LOAcquire register halfword.

LDNP: Load pair of registers, with non-temporal hint.

LDP: Load pair of registers.

LDPSW: Load pair of registers signed word.

LDR (immediate): Load register (immediate).

LDR (literal): Load register (literal).

LDR (register): Load register (register).

LDRAA, LDRAB: Load register, with pointer authentication.

LDRB (immediate): Load register byte (immediate).

LDRB (register): Load register byte (register).

LDRH (immediate): Load register halfword (immediate).

LDRH (register): Load register halfword (register).

LDRSB (immediate): Load register signed byte (immediate).

LDRSB (register): Load register signed byte (register).

LDRSH (immediate): Load register signed halfword (immediate).

LDRSH (register): Load register signed halfword (register).

LDRSW (immediate): Load register signed word (immediate).

LDRSW (literal): Load register signed word (literal).

LDRSW (register): Load register signed word (register).

LDSET, LDSETA, LDSETAL, LDSETL: Atomic bit set on word or doubleword.

LDSETB, LDSETAB, LDSETALB, LDSETLB: Atomic bit set on byte.

LDSETH, LDSETAH, LDSETALH, LDSETLH: Atomic bit set on halfword.

LDSETP, LDSETPA, LDSETPAL, LDSETPL: Atomic bit set on quadword.

LDSMAX, LDSMAXA, LDSMAXAL, LDSMAXL: Atomic signed maximum on word or doubleword.

LDSMAXB, LDSMAXAB, LDSMAXALB, LDSMAXLB: Atomic signed maximum on byte.

LDSMAXH, LDSMAXAH, LDSMAXALH, LDSMAXLH: Atomic signed maximum on halfword.

LDSMIN, LDSMINA, LDSMINAL, LDSMINL: Atomic signed minimum on word or doubleword.

LDSMINB, LDSMINAB, LDSMINALB, LDSMINLB: Atomic signed minimum on byte.

LDSMINH, LDSMINAH, LDSMINALH, LDSMINLH: Atomic signed minimum on halfword.

LDTADD, LDTADDA, LDTADDAL, LDTADDL: Atomic add unprivileged.

LDTCLR, LDTCLRA, LDTCLRAL, LDTCLRL: Atomic bit clear unprivileged.

LDTNP: Load unprivileged pair of registers, with non-temporal hint.

LDTP: Load unprivileged pair of registers.

LDTR: Load register (unprivileged).

LDTRB: Load register byte (unprivileged).

LDTRH: Load register halfword (unprivileged).

LDTRSB: Load register signed byte (unprivileged).

LDTRSH: Load register signed halfword (unprivileged).

LDTRSW: Load register signed word (unprivileged).

LDTSET, LDTSETA, LDTSETAL, LDTSETL: Atomic bit set unprivileged.

LDTXR: Load unprivileged exclusive register.

LDUMAX, LDUMAXA, LDUMAXAL, LDUMAXL: Atomic unsigned maximum on word or doubleword.

LDUMAXB, LDUMAXAB, LDUMAXALB, LDUMAXLB: Atomic unsigned maximum on byte.

LDUMAXH, LDUMAXAH, LDUMAXALH, LDUMAXLH: Atomic unsigned maximum on halfword.

LDUMIN, LDUMINA, LDUMINAL, LDUMINL: Atomic unsigned minimum on word or doubleword.

LDUMINB, LDUMINAB, LDUMINALB, LDUMINLB: Atomic unsigned minimum on byte.

LDUMINH, LDUMINAH, LDUMINALH, LDUMINLH: Atomic unsigned minimum on halfword.

LDUR: Load register (unscaled).

LDURB: Load register byte (unscaled).

LDURH: Load register halfword (unscaled).

LDURSB: Load register signed byte (unscaled).

LDURSH: Load register signed halfword (unscaled).

LDURSW: Load register signed word (unscaled).

LDXP: Load exclusive pair of registers.

LDXR: Load exclusive register.

LDXRB: Load exclusive register byte.

LDXRH: Load exclusive register halfword.

LSL (immediate): Logical shift left (immediate): an alias of UBFM.

LSL (register): Logical shift left (register): an alias of LSLV.

LSLV: Logical shift left variable.

LSR (immediate): Logical shift right (immediate): an alias of UBFM.

LSR (register): Logical shift right (register): an alias of LSRV.

LSRV: Logical shift right variable.

MADD: Multiply-add.

MADDPT: Multiply-add checked pointer.

[MLBI](#): MPAM lookaside buffer invalidate: an alias of SYS.

MNEG: Multiply-negate: an alias of MSUB.

MOV (bitmask immediate): Move bitmask immediate value: an alias of ORR (immediate).

MOV (inverted wide immediate): Move inverted wide immediate value: an alias of MOVN.

MOV (register): Move register value: an alias of ORR (shifted register).

MOV (to/from SP): Move register value to or from SP: an alias of ADD (immediate).

MOV (wide immediate): Move wide immediate value: an alias of MOVZ.

MOVK: Move wide with keep.

MOVN: Move wide with NOT.

MOVZ: Move wide with zero.

MRRS: Move System register to two adjacent general-purpose registers.

MRS: Move System register to general-purpose register.

MSR (immediate): Move immediate value to special register.

MSR (register): Move general-purpose register to System register.

MSRR: Move two adjacent general-purpose registers to System register.

MSUB: Multiply-subtract.

MSUBPT: Multiply-subtract checked pointer.

MUL: Multiply: an alias of MADD.

MVN: Bitwise NOT: an alias of ORN (shifted register).

NEG (shifted register): Negate (shifted register): an alias of SUB (shifted register).

NEGS: Negate, setting flags: an alias of SUBS (shifted register).

NGC: Negate with carry: an alias of SBC.

NGCS: Negate with carry, setting flags: an alias of SBCS.

NOP: No operation.

ORN (shifted register): Bitwise OR NOT (shifted register).

ORR (immediate): Bitwise OR (immediate).

ORR (shifted register): Bitwise OR (shifted register).

PACDA, PACDZA: Pointer Authentication Code for data address, using key A.

PACDB, PACDZB: Pointer Authentication Code for data address, using key B.

PACGA: Pointer Authentication Code, using generic key.

PACIA, PACIA1716, PACIASP, PACIAZ, PACIZA: Pointer Authentication Code for instruction address, using key A.

PACIA171615: Pointer Authentication Code for instruction address, using key A.

PACIASPPC: Pointer Authentication Code for return address, using key A.

PACIB, PACIB1716, PACIBSP, PACIBZ, PACIZB: Pointer Authentication Code for instruction address, using key B.

PACIB171615: Pointer Authentication Code for instruction address, using key B.

PACIBSPPC: Pointer Authentication Code for return address, using key B.

PACM: Pointer authentication modifier.

PACNBIASPPC: Pointer Authentication Code for return address, using key A, not a branch target.

PACNBIBSPPC: Pointer Authentication Code for return address, using key B, not a branch target.

PRFM (immediate): Prefetch memory (immediate).

PRFM (literal): Prefetch memory (literal).

PRFM (register): Prefetch memory (register).

PRFUM: Prefetch memory (unscaled offset).

PSB: Profiling synchronization barrier.

PSSBB: Physical speculative store bypass barrier: an alias of DSB.

RBIT: Reverse bits.

RCWCAS, RCWCASA, RCWCASAL, RCWCASL: Read check write compare and swap doubleword in memory.

RCWCASP, RCWCASPA, RCWCASPAL, RCWCASPL: Read check write compare and swap quadword in memory.

RCWCLR, RCWCLRA, RCWCLRAL, RCWCLRL: Read check write atomic bit clear on doubleword in memory.

RCWCLRP, RCWCLRPA, RCWCLRPA, RCWCLRPL: Read check write atomic bit clear on quadword in memory.

RCWSCAS, RCWSCASA, RCWSCASAL, RCWSCASL: Read check write software compare and swap doubleword in memory.

RCWSCASP, RCWSCASPA, RCWSCASPAL, RCWSCASPL: Read check write software compare and swap quadword in memory.

RCWSCLR, RCWSCLRA, RCWSCLRAL, RCWSCLRL: Read check write software atomic bit clear on doubleword in memory.

RCWSCLRP, RCWSCLRPA, RCWSCLRPA, RCWSCLRPL: Read check write software atomic bit clear on quadword in memory.

[RCWSET, RCWSETA, RCWSETAL, RCWSETL](#): Read check write atomic bit set on doubleword in memory.

RCWSETP, RCWSETPA, RCWSETPAL, RCWSETPL: Read check write atomic bit set on quadword in memory.

[RCWSSET, RCWSSETA, RCWSSETAL, RCWSSETL](#): Read check write software atomic bit set on doubleword in memory.

RCWSSETP, RCWSSETPA, RCWSSETPAL, RCWSSETPL: Read check write software atomic bit set on quadword in memory.

RCWSSWP, RCWSSWPA, RCWSSWPAL, RCWSSWPL: Read check write software swap doubleword in memory.

RCWSSWPP, RCWSSWPPA, RCWSSWPPAL, RCWSSWPPPL: Read check write software swap quadword in memory.

RCWSWP, RCWSWPA, RCWSWPAL, RCWSWPL: Read check write swap doubleword in memory.

RCWSWPP, RCWSWPPA, RCWSWPPAL, RCWSWPPPL: Read check write swap quadword in memory.

RET: Return from subroutine.

RETA, RETAB: Return from subroutine, with pointer authentication.

RETAASPPC, RETABSPPC: Return from subroutine, with enhanced pointer authentication using an immediate offset.

RETAASPPCR, RETABSPPCR: Return from subroutine, with enhanced pointer authentication using a register.

REV: Reverse bytes.

REV16: Reverse bytes in 16-bit halfwords.

REV32: Reverse bytes in 32-bit words.

REV64: Reverse bytes: an alias of REV.

RMIF: Rotate, mask insert flags.

ROR (immediate): Rotate right (immediate): an alias of EXTR.

ROR (register): Rotate right (register): an alias of RORV.

RORV: Rotate right variable.

RPRFM: Range prefetch memory.

SB: Speculation barrier.

SBC: Subtract with carry.

SBCS: Subtract with carry, setting flags.

SBFIZ: Signed bitfield insert in zeros: an alias of SBFM.

SBFM: Signed bitfield move.

SBFX: Signed bitfield extract: an alias of SBFM.

SDIV: Signed divide.

SETF8, SETF16: Evaluation of 8-bit or 16-bit flag values.

[SETGP, SETGM, SETGE](#): Memory set with tag setting.

[SETGPN, SETGMN, SETGEN](#): Memory set with tag setting, non-temporal.

[SETGPT, SETGMT, SETGET](#): Memory set with tag setting, unprivileged.

[SETGPTN, SETGMTN, SETGETN](#): Memory set with tag setting, unprivileged and non-temporal.

[SETP, SETM, SETE](#): Memory set.

[SETPN, SETMN, SETEN](#): Memory set, non-temporal.

[SETPT, SETMT, SETET](#): Memory set, unprivileged.

[SETPTN, SETMTN, SETETN](#): Memory set, unprivileged and non-temporal.

SEV: Send event.

SEVL: Send event local.

SHUH: Shared update hint.

SMADDL: Signed multiply-add long.

SMAx (immediate): Signed maximum (immediate).

SMAx (register): Signed maximum (register).

SMC: Secure monitor call.

SMIN (immediate): Signed minimum (immediate).

SMIN (register): Signed minimum (register).

SMNEGL: Signed multiply-negate long: an alias of SMSUBL.

SMSTART: Enables access to Streaming SVE mode and SME architectural state: an alias of MSR (immediate).

SMSTOP: Disables access to Streaming SVE mode and SME architectural state: an alias of MSR (immediate).

SMSUBL: Signed multiply-subtract long.

SMULH: Signed multiply high.

SMULL: Signed multiply long: an alias of SMADDL.

SSBB: Speculative store bypass barrier: an alias of DSB.

ST2G: Store Allocation Tags.

ST64B: Single-copy atomic 64-byte store without status result.

ST64BV: Single-copy atomic 64-byte store with status result.

ST64BV0: Single-copy atomic 64-byte EL0 store with status result.

STADD, STADDL: Atomic add on word or doubleword, without return: an alias of LDADD, LDADDA, LDADDAL, LDADDL.

STADDB, STADDLB: Atomic add on byte, without return: an alias of LDADDB, LDADDAB, LDADDALB, LDADDLB.

STADDH, STADDLH: Atomic add on halfword, without return: an alias of LDADDH, LDADDAH, LDADDALH, LDADDLH.

STCLR, STCLRL: Atomic bit clear on word or doubleword, without return: an alias of LDCLR, LDCLRA, LDCLRAL, LDCLRL.

STCLRB, STCLRLB: Atomic bit clear on byte, without return: an alias of LDCLRB, LDCLRAB, LDCLRALB, LDCLRLB.

STCLRH, STCLRLH: Atomic bit clear on halfword, without return: an alias of LDCLRH, LDCLRAH, LDCLRALH, LDCLRLH.

STCPH: Store concurrent priority hint.

STEOR, STEORL: Atomic exclusive-OR on word or doubleword, without return: an alias of LDEOR, LDEORA, LDEORAL, LDEORL.

STEORB, STEORLB: Atomic exclusive-OR on byte, without return: an alias of LDEORB, LDEORAB, LDEORALB, LDEORLB.

STEORH, STEORLH: Atomic exclusive-OR on halfword, without return: an alias of LDEORH, LDEORAH, LDEORALH, LDEORLH.

STG: Store Allocation Tag.

STGM: Store Allocation Tag multiple.

STGP: Store Allocation Tag and pair of registers.

STILP: Store-release ordered pair of registers.

STLLR: Store LORelease register.

STLLRB: Store LORelease register byte.

STLLRH: Store LORelease register halfword.

STLP: Store-release pair of registers.

STLR: Store-release register.

STLRB: Store-release register byte.

STLRH: Store-release register halfword.

STLTXR: Store-release unprivileged exclusive register.

STLUR: Store-release register (unscaled).

STLURB: Store-release register byte (unscaled).

STLURH: Store-release register halfword (unscaled).

STLXP: Store-release exclusive pair of registers.

STLXR: Store-release exclusive register.

STLXRB: Store-release exclusive register byte.

STLXRH: Store-release exclusive register halfword.

STNP: Store pair of registers, with non-temporal hint.

STP: Store pair of registers.

STR (immediate): Store register (immediate).

STR (register): Store register (register).

STRB (immediate): Store register byte (immediate).

STRB (register): Store register byte (register).

STRH (immediate): Store register halfword (immediate).

STRH (register): Store register halfword (register).

STSET, STSETL: Atomic bit set on word or doubleword, without return: an alias of LDSET, LDSETA, LDSETAL, LDSETL.

STSETB, STSETLB: Atomic bit set on byte, without return: an alias of LDSETB, LDSETAB, LDSETALB, LDSETLB.

STSETH, STSETLH: Atomic bit set on halfword, without return: an alias of LDSETH, LDSETHA, LDSETALH, LDSETLH.

STSHH: Store shared hint.

STSMAX, STSMAXL: Atomic signed maximum on word or doubleword, without return: an alias of LDSMAX, LDSMAXA, LDSMAXAL, LDSMAXL.

STSMAXB, STSMAXLB: Atomic signed maximum on byte, without return: an alias of LDSMAXB, LDSMAXAB, LDSMAXALB, LDSMAXLB.

STSMAXH, STSMAXLH: Atomic signed maximum on halfword, without return: an alias of LDSMAXH, LDSMAXAH, LDSMAXALH, LDSMAXLH.

STSMIN, STSMINL: Atomic signed minimum on word or doubleword, without return: an alias of LDSMIN, LDSMINA, LDSMINAL, LDSMINL.

STSMINB, STSMINLB: Atomic signed minimum on byte, without return: an alias of LDSMINB, LDSMINAB, LDSMINALB, LDSMINLB.

STSMINH, STSMINLH: Atomic signed minimum on halfword, without return: an alias of LDSMINH, LDSMINAH, LDSMINALH, LDSMINLH.

STTADD, STTADDL: Atomic add unprivileged, without return: an alias of LDTADD, LDTADDA, LDTADDAL, LDTADDL.

STTCLR, STTCLRL: Atomic bit clear unprivileged, without return: an alias of LDTCLR, LDTCLRA, LDTCLRAL, LDTCLRL.

STTNP: Store unprivileged pair of registers, with non-temporal hint.

STTP: Store unprivileged pair of registers.

STTR: Store register (unprivileged).

STTRB: Store register byte (unprivileged).

STTRH: Store register halfword (unprivileged).

STTSET, STTSETL: Atomic bit set unprivileged, without return: an alias of LDTSET, LDTSETA, LDTSETAL, LDTSETL.

STTXR: Store unprivileged exclusive register.

STUMAX, STUMAXL: Atomic unsigned maximum on word or doubleword, without return: an alias of LDUMAX, LDUMAXA, LDUMAXAL, LDUMAXL.

STUMAXB, STUMAXLB: Atomic unsigned maximum on byte, without return: an alias of LDUMAXB, LDUMAXAB, LDUMAXALB, LDUMAXLB.

STUMAXH, STUMAXLH: Atomic unsigned maximum on halfword, without return: an alias of LDUMAXH, LDUMAXAH, LDUMAXALH, LDUMAXLH.

STUMIN, STUMINL: Atomic unsigned minimum on word or doubleword, without return: an alias of LDUMIN, LDUMINA, LDUMINAL, LDUMINL.

STUMINB, STUMINLB: Atomic unsigned minimum on byte, without return: an alias of LDUMINB, LDUMINAB, LDUMINALB, LDUMINLB.

STUMINH, STUMINLH: Atomic unsigned minimum on halfword, without return: an alias of LDUMINH, LDUMINAH, LDUMINALH, LDUMINLH.

STUR: Store register (unscaled).

STURB: Store register byte (unscaled).

STURH: Store register halfword (unscaled).

STXP: Store exclusive pair of registers.

STXR: Store exclusive register.

STXRB: Store exclusive register byte.

STXRH: Store exclusive register halfword.

STZ2G: Store Allocation Tags, zeroing.

STZG: Store Allocation Tag, zeroing.

STZGM: Store Allocation Tag and zero multiple.

SUB (extended register): Subtract extended and scaled register.

SUB (immediate): Subtract immediate value.

SUB (shifted register): Subtract optionally-shifted register.

SUBG: Subtract with tag.

SUBP: Subtract pointer.

SUBPS: Subtract pointer, setting flags.

SUBPT: Subtract checked pointer.

SUBS (extended register): Subtract extended and scaled register, setting flags.

SUBS (immediate): Subtract immediate value, setting flags.

SUBS (shifted register): Subtract optionally-shifted register, setting flags.

SVC: Supervisor call.

SWP, SWPA, SWPAL, SWPL: Swap word or doubleword in memory.

SWPB, SWPAB, SWPALB, SWPLB: Swap byte in memory.

SWPH, SWPAH, SWPALH, SWPLH: Swap halfword in memory.

SWPP, SWPPA, SWPPAL, SWPPL: Swap quadword in memory.

SWPT, SWPTA, SWPTAL, SWPTL: Swap unprivileged.

SXTB: Signed extend byte: an alias of SBFM.

SXTH: Sign extend halfword: an alias of SBFM.

SXTW: Sign extend word: an alias of SBFM.

[SYS](#): System instruction.

SYSL: System instruction with result.

SYSP: 128-bit system instruction.

TBNZ: Test bit and branch if nonzero.

TBZ: Test bit and branch if zero.

TLBI: TLB invalidate operation: an alias of SYS.

TLBIP: TLB invalidate pair operation: an alias of SYSP.

TRCIT: Trace instrumentation: an alias of SYS.

TSB: Trace synchronization barrier.

TST (immediate): Test bits (immediate): an alias of ANDS (immediate).

TST (shifted register): Test (shifted register): an alias of ANDS (shifted register).

UBFIZ: Unsigned bitfield insert in zeros: an alias of UBFM.

UBFM: Unsigned bitfield move.

UBFX: Unsigned bitfield extract: an alias of UBFM.

UDF: Permanently undefined.

UDIV: Unsigned divide.

UMADDL: Unsigned multiply-add long.

UMAX (immediate): Unsigned maximum (immediate).

UMAX (register): Unsigned maximum (register).

UMIN (immediate): Unsigned minimum (immediate).

UMIN (register): Unsigned minimum (register).

UMNEGL: Unsigned multiply-negate long: an alias of UMSUBL.

UMSUBL: Unsigned multiply-subtract long.

UMULH: Unsigned multiply high.

UMULL: Unsigned multiply long: an alias of UMADDL.

UXTB: Unsigned extend byte: an alias of UBFM.

UXTH: Unsigned extend halfword: an alias of UBFM.

WFE: Wait for event.

WFET: Wait for event with timeout.

WFI: Wait for interrupt.

WFIT: Wait for interrupt with timeout.

XAFLAG: Convert floating-point condition flags from external format to Arm format.

XPACD, XPACI, XPACLRI: Strip Pointer Authentication Code.

YIELD: Yield.

(old)	htmldiff from-	(new)
-------	----------------	-------

CAS, CASA, CASAL, CASL

Compare and swap word or doubleword in memory

This instruction reads a 32-bit word or 64-bit doubleword from memory, and compares it against the value held in a first register. If the comparison is equal, the value in a second register is written to memory. If the comparison is not equal, the architecture permits writing the value read from the location to memory. If the write is performed, the read and write occur atomically such that no other modification of the memory location can take place between the read and write.

- If the destination register is not one of WZR or XZR, *CASA* and *CASAL* load from memory with acquire semantics.
- *CASL* and *CASAL* store to memory with release semantics.
- *CAS* has neither acquire nor release semantics.

The architecture permits that the data read clears any exclusive monitors associated with that location, even if the compare subsequently fails.

If the instruction generates a synchronous Data Abort, the register which is compared and loaded, that is <Ws>, or <Xs>, is restored to the value held in the register before the instruction was executed.

For a *CAS* or *CASA* instruction, when <Ws> or <Xs> specifies the same register as <Wt> or <Xt>, this signals to the memory system that an additional subsequent *CAS*, *CASA*, *CASAL*, or *CASL* access to the specified location is **expected** to occur in the near **future in the absence of taking an exception or exceptional software conditions**. The memory system can respond by taking actions that are expected to enable the subsequent *CAS*, *CASA*, *CASAL*, or *CASL* access to succeed when it does occur.

A code sequence starting with a *CAS* or *CASA* instruction for which <Ws> or <Xs> specifies the same register as <Wt> or <Xt>, and ending with a subsequent *CAS*, *CASA*, *CASAL*, or *CASL* to the same location, exhibits the following properties for best performance when the location may be accessed concurrently, on one or more other PEs:

- The sequence does not contain any direct system register writes, address translation instructions, cache or TLB maintenance operations, exception producing instructions, exception returns, or ISB barriers.
- The execution of the sequence includes 32 or fewer instructions.
- The value provided in <Ws> or <Xs> of the first *CAS* or *CASA* is a value likely to result in the comparison failing. A failing comparison result may lead to better performance due to the hardware not performing a write to memory.

Note

For a *CAS* or *CASA* instruction, when <Ws> or <Xs> specifies the same register as <Wt> or <Xt>, the value in memory is not modified, because the *CAS* or *CASA* either fails its compare or writes the same value back to memory.

For performance reasons, Arm recommends that software not use a *CAS* or *CASA* instruction when <Ws> or <Xs> specifies the same register as <Wt> or <Xt> in a routine that includes a subsequent *CAS*, *CASA*, *CASAL*, or *CASL* for any of the following purposes:

- To load the value that will be compared to conditionally determine whether or not to execute the subsequent *CAS*, *CASA*, *CASAL*, or *CASL*.
- As a form of prefetch executed between the instruction loading the value that will be compared and the subsequent compare instruction that conditionally determines whether or not to execute the subsequent *CAS*, *CASA*, *CASAL*, or *CASL*.

This guidance does not apply when <Ws> or <Xs> specifies the same register as <Wt> or <Xt> for a *CAS* or *CASA* instruction and the compare between the *CAS* or *CASA* and the subsequent *CAS*, *CASA*, *CASAL*, or *CASL* is intended to detect an exceptional software condition such that the result of the compare does not prevent the subsequent *CAS*, *CASA*, *CASAL*, or *CASL* from executing under normal operating conditions.

For more information about memory ordering semantics, see [Load-Acquire, Store-Release](#).

For information about addressing modes, see [Load/Store addressing modes](#).

No offset

(FEAT_LSE)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	x	0	0	1	0	0	0	1	L	1	Rs					o0	1	1	1	1	1	Rn					Rt				
size																				Rt2											

Encoding for the 32-bit CAS variant

Applies when (size == 10 && L == 0 && o0 == 0)

CAS <Ws>, <Wt>, [<Xn|SP>{, #0}]

Encoding for the 32-bit CASA variant

Applies when (size == 10 && L == 1 && o0 == 0)

CASA <Ws>, <Wt>, [<Xn|SP>{, #0}]

Encoding for the 32-bit CASAL variant

Applies when (size == 10 && L == 1 && o0 == 1)

CASAL <Ws>, <Wt>, [<Xn|SP>{, #0}]

Encoding for the 32-bit CASL variant

Applies when (size == 10 && L == 0 && o0 == 1)

CASL <Ws>, <Wt>, [<Xn|SP>{, #0}]

Encoding for the 64-bit CAS variant

Applies when (size == 11 && L == 0 && o0 == 0)

CAS <Xs>, <Xt>, [<Xn|SP>{, #0}]

Encoding for the 64-bit CASA variant

Applies when (size == 11 && L == 1 && o0 == 0)

CASA <Xs>, <Xt>, [<Xn|SP>{, #0}]

Encoding for the 64-bit CASAL variant

Applies when (size == 11 && L == 1 && o0 == 1)

CASAL <Xs>, <Xt>, [<Xn|SP>{, #0}]

Encoding for the 64-bit CASL variant

Applies when (size == 11 && L == 0 && o0 == 1)

CASL <Xs>, <Xt>, [<Xn|SP>{, #0}]

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 8 << UInt(size);
let regsize : integer{} = if datasize == 64 then 64 else 32;
let acquire : boolean = L == '1' && t != 31;
let release : boolean = o0 == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register to be compared and loaded, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be conditionally stored, encoded in the "Rt" field.
<Xn SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xs>	Is the 64-bit name of the general-purpose register to be compared and loaded, encoded in the "Rs" field.
<Xt>	Is the 64-bit name of the general-purpose register to be conditionally stored, encoded in the "Rt" field.

Operation

```
var address : bits(64);
var comparevalue : bits(datasize);
var newvalue : bits(datasize);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_CAS, acquire, release,
tagchecked, privileged, t, s);

comparevalue = X{datasize}(s);
newvalue = X{datasize}(t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let data : bits(datasize) = MemAtomic{}(address, comparevalue, newvalue, accdesc);
X{regsize}(s) = ZeroExtend{regsize}(data);
```

2025-12_rel2025-09_diff_tag 2025-12-102025-11-28 16:4129:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)

htmldiff from-

(new)

CASB, CASAB, CASALB, CASLB

Compare and swap byte in memory

This instruction reads an 8-bit byte from memory, and compares it against the value held in a first register. If the comparison is equal, the value in a second register is written to memory. If the comparison is not equal, the architecture permits writing the value read from the location to memory. If the write is performed, the read and write occur atomically such that no other modification of the memory location can take place between the read and write.

- If the destination register is not one of WZR or XZR, CASAB and CASALB load from memory with acquire semantics.
- CASLB and CASALB store to memory with release semantics.
- CASB has neither acquire nor release semantics.

The architecture permits that the data read clears any exclusive monitors associated with that location, even if the compare subsequently fails.

If the instruction generates a synchronous Data Abort, the register which is compared and loaded, that is <Ws>, is restored to the values held in the register before the instruction was executed.

For a CASB or CASAB instruction, when <Ws> or <Xs> specifies the same register as <Wt> or <Xt>, this signals to the memory system that an additional subsequent CASB, CASAB, CASALB, or CASLB access to the specified location is **expected** to occur in the near **future in the absence of taking an exception or exceptional software conditions**. The memory system can respond by taking actions that are expected to enable the subsequent CASB, CASAB, CASALB, or CASLB access to succeed when it does occur.

A code sequence starting with a CASB or CASAB instruction for which <Ws> or <Xs> specifies the same register as <Wt> or <Xt>, and ending with a subsequent CASB, CASAB, CASALB, or CASLB to the same location, exhibits the following properties for best performance when the location may be accessed concurrently, on one or more other PEs:

- The sequence does not contain any direct system register writes, address translation instructions, cache or TLB maintenance operations, exception producing instructions, exception returns, or ISB barriers.
- The execution of the sequence includes 32 or fewer instructions.
- The value provided in <Ws> or <Xs> of the first CASB or CASAB is a value likely to result in the comparison failing. A failing comparison result may lead to better performance due to the hardware not performing a write to memory.

Note

For a CASB or CASAB instruction, when <Ws> or <Xs> specifies the same register as <Wt> or <Xt>, the value in memory is not modified, because the CASB or CASAB either fails its compare or writes the same value back to memory.

For performance reasons, Arm recommends that software not use a CASB or CASAB instruction when <Ws> or <Xs> specifies the same register as <Wt> or <Xt> in a routine that includes a subsequent CASB, CASAB, CASALB, or CASLB for any of the following purposes:

- To load the value that will be compared to conditionally determine whether or not to execute the subsequent CASB, CASAB, CASALB, or CASLB.
- As a form of prefetch executed between the instruction loading the value that will be compared and the subsequent compare instruction that conditionally determines whether or not to execute the subsequent CASB, CASAB, CASALB, or CASLB.

This guidance does not apply when <Ws> or <Xs> specifies the same register as <Wt> or <Xt> for a CASB or CASAB instruction and the compare between the CASB or CASAB and the subsequent CASB, CASAB, CASALB, or CASLB is intended to detect an exceptional software condition such that the result of the compare does not prevent the subsequent CASB, CASAB, CASALB, or CASLB from executing under normal operating conditions.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.

For information about addressing modes, see *Load/Store addressing modes*.

No offset

(FEAT_LSE)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	1	0	0	0	1	L	1	Rs				o0	1	1	1	1	1	Rn				Rt						
size										Rt2																					

Encoding for the CASB variant

Applies when (L == 0 && o0 == 0)

CASB <Ws>, <Wt>, [<Xn|SP>{, #0}]

Encoding for the CASAB variant

Applies when (L == 1 && o0 == 0)

CASAB <Ws>, <Wt>, [<Xn|SP>{, #0}]

Encoding for the CASALB variant

Applies when (L == 1 && o0 == 1)

CASALB <Ws>, <Wt>, [<Xn|SP>{, #0}]

Encoding for the CASLB variant

Applies when (L == 0 && o0 == 1)

CASLB <Ws>, <Wt>, [<Xn|SP>{, #0}]

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let acquire : boolean = L == '1' && t != 31;
let release : boolean = o0 == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register to be compared and loaded, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be conditionally stored, encoded in the "Rt" field.
<Xn SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);
var comparevalue : bits(8);
var newvalue : bits(8);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_CAS, acquire, release,
tagchecked, privileged, t, s);

comparevalue = X{8}(s);
newvalue = X{8}(t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let data : bits(8) = MemAtomic{}(address, comparevalue, newvalue, accdesc);
X{32}(s) = ZeroExtend{32}(data);
```

2025-12_rel2025-09_diff_tag 2025-12-102025-11-28 16:4129:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)

htmldiff from-

(new)

CASH, CASAH, CASALH, CASLH

Compare and swap halfword in memory

This instruction reads a 16-bit halfword from memory, and compares it against the value held in a first register. If the comparison is equal, the value in a second register is written to memory. If the comparison is not equal, the architecture permits writing the value read from the location to memory. If the write is performed, the read and write occur atomically such that no other modification of the memory location can take place between the read and write.

- If the destination register is not one of WZR or XZR, CASAH and CASALH load from memory with acquire semantics.
- CASLH and CASALH store to memory with release semantics.
- CASH has neither acquire nor release semantics.

The architecture permits that the data read clears any exclusive monitors associated with that location, even if the compare subsequently fails.

If the instruction generates a synchronous Data Abort, the register which is compared and loaded, that is <Ws>, is restored to the values held in the register before the instruction was executed.

For a CASH or CASAH instruction, when <Ws> or <Xs> specifies the same register as <Wt> or <Xt>, this signals to the memory system that an additional subsequent CASH, CASAH, CASALH, or CASLH access to the specified location is **expected** to occur in the near **future in the absence of taking an exception or exceptional software conditions**. The memory system can respond by taking actions that are expected to enable the subsequent CASH, CASAH, CASALH, or CASLH access to succeed when it does occur.

A code sequence starting with a CASH or CASAH instruction for which <Ws> or <Xs> specifies the same register as <Wt> or <Xt>, and ending with a subsequent CASH, CASAH, CASALH, or CASLH to the same location, exhibits the following properties for best performance when the location may be accessed concurrently, on one or more other PEs:

- The sequence does not contain any direct system register writes, address translation instructions, cache or TLB maintenance operations, exception producing instructions, exception returns, or ISB barriers.
- The execution of the sequence includes 32 or fewer instructions.
- The value provided in <Ws> or <Xs> of the first CASH or CASAH is a value likely to result in the comparison failing. A failing comparison result may lead to better performance due to the hardware not performing a write to memory.

Note

For a CASH or CASAH instruction, when <Ws> or <Xs> specifies the same register as <Wt> or <Xt>, the value in memory is not modified, because the CASH or CASAH either fails its compare or writes the same value back to memory.

For performance reasons, Arm recommends that software not use a CASH or CASAH instruction when <Ws> or <Xs> specifies the same register as <Wt> or <Xt> in a routine that includes a subsequent CASH, CASAH, CASALH, or CASLH for any of the following purposes:

- To load the value that will be compared to conditionally determine whether or not to execute the subsequent CASH, CASAH, CASALH, or CASLH.
- As a form of prefetch executed between the instruction loading the value that will be compared and the subsequent compare instruction that conditionally determines whether or not to execute the subsequent CASH, CASAH, CASALH, or CASLH.

This guidance does not apply when <Ws> or <Xs> specifies the same register as <Wt> or <Xt> for a CASH or CASAH instruction and the compare between the CASH or CASAH and the subsequent CASH, CASAH, CASALH, or CASLH is intended to detect an exceptional software condition such that the result of the compare does not prevent the subsequent CASH, CASAH, CASALH, or CASLH from executing under normal operating conditions.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.

For information about addressing modes, see *Load/Store addressing modes*.

No offset

(FEAT_LSE)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0	1	0	0	1	0	0	0	1	L	1	Rs					o0	1	1	1	1	1	Rn					Rt					
size											Rt2																					

Encoding for the CASH variant

Applies when (L == 0 && o0 == 0)

CASH <Ws>, <Wt>, [<Xn|SP>{, #0}]

Encoding for the CASAH variant

Applies when (L == 1 && o0 == 0)

CASAH <Ws>, <Wt>, [<Xn|SP>{, #0}]

Encoding for the CASALH variant

Applies when (L == 1 && o0 == 1)

CASALH <Ws>, <Wt>, [<Xn|SP>{, #0}]

Encoding for the CASLH variant

Applies when (L == 0 && o0 == 1)

CASLH <Ws>, <Wt>, [<Xn|SP>{, #0}]

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let acquire : boolean = L == '1' && t != 31;
let release : boolean = o0 == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

- <Ws> Is the 32-bit name of the general-purpose register to be compared and loaded, encoded in the "Rs" field.
- <Wt> Is the 32-bit name of the general-purpose register to be conditionally stored, encoded in the "Rt" field.
- <Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);
var comparevalue : bits(16);
var newvalue : bits(16);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_CAS, acquire, release,
tagchecked, privileged, t, s);

comparevalue = X{16}(s);
newvalue = X{16}(t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let data : bits(16) = MemAtomic{}(address, comparevalue, newvalue, accdesc);
X{32}(s) = ZeroExtend{32}(data);
```

2025-12_rel2025-09_diff_tag 2025-12-102025-11-28 16:4129:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)	htmldiff from-	(new)
-------	----------------	-------

CASP, CASPA, CASPAL, CASPL

Compare and swap pair of words or doublewords in memory

This instruction reads a pair of 32-bit words or 64-bit doublewords from memory, and compares them against the values held in the first pair of registers. If the comparison is equal, the values in the second pair of registers are written to memory. If the comparison is not equal, the architecture permits writing the value read from the location to memory. If the writes are performed, the reads and writes occur atomically such that no other modification of the memory location can take place between the reads and writes.

- CASPA and CASPAL load from memory with acquire semantics.
- CASPL and CASPAL store to memory with release semantics.
- CASP has neither acquire nor release semantics.

The architecture permits that the data read clears any exclusive monitors associated with that location, even if the compare subsequently fails. If the instruction generates a synchronous Data Abort, the registers which are compared and loaded, that is $\langle Ws \rangle$ and $\langle W(s+1) \rangle$, or $\langle Xs \rangle$ and $\langle X(s+1) \rangle$, are restored to the values held in the registers before the instruction was executed.

For a CASP or CASPA instruction, when $\langle Ws \rangle$ or $\langle Xs \rangle$ specifies the same register as $\langle Wt \rangle$ or $\langle Xt \rangle$, this signals to the memory system that an additional subsequent CASP, CASPA, CASPAL, or CASPL access to the specified location is **expected** to occur in the near **future in the absence of taking an exception or exceptional software conditions**. The memory system can respond by taking actions that are expected to enable the subsequent CASP, CASPA, CASPAL, or CASPL access to succeed when it does occur.

A code sequence starting with a CASP or CASPA instruction for which $\langle Ws \rangle$ or $\langle Xs \rangle$ specifies the same register as $\langle Wt \rangle$ or $\langle Xt \rangle$, and ending with a subsequent CASP, CASPA, CASPAL, or CASPL to the same location, exhibits the following properties for best performance when the location may be accessed concurrently, on one or more other PEs:

- The sequence does not contain any direct system register writes, address translation instructions, cache or TLB maintenance operations, exception producing instructions, exception returns, or ISB barriers.
- The execution of the sequence includes 32 or fewer instructions.
- The value provided in $\langle Ws \rangle$ or $\langle Xs \rangle$ of the first CASP or CASPA is a value likely to result in the comparison failing. A failing comparison result may lead to better performance due to the hardware not performing a write to memory.

Note

For a CASP or CASPA instruction, when $\langle Ws \rangle$ or $\langle Xs \rangle$ specifies the same register as $\langle Wt \rangle$ or $\langle Xt \rangle$, the value in memory is not modified, because the CASP or CASPA either fails its compare or writes the same value back to memory.

For performance reasons, Arm recommends that software not use a CASP or CASPA instruction when $\langle Ws \rangle$ or $\langle Xs \rangle$ specifies the same register as $\langle Wt \rangle$ or $\langle Xt \rangle$ in a routine that includes a subsequent CASP, CASPA, CASPAL, or CASPL for any of the following purposes:

- To load the value that will be compared to conditionally determine whether or not to execute the subsequent CASP, CASPA, CASPAL, or CASPL.
- As a form of prefetch executed between the instruction loading the value that will be compared and the subsequent compare instruction that conditionally determines whether or not to execute the subsequent CASP, CASPA, CASPAL, or CASPL.

This guidance does not apply when $\langle Ws \rangle$ or $\langle Xs \rangle$ specifies the same register as $\langle Wt \rangle$ or $\langle Xt \rangle$ for a CASP or CASPA instruction and the compare between the CASP or CASPA and the subsequent CASP, CASPA, CASPAL, or CASPL is intended to detect an exceptional software condition such that the result of the compare does not prevent the subsequent CASP, CASPA, CASPAL, or CASPL from executing under normal operating conditions.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.

For information about addressing modes, see *Load/Store addressing modes*.

No offset

(FEAT_LSE)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	SZ	0	0	1	0	0	0	0	L	1	Rs				o0	1	1	1	1	1	Rn				Rt						
Rt2																															

Rt2

Encoding for the 32-bit CASP variant

Applies when $(sz == 0 \ \&\& \ L == 0 \ \&\& \ o0 == 0)$

CASP $\langle Ws \rangle$, $\langle W(s+1) \rangle$, $\langle Wt \rangle$, $\langle W(t+1) \rangle$, [$\langle Xn|SP \rangle\{, \#0\}$]

Encoding for the 32-bit CASPA variant

Applies when $(sz == 0 \ \&\& \ L == 1 \ \&\& \ o0 == 0)$

CASPA $\langle Ws \rangle$, $\langle W(s+1) \rangle$, $\langle Wt \rangle$, $\langle W(t+1) \rangle$, [$\langle Xn|SP \rangle\{, \#0\}$]

Encoding for the 32-bit CASPAL variant

Applies when $(sz == 0 \ \&\& \ L == 1 \ \&\& \ o0 == 1)$

```
CASPAL <Ws>, <W(s+1)>, <Wt>, <W(t+1)>, [<Xn|SP>{, #0}]
```

Encoding for the 32-bit CASPL variant

Applies when (sz == 0 && L == 0 && o0 == 1)

```
CASPL <Ws>, <W(s+1)>, <Wt>, <W(t+1)>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit CASP variant

Applies when (sz == 1 && L == 0 && o0 == 0)

```
CASP <Xs>, <X(s+1)>, <Xt>, <X(t+1)>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit CASPA variant

Applies when (sz == 1 && L == 1 && o0 == 0)

```
CASPA <Xs>, <X(s+1)>, <Xt>, <X(t+1)>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit CASPAL variant

Applies when (sz == 1 && L == 1 && o0 == 1)

```
CASPAL <Xs>, <X(s+1)>, <Xt>, <X(t+1)>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit CASPL variant

Applies when (sz == 1 && L == 0 && o0 == 1)

```
CASPL <Xs>, <X(s+1)>, <Xt>, <X(t+1)>, [<Xn|SP>{, #0}]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;
if Rs[0] == '1' || Rt[0] == '1' then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 32 << UInt(sz);
let acquire : boolean = L == '1';
let release : boolean = o0 == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the first general-purpose register to be compared and loaded, encoded in the "Rs" field. <Ws> must be an even-numbered register.
<W(s+1)>	Is the 32-bit name of the second general-purpose register to be compared and loaded.
<Wt>	Is the 32-bit name of the first general-purpose register to be conditionally stored, encoded in the "Rt" field. <Wt> must be an even-numbered register.
<W(t+1)>	Is the 32-bit name of the second general-purpose register to be conditionally stored.
<Xn SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xs>	Is the 64-bit name of the first general-purpose register to be compared and loaded, encoded in the "Rs" field. <Xs> must be an even-numbered register.
<X(s+1)>	Is the 64-bit name of the second general-purpose register to be compared and loaded.
<Xt>	Is the 64-bit name of the first general-purpose register to be conditionally stored, encoded in the "Rt" field. <Xt> must be an even-numbered register.
<X(t+1)>	Is the 64-bit name of the second general-purpose register to be conditionally stored.

Operation

```
var address : bits(64);
var comparevalue : bits(2*datasize);
var newvalue : bits(2*datasize);
var data : bits(2*datasize);

let s1 : bits(datasize) = X{}(s);
let s2 : bits(datasize) = X{}(s+1);
let t1 : bits(datasize) = X{}(t);
let t2 : bits(datasize) = X{}(t+1);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp\_CAS, acquire, release,
tagchecked, privileged, t, t+1, s, s+1);

comparevalue = if BigEndian(accdesc.acctype) then s1::s2 else s2::s1;
newvalue      = if BigEndian(accdesc.acctype) then t1::t2 else t2::t1;
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

data = MemAtomic{2*datasize}(address, comparevalue, newvalue, accdesc);

if BigEndian(accdesc.acctype) then
    X{datasize}(s)    = data[2*datasize-1:datasize];
    X{datasize}(s+1) = data[datasize-1:0];
else
    X{datasize}(s)    = data[datasize-1:0];
    X{datasize}(s+1) = data[2*datasize-1:datasize];
end;
```

2025-12-re12025-09-diff-tag 2025-12-102025-11-28 16:4129:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)

htmldiff from-

(new)

CASPT, CASPAT, CASPALT, CASPLT

Compare and swap pair unprivileged

This instruction reads a pair of 64-bit doublewords from memory, and compares them against the values held in the first pair of registers. If the comparison is equal, the values in the second pair of registers are written to memory. If the comparison is not equal, the architecture permits writing the value read from the location to memory. If the writes are performed, the reads and writes occur atomically such that no other modification of the memory location can take place between the reads and writes.

- CASPAT and CASPALT load from memory with acquire semantics.
- CASPLT and CASPALT store to memory with release semantics.
- CASPT has neither acquire nor release semantics.

The architecture permits that the data read clears any exclusive monitors associated with that location, even if the compare subsequently fails.

If the instruction generates a synchronous Data Abort, the registers which are compared and loaded, that is <Xs> and <X(s+1)>, are restored to the values held in the registers before the instruction was executed.

Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of HCR_EL2().{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

For a CASPT or CASPAT instruction, when <Ws> or <Xs> specifies the same register as <Wt> or <Xt>, this signals to the memory system that an additional subsequent CASPT, CASPAT, CASPALT, or CASPLT access to the specified location is *expected* to occur in the near *future in the absence of taking an exception or exceptional software conditions*. The memory system can respond by taking actions that are expected to enable the subsequent CASPT, CASPAT, CASPALT, or CASPLT access to succeed when it does occur.

A code sequence starting with a CASPT or CASPAT instruction for which <Ws> or <Xs> specifies the same register as <Wt> or <Xt>, and ending with a subsequent CASPT, CASPAT, CASPALT, or CASPLT to the same location, exhibits the following properties for best performance when the location may be accessed concurrently, on one or more other PEs:

- The sequence does not contain any direct system register writes, address translation instructions, cache or TLB maintenance operations, exception producing instructions, exception returns, or ISB barriers.
- The execution of the sequence includes 32 or fewer instructions.
- The value provided in <Ws> or <Xs> of the first CASPT or CASPAT is a value likely to result in the comparison failing. A failing comparison result may lead to better performance due to the hardware not performing a write to memory.

Note

For a CASPT or CASPAT instruction, when <Ws> or <Xs> specifies the same register as <Wt> or <Xt>, the value in memory is not modified, because the CASPT or CASPAT either fails its compare or writes the same value back to memory.

For performance reasons, Arm recommends that software not use a CASPT or CASPAT instruction when <Ws> or <Xs> specifies the same register as <Wt> or <Xt> in a routine that includes a subsequent CASPT, CASPAT, CASPALT, or CASPLT for any of the following purposes:

- To load the value that will be compared to conditionally determine whether or not to execute the subsequent CASPT, CASPAT, CASPALT, or CASPLT.
- As a form of prefetch executed between the instruction loading the value that will be compared and the subsequent compare instruction that conditionally determines whether or not to execute the subsequent CASPT, CASPAT, CASPALT, or CASPLT.

This guidance does not apply when <Ws> or <Xs> specifies the same register as <Wt> or <Xt> for a CASPT or CASPAT instruction and the compare between the CASPT or CASPAT and the subsequent CASPT, CASPAT, CASPALT, or CASPLT is intended to detect an exceptional software condition such that the result of the compare does not prevent the subsequent CASPT, CASPAT, CASPALT, or CASPLT from executing under normal operating conditions.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.

For information about addressing modes, see *Load/Store addressing modes*.

No offset

(FEAT_LSUI)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	1	0	0	1	1	L	0	Rs					o0	1	1	1	1	1	Rn					Rt				
SZ										Rt2																					

Encoding for the CASPT variant

Applies when (L == 0 && o0 == 0)

CASPT <Xs>, <X(s+1)>, <Xt>, <X(t+1)>, [<Xn|SP>{, #0}]

Encoding for the CASPAT variant

Applies when (L == 1 && o0 == 0)

CASPAT <Xs>, <X(s+1)>, <Xt>, <X(t+1)>, [<Xn|SP>{, #0}]

Encoding for the CASPALT variant

Applies when (L == 1 && o0 == 1)

CASPALT <Xs>, <X(s+1)>, <Xt>, <X(t+1)>, [<Xn|SP>{, #0}]

Encoding for the CASPLT variant

Applies when (L == 0 && o0 == 1)

CASPLT <Xs>, <X(s+1)>, <Xt>, <X(t+1)>, [<Xn|SP>{, #0}]

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSUI) then EndOfDecode(Decode_UNDEF); end;
if Rs[0] == '1' || Rt[0] == '1' then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 64;
let acquire : boolean = L == '1';
let release : boolean = o0 == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Xs>	Is the 64-bit name of the first general-purpose register to be compared and loaded, encoded in the "Rs" field. <Xs> must be an even-numbered register.
<X(s+1)>	Is the 64-bit name of the second general-purpose register to be compared and loaded.
<Xt>	Is the 64-bit name of the first general-purpose register to be conditionally stored, encoded in the "Rt" field. <Xt> must be an even-numbered register.
<X(t+1)>	Is the 64-bit name of the second general-purpose register to be conditionally stored.
<Xn SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);
var comparevalue : bits(2*datasize);
var newvalue : bits(2*datasize);
var data : bits(2*datasize);

let s1 : bits(datasize) = X{}(s);
let s2 : bits(datasize) = X{}(s+1);
let t1 : bits(datasize) = X{}(t);
let t2 : bits(datasize) = X{}(t+1);

let privileged : boolean = AArch64_IsUnprivAccessPriv();
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_CAS, acquire, release,
tagchecked, privileged, t, t+1, s, s+1);

comparevalue = if BigEndian(accdesc.acctype) then s1::s2 else s2::s1;
newvalue = if BigEndian(accdesc.acctype) then t1::t2 else t2::t1;
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

data = MemAtomic{2*datasize}(address, comparevalue, newvalue, accdesc);

if BigEndian(accdesc.acctype) then
    X{datasize}(s) = data[2*datasize-1:datasize];
    X{datasize}(s+1) = data[datasize-1:0];
else
    X{datasize}(s) = data[datasize-1:0];
    X{datasize}(s+1) = data[2*datasize-1:datasize];
end;
```

2025-12_rel2025-09_diff_tag 2025-12-102025-11-28 16:41:29:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)

htmldiff from-

(new)

CAST, CASAT, CASALT, CASLT

Compare and swap unprivileged

This instruction reads a 64-bit doubleword from memory, and compares it against the value held in a first register. If the comparison is equal, the value in a second register is written to memory. If the comparison is not equal, the architecture permits writing the value read from the location to memory. If the write is performed, the read and write occur atomically such that no other modification of the memory location can take place between the read and write.

- If the destination register is not one of WZR or XZR, `CASAT` and `CASALT` load from memory with acquire semantics.
- `CASLT` and `CASALT` store to memory with release semantics.
- `CAST` has neither acquire nor release semantics.

The architecture permits that the data read clears any exclusive monitors associated with that location, even if the compare subsequently fails.

If the instruction generates a synchronous Data Abort, the register which is compared and loaded, that is `<Xs>`, is restored to the value held in the register before the instruction was executed.

Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of `PSTATE.UAO` is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of `HCR_EL2().{E2H, TGE}` is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

For a `CAST` or `CASAT` instruction, when `<Ws>` or `<Xs>` specifies the same register as `<Wt>` or `<Xt>`, this signals to the memory system that an additional subsequent `CAST`, `CASAT`, `CASALT`, or `CASLT` access to the specified location is *expected* to occur in the near *future in the absence of taking an exception or exceptional software conditions*. The memory system can respond by taking actions that are expected to enable the subsequent `CAST`, `CASAT`, `CASALT`, or `CASLT` access to succeed when it does occur.

A code sequence starting with a `CAST` or `CASAT` instruction for which `<Ws>` or `<Xs>` specifies the same register as `<Wt>` or `<Xt>`, and ending with a subsequent `CAST`, `CASAT`, `CASALT`, or `CASLT` to the same location, exhibits the following properties for best performance when the location may be accessed concurrently, on one or more other PEs:

- The sequence does not contain any direct system register writes, address translation instructions, cache or TLB maintenance operations, exception producing instructions, exception returns, or `ISB` barriers.
- The execution of the sequence includes 32 or fewer instructions.
- The value provided in `<Ws>` or `<Xs>` of the first `CAST` or `CASAT` is a value likely to result in the comparison failing. A failing comparison result may lead to better performance due to the hardware not performing a write to memory.

Note

For a `CAST` or `CASAT` instruction, when `<Ws>` or `<Xs>` specifies the same register as `<Wt>` or `<Xt>`, the value in memory is not modified, because the `CAST` or `CASAT` either fails its compare or writes the same value back to memory.

For performance reasons, Arm recommends that software not use a `CAST` or `CASAT` instruction when `<Ws>` or `<Xs>` specifies the same register as `<Wt>` or `<Xt>` in a routine that includes a subsequent `CAST`, `CASAT`, `CASALT`, or `CASLT` for any of the following purposes:

- To load the value that will be compared to conditionally determine whether or not to execute the subsequent `CAST`, `CASAT`, `CASALT`, or `CASLT`.
- As a form of prefetch executed between the instruction loading the value that will be compared and the subsequent compare instruction that conditionally determines whether or not to execute the subsequent `CAST`, `CASAT`, `CASALT`, or `CASLT`.

This guidance does not apply when `<Ws>` or `<Xs>` specifies the same register as `<Wt>` or `<Xt>` for a `CAST` or `CASAT` instruction and the compare between the `CAST` or `CASAT` and the subsequent `CAST`, `CASAT`, `CASALT`, or `CASLT` is intended to detect an exceptional software condition such that the result of the compare does not prevent the subsequent `CAST`, `CASAT`, `CASALT`, or `CASLT` from executing under normal operating conditions.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.

For information about addressing modes, see *Load/Store addressing modes*.

No offset

(FEAT_LSUI)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
1	1	0	0	1	0	0	1	1	L	0	Rs				o0	1	1	1	1	1	Rn				Rt										
SZ										Rt2																									

Encoding for the CAST variant

Applies when `(L == 0 && o0 == 0)`

CAST <Xs>, <Xt>, [<Xn|SP>{, #0}]

Encoding for the CASAT variant

Applies when (L == 1 && o0 == 0)

CASAT <Xs>, <Xt>, [<Xn|SP>{, #0}]

Encoding for the CASALT variant

Applies when (L == 1 && o0 == 1)

CASALT <Xs>, <Xt>, [<Xn|SP>{, #0}]

Encoding for the CASLT variant

Applies when (L == 0 && o0 == 1)

CASLT <Xs>, <Xt>, [<Xn|SP>{, #0}]

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSUI) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let acquire : boolean = L == '1' && t != 31;
let release : boolean = o0 == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Xs>	Is the 64-bit name of the general-purpose register to be compared and loaded, encoded in the "Rs" field.
<Xt>	Is the 64-bit name of the general-purpose register to be conditionally stored, encoded in the "Rt" field.
<Xn SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);
var comparevalue : bits(64);
var newvalue : bits(64);

let privileged : boolean = AArch64_IsUnprivAccessPriv();
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_CAS, acquire, release,
tagchecked, privileged, t, s);

comparevalue = X{64}(s);
newvalue = X{64}(t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

X{64}(s) = MemAtomic{64}(address, comparevalue, newvalue, accdesc);
```

2025-12_rel2025-09_diff_tag 2025-12-102025-11-28 16:4129:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)

htmldiff from-

(new)

CPYP, CPYM, CPYE

Memory copy

These instructions copy a requested number of bytes in memory from a source address to a destination address. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYP, then CPYM, and then CPYE.

CPYP performs some preconditioning of the arguments suitable for using the CPYM instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYM copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYE copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see [Memory Copy and Memory Set exceptions](#).

The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYP:

- If $Xn[63:55] \neq '000000000'$, the copy size Xn is saturated to $0x007FFFFFFFFFFFFFFF$.
- After saturation is performed, the direction of the memory copy is based on the following:
If $(Xs[55:0] > Xd[55:0])$ and $(Xd[55:0] + \text{saturated copy size}) > Xs[55:0]$, then the direction is forward.
If $(Xs[55:0] < Xd[55:0])$ and $(Xs[55:0] + \text{saturated copy size}) > Xd[55:0]$, then the direction is backward.
Otherwise, the direction is an IMPLEMENTATION DEFINED choice between forward and backward.

On completion of CPYP, option A:

- $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,0,0\}$.
- If the copy is in the forward direction, then:
 - Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the original Xs + saturated copy size.
 - Xd holds the original Xd + saturated copy size.
- If the copy is in the backward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs and Xd are unchanged.

On completion of CPYP, option B:

- If the copy is in the forward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
 - $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,1,0\}$.
- If the copy is in the backward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.
 - $PSTATE.\{N,Z,C,V\}$ are set to $\{1,0,1,0\}$.

For CPYM, option A, when $PSTATE.C = 0$:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
 - Xn holds -1 times the number of bytes remaining to be copied.
 - Xs holds the lowest address to be copied from - Xn .
 - Xd holds the lowest address to be copied to - Xn .
 - On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.
- If the copy is in the backward direction (Xn holds a positive number), then:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address to be copied from + 1 - Xn .
 - Xd holds the highest address to be copied to + 1 - Xn .
 - On completion of the instruction, Xn holds the number of bytes remaining to be copied.

For CPYM, option B, when $PSTATE.C = 1$:

- Xn holds the number of bytes remaining to be copied.

- If the copy is in the forward direction ($PSTATE.N == 0$), then:
 - Xs holds the lowest address to be copied from.
 - Xd holds the lowest address to be copied to.
 - On completion of the instruction:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
- If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - Xs holds the highest address to be copied from + 1.
 - Xd holds the highest address to be copied to + 1.
 - On completion of the instruction:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.

For CPYE, option A, when $PSTATE.C = 0$:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
 - Xn holds -1 times the number of bytes remaining to be copied.
 - Xs holds the lowest address to be copied from - Xn.
 - Xd holds the lowest address to be copied to - Xn.
 - On completion of the instruction, Xn holds 0.
- If the copy is in the backward direction (Xn holds a positive number), then:
 - If the copy is in the backward direction (Xn holds a positive number), then:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address to be copied from + 1 - Xn.
 - Xd holds the highest address to be copied to + 1 - Xn.
 - On completion of the instruction, Xn holds 0.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied from.
 - The highest address that has been copied from + 1.
 - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied to.
 - The highest address that has not been copied to + 1.

For CPYE, option B, when $PSTATE.C = 1$:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction ($PSTATE.N == 0$), then:
 - Xs holds the lowest address to be copied from.
 - Xd holds the lowest address to be copied to.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
- If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - Xs holds the highest address to be copied from + 1.
 - Xd holds the highest address to be copied to + 1.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied from.
 - The highest address that has been copied from + 1.
 - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied to.
 - The highest address that has not been copied to + 1.

Integer

(FEAT_MOPS)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0						
sz	0	1	1	1	0	1	op1	0	Rs							0	0	0	0	0	1	Rn							Rd								
op0							op2																														

Encoding for the Prologue variant

Applies when ($op1 == 00$)

```
CPYP [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when (op1 == 01)

```
CPYM [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when (op1 == 10)

```
CPYE [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rntemporal : boolean = options[3] == '1';
let wntemporal : boolean = options[2] == '1';
case op1 of
  when '00' => memcpy.stage = MOPSStage_Prologue;
  when '01' => memcpy.stage = MOPSStage_Main;
  when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the `CONSTRAINED UNPREDICTABLE` behavior of this instruction, see [Architectural Constraints on UNPREDICTABLE behaviors](#), and particularly [Memory Copy and Memory Set CPY*](#) and [Crossing a page boundary with different memory types or Shareability attributes](#).

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.


```

CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv      = PSTATE.[N,Z,C,V];
memcpy.toaddress = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);

if memcpy.stage == MOPSTage_Prologue then
    memcpy.cpy_size = UInt(X{64}(memcpy.n));
else
    memcpy.cpy_size = SInt(X{64}(memcpy.n));
end;

memcpy.implements_option_a = CPYOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
    else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
    else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD, rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSTage_Prologue then
    if memcpy.cpy_size > ArchMaxMOPSCPysize then
        memcpy.cpy_size = ArchMaxMOPSCPysize;
    end;

    memcpy.forward = IsMemCpyForward(memcpy);

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        if memcpy.forward then
            // Copy in the forward direction offsets the arguments.
            memcpy.toaddress = memcpy.toaddress + memcpy.cpy_size;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpy_size;
            memcpy.cpy_size = 0 - memcpy.cpy_size;
        end;
    else
        if !memcpy.forward then
            // Copy in the reverse direction offsets the arguments.
            memcpy.toaddress = memcpy.toaddress + memcpy.cpy_size;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpy_size;
            memcpy.nzcv = '1010';
        else
            memcpy.nzcv = '0010';
        end;
    end;
end;

memcpy.stagecpy_size = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSTage_Prologue then
    memcpy.forward = memcpy.cpy_size < 0 || (!memcpy.implements_option_a && memcpy.nzcv[3] == '0');
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var fault : boolean = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpy_size != 0 && !fault looplimit ArchMaxMOPSCPysize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPySizeChoice(memcpy);

        if memcpy.forward then
            assert B <= -1 * memcpy.stagecpy_size;
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                memcpy.toaddress + memcpy.cpy_size,
                memcpy.fromaddress + memcpy.cpy_size,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.cpy_size = memcpy.cpy_size + B;
                memcpy.stagecpy_size = memcpy.stagecpy_size + B;
            end;
        else
            assert B <= memcpy.stagecpy_size;
            memcpy.cpy_size = memcpy.cpy_size - B;
            memcpy.stagecpy_size = memcpy.stagecpy_size - B;
        end;
    end;
end;

```

```

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
            memcpy.toaddress + memcpy.cpy_size,
            memcpy.fromaddress + memcpy.cpy_size,
            memcpy.forward, B, raccdesc,
            waccdesc);

    if copied != B then
        fault = TRUE;
        memcpy.cpy_size = memcpy.cpy_size + B;
        memcpy.stagecpy_size = memcpy.stagecpy_size + B;
    end;
end;
end;

else
    while memcpy.stagecpy_size > 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpy_size;

        if memcpy.forward then
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                memcpy.fromaddress,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress + B;
                memcpy.toaddress = memcpy.toaddress + B;
            end;
        else
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress - B,
                memcpy.fromaddress - B,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress - B;
                memcpy.toaddress = memcpy.toaddress - B;
            end;
        end;

        if !fault then
            memcpy.cpy_size = memcpy.cpy_size - B;
            memcpy.stagecpy_size = memcpy.stagecpy_size - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;

```

2025-12_rel2025-09_diff_tag 2025-12-102025-11-28 16:4129:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)

htmldiff from-

(new)

CPYPN, CPYMN, CPYEN

Memory copy, reads and writes non-temporal

These instructions copy a requested number of bytes in memory from a source address to a destination address. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYPN, then CPYMN, and then CPYEN.

CPYPN performs some preconditioning of the arguments suitable for using the CPYMN instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYMN copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYEN copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see [Memory Copy and Memory Set exceptions](#).

The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYPN:

- If $Xn[63:55] \neq '000000000'$, the copy size Xn is saturated to $0x007FFFFFFFFFFFFFFF$.
- After saturation is performed, the direction of the memory copy is based on the following:
If $(Xs[55:0] > Xd[55:0])$ and $(Xd[55:0] + \text{saturated copy size}) > Xs[55:0]$, then the direction is forward.
If $(Xs[55:0] < Xd[55:0])$ and $(Xs[55:0] + \text{saturated copy size}) > Xd[55:0]$, then the direction is backward.
Otherwise, the direction is an IMPLEMENTATION DEFINED choice between forward and backward.

On completion of CPYPN, option A:

- $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,0,0\}$.
- If the copy is in the forward direction, then:
 - Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the original Xs + saturated copy size.
 - Xd holds the original Xd + saturated copy size.
- If the copy is in the backward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs and Xd are unchanged.

On completion of CPYPN, option B:

- If the copy is in the forward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
 - $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,1,0\}$.
- If the copy is in the backward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.
 - $PSTATE.\{N,Z,C,V\}$ are set to $\{1,0,1,0\}$.

For CPYMN, option A, when $PSTATE.C = 0$:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
 - Xn holds -1 times the number of bytes remaining to be copied.
 - Xs holds the lowest address to be copied from - Xn .
 - Xd holds the lowest address to be copied to - Xn .
 - On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.
- If the copy is in the backward direction (Xn holds a positive number), then:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address to be copied from + 1 - Xn .
 - Xd holds the highest address to be copied to + 1 - Xn .
 - On completion of the instruction, Xn holds the number of bytes remaining to be copied.

For CPYMN, option B, when $PSTATE.C = 1$:

- Xn holds the number of bytes remaining to be copied.

- If the copy is in the forward direction ($PSTATE.N == 0$), then:
 - Xs holds the lowest address to be copied from.
 - Xd holds the lowest address to be copied to.
 - On completion of the instruction:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
- If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - Xs holds the highest address to be copied from + 1.
 - Xd holds the highest address to be copied to + 1.
 - On completion of the instruction:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.

For CPYEN, option A, when $PSTATE.C = 0$:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
 - Xn holds -1 times the number of bytes remaining to be copied.
 - Xs holds the lowest address to be copied from - Xn.
 - Xd holds the lowest address to be copied to - Xn.
 - On completion of the instruction, Xn holds 0.
- If the copy is in the backward direction (Xn holds a positive number), then:
 - If the copy is in the backward direction (Xn holds a positive number), then:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address to be copied from + 1 - Xn.
 - Xd holds the highest address to be copied to + 1 - Xn.
 - On completion of the instruction, Xn holds 0.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied from.
 - The highest address that has been copied from + 1.
 - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied to.
 - The highest address that has not been copied to + 1.

For CPYEN, option B, when $PSTATE.C = 1$:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction ($PSTATE.N == 0$), then:
 - Xs holds the lowest address to be copied from.
 - Xd holds the lowest address to be copied to.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
- If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - Xs holds the highest address to be copied from + 1.
 - Xd holds the highest address to be copied to + 1.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied from.
 - The highest address that has been copied from + 1.
 - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied to.
 - The highest address that has not been copied to + 1.

Integer

(FEAT_MOPS)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sz	0	1	1	1	0	1	op1	0	Rs						1	1	0	0	0	1	Rn						Rd				
o0								op2																							

Encoding for the Prologue variant

Applies when ($op1 == 00$)

CPYPN [<Xd>]!, [<Xs>]!, <Xn>!

Encoding for the Main variant

Applies when (op1 == 01)

CPYMN [<Xd>]!, [<Xs>]!, <Xn>!

Encoding for the Epilogue variant

Applies when (op1 == 10)

CPYEN [<Xd>]!, [<Xs>]!, <Xn>!

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
  when '00' => memcpy.stage = MOPSStage_Prologue;
  when '01' => memcpy.stage = MOPSStage_Main;
  when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the `CONSTRAINED UNPREDICTABLE` behavior of this instruction, see [Architectural Constraints on UNPREDICTABLE behaviors](#), and particularly [Memory Copy and Memory Set CPY*](#) and [Crossing a page boundary with different memory types or Shareability attributes](#).

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.


```

CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv      = PSTATE.[N,Z,C,V];
memcpy.toaddress = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);

if memcpy.stage == MOPSTage_Prologue then
    memcpy.cpy_size = UInt(X{64}(memcpy.n));
else
    memcpy.cpy_size = SInt(X{64}(memcpy.n));
end;

memcpy.implements_option_a = CPYOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
    else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
    else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD, rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSTage_Prologue then
    if memcpy.cpy_size > ArchMaxMOPSCPysize then
        memcpy.cpy_size = ArchMaxMOPSCPysize;
    end;

    memcpy.forward = IsMemCpyForward(memcpy);

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        if memcpy.forward then
            // Copy in the forward direction offsets the arguments.
            memcpy.toaddress = memcpy.toaddress + memcpy.cpy_size;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpy_size;
            memcpy.cpy_size = 0 - memcpy.cpy_size;
        end;
    else
        if !memcpy.forward then
            // Copy in the reverse direction offsets the arguments.
            memcpy.toaddress = memcpy.toaddress + memcpy.cpy_size;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpy_size;
            memcpy.nzcv = '1010';
        else
            memcpy.nzcv = '0010';
        end;
    end;
end;

memcpy.stagecpy_size = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSTage_Prologue then
    memcpy.forward = memcpy.cpy_size < 0 || (!memcpy.implements_option_a && memcpy.nzcv[3] == '0');
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var fault : boolean = FALSE;
var B : MOPSTBlock Size = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpy_size != 0 && !fault looplimit ArchMaxMOPSCPysize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPySizeChoice(memcpy);

        if memcpy.forward then
            assert B <= -1 * memcpy.stagecpy_size;
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                memcpy.toaddress + memcpy.cpy_size,
                memcpy.fromaddress + memcpy.cpy_size,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.cpy_size = memcpy.cpy_size + B;
                memcpy.stagecpy_size = memcpy.stagecpy_size + B;
            end;
        else
            assert B <= memcpy.stagecpy_size;
            memcpy.cpy_size = memcpy.cpy_size - B;
            memcpy.stagecpy_size = memcpy.stagecpy_size - B;
        end;
    end;
end;

```

```

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
            memcpy.toaddress + memcpy.cpy_size,
            memcpy.fromaddress + memcpy.cpy_size,
            memcpy.forward, B, raccdesc,
            waccdesc);

    if copied != B then
        fault = TRUE;
        memcpy.cpy_size = memcpy.cpy_size + B;
        memcpy.stagecpy_size = memcpy.stagecpy_size + B;
    end;
end;
end;

else
    while memcpy.stagecpy_size > 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpy_size;

        if memcpy.forward then
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                memcpy.fromaddress,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress + B;
                memcpy.toaddress = memcpy.toaddress + B;
            end;
        else
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress - B,
                memcpy.fromaddress - B,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress - B;
                memcpy.toaddress = memcpy.toaddress - B;
            end;
        end;

        if !fault then
            memcpy.cpy_size = memcpy.cpy_size - B;
            memcpy.stagecpy_size = memcpy.stagecpy_size - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;

```

2025-12_rel2025-09_diff_tag 2025-12-102025-11-28 16:4129:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)

htmldiff from-

(new)

CPYPRN, CPYMRN, CPYERN

Memory copy, reads non-temporal

These instructions copy a requested number of bytes in memory from a source address to a destination address. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYPRN, then CPYMRN, and then CPYERN.

CPYPRN performs some preconditioning of the arguments suitable for using the CPYMRN instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYMRN copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYERN copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see [Memory Copy and Memory Set exceptions](#).

The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYPRN:

- If $Xn[63:55] \neq '000000000'$, the copy size Xn is saturated to $0x007FFFFFFFFFFFFF$.
- After saturation is performed, the direction of the memory copy is based on the following:
If $(Xs[55:0] > Xd[55:0])$ and $(Xd[55:0] + \text{saturated copy size}) > Xs[55:0]$, then the direction is forward.
If $(Xs[55:0] < Xd[55:0])$ and $(Xs[55:0] + \text{saturated copy size}) > Xd[55:0]$, then the direction is backward.
Otherwise, the direction is an IMPLEMENTATION DEFINED choice between forward and backward.

On completion of CPYPRN, option A:

- $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,0,0\}$.
- If the copy is in the forward direction, then:
 - Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the original $Xs + \text{saturated copy size}$.
 - Xd holds the original $Xd + \text{saturated copy size}$.
- If the copy is in the backward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs and Xd are unchanged.

On completion of CPYPRN, option B:

- If the copy is in the forward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
 - $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,1,0\}$.
- If the copy is in the backward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.
 - $PSTATE.\{N,Z,C,V\}$ are set to $\{1,0,1,0\}$.

For CPYMRN, option A, when $PSTATE.C = 0$:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
 - Xn holds -1 times the number of bytes remaining to be copied.
 - Xs holds the lowest address to be copied from - Xn .
 - Xd holds the lowest address to be copied to - Xn .
 - On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.
- If the copy is in the backward direction (Xn holds a positive number), then:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address to be copied from + 1 - Xn .
 - Xd holds the highest address to be copied to + 1 - Xn .
 - On completion of the instruction, Xn holds the number of bytes remaining to be copied.

For CPYMRN, option B, when $PSTATE.C = 1$:

- Xn holds the number of bytes remaining to be copied.

- If the copy is in the forward direction ($PSTATE.N == 0$), then:
 - Xs holds the lowest address to be copied from.
 - Xd holds the lowest address to be copied to.
 - On completion of the instruction:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
- If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - Xs holds the highest address to be copied from + 1.
 - Xd holds the highest address to be copied to + 1.
 - On completion of the instruction:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.

For CPYERN, option A, when $PSTATE.C = 0$:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
 - Xn holds -1 times the number of bytes remaining to be copied.
 - Xs holds the lowest address to be copied from - Xn.
 - Xd holds the lowest address to be copied to - Xn.
 - On completion of the instruction, Xn holds 0.
- If the copy is in the backward direction (Xn holds a positive number), then:
 - If the copy is in the backward direction (Xn holds a positive number), then:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address to be copied from + 1 - Xn.
 - Xd holds the highest address to be copied to + 1 - Xn.
 - On completion of the instruction, Xn holds 0.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied from.
 - The highest address that has been copied from + 1.
 - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied to.
 - The highest address that has not been copied to + 1.

For CPYERN, option B, when $PSTATE.C = 1$:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction ($PSTATE.N == 0$), then:
 - Xs holds the lowest address to be copied from.
 - Xd holds the lowest address to be copied to.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
- If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - Xs holds the highest address to be copied from + 1.
 - Xd holds the highest address to be copied to + 1.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied from.
 - The highest address that has been copied from + 1.
 - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied to.
 - The highest address that has not been copied to + 1.

Integer

(FEAT_MOPS)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sz	0	1	1	1	0	1	op1	0	Rs							1	0	0	0	0	1	Rn							Rd		
o0							op2																								

Encoding for the Prologue variant

Applies when ($op1 == 00$)

CPYPRN [<Xd>]!, [<Xs>]!, <Xn>!

Encoding for the Main variant

Applies when (op1 == 01)

CPYMRN [<Xd>]!, [<Xs>]!, <Xn>!

Encoding for the Epilogue variant

Applies when (op1 == 10)

CPYERN [<Xd>]!, [<Xs>]!, <Xn>!

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
  when '00' => memcpy.stage = MOPSStage_Prologue;
  when '01' => memcpy.stage = MOPSStage_Main;
  when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the `CONSTRAINED UNPREDICTABLE` behavior of this instruction, see [Architectural Constraints on UNPREDICTABLE behaviors](#), and particularly [Memory Copy and Memory Set CPY*](#) and [Crossing a page boundary with different memory types or Shareability attributes](#).

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.


```

CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv      = PSTATE.[N,Z,C,V];
memcpy.toaddress = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);

if memcpy.stage == MOPSTage_Prologue then
    memcpy.cpy_size = UInt(X{64}(memcpy.n));
else
    memcpy.cpy_size = SInt(X{64}(memcpy.n));
end;

memcpy.implements_option_a = CPYOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
    else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
    else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD, rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSTage_Prologue then
    if memcpy.cpy_size > ArchMaxMOPSCPysize then
        memcpy.cpy_size = ArchMaxMOPSCPysize;
    end;

    memcpy.forward = IsMemCpyForward(memcpy);

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        if memcpy.forward then
            // Copy in the forward direction offsets the arguments.
            memcpy.toaddress = memcpy.toaddress + memcpy.cpy_size;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpy_size;
            memcpy.cpy_size = 0 - memcpy.cpy_size;
        end;
    else
        if !memcpy.forward then
            // Copy in the reverse direction offsets the arguments.
            memcpy.toaddress = memcpy.toaddress + memcpy.cpy_size;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpy_size;
            memcpy.nzcv = '1010';
        else
            memcpy.nzcv = '0010';
        end;
    end;
end;

memcpy.stagecpy_size = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSTage_Prologue then
    memcpy.forward = memcpy.cpy_size < 0 || (!memcpy.implements_option_a && memcpy.nzcv[3] == '0');
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var fault : boolean = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpy_size != 0 && !fault looplimit ArchMaxMOPSCPysize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPySizeChoice(memcpy);

        if memcpy.forward then
            assert B <= -1 * memcpy.stagecpy_size;
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                memcpy.toaddress + memcpy.cpy_size,
                memcpy.fromaddress + memcpy.cpy_size,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.cpy_size = memcpy.cpy_size + B;
                memcpy.stagecpy_size = memcpy.stagecpy_size + B;
            end;
        else
            assert B <= memcpy.stagecpy_size;
            memcpy.cpy_size = memcpy.cpy_size - B;
            memcpy.stagecpy_size = memcpy.stagecpy_size - B;
        end;
    end;
end;

```

```

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
            memcpy.toaddress + memcpy.cpy_size,
            memcpy.fromaddress + memcpy.cpy_size,
            memcpy.forward, B, raccdesc,
            waccdesc);

    if copied != B then
        fault = TRUE;
        memcpy.cpy_size = memcpy.cpy_size + B;
        memcpy.stagecpy_size = memcpy.stagecpy_size + B;
    end;
end;
end;

else
    while memcpy.stagecpy_size > 0 && !fault looplimit ArchMaxMOPSCPYSIZE do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSIZECHOICE(memcpy);
        assert B <= memcpy.stagecpy_size;

        if memcpy.forward then
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                memcpy.fromaddress,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress + B;
                memcpy.toaddress = memcpy.toaddress + B;
            end;
        else
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress - B,
                memcpy.fromaddress - B,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress - B;
                memcpy.toaddress = memcpy.toaddress - B;
            end;
        end;

        if !fault then
            memcpy.cpy_size = memcpy.cpy_size - B;
            memcpy.stagecpy_size = memcpy.stagecpy_size - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSSstage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;

```

2025-12-10 16:41:29 2025-09-16 16:41:29 2025-12-10 16:41:29 2025-12-10 16:41:29

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)

htmldiff from-

(new)

CPYPRT, CPYMRT, CPYERT

Memory copy, reads unprivileged

These instructions copy a requested number of bytes in memory from a source address to a destination address. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYPRT, then CPYMRT, and then CPYERT.

CPYPRT performs some preconditioning of the arguments suitable for using the CPYMRT instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYMRT copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYERT copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see [Memory Copy and Memory Set exceptions](#).

The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYPRT:

- If $Xn[63:55] \neq '000000000'$, the copy size Xn is saturated to $0x007FFFFFFFFFFFFF$.
- After saturation is performed, the direction of the memory copy is based on the following:
If $(Xs[55:0] > Xd[55:0])$ and $(Xd[55:0] + \text{saturated copy size}) > Xs[55:0]$, then the direction is forward.
If $(Xs[55:0] < Xd[55:0])$ and $(Xs[55:0] + \text{saturated copy size}) > Xd[55:0]$, then the direction is backward.
Otherwise, the direction is an IMPLEMENTATION DEFINED choice between forward and backward.

On completion of CPYPRT, option A:

- $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,0,0\}$.
- If the copy is in the forward direction, then:
 - Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the original Xs + saturated copy size.
 - Xd holds the original Xd + saturated copy size.
- If the copy is in the backward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs and Xd are unchanged.

On completion of CPYPRT, option B:

- If the copy is in the forward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
 - $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,1,0\}$.
- If the copy is in the backward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.
 - $PSTATE.\{N,Z,C,V\}$ are set to $\{1,0,1,0\}$.

For CPYMRT, option A, when $PSTATE.C = 0$:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
 - Xn holds -1 times the number of bytes remaining to be copied.
 - Xs holds the lowest address to be copied from - Xn .
 - Xd holds the lowest address to be copied to - Xn .
 - On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.
- If the copy is in the backward direction (Xn holds a positive number), then:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address to be copied from + 1 - Xn .
 - Xd holds the highest address to be copied to + 1 - Xn .
 - On completion of the instruction, Xn holds the number of bytes remaining to be copied.

For CPYMRT, option B, when $PSTATE.C = 1$:

- Xn holds the number of bytes remaining to be copied.

- If the copy is in the forward direction ($PSTATE.N == 0$), then:
 - Xs holds the lowest address to be copied from.
 - Xd holds the lowest address to be copied to.
 - On completion of the instruction:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
- If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - Xs holds the highest address to be copied from + 1.
 - Xd holds the highest address to be copied to + 1.
 - On completion of the instruction:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.

For CPYERT, option A, when $PSTATE.C = 0$:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
 - Xn holds -1 times the number of bytes remaining to be copied.
 - Xs holds the lowest address to be copied from - Xn.
 - Xd holds the lowest address to be copied to - Xn.
 - On completion of the instruction, Xn holds 0.
- If the copy is in the backward direction (Xn holds a positive number), then:
 - If the copy is in the backward direction (Xn holds a positive number), then:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address to be copied from + 1 - Xn.
 - Xd holds the highest address to be copied to + 1 - Xn.
 - On completion of the instruction, Xn holds 0.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied from.
 - The highest address that has been copied from + 1.
 - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied to.
 - The highest address that has not been copied to + 1.

For CPYERT, option B, when $PSTATE.C = 1$:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction ($PSTATE.N == 0$), then:
 - Xs holds the lowest address to be copied from.
 - Xd holds the lowest address to be copied to.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
- If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - Xs holds the highest address to be copied from + 1.
 - Xd holds the highest address to be copied to + 1.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied from.
 - The highest address that has been copied from + 1.
 - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied to.
 - The highest address that has not been copied to + 1.

Explicit Memory Read effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of $PSTATE.UAO$ is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of $HCR_EL2().\{E2H, TGE\}$ is '11'.

Otherwise, the Explicit Memory Read effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer (FEAT_MOPS)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
sz	0	1	1	1	0	1	op1	0	Rs							0	0	1	0	0	1	Rn							Rd			
o0								op2																								

Encoding for the Prologue variant

Applies when (op1 == 00)

```
CPYPRT [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when (op1 == 01)

```
CPYMRT [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when (op1 == 10)

```
CPYERT [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
  when '00' => memcpy.stage = MOPSStage_Prologue;
  when '01' => memcpy.stage = MOPSStage_Main;
  when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the `CONSTRAINED UNPREDICTABLE` behavior of this instruction, see [Architectural Constraints on UNPREDICTABLE behaviors](#), and particularly [Memory Copy and Memory Set CPY*](#) and [Crossing a page boundary with different memory types or Shareability attributes](#).

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.


```

CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv      = PSTATE.[N,Z,C,V];
memcpy.toaddress = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);

if memcpy.stage == MOPSStage_Prologue then
    memcpy.cpy_size = UInt(X{64}(memcpy.n));
else
    memcpy.cpy_size = SInt(X{64}(memcpy.n));
end;

memcpy.implements_option_a = CPYOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
    else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
    else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD, rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSStage_Prologue then
    if memcpy.cpy_size > ArchMaxMOPSCPysize then
        memcpy.cpy_size = ArchMaxMOPSCPysize;
    end;

    memcpy.forward = IsMemCpyForward(memcpy);

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        if memcpy.forward then
            // Copy in the forward direction offsets the arguments.
            memcpy.toaddress = memcpy.toaddress + memcpy.cpy_size;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpy_size;
            memcpy.cpy_size = 0 - memcpy.cpy_size;
        end;
    else
        if !memcpy.forward then
            // Copy in the reverse direction offsets the arguments.
            memcpy.toaddress = memcpy.toaddress + memcpy.cpy_size;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpy_size;
            memcpy.nzcv = '1010';
        else
            memcpy.nzcv = '0010';
        end;
    end;
end;

memcpy.stagecpy_size = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSStage_Prologue then
    memcpy.forward = memcpy.cpy_size < 0 || (!memcpy.implements_option_a && memcpy.nzcv[3] == '0');
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var fault : boolean = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpy_size != 0 && !fault looplimit ArchMaxMOPSCPysize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);

        if memcpy.forward then
            assert B <= -1 * memcpy.stagecpy_size;
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                memcpy.toaddress + memcpy.cpy_size,
                memcpy.fromaddress + memcpy.cpy_size,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.cpy_size = memcpy.cpy_size + B;
                memcpy.stagecpy_size = memcpy.stagecpy_size + B;
            end;
        else
            assert B <= memcpy.stagecpy_size;
            memcpy.cpy_size = memcpy.cpy_size - B;
            memcpy.stagecpy_size = memcpy.stagecpy_size - B;
        end;
    end;
end;

```

```

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
            memcpy.toaddress + memcpy.cpy_size,
            memcpy.fromaddress + memcpy.cpy_size,
            memcpy.forward, B, raccdesc,
            waccdesc);

    if copied != B then
        fault = TRUE;
        memcpy.cpy_size = memcpy.cpy_size + B;
        memcpy.stagecpy_size = memcpy.stagecpy_size + B;
    end;
end;
end;

else
    while memcpy.stagecpy_size > 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpy_size;

        if memcpy.forward then
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                memcpy.fromaddress,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress + B;
                memcpy.toaddress = memcpy.toaddress + B;
            end;
        else
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress - B,
                memcpy.fromaddress - B,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress - B;
                memcpy.toaddress = memcpy.toaddress - B;
            end;
        end;

        if !fault then
            memcpy.cpy_size = memcpy.cpy_size - B;
            memcpy.stagecpy_size = memcpy.stagecpy_size - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;

```

2025-12_rel2025-09_diff_tag 2025-12-102025-11-28 16:4129:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)

htmldiff from-

(new)

CPYPRTN, CPYMRTN, CPYERTN

Memory copy, reads unprivileged, reads and writes non-temporal

These instructions copy a requested number of bytes in memory from a source address to a destination address. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYPRTN, then CPYMRTN, and then CPYERTN.

CPYPRTN performs some preconditioning of the arguments suitable for using the CPYMRTN instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYMRTN copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYERTN copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see [Memory Copy and Memory Set exceptions](#).

The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYPRTN:

- If $Xn[63:55] \neq '000000000'$, the copy size Xn is saturated to $0x007FFFFFFFFFFFFFFF$.
- After saturation is performed, the direction of the memory copy is based on the following:
If $(Xs[55:0] > Xd[55:0])$ and $(Xd[55:0] + \text{saturated copy size}) > Xs[55:0]$, then the direction is forward.
If $(Xs[55:0] < Xd[55:0])$ and $(Xs[55:0] + \text{saturated copy size}) > Xd[55:0]$, then the direction is backward.
Otherwise, the direction is an IMPLEMENTATION DEFINED choice between forward and backward.

On completion of CPYPRTN, option A:

- $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,0,0\}$.
- If the copy is in the forward direction, then:
 - Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the original Xs + saturated copy size.
 - Xd holds the original Xd + saturated copy size.
- If the copy is in the backward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs and Xd are unchanged.

On completion of CPYPRTN, option B:

- If the copy is in the forward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
 - $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,1,0\}$.
- If the copy is in the backward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.
 - $PSTATE.\{N,Z,C,V\}$ are set to $\{1,0,1,0\}$.

For CPYMRTN, option A, when $PSTATE.C = 0$:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
 - Xn holds -1 times the number of bytes remaining to be copied.
 - Xs holds the lowest address to be copied from - Xn .
 - Xd holds the lowest address to be copied to - Xn .
 - On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.
- If the copy is in the backward direction (Xn holds a positive number), then:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address to be copied from + 1 - Xn .
 - Xd holds the highest address to be copied to + 1 - Xn .
 - On completion of the instruction, Xn holds the number of bytes remaining to be copied.

For CPYMRTN, option B, when $PSTATE.C = 1$:

- Xn holds the number of bytes remaining to be copied.

- If the copy is in the forward direction ($PSTATE.N == 0$), then:
 - Xs holds the lowest address to be copied from.
 - Xd holds the lowest address to be copied to.
 - On completion of the instruction:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
- If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - Xs holds the highest address to be copied from + 1.
 - Xd holds the highest address to be copied to + 1.
 - On completion of the instruction:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.

For CPYERTN, option A, when $PSTATE.C = 0$:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
 - Xn holds -1 times the number of bytes remaining to be copied.
 - Xs holds the lowest address to be copied from - Xn.
 - Xd holds the lowest address to be copied to - Xn.
 - On completion of the instruction, Xn holds 0.
- If the copy is in the backward direction (Xn holds a positive number), then:
 - If the copy is in the backward direction (Xn holds a positive number), then:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address to be copied from + 1 - Xn.
 - Xd holds the highest address to be copied to + 1 - Xn.
 - On completion of the instruction, Xn holds 0.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied from.
 - The highest address that has been copied from + 1.
 - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied to.
 - The highest address that has not been copied to + 1.

For CPYERTN, option B, when $PSTATE.C = 1$:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction ($PSTATE.N == 0$), then:
 - Xs holds the lowest address to be copied from.
 - Xd holds the lowest address to be copied to.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
- If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - Xs holds the highest address to be copied from + 1.
 - Xd holds the highest address to be copied to + 1.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied from.
 - The highest address that has been copied from + 1.
 - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied to.
 - The highest address that has not been copied to + 1.

Explicit Memory Read effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of $PSTATE.UAO$ is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of $HCR_EL2().\{E2H, TGE\}$ is '11'.

Otherwise, the Explicit Memory Read effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer (FEAT_MOPS)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0										
sz		0		1		1		1		0		1		op1		0		Rs				1		1		1		0		0		1		Rn				Rd			
op0										op2																															

Encoding for the Prologue variant

Applies when (op1 == 00)

```
CPYPRTN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when (op1 == 01)

```
CPYMRTN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when (op1 == 10)

```
CPYERTN [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rntemporal : boolean = options[3] == '1';
let wntemporal : boolean = options[2] == '1';
case op1 of
  when '00' => memcpy.stage = MOPSSStage_Prologue;
  when '01' => memcpy.stage = MOPSSStage_Main;
  when '10' => memcpy.stage = MOPSSStage_Epilogue;
end;
```

For information about the `CONSTRAINED UNPREDICTABLE` behavior of this instruction, see [Architectural Constraints on UNPREDICTABLE behaviors](#), and particularly [Memory Copy and Memory Set CPY*](#) and [Crossing a page boundary with different memory types or Shareability attributes](#).

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.


```

CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv      = PSTATE.[N,Z,C,V];
memcpy.toaddress = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);

if memcpy.stage == MOPSTage_Prologue then
    memcpy.cpy_size = UInt(X{64}(memcpy.n));
else
    memcpy.cpy_size = SInt(X{64}(memcpy.n));
end;

memcpy.implements_option_a = CPYOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
    else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
    else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD, rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSTage_Prologue then
    if memcpy.cpy_size > ArchMaxMOPSCPysize then
        memcpy.cpy_size = ArchMaxMOPSCPysize;
    end;

    memcpy.forward = IsMemCpyForward(memcpy);

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        if memcpy.forward then
            // Copy in the forward direction offsets the arguments.
            memcpy.toaddress = memcpy.toaddress + memcpy.cpy_size;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpy_size;
            memcpy.cpy_size = 0 - memcpy.cpy_size;
        end;
    else
        if !memcpy.forward then
            // Copy in the reverse direction offsets the arguments.
            memcpy.toaddress = memcpy.toaddress + memcpy.cpy_size;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpy_size;
            memcpy.nzcv = '1010';
        else
            memcpy.nzcv = '0010';
        end;
    end;
end;

memcpy.stagecpy_size = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSTage_Prologue then
    memcpy.forward = memcpy.cpy_size < 0 || (!memcpy.implements_option_a && memcpy.nzcv[3] == '0');
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var fault : boolean = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpy_size != 0 && !fault looplimit ArchMaxMOPSCPysize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);

        if memcpy.forward then
            assert B <= -1 * memcpy.stagecpy_size;
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                memcpy.toaddress + memcpy.cpy_size,
                memcpy.fromaddress + memcpy.cpy_size,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.cpy_size = memcpy.cpy_size + B;
                memcpy.stagecpy_size = memcpy.stagecpy_size + B;
            end;
        else
            assert B <= memcpy.stagecpy_size;
            memcpy.cpy_size = memcpy.cpy_size - B;
            memcpy.stagecpy_size = memcpy.stagecpy_size - B;
        end;
    end;
end;

```

```

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
            memcpy.toaddress + memcpy.cpy_size,
            memcpy.fromaddress + memcpy.cpy_size,
            memcpy.forward, B, raccdesc,
            waccdesc);

    if copied != B then
        fault = TRUE;
        memcpy.cpy_size = memcpy.cpy_size + B;
        memcpy.stagecpy_size = memcpy.stagecpy_size + B;
    end;
end;
end;

else
    while memcpy.stagecpy_size > 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpy_size;

        if memcpy.forward then
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                memcpy.fromaddress,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress + B;
                memcpy.toaddress = memcpy.toaddress + B;
            end;
        else
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress - B,
                memcpy.fromaddress - B,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress - B;
                memcpy.toaddress = memcpy.toaddress - B;
            end;
        end;

        if !fault then
            memcpy.cpy_size = memcpy.cpy_size - B;
            memcpy.stagecpy_size = memcpy.stagecpy_size - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSSstage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;

```

2025-12-10 16:41:29 2025-09-16 16:41:29 2025-12-10 16:41:29 2025-12-10 16:41:29

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)

htmldiff from-

(new)

CPYPRTRN, CPYMRTRN, CPYERTRN

Memory copy, reads unprivileged and non-temporal

These instructions copy a requested number of bytes in memory from a source address to a destination address. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYPRTRN, then CPYMRTRN, and then CPYERTRN. CPYPRTRN performs some preconditioning of the arguments suitable for using the CPYMRTRN instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYMRTRN copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYERTRN copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see [Memory Copy and Memory Set exceptions](#).

The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYPRTRN:

- If $Xn[63:55] \neq '000000000'$, the copy size Xn is saturated to $0x007FFFFFFFFFFFFF$.
- After saturation is performed, the direction of the memory copy is based on the following:
If $(Xs[55:0] > Xd[55:0])$ and $(Xd[55:0] + \text{saturated copy size}) > Xs[55:0]$, then the direction is forward.
If $(Xs[55:0] < Xd[55:0])$ and $(Xs[55:0] + \text{saturated copy size}) > Xd[55:0]$, then the direction is backward.
Otherwise, the direction is an IMPLEMENTATION DEFINED choice between forward and backward.

On completion of CPYPRTRN, option A:

- $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,0,0\}$.
- If the copy is in the forward direction, then:
 - Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the original $Xs + \text{saturated copy size}$.
 - Xd holds the original $Xd + \text{saturated copy size}$.
- If the copy is in the backward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs and Xd are unchanged.

On completion of CPYPRTRN, option B:

- If the copy is in the forward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
 - $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,1,0\}$.
- If the copy is in the backward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.
 - $PSTATE.\{N,Z,C,V\}$ are set to $\{1,0,1,0\}$.

For CPYMRTRN, option A, when $PSTATE.C = 0$:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
 - Xn holds -1 times the number of bytes remaining to be copied.
 - Xs holds the lowest address to be copied from - Xn .
 - Xd holds the lowest address to be copied to - Xn .
 - On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.
- If the copy is in the backward direction (Xn holds a positive number), then:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address to be copied from + 1 - Xn .
 - Xd holds the highest address to be copied to + 1 - Xn .
 - On completion of the instruction, Xn holds the number of bytes remaining to be copied.

For CPYMRTRN, option B, when $PSTATE.C = 1$:

- Xn holds the number of bytes remaining to be copied.

- If the copy is in the forward direction ($PSTATE.N == 0$), then:
 - Xs holds the lowest address to be copied from.
 - Xd holds the lowest address to be copied to.
 - On completion of the instruction:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
- If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - Xs holds the highest address to be copied from + 1.
 - Xd holds the highest address to be copied to + 1.
 - On completion of the instruction:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.

For CPYERTRN, option A, when $PSTATE.C = 0$:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
 - Xn holds -1 times the number of bytes remaining to be copied.
 - Xs holds the lowest address to be copied from - Xn.
 - Xd holds the lowest address to be copied to - Xn.
 - On completion of the instruction, Xn holds 0.
- If the copy is in the backward direction (Xn holds a positive number), then:
 - If the copy is in the backward direction (Xn holds a positive number), then:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address to be copied from + 1 - Xn.
 - Xd holds the highest address to be copied to + 1 - Xn.
 - On completion of the instruction, Xn holds 0.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied from.
 - The highest address that has been copied from + 1.
 - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied to.
 - The highest address that has not been copied to + 1.

For CPYERTRN, option B, when $PSTATE.C = 1$:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction ($PSTATE.N == 0$), then:
 - Xs holds the lowest address to be copied from.
 - Xd holds the lowest address to be copied to.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
- If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - Xs holds the highest address to be copied from + 1.
 - Xd holds the highest address to be copied to + 1.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied from.
 - The highest address that has been copied from + 1.
 - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied to.
 - The highest address that has not been copied to + 1.

Explicit Memory Read effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of $PSTATE.UAO$ is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of $HCR_EL2().\{E2H, TGE\}$ is '11'.

Otherwise, the Explicit Memory Read effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer (FEAT_MOPS)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sz	0	1	1	1	0	1	op1	0	Rs				1	0	1	0	0	1	Rn				Rd								
o0										op2																					

Encoding for the Prologue variant

Applies when (op1 == 00)

CPYPRTN [<Xd>]!, [<Xs>]!, <Xn>!

Encoding for the Main variant

Applies when (op1 == 01)

CPYMRTRN [<Xd>]!, [<Xs>]!, <Xn>!

Encoding for the Epilogue variant

Applies when (op1 == 10)

CPYERTN [<Xd>]!, [<Xs>]!, <Xn>!

Decode for all variants of this encoding

```

if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rntemporal : boolean = options[3] == '1';
let wntemporal : boolean = options[2] == '1';
case op1 of
  when '00' => memcpy.stage = MOPSStage_Prologue;
  when '01' => memcpy.stage = MOPSStage_Main;
  when '10' => memcpy.stage = MOPSStage_Epilogue;
end;

```

For information about the `CONSTRAINED UNPREDICTABLE` behavior of this instruction, see [Architectural Constraints on UNPREDICTABLE behaviors](#), and particularly [Memory Copy and Memory Set CPY*](#) and [Crossing a page boundary with different memory types or Shareability attributes](#).

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.


```

CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv      = PSTATE.[N,Z,C,V];
memcpy.toaddress = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);

if memcpy.stage == MOPSTage_Prologue then
    memcpy.cpy_size = UInt(X{64}(memcpy.n));
else
    memcpy.cpy_size = SInt(X{64}(memcpy.n));
end;

memcpy.implements_option_a = CPYOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
    else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
    else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD, rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSTage_Prologue then
    if memcpy.cpy_size > ArchMaxMOPSCPysize then
        memcpy.cpy_size = ArchMaxMOPSCPysize;
    end;

    memcpy.forward = IsMemCpyForward(memcpy);

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        if memcpy.forward then
            // Copy in the forward direction offsets the arguments.
            memcpy.toaddress = memcpy.toaddress + memcpy.cpy_size;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpy_size;
            memcpy.cpy_size = 0 - memcpy.cpy_size;
        end;
    else
        if !memcpy.forward then
            // Copy in the reverse direction offsets the arguments.
            memcpy.toaddress = memcpy.toaddress + memcpy.cpy_size;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpy_size;
            memcpy.nzcv = '1010';
        else
            memcpy.nzcv = '0010';
        end;
    end;
end;

memcpy.stagecpy_size = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSTage_Prologue then
    memcpy.forward = memcpy.cpy_size < 0 || (!memcpy.implements_option_a && memcpy.nzcv[3] == '0');
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var fault : boolean = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpy_size != 0 && !fault looplimit ArchMaxMOPSCPysize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);

        if memcpy.forward then
            assert B <= -1 * memcpy.stagecpy_size;
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                memcpy.toaddress + memcpy.cpy_size,
                memcpy.fromaddress + memcpy.cpy_size,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.cpy_size = memcpy.cpy_size + B;
                memcpy.stagecpy_size = memcpy.stagecpy_size + B;
            end;
        else
            assert B <= memcpy.stagecpy_size;
            memcpy.cpy_size = memcpy.cpy_size - B;
            memcpy.stagecpy_size = memcpy.stagecpy_size - B;
        end;
    end;
end;

```

```

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
            memcpy.toaddress + memcpy.cpy_size,
            memcpy.fromaddress + memcpy.cpy_size,
            memcpy.forward, B, raccdesc,
            waccdesc);

    if copied != B then
        fault = TRUE;
        memcpy.cpy_size = memcpy.cpy_size + B;
        memcpy.stagecpy_size = memcpy.stagecpy_size + B;
    end;
end;
end;

else
    while memcpy.stagecpy_size > 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpy_size;

        if memcpy.forward then
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                memcpy.fromaddress,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress + B;
                memcpy.toaddress = memcpy.toaddress + B;
            end;
        else
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress - B,
                memcpy.fromaddress - B,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress - B;
                memcpy.toaddress = memcpy.toaddress - B;
            end;
        end;

        if !fault then
            memcpy.cpy_size = memcpy.cpy_size - B;
            memcpy.stagecpy_size = memcpy.stagecpy_size - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;

```

2025-12-10 16:41:29 2025-09-16 16:41:29 2025-12-10 16:41:29 2025-12-10 16:41:29

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)

htmldiff from-

(new)

CPYPRTWN, CPYMRTWN, CPYERTWN

Memory copy, reads unprivileged, writes non-temporal

These instructions copy a requested number of bytes in memory from a source address to a destination address. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYPRTWN, then CPYMRTWN, and then CPYERTWN.

CPYPRTWN performs some preconditioning of the arguments suitable for using the CPYMRTWN instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYMRTWN copies a further IMPLEMENTATION DEFINED portion of the remaining bytes.

CPYERTWN copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see [Memory Copy and Memory Set exceptions](#).

The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYPRTWN:

- If $Xn[63:55] \neq '000000000'$, the copy size Xn is saturated to $0x007FFFFFFFFFFFFFFF$.
- After saturation is performed, the direction of the memory copy is based on the following:
If $(Xs[55:0] > Xd[55:0])$ and $(Xd[55:0] + \text{saturated copy size}) > Xs[55:0]$, then the direction is forward.
If $(Xs[55:0] < Xd[55:0])$ and $(Xs[55:0] + \text{saturated copy size}) > Xd[55:0]$, then the direction is backward.
Otherwise, the direction is an IMPLEMENTATION DEFINED choice between forward and backward.

On completion of CPYPRTWN, option A:

- $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,0,0\}$.
- If the copy is in the forward direction, then:
 - Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the original $Xs + \text{saturated copy size}$.
 - Xd holds the original $Xd + \text{saturated copy size}$.
- If the copy is in the backward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs and Xd are unchanged.

On completion of CPYPRTWN, option B:

- If the copy is in the forward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
 - $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,1,0\}$.
- If the copy is in the backward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.
 - $PSTATE.\{N,Z,C,V\}$ are set to $\{1,0,1,0\}$.

For CPYMRTWN, option A, when $PSTATE.C = 0$:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
 - Xn holds -1 times the number of bytes remaining to be copied.
 - Xs holds the lowest address to be copied from - Xn .
 - Xd holds the lowest address to be copied to - Xn .
 - On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.
- If the copy is in the backward direction (Xn holds a positive number), then:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address to be copied from + 1 - Xn .
 - Xd holds the highest address to be copied to + 1 - Xn .
 - On completion of the instruction, Xn holds the number of bytes remaining to be copied.

For CPYMRTWN, option B, when $PSTATE.C = 1$:

- Xn holds the number of bytes remaining to be copied.

- If the copy is in the forward direction ($PSTATE.N == 0$), then:
 - Xs holds the lowest address to be copied from.
 - Xd holds the lowest address to be copied to.
 - On completion of the instruction:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
- If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - Xs holds the highest address to be copied from + 1.
 - Xd holds the highest address to be copied to + 1.
 - On completion of the instruction:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.

For CPYERTWN, option A, when $PSTATE.C = 0$:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
 - Xn holds -1 times the number of bytes remaining to be copied.
 - Xs holds the lowest address to be copied from - Xn.
 - Xd holds the lowest address to be copied to - Xn.
 - On completion of the instruction, Xn holds 0.
- If the copy is in the backward direction (Xn holds a positive number), then:
 - If the copy is in the backward direction (Xn holds a positive number), then:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address to be copied from + 1 - Xn.
 - Xd holds the highest address to be copied to + 1 - Xn.
 - On completion of the instruction, Xn holds 0.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied from.
 - The highest address that has been copied from + 1.
 - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied to.
 - The highest address that has not been copied to + 1.

For CPYERTWN, option B, when $PSTATE.C = 1$:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction ($PSTATE.N == 0$), then:
 - Xs holds the lowest address to be copied from.
 - Xd holds the lowest address to be copied to.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
- If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - Xs holds the highest address to be copied from + 1.
 - Xd holds the highest address to be copied to + 1.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied from.
 - The highest address that has been copied from + 1.
 - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied to.
 - The highest address that has not been copied to + 1.

Explicit Memory Read effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of $PSTATE.UAO$ is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of $HCR_EL2().\{E2H, TGE\}$ is '11'.

Otherwise, the Explicit Memory Read effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer (FEAT_MOPS)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sz	0	1	1	1	0	1	op1	0	Rs				0	1	1	0	0	1	Rn				Rd								
o0								op2																							

Encoding for the Prologue variant

Applies when (op1 == 00)

CPYPRTWN [<Xd>]!, [<Xs>]!, <Xn>!

Encoding for the Main variant

Applies when (op1 == 01)

CPYMRTWN [<Xd>]!, [<Xs>]!, <Xn>!

Encoding for the Epilogue variant

Applies when (op1 == 10)

CPYERTWN [<Xd>]!, [<Xs>]!, <Xn>!

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
  when '00' => memcpy.stage = MOPSSStage_Prologue;
  when '01' => memcpy.stage = MOPSSStage_Main;
  when '10' => memcpy.stage = MOPSSStage_Epilogue;
end;
```

For information about the `CONSTRAINED UNPREDICTABLE` behavior of this instruction, see [Architectural Constraints on UNPREDICTABLE behaviors](#), and particularly [Memory Copy and Memory Set CPY*](#) and [Crossing a page boundary with different memory types or Shareability attributes](#).

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.


```

CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv      = PSTATE.[N,Z,C,V];
memcpy.toaddress = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);

if memcpy.stage == MOPSTage_Prologue then
    memcpy.cpy_size = UInt(X{64}(memcpy.n));
else
    memcpy.cpy_size = SInt(X{64}(memcpy.n));
end;

memcpy.implements_option_a = CPYOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
    else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
    else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD, rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSTage_Prologue then
    if memcpy.cpy_size > ArchMaxMOPSCPysize then
        memcpy.cpy_size = ArchMaxMOPSCPysize;
    end;

    memcpy.forward = IsMemCpyForward(memcpy);

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        if memcpy.forward then
            // Copy in the forward direction offsets the arguments.
            memcpy.toaddress = memcpy.toaddress + memcpy.cpy_size;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpy_size;
            memcpy.cpy_size = 0 - memcpy.cpy_size;
        end;
    else
        if !memcpy.forward then
            // Copy in the reverse direction offsets the arguments.
            memcpy.toaddress = memcpy.toaddress + memcpy.cpy_size;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpy_size;
            memcpy.nzcv = '1010';
        else
            memcpy.nzcv = '0010';
        end;
    end;
end;

memcpy.stagecpy_size = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSTage_Prologue then
    memcpy.forward = memcpy.cpy_size < 0 || (!memcpy.implements_option_a && memcpy.nzcv[3] == '0');
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var fault : boolean = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpy_size != 0 && !fault looplimit ArchMaxMOPSCPysize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);

        if memcpy.forward then
            assert B <= -1 * memcpy.stagecpy_size;
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                memcpy.toaddress + memcpy.cpy_size,
                memcpy.fromaddress + memcpy.cpy_size,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.cpy_size = memcpy.cpy_size + B;
                memcpy.stagecpy_size = memcpy.stagecpy_size + B;
            end;
        else
            assert B <= memcpy.stagecpy_size;
            memcpy.cpy_size = memcpy.cpy_size - B;
            memcpy.stagecpy_size = memcpy.stagecpy_size - B;
        end;
    end;
end;

```

```

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
            memcpy.toaddress + memcpy.cpy_size,
            memcpy.fromaddress + memcpy.cpy_size,
            memcpy.forward, B, raccdesc,
            waccdesc);

    if copied != B then
        fault = TRUE;
        memcpy.cpy_size = memcpy.cpy_size + B;
        memcpy.stagecpy_size = memcpy.stagecpy_size + B;
    end;
end;
end;

else
    while memcpy.stagecpy_size > 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpy_size;

        if memcpy.forward then
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                memcpy.fromaddress,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress + B;
                memcpy.toaddress = memcpy.toaddress + B;
            end;
        else
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress - B,
                memcpy.fromaddress - B,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress - B;
                memcpy.toaddress = memcpy.toaddress - B;
            end;
        end;

        if !fault then
            memcpy.cpy_size = memcpy.cpy_size - B;
            memcpy.stagecpy_size = memcpy.stagecpy_size - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;

```

2025-12-10 16:41:29 2025-09-16 16:41:29 2025-12-10 16:41:29 2025-12-10 16:41:29

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)

htmldiff from-

(new)

CPYPT, CPYMT, CPYET

Memory copy, reads and writes unprivileged

These instructions copy a requested number of bytes in memory from a source address to a destination address. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYPT, then CPYMT, and then CPYET.

CPYPT performs some preconditioning of the arguments suitable for using the CPYMT instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYMT copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYET copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see [Memory Copy and Memory Set exceptions](#).

The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYPT:

- If $X_n[63:55] \neq '000000000'$, the copy size X_n is saturated to $0x007FFFFFFFFFFFFFFF$.
- After saturation is performed, the direction of the memory copy is based on the following:
If $(X_s[55:0] > X_d[55:0])$ and $(X_d[55:0] + \text{saturated copy size}) > X_s[55:0]$, then the direction is forward.
If $(X_s[55:0] < X_d[55:0])$ and $(X_s[55:0] + \text{saturated copy size}) > X_d[55:0]$, then the direction is backward.
Otherwise, the direction is an IMPLEMENTATION DEFINED choice between forward and backward.

On completion of CPYPT, option A:

- $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,0,0\}$.
- If the copy is in the forward direction, then:
 - X_n holds -1 times the number of bytes in the saturated copy size remaining to be copied.
 - X_s holds the original $X_s + \text{saturated copy size}$.
 - X_d holds the original $X_d + \text{saturated copy size}$.
- If the copy is in the backward direction, then:
 - X_n holds the number of bytes in the saturated copy size remaining to be copied.
 - X_s and X_d are unchanged.

On completion of CPYPT, option B:

- If the copy is in the forward direction, then:
 - X_n holds the number of bytes in the saturated copy size remaining to be copied.
 - X_s holds the lowest address that has not been copied from.
 - X_d holds the lowest address that has not been copied to.
 - $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,1,0\}$.
- If the copy is in the backward direction, then:
 - X_n holds the number of bytes in the saturated copy size remaining to be copied.
 - X_s holds the highest address that has not been copied from + 1.
 - X_d holds the highest address that has not been copied to + 1.
 - $PSTATE.\{N,Z,C,V\}$ are set to $\{1,0,1,0\}$.

For CPYMT, option A, when $PSTATE.C = 0$:

- X_n holds a signed 64-bit integer.
- If the copy is in the forward direction (X_n holds a negative number), then:
 - X_n holds -1 times the number of bytes remaining to be copied.
 - X_s holds the lowest address to be copied from - X_n .
 - X_d holds the lowest address to be copied to - X_n .
 - On completion of the instruction, X_n holds -1 times the number of bytes remaining to be copied.
- If the copy is in the backward direction (X_n holds a positive number), then:
 - X_n holds the number of bytes remaining to be copied.
 - X_s holds the highest address to be copied from + 1 - X_n .
 - X_d holds the highest address to be copied to + 1 - X_n .
 - On completion of the instruction, X_n holds the number of bytes remaining to be copied.

For CPYMT, option B, when $PSTATE.C = 1$:

- X_n holds the number of bytes remaining to be copied.

- If the copy is in the forward direction ($PSTATE.N == 0$), then:
 - Xs holds the lowest address to be copied from.
 - Xd holds the lowest address to be copied to.
 - On completion of the instruction:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
- If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - Xs holds the highest address to be copied from + 1.
 - Xd holds the highest address to be copied to + 1.
 - On completion of the instruction:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.

For CPYET, option A, when $PSTATE.C = 0$:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
 - Xn holds -1 times the number of bytes remaining to be copied.
 - Xs holds the lowest address to be copied from - Xn.
 - Xd holds the lowest address to be copied to - Xn.
 - On completion of the instruction, Xn holds 0.
- If the copy is in the backward direction (Xn holds a positive number), then:
 - If the copy is in the backward direction (Xn holds a positive number), then:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address to be copied from + 1 - Xn.
 - Xd holds the highest address to be copied to + 1 - Xn.
 - On completion of the instruction, Xn holds 0.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied from.
 - The highest address that has been copied from + 1.
 - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied to.
 - The highest address that has not been copied to + 1.

For CPYET, option B, when $PSTATE.C = 1$:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction ($PSTATE.N == 0$), then:
 - Xs holds the lowest address to be copied from.
 - Xd holds the lowest address to be copied to.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
- If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - Xs holds the highest address to be copied from + 1.
 - Xd holds the highest address to be copied to + 1.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied from.
 - The highest address that has been copied from + 1.
 - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied to.
 - The highest address that has not been copied to + 1.

Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of $PSTATE.UAO$ is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of $HCR_EL2().\{E2H, TGE\}$ is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer

(FEAT_MOPS)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
sz	0	1	1	1	0	1	op1	0	Rs							0	0	1	1	0	1	Rn							Rd			
op0								op2																								

Encoding for the Prologue variant

Applies when (op1 == 00)

CPYPT [<Xd>]!, [<Xs>]!, <Xn>!

Encoding for the Main variant

Applies when (op1 == 01)

CPYMT [<Xd>]!, [<Xs>]!, <Xn>!

Encoding for the Epilogue variant

Applies when (op1 == 10)

CPYET [<Xd>]!, [<Xs>]!, <Xn>!

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rntemporal : boolean = options[3] == '1';
let wntemporal : boolean = options[2] == '1';
case op1 of
  when '00' => memcpy.stage = MOPSStage_Prologue;
  when '01' => memcpy.stage = MOPSStage_Main;
  when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the `CONSTRAINED UNPREDICTABLE` behavior of this instruction, see [Architectural Constraints on UNPREDICTABLE behaviors](#), and particularly [Memory Copy and Memory Set CPY*](#) and [Crossing a page boundary with different memory types or Shareability attributes](#).

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.


```

CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv      = PSTATE.[N,Z,C,V];
memcpy.toaddress = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);

if memcpy.stage == MOPSTage_Prologue then
    memcpy.cpy_size = UInt(X{64}(memcpy.n));
else
    memcpy.cpy_size = SInt(X{64}(memcpy.n));
end;

memcpy.implements_option_a = CPYOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
    else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
    else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD, rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSTage_Prologue then
    if memcpy.cpy_size > ArchMaxMOPSCPysize then
        memcpy.cpy_size = ArchMaxMOPSCPysize;
    end;

    memcpy.forward = IsMemCpyForward(memcpy);

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        if memcpy.forward then
            // Copy in the forward direction offsets the arguments.
            memcpy.toaddress = memcpy.toaddress + memcpy.cpy_size;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpy_size;
            memcpy.cpy_size = 0 - memcpy.cpy_size;
        end;
    else
        if !memcpy.forward then
            // Copy in the reverse direction offsets the arguments.
            memcpy.toaddress = memcpy.toaddress + memcpy.cpy_size;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpy_size;
            memcpy.nzcv = '1010';
        else
            memcpy.nzcv = '0010';
        end;
    end;
end;

memcpy.stagecpy_size = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSTage_Prologue then
    memcpy.forward = memcpy.cpy_size < 0 || (!memcpy.implements_option_a && memcpy.nzcv[3] == '0');
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var fault : boolean = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpy_size != 0 && !fault looplimit ArchMaxMOPSCPysize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPySizeChoice(memcpy);

        if memcpy.forward then
            assert B <= -1 * memcpy.stagecpy_size;
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                memcpy.toaddress + memcpy.cpy_size,
                memcpy.fromaddress + memcpy.cpy_size,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.cpy_size = memcpy.cpy_size + B;
                memcpy.stagecpy_size = memcpy.stagecpy_size + B;
            end;
        else
            assert B <= memcpy.stagecpy_size;
            memcpy.cpy_size = memcpy.cpy_size - B;
            memcpy.stagecpy_size = memcpy.stagecpy_size - B;
        end;
    end;
end;

```

```

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
            memcpy.toaddress + memcpy.cpy_size,
            memcpy.fromaddress + memcpy.cpy_size,
            memcpy.forward, B, raccdesc,
            waccdesc);

    if copied != B then
        fault = TRUE;
        memcpy.cpy_size = memcpy.cpy_size + B;
        memcpy.stagecpy_size = memcpy.stagecpy_size + B;
    end;
end;
end;

else
    while memcpy.stagecpy_size > 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpy_size;

        if memcpy.forward then
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                memcpy.fromaddress,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress + B;
                memcpy.toaddress = memcpy.toaddress + B;
            end;
        else
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress - B,
                memcpy.fromaddress - B,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress - B;
                memcpy.toaddress = memcpy.toaddress - B;
            end;
        end;

        if !fault then
            memcpy.cpy_size = memcpy.cpy_size - B;
            memcpy.stagecpy_size = memcpy.stagecpy_size - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;

```

2025-12_rel2025-09_diff_tag 2025-12-102025-11-28 16:4129:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)

htmldiff from-

(new)

CPYPTN, CPYMTN, CPYETN

Memory copy, reads and writes unprivileged and non-temporal

These instructions copy a requested number of bytes in memory from a source address to a destination address. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYPTN, then CPYMTN, and then CPYETN.

CPYPTN performs some preconditioning of the arguments suitable for using the CPYMTN instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYMTN copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYETN copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see [Memory Copy and Memory Set exceptions](#).

The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYPTN:

- If $Xn[63:55] \neq '000000000'$, the copy size Xn is saturated to $0x007FFFFFFFFFFFFFFF$.
- After saturation is performed, the direction of the memory copy is based on the following:
If $(Xs[55:0] > Xd[55:0])$ and $(Xd[55:0] + \text{saturated copy size}) > Xs[55:0]$, then the direction is forward.
If $(Xs[55:0] < Xd[55:0])$ and $(Xs[55:0] + \text{saturated copy size}) > Xd[55:0]$, then the direction is backward.
Otherwise, the direction is an IMPLEMENTATION DEFINED choice between forward and backward.

On completion of CPYPTN, option A:

- $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,0,0\}$.
- If the copy is in the forward direction, then:
 - Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the original $Xs + \text{saturated copy size}$.
 - Xd holds the original $Xd + \text{saturated copy size}$.
- If the copy is in the backward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs and Xd are unchanged.

On completion of CPYPTN, option B:

- If the copy is in the forward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
 - $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,1,0\}$.
- If the copy is in the backward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.
 - $PSTATE.\{N,Z,C,V\}$ are set to $\{1,0,1,0\}$.

For CPYMTN, option A, when $PSTATE.C = 0$:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
 - Xn holds -1 times the number of bytes remaining to be copied.
 - Xs holds the lowest address to be copied from - Xn .
 - Xd holds the lowest address to be copied to - Xn .
 - On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.
- If the copy is in the backward direction (Xn holds a positive number), then:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address to be copied from + 1 - Xn .
 - Xd holds the highest address to be copied to + 1 - Xn .
 - On completion of the instruction, Xn holds the number of bytes remaining to be copied.

For CPYMTN, option B, when $PSTATE.C = 1$:

- Xn holds the number of bytes remaining to be copied.

- If the copy is in the forward direction ($PSTATE.N == 0$), then:
 - Xs holds the lowest address to be copied from.
 - Xd holds the lowest address to be copied to.
 - On completion of the instruction:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
- If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - Xs holds the highest address to be copied from + 1.
 - Xd holds the highest address to be copied to + 1.
 - On completion of the instruction:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.

For CPYETN, option A, when $PSTATE.C = 0$:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
 - Xn holds -1 times the number of bytes remaining to be copied.
 - Xs holds the lowest address to be copied from - Xn.
 - Xd holds the lowest address to be copied to - Xn.
 - On completion of the instruction, Xn holds 0.
- If the copy is in the backward direction (Xn holds a positive number), then:
 - If the copy is in the backward direction (Xn holds a positive number), then:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address to be copied from + 1 - Xn.
 - Xd holds the highest address to be copied to + 1 - Xn.
 - On completion of the instruction, Xn holds 0.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied from.
 - The highest address that has been copied from + 1.
 - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied to.
 - The highest address that has not been copied to + 1.

For CPYETN, option B, when $PSTATE.C = 1$:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction ($PSTATE.N == 0$), then:
 - Xs holds the lowest address to be copied from.
 - Xd holds the lowest address to be copied to.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
- If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - Xs holds the highest address to be copied from + 1.
 - Xd holds the highest address to be copied to + 1.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied from.
 - The highest address that has been copied from + 1.
 - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied to.
 - The highest address that has not been copied to + 1.

Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of $PSTATE.UAO$ is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of $HCR_EL2().\{E2H, TGE\}$ is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer (FEAT_MOPS)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
sz	0	1	1	1	0	1	op1	0	Rs							1	1	1	1	0	1	Rn							Rd			
o0								op2																								

Encoding for the Prologue variant

Applies when (op1 == 00)

CPYP TN [<Xd>]!, [<Xs>]!, <Xn>!

Encoding for the Main variant

Applies when (op1 == 01)

CPYMTN [<Xd>]!, [<Xs>]!, <Xn>!

Encoding for the Epilogue variant

Applies when (op1 == 10)

CPYETN [<Xd>]!, [<Xs>]!, <Xn>!

Decode for all variants of this encoding

```

if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
  when '00' => memcpy.stage = MOPSSStage_Prologue;
  when '01' => memcpy.stage = MOPSSStage_Main;
  when '10' => memcpy.stage = MOPSSStage_Epilogue;
end;
```

For information about the `CONSTRAINED UNPREDICTABLE` behavior of this instruction, see [Architectural Constraints on UNPREDICTABLE behaviors](#), and particularly [Memory Copy and Memory Set CPY*](#) and [Crossing a page boundary with different memory types or Shareability attributes](#).

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.


```

CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv      = PSTATE.[N,Z,C,V];
memcpy.toaddress = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);

if memcpy.stage == MOPSTage_Prologue then
    memcpy.cpy_size = UInt(X{64}(memcpy.n));
else
    memcpy.cpy_size = SInt(X{64}(memcpy.n));
end;

memcpy.implements_option_a = CPYOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD, rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSTage_Prologue then
    if memcpy.cpy_size > ArchMaxMOPSCPysize then
        memcpy.cpy_size = ArchMaxMOPSCPysize;
    end;

    memcpy.forward = IsMemCpyForward(memcpy);

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        if memcpy.forward then
            // Copy in the forward direction offsets the arguments.
            memcpy.toaddress = memcpy.toaddress + memcpy.cpy_size;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpy_size;
            memcpy.cpy_size = 0 - memcpy.cpy_size;
        end;
    else
        if !memcpy.forward then
            // Copy in the reverse direction offsets the arguments.
            memcpy.toaddress = memcpy.toaddress + memcpy.cpy_size;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpy_size;
            memcpy.nzcv = '1010';
        else
            memcpy.nzcv = '0010';
        end;
    end;
end;

memcpy.stagecpy_size = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSTage_Prologue then
    memcpy.forward = memcpy.cpy_size < 0 || (!memcpy.implements_option_a && memcpy.nzcv[3] == '0');
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var fault : boolean = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpy_size != 0 && !fault looplimit ArchMaxMOPSCPysize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);

        if memcpy.forward then
            assert B <= -1 * memcpy.stagecpy_size;
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                memcpy.toaddress + memcpy.cpy_size,
                memcpy.fromaddress + memcpy.cpy_size,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.cpy_size = memcpy.cpy_size + B;
                memcpy.stagecpy_size = memcpy.stagecpy_size + B;
            end;
        else
            assert B <= memcpy.stagecpy_size;
            memcpy.cpy_size = memcpy.cpy_size - B;
            memcpy.stagecpy_size = memcpy.stagecpy_size - B;
        end;
    end;
end;

```

```

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
            memcpy.toaddress + memcpy.cpy_size,
            memcpy.fromaddress + memcpy.cpy_size,
            memcpy.forward, B, raccdesc,
            waccdesc);

    if copied != B then
        fault = TRUE;
        memcpy.cpy_size = memcpy.cpy_size + B;
        memcpy.stagecpy_size = memcpy.stagecpy_size + B;
    end;
end;
end;

else
    while memcpy.stagecpy_size > 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpy_size;

        if memcpy.forward then
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                memcpy.fromaddress,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress + B;
                memcpy.toaddress = memcpy.toaddress + B;
            end;
        else
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress - B,
                memcpy.fromaddress - B,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress - B;
                memcpy.toaddress = memcpy.toaddress - B;
            end;
        end;

        if !fault then
            memcpy.cpy_size = memcpy.cpy_size - B;
            memcpy.stagecpy_size = memcpy.stagecpy_size - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;

```

2025-12_rel2025-09_diff_tag 2025-12-102025-11-28 16:4129:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)

htmldiff from-

(new)

CPYPTRN, CPYMTRN, CPYETRN

Memory copy, reads and writes unprivileged, reads non-temporal

These instructions copy a requested number of bytes in memory from a source address to a destination address. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYPTRN, then CPYMTRN, and then CPYETRN.

CPYPTRN performs some preconditioning of the arguments suitable for using the CPYMTRN instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYMTRN copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYETRN copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see [Memory Copy and Memory Set exceptions](#).

The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYPTRN:

- If $Xn[63:55] \neq '000000000'$, the copy size Xn is saturated to $0x007FFFFFFFFFFFFFFF$.
- After saturation is performed, the direction of the memory copy is based on the following:
 If $(Xs[55:0] > Xd[55:0])$ and $(Xd[55:0] + \text{saturated copy size}) > Xs[55:0]$, then the direction is forward.
 If $(Xs[55:0] < Xd[55:0])$ and $(Xs[55:0] + \text{saturated copy size}) > Xd[55:0]$, then the direction is backward.
 Otherwise, the direction is an IMPLEMENTATION DEFINED choice between forward and backward.

On completion of CPYPTRN, option A:

- $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,0,0\}$.
- If the copy is in the forward direction, then:
 - Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the original $Xs + \text{saturated copy size}$.
 - Xd holds the original $Xd + \text{saturated copy size}$.
- If the copy is in the backward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs and Xd are unchanged.

On completion of CPYPTRN, option B:

- If the copy is in the forward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
 - $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,1,0\}$.
- If the copy is in the backward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.
 - $PSTATE.\{N,Z,C,V\}$ are set to $\{1,0,1,0\}$.

For CPYMTRN, option A, when $PSTATE.C = 0$:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
 - Xn holds -1 times the number of bytes remaining to be copied.
 - Xs holds the lowest address to be copied from - Xn .
 - Xd holds the lowest address to be copied to - Xn .
 - On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.
- If the copy is in the backward direction (Xn holds a positive number), then:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address to be copied from + 1 - Xn .
 - Xd holds the highest address to be copied to + 1 - Xn .
 - On completion of the instruction, Xn holds the number of bytes remaining to be copied.

For CPYMTRN, option B, when $PSTATE.C = 1$:

- Xn holds the number of bytes remaining to be copied.

- If the copy is in the forward direction ($PSTATE.N == 0$), then:
 - Xs holds the lowest address to be copied from.
 - Xd holds the lowest address to be copied to.
 - On completion of the instruction:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
- If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - Xs holds the highest address to be copied from + 1.
 - Xd holds the highest address to be copied to + 1.
 - On completion of the instruction:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.

For CPYETRN, option A, when $PSTATE.C = 0$:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
 - Xn holds -1 times the number of bytes remaining to be copied.
 - Xs holds the lowest address to be copied from - Xn.
 - Xd holds the lowest address to be copied to - Xn.
 - On completion of the instruction, Xn holds 0.
- If the copy is in the backward direction (Xn holds a positive number), then:
 - If the copy is in the backward direction (Xn holds a positive number), then:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address to be copied from + 1 - Xn.
 - Xd holds the highest address to be copied to + 1 - Xn.
 - On completion of the instruction, Xn holds 0.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied from.
 - The highest address that has been copied from + 1.
 - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied to.
 - The highest address that has not been copied to + 1.

For CPYETRN, option B, when $PSTATE.C = 1$:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction ($PSTATE.N == 0$), then:
 - Xs holds the lowest address to be copied from.
 - Xd holds the lowest address to be copied to.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
- If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - Xs holds the highest address to be copied from + 1.
 - Xd holds the highest address to be copied to + 1.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied from.
 - The highest address that has been copied from + 1.
 - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied to.
 - The highest address that has not been copied to + 1.

Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of $PSTATE.UAO$ is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of $HCR_EL2().\{E2H, TGE\}$ is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer (FEAT_MOPS)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sz	0	1	1	1	0	1	op1	0	Rs				1	0	1	1	0	1	Rn				Rd								
op0								op2																							

Encoding for the Prologue variant

Applies when (op1 == 00)

```
CPYPTRN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when (op1 == 01)

```
CPYMTRN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when (op1 == 10)

```
CPYETRN [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
  when '00' => memcpy.stage = MOPSStage_Prologue;
  when '01' => memcpy.stage = MOPSStage_Main;
  when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the `CONSTRAINED UNPREDICTABLE` behavior of this instruction, see [Architectural Constraints on UNPREDICTABLE behaviors](#), and particularly [Memory Copy and Memory Set CPY*](#) and [Crossing a page boundary with different memory types or Shareability attributes](#).

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.


```

CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv      = PSTATE.[N,Z,C,V];
memcpy.toaddress = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);

if memcpy.stage == MOPSTage_Prologue then
    memcpy.cpy_size = UInt(X{64}(memcpy.n));
else
    memcpy.cpy_size = SInt(X{64}(memcpy.n));
end;

memcpy.implements_option_a = CPYOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
    else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
    else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD, rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSTage_Prologue then
    if memcpy.cpy_size > ArchMaxMOPSCPysize then
        memcpy.cpy_size = ArchMaxMOPSCPysize;
    end;

    memcpy.forward = IsMemCpyForward(memcpy);

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        if memcpy.forward then
            // Copy in the forward direction offsets the arguments.
            memcpy.toaddress = memcpy.toaddress + memcpy.cpy_size;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpy_size;
            memcpy.cpy_size = 0 - memcpy.cpy_size;
        end;
    else
        if !memcpy.forward then
            // Copy in the reverse direction offsets the arguments.
            memcpy.toaddress = memcpy.toaddress + memcpy.cpy_size;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpy_size;
            memcpy.nzcv = '1010';
        else
            memcpy.nzcv = '0010';
        end;
    end;
end;

memcpy.stagecpy_size = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSTage_Prologue then
    memcpy.forward = memcpy.cpy_size < 0 || (!memcpy.implements_option_a && memcpy.nzcv[3] == '0');
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var fault : boolean = FALSE;
var B : MOPSTBlock Size = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpy_size != 0 && !fault looplimit ArchMaxMOPSCPysize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);

        if memcpy.forward then
            assert B <= -1 * memcpy.stagecpy_size;
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                memcpy.toaddress + memcpy.cpy_size,
                memcpy.fromaddress + memcpy.cpy_size,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.cpy_size = memcpy.cpy_size + B;
                memcpy.stagecpy_size = memcpy.stagecpy_size + B;
            end;
        else
            assert B <= memcpy.stagecpy_size;
            memcpy.cpy_size = memcpy.cpy_size - B;
            memcpy.stagecpy_size = memcpy.stagecpy_size - B;
        end;
    end;
end;

```

```

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
            memcpy.toaddress + memcpy.cpy_size,
            memcpy.fromaddress + memcpy.cpy_size,
            memcpy.forward, B, raccdesc,
            waccdesc);

    if copied != B then
        fault = TRUE;
        memcpy.cpy_size = memcpy.cpy_size + B;
        memcpy.stagecpy_size = memcpy.stagecpy_size + B;
    end;
end;
end;

else
    while memcpy.stagecpy_size > 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpy_size;

        if memcpy.forward then
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                memcpy.fromaddress,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress + B;
                memcpy.toaddress = memcpy.toaddress + B;
            end;
        else
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress - B,
                memcpy.fromaddress - B,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress - B;
                memcpy.toaddress = memcpy.toaddress - B;
            end;
        end;

        if !fault then
            memcpy.cpy_size = memcpy.cpy_size - B;
            memcpy.stagecpy_size = memcpy.stagecpy_size - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;

```

2025-12_rel2025-09_diff_tag 2025-12-102025-11-28 16:4129:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)

htmldiff from-

(new)

CPYPTWN, CPYMTWN, CPYETWN

Memory copy, reads and writes unprivileged, writes non-temporal

These instructions copy a requested number of bytes in memory from a source address to a destination address. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYPTWN, then CPYMTWN, and then CPYETWN.

CPYPTWN performs some preconditioning of the arguments suitable for using the CPYMTWN instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYMTWN copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYETWN copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see [Memory Copy and Memory Set exceptions](#).

The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYPTWN:

- If $Xn[63:55] \neq '000000000'$, the copy size Xn is saturated to $0x007FFFFFFFFFFFFFFF$.
- After saturation is performed, the direction of the memory copy is based on the following:
 If $(Xs[55:0] > Xd[55:0])$ and $(Xd[55:0] + \text{saturated copy size}) > Xs[55:0]$, then the direction is forward.
 If $(Xs[55:0] < Xd[55:0])$ and $(Xs[55:0] + \text{saturated copy size}) > Xd[55:0]$, then the direction is backward.
 Otherwise, the direction is an IMPLEMENTATION DEFINED choice between forward and backward.

On completion of CPYPTWN, option A:

- $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,0,0\}$.
- If the copy is in the forward direction, then:
 - Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the original Xs + saturated copy size.
 - Xd holds the original Xd + saturated copy size.
- If the copy is in the backward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs and Xd are unchanged.

On completion of CPYPTWN, option B:

- If the copy is in the forward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
 - $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,1,0\}$.
- If the copy is in the backward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.
 - $PSTATE.\{N,Z,C,V\}$ are set to $\{1,0,1,0\}$.

For CPYMTWN, option A, when $PSTATE.C = 0$:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
 - Xn holds -1 times the number of bytes remaining to be copied.
 - Xs holds the lowest address to be copied from - Xn .
 - Xd holds the lowest address to be copied to - Xn .
 - On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.
- If the copy is in the backward direction (Xn holds a positive number), then:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address to be copied from + 1 - Xn .
 - Xd holds the highest address to be copied to + 1 - Xn .
 - On completion of the instruction, Xn holds the number of bytes remaining to be copied.

For CPYMTWN, option B, when $PSTATE.C = 1$:

- Xn holds the number of bytes remaining to be copied.

- If the copy is in the forward direction ($PSTATE.N == 0$), then:
 - Xs holds the lowest address to be copied from.
 - Xd holds the lowest address to be copied to.
 - On completion of the instruction:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
- If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - Xs holds the highest address to be copied from + 1.
 - Xd holds the highest address to be copied to + 1.
 - On completion of the instruction:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.

For CPYETWN, option A, when $PSTATE.C = 0$:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
 - Xn holds -1 times the number of bytes remaining to be copied.
 - Xs holds the lowest address to be copied from - Xn.
 - Xd holds the lowest address to be copied to - Xn.
 - On completion of the instruction, Xn holds 0.
- If the copy is in the backward direction (Xn holds a positive number), then:
 - If the copy is in the backward direction (Xn holds a positive number), then:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address to be copied from + 1 - Xn.
 - Xd holds the highest address to be copied to + 1 - Xn.
 - On completion of the instruction, Xn holds 0.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied from.
 - The highest address that has been copied from + 1.
 - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied to.
 - The highest address that has not been copied to + 1.

For CPYETWN, option B, when $PSTATE.C = 1$:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction ($PSTATE.N == 0$), then:
 - Xs holds the lowest address to be copied from.
 - Xd holds the lowest address to be copied to.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
- If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - Xs holds the highest address to be copied from + 1.
 - Xd holds the highest address to be copied to + 1.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied from.
 - The highest address that has been copied from + 1.
 - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied to.
 - The highest address that has not been copied to + 1.

Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of $PSTATE.UAO$ is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of $HCR_EL2().\{E2H, TGE\}$ is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer (FEAT_MOPS)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
sz	0	1	1	1	0	1	op1	0	Rs							0	1	1	1	0	1	Rn							Rd			
op0										op2																						

Encoding for the Prologue variant

Applies when (op1 == 00)

```
CPYPTWN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when (op1 == 01)

```
CPYMTWN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when (op1 == 10)

```
CPYETWN [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
  when '00' => memcpy.stage = MOPSSStage_Prologue;
  when '01' => memcpy.stage = MOPSSStage_Main;
  when '10' => memcpy.stage = MOPSSStage_Epilogue;
end;
```

For information about the `CONSTRAINED UNPREDICTABLE` behavior of this instruction, see [Architectural Constraints on UNPREDICTABLE behaviors](#), and particularly [Memory Copy and Memory Set CPY*](#) and [Crossing a page boundary with different memory types or Shareability attributes](#).

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.


```

CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv      = PSTATE.[N,Z,C,V];
memcpy.toaddress = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);

if memcpy.stage == MOPSTage_Prologue then
    memcpy.cpy_size = UInt(X{64}(memcpy.n));
else
    memcpy.cpy_size = SInt(X{64}(memcpy.n));
end;

memcpy.implements_option_a = CPYOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
    else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
    else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD, rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSTage_Prologue then
    if memcpy.cpy_size > ArchMaxMOPSCPysize then
        memcpy.cpy_size = ArchMaxMOPSCPysize;
    end;

    memcpy.forward = IsMemCpyForward(memcpy);

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        if memcpy.forward then
            // Copy in the forward direction offsets the arguments.
            memcpy.toaddress = memcpy.toaddress + memcpy.cpy_size;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpy_size;
            memcpy.cpy_size = 0 - memcpy.cpy_size;
        end;
    else
        if !memcpy.forward then
            // Copy in the reverse direction offsets the arguments.
            memcpy.toaddress = memcpy.toaddress + memcpy.cpy_size;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpy_size;
            memcpy.nzcv = '1010';
        else
            memcpy.nzcv = '0010';
        end;
    end;
end;

memcpy.stagecpy_size = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSTage_Prologue then
    memcpy.forward = memcpy.cpy_size < 0 || (!memcpy.implements_option_a && memcpy.nzcv[3] == '0');
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var fault : boolean = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpy_size != 0 && !fault looplimit ArchMaxMOPSCPysize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);

        if memcpy.forward then
            assert B <= -1 * memcpy.stagecpy_size;
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                memcpy.toaddress + memcpy.cpy_size,
                memcpy.fromaddress + memcpy.cpy_size,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.cpy_size = memcpy.cpy_size + B;
                memcpy.stagecpy_size = memcpy.stagecpy_size + B;
            end;
        else
            assert B <= memcpy.stagecpy_size;
            memcpy.cpy_size = memcpy.cpy_size - B;
            memcpy.stagecpy_size = memcpy.stagecpy_size - B;
        end;
    end;
end;

```

```

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
            memcpy.toaddress + memcpy.cpy_size,
            memcpy.fromaddress + memcpy.cpy_size,
            memcpy.forward, B, raccdesc,
            waccdesc);

    if copied != B then
        fault = TRUE;
        memcpy.cpy_size = memcpy.cpy_size + B;
        memcpy.stagecpy_size = memcpy.stagecpy_size + B;
    end;
end;
end;

else
    while memcpy.stagecpy_size > 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpy_size;

        if memcpy.forward then
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                memcpy.fromaddress,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress + B;
                memcpy.toaddress = memcpy.toaddress + B;
            end;
        else
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress - B,
                memcpy.fromaddress - B,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress - B;
                memcpy.toaddress = memcpy.toaddress - B;
            end;
        end;

        if !fault then
            memcpy.cpy_size = memcpy.cpy_size - B;
            memcpy.stagecpy_size = memcpy.stagecpy_size - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;

```

2025-12_rel2025-09_diff_tag 2025-12-102025-11-28 16:4129:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)

htmldiff from-

(new)

CPYPWN, CPYMWN, CPYEWN

Memory copy, writes non-temporal

These instructions copy a requested number of bytes in memory from a source address to a destination address. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYPWN, then CPYMWN, and then CPYEWN.

CPYPWN performs some preconditioning of the arguments suitable for using the CPYMWN instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYMWN copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYEWN copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see [Memory Copy and Memory Set exceptions](#).

The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYPWN:

- If $Xn[63:55] \neq '000000000'$, the copy size Xn is saturated to $0x007FFFFFFFFFFFFFFF$.
- After saturation is performed, the direction of the memory copy is based on the following:
If $(Xs[55:0] > Xd[55:0])$ and $(Xd[55:0] + \text{saturated copy size}) > Xs[55:0]$, then the direction is forward.
If $(Xs[55:0] < Xd[55:0])$ and $(Xs[55:0] + \text{saturated copy size}) > Xd[55:0]$, then the direction is backward.
Otherwise, the direction is an IMPLEMENTATION DEFINED choice between forward and backward.

On completion of CPYPWN, option A:

- $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,0,0\}$.
- If the copy is in the forward direction, then:
 - Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the original Xs + saturated copy size.
 - Xd holds the original Xd + saturated copy size.
- If the copy is in the backward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs and Xd are unchanged.

On completion of CPYPWN, option B:

- If the copy is in the forward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
 - $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,1,0\}$.
- If the copy is in the backward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.
 - $PSTATE.\{N,Z,C,V\}$ are set to $\{1,0,1,0\}$.

For CPYMWN, option A, when $PSTATE.C = 0$:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
 - Xn holds -1 times the number of bytes remaining to be copied.
 - Xs holds the lowest address to be copied from - Xn .
 - Xd holds the lowest address to be copied to - Xn .
 - On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.
- If the copy is in the backward direction (Xn holds a positive number), then:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address to be copied from + 1 - Xn .
 - Xd holds the highest address to be copied to + 1 - Xn .
 - On completion of the instruction, Xn holds the number of bytes remaining to be copied.

For CPYMWN, option B, when $PSTATE.C = 1$:

- Xn holds the number of bytes remaining to be copied.

- If the copy is in the forward direction ($PSTATE.N == 0$), then:
 - Xs holds the lowest address to be copied from.
 - Xd holds the lowest address to be copied to.
 - On completion of the instruction:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
- If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - Xs holds the highest address to be copied from + 1.
 - Xd holds the highest address to be copied to + 1.
 - On completion of the instruction:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.

For CPYEWN, option A, when $PSTATE.C = 0$:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
 - Xn holds -1 times the number of bytes remaining to be copied.
 - Xs holds the lowest address to be copied from - Xn.
 - Xd holds the lowest address to be copied to - Xn.
 - On completion of the instruction, Xn holds 0.
- If the copy is in the backward direction (Xn holds a positive number), then:
 - If the copy is in the backward direction (Xn holds a positive number), then:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address to be copied from + 1 - Xn.
 - Xd holds the highest address to be copied to + 1 - Xn.
 - On completion of the instruction, Xn holds 0.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied from.
 - The highest address that has been copied from + 1.
 - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied to.
 - The highest address that has not been copied to + 1.

For CPYEWN, option B, when $PSTATE.C = 1$:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction ($PSTATE.N == 0$), then:
 - Xs holds the lowest address to be copied from.
 - Xd holds the lowest address to be copied to.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
- If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - Xs holds the highest address to be copied from + 1.
 - Xd holds the highest address to be copied to + 1.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied from.
 - The highest address that has been copied from + 1.
 - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied to.
 - The highest address that has not been copied to + 1.

Integer

(FEAT_MOPS)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sz	0	1	1	1	0	1	op1	0	Rs				0	1	0	0	0	1	Rn				Rd								
o0								op2																							

Encoding for the Prologue variant

Applies when ($op1 == 00$)

CPYPWN [<Xd>]!, [<Xs>]!, <Xn>!

Encoding for the Main variant

Applies when (op1 == 01)

CPYMWN [<Xd>]!, [<Xs>]!, <Xn>!

Encoding for the Epilogue variant

Applies when (op1 == 10)

CPYEWN [<Xd>]!, [<Xs>]!, <Xn>!

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rntemporal : boolean = options[3] == '1';
let wntemporal : boolean = options[2] == '1';
case op1 of
  when '00' => memcpy.stage = MOPSStage_Prologue;
  when '01' => memcpy.stage = MOPSStage_Main;
  when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the `CONSTRAINED UNPREDICTABLE` behavior of this instruction, see [Architectural Constraints on UNPREDICTABLE behaviors](#), and particularly [Memory Copy and Memory Set CPY*](#) and [Crossing a page boundary with different memory types or Shareability attributes](#).

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.


```

CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv      = PSTATE.[N,Z,C,V];
memcpy.toaddress = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);

if memcpy.stage == MOPSTage_Prologue then
    memcpy.cpy_size = UInt(X{64}(memcpy.n));
else
    memcpy.cpy_size = SInt(X{64}(memcpy.n));
end;

memcpy.implements_option_a = CPYOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
    else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
    else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD, rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSTage_Prologue then
    if memcpy.cpy_size > ArchMaxMOPSCPysize then
        memcpy.cpy_size = ArchMaxMOPSCPysize;
    end;

    memcpy.forward = IsMemCpyForward(memcpy);

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        if memcpy.forward then
            // Copy in the forward direction offsets the arguments.
            memcpy.toaddress = memcpy.toaddress + memcpy.cpy_size;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpy_size;
            memcpy.cpy_size = 0 - memcpy.cpy_size;
        end;
    else
        if !memcpy.forward then
            // Copy in the reverse direction offsets the arguments.
            memcpy.toaddress = memcpy.toaddress + memcpy.cpy_size;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpy_size;
            memcpy.nzcv = '1010';
        else
            memcpy.nzcv = '0010';
        end;
    end;
end;

memcpy.stagecpy_size = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSTage_Prologue then
    memcpy.forward = memcpy.cpy_size < 0 || (!memcpy.implements_option_a && memcpy.nzcv[3] == '0');
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var fault : boolean = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpy_size != 0 && !fault looplimit ArchMaxMOPSCPysize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPySizeChoice(memcpy);

        if memcpy.forward then
            assert B <= -1 * memcpy.stagecpy_size;
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                memcpy.toaddress + memcpy.cpy_size,
                memcpy.fromaddress + memcpy.cpy_size,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.cpy_size = memcpy.cpy_size + B;
                memcpy.stagecpy_size = memcpy.stagecpy_size + B;
            end;
        else
            assert B <= memcpy.stagecpy_size;
            memcpy.cpy_size = memcpy.cpy_size - B;
            memcpy.stagecpy_size = memcpy.stagecpy_size - B;
        end;
    end;
end;

```

```

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
            memcpy.toaddress + memcpy.cpyssize,
            memcpy.fromaddress + memcpy.cpyssize,
            memcpy.forward, B, raccdesc,
            waccdesc);

    if copied != B then
        fault = TRUE;
        memcpy.cpyssize = memcpy.cpyssize + B;
        memcpy.stagecpyssize = memcpy.stagecpyssize + B;
    end;
end;
end;

else
    while memcpy.stagecpyssize > 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpyssize;

        if memcpy.forward then
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                memcpy.fromaddress,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress + B;
                memcpy.toaddress = memcpy.toaddress + B;
            end;
        else
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress - B,
                memcpy.fromaddress - B,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress - B;
                memcpy.toaddress = memcpy.toaddress - B;
            end;
        end;

        if !fault then
            memcpy.cpyssize = memcpy.cpyssize - B;
            memcpy.stagecpyssize = memcpy.stagecpyssize - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSSstage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;

```

2025-12_rel2025-09_diff_tag 2025-12-102025-11-28 16:4129:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)

htmldiff from-

(new)

CPYPWT, CPYMW, CPYEWT

Memory copy, writes unprivileged

These instructions copy a requested number of bytes in memory from a source address to a destination address. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYPWT, then CPYMW, and then CPYEWT.

CPYPWT performs some preconditioning of the arguments suitable for using the CPYMW instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYMW copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYEWT copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see [Memory Copy and Memory Set exceptions](#).

The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYPWT:

- If $Xn[63:55] \neq '000000000'$, the copy size Xn is saturated to $0x007FFFFFFFFFFFFFFF$.
- After saturation is performed, the direction of the memory copy is based on the following:
If $(Xs[55:0] > Xd[55:0])$ and $(Xd[55:0] + \text{saturated copy size}) > Xs[55:0]$, then the direction is forward.
If $(Xs[55:0] < Xd[55:0])$ and $(Xs[55:0] + \text{saturated copy size}) > Xd[55:0]$, then the direction is backward.
Otherwise, the direction is an IMPLEMENTATION DEFINED choice between forward and backward.

On completion of CPYPWT, option A:

- $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,0,0\}$.
- If the copy is in the forward direction, then:
 - Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the original $Xs + \text{saturated copy size}$.
 - Xd holds the original $Xd + \text{saturated copy size}$.
- If the copy is in the backward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs and Xd are unchanged.

On completion of CPYPWT, option B:

- If the copy is in the forward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
 - $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,1,0\}$.
- If the copy is in the backward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.
 - $PSTATE.\{N,Z,C,V\}$ are set to $\{1,0,1,0\}$.

For CPYMW, option A, when $PSTATE.C = 0$:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
 - Xn holds -1 times the number of bytes remaining to be copied.
 - Xs holds the lowest address to be copied from - Xn .
 - Xd holds the lowest address to be copied to - Xn .
 - On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.
- If the copy is in the backward direction (Xn holds a positive number), then:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address to be copied from + 1 - Xn .
 - Xd holds the highest address to be copied to + 1 - Xn .
 - On completion of the instruction, Xn holds the number of bytes remaining to be copied.

For CPYMW, option B, when $PSTATE.C = 1$:

- Xn holds the number of bytes remaining to be copied.

- If the copy is in the forward direction ($PSTATE.N == 0$), then:
 - Xs holds the lowest address to be copied from.
 - Xd holds the lowest address to be copied to.
 - On completion of the instruction:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
- If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - Xs holds the highest address to be copied from + 1.
 - Xd holds the highest address to be copied to + 1.
 - On completion of the instruction:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.

For CPYEWT, option A, when $PSTATE.C = 0$:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
 - Xn holds -1 times the number of bytes remaining to be copied.
 - Xs holds the lowest address to be copied from - Xn.
 - Xd holds the lowest address to be copied to - Xn.
 - On completion of the instruction, Xn holds 0.
- If the copy is in the backward direction (Xn holds a positive number), then:
 - If the copy is in the backward direction (Xn holds a positive number), then:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address to be copied from + 1 - Xn.
 - Xd holds the highest address to be copied to + 1 - Xn.
 - On completion of the instruction, Xn holds 0.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied from.
 - The highest address that has been copied from + 1.
 - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied to.
 - The highest address that has not been copied to + 1.

For CPYEWT, option B, when $PSTATE.C = 1$:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction ($PSTATE.N == 0$), then:
 - Xs holds the lowest address to be copied from.
 - Xd holds the lowest address to be copied to.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
- If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - Xs holds the highest address to be copied from + 1.
 - Xd holds the highest address to be copied to + 1.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied from.
 - The highest address that has been copied from + 1.
 - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied to.
 - The highest address that has not been copied to + 1.

Explicit Memory Write effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of $PSTATE.UAO$ is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of $HCR_EL2().\{E2H, TGE\}$ is '11'.

Otherwise, the Explicit Memory Write effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer

(FEAT_MOPS)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sz	0	1	1	1	0	1	op1	0	Rs						0	0	0	1	0	1	Rn						Rd				
o0								op2																							

Encoding for the Prologue variant

Applies when (op1 == 00)

CPYPWT [<Xd>]!, [<Xs>]!, <Xn>!

Encoding for the Main variant

Applies when (op1 == 01)

CPYMWWT [<Xd>]!, [<Xs>]!, <Xn>!

Encoding for the Epilogue variant

Applies when (op1 == 10)

CPYEWWT [<Xd>]!, [<Xs>]!, <Xn>!

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
  when '00' => memcpy.stage = MOPSSStage_Prologue;
  when '01' => memcpy.stage = MOPSSStage_Main;
  when '10' => memcpy.stage = MOPSSStage_Epilogue;
end;
```

For information about the `CONSTRAINED UNPREDICTABLE` behavior of this instruction, see [Architectural Constraints on UNPREDICTABLE behaviors](#), and particularly [Memory Copy and Memory Set CPY*](#) and [Crossing a page boundary with different memory types or Shareability attributes](#).

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.


```

CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv      = PSTATE.[N,Z,C,V];
memcpy.toaddress = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);

if memcpy.stage == MOPSTage_Prologue then
    memcpy.cpy_size = UInt(X{64}(memcpy.n));
else
    memcpy.cpy_size = SInt(X{64}(memcpy.n));
end;

memcpy.implements_option_a = CPYOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD, rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSTage_Prologue then
    if memcpy.cpy_size > ArchMaxMOPSCPysize then
        memcpy.cpy_size = ArchMaxMOPSCPysize;
    end;

    memcpy.forward = IsMemCpyForward(memcpy);

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        if memcpy.forward then
            // Copy in the forward direction offsets the arguments.
            memcpy.toaddress = memcpy.toaddress + memcpy.cpy_size;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpy_size;
            memcpy.cpy_size = 0 - memcpy.cpy_size;
        end;
    else
        if !memcpy.forward then
            // Copy in the reverse direction offsets the arguments.
            memcpy.toaddress = memcpy.toaddress + memcpy.cpy_size;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpy_size;
            memcpy.nzcv = '1010';
        else
            memcpy.nzcv = '0010';
        end;
    end;
end;

memcpy.stagecpy_size = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSTage_Prologue then
    memcpy.forward = memcpy.cpy_size < 0 || (!memcpy.implements_option_a && memcpy.nzcv[3] == '0');
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var fault : boolean = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpy_size != 0 && !fault looplimit ArchMaxMOPSCPysize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);

        if memcpy.forward then
            assert B <= -1 * memcpy.stagecpy_size;
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                memcpy.toaddress + memcpy.cpy_size,
                memcpy.fromaddress + memcpy.cpy_size,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.cpy_size = memcpy.cpy_size + B;
                memcpy.stagecpy_size = memcpy.stagecpy_size + B;
            end;
        else
            assert B <= memcpy.stagecpy_size;
            memcpy.cpy_size = memcpy.cpy_size - B;
            memcpy.stagecpy_size = memcpy.stagecpy_size - B;
        end;
    end;
end;

```

```

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
            memcpy.toaddress + memcpy.cpy_size,
            memcpy.fromaddress + memcpy.cpy_size,
            memcpy.forward, B, raccdesc,
            waccdesc);

    if copied != B then
        fault = TRUE;
        memcpy.cpy_size = memcpy.cpy_size + B;
        memcpy.stagecpy_size = memcpy.stagecpy_size + B;
    end;
end;
end;

else
    while memcpy.stagecpy_size > 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpy_size;

        if memcpy.forward then
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                memcpy.fromaddress,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress + B;
                memcpy.toaddress = memcpy.toaddress + B;
            end;
        else
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress - B,
                memcpy.fromaddress - B,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress - B;
                memcpy.toaddress = memcpy.toaddress - B;
            end;
        end;

        if !fault then
            memcpy.cpy_size = memcpy.cpy_size - B;
            memcpy.stagecpy_size = memcpy.stagecpy_size - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;

```

2025-12_rel2025-09_diff_tag 2025-12-102025-11-28 16:4129:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)

htmldiff from-

(new)

CPYPWTN, CPYMWTN, CPYEWTN

Memory copy, writes unprivileged, reads and writes non-temporal

These instructions copy a requested number of bytes in memory from a source address to a destination address. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYPWTN, then CPYMWTN, and then CPYEWTN.

CPYPWTN performs some preconditioning of the arguments suitable for using the CPYMWTN instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYMWTN copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYEWTN copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see [Memory Copy and Memory Set exceptions](#).

The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYPWTN:

- If $X_n[63:55] \neq '000000000'$, the copy size X_n is saturated to $0x007FFFFFFFFFFFFFFF$.
- After saturation is performed, the direction of the memory copy is based on the following:
 If $(X_s[55:0] > X_d[55:0])$ and $(X_d[55:0] + \text{saturated copy size}) > X_s[55:0]$, then the direction is forward.
 If $(X_s[55:0] < X_d[55:0])$ and $(X_s[55:0] + \text{saturated copy size}) > X_d[55:0]$, then the direction is backward.
 Otherwise, the direction is an IMPLEMENTATION DEFINED choice between forward and backward.

On completion of CPYPWTN, option A:

- $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,0,0\}$.
- If the copy is in the forward direction, then:
 - X_n holds -1 times the number of bytes in the saturated copy size remaining to be copied.
 - X_s holds the original X_s + saturated copy size.
 - X_d holds the original X_d + saturated copy size.
- If the copy is in the backward direction, then:
 - X_n holds the number of bytes in the saturated copy size remaining to be copied.
 - X_s and X_d are unchanged.

On completion of CPYPWTN, option B:

- If the copy is in the forward direction, then:
 - X_n holds the number of bytes in the saturated copy size remaining to be copied.
 - X_s holds the lowest address that has not been copied from.
 - X_d holds the lowest address that has not been copied to.
 - $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,1,0\}$.
- If the copy is in the backward direction, then:
 - X_n holds the number of bytes in the saturated copy size remaining to be copied.
 - X_s holds the highest address that has not been copied from + 1.
 - X_d holds the highest address that has not been copied to + 1.
 - $PSTATE.\{N,Z,C,V\}$ are set to $\{1,0,1,0\}$.

For CPYMWTN, option A, when $PSTATE.C = 0$:

- X_n holds a signed 64-bit integer.
- If the copy is in the forward direction (X_n holds a negative number), then:
 - X_n holds -1 times the number of bytes remaining to be copied.
 - X_s holds the lowest address to be copied from - X_n .
 - X_d holds the lowest address to be copied to - X_n .
 - On completion of the instruction, X_n holds -1 times the number of bytes remaining to be copied.
- If the copy is in the backward direction (X_n holds a positive number), then:
 - X_n holds the number of bytes remaining to be copied.
 - X_s holds the highest address to be copied from + 1 - X_n .
 - X_d holds the highest address to be copied to + 1 - X_n .
 - On completion of the instruction, X_n holds the number of bytes remaining to be copied.

For CPYMWTN, option B, when $PSTATE.C = 1$:

- X_n holds the number of bytes remaining to be copied.

- If the copy is in the forward direction ($PSTATE.N == 0$), then:
 - Xs holds the lowest address to be copied from.
 - Xd holds the lowest address to be copied to.
 - On completion of the instruction:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
- If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - Xs holds the highest address to be copied from + 1.
 - Xd holds the highest address to be copied to + 1.
 - On completion of the instruction:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.

For CPYEWTN, option A, when $PSTATE.C = 0$:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
 - Xn holds -1 times the number of bytes remaining to be copied.
 - Xs holds the lowest address to be copied from - Xn.
 - Xd holds the lowest address to be copied to - Xn.
 - On completion of the instruction, Xn holds 0.
- If the copy is in the backward direction (Xn holds a positive number), then:
 - If the copy is in the backward direction (Xn holds a positive number), then:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address to be copied from + 1 - Xn.
 - Xd holds the highest address to be copied to + 1 - Xn.
 - On completion of the instruction, Xn holds 0.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied from.
 - The highest address that has been copied from + 1.
 - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied to.
 - The highest address that has not been copied to + 1.

For CPYEWTN, option B, when $PSTATE.C = 1$:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction ($PSTATE.N == 0$), then:
 - Xs holds the lowest address to be copied from.
 - Xd holds the lowest address to be copied to.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
- If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - If the copy is in the backward direction ($PSTATE.N == 1$), then:
 - Xs holds the highest address to be copied from + 1.
 - Xd holds the highest address to be copied to + 1.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied from.
 - The highest address that has been copied from + 1.
 - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied to.
 - The highest address that has not been copied to + 1.

Explicit Memory Write effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of $PSTATE.UAO$ is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of $HCR_EL2().\{E2H, TGE\}$ is '11'.

Otherwise, the Explicit Memory Write effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer

(FEAT_MOPS)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
sz	0	1	1	1	0	1	op1	0	Rs							1	1	0	1	0	1	Rn							Rd						
o0										op2																									

Encoding for the Prologue variant

Applies when (op1 == 00)

CPYPWTN [<Xd>]!, [<Xs>]!, <Xn>!

Encoding for the Main variant

Applies when (op1 == 01)

CPYMWTN [<Xd>]!, [<Xs>]!, <Xn>!

Encoding for the Epilogue variant

Applies when (op1 == 10)

CPYEWTN [<Xd>]!, [<Xs>]!, <Xn>!

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
  when '00' => memcpy.stage = MOPSSStage_Prologue;
  when '01' => memcpy.stage = MOPSSStage_Main;
  when '10' => memcpy.stage = MOPSSStage_Epilogue;
end;
```

For information about the `CONSTRAINED UNPREDICTABLE` behavior of this instruction, see [Architectural Constraints on UNPREDICTABLE behaviors](#), and particularly [Memory Copy and Memory Set CPY*](#) and [Crossing a page boundary with different memory types or Shareability attributes](#).

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.


```

CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv      = PSTATE.[N,Z,C,V];
memcpy.toaddress = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);

if memcpy.stage == MOPSTage_Prologue then
    memcpy.cpy_size = UInt(X{64}(memcpy.n));
else
    memcpy.cpy_size = SInt(X{64}(memcpy.n));
end;

memcpy.implements_option_a = CPYOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
    else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
    else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD, rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSTage_Prologue then
    if memcpy.cpy_size > ArchMaxMOPSCPysize then
        memcpy.cpy_size = ArchMaxMOPSCPysize;
    end;

    memcpy.forward = IsMemCpyForward(memcpy);

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        if memcpy.forward then
            // Copy in the forward direction offsets the arguments.
            memcpy.toaddress = memcpy.toaddress + memcpy.cpy_size;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpy_size;
            memcpy.cpy_size = 0 - memcpy.cpy_size;
        end;
    else
        if !memcpy.forward then
            // Copy in the reverse direction offsets the arguments.
            memcpy.toaddress = memcpy.toaddress + memcpy.cpy_size;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpy_size;
            memcpy.nzcv = '1010';
        else
            memcpy.nzcv = '0010';
        end;
    end;
end;

memcpy.stagecpy_size = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSTage_Prologue then
    memcpy.forward = memcpy.cpy_size < 0 || (!memcpy.implements_option_a && memcpy.nzcv[3] == '0');
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var fault : boolean = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpy_size != 0 && !fault looplimit ArchMaxMOPSCPysize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);

        if memcpy.forward then
            assert B <= -1 * memcpy.stagecpy_size;
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                memcpy.toaddress + memcpy.cpy_size,
                memcpy.fromaddress + memcpy.cpy_size,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.cpy_size = memcpy.cpy_size + B;
                memcpy.stagecpy_size = memcpy.stagecpy_size + B;
            end;
        else
            assert B <= memcpy.stagecpy_size;
            memcpy.cpy_size = memcpy.cpy_size - B;
            memcpy.stagecpy_size = memcpy.stagecpy_size - B;
        end;
    end;
end;

```

```

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
            memcpy.toaddress + memcpy.cpy_size,
            memcpy.fromaddress + memcpy.cpy_size,
            memcpy.forward, B, raccdesc,
            waccdesc);

    if copied != B then
        fault = TRUE;
        memcpy.cpy_size = memcpy.cpy_size + B;
        memcpy.stagecpy_size = memcpy.stagecpy_size + B;
    end;
end;
end;

else
    while memcpy.stagecpy_size > 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpy_size;

        if memcpy.forward then
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                memcpy.fromaddress,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress + B;
                memcpy.toaddress = memcpy.toaddress + B;
            end;
        else
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress - B,
                memcpy.fromaddress - B,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress - B;
                memcpy.toaddress = memcpy.toaddress - B;
            end;
        end;

        if !fault then
            memcpy.cpy_size = memcpy.cpy_size - B;
            memcpy.stagecpy_size = memcpy.stagecpy_size - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;

```

2025-12_rel2025-09_diff_tag 2025-12-102025-11-28 16:4129:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)

htmldiff from-

(new)

CPYPWTRN, CPYMWTRN, CPYEWTRN

Memory copy, writes unprivileged, reads non-temporal

These instructions copy a requested number of bytes in memory from a source address to a destination address. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYPWTRN, then CPYMWTRN, and then CPYEWTRN.

CPYPWTRN performs some preconditioning of the arguments suitable for using the CPYMWTRN instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYMWTRN copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYEWTRN copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see [Memory Copy and Memory Set exceptions](#).

The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYPWTRN:

- If $Xn[63:55] \neq '00000000'$, the copy size Xn is saturated to $0x007FFFFFFFFFFFFFFF$.
- After saturation is performed, the direction of the memory copy is based on the following:
If $(Xs[55:0] > Xd[55:0])$ and $(Xd[55:0] + \text{saturated copy size}) > Xs[55:0]$, then the direction is forward.
If $(Xs[55:0] < Xd[55:0])$ and $(Xs[55:0] + \text{saturated copy size}) > Xd[55:0]$, then the direction is backward.
Otherwise, the direction is an IMPLEMENTATION DEFINED choice between forward and backward.

On completion of CPYPWTRN, option A:

- $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,0,0\}$.
- If the copy is in the forward direction, then:
 - Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the original $Xs + \text{saturated copy size}$.
 - Xd holds the original $Xd + \text{saturated copy size}$.
- If the copy is in the backward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs and Xd are unchanged.

On completion of CPYPWTRN, option B:

- If the copy is in the forward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
 - $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,1,0\}$.
- If the copy is in the backward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.
 - $PSTATE.\{N,Z,C,V\}$ are set to $\{1,0,1,0\}$.

For CPYMWTRN, option A, when $PSTATE.C = 0$:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
 - Xn holds -1 times the number of bytes remaining to be copied.
 - Xs holds the lowest address to be copied from - Xn .
 - Xd holds the lowest address to be copied to - Xn .
 - On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.
- If the copy is in the backward direction (Xn holds a positive number), then:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address to be copied from + 1 - Xn .
 - Xd holds the highest address to be copied to + 1 - Xn .
 - On completion of the instruction, Xn holds the number of bytes remaining to be copied.

For CPYMWTRN, option B, when PSTATE.C = 1:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction (PSTATE.N == 0), then:
 - Xs holds the lowest address to be copied from.
 - Xd holds the lowest address to be copied to.
 - On completion of the instruction:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
- If the copy is in the backward direction (PSTATE.N == 1), then:
 - Xs holds the highest address to be copied from + 1.
 - Xd holds the highest address to be copied to + 1.
 - On completion of the instruction:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.

For CPYEWTRN, option A, when PSTATE.C = 0:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
 - Xn holds -1 times the number of bytes remaining to be copied.
 - Xs holds the lowest address to be copied from - Xn.
 - Xd holds the lowest address to be copied to - Xn.
 - On completion of the instruction, Xn holds 0.
- ~~If the copy is in the backward direction (Xn holds a positive number), then:~~
 If the copy is in the backward direction (Xn holds a positive number), then:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address to be copied from + 1 - Xn.
 - Xd holds the highest address to be copied to + 1 - Xn.
 - On completion of the instruction, Xn holds 0.
- On completion of the instruction:
 - Xn holds 0.
 - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied from.
 - The highest address that has been copied from + 1.
 - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied to.
 - The highest address that has not been copied to + 1.

For CPYEWTRN, option B, when PSTATE.C = 1:

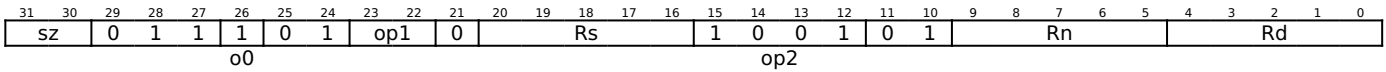
- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction (PSTATE.N == 0), then:
 - Xs holds the lowest address to be copied from.
 - Xd holds the lowest address to be copied to.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
- ~~If the copy is in the backward direction (PSTATE.N == 1), then:~~
 If the copy is in the backward direction (PSTATE.N == 1), then:
 - Xs holds the highest address to be copied from + 1.
 - Xd holds the highest address to be copied to + 1.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.
- On completion of the instruction:
 - Xn holds 0.
 - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied from.
 - The highest address that has been copied from + 1.
 - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied to.
 - The highest address that has not been copied to + 1.

Explicit Memory Write effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of HCR_EL2().{E2H, TGE} is '11'.

Otherwise, the Explicit Memory Write effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer (FEAT_MOPS)



Encoding for the Prologue variant

Applies when (op1 == 00)

CPYPWTRN [<Xd>]!, [<Xs>]!, <Xn>!

Encoding for the Main variant

Applies when (op1 == 01)

CPYMWTRN [<Xd>]!, [<Xs>]!, <Xn>!

Encoding for the Epilogue variant

Applies when (op1 == 10)

CPYEWTRN [<Xd>]!, [<Xs>]!, <Xn>!

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rntemporal : boolean = options[3] == '1';
let wntemporal : boolean = options[2] == '1';
case op1 of
  when '00' => memcpy.stage = MOPSStage_Prologue;
  when '01' => memcpy.stage = MOPSStage_Main;
  when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the **CONSTRAINED UNPREDICTABLE** behavior of this instruction, see [Architectural Constraints on UNPREDICTABLE behaviors](#), and particularly [Memory Copy and Memory Set CPY*](#) and [Crossing a page boundary with different memory types or Shareability attributes](#).

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.


```

CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv      = PSTATE.[N,Z,C,V];
memcpy.toaddress = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);

if memcpy.stage == MOPSTage_Prologue then
    memcpy.cpy_size = UInt(X{64}(memcpy.n));
else
    memcpy.cpy_size = SInt(X{64}(memcpy.n));
end;

memcpy.implements_option_a = CPYOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
    else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
    else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD, rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSTage_Prologue then
    if memcpy.cpy_size > ArchMaxMOPSCPysize then
        memcpy.cpy_size = ArchMaxMOPSCPysize;
    end;

    memcpy.forward = IsMemCpyForward(memcpy);

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        if memcpy.forward then
            // Copy in the forward direction offsets the arguments.
            memcpy.toaddress = memcpy.toaddress + memcpy.cpy_size;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpy_size;
            memcpy.cpy_size = 0 - memcpy.cpy_size;
        end;
    else
        if !memcpy.forward then
            // Copy in the reverse direction offsets the arguments.
            memcpy.toaddress = memcpy.toaddress + memcpy.cpy_size;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpy_size;
            memcpy.nzcv = '1010';
        else
            memcpy.nzcv = '0010';
        end;
    end;
end;

memcpy.stagecpy_size = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSTage_Prologue then
    memcpy.forward = memcpy.cpy_size < 0 || (!memcpy.implements_option_a && memcpy.nzcv[3] == '0');
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var fault : boolean = FALSE;
var B : MOPSTBlock Size = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpy_size != 0 && !fault looplimit ArchMaxMOPSCPysize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);

        if memcpy.forward then
            assert B <= -1 * memcpy.stagecpy_size;
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                memcpy.toaddress + memcpy.cpy_size,
                memcpy.fromaddress + memcpy.cpy_size,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.cpy_size = memcpy.cpy_size + B;
                memcpy.stagecpy_size = memcpy.stagecpy_size + B;
            end;
        else
            assert B <= memcpy.stagecpy_size;
            memcpy.cpy_size = memcpy.cpy_size - B;
            memcpy.stagecpy_size = memcpy.stagecpy_size - B;
        end;
    end;
end;

```

```

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
            memcpy.toaddress + memcpy.cpy_size,
            memcpy.fromaddress + memcpy.cpy_size,
            memcpy.forward, B, raccdesc,
            waccdesc);

    if copied != B then
        fault = TRUE;
        memcpy.cpy_size = memcpy.cpy_size + B;
        memcpy.stagecpy_size = memcpy.stagecpy_size + B;
    end;
end;
end;

else
    while memcpy.stagecpy_size > 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpy_size;

        if memcpy.forward then
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                memcpy.fromaddress,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress + B;
                memcpy.toaddress = memcpy.toaddress + B;
            end;
        else
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress - B,
                memcpy.fromaddress - B,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress - B;
                memcpy.toaddress = memcpy.toaddress - B;
            end;
        end;

        if !fault then
            memcpy.cpy_size = memcpy.cpy_size - B;
            memcpy.stagecpy_size = memcpy.stagecpy_size - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;

```

2025-12_rel2025-09_diff_tag 2025-12-102025-11-28 16:4129:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)

htmldiff from-

(new)

CPYPWTWN, CPYMWTWN, CPYEWTWN

Memory copy, writes unprivileged and non-temporal

These instructions copy a requested number of bytes in memory from a source address to a destination address. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYPWTWN, then CPYMWTWN, and then CPYEWTWN.

CPYPWTWN performs some preconditioning of the arguments suitable for using the CPYMWTWN instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYMWTWN copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYEWTWN copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see [Memory Copy and Memory Set exceptions](#).

The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYPWTWN:

- If $Xn[63:55] \neq '00000000'$, the copy size Xn is saturated to $0x007FFFFFFFFFFFFFFF$.
- After saturation is performed, the direction of the memory copy is based on the following:
If $(Xs[55:0] > Xd[55:0])$ and $(Xd[55:0] + \text{saturated copy size}) > Xs[55:0]$, then the direction is forward.
If $(Xs[55:0] < Xd[55:0])$ and $(Xs[55:0] + \text{saturated copy size}) > Xd[55:0]$, then the direction is backward.
Otherwise, the direction is an IMPLEMENTATION DEFINED choice between forward and backward.

On completion of CPYPWTWN, option A:

- $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,0,0\}$.
- If the copy is in the forward direction, then:
 - Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the original $Xs + \text{saturated copy size}$.
 - Xd holds the original $Xd + \text{saturated copy size}$.
- If the copy is in the backward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs and Xd are unchanged.

On completion of CPYPWTWN, option B:

- If the copy is in the forward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
 - $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,1,0\}$.
- If the copy is in the backward direction, then:
 - Xn holds the number of bytes in the saturated copy size remaining to be copied.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.
 - $PSTATE.\{N,Z,C,V\}$ are set to $\{1,0,1,0\}$.

For CPYMWTWN, option A, when $PSTATE.C = 0$:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
 - Xn holds -1 times the number of bytes remaining to be copied.
 - Xs holds the lowest address to be copied from - Xn .
 - Xd holds the lowest address to be copied to - Xn .
 - On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.
- If the copy is in the backward direction (Xn holds a positive number), then:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address to be copied from + 1 - Xn .
 - Xd holds the highest address to be copied to + 1 - Xn .
 - On completion of the instruction, Xn holds the number of bytes remaining to be copied.

For CPYMWTTWN, option B, when PSTATE.C = 1:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction (PSTATE.N == 0), then:
 - Xs holds the lowest address to be copied from.
 - Xd holds the lowest address to be copied to.
 - On completion of the instruction:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
- If the copy is in the backward direction (PSTATE.N == 1), then:
 - Xs holds the highest address to be copied from + 1.
 - Xd holds the highest address to be copied to + 1.
 - On completion of the instruction:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.

For CPYEWTTWN, option A, when PSTATE.C = 0:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
 - Xn holds -1 times the number of bytes remaining to be copied.
 - Xs holds the lowest address to be copied from - Xn.
 - Xd holds the lowest address to be copied to - Xn.
 - On completion of the instruction, Xn holds 0.
- ~~If the copy is in the backward direction (Xn holds a positive number), then:~~
 If the copy is in the backward direction (Xn holds a positive number), then:
 - Xn holds the number of bytes remaining to be copied.
 - Xs holds the highest address to be copied from + 1 - Xn.
 - Xd holds the highest address to be copied to + 1 - Xn.
 - On completion of the instruction, Xn holds 0.
- On completion of the instruction:
 - Xn holds 0.
 - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied from.
 - The highest address that has been copied from + 1.
 - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied to.
 - The highest address that has not been copied to + 1.

For CPYEWTTWN, option B, when PSTATE.C = 1:

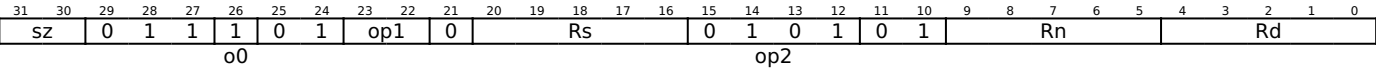
- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction (PSTATE.N == 0), then:
 - Xs holds the lowest address to be copied from.
 - Xd holds the lowest address to be copied to.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds the lowest address that has not been copied from.
 - Xd holds the lowest address that has not been copied to.
- ~~If the copy is in the backward direction (PSTATE.N == 1), then:~~
 If the copy is in the backward direction (PSTATE.N == 1), then:
 - Xs holds the highest address to be copied from + 1.
 - Xd holds the highest address to be copied to + 1.
 - On completion of the instruction:
 - Xn holds 0.
 - Xs holds the highest address that has not been copied from + 1.
 - Xd holds the highest address that has not been copied to + 1.
- On completion of the instruction:
 - Xn holds 0.
 - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied from.
 - The highest address that has been copied from + 1.
 - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
 - The lowest address that has been copied to.
 - The highest address that has not been copied to + 1.

Explicit Memory Write effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of HCR_EL2().{E2H, TGE} is '11'.

Otherwise, the Explicit Memory Write effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer
(FEAT_MOPS)



Encoding for the Prologue variant

Applies when (op1 == 00)

```
CPYPWTWN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when (op1 == 01)

```
CPYMWTWN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when (op1 == 10)

```
CPYEWTTN [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rntemporal : boolean = options[3] == '1';
let wntemporal : boolean = options[2] == '1';
case op1 of
  when '00' => memcpy.stage = MOPSStage_Prologue;
  when '01' => memcpy.stage = MOPSStage_Main;
  when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the **CONSTRAINED UNPREDICTABLE** behavior of this instruction, see [Architectural Constraints on UNPREDICTABLE behaviors](#), and particularly [Memory Copy and Memory Set CPY*](#) and [Crossing a page boundary with different memory types or Shareability attributes](#).

Assembler Symbols

- <Xd> For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field.
For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
- <Xs> For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field.
For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
- <Xn> For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field.
For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field.
For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.


```

CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv      = PSTATE.[N,Z,C,V];
memcpy.toaddress = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);

if memcpy.stage == MOPSTage_Prologue then
    memcpy.cpy_size = UInt(X{64}(memcpy.n));
else
    memcpy.cpy_size = SInt(X{64}(memcpy.n));
end;

memcpy.implements_option_a = CPYOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
    else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
    else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD, rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSTage_Prologue then
    if memcpy.cpy_size > ArchMaxMOPSCPysize then
        memcpy.cpy_size = ArchMaxMOPSCPysize;
    end;

    memcpy.forward = IsMemCpyForward(memcpy);

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        if memcpy.forward then
            // Copy in the forward direction offsets the arguments.
            memcpy.toaddress = memcpy.toaddress + memcpy.cpy_size;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpy_size;
            memcpy.cpy_size = 0 - memcpy.cpy_size;
        end;
    else
        if !memcpy.forward then
            // Copy in the reverse direction offsets the arguments.
            memcpy.toaddress = memcpy.toaddress + memcpy.cpy_size;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpy_size;
            memcpy.nzcv = '1010';
        else
            memcpy.nzcv = '0010';
        end;
    end;
end;

memcpy.stagecpy_size = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSTage_Prologue then
    memcpy.forward = memcpy.cpy_size < 0 || (!memcpy.implements_option_a && memcpy.nzcv[3] == '0');
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var fault : boolean = FALSE;
var B : MOPSTBlock Size = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpy_size != 0 && !fault looplimit ArchMaxMOPSCPysize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);

        if memcpy.forward then
            assert B <= -1 * memcpy.stagecpy_size;
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                memcpy.toaddress + memcpy.cpy_size,
                memcpy.fromaddress + memcpy.cpy_size,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.cpy_size = memcpy.cpy_size + B;
                memcpy.stagecpy_size = memcpy.stagecpy_size + B;
            end;
        else
            assert B <= memcpy.stagecpy_size;
            memcpy.cpy_size = memcpy.cpy_size - B;
            memcpy.stagecpy_size = memcpy.stagecpy_size - B;
        end;
    end;
end;

```

```

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
            memcpy.toaddress + memcpy.cpy_size,
            memcpy.fromaddress + memcpy.cpy_size,
            memcpy.forward, B, raccdesc,
            waccdesc);

    if copied != B then
        fault = TRUE;
        memcpy.cpy_size = memcpy.cpy_size + B;
        memcpy.stagecpy_size = memcpy.stagecpy_size + B;
    end;
end;
end;

else
    while memcpy.stagecpy_size > 0 && !fault looplimit ArchMaxMOPSCPYSIZE do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSIZECHOICE(memcpy);
        assert B <= memcpy.stagecpy_size;

        if memcpy.forward then
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                memcpy.fromaddress,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress + B;
                memcpy.toaddress = memcpy.toaddress + B;
            end;
        else
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress - B,
                memcpy.fromaddress - B,
                memcpy.forward, B,
                raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress - B;
                memcpy.toaddress = memcpy.toaddress - B;
            end;
        end;

        if !fault then
            memcpy.cpy_size = memcpy.cpy_size - B;
            memcpy.stagecpy_size = memcpy.stagecpy_size - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;

```

2025-12-10 16:41:29 2025-09-16 16:41:29 2025-12-10 16:41:29 2025-11-28 16:41:29 2025-12-10 16:41:29

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)

htmldiff from-

(new)

MLBI

MPAM lookaside buffer invalidate

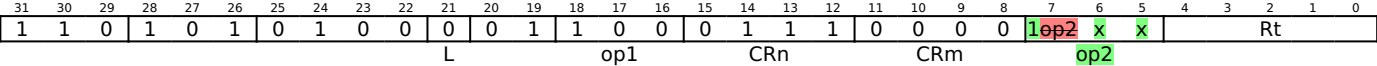
This instruction causes invalidation of cached copies of MPAM virtual-to-physical identifier translations.

This is an alias of [SYS](#). This means:

- The encodings in this description are named to match the encodings of [SYS](#).
- The description of [SYS](#) gives the operational pseudocode, any `CONSTRAINED UNPREDICTABLE` behavior, and any operational information for this instruction.

System

(FEAT_MPAMv2_VID)



Encoding

MLBI `<mlbi_op>{, <Xt>}`

is equivalent to

[SYS](#) #4, C7, C0, #<op2>{, <Xt>}

and is always the preferred disassembly.

and is the preferred disassembly when SysOp('100', '0111', '0000', op2) == Sys_MLBI.

Assembler Symbols

<mlbi_op> Is an MLBI operation name, as listed for the MLBI system instruction group, encoded in “op2[1:0]”:

op2[1:0]	<mlbi_op>
00100	ALLE1
01101	VMALLE1
10110	VPIDE1
11111	VPMGE1

<Xt> Is the 64-bit name of the optional general-purpose source register, defaulting to '11111', encoded in the "Rt" field.

Operation

The description of [SYS](#) gives the operational pseudocode for this instruction.

2025-12-10 16:41:29.4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

RCWSET, RCWSETA, RCWSETAL, RCWSETL

Read check write atomic bit set on doubleword in memory

This instruction atomically loads a 64-bit doubleword from memory, performs a bitwise OR with the **complement of the** value held in a register on it, and conditionally stores the result back to memory. Storing of the result back to memory is conditional on RCW Checks. The value initially loaded from memory is returned in the destination register. This instruction updates the condition flags based on the result of the update of memory.

- If the destination register is not XZR, RCWSETA and RCWSETAL load from memory with acquire semantics.
- RCWSETL and RCWSETAL store to memory with release semantics.
- RCWSET has neither acquire nor release semantics.

Note

This instruction is for performing atomic updates of translation table entries and not for general use.

Integer

(FEAT_THE)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	1	1	0	0	0	A	R	1	Rs					1	0	1	1	0	0	Rn					Rt				
S		VR				o3										opc															

Encoding for the RCWSET variant

Applies when (A == 0 && R == 0)

RCWSET <Xs>, <Xt>, [<Xn|SP>]

Encoding for the RCWSETA variant

Applies when (A == 1 && R == 0)

RCWSETA <Xs>, <Xt>, [<Xn|SP>]

Encoding for the RCWSETAL variant

Applies when (A == 1 && R == 1)

RCWSETAL <Xs>, <Xt>, [<Xn|SP>]

Encoding for the RCWSETL variant

Applies when (A == 0 && R == 1)

RCWSETL <Xs>, <Xt>, [<Xn|SP>]

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_THE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let soft : boolean = FALSE;

let acquire : boolean = A == '1' && t != 31;
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Xs>	Is the 64-bit name of the general-purpose register to be stored, encoded in the "Rs" field.
<Xt>	Is the 64-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
if IsD128Enabled(PSTATE.EL) then Undefined(); end;

var address : bits(64);
let newdata : bits(64) = X{}(s);
var readdata : bits(64);
var nzcvcv : bits(4);

let accdesc : AccessDescriptor = CreateAccDescRCW(MemAtomicOp\_ORR, soft, acquire, release,
tagchecked, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let compdata : bits(64) = ARBITRARY : bits(64); // Irrelevant when not executing CAS
(nzcvcv, readdata) = MemAtomicRCW{64}(address, compdata, newdata, accdesc);

PSTATE.[N,Z,C,V] = nzcvcv;
X{64}(t) = readdata; // Return the old value when t!=31
```

2025-12-10 rel2025-09-diff-tag 2025-12-102025-11-28 16:41:29:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)	htmldiff from-	(new)
-------	----------------	-------

RCWSSET, RCWSSETA, RCWSSETAL, RCWSSETL

Read check write software atomic bit set on doubleword in memory

This instruction atomically loads a 64-bit doubleword from memory, performs a bitwise OR with the complement of the value held in a register on it, and conditionally stores the result back to memory. Storing of the result back to memory is conditional on RCW Checks and RCWS Checks. The value initially loaded from memory is returned in the destination register. This instruction updates the condition flags based on the result of the update of memory.

- If the destination register is not XZR, RCWSSETA and RCWSSETAL load from memory with acquire semantics.
- RCWSSETL and RCWSSETAL store to memory with release semantics.
- RCWSSET has neither acquire nor release semantics.

Note

This instruction is for performing atomic updates of translation table entries and not for general use.

Integer

(FEAT_TTHE)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	1	1	0	0	0	A	R	1	Rs				1	0	1	1	0	0	Rn				Rt						
S				VR								o3				opc															

Encoding for the RCWSSET variant

Applies when (A == 0 && R == 0)

RCWSSET <Xs>, <Xt>, [<Xn|SP>]

Encoding for the RCWSSETA variant

Applies when (A == 1 && R == 0)

RCWSSETA <Xs>, <Xt>, [<Xn|SP>]

Encoding for the RCWSSETAL variant

Applies when (A == 1 && R == 1)

RCWSSETAL <Xs>, <Xt>, [<Xn|SP>]

Encoding for the RCWSSETL variant

Applies when (A == 0 && R == 1)

RCWSSETL <Xs>, <Xt>, [<Xn|SP>]

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_TTHE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let soft : boolean = TRUE;

let acquire : boolean = A == '1' && t != 31;
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Xs>	Is the 64-bit name of the general-purpose register to be stored, encoded in the "Rs" field.
<Xt>	Is the 64-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
if IsD128Enabled(PSTATE.EL) then Undefined(); end;

var address : bits(64);
let newdata : bits(64) = X{}(s);
var readdata : bits(64);
var nzcvcv : bits(4);

let accdesc : AccessDescriptor = CreateAccDescRCW(MemAtomicOp_ORR, soft, acquire, release,
tagchecked, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let compdata : bits(64) = ARBITRARY : bits(64); // Irrelevant when not executing CAS
(nzcvcv, readdata) = MemAtomicRCW{64}(address, compdata, newdata, accdesc);

PSTATE.[N,Z,C,V] = nzcvcv;
X{64}(t) = readdata; // Return the old value when t!=31
```

2025-12-10 rel2025-09-diff-tag 2025-12-102025-11-28 16:41:29:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)	htmldiff from-	(new)
-------	----------------	-------

SETGP, SETGM, SETGE

Memory set with tag setting

These instructions set a requested number of bytes in memory to the value in the least significant byte of the source data register and store an Allocation Tag to memory for each Tag Granule written. The Allocation Tag is calculated from the Logical Address Tag in the register that holds the first address to be set. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: SETGP, then SETGM, and then SETGE.

SETGP performs some preconditioning of the arguments suitable for using the SETGM instruction, and sets an IMPLEMENTATION DEFINED portion of the requested number of bytes. SETGM sets a further IMPLEMENTATION DEFINED portion of the remaining bytes. SETGE sets any final remaining bytes.

Note

The ability to set an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being set are divided between the different instructions.

For more information on exceptions specific to memory set instructions, see [Memory Copy and Memory Set exceptions](#).

The architecture supports two algorithms for the memory set: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For SETGP:

- If $Xn[63] == '1'$, the set size is saturated to $0x7FFFFFFFFFFFFFFF0$.

On completion of SETGP, option A:

- Xn holds -1 times the number of bytes in the saturated set size remaining to be set.
- Xd holds the original Xd + saturated set size.
- $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,0,0\}$.

On completion of SETGP, option B:

- Xn holds the number of bytes in the saturated set size remaining to be set.
- Xd holds the lowest address that has not been set.
- $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,1,0\}$.

For SETGM, option A, when $PSTATE.C = '0'$:

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be set.
- Xd holds the lowest address to be set - Xn .
- On completion of the instruction, Xn holds -1 times the number of bytes remaining to be set.

For SETGM, option B, when $PSTATE.C = '1'$:

- Xn holds the number of bytes remaining to be set.
- Xd holds the lowest address to be set.
- On completion of the instruction:
 - Xn holds the number of bytes remaining to be set.
 - Xd holds the lowest address that has not been set.

For SETGE, option A, when $PSTATE.C = '0'$:

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be set.
- Xd holds the lowest address to be set - Xn .
- On completion of the instruction, Xn holds 0.

For SETGE, option B, when $PSTATE.C = '1'$:

- Xn holds the number of bytes remaining to be set.
- Xd holds the lowest address to be set.
- On completion of the instruction:
 - Xn holds 0.
 - Xd holds the lowest address that has not been set.

Integer

(FEAT_MOPS && FEAT_MTE)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
sz	0	1	1	1	0	1	1	1	0	Rs						x	x	0	0	0	1	Rn						Rd					
o0				op1				op2																									

Encoding for the Prologue variant

Applies when (op2 == 0000)

SETGP [<Xd>]!, <Xn>!, <Xs>

Encoding for the Main variant

Applies when (op2 == 0100)

SETGM [<Xd>]!, <Xn>!, <Xs>

Encoding for the Epilogue variant

Applies when (op2 == 1000)

SETGE [<Xd>]!, <Xn>!, <Xs>

Decode for all variants of this encoding

```

if !IsFeatureImplemented(FEAT_MOPS) || !IsFeatureImplemented(FEAT_MTE) || sz != '00' then
    EndOfDecode(Decode_UNDEF);
end;

var memset : SETParams;
memset.d = UInt(Rd);
memset.s = UInt(Rs);
memset.n = UInt(Rn);
let options : bits(2) = op2[1:0];
let nontemporal : boolean = options[1] == '1';

case op2[3:2] of
    when '00' => memset.stage = MOPSStage_Prologue;
    when '01' => memset.stage = MOPSStage_Main;
    when '10' => memset.stage = MOPSStage_Epilogue;
    otherwise => EndOfDecode(Decode_UNDEF);
end;

```

For information about the `CONSTRAINED UNPREDICTABLE` behavior of this instruction, see [Architectural Constraints on UNPREDICTABLE behaviors](#), and particularly [Memory Copy and Memory Set SET*](#) and [Crossing a page boundary with different memory types or Shareability attributes](#).

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the destination address (an integer multiple of 16) and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address (an integer multiple of 16) and for option B is updated by the instruction, encoded in the "Rd" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be set (an integer multiple of 16) and is updated by the instruction, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be set (an integer multiple of 16) and is updated by the instruction, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be set (an integer multiple of 16) and is set to zero on completion of the instruction, encoded in the "Rn" field.
<Xs>	For the "Main" and "Prologue" variants: is the 64-bit name of the general-purpose register that holds the source data in bits<7:0>, encoded in the "Rs" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds the source data, encoded in the "Rs" field.


```

CheckMOPSEnabled();

CheckSETConstrainedUnpredictable(memset.memset.n, memset.d, memset.s);

let data : bits(8) = X{}(memset.s);
var B : MOPSEBlocksize = 0;

memset.is_setg = TRUE;
memset.nzcv = PSTATE.[N,Z,C,V];
memset.toaddress = PSTATE.[N,Z,C,V];
memset.toaddress = X{64}(memset.d);
if memset.stage == MOPSEStage_Prologue then
    memset.setsize = UInt(X{64}(memset.n));
else
    memset.setsize = SInt(X{64}(memset.n));
end;
memset.implements_option_a = SETGOptionA();

let privileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
    else PSTATE.EL()
    else PSTATE.EL != EL0);

let accdesc : AccessDescriptor = CreateAccDescSTGMOPS(privileged, nontemporal);

if memset.stage == MOPSEStage_Prologue then
    if memset.setsize > ArchMaxMOPSESETGSize then
        memset.setsize = ArchMaxMOPSESETGSize;
    end;

    if ((memset.setsize != 0 && !IsAlignedSize(memset.toaddress, TAG_GRANULE)) ||
        !IsAlignedSize(memset.setsize[63:0], TAG_GRANULE)) then
        let fault : FaultRecord = AlignmentFault(accdesc, memset.toaddress);
        AArch64_Abort(fault);
    end;

    if memset.implements_option_a then
        memset.nzcv = '0000';
        memset.toaddress = memset.toaddress + memset.setsize;
        memset.setsize = 0 - memset.setsize;
    else
        memset.nzcv = '0010';
    end;
end;

memset.stagesetsize = MemSetStageSize(memset);

if memset.stage != MOPSEStage_Prologue then
    CheckMemSetParams(memset, options);

    var fault_address : bits(64);
    if memset.implements_option_a then
        fault_address = memset.toaddress + memset.setsize;
    else
        fault_address = memset.toaddress;
    end;

    if (memset.setsize != 0 && (memset.stagesetsize != 0 || MemStageSetZeroSizeCheck()) &&
        !IsAlignedSize(memset.toaddress, TAG_GRANULE)) then
        let fault : FaultRecord = AlignmentFault(accdesc, fault_address);
        AArch64_Abort(fault);
    end;
    if ((memset.stagesetsize != 0 || MemStageSetZeroSizeCheck()) &&
        !IsAlignedSize(memset.setsize[63:0], TAG_GRANULE)) then
        let fault : FaultRecord = AlignmentFault(accdesc, fault_address);
        AArch64_Abort(fault);
    end;
end;

var tags_set : integer;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var tagmemaddrdesc : AddressDescriptor;
var tagmemstatus : PhysMemRetStatus;
var fault : boolean = FALSE;
var tagfault : boolean = FALSE;

if memset.implements_option_a then
    while memset.stagesetsize < 0 && !fault && !tagfault looplimit ArchMaxMOPSESETGSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = SETSizeChoice(memset, TAG_GRANULE);
        assert B <= -1 * memset.stagesetsize && B[3:0] == '0000';

        (-, memaddrdesc, memstatus) = MemSetBytes(memset.toaddress + memset.setsize,
            data, B, accdesc);

        let tag : bits(4) = AArch64_AllocationTagFromAddress(memset.toaddress + memset.setsize);
        (tags_set, tagmemaddrdesc, tagmemstatus) = MemSetTags(memset.toaddress + memset.setsize,
            tag, B, accdesc);
    end;
end;

```

```

    fault      = IsFault(memaddrdesc)    || IsFault(memstatus);
    tagfault   = IsFault(tagmemaddrdesc) || IsFault(tagmemstatus);

    if !fault && !tagfault then
        memset.setsize      = memset.setsize      + B;
        memset.stagesetsize = memset.stagesetsize + B;
    end;
end;

else
    while memset.stagesetsize > 0 && !fault && !tagfault looplimit ArchMaxMOPSETGSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = SETSizeChoice(memset, TAG GRANULE);
        assert B <= memset.stagesetsize && B[3:0] == '0000';

        (-, memaddrdesc, memstatus) = MemSetBytes(memset.toaddress, data, B, accdesc);
        let tag : bits(4) = AArch64\_AllocationTagFromAddress(memset.toaddress);
        (tags_set, tagmemaddrdesc, tagmemstatus) = MemSetTags(memset.toaddress, tag, B, accdesc);

        fault      = IsFault(memaddrdesc)    || IsFault(memstatus);
        tagfault   = IsFault(tagmemaddrdesc) || IsFault(tagmemstatus);

        if !fault && !tagfault then
            memset.toaddress      = memset.toaddress      + B;
            memset.setsize        = memset.setsize        - B;
            memset.stagesetsize   = memset.stagesetsize   - B;
        end;
    end;
end;

UpdateSetRegisters(memset, fault || tagfault, tags_set * TAG GRANULE);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let iswrite : boolean = TRUE;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
elseif tagfault then
    if IsFault(tagmemaddrdesc) then
        AArch64_Abort(tagmemaddrdesc.fault);
    else
        let iswrite : boolean = TRUE;
        HandleExternalAbort(tagmemstatus, iswrite, tagmemaddrdesc, 1, accdesc);
    end;
end;

if memset.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memset.nzcv;
end;

```

2025-12-10 16:41:29:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)

htmldiff from-

(new)

SETGPN, SETGMN, SETGEN

Memory set with tag setting, non-temporal

These instructions set a requested number of bytes in memory to the value in the least significant byte of the source data register and store an Allocation Tag to memory for each Tag Granule written. The Allocation Tag is calculated from the Logical Address Tag in the register that holds the first address to be set. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: SETGPN, then SETGMN, and then SETGEN.

SETGPN performs some preconditioning of the arguments suitable for using the SETGMN instruction, and sets an IMPLEMENTATION DEFINED portion of the requested number of bytes. SETGMN sets a further IMPLEMENTATION DEFINED portion of the remaining bytes. SETGEN sets any final remaining bytes.

Note

The ability to set an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being set are divided between the different instructions.

For more information on exceptions specific to memory set instructions, see [Memory Copy and Memory Set exceptions](#).

The architecture supports two algorithms for the memory set: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For SETGPN:

- If $Xn[63] == '1'$, the set size is saturated to $0x7FFFFFFFFFFFFFFF0$.

On completion of SETGPN, option A:

- Xn holds -1 times the number of bytes in the saturated set size remaining to be set.
- Xd holds the original Xd + saturated set size.
- $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,0,0\}$.

On completion of SETGPN, option B:

- Xn holds the number of bytes in the saturated set size remaining to be set.
- Xd holds the lowest address that has not been set.
- $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,1,0\}$.

For SETGMN, option A, when $PSTATE.C = '0'$:

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be set.
- Xd holds the lowest address to be set - Xn .
- On completion of the instruction, Xn holds -1 times the number of bytes remaining to be set.

For SETGMN, option B, when $PSTATE.C = '1'$:

- Xn holds the number of bytes remaining to be set.
- Xd holds the lowest address to be set.
- On completion of the instruction:
 - Xn holds the number of bytes remaining to be set.
 - Xd holds the lowest address that has not been set.

For SETGEN, option A, when $PSTATE.C = '0'$:

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be set.
- Xd holds the lowest address to be set - Xn .
- On completion of the instruction, Xn holds 0.

For SETGEN, option B, when $PSTATE.C = '1'$:

- Xn holds the number of bytes remaining to be set.
- Xd holds the lowest address to be set.
- On completion of the instruction:
 - Xn holds 0.
 - Xd holds the lowest address that has not been set.

Integer

(FEAT_MOPS && FEAT_MTE)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sz	0	1	1	1	0	1	1	1	1	0	Rs					x	x	1	0	0	1	Rn					Rd				
o0					op1					op2																					

Encoding for the Prologue variant

Applies when (op2 == 0010)

SETGPN [<Xd>]!, <Xn>!, <Xs>

Encoding for the Main variant

Applies when (op2 == 0110)

SETGMN [<Xd>]!, <Xn>!, <Xs>

Encoding for the Epilogue variant

Applies when (op2 == 1010)

SETGEN [<Xd>]!, <Xn>!, <Xs>

Decode for all variants of this encoding

```

if !IsFeatureImplemented(FEAT_MOPS) || !IsFeatureImplemented(FEAT_MTE) || sz != '00' then
    EndOfDecode(Decode_UNDEF);
end;

var memset : SETParams;
memset.d = UInt(Rd);
memset.s = UInt(Rs);
memset.n = UInt(Rn);
let options : bits(2) = op2[1:0];
let nontemporal : boolean = options[1] == '1';

case op2[3:2] of
    when '00' => memset.stage = MOPSStage_Prologue;
    when '01' => memset.stage = MOPSStage_Main;
    when '10' => memset.stage = MOPSStage_Epilogue;
    otherwise => EndOfDecode(Decode_UNDEF);
end;

```

For information about the `CONSTRAINED UNPREDICTABLE` behavior of this instruction, see [Architectural Constraints on UNPREDICTABLE behaviors](#), and particularly [Memory Copy and Memory Set SET*](#) and [Crossing a page boundary with different memory types or Shareability attributes](#).

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the destination address (an integer multiple of 16) and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address (an integer multiple of 16) and for option B is updated by the instruction, encoded in the "Rd" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be set (an integer multiple of 16) and is updated by the instruction, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be set (an integer multiple of 16) and is updated by the instruction, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be set (an integer multiple of 16) and is set to zero on completion of the instruction, encoded in the "Rn" field.
<Xs>	For the "Main" and "Prologue" variants: is the 64-bit name of the general-purpose register that holds the source data in bits<7:0>, encoded in the "Rs" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds the source data, encoded in the "Rs" field.


```

CheckMOPSEnabled();

CheckSETConstrainedUnpredictable(memset.memset.n, memset.d, memset.s);

let data : bits(8) = X{}(memset.s);
var B : MOPSEBlocksize = 0;

memset.is_setg = TRUE;
memset.nzcv = PSTATE.[N,Z,C,V];
memset.toaddress = PSTATE.[N,Z,C,V];
memset.toaddress = X{64}(memset.d);
if memset.stage == MOPSEStage_Prologue then
    memset.setsize = UInt(X{64}(memset.n));
else
    memset.setsize = SInt(X{64}(memset.n));
end;
memset.implements_option_a = SETGOptionA();

let privileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
    else PSTATE.EL()
    else PSTATE.EL != EL0);

let accdesc : AccessDescriptor = CreateAccDescSTGMOPS(privileged, nontemporal);

if memset.stage == MOPSEStage_Prologue then
    if memset.setsize > ArchMaxMOPSESETGSize then
        memset.setsize = ArchMaxMOPSESETGSize;
    end;

    if ((memset.setsize != 0 && !IsAlignedSize(memset.toaddress, TAG_GRANULE)) ||
        !IsAlignedSize(memset.setsize[63:0], TAG_GRANULE)) then
        let fault : FaultRecord = AlignmentFault(accdesc, memset.toaddress);
        AArch64_Abort(fault);
    end;

    if memset.implements_option_a then
        memset.nzcv = '0000';
        memset.toaddress = memset.toaddress + memset.setsize;
        memset.setsize = 0 - memset.setsize;
    else
        memset.nzcv = '0010';
    end;
end;

memset.stagesetsize = MemSetStageSize(memset);

if memset.stage != MOPSEStage_Prologue then
    CheckMemSetParams(memset, options);

    var fault_address : bits(64);
    if memset.implements_option_a then
        fault_address = memset.toaddress + memset.setsize;
    else
        fault_address = memset.toaddress;
    end;

    if (memset.setsize != 0 && (memset.stagesetsize != 0 || MemStageSetZeroSizeCheck()) &&
        !IsAlignedSize(memset.toaddress, TAG_GRANULE)) then
        let fault : FaultRecord = AlignmentFault(accdesc, fault_address);
        AArch64_Abort(fault);
    end;
    if ((memset.stagesetsize != 0 || MemStageSetZeroSizeCheck()) &&
        !IsAlignedSize(memset.setsize[63:0], TAG_GRANULE)) then
        let fault : FaultRecord = AlignmentFault(accdesc, fault_address);
        AArch64_Abort(fault);
    end;
end;

var tags_set : integer;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var tagmemaddrdesc : AddressDescriptor;
var tagmemstatus : PhysMemRetStatus;
var fault : boolean = FALSE;
var tagfault : boolean = FALSE;

if memset.implements_option_a then
    while memset.stagesetsize < 0 && !fault && !tagfault looplimit ArchMaxMOPSESETGSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = SETSizeChoice(memset, TAG_GRANULE);
        assert B <= -1 * memset.stagesetsize && B[3:0] == '0000';

        (-, memaddrdesc, memstatus) = MemSetBytes(memset.toaddress + memset.setsize,
            data, B, accdesc);

        let tag : bits(4) = AArch64_AllocationTagFromAddress(memset.toaddress + memset.setsize);
        (tags_set, tagmemaddrdesc, tagmemstatus) = MemSetTags(memset.toaddress + memset.setsize,
            tag, B, accdesc);
    end;
end;

```

```

    fault    = IsFault(memaddrdesc)    || IsFault(memstatus);
    tagfault = IsFault(tagmemaddrdesc) || IsFault(tagmemstatus);

    if !fault && !tagfault then
        memset.setsize      = memset.setsize      + B;
        memset.stagesetsize = memset.stagesetsize + B;
    end;
end;

else
    while memset.stagesetsize > 0 && !fault && !tagfault looplimit ArchMaxMOPSETGSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = SETSizeChoice(memset, TAG GRANULE);
        assert B <= memset.stagesetsize && B[3:0] == '0000';

        (-, memaddrdesc, memstatus) = MemSetBytes(memset.toaddress, data, B, accdesc);
        let tag : bits(4) = AArch64\_AllocationTagFromAddress(memset.toaddress);
        (tags_set, tagmemaddrdesc, tagmemstatus) = MemSetTags(memset.toaddress, tag, B, accdesc);

        fault    = IsFault(memaddrdesc)    || IsFault(memstatus);
        tagfault = IsFault(tagmemaddrdesc) || IsFault(tagmemstatus);

        if !fault && !tagfault then
            memset.toaddress      = memset.toaddress      + B;
            memset.setsize        = memset.setsize        - B;
            memset.stagesetsize   = memset.stagesetsize   - B;
        end;
    end;
end;

UpdateSetRegisters(memset, fault || tagfault, tags_set * TAG GRANULE);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let iswrite : boolean = TRUE;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
elseif tagfault then
    if IsFault(tagmemaddrdesc) then
        AArch64_Abort(tagmemaddrdesc.fault);
    else
        let iswrite : boolean = TRUE;
        HandleExternalAbort(tagmemstatus, iswrite, tagmemaddrdesc, 1, accdesc);
    end;
end;

if memset.stage == MOPSTage_Prologue then
    PSTATE.[N,Z,C,V] = memset.nzcv;
end;

```

2025-12-10 2025-09-10 diff tag 2025-12-10 2025-11-28 16:41:29:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)

htmldiff from-

(new)

SETGPT, SETGMT, SETGET

Memory set with tag setting, unprivileged

These instructions set a requested number of bytes in memory to the value in the least significant byte of the source data register and store an Allocation Tag to memory for each Tag Granule written. The Allocation Tag is calculated from the Logical Address Tag in the register that holds the first address to be set. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: SETGPT, then SETGMT, and then SETGET.

SETGPT performs some preconditioning of the arguments suitable for using the SETGMT instruction, and sets an IMPLEMENTATION DEFINED portion of the requested number of bytes. SETGMT sets a further IMPLEMENTATION DEFINED portion of the remaining bytes. SETGET sets any final remaining bytes.

Note

The ability to set an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being set are divided between the different instructions.

For more information on exceptions specific to memory set instructions, see [Memory Copy and Memory Set exceptions](#).

The architecture supports two algorithms for the memory set: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For SETGPT:

- If $Xn[63] == '1'$, the set size is saturated to $0x7FFFFFFFFFFFFFFF0$.

On completion of SETGPT, option A:

- Xn holds -1 times the number of bytes in the saturated set size remaining to be set.
- Xd holds the original Xd + saturated set size.
- $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,0,0\}$.

On completion of SETGPT, option B:

- Xn holds the number of bytes in the saturated set size remaining to be set.
- Xd holds the lowest address that has not been set.
- $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,1,0\}$.

For SETGMT, option A, when $PSTATE.C = '0'$:

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be set.
- Xd holds the lowest address to be set - Xn .
- On completion of the instruction, Xn holds -1 times the number of bytes remaining to be set.

For SETGMT, option B, when $PSTATE.C = '1'$:

- Xn holds the number of bytes remaining to be set.
- Xd holds the lowest address to be set.
- On completion of the instruction:
 - Xn holds the number of bytes remaining to be set.
 - Xd holds the lowest address that has not been set.

For SETGET, option A, when $PSTATE.C = '0'$:

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be set.
- Xd holds the lowest address to be set - Xn .
- On completion of the instruction, Xn holds 0.

For SETGET, option B, when $PSTATE.C = '1'$:

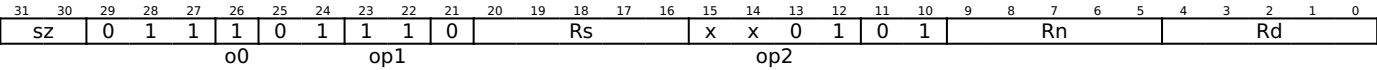
- Xn holds the number of bytes remaining to be set.
- Xd holds the lowest address to be set.
- On completion of the instruction:
 - Xn holds 0.
 - Xd holds the lowest address that has not been set.

Explicit Memory Write effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of HCR_EL2().{E2H, TGE} is '11'.

Otherwise, the Explicit Memory Write effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer
(FEAT_MOPS && FEAT_MTE)



Encoding for the Prologue variant

Applies when (op2 == 0001)

```
SETGPT [<Xd>]!, <Xn>!, <Xs>
```

Encoding for the Main variant

Applies when (op2 == 0101)

```
SETGMT [<Xd>]!, <Xn>!, <Xs>
```

Encoding for the Epilogue variant

Applies when (op2 == 1001)

```
SETGET [<Xd>]!, <Xn>!, <Xs>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || !IsFeatureImplemented(FEAT_MTE) || sz != '00' then
  EndOfDecode(Decode_UNDEF);
end;

var memset : SETParams;
memset.d = UInt(Rd);
memset.s = UInt(Rs);
memset.n = UInt(Rn);
let options : bits(2) = op2[1:0];
let nontemporal : boolean = options[1] == '1';

case op2[3:2] of
  when '00' => memset.stage = MOPSStage_Prologue;
  when '01' => memset.stage = MOPSStage_Main;
  when '10' => memset.stage = MOPSStage_Epilogue;
  otherwise => EndOfDecode(Decode_UNDEF);
end;
```

For information about the *CONSTRAINED UNPREDICTABLE* behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set SET** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

- <Xd> For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the destination address (an integer multiple of 16) and is updated by the instruction, encoded in the "Rd" field.
For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address (an integer multiple of 16) and for option B is updated by the instruction, encoded in the "Rd" field.
- <Xn> For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be set (an integer multiple of 16) and is updated by the instruction, encoded in the "Rn" field.
For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be set (an integer multiple of 16) and is updated by the instruction, encoded in the "Rn" field.
For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be set (an integer multiple of 16) and is set to zero on completion of the instruction, encoded in the "Rn" field.
- <Xs> For the "Main" and "Prologue" variants: is the 64-bit name of the general-purpose register that holds the source data in bits<7:0>, encoded in the "Rs" field.
For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds the source data, encoded in the "Rs" field.


```

CheckMOPSEnabled();

CheckSETConstrainedUnpredictable(memset.memset.n, memset.d, memset.s);

let data : bits(8) = X{}(memset.s);
var B : MOPSEBlocksize = 0;

memset.is_setg = TRUE;
memset.nzcv = PSTATE.[N,Z,C,V];
memset.toaddress = PSTATE.[N,Z,C,V];
memset.toaddress = X{64}(memset.d);
if memset.stage == MOPSEStage_Prologue then
    memset.setsize = UInt(X{64}(memset.n));
else
    memset.setsize = SInt(X{64}(memset.n));
end;
memset.implements_option_a = SETGOptionA();

let privileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
    else PSTATE.EL()
    else PSTATE.EL != EL0);

let accdesc : AccessDescriptor = CreateAccDescSTGMOPS(privileged, nontemporal);

if memset.stage == MOPSEStage_Prologue then
    if memset.setsize > ArchMaxMOPSESETGSize then
        memset.setsize = ArchMaxMOPSESETGSize;
    end;

    if ((memset.setsize != 0 && !IsAlignedSize(memset.toaddress, TAG_GRANULE)) ||
        !IsAlignedSize(memset.setsize[63:0], TAG_GRANULE)) then
        let fault : FaultRecord = AlignmentFault(accdesc, memset.toaddress);
        AArch64_Abort(fault);
    end;

    if memset.implements_option_a then
        memset.nzcv = '0000';
        memset.toaddress = memset.toaddress + memset.setsize;
        memset.setsize = 0 - memset.setsize;
    else
        memset.nzcv = '0010';
    end;
end;

memset.stagesetsize = MemSetStageSize(memset);

if memset.stage != MOPSEStage_Prologue then
    CheckMemSetParams(memset, options);

    var fault_address : bits(64);
    if memset.implements_option_a then
        fault_address = memset.toaddress + memset.setsize;
    else
        fault_address = memset.toaddress;
    end;

    if (memset.setsize != 0 && (memset.stagesetsize != 0 || MemStageSetZeroSizeCheck()) &&
        !IsAlignedSize(memset.toaddress, TAG_GRANULE)) then
        let fault : FaultRecord = AlignmentFault(accdesc, fault_address);
        AArch64_Abort(fault);
    end;

    if ((memset.stagesetsize != 0 || MemStageSetZeroSizeCheck()) &&
        !IsAlignedSize(memset.setsize[63:0], TAG_GRANULE)) then
        let fault : FaultRecord = AlignmentFault(accdesc, fault_address);
        AArch64_Abort(fault);
    end;
end;

var tags_set : integer;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var tagmemaddrdesc : AddressDescriptor;
var tagmemstatus : PhysMemRetStatus;
var fault : boolean = FALSE;
var tagfault : boolean = FALSE;

if memset.implements_option_a then
    while memset.stagesetsize < 0 && !fault && !tagfault looplimit ArchMaxMOPSESETGSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = SETSizeChoice(memset, TAG_GRANULE);
        assert B <= -1 * memset.stagesetsize && B[3:0] == '0000';

        (-, memaddrdesc, memstatus) = MemSetBytes(memset.toaddress + memset.setsize,
            data, B, accdesc);

        let tag : bits(4) = AArch64_AllocationTagFromAddress(memset.toaddress + memset.setsize);
        (tags_set, tagmemaddrdesc, tagmemstatus) = MemSetTags(memset.toaddress + memset.setsize,
            tag, B, accdesc);
    end;
end;

```

```

    fault    = IsFault(memaddrdesc)    || IsFault(memstatus);
    tagfault = IsFault(tagmemaddrdesc) || IsFault(tagmemstatus);

    if !fault && !tagfault then
        memset.setsize      = memset.setsize      + B;
        memset.stagesetsize = memset.stagesetsize + B;
    end;
end;

else
    while memset.stagesetsize > 0 && !fault && !tagfault looplimit ArchMaxMOPSETGSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = SETSizeChoice(memset, TAG GRANULE);
        assert B <= memset.stagesetsize && B[3:0] == '0000';

        (-, memaddrdesc, memstatus) = MemSetBytes(memset.toaddress, data, B, accdesc);
        let tag : bits(4) = AArch64\_AllocationTagFromAddress(memset.toaddress);
        (tags_set, tagmemaddrdesc, tagmemstatus) = MemSetTags(memset.toaddress, tag, B, accdesc);

        fault    = IsFault(memaddrdesc)    || IsFault(memstatus);
        tagfault = IsFault(tagmemaddrdesc) || IsFault(tagmemstatus);

        if !fault && !tagfault then
            memset.toaddress      = memset.toaddress      + B;
            memset.setsize        = memset.setsize        - B;
            memset.stagesetsize   = memset.stagesetsize   - B;
        end;
    end;
end;

UpdateSetRegisters(memset, fault || tagfault, tags_set * TAG GRANULE);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let iswrite : boolean = TRUE;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
elseif tagfault then
    if IsFault(tagmemaddrdesc) then
        AArch64_Abort(tagmemaddrdesc.fault);
    else
        let iswrite : boolean = TRUE;
        HandleExternalAbort(tagmemstatus, iswrite, tagmemaddrdesc, 1, accdesc);
    end;
end;

if memset.stage == MOPSTage_Prologue then
    PSTATE.[N,Z,C,V] = memset.nzcv;
end;

```

2025-12-10 16:41:29:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)

htmldiff from-

(new)

SETGPTN, SETGMTN, SETGETN

Memory set with tag setting, unprivileged and non-temporal

These instructions set a requested number of bytes in memory to the value in the least significant byte of the source data register and store an Allocation Tag to memory for each Tag Granule written. The Allocation Tag is calculated from the Logical Address Tag in the register that holds the first address to be set. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: SETGPTN, then SETGMTN, and then SETGETN.

SETGPTN performs some preconditioning of the arguments suitable for using the SETGMTN instruction, and sets an IMPLEMENTATION DEFINED portion of the requested number of bytes. SETGMTN sets a further IMPLEMENTATION DEFINED portion of the remaining bytes. SETGETN sets any final remaining bytes.

Note

The ability to set an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being set are divided between the different instructions.

For more information on exceptions specific to memory set instructions, see [Memory Copy and Memory Set exceptions](#).

The architecture supports two algorithms for the memory set: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For SETGPTN:

- If $Xn[63] == '1'$, the set size is saturated to $0x7FFFFFFFFFFFFFFF0$.

On completion of SETGPTN, option A:

- Xn holds -1 times the number of bytes in the saturated set size remaining to be set.
- Xd holds the original Xd + saturated set size.
- $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,0,0\}$.

On completion of SETGPTN, option B:

- Xn holds the number of bytes in the saturated set size remaining to be set.
- Xd holds the lowest address that has not been set.
- $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,1,0\}$.

For SETGMTN, option A, when $PSTATE.C = '0'$:

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be set.
- Xd holds the lowest address to be set - Xn .
- On completion of the instruction, Xn holds -1 times the number of bytes remaining to be set.

For SETGMTN, option B, when $PSTATE.C = '1'$:

- Xn holds the number of bytes remaining to be set.
- Xd holds the lowest address to be set.
- On completion of the instruction:
 - Xn holds the number of bytes remaining to be set.
 - Xd holds the lowest address that has not been set.

For SETGETN, option A, when $PSTATE.C = '0'$:

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be set.
- Xd holds the lowest address to be set - Xn .
- On completion of the instruction, Xn holds 0.

For SETGETN, option B, when $PSTATE.C = '1'$:

- Xn holds the number of bytes remaining to be set.
- Xd holds the lowest address to be set.
- On completion of the instruction:
 - Xn holds 0.
 - Xd holds the lowest address that has not been set.

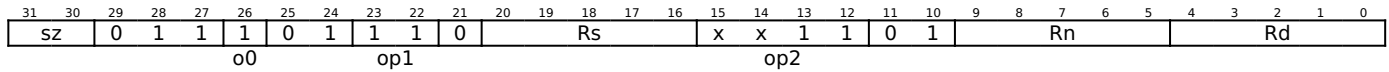
Explicit Memory Write effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of HCR_EL2().{E2H, TGE} is '11'.

Otherwise, the Explicit Memory Write effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer

(FEAT_MOPS && FEAT_MTE)



Encoding for the Prologue variant

Applies when (op2 == 0011)

SETGPTN [*<Xd>*]!, *<Xn>*!, *<Xs>*

Encoding for the Main variant

Applies when (op2 == 0111)

SETGMTN [*<Xd>*]!, *<Xn>*!, *<Xs>*

Encoding for the Epilogue variant

Applies when (op2 == 1011)

SETGETN [*<Xd>*]!, *<Xn>*!, *<Xs>*

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || !IsFeatureImplemented(FEAT_MTE) || sz != '00' then
    EndOfDecode(Decode_UNDEF);
end;

var memset : SETParams;
memset.d = UInt(Rd);
memset.s = UInt(Rs);
memset.n = UInt(Rn);
let options : bits(2) = op2[1:0];
let nontemporal : boolean = options[1] == '1';

case op2[3:2] of
    when '00' => memset.stage = MOPSStage_Prologue;
    when '01' => memset.stage = MOPSStage_Main;
    when '10' => memset.stage = MOPSStage_Epilogue;
    otherwise => EndOfDecode(Decode_UNDEF);
end;
```

For information about the *CONSTRAINED UNPREDICTABLE* behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set SET** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<i><Xd></i>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the destination address (an integer multiple of 16) and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address (an integer multiple of 16) and for option B is updated by the instruction, encoded in the "Rd" field.
<i><Xn></i>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be set (an integer multiple of 16) and is updated by the instruction, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be set (an integer multiple of 16) and is updated by the instruction, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be set (an integer multiple of 16) and is set to zero on completion of the instruction, encoded in the "Rn" field.
<i><Xs></i>	For the "Main" and "Prologue" variants: is the 64-bit name of the general-purpose register that holds the source data in bits<7:0>, encoded in the "Rs" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds the source data, encoded in the "Rs" field.


```

CheckMOPSEnabled();

CheckSETConstrainedUnpredictable(memset.memset.n, memset.d, memset.s);

let data : bits(8) = X{}(memset.s);
var B : MOPSEBlocksize = 0;

memset.is_setg = TRUE;
memset.nzcv = PSTATE.[N,Z,C,V];
memset.toaddress = PSTATE.[N,Z,C,V];
memset.toaddress = X{64}(memset.d);
if memset.stage == MOPSEStage_Prologue then
    memset.setsize = UInt(X{64}(memset.n));
else
    memset.setsize = SInt(X{64}(memset.n));
end;
memset.implements_option_a = SETGOptionA();

let privileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
    else PSTATE.EL()
    else PSTATE.EL != EL0);

let accdesc : AccessDescriptor = CreateAccDescSTGMOPS(privileged, nontemporal);

if memset.stage == MOPSEStage_Prologue then
    if memset.setsize > ArchMaxMOPSESETGSize then
        memset.setsize = ArchMaxMOPSESETGSize;
    end;

    if ((memset.setsize != 0 && !IsAlignedSize(memset.toaddress, TAG_GRANULE)) ||
        !IsAlignedSize(memset.setsize[63:0], TAG_GRANULE)) then
        let fault : FaultRecord = AlignmentFault(accdesc, memset.toaddress);
        AArch64_Abort(fault);
    end;

    if memset.implements_option_a then
        memset.nzcv = '0000';
        memset.toaddress = memset.toaddress + memset.setsize;
        memset.setsize = 0 - memset.setsize;
    else
        memset.nzcv = '0010';
    end;
end;

memset.stagesetsize = MemSetStageSize(memset);

if memset.stage != MOPSEStage_Prologue then
    CheckMemSetParams(memset, options);

    var fault_address : bits(64);
    if memset.implements_option_a then
        fault_address = memset.toaddress + memset.setsize;
    else
        fault_address = memset.toaddress;
    end;

    if (memset.setsize != 0 && (memset.stagesetsize != 0 || MemStageSetZeroSizeCheck()) &&
        !IsAlignedSize(memset.toaddress, TAG_GRANULE)) then
        let fault : FaultRecord = AlignmentFault(accdesc, fault_address);
        AArch64_Abort(fault);
    end;
    if ((memset.stagesetsize != 0 || MemStageSetZeroSizeCheck()) &&
        !IsAlignedSize(memset.setsize[63:0], TAG_GRANULE)) then
        let fault : FaultRecord = AlignmentFault(accdesc, fault_address);
        AArch64_Abort(fault);
    end;
end;

var tags_set : integer;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var tagmemaddrdesc : AddressDescriptor;
var tagmemstatus : PhysMemRetStatus;
var fault : boolean = FALSE;
var tagfault : boolean = FALSE;

if memset.implements_option_a then
    while memset.stagesetsize < 0 && !fault && !tagfault looplimit ArchMaxMOPSESETGSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = SETSizeChoice(memset, TAG_GRANULE);
        assert B <= -1 * memset.stagesetsize && B[3:0] == '0000';

        (-, memaddrdesc, memstatus) = MemSetBytes(memset.toaddress + memset.setsize,
            data, B, accdesc);

        let tag : bits(4) = AArch64_AllocationTagFromAddress(memset.toaddress + memset.setsize);
        (tags_set, tagmemaddrdesc, tagmemstatus) = MemSetTags(memset.toaddress + memset.setsize,
            tag, B, accdesc);
    end;
end;

```

```

    fault    = IsFault(memaddrdesc)    || IsFault(memstatus);
    tagfault = IsFault(tagmemaddrdesc) || IsFault(tagmemstatus);

    if !fault && !tagfault then
        memset.setsize      = memset.setsize      + B;
        memset.stagesetsize = memset.stagesetsize + B;
    end;
end;

else
    while memset.stagesetsize > 0 && !fault && !tagfault looplimit ArchMaxMOPSETGSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = SETSizeChoice(memset, TAG GRANULE);
        assert B <= memset.stagesetsize && B[3:0] == '0000';

        (-, memaddrdesc, memstatus) = MemSetBytes(memset.toaddress, data, B, accdesc);
        let tag : bits(4) = AArch64\_AllocationTagFromAddress(memset.toaddress);
        (tags_set, tagmemaddrdesc, tagmemstatus) = MemSetTags(memset.toaddress, tag, B, accdesc);

        fault    = IsFault(memaddrdesc)    || IsFault(memstatus);
        tagfault = IsFault(tagmemaddrdesc) || IsFault(tagmemstatus);

        if !fault && !tagfault then
            memset.toaddress      = memset.toaddress      + B;
            memset.setsize        = memset.setsize        - B;
            memset.stagesetsize   = memset.stagesetsize   - B;
        end;
    end;
end;

UpdateSetRegisters(memset, fault || tagfault, tags_set * TAG GRANULE);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let iswrite : boolean = TRUE;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
elseif tagfault then
    if IsFault(tagmemaddrdesc) then
        AArch64_Abort(tagmemaddrdesc.fault);
    else
        let iswrite : boolean = TRUE;
        HandleExternalAbort(tagmemstatus, iswrite, tagmemaddrdesc, 1, accdesc);
    end;
end;

if memset.stage == MOPSTage_Prologue then
    PSTATE.[N,Z,C,V] = memset.nzcv;
end;

```

2025-12-10 16:41:29:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)

htmldiff from-

(new)

SETP, SETM, SETE

Memory set

These instructions set a required number of bytes in memory to the value in the least significant byte of the source data register. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: SETP, then SETM, and then SETE.

SETP performs some preconditioning of the arguments suitable for using the SETM instruction, and sets an IMPLEMENTATION DEFINED portion of the requested number of bytes. SETM sets a further IMPLEMENTATION DEFINED portion of the remaining bytes. SETE sets any final remaining bytes.

Note

The ability to set an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being set are divided between the different instructions.

For more information on exceptions specific to memory set instructions, see [Memory Copy and Memory Set exceptions](#).

The architecture supports two algorithms for the memory set: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For SETP:

- If $Xn[63] = '1'$, the set size is saturated to $0x7FFFFFFFFFFFFFFF$.

On completion of SETP, option A:

- Xn holds -1 times the number of bytes in the saturated set size remaining to be set.
- Xd holds the original Xd + saturated set size.
- $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,0,0\}$.

On completion of SETP, option B:

- Xn holds the number of bytes in the saturated set size remaining to be set.
- Xd holds the lowest address that has not been set.
- $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,1,0\}$.

For SETM, option A, when $PSTATE.C = '0'$:

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be set.
- Xd holds the lowest address to be set - Xn .
- On completion of the instruction, Xn holds -1 times the number of bytes remaining to be set.

For SETM, option B, when $PSTATE.C = '1'$:

- Xn holds the number of bytes remaining to be set.
- Xd holds the lowest address to be set.
- On completion of the instruction:
 - Xn holds the number of bytes remaining to be set.
 - Xd holds the lowest address that has not been set.

For SETE, option A, when $PSTATE.C = '0'$:

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be set.
- Xd holds the lowest address to be set - Xn .
- On completion of the instruction, Xn holds 0.

For SETE, option B, when $PSTATE.C = '1'$:

- Xn holds the number of bytes remaining to be set.
- Xd holds the lowest address to be set.
- On completion of the instruction:
 - Xn holds 0.
 - Xd holds the lowest address that has not been set.

Integer

(FEAT_MOPS)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0															
sz		0		1		1		0		0		1		1		1		0		Rs					x		x		0		0		0		1		Rn					Rd				
o0										op1										op2																										

Encoding for the Prologue variant

Applies when (op2 == 0000)

```
SETP [<Xd>]!, <Xn>!, <Xs>
```

Encoding for the Main variant

Applies when (op2 == 0100)

```
SETM [<Xd>]!, <Xn>!, <Xs>
```

Encoding for the Epilogue variant

Applies when (op2 == 1000)

```
SETE [<Xd>]!, <Xn>!, <Xs>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memset : SETParams;
memset.d = UInt(Rd);
memset.s = UInt(Rs);
memset.n = UInt(Rn);
let options : bits(2) = op2[1:0];
let nontemporal : boolean = options[1] == '1';

case op2[3:2] of
  when '00' => memset.stage = MOPSStage_Prologue;
  when '01' => memset.stage = MOPSStage_Main;
  when '10' => memset.stage = MOPSStage_Epilogue;
  otherwise => EndOfDecode(Decode_UNDEF);
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see [Architectural Constraints on UNPREDICTABLE behaviors](#), and particularly [Memory Copy and Memory Set SET*](#) and [Crossing a page boundary with different memory types or Shareability attributes](#).

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address and for option B is updated by the instruction, encoded in the "Rd" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be set and is updated by the instruction, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be set and is updated by the instruction, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be set and is set to zero on completion of the instruction, encoded in the "Rn" field.
<Xs>	Is the 64-bit name of the general-purpose register that holds the source data, encoded in the "Rs" field.


```

CheckMOPSEnabled();

CheckSETConstrainedUnpredictable(memset.memset.n, memset.d, memset.s);

let data : bits(8) = X{}(memset.s);
var B : MOPSBlockSize = 0;

memset.is_setg = FALSE;
memset.nzcv = PSTATE.[N,Z,C,V];
memset.toaddress = PSTATE.[N,Z,C,V];
memset.toaddress = X{64}(memset.d);
if memset.stage == MOPSStage_Prologue then
    memset.setsize = UInt(X{64}(memset.n));
else
    memset.setsize = SInt(X{64}(memset.n));
end;
memset.implements_option_a = SETOptionA();

let privileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
    else PSTATE.EL()
    else PSTATE.EL != EL0);

let accdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, privileged, nontemporal);

if memset.stage == MOPSStage_Prologue then
    if memset.setsize > ArchMaxMOPSBlockSize then
        memset.setsize = ArchMaxMOPSBlockSize;
    end;

    if memset.implements_option_a then
        memset.nzcv = '0000';
        memset.toaddress = memset.toaddress + memset.setsize;
        memset.setsize = 0 - memset.setsize;
    else
        memset.nzcv = '0010';
    end;
end;

memset.stagesetsize = MemSetStageSize(memset);

if memset.stage != MOPSStage_Prologue then
    CheckMemSetParams(memset, options);
end;

var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var memory_set : integer;
var fault : boolean = FALSE;

if memset.implements_option_a then
    while memset.stagesetsize < 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = SETSizeChoice(memset, 1);
        assert B <= -1 * memset.stagesetsize;

        (memory_set, memaddrdesc, memstatus) = MemSetBytes(memset.toaddress + memset.setsize,
            data, B, accdesc);

        if memory_set != B then
            fault = TRUE;
        else
            memset.setsize = memset.setsize + B;
            memset.stagesetsize = memset.stagesetsize + B;
        end;
    end;
else
    while memset.stagesetsize > 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = SETSizeChoice(memset, 1);
        assert B <= memset.stagesetsize;

        (memory_set, memaddrdesc, memstatus) = MemSetBytes(memset.toaddress, data, B, accdesc);

        if memory_set != B then
            fault = TRUE;
        else
            memset.toaddress = memset.toaddress + B;
            memset.setsize = memset.setsize - B;
            memset.stagesetsize = memset.stagesetsize - B;
        end;
    end;
end;

UpdateSetRegisters(memset, fault, memory_set);

if fault then

```

```
if IsFault(memaddrdesc) then
  AArch64_Abort(memaddrdesc.fault);
else
  let iswrite : boolean = TRUE;
  HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
end;
end;

if memset.stage == MOPSTage_Prologue then
  PSTATE.[N,Z,C,V] = memset.nzcv;
end;
```

(old)	htmldiff from-	(new)
-------	----------------	-------

SETPN, SETMN, SETEN

Memory set, non-temporal

These instructions set a required number of bytes in memory to the value in the least significant byte of the source data register. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: SETPN, then SETMN, and then SETEN.

SETPN performs some preconditioning of the arguments suitable for using the SETMN instruction, and sets an IMPLEMENTATION DEFINED portion of the requested number of bytes. SETMN sets a further IMPLEMENTATION DEFINED portion of the remaining bytes. SETEN sets any final remaining bytes.

Note

The ability to set an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being set are divided between the different instructions.

For more information on exceptions specific to memory set instructions, see [Memory Copy and Memory Set exceptions](#).

The architecture supports two algorithms for the memory set: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For SETPN:

- If $Xn[63] = '1'$, the set size is saturated to $0x7FFFFFFFFFFFFFFF$.

On completion of SETPN, option A:

- Xn holds -1 times the number of bytes in the saturated set size remaining to be set.
- Xd holds the original Xd + saturated set size.
- $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,0,0\}$.

On completion of SETPN, option B:

- Xn holds the number of bytes in the saturated set size remaining to be set.
- Xd holds the lowest address that has not been set.
- $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,1,0\}$.

For SETMN, option A, when $PSTATE.C = '0'$:

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be set.
- Xd holds the lowest address to be set - Xn .
- On completion of the instruction, Xn holds -1 times the number of bytes remaining to be set.

For SETMN, option B, when $PSTATE.C = '1'$:

- Xn holds the number of bytes remaining to be set.
- Xd holds the lowest address to be set.
- On completion of the instruction:
 - Xn holds the number of bytes remaining to be set.
 - Xd holds the lowest address that has not been set.

For SETEN, option A, when $PSTATE.C = '0'$:

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be set.
- Xd holds the lowest address to be set - Xn .
- On completion of the instruction, Xn holds 0.

For SETEN, option B, when $PSTATE.C = '1'$:

- Xn holds the number of bytes remaining to be set.
- Xd holds the lowest address to be set.
- On completion of the instruction:
 - Xn holds 0.
 - Xd holds the lowest address that has not been set.

Integer

(FEAT_MOPS)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sz	0	1	1	0	0	1	1	1	0	Rs					x	x	1	0	0	1	Rn					Rd					
o0				op1					op2																						

Encoding for the Prologue variant

Applies when (op2 == 0010)

```
SETPN [<Xd>]!, <Xn>!, <Xs>
```

Encoding for the Main variant

Applies when (op2 == 0110)

```
SETMN [<Xd>]!, <Xn>!, <Xs>
```

Encoding for the Epilogue variant

Applies when (op2 == 1010)

```
SETEN [<Xd>]!, <Xn>!, <Xs>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memset : SETParams;
memset.d = UInt(Rd);
memset.s = UInt(Rs);
memset.n = UInt(Rn);
let options : bits(2) = op2[1:0];
let nontemporal : boolean = options[1] == '1';

case op2[3:2] of
  when '00' => memset.stage = MOPSStage_Prologue;
  when '01' => memset.stage = MOPSStage_Main;
  when '10' => memset.stage = MOPSStage_Epilogue;
  otherwise => EndOfDecode(Decode_UNDEF);
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see [Architectural Constraints on UNPREDICTABLE behaviors](#), and particularly [Memory Copy and Memory Set SET*](#) and [Crossing a page boundary with different memory types or Shareability attributes](#).

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address and for option B is updated by the instruction, encoded in the "Rd" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be set and is updated by the instruction, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be set and is updated by the instruction, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be set and is set to zero on completion of the instruction, encoded in the "Rn" field.
<Xs>	Is the 64-bit name of the general-purpose register that holds the source data, encoded in the "Rs" field.


```

CheckMOPSEnabled();

CheckSETConstrainedUnpredictable(memset.memset.n, memset.d, memset.s);

let data : bits(8) = X{}(memset.s);
var B : MOPSEBlocksize = 0;

memset.is_setg = FALSE;
memset.nzcv = PSTATE.[N,Z,C,V];
memset.toaddress = PSTATE.[N,Z,C,V];
memset.toaddress = X{64}(memset.d);
if memset.stage == MOPSEStage_Prologue then
    memset.setsize = UInt(X{64}(memset.n));
else
    memset.setsize = SInt(X{64}(memset.n));
end;
memset.implements_option_a = SETOptionA();

let privileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
    else PSTATE.EL()
    else PSTATE.EL != EL0);

let accdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, privileged, nontemporal);

if memset.stage == MOPSEStage_Prologue then
    if memset.setsize > ArchMaxMOPSEBlocksize then
        memset.setsize = ArchMaxMOPSEBlocksize;
    end;

    if memset.implements_option_a then
        memset.nzcv = '0000';
        memset.toaddress = memset.toaddress + memset.setsize;
        memset.setsize = 0 - memset.setsize;
    else
        memset.nzcv = '0010';
    end;
end;

memset.stagesetsize = MemSetStageSize(memset);

if memset.stage != MOPSEStage_Prologue then
    CheckMemSetParams(memset, options);
end;

var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var memory_set : integer;
var fault : boolean = FALSE;

if memset.implements_option_a then
    while memset.stagesetsize < 0 && !fault looplimit ArchMaxMOPSEBlocksize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = SETSizeChoice(memset, 1);
        assert B <= -1 * memset.stagesetsize;

        (memory_set, memaddrdesc, memstatus) = MemSetBytes(memset.toaddress + memset.setsize,
            data, B, accdesc);

        if memory_set != B then
            fault = TRUE;
        else
            memset.setsize = memset.setsize + B;
            memset.stagesetsize = memset.stagesetsize + B;
        end;
    end;
else
    while memset.stagesetsize > 0 && !fault looplimit ArchMaxMOPSEBlocksize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = SETSizeChoice(memset, 1);
        assert B <= memset.stagesetsize;

        (memory_set, memaddrdesc, memstatus) = MemSetBytes(memset.toaddress, data, B, accdesc);

        if memory_set != B then
            fault = TRUE;
        else
            memset.toaddress = memset.toaddress + B;
            memset.setsize = memset.setsize - B;
            memset.stagesetsize = memset.stagesetsize - B;
        end;
    end;
end;

UpdateSetRegisters(memset, fault, memory_set);

if fault then

```

```
if IsFault(memaddrdesc) then
    AArch64_Abort(memaddrdesc.fault);
else
    let iswrite : boolean = TRUE;
    HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
end;
end;

if memset.stage == MOPSTage_Prologue then
    PSTATE.[N,Z,C,V] = memset.nzcv;
end;
```

(old)	htmldiff from-	(new)
-------	----------------	-------

SETPT, SETMT, SETET

Memory set, unprivileged

These instructions set a required number of bytes in memory to the value in the least significant byte of the source data register. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: SETPT, then SETMT, and then SETET.

SETPT performs some preconditioning of the arguments suitable for using the SETMT instruction, and sets an IMPLEMENTATION DEFINED portion of the requested number of bytes. SETMT sets a further IMPLEMENTATION DEFINED portion of the remaining bytes. SETET sets any final remaining bytes.

Note

The ability to set an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being set are divided between the different instructions.

For more information on exceptions specific to memory set instructions, see [Memory Copy and Memory Set exceptions](#).

The architecture supports two algorithms for the memory set: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For SETPT:

- If $Xn[63] = '1'$, the set size is saturated to $0x7FFFFFFFFFFFFFFF$.

On completion of SETPT, option A:

- Xn holds -1 times the number of bytes in the saturated set size remaining to be set.
- Xd holds the original Xd + saturated set size.
- $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,0,0\}$.

On completion of SETPT, option B:

- Xn holds the number of bytes in the saturated set size remaining to be set.
- Xd holds the lowest address that has not been set.
- $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,1,0\}$.

For SETMT, option A, when $PSTATE.C = '0'$:

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be set.
- Xd holds the lowest address to be set - Xn .
- On completion of the instruction, Xn holds -1 times the number of bytes remaining to be set.

For SETMT, option B, when $PSTATE.C = '1'$:

- Xn holds the number of bytes remaining to be set.
- Xd holds the lowest address to be set.
- On completion of the instruction:
 - Xn holds the number of bytes remaining to be set.
 - Xd holds the lowest address that has not been set.

For SETET, option A, when $PSTATE.C = '0'$:

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be set.
- Xd holds the lowest address to be set - Xn .
- On completion of the instruction, Xn holds 0.

For SETET, option B, when $PSTATE.C = '1'$:

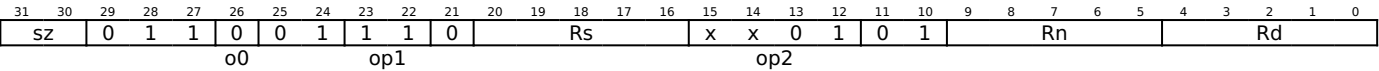
- Xn holds the number of bytes remaining to be set.
- Xd holds the lowest address to be set.
- On completion of the instruction:
 - Xn holds 0.
 - Xd holds the lowest address that has not been set.

Explicit Memory Write effects produced by the instruction behave as if the instruction was executed at EL0 if the [Effective value](#) of $PSTATE.UAO$ is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the [Effective value](#) of $HCR_EL2().\{E2H, TGE\}$ is '11'.

Otherwise, the Explicit Memory Write effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer
(FEAT_MOPS)



Encoding for the Prologue variant

Applies when (op2 == 0001)

```
SETPT [<Xd>]!, <Xn>!, <Xs>
```

Encoding for the Main variant

Applies when (op2 == 0101)

```
SETMT [<Xd>]!, <Xn>!, <Xs>
```

Encoding for the Epilogue variant

Applies when (op2 == 1001)

```
SETET [<Xd>]!, <Xn>!, <Xs>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memset : SETParams;
memset.d = UInt(Rd);
memset.s = UInt(Rs);
memset.n = UInt(Rn);
let options : bits(2) = op2[1:0];
let nontemporal : boolean = options[1] == '1';

case op2[3:2] of
  when '00' => memset.stage = MOPSStage_Prologue;
  when '01' => memset.stage = MOPSStage_Main;
  when '10' => memset.stage = MOPSStage_Epilogue;
  otherwise => EndOfDecode(Decode_UNDEF);
end;
```

For information about the **CONSTRAINED UNPREDICTABLE** behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set SET** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

- <Xd> For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field.
For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address and for option B is updated by the instruction, encoded in the "Rd" field.
- <Xn> For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be set and is updated by the instruction, encoded in the "Rn" field.
For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be set and is updated by the instruction, encoded in the "Rn" field.
For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be set and is set to zero on completion of the instruction, encoded in the "Rn" field.
- <Xs> Is the 64-bit name of the general-purpose register that holds the source data, encoded in the "Rs" field.


```

CheckMOPSEnabled();

CheckSETConstrainedUnpredictable(memset.memset.n, memset.d, memset.s);

let data : bits(8) = X{}(memset.s);
var B : MOPSBlockSize = 0;

memset.is_setg = FALSE;
memset.nzcv = PSTATE.[N,Z,C,V];
memset.toaddress = PSTATE.[N,Z,C,V];
memset.toaddress = X{64}(memset.d);
if memset.stage == MOPSStage_Prologue then
    memset.setsize = UInt(X{64}(memset.n));
else
    memset.setsize = SInt(X{64}(memset.n));
end;
memset.implements_option_a = SETOptionA();

let privileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
    else PSTATE.EL()
    else PSTATE.EL != EL0);

let accdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, privileged, nontemporal);

if memset.stage == MOPSStage_Prologue then
    if memset.setsize > ArchMaxMOPSBlockSize then
        memset.setsize = ArchMaxMOPSBlockSize;
    end;

    if memset.implements_option_a then
        memset.nzcv = '0000';
        memset.toaddress = memset.toaddress + memset.setsize;
        memset.setsize = 0 - memset.setsize;
    else
        memset.nzcv = '0010';
    end;
end;

memset.stagesetsize = MemSetStageSize(memset);

if memset.stage != MOPSStage_Prologue then
    CheckMemSetParams(memset, options);
end;

var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var memory_set : integer;
var fault : boolean = FALSE;

if memset.implements_option_a then
    while memset.stagesetsize < 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = SETSizeChoice(memset, 1);
        assert B <= -1 * memset.stagesetsize;

        (memory_set, memaddrdesc, memstatus) = MemSetBytes(memset.toaddress + memset.setsize,
            data, B, accdesc);

        if memory_set != B then
            fault = TRUE;
        else
            memset.setsize = memset.setsize + B;
            memset.stagesetsize = memset.stagesetsize + B;
        end;
    end;
else
    while memset.stagesetsize > 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = SETSizeChoice(memset, 1);
        assert B <= memset.stagesetsize;

        (memory_set, memaddrdesc, memstatus) = MemSetBytes(memset.toaddress, data, B, accdesc);

        if memory_set != B then
            fault = TRUE;
        else
            memset.toaddress = memset.toaddress + B;
            memset.setsize = memset.setsize - B;
            memset.stagesetsize = memset.stagesetsize - B;
        end;
    end;
end;

UpdateSetRegisters(memset, fault, memory_set);

if fault then

```

```
if IsFault(memaddrdesc) then
  AArch64_Abort(memaddrdesc.fault);
else
  let iswrite : boolean = TRUE;
  HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
end;
end;

if memset.stage == MOPSTage_Prologue then
  PSTATE.[N,Z,C,V] = memset.nzcv;
end;
```

(old)	htmldiff from-	(new)
-------	----------------	-------

SETPTN, SETMTN, SETETN

Memory set, unprivileged and non-temporal

These instructions set a required number of bytes in memory to the value in the least significant byte of the source data register. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: SETPTN, then SETMTN, and then SETETN.

SETPTN performs some preconditioning of the arguments suitable for using the SETMTN instruction, and sets an IMPLEMENTATION DEFINED portion of the requested number of bytes. SETMTN sets a further IMPLEMENTATION DEFINED portion of the remaining bytes. SETETN sets any final remaining bytes.

Note

The ability to set an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being set are divided between the different instructions.

For more information on exceptions specific to memory set instructions, see [Memory Copy and Memory Set exceptions](#).

The architecture supports two algorithms for the memory set: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For SETPTN:

- If $Xn[63] = '1'$, the set size is saturated to $0x7FFFFFFFFFFFFFFF$.

On completion of SETPTN, option A:

- Xn holds -1 times the number of bytes in the saturated set size remaining to be set.
- Xd holds the original Xd + saturated set size.
- $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,0,0\}$.

On completion of SETPTN, option B:

- Xn holds the number of bytes in the saturated set size remaining to be set.
- Xd holds the lowest address that has not been set.
- $PSTATE.\{N,Z,C,V\}$ are set to $\{0,0,1,0\}$.

For SETMTN, option A, when $PSTATE.C = '0'$:

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be set.
- Xd holds the lowest address to be set - Xn .
- On completion of the instruction, Xn holds -1 times the number of bytes remaining to be set.

For SETMTN, option B, when $PSTATE.C = '1'$:

- Xn holds the number of bytes remaining to be set.
- Xd holds the lowest address to be set.
- On completion of the instruction:
 - Xn holds the number of bytes remaining to be set.
 - Xd holds the lowest address that has not been set.

For SETETN, option A, when $PSTATE.C = '0'$:

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be set.
- Xd holds the lowest address to be set - Xn .
- On completion of the instruction, Xn holds 0.

For SETETN, option B, when $PSTATE.C = '1'$:

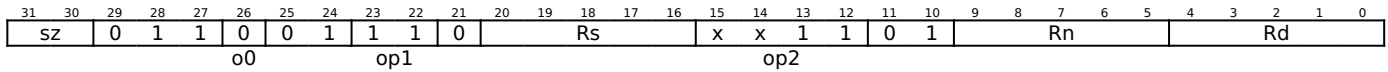
- Xn holds the number of bytes remaining to be set.
- Xd holds the lowest address to be set.
- On completion of the instruction:
 - Xn holds 0.
 - Xd holds the lowest address that has not been set.

Explicit Memory Write effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of HCR_EL2().{E2H, TGE} is '11'.

Otherwise, the Explicit Memory Write effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer (FEAT_MOPS)



Encoding for the Prologue variant

Applies when (op2 == 0011)

SETPTN [<Xd>]!, <Xn>!, <Xs>

Encoding for the Main variant

Applies when (op2 == 0111)

SETMTN [<Xd>]!, <Xn>!, <Xs>

Encoding for the Epilogue variant

Applies when (op2 == 1011)

SETETN [<Xd>]!, <Xn>!, <Xs>

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memset : SETParams;
memset.d = UInt(Rd);
memset.s = UInt(Rs);
memset.n = UInt(Rn);
let options : bits(2) = op2[1:0];
let nontemporal : boolean = options[1] == '1';

case op2[3:2] of
  when '00' => memset.stage = MOPSStage_Prologue;
  when '01' => memset.stage = MOPSStage_Main;
  when '10' => memset.stage = MOPSStage_Epilogue;
  otherwise => EndOfDecode(Decode_UNDEF);
end;
```

For information about the *CONSTRAINED UNPREDICTABLE* behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set SET** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address and for option B is updated by the instruction, encoded in the "Rd" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be set and is updated by the instruction, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be set and is updated by the instruction, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be set and is set to zero on completion of the instruction, encoded in the "Rn" field.
<Xs>	Is the 64-bit name of the general-purpose register that holds the source data, encoded in the "Rs" field.


```

CheckMOPSEnabled();

CheckSETConstrainedUnpredictable(memset.memset.n, memset.d, memset.s);

let data : bits(8) = X{}(memset.s);
var B : MOPSBlockSize = 0;

memset.is_setg = FALSE;
memset.nzcv = PSTATE.[N,Z,C,V];
memset.toaddress = PSTATE.[N,Z,C,V];
memset.toaddress = X{64}(memset.d);
if memset.stage == MOPSStage_Prologue then
    memset.setsize = UInt(X{64}(memset.n));
else
    memset.setsize = SInt(X{64}(memset.n));
end;
memset.implements_option_a = SETOptionA();

let privileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
    else PSTATE.EL()
    else PSTATE.EL != EL0);

let accdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, privileged, nontemporal);

if memset.stage == MOPSStage_Prologue then
    if memset.setsize > ArchMaxMOPSBlockSize then
        memset.setsize = ArchMaxMOPSBlockSize;
    end;

    if memset.implements_option_a then
        memset.nzcv = '0000';
        memset.toaddress = memset.toaddress + memset.setsize;
        memset.setsize = 0 - memset.setsize;
    else
        memset.nzcv = '0010';
    end;
end;

memset.stagesetsize = MemSetStageSize(memset);

if memset.stage != MOPSStage_Prologue then
    CheckMemSetParams(memset, options);
end;

var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var memory_set : integer;
var fault : boolean = FALSE;

if memset.implements_option_a then
    while memset.stagesetsize < 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = SETSizeChoice(memset, 1);
        assert B <= -1 * memset.stagesetsize;

        (memory_set, memaddrdesc, memstatus) = MemSetBytes(memset.toaddress + memset.setsize,
            data, B, accdesc);

        if memory_set != B then
            fault = TRUE;
        else
            memset.setsize = memset.setsize + B;
            memset.stagesetsize = memset.stagesetsize + B;
        end;
    end;
else
    while memset.stagesetsize > 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = SETSizeChoice(memset, 1);
        assert B <= memset.stagesetsize;

        (memory_set, memaddrdesc, memstatus) = MemSetBytes(memset.toaddress, data, B, accdesc);

        if memory_set != B then
            fault = TRUE;
        else
            memset.toaddress = memset.toaddress + B;
            memset.setsize = memset.setsize - B;
            memset.stagesetsize = memset.stagesetsize - B;
        end;
    end;
end;

UpdateSetRegisters(memset, fault, memory_set);

if fault then

```



```
if IsFault(memaddrdesc) then
  AArch64_Abort(memaddrdesc.fault);
else
  let iswrite : boolean = TRUE;
  HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
end;
end;

if memset.stage == MOPSStage_Prologue then
  PSTATE.[N,Z,C,V] = memset.nzcv;
end;
```

(old)	htmldiff from-	(new)
-------	----------------	-------

SYS

System instruction

For more information, see [Op0 equals 0b01, cache maintenance, TLB maintenance, and address translation instructions](#) for the encodings of System instructions.

This instruction is used by the aliases [APAS](#), [AT](#), [BRB](#), [CFP](#), [COSP](#), [CPP](#), [DC](#), [DVP](#), [GCSPOPCX](#), [GCSPOPX](#), [GCSPUSHM](#), [GCSPUSHX](#), [GCSSSI](#), [GIC](#), [GSB](#), [IC](#), [MLBI](#), [TLBI](#), and [TRCIT](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	1	0	1	0	1	0	0	0	0	1	op1			CRn				CRm				op2			Rt				

L

Encoding

SYS #<op1>, <Cn>, <Cm>, #<op2>{, <Xt>}

Decode for this encoding

```
let t : integer      = UInt(Rt);
let sys_L : bits(1)  = L;
let sys_op0 : bits(2) = '01';
let sys_op1 : bits(3) = op1;
let sys_op2 : bits(3) = op2;
let sys_crn : bits(4) = CRn;
let sys_crm : bits(4) = CRm;
```

Assembler Symbols

<op1>	Is a 3-bit unsigned immediate, in the range 0 to 7, encoded in the "op1" field.
<Cn>	Is a name 'Cn', with 'n' in the range 0 to 15, encoded in the "CRn" field.
<Cm>	Is a name 'Cm', with 'm' in the range 0 to 15, encoded in the "CRm" field.
<op2>	Is a 3-bit unsigned immediate, in the range 0 to 7, encoded in the "op2" field.
<Xt>	Is the 64-bit name of the optional general-purpose source register, defaulting to '11111', encoded in the "Rt" field.

Alias Conditions

Alias	Is preferred when
APAS	op1 == '110' && CRn == '0111' && CRm == '0000' && op2 == '000'
AT	CRn == '0111' && CRm IN {'100x'} && SysOp(op1, '0111', CRm, op2) == Sys_AT
BRB	op1 == '001' && CRn == '0111' && CRm == '0010' && SysOp('001', '0111', '0010', op2) == Sys_BRB
CFP	op1 == '011' && CRn == '0111' && CRm == '0011' && op2 == '100'
COSP	op1 == '011' && CRn == '0111' && CRm == '0011' && op2 == '110'
CPP	op1 == '011' && CRn == '0111' && CRm == '0011' && op2 == '111'
DC	CRn == '0111' && SysOp(op1, '0111', CRm, op2) == Sys_DC
DVP	op1 == '011' && CRn == '0111' && CRm == '0011' && op2 == '101'
GCSPOPCX	op1 == '000' && CRn == '0111' && CRm == '0111' && op2 == '101'
GCSPOPX	op1 == '000' && CRn == '0111' && CRm == '0111' && op2 == '110'
GCSPUSHM	op1 == '011' && CRn == '0111' && CRm == '0111' && op2 == '000'
GCSPUSHX	op1 == '000' && CRn == '0111' && CRm == '0111' && op2 == '100'
GCSSSI	op1 == '011' && CRn == '0111' && CRm == '0111' && op2 == '010'
GIC	CRn == '1100' && CRm IN {'0001', '0010'} && SysOp(op1, '1100', CRm, op2) == Sys_GIC
GSB	op1 == '000' && CRn == '1100' && CRm == '0000' && SysOp('000', '1100', '0000', op2) == Sys_GSB
IC	CRn == '0111' && SysOp(op1, '0111', CRm, op2) == Sys_IC
MLBI	op1 == '100' && CRn == '0111' && CRm == '0000' && SysOp('100', '0111', '0000', op2) IN {'1xx'} Sys_MLBI
TLBI	CRn IN {'100x'} && SysOp(op1, CRn, CRm, op2) == Sys_TLBI
TRCIT	op1 == '011' && CRn == '0111' && CRm == '0010' && op2 == '111'

Operation

```
AArch64_SysInstr(sys_op0, sys_op1, sys_crn, sys_crm, sys_op2, t);
```

(old)	htmldiff from-	(new)
-------	----------------	-------

A64 -- SIMD and Floating-point Instructions (alphabetic order)

ABS: Absolute value (vector).

ADD (vector): Add (vector).

ADDHN, ADDHN2: Add returning high narrow.

ADDP (scalar): Add pair of elements (scalar).

ADDP (vector): Add pairwise (vector).

ADDV: Add across vector.

AESD: AES single round decryption.

AESE: AES single round encryption.

AESIMC: AES inverse mix columns.

AESMC: AES mix columns.

AND (vector): Bitwise AND (vector).

BCAX: Bit clear and exclusive-OR.

BF1CVTL, BF1CVTL2, BF2CVTL, BF2CVTL2: 8-bit floating-point convert to BFloat16 (vector).

BFCVT: Single-precision convert to BFloat16 (scalar).

[BFCVTN, BFCVTN2](#): Single-precision convert to BFloat16 (vector).

BFDOT (by element): BFloat16 dot product to single-precision (vector, by element).

BFDOT (vector): BFloat16 dot product to single-precision (vector).

BFMLALB, BFMLALT (by element): BFloat16 multiply-add to single-precision (by element).

BFMLALB, BFMLALT (vector): BFloat16 multiply-add to single-precision (vector).

[BFMMLA \(widening\)](#): BFloat16 matrix multiply-accumulate to single-precision.

BIC (vector, immediate): Bitwise bit clear (vector, immediate).

BIC (vector, register): Bitwise bit clear (vector, register).

BIF: Bitwise insert if false.

BIT: Bitwise insert if true.

BSL: Bitwise select.

CLS (vector): Count leading sign bits (vector).

CLZ (vector): Count leading zero bits (vector).

CMEQ (register): Compare bitwise equal (vector).

CMEQ (zero): Compare bitwise equal to zero (vector).

CMGE (register): Compare signed greater than or equal (vector).

CMGE (zero): Compare signed greater than or equal to zero (vector).

CMGT (register): Compare signed greater than (vector).

CMGT (zero): Compare signed greater than zero (vector).

CMHI (register): Compare unsigned higher (vector).

CMHS (register): Compare unsigned higher or same (vector).

CMLE (zero): Compare signed less than or equal to zero (vector).

CMLT (zero): Compare signed less than zero (vector).

CMTST: Compare bitwise test bits nonzero (vector).

CNT: Population count per byte.

DUP (element): Duplicate vector element to vector or scalar.

DUP (general): Duplicate general-purpose register to vector.

EOR (vector): Bitwise exclusive-OR (vector).

EOR3: Three-way exclusive-OR.

EXT: Extract vector from pair of vectors.

F1CVTL, F1CVTL2, F2CVTL, F2CVTL2: 8-bit floating-point convert to half-precision (vector).

FABD: Floating-point absolute difference (vector).

FABS (scalar): Floating-point absolute value (scalar).

FABS (vector): Floating-point absolute value (vector).

FACGE: Floating-point absolute compare greater than or equal (vector).

FACGT: Floating-point absolute compare greater than (vector).

FADD (scalar): Floating-point add (scalar).

FADD (vector): Floating-point add (vector).

FADDP (scalar): Floating-point add pair of elements (scalar).

FADDP (vector): Floating-point add pairwise (vector).

FAMAX: Floating-point absolute maximum.

FAMIN: Floating-point absolute minimum.

FCADD: Floating-point complex add.

FCCMP: Floating-point conditional quiet compare (scalar).

FCCMPE: Floating-point conditional signaling compare (scalar).

FCMEQ (register): Floating-point compare equal (vector).

FCMEQ (zero): Floating-point compare equal to zero (vector).

FCMGE (register): Floating-point compare greater than or equal (vector).

FCMGE (zero): Floating-point compare greater than or equal to zero (vector).

FCMGT (register): Floating-point compare greater than (vector).

FCMGT (zero): Floating-point compare greater than zero (vector).

FCMLA: Floating-point complex multiply accumulate.

FCMLA (by element): Floating-point complex multiply accumulate (by element).

FCMLE (zero): Floating-point compare less than or equal to zero (vector).

FCMLT (zero): Floating-point compare less than zero (vector).

FCMP: Floating-point quiet compare (scalar).

FCMPE: Floating-point signaling compare (scalar).

FCSEL: Floating-point conditional select (scalar).

FCVT: Floating-point convert precision (scalar).

FCVTAS (scalar SIMD&FP): Floating-point convert to signed integer, rounding to nearest with ties to away (scalar SIMD&FP).

FCVTAS (scalar): Floating-point convert to signed integer, rounding to nearest with ties to away (scalar).

FCVTAS (vector): Floating-point convert to signed integer, rounding to nearest with ties to away (vector).

FCVTAU (scalar SIMD&FP): Floating-point convert to unsigned integer, rounding to nearest with ties to away (scalar SIMD&FP).

FCVTAU (scalar): Floating-point convert to unsigned integer, rounding to nearest with ties to away (scalar).

FCVTAU (vector): Floating-point convert to unsigned integer, rounding to nearest with ties to away (vector).

FCVTL, FCVTL2: Floating-point convert to higher precision long (vector).

FCVTMS (scalar SIMD&FP): Floating-point convert to signed integer, rounding toward minus infinity (scalar SIMD&FP).

FCVTMS (scalar): Floating-point convert to signed integer, rounding toward minus infinity (scalar).

FCVTMS (vector): Floating-point convert to signed integer, rounding toward minus infinity (vector).

FCVTMU (scalar SIMD&FP): Floating-point convert to unsigned integer, rounding toward minus infinity (scalar SIMD&FP).

FCVTMU (scalar): Floating-point convert to unsigned integer, rounding toward minus infinity (scalar).

FCVTMU (vector): Floating-point convert to unsigned integer, rounding toward minus infinity (vector).

FCVTN (half-precision to 8-bit floating-point): Half-precision convert to 8-bit floating-point (vector).

FCVTN, FCVTN2 (double to single-precision, single to half-precision): Floating-point convert to lower precision narrow (vector).

FCVTN, FCVTN2 (single-precision to 8-bit floating-point): Single-precision convert to 8-bit floating-point (vector).

FCVTNS (scalar SIMD&FP): Floating-point convert to signed integer, rounding to nearest with ties to even (scalar SIMD&FP).

FCVTNS (scalar): Floating-point convert to signed integer, rounding to nearest with ties to even (scalar).

FCVTNS (vector): Floating-point convert to signed integer, rounding to nearest with ties to even (vector).

FCVTNU (scalar SIMD&FP): Floating-point convert to unsigned integer, rounding to nearest with ties to even (scalar SIMD&FP).

FCVTNU (scalar): Floating-point convert to unsigned integer, rounding to nearest with ties to even (scalar).

FCVTNU (vector): Floating-point convert to unsigned integer, rounding to nearest with ties to even (vector).

FCVTPS (scalar SIMD&FP): Floating-point convert to signed integer, rounding toward plus infinity (scalar SIMD&FP).

FCVTPS (scalar): Floating-point convert to signed integer, rounding toward plus infinity (scalar).

FCVTPS (vector): Floating-point convert to signed integer, rounding toward plus infinity (vector).

FCVTPU (scalar SIMD&FP): Floating-point convert to unsigned integer, rounding toward plus infinity (scalar SIMD&FP).

FCVTPU (scalar): Floating-point convert to unsigned integer, rounding toward plus infinity (scalar).

FCVTPU (vector): Floating-point convert to unsigned integer, rounding toward plus infinity (vector).

FCVTXN, FCVTXN2: Floating-point convert to lower precision narrow, rounding to odd (vector).

FCVTZS (scalar SIMD&FP): Floating-point convert to signed integer, rounding toward zero (scalar SIMD&FP).

FCVTZS (scalar, fixed-point): Floating-point convert to signed fixed-point, rounding toward zero (scalar).

FCVTZS (scalar, integer): Floating-point convert to signed integer, rounding toward zero (scalar).

FCVTZS (vector, fixed-point): Floating-point convert to signed fixed-point, rounding toward zero (vector).

FCVTZS (vector, integer): Floating-point convert to signed integer, rounding toward zero (vector).

FCVTZU (scalar SIMD&FP): Floating-point convert to unsigned integer, rounding toward zero (scalar SIMD&FP).

FCVTZU (scalar, fixed-point): Floating-point convert to unsigned fixed-point, rounding toward zero (scalar).

FCVTZU (scalar, integer): Floating-point convert to unsigned integer, rounding toward zero (scalar).

FCVTZU (vector, fixed-point): Floating-point convert to unsigned fixed-point, rounding toward zero (vector).

FCVTZU (vector, integer): Floating-point convert to unsigned integer, rounding toward zero (vector).

FDIV (scalar): Floating-point divide (scalar).

FDIV (vector): Floating-point divide (vector).

FDOT (8-bit floating-point to half-precision, by element): 8-bit floating-point dot product to half-precision (vector, by element).

FDOT (8-bit floating-point to half-precision, vector): 8-bit floating-point dot product to half-precision (vector).

FDOT (8-bit floating-point to single-precision, by element): 8-bit floating-point dot product to single-precision (vector, by element).

FDOT (8-bit floating-point to single-precision, vector): 8-bit floating-point dot product to single-precision (vector).

FDOT (half-precision to single-precision, by element): Half-precision dot product to single-precision (vector, by element).

FDOT (half-precision to single-precision, vector): Half-precision dot product to single-precision (vector).

FJCVTZS: Floating-point Javascript convert to signed fixed-point, rounding toward zero.

FMADD: Floating-point fused multiply-add (scalar).

FMAX (scalar): Floating-point maximum (scalar).

FMAX (vector): Floating-point maximum (vector).

FMAXNM (scalar): Floating-point maximum number (scalar).

FMAXNM (vector): Floating-point maximum number (vector).

FMAXNMP (scalar): Floating-point maximum number of pair of elements (scalar).

FMAXNMP (vector): Floating-point maximum number pairwise (vector).

FMAXNMV: Floating-point maximum number across vector.

FMAXP (scalar): Floating-point maximum of pair of elements (scalar).

FMAXP (vector): Floating-point maximum pairwise (vector).

FMAXV: Floating-point maximum across vector.

FMIN (scalar): Floating-point minimum (scalar).

FMIN (vector): Floating-point minimum (vector).

FMINNM (scalar): Floating-point minimum number (scalar).

FMINNM (vector): Floating-point minimum number (vector).

FMINNMP (scalar): Floating-point minimum number of pair of elements (scalar).

FMINNMP (vector): Floating-point minimum number pairwise (vector).

FMINNMV: Floating-point minimum number across vector.

FMINP (scalar): Floating-point minimum of pair of elements (scalar).

FMINP (vector): Floating-point minimum pairwise (vector).

FMINV: Floating-point minimum across vector.

FMLA (by element): Floating-point fused multiply-add to accumulator (by element).

FMLA (vector): Floating-point fused multiply-add to accumulator (vector).

FMLAL, FMLAL2 (by element): Floating-point fused multiply-add long to accumulator (by element).

FMLAL, FMLAL2 (vector): Floating-point fused multiply-add long to accumulator (vector).

FMLALB, FMLALT (by element): 8-bit floating-point multiply-add to half-precision (vector, by element).

FMLALB, FMLALT (vector): 8-bit floating-point multiply-add to half-precision (vector).

FMLALLBB, FMLALLBT, FMLALLTB, FMLALLTT (by element): 8-bit floating-point multiply-add to single-precision (vector, by element).

FMLALLBB, FMLALLBT, FMLALLTB, FMLALLTT (vector): 8-bit floating-point multiply-add to single-precision (vector).

FMLS (by element): Floating-point fused multiply-subtract from accumulator (by element).

FMLS (vector): Floating-point fused multiply-subtract from accumulator (vector).

FMLSL, FMLSL2 (by element): Floating-point fused multiply-subtract long from accumulator (by element).

FMLSL, FMLSL2 (vector): Floating-point fused multiply-subtract long from accumulator (vector).

FMMLA (non-widening): Half-precision matrix multiply-accumulate.

[FMMLA \(widening, 8-bit floating-point to half-precision\)](#): 8-bit floating-point matrix multiply-accumulate to half-precision.

[FMMLA \(widening, 8-bit floating-point to single-precision\)](#): 8-bit floating-point matrix multiply-accumulate to single-precision.

[FMMLA \(widening, half-precision to single-precision\)](#): Half-precision matrix multiply-accumulate to single-precision.

FMOV (general): Floating-point move to or from general-purpose register without conversion.

FMOV (register): Floating-point move register without conversion.

FMOV (scalar, immediate): Floating-point move immediate (scalar).

FMOV (vector, immediate): Floating-point move immediate (vector).

FMSUB: Floating-point fused multiply-subtract (scalar).

FMUL (by element): Floating-point multiply (by element).

FMUL (scalar): Floating-point multiply (scalar).

FMUL (vector): Floating-point multiply (vector).

FMULX: Floating-point multiply extended.

FMULX (by element): Floating-point multiply extended (by element).

FNEG (scalar): Floating-point negate (scalar).

FNEG (vector): Floating-point negate (vector).

FNMADD: Floating-point negated fused multiply-add (scalar).

FNMSUB: Floating-point negated fused multiply-subtract (scalar).

FNMUL (scalar): Floating-point multiply-negate (scalar).

FRECPE: Floating-point reciprocal estimate.

FRECPS: Floating-point reciprocal step.

FRECPX: Floating-point reciprocal exponent (scalar).

FRINT32X (scalar): Floating-point round to 32-bit integer, using current rounding mode (scalar).

FRINT32X (vector): Floating-point round to 32-bit integer, using current rounding mode (vector).

FRINT32Z (scalar): Floating-point round to 32-bit integer toward zero (scalar).

FRINT32Z (vector): Floating-point round to 32-bit integer toward zero (vector).

FRINT64X (scalar): Floating-point round to 64-bit integer, using current rounding mode (scalar).

FRINT64X (vector): Floating-point round to 64-bit integer, using current rounding mode (vector).

FRINT64Z (scalar): Floating-point round to 64-bit integer toward zero (scalar).

FRINT64Z (vector): Floating-point round to 64-bit integer toward zero (vector).

FRINTA (scalar): Floating-point round to integral, to nearest with ties to away (scalar).

FRINTA (vector): Floating-point round to integral, to nearest with ties to away (vector).

FRINTI (scalar): Floating-point round to integral, using current rounding mode (scalar).

FRINTI (vector): Floating-point round to integral, using current rounding mode (vector).

FRINTM (scalar): Floating-point round to integral, toward minus infinity (scalar).

FRINTM (vector): Floating-point round to integral, toward minus infinity (vector).

FRINTN (scalar): Floating-point round to integral, to nearest with ties to even (scalar).

FRINTN (vector): Floating-point round to integral, to nearest with ties to even (vector).

FRINTP (scalar): Floating-point round to integral, toward plus infinity (scalar).

FRINTP (vector): Floating-point round to integral, toward plus infinity (vector).

FRINTX (scalar): Floating-point round to integral exact, using current rounding mode (scalar).

FRINTX (vector): Floating-point round to integral exact, using current rounding mode (vector).

FRINTZ (scalar): Floating-point round to integral, toward zero (scalar).

FRINTZ (vector): Floating-point round to integral, toward zero (vector).

FRSQRT: Floating-point reciprocal square root estimate.

FRSQRTS: Floating-point reciprocal square root step.

FSCALE: Floating-point adjust exponent by vector.

FSQRT (scalar): Floating-point square root (scalar).

FSQRT (vector): Floating-point square root (vector).

FSUB (scalar): Floating-point subtract (scalar).

FSUB (vector): Floating-point subtract (vector).

INS (element): Insert vector element from another vector element.

INS (general): Insert vector element from general-purpose register.

LD1 (multiple structures): Load multiple single-element structures to one, two, three, or four registers.

LD1 (single structure): Load one single-element structure to one lane of one register.

LD1R: Load one single-element structure and replicate to all lanes (of one register).

LD2 (multiple structures): Load multiple 2-element structures to two registers.

LD2 (single structure): Load single 2-element structure to one lane of two registers.

LD2R: Load single 2-element structure and replicate to all lanes of two registers.

LD3 (multiple structures): Load multiple 3-element structures to three registers.

LD3 (single structure): Load single 3-element structure to one lane of three registers.

LD3R: Load single 3-element structure and replicate to all lanes of three registers.

LD4 (multiple structures): Load multiple 4-element structures to four registers.

LD4 (single structure): Load single 4-element structure to one lane of four registers.

LD4R: Load single 4-element structure and replicate to all lanes of four registers.

LDAP1 (SIMD&FP): Load-acquire RCpc one single-element structure to one lane of one register.

LDAPUR (SIMD&FP): Load-acquire RCpc SIMD&FP register (unscaled offset).

LDBFADD, LDBFADDA, LDBFADDAL, LDBFADDL: Atomic BFloat16 add.

LDBFMAX, LDBFMAXA, LDBFMAXAL, LDBFMAXL: Atomic BFloat16 maximum.

LDBFMAXNM, LDBFMAXNMA, LDBFMAXNMAL, LDBFMAXNML: Atomic BFloat16 maximum number.

LDBFMIN, LDBFMINA, LDBFMINAL, LDBFMINL: Atomic BFloat16 minimum.

LDBFMINNM, LDBFMINNMA, LDBFMINNMAL, LDBFMINNML: Atomic BFloat16 minimum number.

LDFADD, LDFADDA, LDFADDAL, LDFADDL: Atomic floating-point add.

LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL: Atomic floating-point maximum.

LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML: Atomic floating-point maximum number.

LDFMIN, LDFMINA, LDFMINAL, LDFMINL: Atomic floating-point minimum.

LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML: Atomic floating-point minimum number.

LDNP (SIMD&FP): Load pair of SIMD&FP registers, with non-temporal hint.

LDP (SIMD&FP): Load pair of SIMD&FP registers.

LDR (immediate, SIMD&FP): Load SIMD&FP register (immediate offset).

LDR (literal, SIMD&FP): Load SIMD&FP register (PC-relative literal).

LDR (register, SIMD&FP): Load SIMD&FP register (register offset).

LDTNP (SIMD&FP): Load unprivileged pair of SIMD&FP registers, with non-temporal hint.

LDTP (SIMD&FP): Load unprivileged pair of SIMD&FP registers.

LDUR (SIMD&FP): Load SIMD&FP register (unscaled offset).

LUTI2: Lookup table read with 2-bit indices.

LUTI4: Lookup table read with 4-bit indices.

MLA (by element): Multiply-add to accumulator (vector, by element).

MLA (vector): Multiply-add to accumulator (vector).

MLS (by element): Multiply-subtract from accumulator (vector, by element).

MLS (vector): Multiply-subtract from accumulator (vector).

MOV (element): Move vector element to another vector element: an alias of INS (element).

MOV (from general): Move general-purpose register to a vector element: an alias of INS (general).

MOV (scalar): Move vector element to scalar: an alias of DUP (element).

MOV (to general): Move vector element to general-purpose register: an alias of UMOV.

MOV (vector): Move vector: an alias of ORR (vector, register).

MOVI: Move immediate (vector).

MUL (by element): Multiply (vector, by element).

MUL (vector): Multiply (vector).

MVN: Bitwise NOT (vector): an alias of NOT.

MVNI: Move inverted immediate (vector).

NEG (vector): Negate (vector).

NOT: Bitwise NOT (vector).

ORN (vector): Bitwise inclusive OR NOT (vector).

ORR (vector, immediate): Bitwise inclusive OR (vector, immediate).

ORR (vector, register): Bitwise inclusive OR (vector, register).

PMUL: Polynomial multiply.

PMULL, PMULL2: Polynomial multiply long.

RADDHN, RADDHN2: Rounding add returning high narrow.

RAX1: Rotate and exclusive-OR.

RBIT (vector): Reverse bit order (vector).

REV16 (vector): Reverse elements in 16-bit halfwords (vector).

REV32 (vector): Reverse elements in 32-bit words (vector).

REV64: Reverse elements in 64-bit doublewords (vector).

RSHRN, RSHRN2: Rounding shift right narrow (immediate).

RSUBHN, RSUBHN2: Rounding subtract returning high narrow.

SABA: Signed absolute difference and accumulate.

SABAL, SABAL2: Signed absolute difference and accumulate long.

SABD: Signed absolute difference.

SABDL, SABDL2: Signed absolute difference long.

SADALP: Signed add and accumulate long pairwise.

SADDL, SADDL2: Signed add long (vector).

SADDLP: Signed add long pairwise.

SADDLV: Signed add long across vector.

SADDW, SADDW2: Signed add wide.

SCVTF (scalar SIMD&FP): Signed integer convert to floating-point (scalar SIMD&FP).

SCVTF (scalar, fixed-point): Signed fixed-point convert to floating-point (scalar).

SCVTF (scalar, integer): Signed integer convert to floating-point (scalar).

SCVTF (vector, fixed-point): Signed fixed-point convert to floating-point (vector).

SCVTF (vector, integer): Signed integer convert to floating-point (vector).

SDOT (by element): Dot product signed arithmetic (vector, by element).

SDOT (vector): Dot product signed arithmetic (vector).

SHA1C: SHA1 hash update (choose).

SHA1H: SHA1 fixed rotate.

SHA1M: SHA1 hash update (majority).

SHA1P: SHA1 hash update (parity).

SHA1SU0: SHA1 schedule update 0.

SHA1SU1: SHA1 schedule update 1.

SHA256H: SHA256 hash update (part 1).

SHA256H2: SHA256 hash update (part 2).

SHA256SU0: SHA256 schedule update 0.

SHA256SU1: SHA256 schedule update 1.

SHA512H: SHA512 hash update part 1.

SHA512H2: SHA512 hash update part 2.

SHA512SU0: SHA512 schedule update 0.

SHA512SU1: SHA512 schedule update 1.

SHADD: Signed halving add.

SHL: Shift left (immediate).

SHLL, SHLL2: Shift left long (by element size).

SHRN, SHRN2: Shift right narrow (immediate).

SHSUB: Signed halving subtract.

SLI: Shift left and insert (immediate).

SM3PARTW1: SM3PARTW1.

SM3PARTW2: SM3PARTW2.

SM3SS1: SM3SS1.

SM3TT1A: SM3TT1A.

SM3TT1B: SM3TT1B.

SM3TT2A: SM3TT2A.

SM3TT2B: SM3TT2B.

SM4E: SM4 encode.

SM4EKEY: SM4 key.

SMAX: Signed maximum (vector).

SMAXP: Signed maximum pairwise.

SMAV: Signed maximum across vector.

SMIN: Signed minimum (vector).

SMINP: Signed minimum pairwise.

SMINV: Signed minimum across vector.

SMLAL, SMLAL2 (by element): Signed multiply-add long (vector, by element).

SMLAL, SMLAL2 (vector): Signed multiply-add long (vector).

SMLSL, SMLSL2 (by element): Signed multiply-subtract long (vector, by element).

SMLSL, SMLSL2 (vector): Signed multiply-subtract long (vector).

SMMLA (vector): Signed 8-bit integer matrix multiply-accumulate to 32-bit integer (vector).

SMOV: Signed move vector element to general-purpose register.

SMULL, SMULL2 (by element): Signed multiply long (vector, by element).

SMULL, SMULL2 (vector): Signed multiply long (vector).

SQABS: Signed saturating absolute value.

SQADD: Signed saturating add.

SQDMLAL, SQDMLAL2 (by element): Signed saturating doubling multiply-add long (by element).

SQDMLAL, SQDMLAL2 (vector): Signed saturating doubling multiply-add long.

SQDMLSL, SQDMLSL2 (by element): Signed saturating doubling multiply-subtract long (by element).

SQDMLSL, SQDMLSL2 (vector): Signed saturating doubling multiply-subtract long.

SQDMULH (by element): Signed saturating doubling multiply returning high half (by element).

SQDMULH (vector): Signed saturating doubling multiply returning high half.

SQDMULL, SQDMULL2 (by element): Signed saturating doubling multiply long (by element).

SQDMULL, SQDMULL2 (vector): Signed saturating doubling multiply long.

SQNEG: Signed saturating negate.

SQRDMLAH (by element): Signed saturating rounding doubling multiply accumulate returning high half (by element).

SQRDMLAH (vector): Signed saturating rounding doubling multiply accumulate returning high half (vector).

SQRDMLSH (by element): Signed saturating rounding doubling multiply subtract returning high half (by element).

SQRDMLSH (vector): Signed saturating rounding doubling multiply subtract returning high half (vector).

SQRDMULH (by element): Signed saturating rounding doubling multiply returning high half (by element).

SQRDMULH (vector): Signed saturating rounding doubling multiply returning high half.

SQRSHL: Signed saturating rounding shift left (register).

SQRSHRN, SQRSHRN2: Signed saturating rounded shift right narrow (immediate).

SQRSHRUN, SQRSHRUN2: Signed saturating rounded shift right unsigned narrow (immediate).

QSHL (immediate): Signed saturating shift left (immediate).

QSHL (register): Signed saturating shift left (register).

QSHLU: Signed saturating shift left unsigned (immediate).

QSHRN, QSHRN2: Signed saturating shift right narrow (immediate).

QSHRUN, QSHRUN2: Signed saturating shift right unsigned narrow (immediate).

SQSUB: Signed saturating subtract.

SQXTN, SQXTN2: Signed saturating extract narrow.

SQXTUN, SQXTUN2: Signed saturating extract unsigned narrow.

SRHADD: Signed rounding halving add.

SRI: Shift right and insert (immediate).

SRSHL: Signed rounding shift left (register).

SRSHR: Signed rounding shift right (immediate).

SRSRA: Signed rounding shift right and accumulate (immediate).

SSHL: Signed shift left (register).

SSHLL, SSHLL2: Signed shift left long (immediate).

SSHR: Signed shift right (immediate).

SSRA: Signed shift right and accumulate (immediate).

SSUBL, SSUBL2: Signed subtract long.

SSUBW, SSUBW2: Signed subtract wide.

ST1 (multiple structures): Store multiple single-element structures from one, two, three, or four registers.

ST1 (single structure): Store a single-element structure from one lane of one register.

ST2 (multiple structures): Store multiple 2-element structures from two registers.

ST2 (single structure): Store single 2-element structure from one lane of two registers.

ST3 (multiple structures): Store multiple 3-element structures from three registers.

ST3 (single structure): Store single 3-element structure from one lane of three registers.

ST4 (multiple structures): Store multiple 4-element structures from four registers.

ST4 (single structure): Store single 4-element structure from one lane of four registers.

STBFADD, STBFADDL: Atomic BFloat16 floating-point add, without return.

STBFMAX, STBFMAXL: Atomic BFloat16 floating-point maximum, without return.

STBFMAXNM, STBFMAXNML: Atomic BFloat16 floating-point maximum number, without return.

STBFMIN, STBFMINL: Atomic BFloat16 floating-point minimum, without return.

STBFMINNM, STBFMINNML: Atomic BFloat16 floating-point minimum number, without return.

STFADD, STFADDL: Atomic floating-point add, without return.

STFMAX, STFMAXL: Atomic floating-point maximum, without return.

STFMAXNM, STFMAXNML: Atomic floating-point maximum number, without return.

STFMIN, STFMINL: Atomic floating-point minimum, without return.

STFMINNM, STFMINNML: Atomic floating-point minimum number, without return.

STL1 (SIMD&FP): Store-release a single-element structure from one lane of one register.

STLUR (SIMD&FP): Store-release SIMD&FP register (unscaled offset).

STNP (SIMD&FP): Store pair of SIMD&FP registers, with non-temporal hint.

STP (SIMD&FP): Store pair of SIMD&FP registers.

STR (immediate, SIMD&FP): Store SIMD&FP register (immediate offset).

STR (register, SIMD&FP): Store SIMD&FP register (register offset).

STTNP (SIMD&FP): Store unprivileged pair of SIMD&FP registers, with non-temporal hint.

STTP (SIMD&FP): Store unprivileged pair of SIMD&FP registers.

STUR (SIMD&FP): Store SIMD&FP register (unscaled offset).

SUB (vector): Subtract (vector).

SUBHN, SUBHN2: Subtract returning high narrow.

SUDOT (by element): Dot product with signed and unsigned integers (vector, by element).

SUQADD: Signed saturating accumulate of unsigned value.

SXTL, SXTL2: Signed extend long: an alias of SSHLL, SSHLL2.

TBL: Table vector lookup.

TBX: Table vector lookup extension.

TRN1: Transpose vectors (primary).

TRN2: Transpose vectors (secondary).

UABA: Unsigned absolute difference and accumulate.

UABAL, UABAL2: Unsigned absolute difference and accumulate long.

UABD: Unsigned absolute difference (vector).

UABDL, UABDL2: Unsigned absolute difference long.

UADALP: Unsigned add and accumulate long pairwise.

UADDL, UADDL2: Unsigned add long (vector).

UADDLP: Unsigned add long pairwise.

UADDLV: Unsigned sum long across vector.

UADDW, UADDW2: Unsigned add wide.

UCVTF (scalar SIMD&FP): Unsigned integer convert to floating-point (scalar SIMD&FP).

UCVTF (scalar, fixed-point): Unsigned fixed-point convert to floating-point (scalar).

UCVTF (scalar, integer): Unsigned integer convert to floating-point (scalar).

UCVTF (vector, fixed-point): Unsigned fixed-point convert to floating-point (vector).

UCVTF (vector, integer): Unsigned integer convert to floating-point (vector).

UDOT (by element): Dot product unsigned arithmetic (vector, by element).

UDOT (vector): Dot product unsigned arithmetic (vector).

UHADD: Unsigned halving add.

UHSUB: Unsigned halving subtract.

UMAX: Unsigned maximum (vector).

UMAXP: Unsigned maximum pairwise.

UMAXV: Unsigned maximum across vector.

UMIN: Unsigned minimum (vector).

UMINP: Unsigned minimum pairwise.

UMINV: Unsigned minimum across vector.

UMLAL, UMLAL2 (by element): Unsigned multiply-add long (vector, by element).

UMLAL, UMLAL2 (vector): Unsigned multiply-add long (vector).

UMLSL, UMLSL2 (by element): Unsigned multiply-subtract long (vector, by element).

UMLSL, UMLSL2 (vector): Unsigned multiply-subtract long (vector).

UMMLA (vector): Unsigned 8-bit integer matrix multiply-accumulate to 32-bit integer (vector).

UMOV: Unsigned move vector element to general-purpose register.

UMULL, UMULL2 (by element): Unsigned multiply long (vector, by element).

UMULL, UMULL2 (vector): Unsigned multiply long (vector).

UQADD: Unsigned saturating add.

UQRSHL: Unsigned saturating rounding shift left (register).

UQRSHRN, UQRSHRN2: Unsigned saturating rounded shift right narrow (immediate).

UQSHL (immediate): Unsigned saturating shift left (immediate).

UQSHL (register): Unsigned saturating shift left (register).

UQSHRN, UQSHRN2: Unsigned saturating shift right narrow (immediate).

UQSUB: Unsigned saturating subtract.

UQXTN, UQXTN2: Unsigned saturating extract narrow.

URECPE: Unsigned reciprocal estimate.

URHADD: Unsigned rounding halving add.

URSHL: Unsigned rounding shift left (register).

URSHR: Unsigned rounding shift right (immediate).

URSQRTE: Unsigned reciprocal square root estimate.

URSRA: Unsigned rounding shift right and accumulate (immediate).

USDOT (by element): Dot product with unsigned and signed integers (vector, by element).

USDOT (vector): Dot product with unsigned and signed integers (vector).

USHL: Unsigned shift left (register).

USHLL, USHLL2: Unsigned shift left long (immediate).

USHR: Unsigned shift right (immediate).

USMMLA (vector): Unsigned and signed 8-bit integer matrix multiply-accumulate to 32-bit integer (vector).

USQADD: Unsigned saturating accumulate of signed value.

USRA: Unsigned shift right and accumulate (immediate).

USUBL, USUBL2: Unsigned subtract long.

USUBW, USUBW2: Unsigned subtract wide.

UXTL, UXTL2: Unsigned extend long: an alias of USHLL, USHLL2.

UZP1: Unzip vectors (primary).

UZP2: Unzip vectors (secondary).

XAR: Exclusive-OR and rotate.

XTN, XTN2: Extract narrow.

ZIP1: Zip vectors (primary).

ZIP2: Zip vectors (secondary).

2025-12_rel2025-09_diff_tag2025-12-102025-11-28 16:4129:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)

htmldiff from-

(new)

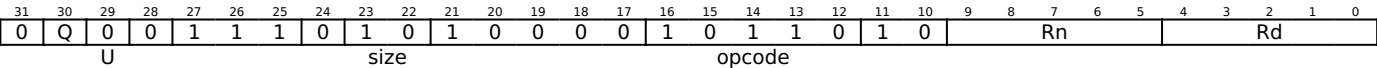
BFCVTN, BFCVTN2

Single-precision convert to BFloat16 (vector)

This instruction reads each single-precision element in the SIMD&FP source vector, converts each value to BFloat16 format, and writes the results in the lower or upper half of the SIMD&FP destination vector. The result elements are half the width of the source elements.

The BFCVTN instruction writes the half-width results to the lower half of the 128-bit SIMD&FP destination vector and clears the upper half to zero. The BFCVTN2 instruction writes the results to the upper half of the 128-bit SIMD&FP destination vector without affecting the lower other half bits in the register.

Vector single-precision to BFloat16 (FEAT_BF16)



Encoding

BFCVTN{2} <Vd>.<Ta>, <Vn>.4S

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_BF16) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let part : integer = UInt(Q);
let elements : integer = 64 DIV 16;
```

Assembler Symbols

- 2

Is the second and upper half specifier. If present it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]
- <Vd>

Is the name of the SIMD&FP destination register, encoded in the "Rd" field.
- <Ta>

Is an arrangement specifier, encoded in “Q”:

Q	<Ta>
0	4H
1	8H
- <Vn>

Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(128) = V{}(n);
var result : bits(64);

for e = 0 to elements-1 do
    result[e*16] = FPConvertBF(operand[e*32], FPCR());
end;

Vpart{64}(d, part) = result;
```


BFMMLA (widening)

BFloat16 matrix multiply-accumulate to single-precision

If FEAT_EBF16 is not implemented or FPCR.EBF is 0, this instruction:

- MultipliesPerforms two unfused sums of products within each two pairs of adjacent BFloat16 elements while multiplying the 2×4 2×4 matrix of BFloat16 values held in the first source vector by with the 4×2 4×2 matrix of BFloat16 values in the second source vector. The intermediate single-precision resultsproducts are calculatedrounded as before a sumthey of are two summed unfused and sum-of-products using two pairs of adjacent BFloat16 values from each of the sourceintermediate vectors, sum and is rounded before accumulation accumulation into the 2×2 2×2 matrix of single-precision resultsmatrix in the destination vector. This is equivalent to accumulating two 2-way unfused dot products per destination element.
- Uses the non-IEEE 754 Round-to-Odd rounding mode, which forces bit 0 of an inexact result to 1, and rounds an overflow to an appropriately signed Infinity.
- Flushes denormalized inputs and results to zero, as if FPCR.{FZ, FIZ} is {1, 1}.
- Honors FPCR.AH when generating a default NaN result, otherwise disables alternative floating point behaviors, as if FPCR.AH is 0.

If FEAT_EBF16 is implemented and FPCR.EBF is 1, then this instruction:

- MultipliesPerforms two fused sums of products within each two pairs of adjacent BFloat16 elements while multiplying the 2×4 2×4 matrix of BFloat16 values held in the first source vector by with the 4×2 4×2 matrix of BFloat16 values in the second source vector. The intermediate single-precision resultsproducts are calculatednot as rounded a before sum of two fused sum-of-products they using are two summed, pairs but of the adjacentintermediate BFloat16sum values is from each of the source vectors, and rounded before accumulation into into the 2×2 2×2 matrix of single-precision resultsmatrix in the destination vector. This is equivalent to accumulating two 2-way fused dot products per destination element.
- Follows all other floating-point behaviors that apply to single-precision arithmetic, as governed by FPCR.RMode, FPCR.FZ, FPCR.AH, and FPCR.FIZ.

Irrespective of FEAT_EBF16 and FPCR.EBF, this instruction:

- Does not modify the cumulative FPSR exception bits (IDC, IXC, UFC, OFC, DZC, and IOC).
- Disables trapped floating-point exceptions, as if the FPCR trap enable bits (IDE, IXE, UFE, OFE, DZE, and IOE) are all zero.
- Generates only the default NaN, as if FPCR.DN is 1.

ID_AA64ISAR1_EL1.BF16 indicates whether this instruction is supported.

Advanced SIMD

(FEAT_BF16)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	1	1	1	0	0	1	0	Rm				1	1	1	0	1	1	Rn				Rd						
Q		U		size								opcode																			

Encoding

BFMMLA <Vd>.4S, <Vn>.8H, <Vm>.8H

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_BF16) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP third source and destination register, encoded in the "Rd" field.
<Vn>	Is the name of the first SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let op1 : bits(128) = V{}(n);
let op2 : bits(128) = V{}(m);
let acc : bits(128) = V{}(d);

V{128}(d) = BFMatMulAddH(acc, op1, op2, FPCR());
```

Operational information

Arm expects that the BFMMLA instruction will deliver a peak BFloat16 multiply throughput that is at least as high as can be achieved using two BFDOT (vector) instructions, with a goal that it should have significantly higher throughput.

(old)	htmldiff from-	(new)
-------	----------------	-------

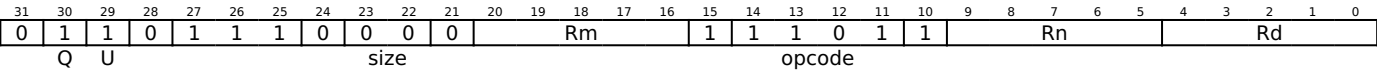
FMMLA (widening, 8-bit floating-point to half-precision)

8-bit floating-point matrix multiply-accumulate to half-precision

This instruction multipliesperforms the fused sum-of-products within each four adjacent 8-bit elements while multiplying the 2×4 matrix of 8-bit floating-point values values held in each 64-bit segment of the first source vector by the 4×2 matrix of 8-bit floating-point values in the corresponding segment of the second source vector. The intermediate half-precision results, calculated as a fused sum-of-products usingare four adjacent 8-bit floating-point values from each of the source vectors, are scaled by $2^{-\text{UInt}(\text{FPMR.LSCALE}[3:0])}$ before being destructively added without intermediate rounding to the 2x2 half-precision matrix in the corresponding segment of the destination vector. This is equivalent to accumulating 4-way dot product per destination element.

The 8-bit floating-point encoding format for the elements of the first source vector is selected by FPMR.F8S1. The 8-bit floating-point encoding format for the elements of the second source vector is selected by FPMR.F8S2.

Advanced SIMD (FEAT_F8F16MM)



Encoding

FMMLA <Vd>.8H, <Vn>.16B, <Vm>.16B

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_F8F16MM) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let d : integer = UInt(Rd);
```

Assembler Symbols

- <Vd>

Is the name of the SIMD&FP third source and destination register, encoded in the "Rd" field.
- <Vn>

Is the name of the first SIMD&FP source register, encoded in the "Rn" field.
- <Vm>

Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
CheckFPMREnabled();
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(128) = V{}(n);
let operand2 : bits(128) = V{}(m);
let operand3 : bits(128) = V{}(d);

var result : bits(128);
var op1, op2, acc : bits(64);

for s = 0 to 1 do
  op1 = operand1[s*:64];
  op2 = operand2[s*:64];
  acc = operand3[s*:64];
  result[s*:64] = FP8MatMulAddFP{64}(acc, op1, op2, 4, FPCR(), FPMR());
end;

V{128}(d) = result;
```

2025-12-re2025-09-diff-tag2025-12-102025-11-2816:4129:4505

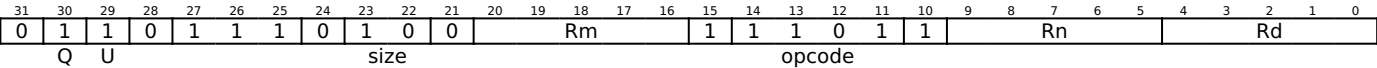
Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMMLA (widening, 8-bit floating-point to single-precision)

8-bit floating-point matrix multiply-accumulate to single-precision

This instruction multipliesperforms the fused sum-of-products within each eight adjacent 8-bit elements while multiplying the 2×8 matrix of 8-bit floating-point-values values held in the first source vector by the 8×2 matrix of 8-bit-floating-point floating-point values in the second source vector. The intermediate single-precision results, calculated as a fused sum-of-products using eight adjacent 8-bit floating-point values from each of the source vectors, are scaled by $2^{-\text{UInt}(\text{FPMR.LSCALE})}$ before being destructively added without intermediate rounding to the 2x2 single-precision matrix in the destination vector. This is equivalent to accumulating 8-way dot product per destination element. The 8-bit floating-point encoding format for the elements of the first source vector is selected by FPMR.F8S1. The 8-bit floating-point encoding format for the elements of the second source vector is selected by FPMR.F8S2.

Advanced SIMD (FEAT_F8F32MM)



Encoding

FMMLA <Vd>.4S, <Vn>.16B, <Vm>.16B

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_F8F32MM) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let d : integer = UInt(Rd);
```

Assembler Symbols

- <Vd>
- Is the name of the SIMD&FP third source and destination register, encoded in the "Rd" field.
- <Vn>
- Is the name of the first SIMD&FP source register, encoded in the "Rn" field.
- <Vm>
- Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
CheckFPMREnabled();
AArch64_CheckFPAdvSIMDEnabled();
let op1 : bits(128) = V{}(n);
let op2 : bits(128) = V{}(m);
let acc : bits(128) = V{}(d);

V{128}(d) = FP8MatMulAddFP{128}(acc, op1, op2, 8, FPCR(), FPMR());
```

2025-12_rel2025-09_diff_tag 2025-12-102025-11-28 16:4129:4505

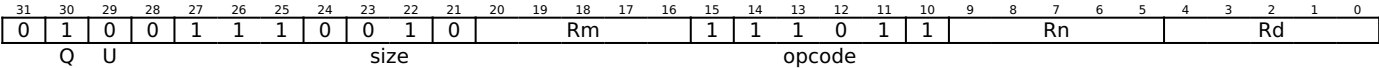
Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMMLA (widening, half-precision to single-precision)

Half-precision matrix multiply-accumulate to single-precision

This instruction multipliesperforms two fused sums-of-products within each two pairs of adjacent half-precision elements while multiplying the 2x42x4 matrix of half-precision values held in the first source vector by the 4x24x2 matrix of half-precision values in the second source vector. The intermediate single-precision resultssums-of-products are calculatedrounded as a sum of two fused sum-of-products usingbefore twothey pairsare ofsummed, adjacentand half-precisiontheir valuesintermediate fromsum eachis of the source vectors, and rounded before accumulation into the 2x22x2 matrix of single-precision resultsmatrix in the destination vector. This is equivalent equivalent to performing a 4-way dot product per destination element.

Advanced SIMD
(FEAT_F16F32MM)



Encoding

FMMLA <Vd>.4S, <Vn>.8H, <Vm>.8H

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_F16F32MM) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
```

Assembler Symbols

- <Vd>Is the name of the SIMD&FP third source and destination register, encoded in the "Rd" field.
- <Vn>Is the name of the first SIMD&FP source register, encoded in the "Rn" field.
- <Vm>Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let op1 : bits(128) = V{}(n);
let op2 : bits(128) = V{}(m);
let acc : bits(128) = V{}(d);

V{128}(d) = FPMatMulAddH{128}(acc, op1, op2, FPCR());
```

2025-12_rel2025-09_diff_tag 2025-12-102025-11-28 16:4129:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

A64 -- SVE Instructions (alphabetic order)

ABS: Absolute value (predicated).

ADCLB: Add with carry long (bottom).

ADCLT: Add with carry long (top).

[ADD \(immediate\)](#): Add immediate (unpredicated).

ADD (vectors, predicated): Add (predicated).

ADD (vectors, unpredicated): Add (unpredicated).

ADDHNB: Add narrow high part (bottom).

ADDHNT: Add narrow high part (top).

ADDP: Add pairwise.

ADDPL: Add multiple of predicate register size to scalar register.

ADDPT (predicated): Add checked pointer vectors (predicated).

ADDPT (unpredicated): Add checked pointer vectors (unpredicated).

ADDQP: Add pairwise within quadword vector segments (unpredicated).

ADDQV: Add reduction of quadword vector segments.

ADDSUBP: Add subtract pairwise.

ADDVL: Add multiple of vector register size to scalar register.

ADR: Calculate vector address.

AESD (indexed): Multi-vector AES single round decryption.

AESD (vectors): AES single round decryption.

AESDIMC: Multi-vector AES single round decryption and inverse mix columns.

AESE (indexed): Multi-vector AES single round encryption.

AESE (vectors): AES single round encryption.

AESEMC: Multi-vector AES single round encryption and mix columns.

AESIMC: AES inverse mix columns.

AESMC: AES mix columns.

AND (immediate): Bitwise AND with immediate (unpredicated).

AND (predicates): Bitwise AND predicates.

AND (vectors, predicated): Bitwise AND (predicated).

AND (vectors, unpredicated): Bitwise AND (unpredicated).

ANDQV: Bitwise AND reduction of quadword vector segments.

ANDS: Bitwise AND predicates, setting the condition flags.

ANDV: Bitwise AND reduction to scalar.

ASR (immediate, predicated): Arithmetic shift right by immediate (predicated).

ASR (immediate, unpredicated): Arithmetic shift right by immediate (unpredicated).

ASR (vectors): Arithmetic shift right by vector (predicated).

ASR (wide elements, predicated): Arithmetic shift right by 64-bit wide elements (predicated).

ASR (wide elements, unpredicated): Arithmetic shift right by 64-bit wide elements (unpredicated).

ASRD: Arithmetic shift right for divide by immediate (predicated).

ASRR: Reversed arithmetic shift right by vector (predicated).

BCAX: Bitwise clear and exclusive-OR.

BDEP: Scatter lower bits into positions selected by bitmask.

BEXT: Gather lower bits from positions selected by bitmask.

BF1CVT, BF2CVT: 8-bit floating-point convert to BFloat16.

BF1CVTLT, BF2CVTLT: 8-bit floating-point convert to BFloat16 (top).

BFADD (predicated): BFloat16 add (predicated).

BFADD (unpredicated): BFloat16 add (unpredicated).

BFCLAMP: BFloat16 clamp to minimum/maximum number.

BFCVT: Single-precision convert to BFloat16 (predicated).

BFCVTN: BFloat16 convert to interleaved 8-bit floating-point.

BFCVTNT: Single-precision convert to BFloat16 (top, predicated).

BFDOT (indexed): BFloat16 dot product by indexed element to single-precision.

BFDOT (vectors): BFloat16 dot product to single-precision.

BFMAX: BFloat16 maximum (predicated).

BFMAXNM: BFloat16 maximum number (predicated).

BFMIN: BFloat16 minimum (predicated).

BFMINNM: BFloat16 minimum number (predicated).

BFMLA (indexed): BFloat16 fused multiply-add by indexed element.

BFMLA (vectors): BFloat16 fused multiply-add.

BFMLALB (indexed): BFloat16 multiply-add by indexed element to single-precision (bottom).

BFMLALB (vectors): BFloat16 multiply-add to single-precision (bottom).

BFMLALT (indexed): BFloat16 multiply-add by indexed element to single-precision (top).

BFMLALT (vectors): BFloat16 multiply-add to single-precision (top).

BFMLS (indexed): BFloat16 fused multiply-subtract by indexed element.

BFMLS (vectors): BFloat16 fused multiply-subtract.

BFMLSLB (indexed): BFloat16 multiply-subtract by indexed element from single-precision (bottom).

BFMLSLB (vectors): BFloat16 multiply-subtract from single-precision (bottom).

BFMLSLT (indexed): BFloat16 multiply-subtract by indexed element from single-precision (top).

BFMLSLT (vectors): BFloat16 multiply-subtract from single-precision (top).

BFMMLA (non-widening): BFloat16 matrix multiply-accumulate.

[BFMMLA \(widening\)](#): BFloat16 matrix multiply-accumulate to single-precision.

BFMUL (indexed): BFloat16 multiply by indexed element.

BFMUL (vectors, predicated): BFloat16 multiply (predicated).

BFMUL (vectors, unpredicated): BFloat16 multiply (unpredicated).

BFSCALE: BFloat16 adjust exponent (predicated).

BFSUB (predicated): BFloat16 subtract (predicated).

BFSUB (unpredicated): BFloat16 subtract (unpredicated).

BGRP: Group bits to right or left as selected by bitmask.

BIC (immediate): Bitwise clear bits using immediate (unpredicated): an alias of AND (immediate).

BIC (predicates): Bitwise clear predicates.

BIC (vectors, predicated): Bitwise clear (predicated).

BIC (vectors, unpredicated): Bitwise clear (unpredicated).

BICS: Bitwise clear predicates, setting the condition flags.

BRKA: Break after first true condition.

BRKAS: Break after first true condition, setting the condition flags.

BRKB: Break before first true condition.

BRKBS: Break before first true condition, setting the condition flags.

BRKN: Propagate break to next partition.

BRKNS: Propagate break to next partition, setting the condition flags.

BRKPA: Break after first true condition, propagating from previous partition.

BRKPAS: Break after first true condition, propagating from previous partition and setting the condition flags.

BRKPB: Break before first true condition, propagating from previous partition.

BRKPBS: Break before first true condition, propagating from previous partition and setting the condition flags.

BSL: Bitwise select.

BSL1N: Bitwise select with first input inverted.

BSL2N: Bitwise select with second input inverted.

CADD: Complex integer add.

CDOT (indexed): Complex integer dot product by indexed element.

CDOT (vectors): Complex integer dot product.

CLASTA (scalar): Conditionally extract element after last to general-purpose register.

CLASTA (SIMD&FP scalar): Conditionally extract element after last to SIMD&FP scalar register.

CLASTA (vectors): Conditionally extract element after last to vector register.

CLASTB (scalar): Conditionally extract last element to general-purpose register.

CLASTB (SIMD&FP scalar): Conditionally extract last element to SIMD&FP scalar register.

CLASTB (vectors): Conditionally extract last element to vector register.

CLS: Count leading sign bits (predicated).

CLZ: Count leading zero bits (predicated).

CMLA (indexed): Complex integer multiply-add by indexed element.

CMLA (vectors): Complex integer multiply-add.

CMP<cc> (immediate): Compare vector to immediate.

CMP<cc> (vectors): Compare vectors.

CMP<cc> (wide elements): Compare vector to 64-bit wide elements.

CMPLE (vectors): Compare signed less than or equal to vector, setting the condition flags: an alias of CMP<cc> (vectors).

CMPLO (vectors): Compare unsigned lower than vector, setting the condition flags: an alias of CMP<cc> (vectors).

CMPLS (vectors): Compare unsigned lower than or same as vector, setting the condition flags: an alias of CMP<cc> (vectors).

CMPLT (vectors): Compare signed less than vector, setting the condition flags: an alias of CMP<cc> (vectors).

CNOT: Logically invert boolean condition (predicated).

CNT: Count non-zero bits (predicated).

CNTB, CNTD, CNTH, CNTW: Set scalar to multiple of predicate constraint element count.

CNTP (predicate as counter): Set scalar to count from predicate-as-counter.

CNTP (predicate): Set scalar to count of true predicate elements.

COMPACT: Copy Active vector elements to lower-numbered elements.

[CPY \(immediate, merging\)](#): Copy signed integer immediate to vector elements (merging).

[CPY \(immediate, zeroing\)](#): Copy signed integer immediate to vector elements (zeroing).

CPY (scalar): Copy general-purpose register to vector elements (predicated).

CPY (SIMD&FP scalar): Copy SIMD&FP scalar register to vector elements (predicated).

CTERMEQ, CTERMNE: Compare and terminate loop.

DECB, DECD, DECH, DECW (scalar): Decrement scalar by multiple of predicate constraint element count.

DECD, DECH, DECW (vector): Decrement vector by multiple of predicate constraint element count.

DECP (scalar): Decrement scalar by count of true predicate elements.

DECP (vector): Decrement vector by count of true predicate elements.

[DUP \(immediate\)](#): Broadcast signed immediate to vector elements (unpredicated).

DUP (indexed): Broadcast indexed element to vector (unpredicated).

DUP (scalar): Broadcast general-purpose register to vector elements (unpredicated).

DUPM: Broadcast logical bitmask immediate to vector (unpredicated).

DUPQ: Broadcast indexed element within each quadword vector segment (unpredicated).

EON: Bitwise exclusive-OR with inverted immediate (unpredicated): an alias of EOR (immediate).

EOR (immediate): Bitwise exclusive-OR with immediate (unpredicated).

EOR (predicates): Bitwise exclusive-OR predicates.

EOR (vectors, predicated): Bitwise exclusive-OR (predicated).

EOR (vectors, unpredicated): Bitwise exclusive-OR (unpredicated).

EOR3: Bitwise exclusive-OR between three vectors.

EORBT: Interleaving exclusive-OR (bottom, top).

EORQV: Bitwise exclusive-OR reduction of quadword vector segments.

EORS: Bitwise exclusive-OR predicates, setting the condition flags.

EORTB: Interleaving exclusive-OR (top, bottom).

EORV: Bitwise exclusive-OR reduction to scalar.

EXPAND: Copy lower-numbered vector elements to Active elements.

EXT: Extract vector from pair of vectors.

EXTQ: Extract vector segment from each pair of quadword vector segments.

F1CVT, F2CVT: 8-bit floating-point convert to half-precision.

F1CVTLT, F2CVTLT: 8-bit floating-point convert to half-precision (top).

FABD: Floating-point absolute difference (predicated).

FABS: Floating-point absolute value (predicated).

FAC<cc>: Floating-point absolute compare.

FACLE: Floating-point absolute compare less than or equal: an alias of FAC<cc>.

FACLT: Floating-point absolute compare less than: an alias of FAC<cc>.

FADD (immediate): Floating-point add immediate (predicated).

FADD (vectors, predicated): Floating-point add (predicated).

FADD (vectors, unpredicated): Floating-point add (unpredicated).

FADDA: Floating-point add strictly-ordered reduction, accumulating in scalar.

FADDP: Floating-point add pairwise.

FADDQV: Floating-point add recursive reduction of quadword vector segments.

FADDV: Floating-point add recursive reduction to scalar.

FAMAX: Floating-point absolute maximum (predicated).

FAMIN: Floating-point absolute minimum (predicated).

FCADD: Floating-point complex add (predicated).

FCLAMP: Floating-point clamp to minimum/maximum number.

FCM<cc> (vectors): Floating-point compare.

FCM<cc> (zero): Floating-point compare with zero.

FCMLA (indexed): Floating-point complex multiply-add by indexed element.

FCMLA (vectors): Floating-point complex multiply-add (predicated).

FCMLE (vectors): Floating-point compare less than or equal to vector: an alias of FCM<cc> (vectors).

FCMLT (vectors): Floating-point compare less than vector: an alias of FCM<cc> (vectors).

FCPY: Copy floating-point immediate to vector elements (predicated).

FCVT: Floating-point convert (predicated).

FCVTLT: Floating-point widening convert (top, predicated).

FCVTN: Half-precision convert to interleaved 8-bit floating-point.

FCVTNB: Single-precision convert to interleaved 8-bit floating-point (bottom).

FCVTNT (predicated): Floating-point narrowing convert (top, predicated).

FCVTNT (unpredicated): Single-precision convert to interleaved 8-bit floating-point (top).

FCVTX: Double-precision convert to single-precision, rounding to odd (predicated).

FCVTXNT: Double-precision convert to single-precision, rounding to odd (top, predicated).

FCVTZS: Floating-point convert to signed integer, rounding toward zero (predicated).

FCVTZSN: Floating-point narrowing convert to interleaved signed integer, rounding toward zero.

FCVTZU: Floating-point convert to unsigned integer, rounding toward zero (predicated).

FCVTZUN: Floating-point narrowing convert to interleaved unsigned integer, rounding toward zero.

FDIV: Floating-point divide (predicated).

FDIVR: Floating-point reversed divide (predicated).

FDOT (2-way, indexed, FP16 to FP32): Half-precision dot product by indexed element to single-precision.

FDOT (2-way, indexed, FP8 to FP16): 8-bit floating-point dot product by indexed element to half-precision.

FDOT (2-way, vectors, FP16 to FP32): Half-precision dot product to single-precision.

FDOT (2-way, vectors, FP8 to FP16): 8-bit floating-point dot product to half-precision.

FDOT (4-way, indexed): 8-bit floating-point dot product by indexed element to single-precision.

FDOT (4-way, vectors): 8-bit floating-point dot product to single-precision.

FDUP: Broadcast floating-point immediate to vector elements (unpredicated).

FEXPA: Floating-point exponential accelerator.

FIRSTP: Scalar index of first true predicate element (predicated).

FLOGB: Floating-point base 2 logarithm as integer.

FMAD: Floating-point fused multiply-add to multiplicand (predicated).

FMAX (immediate): Floating-point maximum with immediate (predicated).

FMAX (vectors): Floating-point maximum (predicated).

FMAXNM (immediate): Floating-point maximum number with immediate (predicated).

FMAXNM (vectors): Floating-point maximum number (predicated).

FMAXNMP: Floating-point maximum number pairwise.

FMAXNMQV: Floating-point maximum number recursive reduction of quadword vector segments.

FMAXNMV: Floating-point maximum number recursive reduction to scalar.

FMAXP: Floating-point maximum pairwise.

FMAXQV: Floating-point maximum reduction of quadword vector segments.

FMAXV: Floating-point maximum recursive reduction to scalar.

FMIN (immediate): Floating-point minimum with immediate (predicated).

FMIN (vectors): Floating-point minimum (predicated).

FMINNM (immediate): Floating-point minimum number with immediate (predicated).

FMINNM (vectors): Floating-point minimum number (predicated).

FMINNMP: Floating-point minimum number pairwise.

FMINNMQV: Floating-point minimum number recursive reduction of quadword vector segments.

FMINNMV: Floating-point minimum number recursive reduction to scalar.

FMINP: Floating-point minimum pairwise.

FMINQV: Floating-point minimum recursive reduction of quadword vector segments.

FMINV: Floating-point minimum recursive reduction to scalar.

FMLA (indexed): Floating-point fused multiply-add by indexed element.

FMLA (vectors): Floating-point fused multiply-add (predicated).

FMLALB (indexed, FP16 to FP32): Half-precision multiply-add by indexed element to single-precision (bottom).

FMLALB (indexed, FP8 to FP16): 8-bit floating-point multiply-add by indexed element to half-precision (bottom).

FMLALB (vectors, FP16 to FP32): Half-precision multiply-add to single-precision (bottom).

FMLALB (vectors, FP8 to FP16): 8-bit floating-point multiply-add to half-precision (bottom).

FMLALLBB (indexed): 8-bit floating-point multiply-add by indexed element to single-precision (bottom bottom).

FMLALLBB (vectors): 8-bit floating-point multiply-add to single-precision (bottom bottom).

FMLALLBT (indexed): 8-bit floating-point multiply-add by indexed element to single-precision (bottom top).

FMLALLBT (vectors): 8-bit floating-point multiply-add to single-precision (bottom top).

FMLALLTB (indexed): 8-bit floating-point multiply-add by indexed element to single-precision (top bottom).

FMLALLTB (vectors): 8-bit floating-point multiply-add to single-precision (top bottom).

FMLALLTT (indexed): 8-bit floating-point multiply-add by indexed element to single-precision (top top).

FMLALLTT (vectors): 8-bit floating-point multiply-add to single-precision (top top).

FMLALT (indexed, FP16 to FP32): Half-precision multiply-add by indexed element to single-precision (top).

FMLALT (indexed, FP8 to FP16): 8-bit floating-point multiply-add by indexed element to half-precision (top).

FMLALT (vectors, FP16 to FP32): Half-precision multiply-add to single-precision (top).

FMLALT (vectors, FP8 to FP16): 8-bit floating-point multiply-add to half-precision (top).

FMLS (indexed): Floating-point fused multiply-subtract by indexed element.

FMLS (vectors): Floating-point fused multiply-subtract (predicated).

FMLSLB (indexed): Half-precision multiply-subtract by indexed element from single-precision (bottom).

FMLSLB (vectors): Half-precision multiply-subtract from single-precision (bottom).

FMLSLT (indexed): Half-precision multiply-subtract by indexed element from single-precision (top).

FMLSLT (vectors): Half-precision multiply-subtract from single-precision (top).

FMMLA (non-widening): Floating-point matrix multiply-accumulate.

[FMMLA \(widening, FP16 to FP32\)](#): Half-precision matrix multiply-accumulate to single-precision.

[FMMLA \(widening, FP8 to FP16\)](#): 8-bit floating-point matrix multiply-accumulate to half-precision.

[FMMLA \(widening, FP8 to FP32\)](#): 8-bit floating-point matrix multiply-accumulate to single-precision.

FMOV (immediate, predicated): Move floating-point immediate to vector elements (predicated): an alias of FCPY.

FMOV (immediate, unpredicated): Move floating-point immediate to vector elements (unpredicated): an alias of FDUP.

FMOV (zero, predicated): Move floating-point +0.0 to vector elements (predicated): an alias of CPY (immediate, merging).

FMOV (zero, unpredicated): Move floating-point +0.0 to vector elements (unpredicated): an alias of DUP (immediate).

FMSB: Floating-point fused multiply-subtract to multiplicand (predicated).

FMUL (immediate): Floating-point multiply by immediate (predicated).

FMUL (indexed): Floating-point multiply by indexed element.

FMUL (vectors, predicated): Floating-point multiply (predicated).

FMUL (vectors, unpredicated): Floating-point multiply (unpredicated).

FMULX: Floating-point multiply extended (predicated).

FNEG: Floating-point negate (predicated).

FNMAAD: Floating-point negated fused multiply-add to multiplicand (predicated).

FNMLA: Floating-point negated fused multiply-add (predicated).

FNMLS: Floating-point negated fused multiply-subtract (predicated).

FNMSB: Floating-point negated fused multiply-subtract to multiplicand (predicated).

FRECPE: Floating-point reciprocal estimate (unpredicated).

FRECPS: Floating-point reciprocal step (unpredicated).

FRECPX: Floating-point reciprocal exponent (predicated).

FRINT32X: Floating-point round to 32-bit integer (predicated).

FRINT32Z: Floating-point round to 32-bit integer, rounding toward zero (predicated).

FRINT64X: Floating-point round to 64-bit integer (predicated).

FRINT64Z: Floating-point round to 64-bit integer, rounding toward zero (predicated).

FRINT<r>: Floating-point round to integral value (predicated).

FRSQRT: Floating-point reciprocal square root estimate (unpredicated).

FRSQRTS: Floating-point reciprocal square root step (unpredicated).

FSCALE: Floating-point adjust exponent (predicated).

FSQRT: Floating-point square root (predicated).

FSUB (immediate): Floating-point subtract immediate (predicated).

FSUB (vectors, predicated): Floating-point subtract (predicated).

FSUB (vectors, unpredicated): Floating-point subtract (unpredicated).

FSUBR (immediate): Floating-point reversed subtract from immediate (predicated).

FSUBR (vectors): Floating-point reversed subtract (predicated).

FTMAD: Floating-point trigonometric multiply-add coefficient.

FTSMUL: Floating-point trigonometric starting value.

FTSSEL: Floating-point trigonometric select coefficient.

HISTCNT: Count matching elements in vector.

HISTSEG: Count matching elements in vector segments.

INCB, INCD, INCH, INCW (scalar): Increment scalar by multiple of predicate constraint element count.

INCD, INCH, INCW (vector): Increment vector by multiple of predicate constraint element count.

INCP (scalar): Increment scalar by count of true predicate elements.

INCP (vector): Increment vector by count of true predicate elements.

INDEX (immediate, scalar): Create index starting from immediate and incremented by general-purpose register.

INDEX (immediates): Create index starting from and incremented by immediate.

INDEX (scalar, immediate): Create index starting from general-purpose register and incremented by immediate.

INDEX (scalars): Create index starting from and incremented by general-purpose register.

INSR (scalar): Insert general-purpose register in shifted vector.

INSR (SIMD&FP scalar): Insert SIMD&FP scalar register in shifted vector.

LASTA (scalar): Extract element after last to general-purpose register.

LASTA (SIMD&FP scalar): Extract element after last to SIMD&FP scalar register.

LASTB (scalar): Extract last element to general-purpose register.

LASTB (SIMD&FP scalar): Extract last element to SIMD&FP scalar register.

LASTP: Scalar index of last true predicate element (predicated).

LD1B (scalar plus immediate, consecutive registers): Contiguous load of bytes to multiple consecutive vectors (immediate index).

LD1B (scalar plus immediate, single register): Contiguous load unsigned bytes to vector (immediate index).

LD1B (scalar plus scalar, consecutive registers): Contiguous load of bytes to multiple consecutive vectors (scalar index).

LD1B (scalar plus scalar, single register): Contiguous load unsigned bytes to vector (scalar index).

LD1B (scalar plus vector): Gather load unsigned bytes to vector (vector index).

LD1B (vector plus immediate): Gather load unsigned bytes to vector (immediate index).

LD1D (scalar plus immediate, consecutive registers): Contiguous load of doublewords to multiple consecutive vectors (immediate index).

LD1D (scalar plus immediate, single register): Contiguous load unsigned doublewords to vector (immediate index).

LD1D (scalar plus scalar, consecutive registers): Contiguous load of doublewords to multiple consecutive vectors (scalar index).

LD1D (scalar plus scalar, single register): Contiguous load unsigned doublewords to vector (scalar index).

LD1D (scalar plus vector): Gather load doublewords to vector (vector index).

LD1D (vector plus immediate): Gather load doublewords to vector (immediate index).

LD1H (scalar plus immediate, consecutive registers): Contiguous load of halfwords to multiple consecutive vectors (immediate index).

LD1H (scalar plus immediate, single register): Contiguous load unsigned halfwords to vector (immediate index).

LD1H (scalar plus scalar, consecutive registers): Contiguous load of halfwords to multiple consecutive vectors (scalar index).

LD1H (scalar plus scalar, single register): Contiguous load unsigned halfwords to vector (scalar index).

LD1H (scalar plus vector): Gather load unsigned halfwords to vector (vector index).

LD1H (vector plus immediate): Gather load unsigned halfwords to vector (immediate index).

LD1Q: Gather load quadwords.

LD1RB: Load and broadcast unsigned byte to vector.

LD1RD: Load and broadcast doubleword to vector.

LD1RH: Load and broadcast unsigned halfword to vector.

LD1ROB (scalar plus immediate): Contiguous load and replicate thirty-two bytes (immediate index).

LD1ROB (scalar plus scalar): Contiguous load and replicate thirty-two bytes (scalar index).

LD1ROD (scalar plus immediate): Contiguous load and replicate four doublewords (immediate index).

LD1ROD (scalar plus scalar): Contiguous load and replicate four doublewords (scalar index).

LD1ROH (scalar plus immediate): Contiguous load and replicate sixteen halfwords (immediate index).

LD1ROH (scalar plus scalar): Contiguous load and replicate sixteen halfwords (scalar index).

LD1ROW (scalar plus immediate): Contiguous load and replicate eight words (immediate index).

LD1ROW (scalar plus scalar): Contiguous load and replicate eight words (scalar index).

LD1RQB (scalar plus immediate): Contiguous load and replicate sixteen bytes (immediate index).

LD1RQB (scalar plus scalar): Contiguous load and replicate sixteen bytes (scalar index).

LD1RQD (scalar plus immediate): Contiguous load and replicate two doublewords (immediate index).

LD1RQD (scalar plus scalar): Contiguous load and replicate two doublewords (scalar index).

LD1RQH (scalar plus immediate): Contiguous load and replicate eight halfwords (immediate index).

LD1RQH (scalar plus scalar): Contiguous load and replicate eight halfwords (scalar index).

LD1RQW (scalar plus immediate): Contiguous load and replicate four words (immediate index).

LD1RQW (scalar plus scalar): Contiguous load and replicate four words (scalar index).

LD1RSB: Load and broadcast signed byte to vector.

LD1RSH: Load and broadcast signed halfword to vector.

LD1RSW: Load and broadcast signed word to vector.

LD1RW: Load and broadcast unsigned word to vector.

LD1SB (scalar plus immediate): Contiguous load signed bytes to vector (immediate index).

LD1SB (scalar plus scalar): Contiguous load signed bytes to vector (scalar index).

LD1SB (scalar plus vector): Gather load signed bytes to vector (vector index).

LD1SB (vector plus immediate): Gather load signed bytes to vector (immediate index).

LD1SH (scalar plus immediate): Contiguous load signed halfwords to vector (immediate index).

LD1SH (scalar plus scalar): Contiguous load signed halfwords to vector (scalar index).

LD1SH (scalar plus vector): Gather load signed halfwords to vector (vector index).

LD1SH (vector plus immediate): Gather load signed halfwords to vector (immediate index).

LD1SW (scalar plus immediate): Contiguous load signed words to vector (immediate index).

LD1SW (scalar plus scalar): Contiguous load signed words to vector (scalar index).

LD1SW (scalar plus vector): Gather load signed words to vector (vector index).

LD1SW (vector plus immediate): Gather load signed words to vector (immediate index).

LD1W (scalar plus immediate, consecutive registers): Contiguous load of words to multiple consecutive vectors (immediate index).

LD1W (scalar plus immediate, single register): Contiguous load unsigned words to vector (immediate index).

LD1W (scalar plus scalar, consecutive registers): Contiguous load of words to multiple consecutive vectors (scalar index).

LD1W (scalar plus scalar, single register): Contiguous load unsigned words to vector (scalar index).

LD1W (scalar plus vector): Gather load unsigned words to vector (vector index).

LD1W (vector plus immediate): Gather load unsigned words to vector (immediate index).

LD2B (scalar plus immediate): Contiguous load two-byte structures to two vectors (immediate index).

LD2B (scalar plus scalar): Contiguous load two-byte structures to two vectors (scalar index).

LD2D (scalar plus immediate): Contiguous load two-doubleword structures to two vectors (immediate index).

LD2D (scalar plus scalar): Contiguous load two-doubleword structures to two vectors (scalar index).

LD2H (scalar plus immediate): Contiguous load two-halfword structures to two vectors (immediate index).

LD2H (scalar plus scalar): Contiguous load two-halfword structures to two vectors (scalar index).

LD2Q (scalar plus immediate): Contiguous load two-quadword structures to two vectors (immediate index).

LD2Q (scalar plus scalar): Contiguous load two-quadword structures to two vectors (scalar index).

LD2W (scalar plus immediate): Contiguous load two-word structures to two vectors (immediate index).

LD2W (scalar plus scalar): Contiguous load two-word structures to two vectors (scalar index).

LD3B (scalar plus immediate): Contiguous load three-byte structures to three vectors (immediate index).

LD3B (scalar plus scalar): Contiguous load three-byte structures to three vectors (scalar index).

LD3D (scalar plus immediate): Contiguous load three-doubleword structures to three vectors (immediate index).

LD3D (scalar plus scalar): Contiguous load three-doubleword structures to three vectors (scalar index).

LD3H (scalar plus immediate): Contiguous load three-halfword structures to three vectors (immediate index).

LD3H (scalar plus scalar): Contiguous load three-halfword structures to three vectors (scalar index).

LD3Q (scalar plus immediate): Contiguous load three-quadword structures to three vectors (immediate index).

LD3Q (scalar plus scalar): Contiguous load three-quadword structures to three vectors (scalar index).

LD3W (scalar plus immediate): Contiguous load three-word structures to three vectors (immediate index).

LD3W (scalar plus scalar): Contiguous load three-word structures to three vectors (scalar index).

LD4B (scalar plus immediate): Contiguous load four-byte structures to four vectors (immediate index).

LD4B (scalar plus scalar): Contiguous load four-byte structures to four vectors (scalar index).

LD4D (scalar plus immediate): Contiguous load four-doubleword structures to four vectors (immediate index).

LD4D (scalar plus scalar): Contiguous load four-doubleword structures to four vectors (scalar index).

LD4H (scalar plus immediate): Contiguous load four-halfword structures to four vectors (immediate index).

LD4H (scalar plus scalar): Contiguous load four-halfword structures to four vectors (scalar index).

LD4Q (scalar plus immediate): Contiguous load four-quadword structures to four vectors (immediate index).

LD4Q (scalar plus scalar): Contiguous load four-quadword structures to four vectors (scalar index).

LD4W (scalar plus immediate): Contiguous load four-word structures to four vectors (immediate index).

LD4W (scalar plus scalar): Contiguous load four-word structures to four vectors (scalar index).

LDFF1B (scalar plus scalar): Contiguous load first-fault unsigned bytes to vector (scalar index).

LDFF1B (scalar plus vector): Gather load first-fault unsigned bytes to vector (vector index).

LDFF1B (vector plus immediate): Gather load first-fault unsigned bytes to vector (immediate index).

LDFF1D (scalar plus scalar): Contiguous load first-fault doublewords to vector (scalar index).

LDFF1D (scalar plus vector): Gather load first-fault doublewords to vector (vector index).

LDFF1D (vector plus immediate): Gather load first-fault doublewords to vector (immediate index).

LDFF1H (scalar plus scalar): Contiguous load first-fault unsigned halfwords to vector (scalar index).

LDFF1H (scalar plus vector): Gather load first-fault unsigned halfwords to vector (vector index).

LDFF1H (vector plus immediate): Gather load first-fault unsigned halfwords to vector (immediate index).

LDFF1SB (scalar plus scalar): Contiguous load first-fault signed bytes to vector (scalar index).

LDFF1SB (scalar plus vector): Gather load first-fault signed bytes to vector (vector index).

LDFF1SB (vector plus immediate): Gather load first-fault signed bytes to vector (immediate index).

LDFF1SH (scalar plus scalar): Contiguous load first-fault signed halfwords to vector (scalar index).

LDFF1SH (scalar plus vector): Gather load first-fault signed halfwords to vector (vector index).

LDFF1SH (vector plus immediate): Gather load first-fault signed halfwords to vector (immediate index).

LDFF1SW (scalar plus scalar): Contiguous load first-fault signed words to vector (scalar index).

LDFF1SW (scalar plus vector): Gather load first-fault signed words to vector (vector index).

LDFF1SW (vector plus immediate): Gather load first-fault signed words to vector (immediate index).

LDFF1W (scalar plus scalar): Contiguous load first-fault unsigned words to vector (scalar index).

LDFF1W (scalar plus vector): Gather load first-fault unsigned words to vector (vector index).

LDFF1W (vector plus immediate): Gather load first-fault unsigned words to vector (immediate index).

LDNF1B: Contiguous load non-fault unsigned bytes to vector (immediate index).

LDNF1D: Contiguous load non-fault doublewords to vector (immediate index).

LDNF1H: Contiguous load non-fault unsigned halfwords to vector (immediate index).

LDNF1SB: Contiguous load non-fault signed bytes to vector (immediate index).

LDNF1SH: Contiguous load non-fault signed halfwords to vector (immediate index).

LDNF1SW: Contiguous load non-fault signed words to vector (immediate index).

LDNF1W: Contiguous load non-fault unsigned words to vector (immediate index).

LDNT1B (scalar plus immediate, consecutive registers): Contiguous load non-temporal of bytes to multiple consecutive vectors (immediate index).

LDNT1B (scalar plus immediate, single register): Contiguous load non-temporal bytes to vector (immediate index).

LDNT1B (scalar plus scalar, consecutive registers): Contiguous load non-temporal of bytes to multiple consecutive vectors (scalar index).

LDNT1B (scalar plus scalar, single register): Contiguous load non-temporal bytes to vector (scalar index).

LDNT1B (vector plus scalar): Gather load non-temporal unsigned bytes.

LDNT1D (scalar plus immediate, consecutive registers): Contiguous load non-temporal of doublewords to multiple consecutive vectors (immediate index).

LDNT1D (scalar plus immediate, single register): Contiguous load non-temporal doublewords to vector (immediate index).

LDNT1D (scalar plus scalar, consecutive registers): Contiguous load non-temporal of doublewords to multiple consecutive vectors (scalar index).

LDNT1D (scalar plus scalar, single register): Contiguous load non-temporal doublewords to vector (scalar index).

LDNT1D (vector plus scalar): Gather load non-temporal unsigned doublewords.

LDNT1H (scalar plus immediate, consecutive registers): Contiguous load non-temporal of halfwords to multiple consecutive vectors (immediate index).

LDNT1H (scalar plus immediate, single register): Contiguous load non-temporal halfwords to vector (immediate index).

LDNT1H (scalar plus scalar, consecutive registers): Contiguous load non-temporal of halfwords to multiple consecutive vectors (scalar index).

LDNT1H (scalar plus scalar, single register): Contiguous load non-temporal halfwords to vector (scalar index).

LDNT1H (vector plus scalar): Gather load non-temporal unsigned halfwords.

LDNT1SB: Gather load non-temporal signed bytes.

LDNT1SH: Gather load non-temporal signed halfwords.

LDNT1SW: Gather load non-temporal signed words.

LDNT1W (scalar plus immediate, consecutive registers): Contiguous load non-temporal of words to multiple consecutive vectors (immediate index).

LDNT1W (scalar plus immediate, single register): Contiguous load non-temporal words to vector (immediate index).

LDNT1W (scalar plus scalar, consecutive registers): Contiguous load non-temporal of words to multiple consecutive vectors (scalar index).

LDNT1W (scalar plus scalar, single register): Contiguous load non-temporal words to vector (scalar index).

LDNT1W (vector plus scalar): Gather load non-temporal unsigned words.

[LDR \(predicate\)](#): Load predicate register.

[LDR \(vector\)](#): Load vector register.

LSL (immediate, predicated): Logical shift left by immediate (predicated).

LSL (immediate, unpredicated): Logical shift left by immediate (unpredicated).

LSL (vectors): Logical shift left by vector (predicated).

LSL (wide elements, predicated): Logical shift left by 64-bit wide elements (predicated).

LSL (wide elements, unpredicated): Logical shift left by 64-bit wide elements (unpredicated).

LSLR: Reversed logical shift left by vector (predicated).

LSR (immediate, predicated): Logical shift right by immediate (predicated).

LSR (immediate, unpredicated): Logical shift right by immediate (unpredicated).

LSR (vectors): Logical shift right by vector (predicated).

LSR (wide elements, predicated): Logical shift right by 64-bit wide elements (predicated).

LSR (wide elements, unpredicated): Logical shift right by 64-bit wide elements (unpredicated).

LSRR: Reversed logical shift right by vector (predicated).

LUT12 (8-bit and 16-bit): Lookup table read with 2-bit indices (8-bit and 16-bit).

LUT14 (8-bit and 16-bit): Lookup table read with 4-bit indices (8-bit and 16-bit).

LUT16 (16-bit): Lookup table read with 6-bit indices (16-bit).

LUT16 (8-bit): Lookup table read with 6-bit indices (8-bit).

MAD: Multiply-add to multiplicand (predicated).

MADPT: Multiply-add checked pointer vectors to multiplicand.

MATCH: Detect any matching elements, setting the condition flags.

MLA (indexed): Multiply-add by indexed element.

MLA (vectors): Multiply-add (predicated).

MLAPT: Multiply-add checked pointer vectors.

MLS (indexed): Multiply-subtract by indexed element.

MLS (vectors): Multiply-subtract (predicated).

MOV: Move logical bitmask immediate to vector (unpredicated): an alias of DUPM.

MOV: Move predicate (unpredicated): an alias of ORR (predicates).

[MOV \(immediate, merging\)](#): Move signed integer immediate to vector elements (merging): an alias of CPY (immediate, merging).

[MOV \(immediate, unpredicated\)](#): Move signed integer immediate to vector elements (unpredicated): an alias of DUP (immediate).

[MOV \(immediate, zeroing\)](#): Move signed integer immediate to vector elements (zeroing): an alias of CPY (immediate, zeroing).

MOV (predicate, merging): Move predicates (merging): an alias of SEL (predicates).

MOV (predicate, zeroing): Move predicates (zeroing): an alias of AND (predicates).

MOV (scalar, predicated): Move general-purpose register to vector elements (predicated): an alias of CPY (scalar).

MOV (scalar, unpredicated): Move general-purpose register to vector elements (unpredicated): an alias of DUP (scalar).

MOV (SIMD&FP scalar, predicated): Move SIMD&FP scalar register to vector elements (predicated): an alias of CPY (SIMD&FP scalar).

MOV (SIMD&FP scalar, unpredicated): Move indexed element or SIMD&FP scalar to vector (unpredicated): an alias of DUP (indexed).

MOV (vector, predicated): Move vector elements (predicated): an alias of SEL (vectors).

MOV (vector, unpredicated): Move vector register (unpredicated): an alias of ORR (vectors, unpredicated).

MOVPRFX (predicated): Move prefix (predicated).

MOVPRFX (unpredicated): Move prefix (unpredicated).

MOVS (predicated): Move predicates (zeroing), setting the condition flags: an alias of ANDS.

MOVS (unpredicated): Move predicate (unpredicated), setting the condition flags: an alias of ORRS.

MSB: Multiply-subtract to multiplicand.

MUL (immediate): Multiply by immediate (unpredicated).

MUL (indexed): Multiply by indexed element.

MUL (vectors, predicated): Multiply (predicated).

MUL (vectors, unpredicated): Multiply (unpredicated).

NAND: Bitwise NAND predicates.

NANDS: Bitwise NAND predicates, setting the condition flags.

NBSL: Bitwise inverted select.

NEG: Negate (predicated).

NMATCH: Detect no matching elements, setting the condition flags.

NOR: Bitwise NOR predicates.

NORS: Bitwise NOR predicates, setting the condition flags.

NOT (predicate): Bitwise invert predicate: an alias of EOR (predicates).

NOT (vector): Bitwise invert (predicated).

NOTS: Bitwise invert predicate, setting the condition flags: an alias of EORS.

ORN (immediate): Bitwise inclusive OR with inverted immediate (unpredicated): an alias of ORR (immediate).

ORN (predicates): Bitwise inclusive OR inverted predicate.

ORNS: Bitwise inclusive OR inverted predicate, setting the condition flags.

ORQV: Bitwise inclusive OR reduction of quadword vector segments.

ORR (immediate): Bitwise inclusive OR with immediate (unpredicated).

ORR (predicates): Bitwise inclusive OR predicates.

ORR (vectors, predicated): Bitwise inclusive OR (predicated).

ORR (vectors, unpredicated): Bitwise inclusive OR (unpredicated).

ORRS: Bitwise inclusive OR predicates, setting the condition flags.

ORV: Bitwise inclusive OR reduction to scalar.

PEXT (predicate pair): Predicate extract pair from predicate-as-counter.

PEXT (predicate): Predicate extract from predicate-as-counter.

PFALSE: Set all predicate elements to false.

PFIRST: Set the First active predicate element to true.

PMLAL: Multi-vector polynomial multiply long and accumulate.

PMOV (to predicate): Move predicate from vector.

PMOV (to vector): Move predicate to vector.

PMUL: Polynomial multiply (unpredicated).

PMULL: Multi-vector polynomial multiply long.

PMULLB: Polynomial multiply long (bottom).

PMULLT: Polynomial multiply long (top).

PNEXT: Find next active predicate.

PRFB (scalar plus immediate): Contiguous prefetch bytes (immediate index).

PRFB (scalar plus scalar): Contiguous prefetch bytes (scalar index).

PRFB (scalar plus vector): Gather prefetch bytes (scalar plus vector).

PRFB (vector plus immediate): Gather prefetch bytes (vector plus immediate).

PRFD (scalar plus immediate): Contiguous prefetch doublewords (immediate index).

PRFD (scalar plus scalar): Contiguous prefetch doublewords (scalar index).

PRFD (scalar plus vector): Gather prefetch doublewords (scalar plus vector).

PRFD (vector plus immediate): Gather prefetch doublewords (vector plus immediate).

PRFH (scalar plus immediate): Contiguous prefetch halfwords (immediate index).

PRFH (scalar plus scalar): Contiguous prefetch halfwords (scalar index).

PRFH (scalar plus vector): Gather prefetch halfwords (scalar plus vector).

PRFH (vector plus immediate): Gather prefetch halfwords (vector plus immediate).

PRFW (scalar plus immediate): Contiguous prefetch words (immediate index).

PRFW (scalar plus scalar): Contiguous prefetch words (scalar index).

PRFW (scalar plus vector): Gather prefetch words (scalar plus vector).

PRFW (vector plus immediate): Gather prefetch words (vector plus immediate).

PSEL: Predicate select between predicate register or all-false.

PTTEST: Set condition flags for predicate.

PTRUE (predicate as counter): Initialize predicate-as-counter to all active.

PTRUE (predicate): Initialize predicate from named constraint.

PTRES: Initialize predicate from named constraint and set the condition flags.

PUNPKHI, PUNPKLO: Unpack and widen half of predicate.

RADDHNB: Rounding add narrow high part (bottom).

RADDHNT: Rounding add narrow high part (top).

RAX1: Bitwise rotate left by 1 and exclusive-OR.

RBIT: Reverse bits (predicated).

RDFFR (predicated): Return predicate of successfully loaded elements.

RDFFR (unpredicated): Read the first-fault register.

RDFFRS: Return predicate of successfully loaded elements, setting the condition flags.

RDVL: Read multiple of vector register size to scalar register.

REV (predicate): Reverse all elements in a predicate.

REV (vector): Reverse all elements in a vector (unpredicated).

REVB, REVH, REVW: Reverse bytes / halfwords / words within elements (predicated).

REVD: Reverse 64-bit doublewords in elements (predicated).

RSHRNB: Rounding shift right narrow by immediate (bottom).

RSHRNT: Rounding shift right narrow by immediate (top).

RSUBHNB: Rounding subtract narrow high part (bottom).

RSUBHNT: Rounding subtract narrow high part (top).

SABA: Signed absolute difference and accumulate.

SABAL: Two-way signed absolute difference sum and accumulate long.

SABALB: Signed absolute difference and accumulate long (bottom).

SABALT: Signed absolute difference and accumulate long (top).

SABD: Signed absolute difference (predicated).

SABDLB: Signed absolute difference long (bottom).

SABDLT: Signed absolute difference long (top).

SADALP: Signed add and accumulate long pairwise.

SADDLB: Signed add long (bottom).

SADDLBT: Signed add long (bottom + top).

SADDLT: Signed add long (top).

SADDV: Signed add reduction to scalar.

SADDWB: Signed add wide (bottom).

SADDWT: Signed add wide (top).

SBCLB: Subtract with carry long (bottom).

SBCLT: Subtract with carry long (top).

SCLAMP: Signed clamp to minimum/maximum.

SCVTF (bottom, unpredicated): Signed integer convert to floating-point (bottom, unpredicated).

SCVTF (predicated): Signed integer convert to floating-point (predicated).

SCVTFLT: Signed integer convert to floating-point (top, unpredicated).

SDIV: Signed divide (predicated).

SDIVR: Signed reversed divide (predicated).

SDOT (2-way, indexed): Signed integer dot product by indexed element (two-way).

SDOT (2-way, vectors): Signed integer dot product (two-way).

SDOT (4-way, indexed): Signed integer dot product by indexed element (four-way).

SDOT (4-way, vectors): Signed integer dot product (four-way).

SEL (predicates): Conditionally select elements from two predicates.

SEL (vectors): Conditionally select elements from two vectors.

SETFFR: Initialize the first-fault register to all true.

SHADD: Signed halving add.

SHRNB: Shift right narrow by immediate (bottom).

SHRNT: Shift right narrow by immediate (top).

SHSUB: Signed halving subtract.

SHSUBR: Signed halving subtract reversed.

SLI: Shift left and insert (immediate).

SM4E: SM4 encryption and decryption.

SM4EKEY: SM4 key updates.

SMAX (immediate): Signed maximum with immediate (unpredicated).

SMAX (vectors): Signed maximum (predicated).

SMAXP: Signed maximum pairwise.

SMAXQV: Signed maximum reduction of quadword vector segments.

SMAXV: Signed maximum reduction to scalar.

SMIN (immediate): Signed minimum with immediate (unpredicated).

SMIN (vectors): Signed minimum (predicated).

SMINP: Signed minimum pairwise.

SMINQV: Signed minimum reduction of quadword vector segments.

SMINV: Signed minimum reduction to scalar.

SMLALB (indexed): Signed multiply-add long by indexed element (bottom).

SMLALB (vectors): Signed multiply-add long (bottom).

SMLALT (indexed): Signed multiply-add long by indexed element (top).

SMLALT (vectors): Signed multiply-add long (top).

SMLSLB (indexed): Signed multiply-subtract long by indexed element (bottom).

SMLSLB (vectors): Signed multiply-subtract long (bottom).

SMLSLT (indexed): Signed multiply-subtract long by indexed element (top).

SMLSLT (vectors): Signed multiply-subtract long (top).

SMMLA: Signed 8-bit integer matrix multiply-accumulate to 32-bit integer.

SMULH (predicated): Signed multiply returning high half (predicated).

SMULH (unpredicated): Signed multiply returning high half (unpredicated).

SMULLB (indexed): Signed multiply long by indexed element (bottom).

SMULLB (vectors): Signed multiply long (bottom).

SMULLT (indexed): Signed multiply long by indexed element (top).

SMULLT (vectors): Signed multiply long (top).

SPLICE: Splice two vectors under predicate control.

SQABS: Signed saturating absolute value.

[SQADD \(immediate\)](#): Signed saturating add immediate (unpredicated).

SQADD (vectors, predicated): Signed saturating add (predicated).

SQADD (vectors, unpredicated): Signed saturating add (unpredicated).

SQCADD: Saturating complex integer add.

SQCVTN: Signed 32-bit integer saturating extract narrow to interleaved 16-bit integer.

SQCVTUN: Signed 32-bit integer saturating extract narrow to interleaved unsigned 16-bit integer.

SQDECB: Signed saturating decrement scalar by multiple of 8-bit predicate constraint element count.

SQDECD (scalar): Signed saturating decrement scalar by multiple of 64-bit predicate constraint element count.

SQDECD (vector): Signed saturating decrement vector by multiple of 64-bit predicate constraint element count.

SQDECH (scalar): Signed saturating decrement scalar by multiple of 16-bit predicate constraint element count.

SQDECH (vector): Signed saturating decrement vector by multiple of 16-bit predicate constraint element count.

SQDECP (scalar): Signed saturating decrement scalar by count of true predicate elements.

SQDECP (vector): Signed saturating decrement vector by count of true predicate elements.

SQDECW (scalar): Signed saturating decrement scalar by multiple of 32-bit predicate constraint element count.

SQDECW (vector): Signed saturating decrement vector by multiple of 32-bit predicate constraint element count.

SQDMLALB (indexed): Signed saturating doubling multiply-add by indexed element (bottom).

SQDMLALB (vectors): Signed saturating doubling multiply-add (bottom).

SQDMLALBT: Signed saturating doubling multiply-add (bottom $\times$ top).

SQDMLALT (indexed): Signed saturating doubling multiply-add by indexed element (top).

SQDMLALT (vectors): Signed saturating doubling multiply-add (top).

SQDMLSLB (indexed): Signed saturating doubling multiply-subtract by indexed element (bottom).

SQDMLSLB (vectors): Signed saturating doubling multiply-subtract (bottom).

SQDMLSLBT: Signed saturating doubling multiply-subtract (bottom $\times$ top).

SQDMLSLT (indexed): Signed saturating doubling multiply-subtract by indexed element (top).

SQDMLSLT (vectors): Signed saturating doubling multiply-subtract (top).

SQDMULH (indexed): Signed saturating doubling multiply high by indexed element.

SQDMULH (vectors): Signed saturating doubling multiply high (unpredicated).

SQDMULLB (indexed): Signed saturating doubling multiply by indexed element (bottom).

SQDMULLB (vectors): Signed saturating doubling multiply (bottom).

SQDMULLT (indexed): Signed saturating doubling multiply by indexed element (top).

SQDMULLT (vectors): Signed saturating doubling multiply (top).

SQINCB: Signed saturating increment scalar by multiple of 8-bit predicate constraint element count.

SQINCD (scalar): Signed saturating increment scalar by multiple of 64-bit predicate constraint element count.

SQINCD (vector): Signed saturating increment vector by multiple of 64-bit predicate constraint element count.

SQINCH (scalar): Signed saturating increment scalar by multiple of 16-bit predicate constraint element count.

SQINCH (vector): Signed saturating increment vector by multiple of 16-bit predicate constraint element count.

SQINCP (scalar): Signed saturating increment scalar by count of true predicate elements.

SQINCP (vector): Signed saturating increment vector by count of true predicate elements.

SQINCW (scalar): Signed saturating increment scalar by multiple of 32-bit predicate constraint element count.

SQINCW (vector): Signed saturating increment vector by multiple of 32-bit predicate constraint element count.

SQNEG: Signed saturating negate.

SQRDCMLAH (indexed): Saturating rounding doubling complex integer multiply-add high by indexed element.

SQRDCMLAH (vectors): Saturating rounding doubling complex integer multiply-add high.

SQRDMLAH (indexed): Signed saturating rounding doubling multiply-add high by indexed element.

SQRDMLAH (vectors): Signed saturating rounding doubling multiply-add high (unpredicated).

SQRDMLSH (indexed): Signed saturating rounding doubling multiply-subtract high by indexed element.

SQRDMLSH (vectors): Signed saturating rounding doubling multiply-subtract high (unpredicated).

SQRDMULH (indexed): Signed saturating rounding doubling multiply high by indexed element.

SQRDMULH (vectors): Signed saturating rounding doubling multiply high (unpredicated).

SQRSHL: Signed saturating rounding shift left (predicated).

SQRSHLR: Signed saturating rounding shift left reversed (predicated).

SQRSHRN: Signed saturating rounding shift right narrow by immediate to interleaved integer.

SQRSHRNB: Signed saturating rounding shift right narrow by immediate (bottom).

SQRSHRNT: Signed saturating rounding shift right narrow by immediate (top).

SQRSHRUN: Signed saturating rounding shift right unsigned narrow by immediate and interleave.
 SQRSHRUNB: Signed saturating rounding shift right narrow by immediate to unsigned integer (bottom).
 SQRSHRUNT: Signed saturating rounding shift right narrow by immediate to unsigned integer (top).
 SQSHL (immediate): Signed saturating shift left by immediate.
 SQSHL (vectors): Signed saturating shift left (predicated).
 SQSHLR: Signed saturating shift left reversed (predicated).
 SQSHLU: Signed saturating shift left unsigned by immediate.
 SQSHRN: Signed saturating shift right narrow by immediate to interleaved integer.
 SQSHRNB: Signed saturating shift right narrow by immediate (bottom).
 SQSHRNT: Signed saturating shift right narrow by immediate (top).
 SQSHRUN: Signed saturating shift right narrow by immediate to interleaved unsigned integer.
 SQSHRUNB: Signed saturating shift right narrow by immediate to unsigned integer (bottom).
 SQSHRUNT: Signed saturating shift right narrow by immediate to unsigned integer (top).
SQSUB (immediate): Signed saturating subtract immediate (unpredicated).
 SQSUB (vectors, predicated): Signed saturating subtract (predicated).
 SQSUB (vectors, unpredicated): Signed saturating subtract (unpredicated).
 QSUBR: Signed saturating subtract reversed (predicated).
 SQXTNB: Signed saturating extract narrow (bottom).
 SQXTNT: Signed saturating extract narrow (top).
 SQXTUNB: Signed saturating extract narrow to unsigned integer (bottom).
 SQXTUNT: Signed saturating extract narrow to unsigned integer (top).
 SRHADD: Signed rounding halving add.
 SRI: Shift right and insert (immediate).
 RSHL: Signed rounding shift left (predicated).
 RSHLR: Signed rounding shift left reversed (predicated).
 RSHR: Signed rounding shift right by immediate.
 RSRRA: Signed rounding shift right and accumulate (immediate).
 SSHLLB: Signed shift left long by immediate (bottom).
 SSHLLT: Signed shift left long by immediate (top).
 SSRA: Signed shift right and accumulate (immediate).
 SSUBLB: Signed subtract long (bottom).
 SSUBLBT: Signed subtract long (bottom - top).
 SSUBLT: Signed subtract long (top).
 SSUBLTB: Signed subtract long (top - bottom).
 SSUBWB: Signed subtract wide (bottom).
 SSUBWT: Signed subtract wide (top).
 ST1B (scalar plus immediate, consecutive registers): Contiguous store of bytes from multiple consecutive vectors (immediate index).
 ST1B (scalar plus immediate, single register): Contiguous store bytes from vector (immediate index).
 ST1B (scalar plus scalar, consecutive registers): Contiguous store of bytes from multiple consecutive vectors (scalar index).
 ST1B (scalar plus scalar, single register): Contiguous store bytes from vector (scalar index).

ST1B (scalar plus vector): Scatter store bytes from a vector (vector index).

ST1B (vector plus immediate): Scatter store bytes from a vector (immediate index).

ST1D (scalar plus immediate, consecutive registers): Contiguous store of doublewords from multiple consecutive vectors (immediate index).

ST1D (scalar plus immediate, single register): Contiguous store doublewords from vector (immediate index).

ST1D (scalar plus scalar, consecutive registers): Contiguous store of doublewords from multiple consecutive vectors (scalar index).

ST1D (scalar plus scalar, single register): Contiguous store doublewords from vector (scalar index).

ST1D (scalar plus vector): Scatter store doublewords from a vector (vector index).

ST1D (vector plus immediate): Scatter store doublewords from a vector (immediate index).

ST1H (scalar plus immediate, consecutive registers): Contiguous store of halfwords from multiple consecutive vectors (immediate index).

ST1H (scalar plus immediate, single register): Contiguous store halfwords from vector (immediate index).

ST1H (scalar plus scalar, consecutive registers): Contiguous store of halfwords from multiple consecutive vectors (scalar index).

ST1H (scalar plus scalar, single register): Contiguous store halfwords from vector (scalar index).

ST1H (scalar plus vector): Scatter store halfwords from a vector (vector index).

ST1H (vector plus immediate): Scatter store halfwords from a vector (immediate index).

ST1Q: Scatter store quadwords.

ST1W (scalar plus immediate, consecutive registers): Contiguous store of words from multiple consecutive vectors (immediate index).

ST1W (scalar plus immediate, single register): Contiguous store words from vector (immediate index).

ST1W (scalar plus scalar, consecutive registers): Contiguous store of words from multiple consecutive vectors (scalar index).

ST1W (scalar plus scalar, single register): Contiguous store words from vector (scalar index).

ST1W (scalar plus vector): Scatter store words from a vector (vector index).

ST1W (vector plus immediate): Scatter store words from a vector (immediate index).

ST2B (scalar plus immediate): Contiguous store two-byte structures from two vectors (immediate index).

ST2B (scalar plus scalar): Contiguous store two-byte structures from two vectors (scalar index).

ST2D (scalar plus immediate): Contiguous store two-doubleword structures from two vectors (immediate index).

ST2D (scalar plus scalar): Contiguous store two-doubleword structures from two vectors (scalar index).

ST2H (scalar plus immediate): Contiguous store two-halfword structures from two vectors (immediate index).

ST2H (scalar plus scalar): Contiguous store two-halfword structures from two vectors (scalar index).

ST2Q (scalar plus immediate): Contiguous store two-quadword structures from two vectors (immediate index).

ST2Q (scalar plus scalar): Contiguous store two-quadword structures from two vectors (scalar index).

ST2W (scalar plus immediate): Contiguous store two-word structures from two vectors (immediate index).

ST2W (scalar plus scalar): Contiguous store two-word structures from two vectors (scalar index).

ST3B (scalar plus immediate): Contiguous store three-byte structures from three vectors (immediate index).

ST3B (scalar plus scalar): Contiguous store three-byte structures from three vectors (scalar index).

ST3D (scalar plus immediate): Contiguous store three-doubleword structures from three vectors (immediate index).

ST3D (scalar plus scalar): Contiguous store three-doubleword structures from three vectors (scalar index).

ST3H (scalar plus immediate): Contiguous store three-halfword structures from three vectors (immediate index).

ST3H (scalar plus scalar): Contiguous store three-halfword structures from three vectors (scalar index).

ST3Q (scalar plus immediate): Contiguous store three-quadword structures from three vectors (immediate index).

ST3Q (scalar plus scalar): Contiguous store three-quadword structures from three vectors (scalar index).

ST3W (scalar plus immediate): Contiguous store three-word structures from three vectors (immediate index).

ST3W (scalar plus scalar): Contiguous store three-word structures from three vectors (scalar index).

ST4B (scalar plus immediate): Contiguous store four-byte structures from four vectors (immediate index).

ST4B (scalar plus scalar): Contiguous store four-byte structures from four vectors (scalar index).

ST4D (scalar plus immediate): Contiguous store four-doubleword structures from four vectors (immediate index).

ST4D (scalar plus scalar): Contiguous store four-doubleword structures from four vectors (scalar index).

ST4H (scalar plus immediate): Contiguous store four-halfword structures from four vectors (immediate index).

ST4H (scalar plus scalar): Contiguous store four-halfword structures from four vectors (scalar index).

ST4Q (scalar plus immediate): Contiguous store four-quadword structures from four vectors (immediate index).

ST4Q (scalar plus scalar): Contiguous store four-quadword structures from four vectors (scalar index).

ST4W (scalar plus immediate): Contiguous store four-word structures from four vectors (immediate index).

ST4W (scalar plus scalar): Contiguous store four-word structures from four vectors (scalar index).

STNT1B (scalar plus immediate, consecutive registers): Contiguous store non-temporal of bytes from multiple consecutive vectors (immediate index).

STNT1B (scalar plus immediate, single register): Contiguous store non-temporal bytes from vector (immediate index).

STNT1B (scalar plus scalar, consecutive registers): Contiguous store non-temporal of bytes from multiple consecutive vectors (scalar index).

STNT1B (scalar plus scalar, single register): Contiguous store non-temporal bytes from vector (scalar index).

STNT1B (vector plus scalar): Scatter store non-temporal bytes.

STNT1D (scalar plus immediate, consecutive registers): Contiguous store non-temporal of doublewords from multiple consecutive vectors (immediate index).

STNT1D (scalar plus immediate, single register): Contiguous store non-temporal doublewords from vector (immediate index).

STNT1D (scalar plus scalar, consecutive registers): Contiguous store non-temporal of doublewords from multiple consecutive vectors (scalar index).

STNT1D (scalar plus scalar, single register): Contiguous store non-temporal doublewords from vector (scalar index).

STNT1D (vector plus scalar): Scatter store non-temporal doublewords.

STNT1H (scalar plus immediate, consecutive registers): Contiguous store non-temporal of halfwords from multiple consecutive vectors (immediate index).

STNT1H (scalar plus immediate, single register): Contiguous store non-temporal halfwords from vector (immediate index).

STNT1H (scalar plus scalar, consecutive registers): Contiguous store non-temporal of halfwords from multiple consecutive vectors (scalar index).

STNT1H (scalar plus scalar, single register): Contiguous store non-temporal halfwords from vector (scalar index).

STNT1H (vector plus scalar): Scatter store non-temporal halfwords.

STNT1W (scalar plus immediate, consecutive registers): Contiguous store non-temporal of words from multiple consecutive vectors (immediate index).

STNT1W (scalar plus immediate, single register): Contiguous store non-temporal words from vector (immediate index).

STNT1W (scalar plus scalar, consecutive registers): Contiguous store non-temporal of words from multiple consecutive vectors (scalar index).

STNT1W (scalar plus scalar, single register): Contiguous store non-temporal words from vector (scalar index).

STNT1W (vector plus scalar): Scatter store non-temporal words.

[STR \(predicate\)](#): Store predicate register.

[STR \(vector\)](#): Store vector register.

[SUB \(immediate\)](#): Subtract immediate (unpredicated).

SUB (vectors, predicated): Subtract (predicated).

SUB (vectors, unpredicated): Subtract (unpredicated).

SUBHNB: Subtract narrow high part (bottom).

SUBHNT: Subtract narrow high part (top).

SUBP: Subtract pairwise.

SUBPT (predicated): Subtract checked pointer vectors (predicated).

SUBPT (unpredicated): Subtract checked pointer vectors (unpredicated).

[SUBR \(immediate\)](#): Reversed subtract from immediate (unpredicated).

SUBR (vectors): Reversed subtract (predicated).

SUDOT: Signed by unsigned 8-bit integer dot product by indexed element to 32-bit integer.

SUNPKHI, SUNPKLO: Signed unpack and extend half of vector.

SUQADD: Signed saturating unsigned add.

SXTB, SXTH, SXTW: Signed byte / halfword / word extend (predicated).

TBL: Programmable table lookup in one or two vector table (zeroing).

TBLQ: Programmable table lookup within each quadword vector segment (zeroing).

TBX: Programmable table lookup in single vector table (merging).

TBXQ: Programmable table lookup within each quadword vector segment (merging).

TRN1, TRN2 (predicates): Interleave even or odd elements from two predicates.

TRN1, TRN2 (vectors): Interleave even or odd elements from two vectors.

UABA: Unsigned absolute difference and accumulate.

UABAL: Two-way unsigned absolute difference sum and accumulate long.

UABALB: Unsigned absolute difference and accumulate long (bottom).

UABALT: Unsigned absolute difference and accumulate long (top).

UABD: Unsigned absolute difference (predicated).

UABDLB: Unsigned absolute difference long (bottom).

UABDLT: Unsigned absolute difference long (top).

UADALP: Unsigned add and accumulate long pairwise.

UADDLB: Unsigned add long (bottom).

UADDLT: Unsigned add long (top).

UADDV: Unsigned add reduction to scalar.

UADDWB: Unsigned add wide (bottom).

UADDWT: Unsigned add wide (top).

UCLAMP: Unsigned clamp to minimum/maximum.

UCVTF (bottom, unpredicated): Unsigned integer convert to floating-point (bottom, unpredicated).

UCVTF (predicated): Unsigned integer convert to floating-point (predicated).

UCVTFLT: Unsigned integer convert to floating-point (top, unpredicated).

UDIV: Unsigned divide (predicated).

UDIVR: Unsigned reversed divide (predicated).

UDOT (2-way, indexed): Unsigned integer dot product by indexed element (two-way).

UDOT (2-way, vectors): Unsigned integer dot product (two-way).

UDOT (4-way, indexed): Unsigned integer dot product by indexed element (four-way).

UDOT (4-way, vectors): Unsigned integer dot product (four-way).

UHADD: Unsigned halving add.

UHSUB: Unsigned halving subtract.

UHSUBR: Unsigned halving subtract reversed.

UMAX (immediate): Unsigned maximum with immediate (unpredicated).

UMAX (vectors): Unsigned maximum (predicated).

UMAXP: Unsigned maximum pairwise.

UMAXQV: Unsigned maximum reduction of quadword vector segments.

UMAXV: Unsigned maximum reduction to scalar.

UMIN (immediate): Unsigned minimum with immediate (unpredicated).

UMIN (vectors): Unsigned minimum (predicated).

UMINP: Unsigned minimum pairwise.

UMINQV: Unsigned minimum reduction of quadword vector segments.

UMINV: Unsigned minimum reduction to scalar.

UMLALB (indexed): Unsigned multiply-add long by indexed element (bottom).

UMLALB (vectors): Unsigned multiply-add long (bottom).

UMLALT (indexed): Unsigned multiply-add long by indexed element (top).

UMLALT (vectors): Unsigned multiply-add long (top).

UMLSLB (indexed): Unsigned multiply-subtract long by indexed element (bottom).

UMLSLB (vectors): Unsigned multiply-subtract long (bottom).

UMLSLT (indexed): Unsigned multiply-subtract long by indexed element (top).

UMLSLT (vectors): Unsigned multiply-subtract long (top).

UMMLA: Unsigned 8-bit integer matrix multiply-accumulate to 32-bit integer.

UMULH (predicated): Unsigned multiply returning high half (predicated).

UMULH (unpredicated): Unsigned multiply returning high half (unpredicated).

UMULLB (indexed): Unsigned multiply long by indexed element (bottom).

UMULLB (vectors): Unsigned multiply long (bottom).

UMULLT (indexed): Unsigned multiply long by indexed element (top).

UMULLT (vectors): Unsigned multiply long (top).

UQADD (immediate): Unsigned saturating add immediate (unpredicated).

UQADD (vectors, predicated): Unsigned saturating add (predicated).

UQADD (vectors, unpredicated): Unsigned saturating add (unpredicated).

UQCVTN: Unsigned 32-bit integer saturating extract narrow to interleaved 16-bit integer.

UQDECB: Unsigned saturating decrement scalar by multiple of 8-bit predicate constraint element count.

UQDECD (scalar): Unsigned saturating decrement scalar by multiple of 64-bit predicate constraint element count.

UQDECD (vector): Unsigned saturating decrement vector by multiple of 64-bit predicate constraint element count.

UQDECH (scalar): Unsigned saturating decrement scalar by multiple of 16-bit predicate constraint element count.

UQDECH (vector): Unsigned saturating decrement vector by multiple of 16-bit predicate constraint element count.

UQDECP (scalar): Unsigned saturating decrement scalar by count of true predicate elements.

UQDECP (vector): Unsigned saturating decrement vector by count of true predicate elements.

UQDECW (scalar): Unsigned saturating decrement scalar by multiple of 32-bit predicate constraint element count.

UQDECW (vector): Unsigned saturating decrement vector by multiple of 32-bit predicate constraint element count.

UQINCB: Unsigned saturating increment scalar by multiple of 8-bit predicate constraint element count.

UQINCD (scalar): Unsigned saturating increment scalar by multiple of 64-bit predicate constraint element count.

UQINCD (vector): Unsigned saturating increment vector by multiple of 64-bit predicate constraint element count.

UQINCH (scalar): Unsigned saturating increment scalar by multiple of 16-bit predicate constraint element count.

UQINCH (vector): Unsigned saturating increment vector by multiple of 16-bit predicate constraint element count.

UQINCP (scalar): Unsigned saturating increment scalar by count of true predicate elements.

UQINCP (vector): Unsigned saturating increment vector by count of true predicate elements.

UQINCW (scalar): Unsigned saturating increment scalar by multiple of 32-bit predicate constraint element count.

UQINCW (vector): Unsigned saturating increment vector by multiple of 32-bit predicate constraint element count.

UQRSHL: Unsigned saturating rounding shift left (predicated).

UQRSHLR: Unsigned saturating rounding shift left reversed (predicated).

UQRSHRN: Unsigned saturating rounding shift right narrow by immediate to interleaved integer.

UQRSHRNB: Unsigned saturating rounding shift right narrow by immediate (bottom).

UQRSHRNT: Unsigned saturating rounding shift right narrow by immediate (top).

UQSHL (immediate): Unsigned saturating shift left by immediate.

UQSHL (vectors): Unsigned saturating shift left (predicated).

UQSHLR: Unsigned saturating shift left reversed (predicated).

UQSHRN: Unsigned saturating shift right narrow by immediate to interleaved integer.

UQSHRNB: Unsigned saturating shift right narrow by immediate (bottom).

UQSHRNT: Unsigned saturating shift right narrow by immediate (top).

UQSUB (immediate): Unsigned saturating subtract immediate (unpredicated).

UQSUB (vectors, predicated): Unsigned saturating subtract (predicated).

UQSUB (vectors, unpredicated): Unsigned saturating subtract (unpredicated).

UQSUBR: Unsigned saturating subtract reversed (predicated).

UQXTNB: Unsigned saturating extract narrow (bottom).

UQXTNT: Unsigned saturating extract narrow (top).

URECPE: Unsigned reciprocal estimate (predicated).

URHADD: Unsigned rounding halving add.

URSHL: Unsigned rounding shift left (predicated).

URSHLR: Unsigned rounding shift left reversed (predicated).

URSHR: Unsigned rounding shift right by immediate.

URSQRTE: Unsigned reciprocal square root estimate (predicated).

URSRA: Unsigned rounding shift right and accumulate (immediate).

USDOT (indexed): Unsigned by signed 8-bit integer dot product by indexed element to 32-bit integer.

USDOT (vectors): Unsigned by signed 8-bit integer dot product to 32-bit integer.

USHLLB: Unsigned shift left long by immediate (bottom).

USHLLT: Unsigned shift left long by immediate (top).

USMMLA: Unsigned by signed 8-bit integer matrix multiply-accumulate to 32-bit integer.

USQADD: Unsigned saturating signed add.

USRA: Unsigned shift right and accumulate (immediate).

USUBLB: Unsigned subtract long (bottom).

USUBLT: Unsigned subtract long (top).

USUBWB: Unsigned subtract wide (bottom).

USUBWT: Unsigned subtract wide (top).

UUNPKHI, UUNPKLO: Unsigned unpack and extend half of vector.

UXTB, UXTH, UXTW: Unsigned byte / halfword / word extend (predicated).

UZP1, UZP2 (predicates): Concatenate even or odd elements from two predicates.

UZP1, UZP2 (vectors): Concatenate even or odd elements from two vectors.

UZPQ1: Concatenate even elements within each pair of quadword vector segments.

UZPQ2: Concatenate odd elements within each pair of quadword vector segments.

WHILEGE (predicate as counter): While decrementing signed scalar greater than or equal to scalar (predicate-as-counter).

WHILEGE (predicate pair): While decrementing signed scalar greater than or equal to scalar (pair of predicates).

WHILEGE (predicate): While decrementing signed scalar greater than or equal to scalar.

WHILEGT (predicate as counter): While decrementing signed scalar greater than scalar (predicate-as-counter).

WHILEGT (predicate pair): While decrementing signed scalar greater than scalar (pair of predicates).

WHILEGT (predicate): While decrementing signed scalar greater than scalar.

WHILEHI (predicate as counter): While decrementing unsigned scalar higher than scalar (predicate-as-counter).

WHILEHI (predicate pair): While decrementing unsigned scalar higher than scalar (pair of predicates).

WHILEHI (predicate): While decrementing unsigned scalar higher than scalar.

WHILEHS (predicate as counter): While decrementing unsigned scalar higher than or same as scalar (predicate-as-counter).

WHILEHS (predicate pair): While decrementing unsigned scalar higher than or same as scalar (pair of predicates).

WHILEHS (predicate): While decrementing unsigned scalar higher than or same as scalar.

WHILELE (predicate as counter): While incrementing signed scalar less than or equal to scalar (predicate-as-counter).

WHILELE (predicate pair): While incrementing signed scalar less than or equal to scalar (pair of predicates).

WHILELE (predicate): While incrementing signed scalar less than or equal to scalar.

WHILELO (predicate as counter): While incrementing unsigned scalar lower than scalar (predicate-as-counter).

WHILELO (predicate pair): While incrementing unsigned scalar lower than scalar (pair of predicates).

WHILELO (predicate): While incrementing unsigned scalar lower than scalar.

WHILELS (predicate as counter): While incrementing unsigned scalar lower than or same as scalar (predicate-as-counter).

WHILELS (predicate pair): While incrementing unsigned scalar lower than or same as scalar (pair of predicates).

WHILELS (predicate): While incrementing unsigned scalar lower than or same as scalar.

WHILELT (predicate as counter): While incrementing signed scalar less than scalar (predicate-as-counter).

WHILELT (predicate pair): While incrementing signed scalar less than scalar (pair of predicates).

WHILELT (predicate): While incrementing signed scalar less than scalar.

WHILERW: While free of read-after-write conflicts.

WHILEWR: While free of write-after-read/write conflicts.

WRFFR: Write the first-fault register.

XAR: Bitwise exclusive-OR and rotate right by immediate.

ZIP1, ZIP2 (predicates): Interleave elements from two half predicates.

ZIP1, ZIP2 (vectors): Interleave elements from two half vectors.

ZIPQ1: Interleave elements from low halves of each pair of quadword vector segments.

ZIPQ2: Interleave elements from high halves of each pair of quadword vector segments.

(old)	htmldiff from-	(new)
-------	----------------	-------

ADD (immediate)

Add immediate (unpredicated)

This instruction adds an unsigned immediate to each element of the source vector, and destructively places the results in the corresponding elements of the source vector. This instruction is unpredicated.

The immediate is an unsigned value in the range 0 to 255, and for element widths of 16 bits or higher it may also be a positive multiple of 256 in the range 256 to 65280.

The immediate is encoded in 8 bits with an optional left shift by 8. The preferred disassembly when the shift option is specified is "**#<imm>imm8**", LSL #8". However an assembler and disassembler may also allow use of the shifted 16-bit value unless the immediate is 0 and the shift amount is 8, which must be unambiguously described as "#0, LSL #8".

SVE

(FEAT_SVE || FEAT_SME)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1	size	1	0	0	0	0	0	1	1	sh	imm8								Zdn					
opc																															

Encoding

ADD <Zdn>.<T>, <Zdn>.<T>, #<imm>{, <shift>}

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
  EndOfDecode(Decode_UNDEF);
end;
if size::sh == '001' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn);
var imm : integer = UInt(imm8);
if sh == '1' then imm = imm << 8; end;
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in "size":

size	<T>
00	B
01	H
10	S
11	D

<imm> Is an unsigned immediate in the range 0 to 255, encoded in the "imm8" field.

<shift> Is the optional left shift to apply to the immediate, defaulting to LSL #0 and encoded in "sh":

sh	<shift>
0	LSL #0
1	LSL #8

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
  let element1 : bits(esize) = operand1[e*esize];
  result[e*esize] = element1 + imm;
end;
Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in [About PSTATE.DIT](#).

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2025-12-16 16:41:29 2025-09-10 2025-12-10 2025-11-28 rel diff tag 41294505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)	htmldiff from-	(new)
-------	----------------	-------

BFMMLA (widening)

BFloat16 matrix multiply-accumulate to single-precision

If FEAT_EBF16 is not implemented or FPCR.EBF is 0, this instruction:

- MultipliesPerforms two unfused sums-of-products within each two pairs of adjacent BFloat16 elements while multiplying the 2×4 matrix of BFloat16 values held in each 128-bit segment of the first source vector by the 4×2 matrix of BFloat16 values in the corresponding corresponding segment of the second source vector. The intermediate single-precision results products are calculatedrounded as a sum of two unfused sum-of-products using two pairs ofbefore adjacentthey BFloat16are valuessummed fromand each of the sourceintermediate vectors,sum andis rounded before accumulation into the 2×2 single-precision matrix of single-precision results in the corresponding segment of the destination vector. This is equivalent to accumulating two 2-way unfused dot products per destination element.
- Uses the non-IEEE 754 Round-to-Odd rounding mode, which forces bit 0 of an inexact result to 1, and rounds an overflow to an appropriately signed Infinity.
- Flushes denormalized inputs and results to zero, as if FPCR.{FZ, FIZ} is {1, 1}.
- Honors FPCR.AH when generating a default NaN result, otherwise disables alternative floating point behaviors, as if FPCR.AH is 0.

If FEAT_EBF16 is implemented and FPCR.EBF is 1, then this instruction:

- MultipliesPerforms two fused sums-of-products within each two pairs of adjacent BFloat16 elements while multiplying the 2×4 matrix of BFloat16 values held in each 128-bit segment of the first source vector by the 4×2 matrix of BFloat16 values in the corresponding corresponding segment of the second source vector. The intermediate single-precision resultsproducts are calculatedasnot rounded sumbefore they ofare twosummed, fusedbut sum-of-products using two pairs of adjacent BFloat16 values from each of the sourceintermediate vectors,sum andis rounded before accumulation into the 2×2 single-precision matrix ofin single-precisionthe resultscorresponding insegment of the destination vector. This is equivalent to accumulating two 2-way fused dot products per destination element.
- Follows all other floating-point behaviors that apply to single-precision arithmetic, as governed by FPCR.RMode, FPCR.FZ, FPCR.AH, and FPCR.FIZ.

Irrespective of FEAT_EBF16 and FPCR.EBF, this instruction:

- Does not modify the cumulative FPSR exception bits (IDC, IXC, UFC, OFC, DZC, and IOC).
- Disables trapped floating-point exceptions, as if the FPCR trap enable bits (IDE, IXE, UFE, OFE, DZE, and IOE) are all zero.
- Generates only the default NaN, as if FPCR.DN is 1.

This instruction is unpredicated and vector length agnostic.

ID_AA64ZFR0_EL1.BF16 indicates whether this instruction is implemented.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE

(FEAT_SVE && FEAT_BF16)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	0	0	1	1	Zm				1	1	1	0	0	1	Zn				Zda						
opc																															

Encoding

BFMMLA <Zda>.S, <Zn>.H, <Zm>.H

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) || !IsFeatureImplemented(FEAT_BF16) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

- <Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
- <Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.
- <Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let segments : integer = VL DIV 128;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);
var op1, op2 : bits(128);
var res, addend : bits(128);

for s = 0 to segments-1 do
    op1 = operand1[s*:(128)];
    op2 = operand2[s*:(128)];
    addend = operand3[s*:(128)];
    res = BFMatMulAddH(addend, op1, op2, FPCR());
    result[s*:(128)] = res;
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2025-12-re2025-09-diff-tag 2025-12-102025-11-28 16:4129:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)	htmldiff from-	(new)
-------	----------------	-------

CPY (immediate, merging)

Copy signed integer immediate to vector elements (merging)

This instruction copies a signed integer immediate to each Active element in the destination vector. Inactive elements in the destination vector register remain unmodified.

The immediate operand is a signed value in the range -128 to +127, and for element widths of 16 bits or higher it may also be a signed multiple of 256 in the range -32768 to +32512 (excluding 0).

The immediate is encoded in 8 bits with an optional left shift by 8. The preferred disassembly when the shift option is specified is "**#<imm>imm8**", LSL #8". However an assembler and disassembler may also allow use of the shifted 16-bit value unless the immediate is 0 and the shift amount is 8, which must be unambiguously described as "#0, LSL #8".

This instruction is used by the alias [MOV \(immediate, merging\)](#).

This instruction is used by the pseudo-instruction [FMOV \(zero, predicated\)](#).

SVE

(FEAT_SVE || FEAT_SME)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1	size	0	1			Pg			0	1	sh												Zd	

M

Encoding

CPY <Zd>.<T>, <Pg>/M, #<imm>{, <shift>}

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size::sh == '001' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let d : integer = UInt(Zd);
let merging : boolean = TRUE;
var imm : integer = SInt(imm8);
if sh == '1' then imm = imm << 8; end;
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<T>	Is the size specifier, encoded in "size":
size	<T>
00	B
01	H
10	S
11	D
<Pg>	Is the name of the governing scalable predicate register, encoded in the "Pg" field.
<imm>	Is a signed immediate in the range -128 to 127, encoded in the "imm8" field.
<shift>	Is the optional left shift to apply to the immediate, defaulting to LSL #0 and encoded in "sh":
sh	<shift>
0	LSL #0
1	LSL #8

Alias Conditions

Alias	Is preferred when
FMOV (zero, predicated)	Never
MOV (immediate, merging)	Unconditionally

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let dest : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
  if ActivePredicateElement{PL}(mask, e, esize) then
    result[e*:esize] = imm[esize-1:0];
  else
    result[e*:esize] = dest[e*:esize];
  end;
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in [About PSTATE.DIT](#).
This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2025-12_rel2025-09_diff_tag 2025-12-102025-11-28 16:4129:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)	htmldiff from-	(new)
-------	----------------	-------

CPY (immediate, zeroing)

Copy signed integer immediate to vector elements (zeroing)

This instruction copies a signed integer immediate to each Active element in the destination vector. Inactive elements in the destination vector register are set to zero.

The immediate operand is a signed value in the range -128 to +127, and for element widths of 16 bits or higher it may also be a signed multiple of 256 in the range -32768 to +32512 (excluding 0).

The immediate is encoded in 8 bits with an optional left shift by 8. The preferred disassembly when the shift option is specified is "[#<imm>imm8](#)", LSL #8". However an assembler and disassembler may also allow use of the shifted 16-bit value unless the immediate is 0 and the shift amount is 8, which must be unambiguously described as "#0, LSL #8".

This instruction is used by the alias [MOV \(immediate, zeroing\)](#).

SVE

(FEAT_SVE || FEAT_SME)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1	size		0	1	Pg				0	0	sh	imm8								Zd				
																	M														

Encoding

CPY [<Zd>.<T>](#), [<Pg>/Z](#), #[<imm>](#){, [<shift>](#)}

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode\_UNDEF);
end;
if size::sh == '001' then EndOfDecode(Decode\_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let d : integer = UInt(Zd);
let merging : boolean = FALSE;
var imm : integer = SInt(imm8);
if sh == '1' then imm = imm << 8; end;
```

Assembler Symbols

[<Zd>](#) Is the name of the destination scalable vector register, encoded in the "Zd" field.

[<T>](#) Is the size specifier, encoded in "size":

size	<T>
00	B
01	H
10	S
11	D

[<Pg>](#) Is the name of the governing scalable predicate register, encoded in the "Pg" field.

[<imm>](#) Is a signed immediate in the range -128 to 127, encoded in the "imm8" field.

[<shift>](#) Is the optional left shift to apply to the immediate, defaulting to LSL #0 and encoded in "sh":

sh	<shift>
0	LSL #0
1	LSL #8

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let dest : bits(VL) = if merging then Z{VL}(d) else Zeros(VL);
var result : bits(VL);

for e = 0 to elements-1 do
  if ActivePredicateElement{PL}(mask, e, esize) then
    result[e*:esize] = imm[esize-1:0];
  else
    result[e*:esize] = dest[e*:esize];
  end;
end;

Z{VL}(d) = result;
```

2025-12-re2025-09-diff-tag 2025-12-102025-11-28 16:4129:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)	htmldiff from-	(new)
-------	----------------	-------

DUP (immediate)

Broadcast signed immediate to vector elements (unpredicated)

This instruction unconditionally broadcasts the signed integer immediate into each element of the destination vector. This instruction is unpredicated.

The immediate operand is a signed value in the range -128 to +127, and for element widths of 16 bits or higher it may also be a signed multiple of 256 in the range -32768 to +32512 (excluding 0).

The immediate is encoded in 8 bits with an optional left shift by 8. The preferred disassembly when the shift option is specified is "#<imm>imm8", LSL #8". However an assembler and disassembler may also allow use of the shifted 16-bit value unless the immediate is 0 and the shift amount is 8, which must be unambiguously described as "#0, LSL #8".

This instruction is used by the alias [MOV \(immediate, unpredicated\)](#).

This instruction is used by the pseudo-instruction [FMOV \(zero, unpredicated\)](#).

SVE

(FEAT_SVE || FEAT_SME)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1	size	1	1	1	0	0	0	1	1	sh	imm8								Zd					
opc																															

Encoding

DUP <Zd>.<T>, #<imm>{, <shift>}

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
  EndOfDecode(Decode_UNDEF);
end;
if size::sh == '001' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let d : integer = UInt(Zd);
var imm : integer = SInt(imm8);
if sh == '1' then imm = imm << 8; end;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in "size":

size	<T>
00	B
01	H
10	S
11	D

<imm> Is a signed immediate in the range -128 to 127, encoded in the "imm8" field.

<shift> Is the optional left shift to apply to the immediate, defaulting to LSL #0 and encoded in "sh":

sh	<shift>
0	LSL #0
1	LSL #8

Alias Conditions

Alias	Is preferred when
FMOV (zero, unpredicated)	Never
MOV (immediate, unpredicated)	Unconditionally

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let result : bits(VL) = Replicate{}(imm[esize-1:0]);
Z{VL}(d) = result;
```


FMMLA (widening, FP16 to FP32)

Half-precision matrix multiply-accumulate to single-precision

This instruction multiplies performs two fused sums-of-products within each two pairs of adjacent half-precision elements while multiplying the 2×4 matrix of half-precision values held in each 128-bit segment of the first source vector by the 4×2 matrix of half-precision values in the corresponding segment of the second source vector. The intermediate single-precision sums-of-products results are calculated rounded as before as they sum are of two fused summed, sum-of-products and using their two intermediate pair sum of adjacent half-precision values from each of the source vectors, and rounded before accumulation into the 2×2 single-precision matrix of single-precision results in the corresponding segment of the destination vector. This is equivalent to performing a 4-way dot product per destination element.

This instruction is unpredicated.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE2

(FEAT_SVE_F16F32MM)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	0	0	0	1	Zm					1	1	1	0	0	1	Zn					Zda				
opc																															

Encoding

FMMLA <Zda>.S, <Zn>.H, <Zm>.H

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_F16F32MM) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let segments : integer = VL DIV 128;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for s = 0 to segments-1 do
    let op1 : bits(128) = operand1[s*:(128)];
    let op2 : bits(128) = operand2[s*:(128)];
    let addend : bits(128) = operand3[s*:(128)];
    result[s*:(128)] = FPMatMulAddH{128}(addend, op1, op2, FPCR());
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2025-12_rel2025-09_diff_tag 2025-12-102025-11-28 16:4129:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMMLA (widening, FP8 to FP16)

8-bit floating-point matrix multiply-accumulate to half-precision

This instruction ~~multiplies~~~~performs~~ the ~~fused sum-of-products within each four adjacent 8-bit elements while multiplying the~~ 2×4 matrix of 8-bit floating-point values held in each 64-bit segment of the first source vector ~~by with the~~ the 4×2 matrix of 8-bit floating-point values in the corresponding ~~segment~~ segment of the second source vector. The ~~intermediate~~ half-precision ~~results calculated as a fused~~ sum-of-products ~~using four adjacent 8-bit floating-point values from each of the source vectors,~~ are scaled by $2^{-\text{UInt}(\text{FPMR.LSCALE}[3:0])}$, before being destructively added without intermediate rounding to the 2×2 half-precision matrix in the corresponding segment of the destination vector. This is equivalent to accumulating 4-way dot product per destination element.

The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

This instruction is unpredicated.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE2

(FEAT_SVE2 && FEAT_F8F16MM)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	0	0	1	1	Zm					1	1	1	0	0	0	Zn					Zda				
op																															

Encoding

FMMLA <Zda>.H, <Zn>.B, <Zm>.B

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) || !IsFeatureImplemented(FEAT_F8F16MM) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckFPMREnabled();
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let segments : integer = VL DIV 64;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for s = 0 to segments-1 do
    let op1 : bits(64) = operand1[s*:(64)];
    let op2 : bits(64) = operand2[s*:(64)];
    let addend : bits(64) = operand3[s*:(64)];
    let way : integer{} = 4;
    result[s*:(64)] = FP8MatMulAddFP[64](addend, op1, op2, way, FPCR(), FPMR());
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

FMMLA (widening, FP8 to FP32)

8-bit floating-point matrix multiply-accumulate to single-precision

This instruction multiplies performs the fused sum-of-products within each eight adjacent 8-bit elements while multiplying the 2×8 matrix of 8-bit floating-point values held in each 128-bit segment of the first source vector by with the the 8×2 matrix of 8-bit floating-point values in the corresponding segment segment of the second source vector. The intermediate single-precision results calculated as a fused sum-of-products using eight adjacent 8-bit floating-point values from each of the source vectors, are scaled by $2^{-\text{UInt}(\text{FPMR.LSCALE})}$, before being destructively added without intermediate rounding to the 2×2 single-precision matrix in the corresponding segment of the destination vector. This is equivalent to accumulating 8-way dot product per destination element.

The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

This instruction is unpredicated.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE2

(FEAT_SVE2 && FEAT_F8F32MM)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	0	0	0	1	Zm				1	1	1	0	0	0	Zn				Zda						
op																															

Encoding

FMMLA <Zda>.S, <Zn>.B, <Zm>.B

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) || !IsFeatureImplemented(FEAT_F8F32MM) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckFPMREnabled();
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let segments : integer = VL DIV 128;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for s = 0 to segments-1 do
    let op1 : bits(128) = operand1[s*:(128)];
    let op2 : bits(128) = operand2[s*:(128)];
    let addend : bits(128) = operand3[s*:(128)];
    let way : integer{} = 8;
    result[s*:(128)] = FP8MatMulAddFP{128}(addend, op1, op2, way, FPCR(), FPMR());
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

LDR (predicate)

Load predicate register

This instruction loads a predicate register from a memory address generated by a 64-bit scalar base, plus an immediate offset in the range -256 to 255 that is multiplied by the current predicate register size in bytes. This instruction is unpredicated.

The load is performed as contiguous byte accesses, each containing 8 consecutive predicate bits in ascending element order, with no endian conversion and no guarantee of single-copy atomicity larger than a byte. However, if alignment is checked, then a general-purpose base register must be aligned to 2 bytes.

For programmer convenience, an assembler must also accept a predicate-as-counter register name for the destination predicate register.

SVE

(FEAT_SVE || FEAT_SME)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	1	0	1	1	0	imm9h						0	0	0	imm9l			Rn			0	Pt					

Encoding

```
LDR <P>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Pt);
let n : integer = UInt(Rn);
let imm : integer = SInt(imm9h::imm9l);
```

Assembler Symbols

<P>	Is the name of the destination scalable predicate register, encoded in the "Pt" field.
<Xn SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, in the range -256 to 255, defaulting to 0, encoded in the "imm9h:imm9l" fields.

Operation

```
CheckSVEEnabled();
let VL : integer() = CurrentVL();
let PL : integer() = VL DIV 8;
let elements : integer = PL DIV 8;
var base : bits(64);
let offset : integer = imm * elements;
var result : bits(PL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = FALSE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                    predicated, tagchecked);

if n == 31 then
    CheckSPAlignment();
    base = SP{64}();
else
    base = X{64}(n);
end;

var addr : bits(64) = AddressAdd(base, offset, accdesc);

let aligned : boolean = IsAlignedSize(base+addr, 2);

if !aligned && AlignmentEnforced() then
    let fault : FaultRecord = AlignmentFault(accdesc, addr);
    AArch64_Abort(fault);
end;

for e = 0 to elements-1 do
    result[e*8] = AArch64\_MemSingle{8}(addr, accdesc, aligned);
    addr = AddressIncrement(addr, 1, accdesc);
end;

P{PL}(t) = result;
```

(old)	htmldiff from-	(new)
-------	----------------	-------

LDR (vector)

Load vector register

This instruction loads a vector register from a memory address generated by a 64-bit scalar base, plus an immediate offset in the range -256 to 255 that is multiplied by the current vector register size in bytes. This instruction is unpredicated.

The load is performed as contiguous byte accesses, with no endian conversion and no guarantee of single-copy atomicity larger than a byte. However, if alignment is checked, then the base register must be aligned to 16 bytes.

SVE

(FEAT_SVE || FEAT_SME)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	1	0	1	1	0	imm9h						0	1	0	imm9l			Rn				Zt					

Encoding

```
LDR <Zt>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let imm : integer = SInt(imm9h::imm9l);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Xn SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, in the range -256 to 255, defaulting to 0, encoded in the "imm9h:imm9l" fields.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV 8;
var base : bits(64);
let offset : integer = imm * elements;
var result : bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = FALSE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                    predicated, tagchecked);

if n == 31 then
    CheckSPAlignment();
    base = SP{64}();
else
    base = X{64}(n);
end;

var addr : bits(64) = AddressAdd(base, offset, accdesc);

let aligned : boolean = IsAlignedSize(baseaddr, 16);

if !aligned && AlignmentEnforced() then
    let fault : FaultRecord = AlignmentFault(accdesc, addr);
    AArch64_Abort(fault);
end;

for e = 0 to elements-1 do
    result[e*8] = AArch64\_MemSingle{8}(addr, accdesc, aligned);
    addr = AddressIncrement(addr, 1, accdesc);
end;

Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in [About PSTATE.DIT](#).

(old)	htmldiff from-	(new)
-------	----------------	-------

MOV (immediate, merging)

Move signed integer immediate to vector elements (merging)

This instruction moves a signed integer immediate to each Active element in the destination vector. Inactive elements in the destination vector register remain unmodified.

The immediate operand is a signed value in the range -128 to +127, and for element widths of 16 bits or higher it may also be a signed multiple of 256 in the range -32768 to +32512 (excluding 0).

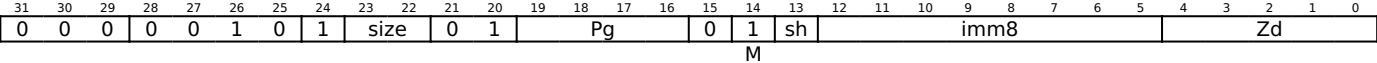
The immediate is encoded in 8 bits with an optional left shift by 8. The preferred disassembly when the shift option is specified is "**#<imm>imm8**", LSL #8". However an assembler and disassembler may also allow use of the shifted 16-bit value unless the immediate is 0 and the shift amount is 8, which must be unambiguously described as "#0, LSL #8".

This is an alias of [CPY \(immediate, merging\)](#). This means:

- The encodings in this description are named to match the encodings of [CPY \(immediate, merging\)](#).
- The description of [CPY \(immediate, merging\)](#) gives the operational pseudocode, any **CONSTRAINED UNPREDICTABLE** behavior, and any operational information for this instruction.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

MOV <Zd>.<T>, <Pg>/M, #<imm>{, <shift>}

is equivalent to

CPY <Zd>.<T>, <Pg>/M, #<imm>{, <shift>}

and is always the preferred disassembly.

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.										
<T>	Is the size specifier, encoded in "size":										
<table><tr><th>size</th><th><T></th></tr><tr><td>00</td><td>B</td></tr><tr><td>01</td><td>H</td></tr><tr><td>10</td><td>S</td></tr><tr><td>11</td><td>D</td></tr></table>		size	<T>	00	B	01	H	10	S	11	D
size	<T>										
00	B										
01	H										
10	S										
11	D										
<Pg>	Is the name of the governing scalable predicate register, encoded in the "Pg" field.										
<imm>	Is a signed immediate in the range -128 to 127, encoded in the "imm8" field.										
<shift>	Is the optional left shift to apply to the immediate, defaulting to LSL #0 and encoded in "sh":										
<table><tr><th>sh</th><th><shift></th></tr><tr><td>0</td><td>LSL #0</td></tr><tr><td>1</td><td>LSL #8</td></tr></table>		sh	<shift>	0	LSL #0	1	LSL #8				
sh	<shift>										
0	LSL #0										
1	LSL #8										

Operation

The description of [CPY \(immediate, merging\)](#) gives the operational pseudocode for this instruction.

Operational Information

The description of [CPY \(immediate, merging\)](#) gives the operational information for this instruction.

MOV (immediate, unpredicated)

Move signed immediate to vector elements (unpredicated)

This instruction unconditionally broadcasts the signed integer immediate into each element of the destination vector. This instruction is unpredicated.

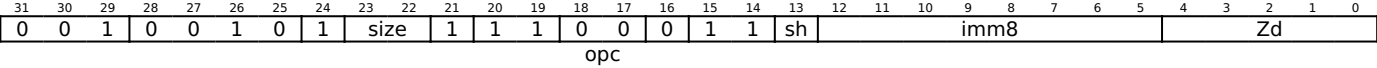
The immediate operand is a signed value in the range -128 to +127, and for element widths of 16 bits or higher it may also be a signed multiple of 256 in the range -32768 to +32512 (excluding 0).

The immediate is encoded in 8 bits with an optional left shift by 8. The preferred disassembly when the shift option is specified is "**#<imm>**", LSL #8". However an assembler and disassembler may also allow use of the shifted 16-bit value unless the immediate is 0 and the shift amount is 8, which must be unambiguously described as "#0, LSL #8".

This is an alias of [DUP \(immediate\)](#). This means:

- The encodings in this description are named to match the encodings of [DUP \(immediate\)](#).
- The description of [DUP \(immediate\)](#) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

SVE
(FEAT_SVE || FEAT_SME)



Encoding

MOV <Zd>.<T>, #<imm>{, <shift>}

is equivalent to

[DUP](#) <Zd>.<T>, #<imm>{, <shift>}

and is always the preferred disassembly.

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in "size":

size	<T>
00	B
01	H
10	S
11	D

<imm> Is a signed immediate in the range -128 to 127, encoded in the "imm8" field.

<shift> Is the optional left shift to apply to the immediate, defaulting to LSL #0 and encoded in "sh":

sh	<shift>
0	LSL #0
1	LSL #8

Operation

The description of [DUP \(immediate\)](#) gives the operational pseudocode for this instruction.

2025-12-10 16:41:29

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOV (immediate, zeroing)

Move signed integer immediate to vector elements (zeroing)

This instruction moves a signed integer immediate to each Active element in the destination vector. Inactive elements in the destination vector register are set to zero.

The immediate operand is a signed value in the range -128 to +127, and for element widths of 16 bits or higher it may also be a signed multiple of 256 in the range -32768 to +32512 (excluding 0).

The immediate is encoded in 8 bits with an optional left shift by 8. The preferred disassembly when the shift option is specified is "**#<imm>imm8**", LSL #8". However an assembler and disassembler may also allow use of the shifted 16-bit value unless the immediate is 0 and the shift amount is 8, which must be unambiguously described as "#0, LSL #8".

This is an alias of [CPY \(immediate, zeroing\)](#). This means:

- The encodings in this description are named to match the encodings of [CPY \(immediate, zeroing\)](#).
- The description of [CPY \(immediate, zeroing\)](#) gives the operational pseudocode, any **CONSTRAINED UNPREDICTABLE** behavior, and any operational information for this instruction.

SVE

(FEAT_SVE || FEAT_SME)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1	size	0	1	Pg			0	0	sh	imm8								Zd						
																	M														

Encoding

MOV <Zd>.<T>, <Pg>/Z, #<imm>{, <shift>}

is equivalent to

CPY <Zd>.<T>, <Pg>/Z, #<imm>{, <shift>}

and is always the preferred disassembly.

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.										
<T>	Is the size specifier, encoded in "size":										
<table><tr><th>size</th><th><T></th></tr><tr><td>00</td><td>B</td></tr><tr><td>01</td><td>H</td></tr><tr><td>10</td><td>S</td></tr><tr><td>11</td><td>D</td></tr></table>		size	<T>	00	B	01	H	10	S	11	D
size	<T>										
00	B										
01	H										
10	S										
11	D										
<Pg>	Is the name of the governing scalable predicate register, encoded in the "Pg" field.										
<imm>	Is a signed immediate in the range -128 to 127, encoded in the "imm8" field.										
<shift>	Is the optional left shift to apply to the immediate, defaulting to LSL #0 and encoded in "sh":										
<table><tr><th>sh</th><th><shift></th></tr><tr><td>0</td><td>LSL #0</td></tr><tr><td>1</td><td>LSL #8</td></tr></table>		sh	<shift>	0	LSL #0	1	LSL #8				
sh	<shift>										
0	LSL #0										
1	LSL #8										

Operation

The description of [CPY \(immediate, zeroing\)](#) gives the operational pseudocode for this instruction.

SQADD (immediate)

Signed saturating add immediate (unpredicated)

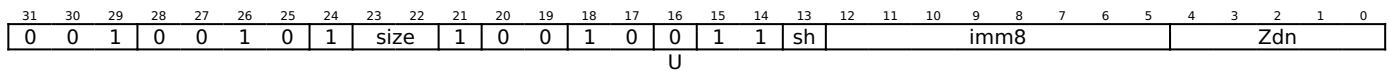
This instruction performs a signed saturating addition of an unsigned immediate to each element of the source vector, and destructively places the results in the corresponding elements of the source vector. Each result element is saturated to the N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. This instruction is unpredicated.

The immediate is an unsigned value in the range 0 to 255, and for element widths of 16 bits or higher it may also be a positive multiple of 256 in the range 256 to 65280.

The immediate is encoded in 8 bits with an optional left shift by 8. The preferred disassembly when the shift option is specified is "#<imm>imm8, LSL #8". However an assembler and disassembler may also allow use of the shifted 16-bit value unless the immediate is 0 and the shift amount is 8, which must be unambiguously described as "#0, LSL #8".

SVE

(FEAT_SVE || FEAT_SME)



Encoding

SQADD <Zdn>.<T>, <Zdn>.<T>, #<imm>{, <shift>}

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
  EndOfDecode(Decode_UNDEF);
end;
if size::sh == '001' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn);
var imm : integer = UInt(imm8);
if sh == '1' then imm = imm << 8; end;
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in "size":

size	<T>
00	B
01	H
10	S
11	D

<imm> Is an unsigned immediate in the range 0 to 255, encoded in the "imm8" field.

<shift> Is the optional left shift to apply to the immediate, defaulting to LSL #0 and encoded in "sh":

sh	<shift>
0	LSL #0
1	LSL #8

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
  let oplelt : bits(esize) = operand1[e*esize];
  let element1 : integer = if unsigned then UInt(oplelt) else SInt(oplelt);
  (result[e*esize], -) = SatQ(esize)(element1 + imm, unsigned);
end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2025-12-re2025-09-diff-tag 2025-12-102025-11-28 16:4129:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)	htmldiff from-	(new)
-------	----------------	-------

SQSUB (immediate)

Signed saturating subtract immediate (unpredicated)

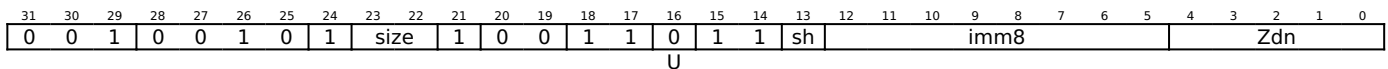
This instruction performs a signed saturating subtraction of an unsigned immediate from each element of the source vector, and destructively places the results in the corresponding elements of the source vector. Each result element is saturated to the N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. This instruction is unpredicated.

The immediate is an unsigned value in the range 0 to 255, and for element widths of 16 bits or higher it may also be a positive multiple of 256 in the range 256 to 65280.

The immediate is encoded in 8 bits with an optional left shift by 8. The preferred disassembly when the shift option is specified is "#<imm>imm8, LSL #8". However an assembler and disassembler may also allow use of the shifted 16-bit value unless the immediate is 0 and the shift amount is 8, which must be unambiguously described as "#0, LSL #8".

SVE

(FEAT_SVE || FEAT_SME)



Encoding

SQSUB <Zdn>.<T>, <Zdn>.<T>, #<imm>{, <shift>}

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
  EndOfDecode(Decode_UNDEF);
end;
if size::sh == '001' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn);
var imm : integer = UInt(imm8);
if sh == '1' then imm = imm << 8; end;
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in "size":

size	<T>
00	B
01	H
10	S
11	D

<imm> Is an unsigned immediate in the range 0 to 255, encoded in the "imm8" field.

<shift> Is the optional left shift to apply to the immediate, defaulting to LSL #0 and encoded in "sh":

sh	<shift>
0	LSL #0
1	LSL #8

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
  let oplelt : bits(esize) = operand1[e*esize];
  let element1 : integer = if unsigned then UInt(oplelt) else SInt(oplelt);
  (result[e*esize], -) = SatQ(esize)(element1 - imm, unsigned);
end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2025-12-re2025-09-diff-tag 2025-12-102025-11-28 16:4129:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)	htmldiff from-	(new)
-------	----------------	-------

STR (predicate)

Store predicate register

This instruction stores a predicate register to a memory address generated by a 64-bit scalar base, plus an immediate offset in the range -256 to 255 that is multiplied by the current predicate register size in bytes. This instruction is unpredicated.

The store is performed as contiguous byte accesses, each containing 8 consecutive predicate bits in ascending element order, with no endian conversion and no guarantee of single-copy atomicity larger than a byte. However, if alignment is checked, then a general-purpose base register must be aligned to 2 bytes.

For programmer convenience, an assembler must also accept a predicate-as-counter register name for the source predicate register.

SVE

(FEAT_SVE || FEAT_SME)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	1	0	1	1	0	imm9h					0	0	0	imm9l				Rn				0	Pt				

Encoding

STR <P>, [<Xn|SP>{, #<imm>, MUL VL}]

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Pt);
let n : integer = UInt(Rn);
let imm : integer = SInt(imm9h::imm9l);
```

Assembler Symbols

<P>	Is the name of the scalable predicate transfer register, encoded in the "Pt" field.
<Xn SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, in the range -256 to 255, defaulting to 0, encoded in the "imm9h:imm9l" fields.

Operation

```
CheckSVEEnabled();
let PL : integer{} = CurrentVL() DIV 8;
let elements : integer = PL DIV 8;
var src : bits(PL);
var base : bits(64);
let offset : integer = imm * elements;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = FALSE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                    predicated, tagchecked);

if n == 31 then
    CheckSPAlignment();
    base = SP{64}();
else
    base = X{64}(n);
end;

src = P{PL}(t);

var addr : bits(64) = AddressAdd(base, offset, accdesc);

let aligned : boolean = IsAlignedSize(baseaddr, 2);

if !aligned && AlignmentEnforced() then
    let fault : FaultRecord = AlignmentFault(accdesc, addr);
    AArch64_Abort(fault);
end;

for e = 0 to elements-1 do
    AArch64\_MemSingle{8}(addr, accdesc, aligned) = src[e*8];
    addr = AddressIncrement(addr, 1, accdesc);
end;
```


STR (vector)

Store vector register

This instruction stores a vector register to a memory address generated by a 64-bit scalar base, plus an immediate offset in the range -256 to 255 that is multiplied by the current vector register size in bytes. This instruction is unpredicated.

The store is performed as contiguous byte accesses, with no endian conversion and no guarantee of single-copy atomicity larger than a byte. However, if alignment is checked, then the base register must be aligned to 16 bytes.

SVE

(FEAT_SVE || FEAT_SME)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	1	0	1	1	0	imm9h						0	1	0	imm9l			Rn				Zt					

Encoding

STR <Zt>, [<Xn|SP>{, #<imm>, MUL VL}]

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let imm : integer = SInt(imm9h::imm9l);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Xn SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, in the range -256 to 255, defaulting to 0, encoded in the "imm9h:imm9l" fields.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 8;
var src : bits(VL);
var base : bits(64);
let offset : integer = imm * elements;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = FALSE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                    predicated, tagchecked);

if n == 31 then
    CheckSPAlignment();
    base = SP{64}();
else
    base = X{64}(n);
end;

src = Z{VL}(t);

var addr : bits(64) = AddressAdd(base, offset, accdesc);

let aligned : boolean = IsAlignedSize(baseaddr, 16);

if !aligned && AlignmentEnforced() then
    let fault : FaultRecord = AlignmentFault(accdesc, addr);
    AArch64_Abort(fault);
end;

for e = 0 to elements-1 do
    AArch64\_MemSingle{8}(addr, accdesc, aligned) = src[e*8];
    addr = AddressIncrement(addr, 1, accdesc);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in [About PSTATE.DIT](#).

(old)	htmldiff from-	(new)
-------	----------------	-------

SUB (immediate)

Subtract immediate (unpredicated)

This instruction subtracts an unsigned immediate from each element of the source vector, and destructively places the results in the corresponding elements of the source vector. This instruction is unpredicated.

The immediate is an unsigned value in the range 0 to 255, and for element widths of 16 bits or higher it may also be a positive multiple of 256 in the range 256 to 65280.

The immediate is encoded in 8 bits with an optional left shift by 8. The preferred disassembly when the shift option is specified is "**#<imm>imm8**", LSL #8". However an assembler and disassembler may also allow use of the shifted 16-bit value unless the immediate is 0 and the shift amount is 8, which must be unambiguously described as "#0, LSL #8".

SVE

(FEAT_SVE || FEAT_SME)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1	size	1	0	0	0	0	1	1	1	sh	imm8								Zdn					
opc																															

Encoding

SUB <Zdn>.<T>, <Zdn>.<T>, #<imm>{, <shift>}

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
  EndOfDecode(Decode_UNDEF);
end;
if size::sh == '001' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn);
var imm : integer = UInt(imm8);
if sh == '1' then imm = imm << 8; end;
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in "size":

size	<T>
00	B
01	H
10	S
11	D

<imm> Is an unsigned immediate in the range 0 to 255, encoded in the "imm8" field.

<shift> Is the optional left shift to apply to the immediate, defaulting to LSL #0 and encoded in "sh":

sh	<shift>
0	LSL #0
1	LSL #8

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
  let element1 : bits(esize) = operand1[e*esize];
  result[e*esize] = element1 - imm;
end;
Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in [About PSTATE.DIT](#).

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2025-12-10 16:41:29 2025-09-10 16:41:29 2025-12-10 16:41:29 2025-11-28 16:41:29 2025-12-10 16:41:29

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)	htmldiff from-	(new)
-------	----------------	-------

SUBR (immediate)

Reversed subtract from immediate (unpredicated)

This instruction performs a reversed subtraction from an unsigned immediate each element of the source vector, and destructively places the results in the corresponding elements of the source vector. This instruction is unpredicated.

The immediate is an unsigned value in the range 0 to 255, and for element widths of 16 bits or higher it may also be a positive multiple of 256 in the range 256 to 65280.

The immediate is encoded in 8 bits with an optional left shift by 8. The preferred disassembly when the shift option is specified is "#<imm>imm8", LSL #8". However an assembler and disassembler may also allow use of the shifted 16-bit value unless the immediate is 0 and the shift amount is 8, which must be unambiguously described as "#0, LSL #8".

SVE

(FEAT_SVE || FEAT_SME)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1	size	1	0	0	0	1	1	1	1	sh	imm8								Zdn					
opc																															

Encoding

SUBR <Zdn>.<T>, <Zdn>.<T>, #<imm>{, <shift>}

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
  EndOfDecode(Decode_UNDEF);
end;
if size::sh == '001' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn);
var imm : integer = UInt(imm8);
if sh == '1' then imm = imm << 8; end;
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in "size":

size	<T>
00	B
01	H
10	S
11	D

<imm> Is an unsigned immediate in the range 0 to 255, encoded in the "imm8" field.

<shift> Is the optional left shift to apply to the immediate, defaulting to LSL #0 and encoded in "sh":

sh	<shift>
0	LSL #0
1	LSL #8

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
  let element1 : integer = UInt(operand1[e*:esize]);
  result[e*:esize] = (imm - element1)[esize-1:0];
end;
Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in [About PSTATE.DIT](#).

This instruction might be immediately preceded in program order by a `MOVPRFX` instruction. The `MOVPRFX` must conform to all of the following requirements, otherwise the behavior of the `MOVPRFX` and this instruction is `CONSTRAINED UNPREDICTABLE`:

- The `MOVPRFX` must be unpredicated.
- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2025-12-10 16:41:29 4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)	htmldiff from-	(new)
-------	----------------	-------

UQADD (immediate)

Unsigned saturating add immediate (unpredicated)

This instruction performs an unsigned saturating addition of an unsigned immediate to each element of the source vector, and destructively places the results in the corresponding elements of the source vector. Each result element is saturated to the N-bit element's unsigned integer range 0 to $(2^N)-1$. This instruction is unpredicated.

The immediate is an unsigned value in the range 0 to 255, and for element widths of 16 bits or higher it may also be a positive multiple of 256 in the range 256 to 65280.

The immediate is encoded in 8 bits with an optional left shift by 8. The preferred disassembly when the shift option is specified is "#<imm>imm8", LSL #8". However an assembler and disassembler may also allow use of the shifted 16-bit value unless the immediate is 0 and the shift amount is 8, which must be unambiguously described as "#0, LSL #8".

SVE

(FEAT_SVE || FEAT_SME)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
0	0	1	0	0	1	0	1	size	1	0	0	1	0	1	1	1	sh	imm8								Zdn							
U																																	

Encoding

UQADD <Zdn>.<T>, <Zdn>.<T>, #<imm>{, <shift>}

Decode for this encoding

```

if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
  EndOfDecode(Decode_UNDEF);
end;
if size::sh == '001' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn);
var imm : integer = UInt(imm8);
if sh == '1' then imm = imm << 8; end;
let unsigned : boolean = TRUE;

```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in "size":

size	<T>
00	B
01	H
10	S
11	D

<imm> Is an unsigned immediate in the range 0 to 255, encoded in the "imm8" field.

<shift> Is the optional left shift to apply to the immediate, defaulting to LSL #0 and encoded in "sh":

sh	<shift>
0	LSL #0
1	LSL #8

Operation

```

CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
  let oplelt : bits(esize) = operand1[e*esize];
  let element1 : integer = if unsigned then UInt(oplelt) else SInt(oplelt);
  (result[e*esize], -) = SatQ(esize)(element1 + imm, unsigned);
end;
Z{VL}(dn) = result;

```

Operational information

This instruction might be immediately preceded in program order by a `MOVPRFX` instruction. The `MOVPRFX` must conform to all of the following requirements, otherwise the behavior of the `MOVPRFX` and this instruction is `CONSTRAINED UNPREDICTABLE`:

- The `MOVPRFX` must be unpredicated.
- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2025-12-re2025-09-diff-tag 2025-12-102025-11-28 16:4129:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)	htmldiff from-	(new)
-------	----------------	-------

UQSUB (immediate)

Unsigned saturating subtract immediate (unpredicated)

This instruction performs an unsigned saturating subtraction of an unsigned immediate from each element of the source vector, and destructively places the results in the corresponding elements of the source vector. Each result element is saturated to the N-bit element's unsigned integer range 0 to $(2^N)-1$. This instruction is unpredicated.

The immediate is an unsigned value in the range 0 to 255, and for element widths of 16 bits or higher it may also be a positive multiple of 256 in the range 256 to 65280.

The immediate is encoded in 8 bits with an optional left shift by 8. The preferred disassembly when the shift option is specified is "#<imm>imm8>, LSL #8". However an assembler and disassembler may also allow use of the shifted 16-bit value unless the immediate is 0 and the shift amount is 8, which must be unambiguously described as "#0, LSL #8".

SVE

(FEAT_SVE || FEAT_SME)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1	size	1	0	0	1	1	1	1	1	1	sh	imm8								Zdn				
U																															

Encoding

UQSUB <Zdn>.<T>, <Zdn>.<T>, #<imm>{, <shift>}

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
  EndOfDecode(Decode_UNDEF);
end;
if size::sh == '001' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn);
var imm : integer = UInt(imm8);
if sh == '1' then imm = imm << 8; end;
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in "size":

size	<T>
00	B
01	H
10	S
11	D

<imm> Is an unsigned immediate in the range 0 to 255, encoded in the "imm8" field.

<shift> Is the optional left shift to apply to the immediate, defaulting to LSL #0 and encoded in "sh":

sh	<shift>
0	LSL #0
1	LSL #8

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
  let op1elt : bits(esize) = operand1[e*esize];
  let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
  (result[e*esize], -) = SatQ(esize)(element1 - imm, unsigned);
end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2025-12-re2025-09-diff-tag 2025-12-102025-11-28 16:4129:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)	htmldiff from-	(new)
-------	----------------	-------

A64 -- SME Instructions (alphabetic order)

ADD (to array, array and multiple vectors): Multi-vector accumulate to ZA array vectors.

ADD (to array, multiple and single vector): Multi-vector add by vector to ZA array vectors.

ADD (to array, multiple vectors): Multi-vector add to ZA array vectors.

ADD (to vector, multiple vectors): Multi-vector add by vector to multi-vector.

ADDHA: Add horizontally vector elements to ZA tile.

ADDSPIL: Add multiple of Streaming SVE predicate register size to scalar register.

ADDSPVL: Add multiple of Streaming SVE vector register size to scalar register.

ADDVA: Add vertically vector elements to ZA tile.

BF1CVT, BF2CVT: Multi-vector 8-bit floating-point convert to BFloat16.

BF1CVTL, BF2CVTL: Multi-vector 8-bit floating-point convert to deinterleaved BFloat16.

BFADD: Multi-vector BFloat16 accumulate to ZA array vectors.

BFCLAMP: Multi-vector BFloat16 clamp to minimum/maximum number.

BFCVT (BFloat16 to 8-bit floating-point): Multi-vector BFloat16 convert to 8-bit floating-point.

BFCVT (single-precision to BFloat16): Multi-vector single-precision convert to BFloat16.

BFCVTN: Multi-vector single-precision convert to interleaved BFloat16.

BFDOT (multiple and indexed vector): Multi-vector BFloat16 dot product by indexed element to single-precision.

BFDOT (multiple and single vector): Multi-vector BFloat16 dot product by vector to single-precision.

BFDOT (multiple vectors): Multi-vector BFloat16 dot product to single-precision.

BFMAX (multiple and single vector): Multi-vector BFloat16 maximum by vector.

BFMAX (multiple vectors): Multi-vector BFloat16 maximum.

BFMAXNM (multiple and single vector): Multi-vector BFloat16 maximum number by vector.

BFMAXNM (multiple vectors): Multi-vector BFloat16 maximum number.

BFMIN (multiple and single vector): Multi-vector BFloat16 minimum by vector.

BFMIN (multiple vectors): Multi-vector BFloat16 minimum.

BFMINNM (multiple and single vector): Multi-vector BFloat16 minimum number by vector.

BFMINNM (multiple vectors): Multi-vector BFloat16 minimum number.

BFMLA (multiple and indexed vector): Multi-vector BFloat16 fused multiply-add by indexed element.

BFMLA (multiple and single vector): Multi-vector BFloat16 fused multiply-add by vector.

BFMLA (multiple vectors): Multi-vector BFloat16 fused multiply-add.

BFMLAL (multiple and indexed vector): Multi-vector BFloat16 multiply-add by indexed element to single-precision.

BFMLAL (multiple and single vector): Multi-vector BFloat16 multiply-add by vector to single-precision.

BFMLAL (multiple vectors): Multi-vector BFloat16 multiply-add to single-precision.

BFMLS (multiple and indexed vector): Multi-vector BFloat16 fused multiply-subtract by indexed element.

BFMLS (multiple and single vector): Multi-vector BFloat16 fused multiply-subtract by vector.

BFMLS (multiple vectors): Multi-vector BFloat16 fused multiply-subtract.

BFMLSL (multiple and indexed vector): Multi-vector BFloat16 multiply-subtract by indexed element from single-precision.

BFMLSL (multiple and single vector): Multi-vector BFloat16 multiply-subtract by vector from single-precision.

BFMLSL (multiple vectors): Multi-vector BFloat16 multiply-subtract from single-precision.

BFMOP4A (non-widening): BFloat16 quarter-tile outer product, accumulating.

BFMOP4A (widening): BFloat16 quarter-tile sum of outer products to single-precision, accumulating.

BFMOP4S (non-widening): BFloat16 quarter-tile outer product, subtracting.

BFMOP4S (widening): BFloat16 quarter-tile sum of outer products to single-precision, subtracting.

BFMOPA (non-widening): BFloat16 outer product, accumulating.

BFMOPA (widening): BFloat16 sum of outer products to single-precision, accumulating.

BFMOPS (non-widening): BFloat16 outer product, subtracting.

BFMOPS (widening): BFloat16 sum of outer products to single-precision, subtracting.

BFMUL (multiple and single vector): Multi-vector BFloat16 multiply by vector.

BFMUL (multiple vectors): Multi-vector BFloat16 multiply.

BFSCALE (multiple and single vector): Multi-vector BFloat16 adjust exponent by vector.

BFSCALE (multiple vectors): Multi-vector BFloat16 adjust exponent.

BFSUB: Multi-vector BFloat16 subtract from ZA array vectors.

BFTMOPA (non-widening): BFloat16 sparse outer product, accumulating.

BFTMOPA (widening): BFloat16 sparse sum of outer products to single-precision, accumulating.

BFVDOT: Multi-vector BFloat16 vertical dot product by indexed element to single-precision.

BMOPA: Bitwise exclusive NOR population count outer product, accumulating.

BMOPS: Bitwise exclusive NOR population count outer product, subtracting.

F1CVT, F2CVT: Multi-vector 8-bit floating-point convert to half-precision.

F1CVTL, F2CVTL: Multi-vector 8-bit floating-point convert to deinterleaved half-precision.

FADD: Multi-vector floating-point accumulate to ZA array vectors.

FAMAX: Multi-vector floating-point absolute maximum.

FAMIN: Multi-vector floating-point absolute minimum.

FCLAMP: Multi-vector floating-point clamp to minimum/maximum number.

FCVT (narrowing, FP16 to FP8): Multi-vector half-precision convert to 8-bit floating-point.

FCVT (narrowing, FP32 to FP16): Multi-vector single-precision convert to half-precision.

FCVT (narrowing, FP32 to FP8): Multi-vector single-precision convert to 8-bit floating-point.

FCVT (widening): Multi-vector half-precision convert to single-precision.

FCVTL: Multi-vector half-precision convert to deinterleaved single-precision.

FCVTN (FP32 to FP16): Multi-vector single-precision convert to interleaved half-precision.

FCVTN (FP32 to FP8): Multi-vector single-precision convert to interleaved 8-bit floating-point.

FCVTZS: Multi-vector single-precision convert to signed 32-bit integer, rounding toward zero.

FCVTZU: Multi-vector single-precision convert to unsigned 32-bit integer, rounding toward zero.

FDOT (2-way, multiple and indexed vector, FP16 to FP32): Multi-vector half-precision dot product by indexed element to single-precision.

FDOT (2-way, multiple and indexed vector, FP8 to FP16): Multi-vector 8-bit floating-point dot product by indexed element to half-precision.

FDOT (2-way, multiple and single vector, FP16 to FP32): Multi-vector half-precision dot product by vector to single-precision.

FDOT (2-way, multiple and single vector, FP8 to FP16): Multi-vector 8-bit floating-point dot product by vector to half-precision.

FDOT (2-way, multiple vectors, FP16 to FP32): Multi-vector half-precision dot product to single-precision.

FDOT (2-way, multiple vectors, FP8 to FP16): Multi-vector 8-bit floating-point dot product to half-precision.

FDOT (4-way, multiple and indexed vector): Multi-vector 8-bit floating-point dot product by indexed element to single-precision.

FDOT (4-way, multiple and single vector): Multi-vector 8-bit floating-point dot product by vector to single-precision.

FDOT (4-way, multiple vectors): Multi-vector 8-bit floating-point dot product to single-precision.

FMAX (multiple and single vector): Multi-vector floating-point maximum by vector.

FMAX (multiple vectors): Multi-vector floating-point maximum.

FMAXNM (multiple and single vector): Multi-vector floating-point maximum number by vector.

FMAXNM (multiple vectors): Multi-vector floating-point maximum number.

FMIN (multiple and single vector): Multi-vector floating-point minimum by vector.

FMIN (multiple vectors): Multi-vector floating-point minimum.

FMINNM (multiple and single vector): Multi-vector floating-point minimum number by vector.

FMINNM (multiple vectors): Multi-vector floating-point minimum number.

FMLA (multiple and indexed vector): Multi-vector floating-point fused multiply-add by indexed element.

FMLA (multiple and single vector): Multi-vector floating-point fused multiply-add by vector.

FMLA (multiple vectors): Multi-vector floating-point fused multiply-add.

FMLAL (multiple and indexed vector, FP16 to FP32): Multi-vector half-precision multiply-add by indexed element to single-precision.

FMLAL (multiple and indexed vector, FP8 to FP16): Multi-vector 8-bit floating-point multiply-add by indexed element to half-precision.

FMLAL (multiple and single vector, FP16 to FP32): Multi-vector half-precision multiply-add by vector to single-precision.

FMLAL (multiple and single vector, FP8 to FP16): Multi-vector 8-bit floating-point multiply-add by vector to half-precision.

FMLAL (multiple vectors, FP16 to FP32): Multi-vector half-precision multiply-add to single-precision.

FMLAL (multiple vectors, FP8 to FP16): Multi-vector 8-bit floating-point multiply-add to half-precision.

FMLALL (multiple and indexed vector): Multi-vector 8-bit floating-point multiply-add by indexed element to single-precision.

FMLALL (multiple and single vector): Multi-vector 8-bit floating-point multiply-add by vector to single-precision.

FMLALL (multiple vectors): Multi-vector 8-bit floating-point multiply-add to single-precision.

FMLS (multiple and indexed vector): Multi-vector floating-point fused multiply-subtract by indexed element.

FMLS (multiple and single vector): Multi-vector floating-point fused multiply-subtract by vector.

FMLS (multiple vectors): Multi-vector floating-point fused multiply-subtract.

FMLSL (multiple and indexed vector): Multi-vector half-precision multiply-subtract by indexed element from single-precision.

FMLSL (multiple and single vector): Multi-vector half-precision multiply-subtract by vector from single-precision.

FMLSL (multiple vectors): Multi-vector half-precision multiply-subtract from single-precision.

FMOP4A (non-widening): Floating-point quarter-tile outer product, accumulating.

FMOP4A (widening, 2-way, FP16 to FP32): Half-precision quarter-tile sum of outer products to single-precision, accumulating.

FMOP4A (widening, 2-way, FP8 to FP16): 8-bit floating-point quarter-tile sum of outer products to half-precision, accumulating.

FMOP4A (widening, 4-way): 8-bit floating-point quarter-tile sum of outer products to single-precision, accumulating.

FMOP4S (non-widening): Floating-point quarter-tile outer product, subtracting.

FMOP4S (widening): Half-precision quarter-tile sum of outer products to single-precision, subtracting.

FMOPA (non-widening): Floating-point outer product, accumulating.

FMOPA (widening, 2-way, FP16 to FP32): Half-precision sum of outer products to single-precision, accumulating.

FMOPA (widening, 2-way, FP8 to FP16): 8-bit floating-point sum of outer products to half-precision, accumulating.

FMOPA (widening, 4-way): 8-bit floating-point sum of outer products to single-precision, accumulating.

FMOPS (non-widening): Floating-point outer product, subtracting.

FMOPS (widening): Half-precision sum of outer products to single-precision, subtracting.

FMUL (multiple and single vector): Multi-vector floating-point multiply by vector.

FMUL (multiple vectors): Multi-vector floating-point multiply.

FRINTA: Multi-vector single-precision round to integral value, to nearest with ties away from zero.

FRINTM: Multi-vector single-precision round to integral value, toward minus Infinity.

FRINTN: Multi-vector single-precision round to integral value, to nearest with ties to even.

FRINTP: Multi-vector single-precision round to integral value, toward plus Infinity.

FSCALE (multiple and single vector): Multi-vector floating-point adjust exponent by vector.

FSCALE (multiple vectors): Multi-vector floating-point adjust exponent.

FSUB: Multi-vector floating-point subtract from ZA array vectors.

FTMOPA (non-widening): Floating-point sparse outer product, accumulating.

FTMOPA (widening, 2-way, FP16 to FP32): Half-precision sparse sum of outer products to single-precision, accumulating.

FTMOPA (widening, 2-way, FP8 to FP16): 8-bit floating-point sparse sum of outer products to half-precision, accumulating.

FTMOPA (widening, 4-way): 8-bit floating-point sparse sum of outer products to single-precision, accumulating.

FVDOT (FP16 to FP32): Multi-vector half-precision vertical dot product by indexed element to single-precision.

FVDOT (FP8 to FP16): Multi-vector 8-bit floating-point vertical dot product by indexed element to half-precision.

FVDOTB: Multi-vector 8-bit floating-point vertical dot product by indexed element to single-precision (bottom).

FVDOTT: Multi-vector 8-bit floating-point vertical dot product by indexed element to single-precision (top).

LD1B (scalar plus immediate, strided registers): Contiguous load of bytes to multiple strided vectors (immediate index).

LD1B (scalar plus scalar, strided registers): Contiguous load of bytes to multiple strided vectors (scalar index).

LD1B (scalar plus scalar, tile slice): Contiguous load of bytes to 8-bit element ZA tile slice.

LD1D (scalar plus immediate, strided registers): Contiguous load of doublewords to multiple strided vectors (immediate index).

LD1D (scalar plus scalar, strided registers): Contiguous load of doublewords to multiple strided vectors (scalar index).

LD1D (scalar plus scalar, tile slice): Contiguous load of doublewords to 64-bit element ZA tile slice.

LD1H (scalar plus immediate, strided registers): Contiguous load of halfwords to multiple strided vectors (immediate index).

LD1H (scalar plus scalar, strided registers): Contiguous load of halfwords to multiple strided vectors (scalar index).

LD1H (scalar plus scalar, tile slice): Contiguous load of halfwords to 16-bit element ZA tile slice.

LD1Q: Contiguous load of quadwords to 128-bit element ZA tile slice.

LD1W (scalar plus immediate, strided registers): Contiguous load of words to multiple strided vectors (immediate index).

LD1W (scalar plus scalar, strided registers): Contiguous load of words to multiple strided vectors (scalar index).

LD1W (scalar plus scalar, tile slice): Contiguous load of words to 32-bit element ZA tile slice.

LDNT1B (scalar plus immediate, strided registers): Contiguous load non-temporal of bytes to multiple strided vectors (immediate index).

LDNT1B (scalar plus scalar, strided registers): Contiguous load non-temporal of bytes to multiple strided vectors (scalar index).

LDNT1D (scalar plus immediate, strided registers): Contiguous load non-temporal of doublewords to multiple strided vectors (immediate index).

LDNT1D (scalar plus scalar, strided registers): Contiguous load non-temporal of doublewords to multiple strided vectors (scalar index).

LDNT1H (scalar plus immediate, strided registers): Contiguous load non-temporal of halfwords to multiple strided vectors (immediate index).

LDNT1H (scalar plus scalar, strided registers): Contiguous load non-temporal of halfwords to multiple strided vectors (scalar index).

LDNT1W (scalar plus immediate, strided registers): Contiguous load non-temporal of words to multiple strided vectors (immediate index).

LDNT1W (scalar plus scalar, strided registers): Contiguous load non-temporal of words to multiple strided vectors (scalar index).

[LDR \(array vector\)](#): Load ZA array vector.

LDR (table): Load ZT0 register.

LUT12 (four registers): Lookup table read with 2-bit indexes (four registers).

LUT12 (single): Lookup table read with 2-bit indexes (single).

LUT12 (two registers): Lookup table read with 2-bit indexes (two registers).

LUT14 (four registers, 16-bit and 32-bit): Lookup table read with 4-bit indexes (four registers).

LUT14 (four registers, 8-bit): Lookup table read with 4-bit indexes and 8-bit elements (four registers).

LUT14 (single): Lookup table read with 4-bit indexes (single).

LUT14 (two registers): Lookup table read with 4-bit indexes (two registers).

LUT16 (table, four registers, 8-bit): Lookup table read with 6-bit indices (8-bit).

LUT16 (table, single, 8-bit): Lookup table read with 6-bit indices (single).

[LUT16 \(vector, 16-bit\)](#): Lookup table read with 6-bit indices (16-bit).

MOV (array to vector, four registers): Move four ZA single-vector groups to Z four-vector operand: an alias of MOVA (array to vector, four registers).

MOV (array to vector, two registers): Move two ZA single-vector groups to Z two-vector operand: an alias of MOVA (array to vector, two registers).

MOV (tile to vector, four registers): Move ZA four-slice operand to Z four-vector operand: an alias of MOVA (tile to vector, four registers).

MOV (tile to vector, single): Move ZA tile slice to Z vector: an alias of MOVA (tile to vector, single).

MOV (tile to vector, two registers): Move ZA two-slice operand to Z two-vector operand: an alias of MOVA (tile to vector, two registers).

MOV (vector to array, four registers): Move Z four-vector operand to four ZA single-vector groups: an alias of MOVA (vector to array, four registers).

MOV (vector to array, two registers): Move Z two-vector operand to two ZA single-vector groups: an alias of MOVA (vector to array, two registers).

MOV (vector to tile, four registers): Move Z four-vector operand to ZA four-slice operand: an alias of MOVA (vector to tile, four registers).

MOV (vector to tile, single): Move Z vector to ZA tile slice: an alias of MOVA (vector to tile, single).

MOV (vector to tile, two registers): Move Z two-vector operand to ZA two-slice operand: an alias of MOVA (vector to tile, two registers).

MOVA (array to vector, four registers): Move four ZA single-vector groups to Z four-vector operand.

MOVA (array to vector, two registers): Move two ZA single-vector groups to Z two-vector operand.

MOVA (tile to vector, four registers): Move ZA four-slice operand to Z four-vector operand.

MOVA (tile to vector, single): Move ZA tile slice to Z vector.

MOVA (tile to vector, two registers): Move ZA two-slice operand to Z two-vector operand.

MOVA (vector to array, four registers): Move Z four-vector operand to four ZA single-vector groups.

MOVA (vector to array, two registers): Move Z two-vector operand to two ZA single-vector groups.

MOVA (vector to tile, four registers): Move Z four-vector operand to ZA four-slice operand.

MOVA (vector to tile, single): Move Z vector to ZA tile slice.

MOVA (vector to tile, two registers): Move Z two-vector operand to ZA two-slice operand.

MOVZ (array to vector, four registers): Move and zero four ZA single-vector groups to Z four-vector operand.

MOVZ (array to vector, two registers): Move and zero two ZA single-vector groups to Z two-vector operand.

MOVZ (tile to vector, four registers): Move and zero ZA four-slice operand to Z four-vector operand.

MOVZ (tile to vector, single): Move and zero ZA tile slice to Z vector.

MOVZ (tile to vector, two registers): Move and zero ZA two-slice operand to Z two-vector operand.

MOVT (scalar to table): Move 8 bytes from general-purpose register to ZT0.

MOVT (table to scalar): Move 8 bytes from ZT0 to general-purpose register.

MOVT (vector to table): Move vector register to ZT0.

RDSVL: Read multiple of Streaming SVE vector register size to scalar register.

SCLAMP: Multi-vector signed clamp to minimum/maximum.

SCVTF: Multi-vector signed 32-bit integer convert to single-precision.

SDOT (2-way, multiple and indexed vector): Multi-vector signed 16-bit integer dot product by indexed element to 32-bit integer.

SDOT (2-way, multiple and single vector): Multi-vector signed 16-bit integer dot product by vector to 32-bit integer.

SDOT (2-way, multiple vectors): Multi-vector signed 16-bit integer dot product to 32-bit integer.

SDOT (4-way, multiple and indexed vector): Multi-vector signed integer dot product by indexed element.

SDOT (4-way, multiple and single vector): Multi-vector signed integer dot product by vector.

SDOT (4-way, multiple vectors): Multi-vector signed integer dot product.

[SEL](#): Multi-vector conditional select.

SMAX (multiple and single vector): Multi-vector signed maximum by vector.

SMAX (multiple vectors): Multi-vector signed maximum.

SMIN (multiple and single vector): Multi-vector signed minimum by vector.

SMIN (multiple vectors): Multi-vector signed minimum.

SMLAL (multiple and indexed vector): Multi-vector signed 16-bit integer multiply-add by indexed element to 32-bit integer.

SMLAL (multiple and single vector): Multi-vector signed 16-bit integer multiply-add by vector to 32-bit integer.

SMLAL (multiple vectors): Multi-vector signed 16-bit integer multiply-add to 32-bit integer.

SMLALL (multiple and indexed vector): Multi-vector signed integer multiply-add long long by indexed element.

SMLALL (multiple and single vector): Multi-vector signed integer multiply-add long long by vector.

SMLALL (multiple vectors): Multi-vector signed integer multiply-add long long.

SMLSL (multiple and indexed vector): Multi-vector signed 16-bit integer multiply-subtract by indexed element from 32-bit integer.

SMLSL (multiple and single vector): Multi-vector signed 16-bit integer multiply-subtract by vector from 32-bit integer.

SMLSL (multiple vectors): Multi-vector signed 16-bit integer multiply-subtract from 32-bit integer.

SMLSLL (multiple and indexed vector): Multi-vector signed integer multiply-subtract long long by indexed element.

SMLSLL (multiple and single vector): Multi-vector signed integer multiply-subtract long long by vector.

SMLSLL (multiple vectors): Multi-vector signed integer multiply-subtract long long.

SMOP4A (2-way): Signed 16-bit integer quarter-tile sum of outer products to 32-bit integer, accumulating.

SMOP4A (4-way): Signed integer quarter-tile sum of outer products, accumulating.

SMOP4S (2-way): Signed 16-bit integer quarter-tile sum of outer products to 32-bit integer, subtracting.

SMOP4S (4-way): Signed integer quarter-tile sum of outer products, subtracting.

SMOPA (2-way): Signed 16-bit integer sum of outer products to 32-bit integer, accumulating.

SMOPA (4-way): Signed integer sum of outer products, accumulating.

SMOPS (2-way): Signed 16-bit integer sum of outer products to 32-bit integer, subtracting.

SMOPS (4-way): Signed integer sum of outer products, subtracting.

SQCVT (four registers): Multi-vector signed saturating extract narrow.

SQCVT (two registers): Multi-vector signed 32-bit integer saturating extract narrow to 16-bit integer.

SQCVTN: Multi-vector signed saturating extract narrow to interleaved integer.

SQCVTU (four registers): Multi-vector signed saturating extract narrow to unsigned integer.

SQCVTU (two registers): Multi-vector signed 32-bit integer saturating extract narrow to unsigned 16-bit integer.

SQCVTUN: Multi-vector signed saturating extract narrow to interleaved unsigned integer.

SQDMULH (multiple and single vector): Multi-vector signed saturating doubling multiply high by vector.

SQDMULH (multiple vectors): Multi-vector signed saturating doubling multiply high.

SQRSHR (four registers): Multi-vector signed saturating rounding shift right narrow by immediate.

SQRSHR (two registers): Multi-vector signed 32-bit integer saturating rounding shift right narrow by immediate to 16-bit integer.

SQRSHRN: Multi-vector signed saturating rounding shift right narrow by immediate to interleaved integer.

SQRSHRU (four registers): Multi-vector signed saturating rounding shift right narrow by immediate to unsigned integer.

SQRSHRU (two registers): Multi-vector signed 32-bit integer saturating rounding shift right narrow by immediate to unsigned 16-bit integer.

SQRSHRUN: Multi-vector signed saturating rounding shift right narrow by immediate to interleaved unsigned integer.

SRSHL (multiple and single vector): Multi-vector signed rounding shift left by vector.

SRSHL (multiple vectors): Multi-vector signed rounding shift left.

ST1B (scalar plus immediate, strided registers): Contiguous store of bytes from multiple strided vectors (immediate index).

ST1B (scalar plus scalar, strided registers): Contiguous store of bytes from multiple strided vectors (scalar index).

ST1B (scalar plus scalar, tile slice): Contiguous store of bytes from 8-bit element ZA tile slice.

ST1D (scalar plus immediate, strided registers): Contiguous store of doublewords from multiple strided vectors (immediate index).

ST1D (scalar plus scalar, strided registers): Contiguous store of doublewords from multiple strided vectors (scalar index).

ST1D (scalar plus scalar, tile slice): Contiguous store of doublewords from 64-bit element ZA tile slice.

ST1H (scalar plus immediate, strided registers): Contiguous store of halfwords from multiple strided vectors (immediate index).

ST1H (scalar plus scalar, strided registers): Contiguous store of halfwords from multiple strided vectors (scalar index).

ST1H (scalar plus scalar, tile slice): Contiguous store of halfwords from 16-bit element ZA tile slice.

ST1Q: Contiguous store of quadwords from 128-bit element ZA tile slice.

ST1W (scalar plus immediate, strided registers): Contiguous store of words from multiple strided vectors (immediate index).

ST1W (scalar plus scalar, strided registers): Contiguous store of words from multiple strided vectors (scalar index).

ST1W (scalar plus scalar, tile slice): Contiguous store of words from 32-bit element ZA tile slice.

STMOPA (2-way): Signed 16-bit integer sparse sum of outer products to 32-bit integer, accumulating.

STMOPA (4-way): Signed 8-bit integer sparse sum of outer products to 32-bit integer, accumulating.

STNT1B (scalar plus immediate, strided registers): Contiguous store non-temporal of bytes from multiple strided vectors (immediate index).

STNT1B (scalar plus scalar, strided registers): Contiguous store non-temporal of bytes from multiple strided vectors (scalar index).

STNT1D (scalar plus immediate, strided registers): Contiguous store non-temporal of doublewords from multiple strided vectors (immediate index).

STNT1D (scalar plus scalar, strided registers): Contiguous store non-temporal of doublewords from multiple strided vectors (scalar index).

STNT1H (scalar plus immediate, strided registers): Contiguous store non-temporal of halfwords from multiple strided vectors (immediate index).

STNT1H (scalar plus scalar, strided registers): Contiguous store non-temporal of halfwords from multiple strided vectors (scalar index).

STNT1W (scalar plus immediate, strided registers): Contiguous store non-temporal of words from multiple strided vectors (immediate index).

STNT1W (scalar plus scalar, strided registers): Contiguous store non-temporal of words from multiple strided vectors (scalar index).

[STR \(array vector\)](#): Store ZA array vector.

STR (table): Store ZT0 register.

SUB (to array, array and multiple vectors): Multi-vector subtract from ZA array vectors.

SUB (to array, multiple and single vector): Multi-vector subtract by vector to ZA array vectors.

SUB (to array, multiple vectors): Multi-vector subtract to ZA array vectors.

SUDOT (4-way, multiple and indexed vector): Multi-vector signed by unsigned 8-bit integer dot product by indexed elements to 32-bit integer.

SUDOT (4-way, multiple and single vector): Multi-vector signed by unsigned 8-bit integer dot product by vector to 32-bit integer.

SUMLALL (multiple and indexed vector): Multi-vector signed by unsigned 8-bit integer multiply-add by indexed element to 32-bit integer.

SUMLALL (multiple and single vector): Multi-vector signed by unsigned 8-bit integer multiply-add by vector to 32-bit integer.

SUMOP4A: Signed by unsigned integer quarter-tile sum of outer products, accumulating.

SUMOP4S: Signed by unsigned integer quarter-tile sum of outer products, subtracting.

SUMOPA (4-way): Signed by unsigned integer sum of outer products, accumulating.

SUMOPS: Signed by unsigned integer sum of outer products, subtracting.

SUNPK: Unpack and sign-extend multi-vector elements.

SUTMOPA: Signed by unsigned 8-bit integer sparse sum of outer products to 32-bit integer, accumulating.

SUVDOT: Multi-vector signed by unsigned 8-bit integer vertical dot product by indexed element to 32-bit integer.

SVDOT (2-way): Multi-vector signed 16-bit integer vertical dot product by indexed element to 32-bit integer.

SVDOT (4-way): Multi-vector signed integer vertical dot product by indexed element.

UCLAMP: Multi-vector unsigned clamp to minimum/maximum.

UCVTF: Multi-vector unsigned 32-bit integer convert to single-precision.

UDOT (2-way, multiple and indexed vector): Multi-vector unsigned 16-bit integer dot product by indexed element to 32-bit integer.

UDOT (2-way, multiple and single vector): Multi-vector unsigned 16-bit integer dot product by vector to 32-bit integer.

UDOT (2-way, multiple vectors): Multi-vector unsigned 16-bit integer dot product to 32-bit integer.

UDOT (4-way, multiple and indexed vector): Multi-vector unsigned integer dot product by indexed element.

UDOT (4-way, multiple and single vector): Multi-vector unsigned integer dot product by vector.

UDOT (4-way, multiple vectors): Multi-vector unsigned integer dot product.

UMAX (multiple and single vector): Multi-vector unsigned maximum by vector.

UMAX (multiple vectors): Multi-vector unsigned maximum.

UMIN (multiple and single vector): Multi-vector unsigned minimum by vector.

UMIN (multiple vectors): Multi-vector unsigned minimum.

UMLAL (multiple and indexed vector): Multi-vector unsigned 16-bit integer multiply-add by indexed element to 32-bit integer.

UMLAL (multiple and single vector): Multi-vector unsigned 16-bit integer multiply-add by vector to 32-bit integer.

UMLAL (multiple vectors): Multi-vector unsigned 16-bit integer multiply-add to 32-bit integer.

UMLALL (multiple and indexed vector): Multi-vector unsigned integer multiply-add long long by indexed element.

UMLALL (multiple and single vector): Multi-vector unsigned integer multiply-add long long by vector.

UMLALL (multiple vectors): Multi-vector unsigned integer multiply-add long long.

UMLSL (multiple and indexed vector): Multi-vector unsigned 16-bit integer multiply-subtract by indexed element from 32-bit integer.

UMLSL (multiple and single vector): Multi-vector unsigned 16-bit integer multiply-subtract by vector from 32-bit integer.

UMLSL (multiple vectors): Multi-vector unsigned 16-bit integer multiply-subtract from 32-bit integer.

UMLSLL (multiple and indexed vector): Multi-vector unsigned integer multiply-subtract long long by indexed element.

UMLSLL (multiple and single vector): Multi-vector unsigned integer multiply-subtract long long by vector.

UMLSLL (multiple vectors): Multi-vector unsigned integer multiply-subtract long long.

UMOP4A (2-way): Unsigned 16-bit integer quarter-tile sum of outer products to 32-bit integer, accumulating.

UMOP4A (4-way): Unsigned integer quarter-tile sum of outer products, accumulating.

UMOP4S (2-way): Unsigned 16-bit integer quarter-tile sum of outer products to 32-bit integer, subtracting.

UMOP4S (4-way): Unsigned integer quarter-tile sum of outer products, subtracting.

UMOPA (2-way): Unsigned 16-bit integer sum of outer products to 32-bit integer, accumulating.

UMOPA (4-way): Unsigned integer sum of outer products, accumulating.

UMOPS (2-way): Unsigned 16-bit integer sum of outer products to 32-bit integer, subtracting.

UMOPS (4-way): Unsigned integer sum of outer products, subtracting.

UQCVT (four registers): Multi-vector unsigned saturating extract narrow.

UQCVT (two registers): Multi-vector unsigned 32-bit integer saturating extract narrow to 16-bit integer.

UQCVTN: Multi-vector unsigned saturating extract narrow to interleaved integer.

UQRSHR (four registers): Multi-vector unsigned saturating rounding shift right narrow by immediate.

UQRSHR (two registers): Multi-vector unsigned 32-bit integer saturating rounding shift right narrow by immediate to 16-bit integer.

UQRSHRN: Multi-vector unsigned saturating rounding shift right narrow by immediate to interleaved integer.

URSHL (multiple and single vector): Multi-vector unsigned rounding shift left by vector.

URSHL (multiple vectors): Multi-vector unsigned rounding shift left.

USDOT (4-way, multiple and indexed vector): Multi-vector unsigned by signed 8-bit integer dot product by indexed element to 32-bit integer.

USDOT (4-way, multiple and single vector): Multi-vector unsigned by signed 8-bit integer dot product by vector to 32-bit integer.

USDOT (4-way, multiple vectors): Multi-vector unsigned by signed 8-bit integer dot product to 32-bit integer.

USMLALL (multiple and indexed vector): Multi-vector unsigned by signed 8-bit integer multiply-add by indexed element to 32-bit integer.

USMLALL (multiple and single vector): Multi-vector unsigned by signed 8-bit integer multiply-add by vector to 32-bit integer.

USMLALL (multiple vectors): Multi-vector unsigned by signed 8-bit integer multiply-add to 32-bit integer.

USMOP4A: Unsigned by signed integer quarter-tile sum of outer products, accumulating.

USMOP4S: Unsigned by signed integer quarter-tile sum of outer products, subtracting.

USMOPA (4-way): Unsigned by signed integer sum of outer products, accumulating.

USMOPS: Unsigned by signed integer sum of outer products, subtracting.

USTMOPA: Unsigned by signed 8-bit integer sparse sum of outer products to 32-bit integer, accumulating.

USVDOT: Multi-vector unsigned by signed 8-bit integer vertical dot product by indexed element to 32-bit integer.

UTMOPA (2-way): Unsigned 16-bit integer sparse sum of outer products to 32-bit integer, accumulating.

UTMOPA (4-way): Unsigned 8-bit integer sparse sum of outer products to 32-bit integer, accumulating.

UUNPK: Unpack and zero-extend multi-vector elements.

UVDOT (2-way): Multi-vector unsigned 16-bit integer vertical dot product by indexed element to 32-bit integer.

UVDOT (4-way): Multi-vector unsigned integer vertical dot product by indexed element.

UZP (four registers): Concatenate elements from four vectors.

UZP (two registers): Concatenate elements from two vectors.

ZERO (double-vector): Zero ZA double-vector groups.

ZERO (quad-vector): Zero ZA quad-vector groups.

ZERO (single-vector): Zero ZA single-vector groups.

ZERO (table): Zero ZT0.

ZERO (tiles): Zero a list of 64-bit element ZA tiles.

ZIP (four registers): Interleave elements from four vectors.

ZIP (two registers): Interleave elements from two vectors.

(old)	htmldiff from-	(new)
-------	----------------	-------

LDR (array vector)

Load ZA array vector

This instruction performs a contiguous load of bytes to a ZA array vector from the memory address generated by a 64-bit scalar base plus an optional immediate offset multiplied by the current vector length in bytes. The ZA array vector is selected by the sum of the vector select register and the same immediate offset, modulo the number of bytes in a Streaming SVE vector.

This instruction is unpredicated.

The load is performed as contiguous byte accesses, with no endian conversion and no guarantee of single-copy atomicity larger than a byte. However, if alignment is checked, then the base register must be aligned to 16 bytes.

This instruction does not require the PE to be in Streaming SVE mode, and it is expected that this instruction will not experience a significant slowdown due to contention with other PEs that are executing in Streaming SVE mode.

SME

(FEAT_SME)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	0	0	1	0	0	0	0	0	0	0	0	0	Rv	0	0	0	Rn			0	off4						
op																															

Encoding

```
LDR ZA[<Wv>, <offs>], [<Xn|SP>{, #<offs>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let v : integer = UInt('011'::Rv);
let offset : integer = UInt(off4);
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W12-W15, encoded in the "Rv" field.
<offs>	Is the vector select offset and optional memory offset, in the range 0 to 15, defaulting to 0, encoded in the "off4" field.
<Xn SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
CheckSMEAndZEnabled();
let SVL : integer{} = CurrentSVL();
let dim : integer{} = SVL DIV 8;
var base : bits(64);
let moffs : integer = offset * dim;
var result : bits(SVL);
let vbase : bits(32) = X{}(v);
let vec : integer = (UInt(vbase) + offset) MOD dim;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSME(MemOp_LOAD, nontemporal, contiguous, tagchecked);

if n == 31 then
    CheckSPAlignment();
    base = SP{64}();
else
    base = X{64}(n);
end;

var addr : bits(64) = AddressAdd(base, moffs, accdesc);

let aligned : boolean = IsAlignedSize(baseaddr, 16);

if !aligned && AlignmentEnforced() then
    let fault : FaultRecord = AlignmentFault(accdesc, addr);
    AArch64_Abort(fault);
end;

for e = 0 to dim-1 do
    result[e*8] = AArch64\_MemSingle{8}(addr, accdesc, aligned);
    addr = AddressIncrement(addr, 1, accdesc);
end;

ZAVector{SVL}(vec) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2025-12-re2025-09-diff-tag2025-12-102025-11-2816:41:29:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)	htmldiff from-	(new)
-------	----------------	-------

LUTI6 (vector, 16-bit)

Lookup table read with 6-bit indices (16-bit)

This instruction copies indexed 16-bit elements from the low 512 bits of the two table vectors to four destination vectors using packed 6-bit indices in the low or high one and a half vector segment of the two source vectors. The source vectors segment consumed in order to fill the destination vectors is selected by the source vector segment index. This instruction is unpredicated. This instruction requires that the Effective SVE vector length is at least 512 bits.

It has encodings from 2 classes: [Consecutive](#) and [Strided](#)

Consecutive

(FEAT_SME2p3)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	i1	1	Zm					1	1	1	1	0	1	Zn					Zd			0	0

Encoding

```
LUTI6 { <Zd1>.H-<Zd4>.H }, { <Zn1>.H, <Zn2>.H }, { <Zm1>-<Zm2> }[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p3) then EndOfDecode(Decode_UNDEF); end;
if MaxImplementedSVL() < 512 then EndOfDecode(MaxImplementedAnyVL() < 512 then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 16;
let isize : integer{} = 6;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let dstride : integer = 1;
let d : integer = UInt(Zd::'00');
let select : integer = UInt(i1);
```

Strided

(FEAT_SME2p3)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	i1	1	Zm					1	1	1	1	1	1	Zn					D	0	0	Zd	

Encoding

```
LUTI6 { <Zd1>.H, <Zd2>.H, <Zd3>.H, <Zd4>.H }, { <Zn1>.H, <Zn2>.H }, { <Zm1>-<Zm2> }[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p3) then EndOfDecode(Decode_UNDEF); end;
if MaxImplementedSVL() < 512 then EndOfDecode(MaxImplementedAnyVL() < 512 then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 16;
let isize : integer{} = 6;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let dstride : integer = 4;
let d : integer = UInt(D::'00'::Zd);
let select : integer = UInt(i1);
```

Assembler Symbols

<Zd1>	For the "Consecutive" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 4. For the "Strided" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 of the destination multi-vector group, encoded as "D:'00':Zd".
<Zd4>	For the "Consecutive" variant: is the name of the fourth scalable vector register of the destination multi-vector group, encoded as "Zd" times 4 plus 3. For the "Strided" variant: is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 of the destination multi-vector group, encoded as "D:'11':Zd".
<Zn1>	Is the name of the first table vector register, encoded as "Zn".
<Zn2>	Is the name of the second table vector register, encoded as "Zn" plus 1 modulo 32.
<Zm1>	Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zm".
<Zm2>	Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zm" plus 1 modulo 32.
<index>	Is the vector segment index, in the range 0 to 1, encoded in the "i1" field.

- <Zd2>Is the name of the second scalable vector register Z4-Z7 or Z20-Z23 of the destination multi-vector group, encoded as "D:'01':Zd".
- <Zd3>Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 of the destination multi-vector group, encoded as "D:'10':Zd".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
if VL < 512 then EndOfDecode(Decode_UNDEF); end;
let elements : integer = VL DIV esize;
let operand : bits(2*VL) = Z{VL}((m+1) MOD 32) :: Z{VL}(m+0);
let idxsize : integer{} = VL + (VL DIV 2);
let idxpos : integer{} = if select == 1 then (VL DIV 2) else 0;
let indexes : bits(idxsize) = operand[idxpos+:idxsize];
let table : bits(1024) = Z{VL}((n+1) MOD 32) [511:0] :: Z{VL}(n+0) [511:0];
var dst : integer = d;
var result : bits(VL);

for r = 0 to 3 do
  let base : integer = r * elements;
  for e = 0 to elements-1 do
    let index : integer = UInt(indexes[(base+e)*:isize]);
    result[e*:esize] = table[index*:esize];
  end;
  Z{VL}(dst) = result;
  dst = dst + dstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2025-12-16 16:41:29:4505 rel2025-09-diff-tag 2025-12-102025-11-28

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)	htmldiff from-	(new)
-------	----------------	-------

SEL

Multi-vector conditional select

This instruction selects consecutive elements from the two or four first source vectors where **vector select** predicate elements are true, and places them in the corresponding elements of the two or four destination vectors. **Where the vector select predicate elements are false, the corresponding consecutive elements are selected** from the two or four second source vectors **and** are placed in the remaining elements of the destination vectors.

It has encodings from 2 classes: [Two registers](#) and [Four registers](#)

Two registers

(FEAT_SME2)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size	1	Zm			0	1	0	0	PNvPNg		Zn			0	Zd			0					

Encoding

```
SEL { <Zd1>.<T>-<Zd2>.<T> }, <PNv><PNg>, { <Zn1>.<T>-<Zn2>.<T> }, { <Zm1>.<T>-<Zm2>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm::'0');
let d : integer = UInt(Zd::'0');
let vg : integer = UInt('1'::PNvPNg);
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size	1		Zm	0	1	1	0	0			PNv	PNg			Zn	0	0		Zd	0	0		

Encoding

```
SEL { <Zd1>.<T>-<Zd4>.<T> }, <PNv><PNg>, { <Zn1>.<T>-<Zn4>.<T> }, { <Zm1>.<T>-<Zm4>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn::'00');
let m : integer = UInt(Zm::'00');
let d : integer = UInt(Zd::'00');
let vg : integer = UInt('1'::PNvPNg);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zd1> For the "Two registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 4.

<T> Is the size specifier, encoded in "size":

size	<T>
00	B
01	H
10	S
11	D

<Zd2> Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1.

<PNvPNg> Is the name of the **vector governing selectscalable** predicate register PN8-PN15, with predicate-as-counter encoding, encoded in the "PNvPNg" field.

<Zn1> For the "Two registers" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.

<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm1>	For the "Two registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.
<Zd4>	Is the name of the fourth scalable vector register of the destination multi-vector group, encoded as "Zd" times 4 plus 3.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.
<Zm4>	Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```

CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var results : array[[4]] of bits(VL);
let pred : bits(PL) = P{} (vg);
let mask : bits(PL * nreg) = CounterToPredicate{}(pred[15:0]);

for r = 0 to nreg-1 do
  let operand1 : bits(VL) = Z{}(n+r);
  let operand2 : bits(VL) = Z{}(m+r);
  for e = 0 to elements-1 do
    if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
      results[[r]][e*:esize] = operand1[e*:esize];
    else
      results[[r]][e*:esize] = operand2[e*:esize];
    end;
  end;
end;

for r = 0 to nreg-1 do
  Z{VL}(d+r) = results[[r]];
end;

```

Operational information

This instruction is a data-independent-time instruction as described in [About PSTATE.DIT](#).

2025-12_rel2025-09_diff_tag 2025-12-102025-11-28 16:4129:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)	htmldiff from-	(new)
-------	----------------	-------

STR (array vector)

Store ZA array vector

This instruction performs a contiguous store of bytes from a ZA array vector to the memory address generated by a 64-bit scalar base plus an optional immediate offset multiplied by the current vector length in bytes. The ZA array vector is selected by the sum of the vector select register and the same immediate offset, modulo the number of bytes in a Streaming SVE vector.

This instruction is unpredicated.

The store is performed as contiguous byte accesses, with no endian conversion and no guarantee of single-copy atomicity larger than a byte. However, if alignment is checked, then the base register must be aligned to 16 bytes.

This instruction does not require the PE to be in Streaming SVE mode, and it is expected that this instruction will not experience a significant slowdown due to contention with other PEs that are executing in Streaming SVE mode.

SME

(FEAT_SME)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	0	0	1	0	0	1	0	0	0	0	0	0	Rv	0	0	0	Rn			0			off4			0	
op																															

op

Encoding

```
STR ZA[<Wv>, <offs>], [<Xn|SP>{, #<offs>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let v : integer = UInt('011'::Rv);
let offset : integer = UInt(off4);
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W12-W15, encoded in the "Rv" field.
<offs>	Is the vector select offset and optional memory offset, in the range 0 to 15, defaulting to 0, encoded in the "off4" field.
<Xn SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
CheckSMEAndZEnabled();
let SVL : integer{} = CurrentSVL();
let dim : integer{} = SVL DIV 8;
var base : bits(64);
let moffs : integer = offset * dim;
var src : bits(SVL);
let vbase : bits(32) = X{}(v);
let vec : integer = (UInt(vbase) + offset) MOD dim;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSME(MemOp_STORE, nontemporal, contiguous, tagchecked);

if n == 31 then
    CheckSPAlignment();
    base = SP{64}();
else
    base = X{64}(n);
end;

src = ZAVector{SVL}(vec);

var addr : bits(64) = AddressAdd(base, moffs, accdesc);

let aligned : boolean = IsAlignedSize(baseaddr, 16);

if !aligned && AlignmentEnforced() then
    let fault : FaultRecord = AlignmentFault(accdesc, addr);
    AArch64_Abort(fault);
end;

for e = 0 to dim-1 do
    AArch64\_MemSingle{8}(addr, accdesc, aligned) = src[e*8];
    addr = AddressIncrement(addr, 1, accdesc);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2025-12-re2025-09-diff-tag2025-12-102025-11-2816:41:29:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old)	htmldiff from-	(new)
-------	----------------	-------

A64 instruction set encoding

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
op0																															

Decode fields

Instruction details

op0	op1	
0	0000	Reserved
1	0000	SME encodings
	0010	SVE encodings
	00x1	UNALLOCATED
	100x	Data Processing -- Immediate
	101x	Branches, Exception Generating and System instructions
	x101	Data Processing -- Register
	x111	Data Processing -- Scalar Floating-Point and Advanced SIMD
	x1x0	Loads and Stores

Reserved

The encodings in this section are decoded from [A64 instruction set encoding](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	op0	0	0	0	0																										

Decode fields

Instruction details

op0	op1	
00	000000000	UDF
00	!= 000000000	UNALLOCATED
!= 00		UNALLOCATED

SME encodings

The encodings in this section are decoded from [A64 instruction set encoding](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	op0	0	0	0	0																										

Decode fields			Instruction details
op0	op1	op2	
00	x00xxxxx0x00000	0xx0xx	SME2 Quarter Tile Outer Product - 16-bit and 32-bit
00	x00xxxxx0x00000	1xx0xx	UNALLOCATED
00	x00xxxxx0x00001	xxx0xx	UNALLOCATED
00	x00xxxxx0x0001x	xxx0xx	UNALLOCATED
00	x00xxxxx0x001xx	xxx0xx	UNALLOCATED
00	x00xxxxx0x01xxx	xxx0xx	UNALLOCATED
00	x00xxxxx1x0xxxx	xxx0xx	UNALLOCATED
00	x01xxxxxxxx0xxxx	xxx0xx	SME2 Sparse Outer Product
00	x0xxxxxxxxx0xxxx	xxx1xx	UNALLOCATED
00	x0xxxxxxxxx1xxxx		UNALLOCATED
00	x10xxxxxxxxxxxxxx	xx00xx	SME FP Outer Product - 32 bit
00	x10xxxxxxxxxxxxxx	xx10xx	SME2 Outer Product - Misc
01	00xxxxxxxxxxxxxx		SME2 Multi-vector - Memory (Contiguous)
01	10xxxxxxxxxxxxxx		SME2 Multi-vector - Memory (Strided)
01	x10xxxxxxxxxxxxxx	xxx0xx	SME Integer Outer Product - 32 bit
0x	x10xxxxxxxxxxxxxx	xxx1xx	UNALLOCATED
0x	x11xxxxx0000000	0x1xxx	SME2 Quarter Tile Outer Product - 64-bit
0x	x11xxxxx0000000	1x1xxx	UNALLOCATED
0x	x11xxxxx0000001	xx1xxx	UNALLOCATED
0x	x11xxxxx000001x	xx1xxx	UNALLOCATED
0x	x11xxxxx00001xx	xx1xxx	UNALLOCATED
0x	x11xxxxx0001xxx	xx1xxx	UNALLOCATED
0x	x11xxxxx001xxxx	xx1xxx	UNALLOCATED
0x	x11xxxxx01xxxxx	xx1xxx	UNALLOCATED
0x	x11xxxxx1xxxxxx	xx1xxx	UNALLOCATED
0x	x11xxxxxxxxxxxxxx	xx0xxx	SME Outer Product - 64 bit
10	0000010xxxxxxxxx		SME zero array
10	0000011xxxxxxxxx		SME2 Multiple Zero
10	0010010xxxxxxxxx		SME2 zero lookup table
10	0010011xxxxxxxxx		SME2 Move Lookup Table
10	00x011xxxxxxxxx		UNALLOCATED
10	010011xxxxxxxxx		SME2 Expand Lookup Table (Non-contiguous)
10	011011xxxxxxxxx		UNALLOCATED
10	01x001xxxxxxxxx		SME2 Expand Lookup Table (Contiguous)
10	0xx000x0xxxxxxxx	x0xxxx	SME Move into Array
10	0xx000x0xxxxxxxx	x1xxxx	UNALLOCATED
10	0xx000x1xxxxxxxx		SME Move from Array
10	0xx010xxxxxxxxxx	xx0xxx	SME Add Vector to Array
10	0xx010xxxxxxxxxx	xx1xxx	UNALLOCATED
10	0xx1xxxxxxxxxxxxx		UNALLOCATED
10	10x10xxxx0xxxxxx		SME2 Multi-vector - Multiple and Single Array Vectors (Two registers)
10	10x11xxxx0xxxxxx		SME2 Multi-vector - Multiple and Single Array Vectors (Four registers)
10	10x1xxxxx1111xx		SME2 Multi-vector - Lookup table with 6-bit indices
10	11x1xxxx00xxxxxx		SME2 Multi-vector - Multiple Array Vectors (Two registers)
10	11x1xxxx10xxxxxx		SME2 Multi-vector - Multiple Array Vectors (Four registers)

10	11x1xxxxx1111xx		UNALLOCATED
10	1xx00xxxxxxxxxxx		SME2 Multi-vector - Indexed (One register)
10	1xx01xxxx0xxxxx		SME2 Multi-vector - Indexed (Two registers)
10	1xx01xxxx1xxxxx		SME2 Multi-vector - Indexed (Four registers)
10	1xx10xxxx10100x		SME2 Multi-vector - Multiple and Single SVE Destructive (Two registers)
10	1xx10xxxx10101x	xxxx0x	SME2 Multi-vector - Multiple and Single SVE Destructive (Four registers)
10	1xx10xxxx10101x	xxxx1x	UNALLOCATED
10	1xx11xxxx1010xx		UNALLOCATED
10	1xx1xxx00101110	xxxx0x	SME2 Multi-vector - Multiple Vectors SVE Destructive (Four registers)
10	1xx1xxx00101111	xxxx0x	SME2 Multi-vector - Multiple Vectors SVE Saturating Multiply (Four registers)
10	1xx1xxx1010111x	xxxx0x	UNALLOCATED
10	1xx1xxxx0101100		SME2 Multi-vector - Multiple Vectors SVE Destructive (Two registers)
10	1xx1xxxx0101101		SME2 Multi-vector - Multiple Vectors SVE Saturating Multiply (Two registers)
10	1xx1xxxx010111x	xxxx1x	UNALLOCATED
10	1xx1xxxx11011xx		UNALLOCATED
10	1xx1xxxxx100xxx		SME2 Multi-vector - SVE Select
10	1xx1xxxxx110xxx		SME2 Multi-vector - SVE Constructive Binary
10	1xx1xxxxx111000		SME2 Multi-vector - SVE Constructive Unary
10	1xx1xxxxx111001	0xxxx0	SME2 Multi-vector - FP Multiply
10	1xx1xxxxx111001	1xxxxx	UNALLOCATED
10	1xx1xxxxx111001	!= 0xxxx00xxxx1	UNALLOCATED
10	1xx1xxxxx111010	0xxxx0	SME2 Multiple and Single Vector - FP multiply
10	1xx1xxxxx111010	1xxxxx	UNALLOCATED
10	1xx1xxxxx111010	!= 0xxxx00xxxx1	UNALLOCATED
10	1xx1xxxxx111011		UNALLOCATED
11			SME Memory

SME2 Quarter Tile Outer Product - 16-bit and 32-bit

The encodings in this section are decoded from [SME encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
1	0	0	0	0	0	0	op0	0	0	op1					0	op2	0	0	0	0	0					0	op3		0	op4			

Decode fields					Instruction details
op0	op1	op2	op3	op4	
0	0	0	x0		SME2 FP32 non-widening quarter tile outer product
0	0	0	x1	0	UNALLOCATED
0	1	0	00		SME2 FP8 to FP32 quarter tile outer product
0	1	0	01	0	SME2 FP8 to FP16 quarter tile outer product
0	1	0	10	1	UNALLOCATED
0	1	0	1x	0	UNALLOCATED
1	0	0	x0		SME2 BF16 to FP32 quarter tile outer product
1	0	0	x1	0	SME2 FP16 non-widening quarter tile outer product
1	1	0	x0		SME2 FP16 to FP32 quarter tile outer product
1	1	0	x1	0	SME2 BF16 non-widening quarter tile outer product
	0	1	x1		SME2 Int16 to Int32 quarter tile outer product
	1	1	x1		UNALLOCATED
		0	x1	1	UNALLOCATED
		1	x0		SME2 Int8 to Int32 quarter tile outer product

SME2 FP32 non-widening quarter tile outer product

The encodings in this section are decoded from [SME2 Quarter Tile Outer Product - 16-bit and 32-bit](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	0	0	0	0	M		Zm		0	0	0	0	0	0	0	N		Zn		0	S	0	0	Z	Ada

Decode fields			Instruction Details	Feature
M	N	S		
0	0	0	FMOP4A (non-widening) — Single-precision, single vectors	FEAT_SME_MOP4
0	0	1	FMOP4S (non-widening) — Single-precision, single vectors	FEAT_SME_MOP4
0	1	0	FMOP4A (non-widening) — Single-precision, multiple and single vectors	FEAT_SME_MOP4
0	1	1	FMOP4S (non-widening) — Single-precision, multiple and single vectors	FEAT_SME_MOP4
1	0	0	FMOP4A (non-widening) — Single-precision, single and multiple vectors	FEAT_SME_MOP4
1	0	1	FMOP4S (non-widening) — Single-precision, single and multiple vectors	FEAT_SME_MOP4
1	1	0	FMOP4A (non-widening) — Single-precision, multiple vectors	FEAT_SME_MOP4
1	1	1	FMOP4S (non-widening) — Single-precision, multiple vectors	FEAT_SME_MOP4

SME2 FP8 to FP32 quarter tile outer product

The encodings in this section are decoded from [SME2 Quarter Tile Outer Product - 16-bit and 32-bit](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	0	0	0	1	M		Zm		0	0	0	0	0	0	0	N		Zn		0	0	0	0	Z	Ada

Decode fields		Instruction Details	Feature
M	N		
0	0	FMOP4A (widening, 4-way) — Single vectors	FEAT_SME_MOP4 && FEAT_SME_F8F32
0	1	FMOP4A (widening, 4-way) — Multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_F8F32
1	0	FMOP4A (widening, 4-way) — Single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_F8F32
1	1	FMOP4A (widening, 4-way) — Multiple vectors	FEAT_SME_MOP4 && FEAT_SME_F8F32

SME2 FP8 to FP16 quarter tile outer product

The encodings in this section are decoded from [SME2 Quarter Tile Outer Product - 16-bit and 32-bit](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	0	0	0	1	M		Zm		0	0	0	0	0	0	0	N		Zn		0	0	1	0	0	ZA

Decode fields		Instruction Details	Feature
M	N		
0	0	FMOP4A (widening, 2-way, FP8 to FP16) — Single vectors	FEAT_SME_MOP4 && FEAT_SME_F8F16
0	1	FMOP4A (widening, 2-way, FP8 to FP16) — Multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_F8F16
1	0	FMOP4A (widening, 2-way, FP8 to FP16) — Single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_F8F16
1	1	FMOP4A (widening, 2-way, FP8 to FP16) — Multiple vectors	FEAT_SME_MOP4 && FEAT_SME_F8F16

SME2 BF16 to FP32 quarter tile outer product

The encodings in this section are decoded from [SME2 Quarter Tile Outer Product - 16-bit and 32-bit](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	1	0	0	0	M		Zm		0	0	0	0	0	0	0	N		Zn		0	S	0	0	ZAda	

Decode fields			Instruction Details	Feature
M	N	S		
0	0	0	BFMOP4A (widening) — Single vectors	FEAT_SME_MOP4
0	0	1	BFMOP4S (widening) — Single vectors	FEAT_SME_MOP4
0	1	0	BFMOP4A (widening) — Multiple and single vectors	FEAT_SME_MOP4
0	1	1	BFMOP4S (widening) — Multiple and single vectors	FEAT_SME_MOP4
1	0	0	BFMOP4A (widening) — Single and multiple vectors	FEAT_SME_MOP4
1	0	1	BFMOP4S (widening) — Single and multiple vectors	FEAT_SME_MOP4
1	1	0	BFMOP4A (widening) — Multiple vectors	FEAT_SME_MOP4
1	1	1	BFMOP4S (widening) — Multiple vectors	FEAT_SME_MOP4

SME2 FP16 non-widening quarter tile outer product

The encodings in this section are decoded from [SME2 Quarter Tile Outer Product - 16-bit and 32-bit](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	1	0	0	0	M		Zm		0	0	0	0	0	0	0	N		Zn		0	S	1	0	0	ZA

Decode fields			Instruction Details	Feature
M	N	S		
0	0	0	FMOP4A (non-widening) — Half-precision, single vectors	FEAT_SME_MOP4 && FEAT_SME_F16F16
0	0	1	FMOP4S (non-widening) — Half-precision, single vectors	FEAT_SME_MOP4 && FEAT_SME_F16F16
0	1	0	FMOP4A (non-widening) — Half-precision, multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_F16F16
0	1	1	FMOP4S (non-widening) — Half-precision, multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_F16F16
1	0	0	FMOP4A (non-widening) — Half-precision, single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_F16F16
1	0	1	FMOP4S (non-widening) — Half-precision, single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_F16F16
1	1	0	FMOP4A (non-widening) — Half-precision, multiple vectors	FEAT_SME_MOP4 && FEAT_SME_F16F16
1	1	1	FMOP4S (non-widening) — Half-precision, multiple vectors	FEAT_SME_MOP4 && FEAT_SME_F16F16

SME2 FP16 to FP32 quarter tile outer product

The encodings in this section are decoded from [SME2 Quarter Tile Outer Product - 16-bit and 32-bit](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	1	0	0	1	M		Zm		0	0	0	0	0	0	0	N		Zn		0	S	0	0	ZAda	

Decode fields			Instruction Details	Feature
M	N	S		
0	0	0	FMOP4A (widening, 2-way, FP16 to FP32) — Single vectors	FEAT_SME_MOP4
0	0	1	FMOP4S (widening) — Single vectors	FEAT_SME_MOP4
0	1	0	FMOP4A (widening, 2-way, FP16 to FP32) — Multiple and single vectors	FEAT_SME_MOP4
0	1	1	FMOP4S (widening) — Multiple and single vectors	FEAT_SME_MOP4
1	0	0	FMOP4A (widening, 2-way, FP16 to FP32) — Single and multiple vectors	FEAT_SME_MOP4
1	0	1	FMOP4S (widening) — Single and multiple vectors	FEAT_SME_MOP4
1	1	0	FMOP4A (widening, 2-way, FP16 to FP32) — Multiple vectors	FEAT_SME_MOP4
1	1	1	FMOP4S (widening) — Multiple vectors	FEAT_SME_MOP4

SME2 BF16 non-widening quarter tile outer product

The encodings in this section are decoded from [SME2 Quarter Tile Outer Product - 16-bit and 32-bit](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	1	0	0	1	M		Zm		0	0	0	0	0	0	0	N		Zn		0	S	1	0	0	ZA

Decode fields			Instruction Details	Feature
M	N	S		
0	0	0	BFMOP4A (non-widening) — Single vectors	FEAT_SME_MOP4 && FEAT_SME_B16B16
0	0	1	BFMOP4S (non-widening) — Single vectors	FEAT_SME_MOP4 && FEAT_SME_B16B16
0	1	0	BFMOP4A (non-widening) — Multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_B16B16
0	1	1	BFMOP4S (non-widening) — Multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_B16B16
1	0	0	BFMOP4A (non-widening) — Single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_B16B16
1	0	1	BFMOP4S (non-widening) — Single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_B16B16
1	1	0	BFMOP4A (non-widening) — Multiple vectors	FEAT_SME_MOP4 && FEAT_SME_B16B16
1	1	1	BFMOP4S (non-widening) — Multiple vectors	FEAT_SME_MOP4 && FEAT_SME_B16B16

SME2 Int16 to Int32 quarter tile outer product

The encodings in this section are decoded from [SME2 Quarter Tile Outer Product - 16-bit and 32-bit](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	u0	0	0	0	M	Zm		0		1	0	0	0	0	0	N	Zn		0		S	1	0	ZAda	

Decode fields				Instruction Details	Feature
u0	M	N	S		
0	0	0	0	SMOP4A (2-way) — Single vectors	FEAT_SME_MOP4
0	0	0	1	SMOP4S (2-way) — Single vectors	FEAT_SME_MOP4
0	0	1	0	SMOP4A (2-way) — Multiple and single vectors	FEAT_SME_MOP4
0	0	1	1	SMOP4S (2-way) — Multiple and single vectors	FEAT_SME_MOP4
0	1	0	0	SMOP4A (2-way) — Single and multiple vectors	FEAT_SME_MOP4
0	1	0	1	SMOP4S (2-way) — Single and multiple vectors	FEAT_SME_MOP4
0	1	1	0	SMOP4A (2-way) — Multiple vectors	FEAT_SME_MOP4
0	1	1	1	SMOP4S (2-way) — Multiple vectors	FEAT_SME_MOP4
1	0	0	0	UMOP4A (2-way) — Single vectors	FEAT_SME_MOP4
1	0	0	1	UMOP4S (2-way) — Single vectors	FEAT_SME_MOP4
1	0	1	0	UMOP4A (2-way) — Multiple and single vectors	FEAT_SME_MOP4
1	0	1	1	UMOP4S (2-way) — Multiple and single vectors	FEAT_SME_MOP4
1	1	0	0	UMOP4A (2-way) — Single and multiple vectors	FEAT_SME_MOP4
1	1	0	1	UMOP4S (2-way) — Single and multiple vectors	FEAT_SME_MOP4
1	1	1	0	UMOP4A (2-way) — Multiple vectors	FEAT_SME_MOP4
1	1	1	1	UMOP4S (2-way) — Multiple vectors	FEAT_SME_MOP4

SME2 Int8 to Int32 quarter tile outer product

The encodings in this section are decoded from [SME2 Quarter Tile Outer Product - 16-bit and 32-bit](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	u0	0	0	u1	M		Zm		0	1	0	0	0	0	0	N		Zn		0	S	0	0	ZAda	

Decode fields					Instruction Details	Feature
u0	u1	M	N	S		
0	0	0	0	0	SMOP4A (4-way) — 32-bit, single vectors	FEAT_SME_MOP4
0	0	0	0	1	SMOP4S (4-way) — 32-bit, single vectors	FEAT_SME_MOP4
0	0	0	1	0	SMOP4A (4-way) — 32-bit, multiple and single vectors	FEAT_SME_MOP4
0	0	0	1	1	SMOP4S (4-way) — 32-bit, multiple and single vectors	FEAT_SME_MOP4
0	0	1	0	0	SMOP4A (4-way) — 32-bit, single and multiple vectors	FEAT_SME_MOP4
0	0	1	0	1	SMOP4S (4-way) — 32-bit, single and multiple vectors	FEAT_SME_MOP4
0	0	1	1	0	SMOP4A (4-way) — 32-bit, multiple vectors	FEAT_SME_MOP4
0	0	1	1	1	SMOP4S (4-way) — 32-bit, multiple vectors	FEAT_SME_MOP4
0	1	0	0	0	SUMOP4A — 32-bit, single vectors	FEAT_SME_MOP4
0	1	0	0	1	SUMOP4S — 32-bit, single vectors	FEAT_SME_MOP4
0	1	0	1	0	SUMOP4A — 32-bit, multiple and single vectors	FEAT_SME_MOP4
0	1	0	1	1	SUMOP4S — 32-bit, multiple and single vectors	FEAT_SME_MOP4
0	1	1	0	0	SUMOP4A — 32-bit, single and multiple vectors	FEAT_SME_MOP4
0	1	1	0	1	SUMOP4S — 32-bit, single and multiple vectors	FEAT_SME_MOP4
0	1	1	1	0	SUMOP4A — 32-bit, multiple vectors	FEAT_SME_MOP4
0	1	1	1	1	SUMOP4S — 32-bit, multiple vectors	FEAT_SME_MOP4
1	0	0	0	0	USMOP4A — 32-bit, single vectors	FEAT_SME_MOP4
1	0	0	0	1	USMOP4S — 32-bit, single vectors	FEAT_SME_MOP4
1	0	0	1	0	USMOP4A — 32-bit, multiple and single vectors	FEAT_SME_MOP4
1	0	0	1	1	USMOP4S — 32-bit, multiple and single vectors	FEAT_SME_MOP4
1	0	1	0	0	USMOP4A — 32-bit, single and multiple vectors	FEAT_SME_MOP4
1	0	1	0	1	USMOP4S — 32-bit, single and multiple vectors	FEAT_SME_MOP4
1	0	1	1	0	USMOP4A — 32-bit, multiple vectors	FEAT_SME_MOP4
1	0	1	1	1	USMOP4S — 32-bit, multiple vectors	FEAT_SME_MOP4
1	1	0	0	0	UMOP4A (4-way) — 32-bit, single vectors	FEAT_SME_MOP4
1	1	0	0	1	UMOP4S (4-way) — 32-bit, single vectors	FEAT_SME_MOP4
1	1	0	1	0	UMOP4A (4-way) — 32-bit, multiple and single vectors	FEAT_SME_MOP4
1	1	0	1	1	UMOP4S (4-way) — 32-bit, multiple and single vectors	FEAT_SME_MOP4
1	1	1	0	0	UMOP4A (4-way) — 32-bit, single and multiple vectors	FEAT_SME_MOP4
1	1	1	0	1	UMOP4S (4-way) — 32-bit, single and multiple vectors	FEAT_SME_MOP4
1	1	1	1	0	UMOP4A (4-way) — 32-bit, multiple vectors	FEAT_SME_MOP4
1	1	1	1	1	UMOP4S (4-way) — 32-bit, multiple vectors	FEAT_SME_MOP4

SME2 Sparse Outer Product

The encodings in this section are decoded from [SME encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	op0	0	1	op1						op2	0	op3									op4	0	op5		

Decode fields						Instruction details	Feature
op0	op1	op2	op3	op4	op5		
0	0	0	0	0		FTMOPA (non-widening) — Single-precision	FEAT_SME_TMOP
0	0	0	0	1	0	UNALLOCATED	-
0	1	0	0	0		FTMOPA (widening, 4-way)	FEAT_SME_TMOP && FEAT_SME_F8F32
0	1	0	0	1	0	FTMOPA (widening, 2-way, FP8 to FP16)	FEAT_SME_TMOP && FEAT_SME_F8F16
1	0	0	0	0		BFTMOPA (widening)	FEAT_SME_TMOP
1	0	0	0	1	0	FTMOPA (non-widening) — Half-precision	FEAT_SME_TMOP && FEAT_SME_F16F16
1	1	0	0	0		FTMOPA (widening, 2-way, FP16 to FP32)	FEAT_SME_TMOP
1	1	0	0	1	0	BFTMOPA (non-widening)	FEAT_SME_TMOP && FEAT_SME_B16B16
	0	1	0	1		SME2 Int16 to Int32 sparse outer product	-
	1	1	0	1		UNALLOCATED	-
		0	0	1	1	UNALLOCATED	-
		1	0	0		SME Int8 to Int32 sparse outer product	-
			1			UNALLOCATED	-

SME2 Int16 to Int32 sparse outer product

The encodings in this section are decoded from [SME2 Sparse Outer Product](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	u0	0	1	0	Zm				1	0	0	K	Zk	Zn				i2	1	0	ZAda				

Decode fields		Instruction Details	Feature
u0			
0		STMOPA (2-way)	FEAT_SME_TMOP
1		UTMOPA (2-way)	FEAT_SME_TMOP

SME Int8 to Int32 sparse outer product

The encodings in this section are decoded from [SME2 Sparse Outer Product](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	u0	0	1	u1	Zm					1	0	0	K	Zk	Zn					i2	0	0	ZAda		

Decode fields		Instruction Details	Feature
u0	u1		
0	0	STMOPA (4-way)	FEAT_SME_TMOP
0	1	SUTMOPA	FEAT_SME_TMOP
1	0	USTMOPA	FEAT_SME_TMOP
1	1	UTMOPA (4-way)	FEAT_SME_TMOP

SME FP Outer Product - 32 bit

The encodings in this section are decoded from [SME encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	op0	1	0	op1																	op2	0	0		

Decode fields			Instruction details	Feature
op0	op1	op2		
0	0		SME FP32 outer product	-
0	1	0	FMOPA (widening, 4-way)	FEAT_SME_F8F32
0	1	1	UNALLOCATED	-
1	0		SME BF16 widening outer product	-
1	1		SME FP16 widening outer product	-

SME FP32 outer product

The encodings in this section are decoded from [SME FP Outer Product - 32 bit](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	0	1	0	0	Zm				Pm				Pn				Zn				S	0	0	ZAda	

Decode fields		Instruction Details	Feature
S			
0		FMOPA (non-widening)	FEAT_SME
1		FMOPS (non-widening)	FEAT_SME

SME BF16 widening outer product

The encodings in this section are decoded from [SME FP Outer Product - 32 bit](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	1	1	0	0	Zm				Pm				Pn				Zn				S	0	0	ZAda	

Decode fields		Instruction Details	Feature
S			
0		BFMOPA (widening)	FEAT_SME
1		BFMOPS (widening)	FEAT_SME

SME FP16 widening outer product

The encodings in this section are decoded from [SME FP Outer Product - 32 bit](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	1	1	0	1	Zm				Pm				Pn				Zn				S	0	0	ZAda	

Decode fields		Instruction Details	Feature
S			
0		FMOPA (widening, 2-way, FP16 to FP32)	FEAT_SME
1		FMOPS (widening)	FEAT_SME

SME2 Outer Product - Misc

The encodings in this section are decoded from [SME encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	op0	1	0	op1																	op2	1	0	op3	

Decode fields				Instruction details	Feature
op0	op1	op2	op3		
0	0			SME2 32-bit binary outer product	-
0	1	0	0	FMOPA (widening, 2-way, FP8 to FP16)	FEAT_SME_F8F16
0	1	1	0	UNALLOCATED	-
0	1		1	UNALLOCATED	-
1	0		0	SME2 FP16 non-widening outer product	-
1	1		0	SME2 BF16 non-widening outer product	-
1			1	UNALLOCATED	-

SME2 32-bit binary outer product

The encodings in this section are decoded from [SME2 Outer Product - Misc.](#)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	0	1	0	0	Zm				Pm		Pn		Zn				S	1	0	ZAda					

Decode fields		Instruction Details	Feature
S			
0		BMOPA	FEAT_SME2
1		BMOPS	FEAT_SME2

SME2 FP16 non-widening outer product

The encodings in this section are decoded from [SME2 Outer Product - Misc.](#)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	1	1	0	0	Zm				Pm			Pn			Zn				S	1	0	0	ZAda		

Decode fields		Instruction Details	Feature
S			
0		FMOPA (non-widening)	FEAT_SME_F16F16
1		FMOPS (non-widening)	FEAT_SME_F16F16

SME2 BF16 non-widening outer product

The encodings in this section are decoded from [SME2 Outer Product - Misc.](#)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	1	1	0	1	Zm				Pm			Pn			Zn				S	1	0	0	ZAda		

Decode fields		Instruction Details	Feature
S			
0		BFMOPA (non-widening)	FEAT_SME_B16B16
1		BFMOPS (non-widening)	FEAT_SME_B16B16

SME2 Multi-vector - Memory (Contiguous)

The encodings in this section are decoded from [SME encodings.](#)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	0	0	0		op0						op1														op2	

Decode fields			Instruction details
op0	op1	op2	
00x	0		SME2 multi-vec contiguous load (scalar plus scalar, two registers)
00x	1	0	SME2 multi-vec contiguous load (scalar plus scalar, four registers)
01x	0		SME2 multi-vec contiguous store (scalar plus scalar, two registers)
01x	1	0	SME2 multi-vec contiguous store (scalar plus scalar, four registers)
0x1	1	1	UNALLOCATED
100	0		SME2 multi-vec contiguous load (scalar plus immediate, two registers)
100	1	0	SME2 multi-vec contiguous load (scalar plus immediate, four registers)
110	0		SME2 multi-vec contiguous store (scalar plus immediate, two registers)
110	1	0	SME2 multi-vec contiguous store (scalar plus immediate, four registers)
1x1			UNALLOCATED
!= 1x1xx0	1	1	UNALLOCATED

SME2 multi-vec contiguous load (scalar plus scalar, two registers)

The encodings in this section are decoded from [SME2 Multi-vector - Memory \(Contiguous\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	0	0	0	0	0	Rm					0	msz	PNg			Rn					Zt			N		

Decode fields		Instruction Details	Feature
msz	N		
00	0	LD1B (scalar plus scalar, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
00	1	LDNT1B (scalar plus scalar, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
01	0	LD1H (scalar plus scalar, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
01	1	LDNT1H (scalar plus scalar, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
10	0	LD1W (scalar plus scalar, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
10	1	LDNT1W (scalar plus scalar, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
11	0	LD1D (scalar plus scalar, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
11	1	LDNT1D (scalar plus scalar, consecutive registers)	FEAT_SME2 FEAT_SVE2p1

SME2 multi-vec contiguous load (scalar plus scalar, four registers)

The encodings in this section are decoded from [SME2 Multi-vector - Memory \(Contiguous\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	0	0	0	0	0	Rm				1	msz	PNg		Rn				Zt		0	N					

Decode fields		Instruction Details	Feature
msz	N		
00	0	LD1B (scalar plus scalar, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
00	1	LDNT1B (scalar plus scalar, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
01	0	LD1H (scalar plus scalar, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
01	1	LDNT1H (scalar plus scalar, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
10	0	LD1W (scalar plus scalar, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
10	1	LDNT1W (scalar plus scalar, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
11	0	LD1D (scalar plus scalar, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
11	1	LDNT1D (scalar plus scalar, consecutive registers)	FEAT_SME2 FEAT_SVE2p1

SME2 multi-vec contiguous store (scalar plus scalar, two registers)

The encodings in this section are decoded from [SME2 Multi-vector - Memory \(Contiguous\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	0	0	0	0	1	Rm				0	msz	PNg				Rn				Zt				N		

Decode fields		Instruction Details	Feature
msz	N		
00	0	ST1B (scalar plus scalar, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
00	1	STNT1B (scalar plus scalar, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
01	0	ST1H (scalar plus scalar, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
01	1	STNT1H (scalar plus scalar, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
10	0	ST1W (scalar plus scalar, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
10	1	STNT1W (scalar plus scalar, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
11	0	ST1D (scalar plus scalar, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
11	1	STNT1D (scalar plus scalar, consecutive registers)	FEAT_SME2 FEAT_SVE2p1

SME2 multi-vec contiguous store (scalar plus scalar, four registers)

The encodings in this section are decoded from [SME2 Multi-vector - Memory \(Contiguous\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	0	0	0	0	1	Rm				1	msz	PNa				Rn				Zt				0	N	

Decode fields		Instruction Details	Feature
msz	N		
00	0	ST1B (scalar plus scalar, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
00	1	STNT1B (scalar plus scalar, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
01	0	ST1H (scalar plus scalar, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
01	1	STNT1H (scalar plus scalar, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
10	0	ST1W (scalar plus scalar, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
10	1	STNT1W (scalar plus scalar, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
11	0	ST1D (scalar plus scalar, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
11	1	STNT1D (scalar plus scalar, consecutive registers)	FEAT_SME2 FEAT_SVE2p1

SME2 multi-vec contiguous load (scalar plus immediate, two registers)

The encodings in this section are decoded from [SME2 Multi-vector - Memory \(Contiguous\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	0	0	0	1	0	0	imm4				0	msz	PNg				Rn				Zt					N

Decode fields		Instruction Details	Feature
msz	N		
00	0	LD1B (scalar plus immediate, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
00	1	LDNT1B (scalar plus immediate, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
01	0	LD1H (scalar plus immediate, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
01	1	LDNT1H (scalar plus immediate, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
10	0	LD1W (scalar plus immediate, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
10	1	LDNT1W (scalar plus immediate, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
11	0	LD1D (scalar plus immediate, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
11	1	LDNT1D (scalar plus immediate, consecutive registers)	FEAT_SME2 FEAT_SVE2p1

SME2 multi-vec contiguous load (scalar plus immediate, four registers)

The encodings in this section are decoded from [SME2 Multi-vector - Memory \(Contiguous\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	0	0	0	1	0	0	imm4				1	msz	PNg				Rn				Zt				0	N

Decode fields		Instruction Details	Feature
msz	N		
00	0	LD1B (scalar plus immediate, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
00	1	LDNT1B (scalar plus immediate, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
01	0	LD1H (scalar plus immediate, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
01	1	LDNT1H (scalar plus immediate, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
10	0	LD1W (scalar plus immediate, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
10	1	LDNT1W (scalar plus immediate, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
11	0	LD1D (scalar plus immediate, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
11	1	LDNT1D (scalar plus immediate, consecutive registers)	FEAT_SME2 FEAT_SVE2p1

SME2 multi-vec contiguous store (scalar plus immediate, two registers)

The encodings in this section are decoded from [SME2 Multi-vector - Memory \(Contiguous\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	0	0	0	1	1	0	imm4			0	msz		PNg			Rn			Zt			N				

Decode fields		Instruction Details	Feature
msz	N		
00	0	ST1B (scalar plus immediate, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
00	1	STNT1B (scalar plus immediate, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
01	0	ST1H (scalar plus immediate, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
01	1	STNT1H (scalar plus immediate, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
10	0	ST1W (scalar plus immediate, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
10	1	STNT1W (scalar plus immediate, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
11	0	ST1D (scalar plus immediate, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
11	1	STNT1D (scalar plus immediate, consecutive registers)	FEAT_SME2 FEAT_SVE2p1

SME2 multi-vec contiguous store (scalar plus immediate, four registers)

The encodings in this section are decoded from [SME2 Multi-vector - Memory \(Contiguous\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	0	0	0	1	1	0	imm4			1	msz		PNg			Rn			Zt			0			N	

Decode fields		Instruction Details	Feature
msz	N		
00	0	ST1B (scalar plus immediate, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
00	1	STNT1B (scalar plus immediate, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
01	0	ST1H (scalar plus immediate, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
01	1	STNT1H (scalar plus immediate, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
10	0	ST1W (scalar plus immediate, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
10	1	STNT1W (scalar plus immediate, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
11	0	ST1D (scalar plus immediate, consecutive registers)	FEAT_SME2 FEAT_SVE2p1
11	1	STNT1D (scalar plus immediate, consecutive registers)	FEAT_SME2 FEAT_SVE2p1

SME2 Multi-vector - Memory (Strided)

The encodings in this section are decoded from [SME encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	0	1	0	op0						op1												op2				

Decode fields			Instruction details
op0	op1	op2	
00x	0		SME2 multi-vec non-contiguous load (scalar plus scalar, two registers)
00x	1	0	SME2 multi-vec non-contiguous load (scalar plus scalar, four registers)
01x	0		SME2 multi-vec non-contiguous store (scalar plus scalar, two registers)
01x	1	0	SME2 multi-vec non-contiguous store (scalar plus scalar, four registers)
0x1	1	1	UNALLOCATED
100	0		SME2 multi-vec non-contiguous load (scalar plus immediate, two registers)
100	1	0	SME2 multi-vec non-contiguous load (scalar plus immediate, four registers)
110	0		SME2 multi-vec non-contiguous store (scalar plus immediate, two registers)
110	1	0	SME2 multi-vec non-contiguous store (scalar plus immediate, four registers)
1x1			UNALLOCATED
!= 1x1xx0	1	1	UNALLOCATED

SME2 multi-vec non-contiguous load (scalar plus scalar, two registers)

The encodings in this section are decoded from [SME2 Multi-vector - Memory \(Strided\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	0	1	0	0	0	Rm				0	msz	PNg				Rn				T	N	Zt				

Decode fields		Instruction Details	Feature
msz	N		
00	0	LD1B (scalar plus scalar, strided registers)	FEAT_SME2
00	1	LDNT1B (scalar plus scalar, strided registers)	FEAT_SME2
01	0	LD1H (scalar plus scalar, strided registers)	FEAT_SME2
01	1	LDNT1H (scalar plus scalar, strided registers)	FEAT_SME2
10	0	LD1W (scalar plus scalar, strided registers)	FEAT_SME2
10	1	LDNT1W (scalar plus scalar, strided registers)	FEAT_SME2
11	0	LD1D (scalar plus scalar, strided registers)	FEAT_SME2
11	1	LDNT1D (scalar plus scalar, strided registers)	FEAT_SME2

SME2 multi-vec non-contiguous load (scalar plus scalar, four registers)

The encodings in this section are decoded from [SME2 Multi-vector - Memory \(Strided\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	0	1	0	0	0	Rm					1	msz	PNg				Rn				T	N	0	Zt		

Decode fields		Instruction Details	Feature
msz	N		
00	0	LD1B (scalar plus scalar, strided registers)	FEAT_SME2
00	1	LDNT1B (scalar plus scalar, strided registers)	FEAT_SME2
01	0	LD1H (scalar plus scalar, strided registers)	FEAT_SME2
01	1	LDNT1H (scalar plus scalar, strided registers)	FEAT_SME2
10	0	LD1W (scalar plus scalar, strided registers)	FEAT_SME2
10	1	LDNT1W (scalar plus scalar, strided registers)	FEAT_SME2
11	0	LD1D (scalar plus scalar, strided registers)	FEAT_SME2
11	1	LDNT1D (scalar plus scalar, strided registers)	FEAT_SME2

SME2 multi-vec non-contiguous store (scalar plus scalar, two registers)

The encodings in this section are decoded from [SME2 Multi-vector - Memory \(Strided\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	0	1	0	0	1	Rm				0	msz	PNg				Rn				T	N	Zt				

Decode fields		Instruction Details	Feature
msz	N		
00	0	ST1B (scalar plus scalar, strided registers)	FEAT_SME2
00	1	STNT1B (scalar plus scalar, strided registers)	FEAT_SME2
01	0	ST1H (scalar plus scalar, strided registers)	FEAT_SME2
01	1	STNT1H (scalar plus scalar, strided registers)	FEAT_SME2
10	0	ST1W (scalar plus scalar, strided registers)	FEAT_SME2
10	1	STNT1W (scalar plus scalar, strided registers)	FEAT_SME2
11	0	ST1D (scalar plus scalar, strided registers)	FEAT_SME2
11	1	STNT1D (scalar plus scalar, strided registers)	FEAT_SME2

SME2 multi-vec non-contiguous store (scalar plus scalar, four registers)

The encodings in this section are decoded from [SME2 Multi-vector - Memory \(Strided\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	0	1	0	0	1	Rm				1	msz	PNg				Rn				T	N	0	Zt			

Decode fields		Instruction Details	Feature
msz	N		
00	0	ST1B (scalar plus scalar, strided registers)	FEAT_SME2
00	1	STNT1B (scalar plus scalar, strided registers)	FEAT_SME2
01	0	ST1H (scalar plus scalar, strided registers)	FEAT_SME2
01	1	STNT1H (scalar plus scalar, strided registers)	FEAT_SME2
10	0	ST1W (scalar plus scalar, strided registers)	FEAT_SME2
10	1	STNT1W (scalar plus scalar, strided registers)	FEAT_SME2
11	0	ST1D (scalar plus scalar, strided registers)	FEAT_SME2
11	1	STNT1D (scalar plus scalar, strided registers)	FEAT_SME2

SME2 multi-vec non-contiguous load (scalar plus immediate, two registers)

The encodings in this section are decoded from [SME2 Multi-vector - Memory \(Strided\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	0	1	0	1	0	0	imm4				0	msz		PNg				Rn				T	N	Zt		

Decode fields		Instruction Details	Feature
msz	N		
00	0	LD1B (scalar plus immediate, strided registers)	FEAT_SME2
00	1	LDNT1B (scalar plus immediate, strided registers)	FEAT_SME2
01	0	LD1H (scalar plus immediate, strided registers)	FEAT_SME2
01	1	LDNT1H (scalar plus immediate, strided registers)	FEAT_SME2
10	0	LD1W (scalar plus immediate, strided registers)	FEAT_SME2
10	1	LDNT1W (scalar plus immediate, strided registers)	FEAT_SME2
11	0	LD1D (scalar plus immediate, strided registers)	FEAT_SME2
11	1	LDNT1D (scalar plus immediate, strided registers)	FEAT_SME2

SME2 multi-vec non-contiguous load (scalar plus immediate, four registers)

The encodings in this section are decoded from [SME2 Multi-vector - Memory \(Strided\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	0	1	0	1	0	0	imm4				1	msz	PNg				Rn				T	N	0	Zt		

Decode fields		Instruction Details	Feature
msz	N		
00	0	LD1B (scalar plus immediate, strided registers)	FEAT_SME2
00	1	LDNT1B (scalar plus immediate, strided registers)	FEAT_SME2
01	0	LD1H (scalar plus immediate, strided registers)	FEAT_SME2
01	1	LDNT1H (scalar plus immediate, strided registers)	FEAT_SME2
10	0	LD1W (scalar plus immediate, strided registers)	FEAT_SME2
10	1	LDNT1W (scalar plus immediate, strided registers)	FEAT_SME2
11	0	LD1D (scalar plus immediate, strided registers)	FEAT_SME2
11	1	LDNT1D (scalar plus immediate, strided registers)	FEAT_SME2

SME2 multi-vec non-contiguous store (scalar plus immediate, two registers)

The encodings in this section are decoded from [SME2 Multi-vector - Memory \(Strided\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	0	1	0	1	1	0	imm4			0	msz		PNg			Rn			T	N	Zt					

Decode fields		Instruction Details	Feature
msz	N		
00	0	ST1B (scalar plus immediate, strided registers)	FEAT_SME2
00	1	STNT1B (scalar plus immediate, strided registers)	FEAT_SME2
01	0	ST1H (scalar plus immediate, strided registers)	FEAT_SME2
01	1	STNT1H (scalar plus immediate, strided registers)	FEAT_SME2
10	0	ST1W (scalar plus immediate, strided registers)	FEAT_SME2
10	1	STNT1W (scalar plus immediate, strided registers)	FEAT_SME2
11	0	ST1D (scalar plus immediate, strided registers)	FEAT_SME2
11	1	STNT1D (scalar plus immediate, strided registers)	FEAT_SME2

SME2 multi-vec non-contiguous store (scalar plus immediate, four registers)

The encodings in this section are decoded from [SME2 Multi-vector - Memory \(Strided\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	0	1	0	1	1	0	imm4			1	msz		PNg			Rn			T	N	0	Zt				

Decode fields		Instruction Details	Feature
msz	N		
00	0	ST1B (scalar plus immediate, strided registers)	FEAT_SME2
00	1	STNT1B (scalar plus immediate, strided registers)	FEAT_SME2
01	0	ST1H (scalar plus immediate, strided registers)	FEAT_SME2
01	1	STNT1H (scalar plus immediate, strided registers)	FEAT_SME2
10	0	ST1W (scalar plus immediate, strided registers)	FEAT_SME2
10	1	STNT1W (scalar plus immediate, strided registers)	FEAT_SME2
11	0	ST1D (scalar plus immediate, strided registers)	FEAT_SME2
11	1	STNT1D (scalar plus immediate, strided registers)	FEAT_SME2

SME Integer Outer Product - 32 bit

The encodings in this section are decoded from [SME encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	0		1	0	op0																	op1	0			

Decode fields		Instruction details
op0	op1	
0	1	SME2 Int16 two-way outer product
1	1	UNALLOCATED
	0	SME Int8 outer product

SME2 Int16 two-way outer product

The encodings in this section are decoded from [SME Integer Outer Product - 32 bit](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	0	u0	1	0	0	Zm				Pm				Pn				Zn				S	1	0	ZAda	

Decode fields		Instruction Details	Feature
u0	S		
0	0	SMOPA (2-way)	FEAT_SME2
0	1	SMOPS (2-way)	FEAT_SME2
1	0	UMOPA (2-way)	FEAT_SME2
1	1	UMOPS (2-way)	FEAT_SME2

SME Int8 outer product

The encodings in this section are decoded from [SME Integer Outer Product - 32 bit](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	0	u0	1	0	u1	Zm				Pm				Pn				Zn				S	0	0	ZAda	

Decode fields			Instruction Details	Feature
u0	u1	S		
0	0	0	SMOPA (4-way)	FEAT_SME
0	0	1	SMOPS (4-way)	FEAT_SME
0	1	0	SUMOPA (4-way)	FEAT_SME
0	1	1	SUMOPS	FEAT_SME
1	0	0	USMOPA (4-way)	FEAT_SME
1	0	1	USMOPS	FEAT_SME
1	1	0	UMOPA (4-way)	FEAT_SME
1	1	1	UMOPS (4-way)	FEAT_SME

SME2 Quarter Tile Outer Product - 64-bit

The encodings in this section are decoded from [SME encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	op0	0	0	0	0	op1	1	1	op2					0	0	0	0	0	0	0					0			1		

Decode fields			Instruction details
op0	op1	op2	
0	0	0	SME2 FP64 non-widening quarter tile outer product
0	0	1	UNALLOCATED
0	1		UNALLOCATED
1			SME2 Int16 to Int64 quarter tile outer product

SME2 FP64 non-widening quarter tile outer product

The encodings in this section are decoded from [SME2 Quarter Tile Outer Product - 64-bit](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	0	1	1	0	M	Zm				0	0	0	0	0	0	0	N	Zn				0	S	1	ZAda

Decode fields			Instruction Details	Feature
M	N	S		
0	0	0	FMOP4A (non-widening) — Double-precision, single vectors	FEAT_SME_MOP4 && FEAT_SME_F64F64
0	0	1	FMOP4S (non-widening) — Double-precision, single vectors	FEAT_SME_MOP4 && FEAT_SME_F64F64
0	1	0	FMOP4A (non-widening) — Double-precision, multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_F64F64
0	1	1	FMOP4S (non-widening) — Double-precision, multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_F64F64
1	0	0	FMOP4A (non-widening) — Double-precision, single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_F64F64
1	0	1	FMOP4S (non-widening) — Double-precision, single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_F64F64
1	1	0	FMOP4A (non-widening) — Double-precision, multiple vectors	FEAT_SME_MOP4 && FEAT_SME_F64F64
1	1	1	FMOP4S (non-widening) — Double-precision, multiple vectors	FEAT_SME_MOP4 && FEAT_SME_F64F64

SME2 Int16 to Int64 quarter tile outer product

The encodings in this section are decoded from [SME2 Quarter Tile Outer Product - 64-bit](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	0	u0	1	1	u1	M		Zm		0	0	0	0	0	0	0	0	N		Zn		0	S	1		ZAda

Decode fields					Instruction Details	Feature
u0	u1	M	N	S		
0	0	0	0	0	SMOP4A (4-way) — 64-bit, single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	0	0	0	1	SMOP4S (4-way) — 64-bit, single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	0	0	1	0	SMOP4A (4-way) — 64-bit, multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	0	0	1	1	SMOP4S (4-way) — 64-bit, multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	0	1	0	0	SMOP4A (4-way) — 64-bit, single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	0	1	0	1	SMOP4S (4-way) — 64-bit, single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	0	1	1	0	SMOP4A (4-way) — 64-bit, multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	0	1	1	1	SMOP4S (4-way) — 64-bit, multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	1	0	0	0	SUMOP4A — 64-bit, single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	1	0	0	1	SUMOP4S — 64-bit, single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	1	0	1	0	SUMOP4A — 64-bit, multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	1	0	1	1	SUMOP4S — 64-bit, multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	1	1	0	0	SUMOP4A — 64-bit, single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	1	1	0	1	SUMOP4S — 64-bit, single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	1	1	1	0	SUMOP4A — 64-bit, multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	1	1	1	1	SUMOP4S — 64-bit, multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	0	0	0	0	USMOP4A — 64-bit, single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	0	0	0	1	USMOP4S — 64-bit, single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	0	0	1	0	USMOP4A — 64-bit, multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	0	0	1	1	USMOP4S — 64-bit, multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	0	1	0	0	USMOP4A — 64-bit, single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	0	1	0	1	USMOP4S — 64-bit, single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	0	1	1	0	USMOP4A — 64-bit, multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	0	1	1	1	USMOP4S — 64-bit, multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	1	0	0	0	UMOP4A (4-way) — 64-bit, single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	1	0	0	1	UMOP4S (4-way) — 64-bit, single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	1	0	1	0	UMOP4A (4-way) — 64-bit, multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	1	0	1	1	UMOP4S (4-way) — 64-bit, multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	1	1	0	0	UMOP4A (4-way) — 64-bit, single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	1	1	0	1	UMOP4S (4-way) — 64-bit, single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	1	1	1	0	UMOP4A (4-way) — 64-bit, multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	1	1	1	1	UMOP4S (4-way) — 64-bit, multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64

SME Outer Product - 64 bit

The encodings in this section are decoded from [SME encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	op0	0	0	0	0	op1	1	1	op2																		0			

Decode fields			Instruction details
op0	op1	op2	
0	0	0	SME FP64 outer product
0	0	1	UNALLOCATED
0	1		UNALLOCATED
1			SME Int16 outer product

Decode fields		Instruction Details	Feature
opc	opc2		
x1x	1xx	UNALLOCATED	-
000		ZERO (single-vector) — Two ZA single-vectors	FEAT_SME2p1
001		ZERO (double-vector) — One ZA double-vector	FEAT_SME2p1
010	0xx	ZERO (double-vector) — Two ZA double-vectors	FEAT_SME2p1
011	0xx	ZERO (double-vector) — Four ZA double-vectors	FEAT_SME2p1
100		ZERO (single-vector) — Four ZA single-vectors	FEAT_SME2p1
101	0xx	ZERO (quad-vector) — One ZA quad-vector	FEAT_SME2p1
101	1xx	UNALLOCATED	-
11x	01x	UNALLOCATED	-
110	00x	ZERO (quad-vector) — Two ZA quad-vectors	FEAT_SME2p1
111	00x	ZERO (quad-vector) — Four ZA quad-vectors	FEAT_SME2p1

SME2 zero lookup table

The encodings in this section are decoded from [SME encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	0	1	0	0	1	0	op0												opc					

Decode fields		Instruction Details	Feature
op0	opc		
0000000000000000	0001	ZERO (table)	FEAT_SME2
0000000000000000	!= 0001	UNALLOCATED	-
!= 0000000000000000		UNALLOCATED	-

SME2 Move Lookup Table

The encodings in this section are decoded from [SME encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	0	1	0	0	1	1	op0	op1	op2															

Decode fields			Instruction details
op0	op1	op2	
0	00		SME2 move from lookup table
0	10		UNALLOCATED
1	00		SME2 move into lookup table
1	10	0	SME2 move vector to lookup table
1	10	1	UNALLOCATED
	x1		UNALLOCATED

SME2 move from lookup table

The encodings in this section are decoded from [SME2 Move Lookup Table](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	0	1	0	0	1	1	0	0	0	imm3			opc						Rt					

Decode fields		Instruction Details	Feature
opc			
0011111		MOVT (table to scalar)	FEAT_SME2
!= 0011111		UNALLOCATED	-

SME2 move into lookup table

The encodings in this section are decoded from [SME2 Move Lookup Table](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	0	1	0	0	1	1	1	0	0	imm3			opc						Rt					

Decode fields		Instruction Details		Feature
opc				
0011111		MOVT (scalar to table)		FEAT_SME2
!= 0011111		UNALLOCATED		-

SME2 move vector to lookup table

The encodings in this section are decoded from [SME2 Move Lookup Table](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	0	1	0	0	1	1	1	1	0	0	off2		opc						Zt					

Decode fields		Instruction Details		Feature
opc				
0011111		MOVT (vector to table)		FEAT_SME_LUTv2
!= 0011111		UNALLOCATED		-

SME2 Expand Lookup Table (Non-contiguous)

The encodings in this section are decoded from [SME encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	0	0	0	0	0	0	1	0	0	1	1	op0		op1								op4		op2				op3			

Decode fields					Instruction details	
op0	op1	op4	op2	op3		
00x	00			00	UNALLOCATED	
010	00	0	0	00	SME2 lookup table three source registers expand to four non-contiguous destination registers	
010	00	1	0	00	UNALLOCATED	
011	00		0	00	SME2 lookup table two source registers expand to four non-contiguous destination registers	
01x	00		1	00	UNALLOCATED	
1xx	00			00	UNALLOCATED	
	10			00	SME2 lookup table expand four non-contiguous registers	
	x0			01	UNALLOCATED	
	x1			0x	SME2 lookup table expand two non-contiguous registers	
				1x	UNALLOCATED	
!= 01x	00			00	UNALLOCATED	

SME2 lookup table three source registers expand to four non-contiguous destination registers

The encodings in this section are decoded from [SME2 Expand Lookup Table \(Non-contiguous\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	1	0	0	1	1	0	1	0	0	0	size		opc		Zn		0	0	D	0	0	Zd		

Decode fields		Instruction Details		Feature
size	opc			
00	00	LUTi6 (table, four registers, 8-bit)		FEAT_SME2p3
00	!= 00	UNALLOCATED		-
!= 00		UNALLOCATED		-

SME2 lookup table two source registers expand to four non-contiguous destination registers

The encodings in this section are decoded from [SME2 Expand Lookup Table \(Non-contiguous\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	1	0	0	1	1	0	1	1	0	0	size	opc	Zn			0	D	0	0	Zd				

Decode fields	Instruction Details	Feature
opc		
00	LUTI4 (four registers, 8-bit)	FEAT_SME2p1 && FEAT_SME_LUTv2
!= 00	UNALLOCATED	-

SME2 lookup table expand four non-contiguous registers

The encodings in this section are decoded from [SME2 Expand Lookup Table \(Non-contiguous\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	1	0	0	1	1	opc		1		0	size		opc2		Zn				D		0	0	Zd	

Decode fields	Instruction Details	Feature
opc	opc2	
	!= 00	UNALLOCATED
00x	00	UNALLOCATED
01x	00	LUTI4 (four registers, 16-bit and 32-bit)
1xx	00	LUTI2 (four registers)

SME2 lookup table expand two non-contiguous registers

The encodings in this section are decoded from [SME2 Expand Lookup Table \(Non-contiguous\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	1	0	0	1	1	opc		1		size	opc2		Zn				D		0		Zd			

Decode fields	Instruction Details	Feature
opc	opc2	
	!= 00	UNALLOCATED
00xx	00	UNALLOCATED
01xx	00	LUTI4 (two registers)
1xxx	00	LUTI2 (two registers)

SME2 Expand Lookup Table (Contiguous)

The encodings in this section are decoded from [SME encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	1	op0	0	0	1		op1											op4	op2					op3

Decode fields					Instruction details
op0	op1	op4	op2	op3	
0	00x00			00	UNALLOCATED
0	01000	0	0	00	SME2 lookup table three source registers expand to four contiguous destination registers
0	01000	1	0	00	UNALLOCATED
0	01100		0	00	SME2 lookup table two source registers expand to four contiguous destination registers
0	01x00		1	00	UNALLOCATED
0	1xx00			00	UNALLOCATED
0	xxx10			00	SME2 lookup table expand four contiguous registers
0	xxxxx0			10	UNALLOCATED
0	xxxxx1			x0	SME2 lookup table expand two contiguous registers
0				x1	UNALLOCATED
1					SME2 lookup table expand one register

SME2 lookup table three source registers expand to four contiguous destination registers

The encodings in this section are decoded from [SME2 Expand Lookup Table \(Contiguous\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	1	0	0	0	1	0	1	0	0	0	size	opc			Zn		0	0			Zd		0	0

Decode fields		Instruction Details	Feature
size	opc		
00	00	LUTi6 (table, four registers, 8-bit)	FEAT_SME2p3
00	!= 00	UNALLOCATED	-
!= 00		UNALLOCATED	-

SME2 lookup table two source registers expand to four contiguous destination registers

The encodings in this section are decoded from [SME2 Expand Lookup Table \(Contiguous\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	1	0	0	0	1	0	1	1	0	0	size	opc			Zn		0				Zd		0	0

Decode fields	Instruction Details		Feature
opc			
00	LUTi4 (four registers, 8-bit)		FEAT_SME_LUTv2
!= 00	UNALLOCATED		-

SME2 lookup table expand four contiguous registers

The encodings in this section are decoded from [SME2 Expand Lookup Table \(Contiguous\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	1	0	0	0	1		opc		1	0	size	opc2			Zn						Zd		0	0

Decode fields		Instruction Details	Feature
opc	opc2		
	!= 00	UNALLOCATED	-
00x	00	UNALLOCATED	-
01x	00	LUTI4 (four registers, 16-bit and 32-bit)	FEAT_SME2
1xx	00	LUTI2 (four registers)	FEAT_SME2

SME2 lookup table expand two contiguous registers

The encodings in this section are decoded from [SME2 Expand Lookup Table \(Contiguous\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	1	0	0	0	1	opc			1	size	opc2	Zn				Zd							0	

Decode fields		Instruction Details		Feature
opc	opc2			
	!= 00	UNALLOCATED		-
00xx	00	UNALLOCATED		-
01xx	00	LUTi4 (two registers)		FEAT_SME2
1xxx	00	LUTi2 (two registers)		FEAT_SME2

SME2 lookup table expand one register

The encodings in this section are decoded from [SME2 Expand Lookup Table \(Contiguous\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	1	1	0	0	1	opc			size	opc2	Zn				Zd									

Decode fields		Instruction Details		Feature
opc	size	opc2		
		!= 00	UNALLOCATED	-
00000		00	UNALLOCATED	-
00001	00	00	LUTi6 (table, single, 8-bit)	FEAT_SME2p3
00001	!= 00	00	UNALLOCATED	-
0001x		00	UNALLOCATED	-
001xx		00	UNALLOCATED	-
01xxx		00	LUTi4 (single)	FEAT_SME2
1xxxx		00	LUTi2 (single)	FEAT_SME2

SME Move into Array

The encodings in this section are decoded from [SME encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	op0	0	0	0	op1	0	op2					op3						op4	0	op5				

Decode fields						Instruction details		Feature
op0	op1	op2	op3	op4	op5			
00	1	00	010	x0	0	MOVA (vector to array, two registers)		FEAT_SME2
00	1	00	011	00	0	MOVA (vector to array, four registers)		FEAT_SME2
00	1	00	011	10	0	UNALLOCATED		-
00	1	01	01x	x0	0	UNALLOCATED		-
	0					SME move vector to array		-
	1	0x	000	x0	0	SME2 move vector to tile, two registers		-
	1	0x	001	00	0	SME2 move vector to tile, four registers		-
	1	0x	001	10	0	UNALLOCATED		-
	1	0x	0xx	x0	1	UNALLOCATED		-
	1	0x	0xx	x1		UNALLOCATED		-
	1	0x	1xx			UNALLOCATED		-
	1	1x				UNALLOCATED		-
!= 00	1	0x	01x	x0	0	UNALLOCATED		-

SME move vector to array

The encodings in this section are decoded from [SME Move into Array](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	size	0	0	0	0	0	Q	V	Rs	Pg				Zn				0	opc					

Decode fields size	Q	Instruction Details	Feature
00	0	MOVA (vector to tile, single) — 8-bit	FEAT_SME
01	0	MOVA (vector to tile, single) — 16-bit	FEAT_SME
10	0	MOVA (vector to tile, single) — 32-bit	FEAT_SME
11	0	MOVA (vector to tile, single) — 64-bit	FEAT_SME
11	1	MOVA (vector to tile, single) — 128-bit	FEAT_SME
!= 11	1	UNALLOCATED	-

SME2 move vector to tile, two registers

The encodings in this section are decoded from [SME Move into Array](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	size	0	0	0	1	0	0	V	Rs	0	0	0	Zn	0	0	0	0	0	0	0	0	opc		

Decode fields size	Instruction Details	Feature
00	MOVA (vector to tile, two registers) — 8-bit	FEAT_SME2
01	MOVA (vector to tile, two registers) — 16-bit	FEAT_SME2
10	MOVA (vector to tile, two registers) — 32-bit	FEAT_SME2
11	MOVA (vector to tile, two registers) — 64-bit	FEAT_SME2

SME2 move vector to tile, four registers

The encodings in this section are decoded from [SME Move into Array](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	size	0	0	0	1	0	0	V	Rs	0	0	1	Zn	0	0	0	0	0	0	0	0	opc		

Decode fields size	opc	Instruction Details	Feature
00	0xx	MOVA (vector to tile, four registers) — 8-bit	FEAT_SME2
01	0xx	MOVA (vector to tile, four registers) — 16-bit	FEAT_SME2
10	0xx	MOVA (vector to tile, four registers) — 32-bit	FEAT_SME2
11		MOVA (vector to tile, four registers) — 64-bit	FEAT_SME2
!= 11	1xx	UNALLOCATED	-

SME Move from Array

The encodings in this section are decoded from [SME encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	op0	0	0	0	op1	1	op2					op3		op4									op5	

Decode fields						Instruction details	Feature
op0	op1	op2	op3	op4	op5		
00	1	00	010	00	x0	MOVA (array to vector, two registers)	FEAT_SME2
00	1	00	010	10	x0	MOVAZ (array to vector, two registers)	FEAT_SME2p1
00	1	00	011	00	00	MOVA (array to vector, four registers)	FEAT_SME2
00	1	00	011	10	00	MOVAZ (array to vector, four registers)	FEAT_SME2p1
00	1	00	011	x0	10	UNALLOCATED	-
00	1	01	01x	x0	x0	UNALLOCATED	-
	0		000	1x		SME zeroing move array to vector	-
	0			0x		SME move array to vector	-
	0		!= 000	1x		UNALLOCATED	-
	1	0x	000	00	x0	SME2 move tile to vector, two registers	-
	1	0x	000	10	x0	SME2 zeroing move tile to vector, two registers	-
	1	0x	001	00	00	SME2 move tile to vector, four registers	-
	1	0x	001	10	00	SME2 zeroing move tile to vector, four registers	-
	1	0x	001	x0	10	UNALLOCATED	-
	1	0x	0xx	x0	x1	UNALLOCATED	-
	1	0x	0xx	x1		UNALLOCATED	-
	1	0x	1xx			UNALLOCATED	-
	1	1x				UNALLOCATED	-
!= 00	1	0x	01x	x0	x0	UNALLOCATED	-

SME zeroing move array to vector

The encodings in this section are decoded from [SME Move from Array](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	size	0	0	0	0	1	Q	V	Rs	0	0	0	1	opc				Zd						

Decode fields		Instruction Details	Feature
size	Q		
00	0	MOVAZ (tile to vector, single) — 8-bit	FEAT_SME2p1
01	0	MOVAZ (tile to vector, single) — 16-bit	FEAT_SME2p1
10	0	MOVAZ (tile to vector, single) — 32-bit	FEAT_SME2p1
11	0	MOVAZ (tile to vector, single) — 64-bit	FEAT_SME2p1
11	1	MOVAZ (tile to vector, single) — 128-bit	FEAT_SME2p1
!= 11	1	UNALLOCATED	-

SME move array to vector

The encodings in this section are decoded from [SME Move from Array](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	size	0	0	0	0	1	Q	V	Rs	Pg		0	opc				Zd							

Decode fields size	Q	Instruction Details	Feature
00	0	MOVA (tile to vector, single) — 8-bit	FEAT_SME
01	0	MOVA (tile to vector, single) — 16-bit	FEAT_SME
10	0	MOVA (tile to vector, single) — 32-bit	FEAT_SME
11	0	MOVA (tile to vector, single) — 64-bit	FEAT_SME
11	1	MOVA (tile to vector, single) — 128-bit	FEAT_SME
!= 11	1	UNALLOCATED	-

SME2 move tile to vector, two registers

The encodings in this section are decoded from [SME Move from Array](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	size	0	0	0	1	1	0	V	Rs	0	0	0	0	0	0	opc			Zd					0

Decode fields size	Instruction Details	Feature
00	MOVA (tile to vector, two registers) — 8-bit	FEAT_SME2
01	MOVA (tile to vector, two registers) — 16-bit	FEAT_SME2
10	MOVA (tile to vector, two registers) — 32-bit	FEAT_SME2
11	MOVA (tile to vector, two registers) — 64-bit	FEAT_SME2

SME2 zeroing move tile to vector, two registers

The encodings in this section are decoded from [SME Move from Array](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	size	0	0	0	1	1	0	V	Rs	0	0	0	1	0	0	opc			Zd					0

Decode fields size	Instruction Details	Feature
00	MOVAZ (tile to vector, two registers) — 8-bit	FEAT_SME2p1
01	MOVAZ (tile to vector, two registers) — 16-bit	FEAT_SME2p1
10	MOVAZ (tile to vector, two registers) — 32-bit	FEAT_SME2p1
11	MOVAZ (tile to vector, two registers) — 64-bit	FEAT_SME2p1

SME2 move tile to vector, four registers

The encodings in this section are decoded from [SME Move from Array](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	size	0	0	0	1	1	0	V	Rs	0	0	1	0	0	opc			Zd			0	0		

Decode fields size	opc	Instruction Details	Feature
00	0xx	MOVA (tile to vector, four registers) — 8-bit	FEAT_SME2
01	0xx	MOVA (tile to vector, four registers) — 16-bit	FEAT_SME2
10	0xx	MOVA (tile to vector, four registers) — 32-bit	FEAT_SME2
11		MOVA (tile to vector, four registers) — 64-bit	FEAT_SME2
!= 11	1xx	UNALLOCATED	-

SME2 zeroing move tile to vector, four registers

The encodings in this section are decoded from [SME Move from Array](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	size	0	0	0	1	1	0	V	Rs	0	0	1	1	0	opc	Zd			0	0				

Decode fields		Instruction Details	Feature
size	opc		
00	0xx	MOVAZ (tile to vector, four registers) — 8-bit	FEAT_SME2p1
01	0xx	MOVAZ (tile to vector, four registers) — 16-bit	FEAT_SME2p1
10	0xx	MOVAZ (tile to vector, four registers) — 32-bit	FEAT_SME2p1
11		MOVAZ (tile to vector, four registers) — 64-bit	FEAT_SME2p1
!= 11	1xx	UNALLOCATED	-

SME Add Vector to Array

The encodings in this section are decoded from [SME encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	op0		0	1	0	op1													op2	0				

Decode fields		Instruction details
op0	op1	
0		UNALLOCATED
1	00	SME add vector to array
1	00	UNALLOCATED
1	!= 00	UNALLOCATED

SME add vector to array

The encodings in this section are decoded from [SME Add Vector to Array](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	1	op	0	1	0	0	0	V		Pm			Pn				Zn		0	0		opc2		

Decode fields		Instruction Details	Feature
op	V		
0		UNALLOCATED	-
0	0	ADDHA — 32-bit	FEAT_SME
0	1	ADDVA — 32-bit	FEAT_SME
1	0	ADDHA — 64-bit	FEAT_SME_I16I64
1	1	ADDVA — 64-bit	FEAT_SME_I16I64

SME2 Multi-vector - Multiple and Single Array Vectors (Two registers)

The encodings in this section are decoded from [SME encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	op0	1	0				0					op1							op2		op3		

Decode fields				Instruction details	Feature
op0	op1	op2	op3		
0	000	000	1	FMLALL (multiple and single vector) — Two ZA quad-vectors	FEAT_SME_F8F32
0	000	!= 000	1	UNALLOCATED	-
0	010			SME2 single-multi long FMA two sources	-
0	011			SME2 multiple and single vector long FMA one source	-
0	100			SME2 single-multi FP dot product two registers	-
0	101	x1x		SME2 single-multi mixed dot product two registers	-
1	000		1	UNALLOCATED	-
1	010			SME2 single-multi long MLA two sources	-
1	011			SME2 multiple and single vector long MLA one source	-
1	100			UNALLOCATED	-
1	101	x1x		SME2 single-multi two-way dot product two registers	-
	000		0	SME2 single-multi long long MLA two sources	-
	001			SME2 multiple and single vector long long FMA one source	-
	101	x0x		SME2 single-multi four-way dot product two registers	-
	110	0xx		SME2 single-multi ternary FP two registers	-
	110	1xx		SME2 single-multi ternary int two registers	-
	111	0xx		SME2 single-multi ternary FP16 two registers	-
	111	1xx		UNALLOCATED	-

SME2 single-multi long FMA two sources

The encodings in this section are decoded from [SME2 Multi-vector - Multiple and Single Array Vectors \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	0	1	0	Zm			0	Rv		0	1	0	Zn			op		S	o2	off2			

Decode fields			Instruction Details	Feature
op	S	o2		
0	0	0	FMLAL (multiple and single vector, FP16 to FP32)	FEAT_SME2
0	0	1	FMLAL (multiple and single vector, FP8 to FP16)	FEAT_SME_F8F16
0	1	0	FMLSL (multiple and single vector)	FEAT_SME2
0	1	1	UNALLOCATED	-
1		1	UNALLOCATED	-
1	0	0	BFMLAL (multiple and single vector)	FEAT_SME2
1	1	0	BFMLSL (multiple and single vector)	FEAT_SME2

SME2 multiple and single vector long FMA one source

The encodings in this section are decoded from [SME2 Multi-vector - Multiple and Single Array Vectors \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	0	1	0	Zm			0	Rv		0	1	1	Zn			op		S	off3				

Decode fields		Instruction Details	Feature
op	S		
0	0	FMLAL (multiple and single vector, FP16 to FP32)	FEAT_SME2
0	1	FMLSL (multiple and single vector)	FEAT_SME2
1	0	BFMLAL (multiple and single vector)	FEAT_SME2
1	1	BFMLSL (multiple and single vector)	FEAT_SME2

SME2 single-multi FP dot product two registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple and Single Array Vectors \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	0	1	0		Zm		0		Rv	1	0	0			Zn				opc			off3	

Decode fields	Instruction Details	Feature
opc		
00	FDOT (2-way, multiple and single vector, FP16 to FP32)	FEAT_SME2
01	FDOT (2-way, multiple and single vector, FP8 to FP16)	FEAT_SME_F8F16
10	BFDOT (multiple and single vector)	FEAT_SME2
11	FDOT (4-way, multiple and single vector)	FEAT_SME_F8F32

SME2 single-multi mixed dot product two registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple and Single Array Vectors \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	0	1	0		Zm		0		Rv	1	0	1			Zn			U	1		off3		

Decode fields	Instruction Details	Feature
U		
0	USDOT (4-way, multiple and single vector)	FEAT_SME2
1	SUDOT (4-way, multiple and single vector)	FEAT_SME2

SME2 single-multi long MLA two sources

The encodings in this section are decoded from [SME2 Multi-vector - Multiple and Single Array Vectors \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	1	1	0		Zm		0		Rv	0	1	0			Zn			U	S	op		off2	

Decode fields			Instruction Details	Feature
U	S	op		
		1	UNALLOCATED	-
0	0	0	SMLAL (multiple and single vector)	FEAT_SME2
0	1	0	SMLSL (multiple and single vector)	FEAT_SME2
1	0	0	UMLAL (multiple and single vector)	FEAT_SME2
1	1	0	UMLSL (multiple and single vector)	FEAT_SME2

SME2 multiple and single vector long MLA one source

The encodings in this section are decoded from [SME2 Multi-vector - Multiple and Single Array Vectors \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	1	1	0	Zm			0	Rv		0	1	1	Zn			U		S	off3				

Decode fields		Instruction Details	Feature
U	S		
0	0	SMLAL (multiple and single vector)	FEAT_SME2
0	1	SMLSL (multiple and single vector)	FEAT_SME2
1	0	UMLAL (multiple and single vector)	FEAT_SME2
1	1	UMLSL (multiple and single vector)	FEAT_SME2

SME2 single-multi two-way dot product two registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple and Single Array Vectors \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	1	1	0		Zm		0		Rv	1	0	1			Zn			U	1		off3		

Decode fields	Instruction Details	Feature
U		
0	SDOT (2-way, multiple and single vector)	FEAT_SME2
1	UDOT (2-way, multiple and single vector)	FEAT_SME2

SME2 single-multi long long MLA two sources

The encodings in this section are decoded from [SME2 Multi-vector - Multiple and Single Array Vectors \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	sz	1	0	Zm			0	Rv	0	0	0	Zn			U	S	op	0	ol				

Decode fields				Instruction Details	Feature
sz	U	S	op		
	0	0	0	SMLALL (multiple and single vector)	FEAT_SME2
	0	1	0	SMLSLL (multiple and single vector)	FEAT_SME2
	1	0	0	UMLALL (multiple and single vector)	FEAT_SME2
	1	1	0	UMLSLL (multiple and single vector)	FEAT_SME2
0		1	1	UNALLOCATED	-
0	0	0	1	USMLALL (multiple and single vector)	FEAT_SME2
0	1	0	1	SUMLALL (multiple and single vector)	FEAT_SME2
1			1	UNALLOCATED	-

SME2 multiple and single vector long long FMA one source

The encodings in this section are decoded from [SME2 Multi-vector - Multiple and Single Array Vectors \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	sz	1	0	Zm			0	Rv		0	0	1	Zn			U	S	op	off2				

Decode fields				Instruction Details	Feature
sz	U	S	op		
	0	0	0	SMLALL (multiple and single vector)	FEAT_SME2
	0	1	0	SMLSLL (multiple and single vector)	FEAT_SME2
	1	0	0	UMLALL (multiple and single vector)	FEAT_SME2
	1	1	0	UMLSLL (multiple and single vector)	FEAT_SME2
0	0	0	1	USMLALL (multiple and single vector)	FEAT_SME2
0	0	1	1	UNALLOCATED	-
0	1		1	UNALLOCATED	-
1			1	UNALLOCATED	-

SME2 single-multi four-way dot product two registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple and Single Array Vectors \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	sz	1	0	Zm			0		Rv		1	0	1	Zn			U		0	off3			

Decode fields	Instruction Details	Feature
U		
0	SDOT (4-way, multiple and single vector)	FEAT_SME2
1	UDOT (4-way, multiple and single vector)	FEAT_SME2

SME2 single-multi ternary FP two registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple and Single Array Vectors \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	sz	1	0		Zm			0		Rv	1	1	0			Zn		0	S		off3		

Decode fields	Instruction Details																Feature									
S																										
0	FMLA (multiple and single vector)																FEAT_SME2									
1	FMLS (multiple and single vector)																FEAT_SME2									

SME2 single-multi ternary int two registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple and Single Array Vectors \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	sz	1	0		Zm			0		Rv	1	1	0			Zn		1	S		off3		

Decode fields	Instruction Details																Feature									
S																										
0	ADD (to array, multiple and single vector)																FEAT_SME2									
1	SUB (to array, multiple and single vector)																FEAT_SME2									

SME2 single-multi ternary FP16 two registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple and Single Array Vectors \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	sz	1	0		Zm			0		Rv	1	1	1			Zn		0	S		off3		

Decode fields		Instruction Details																Feature									
sz	S																										
0	0	FMLA (multiple and single vector)																FEAT_SME_F16F16									
0	1	FMLS (multiple and single vector)																FEAT_SME_F16F16									
1	0	BFMLA (multiple and single vector)																FEAT_SME_B16B16									
1	1	BFMLS (multiple and single vector)																FEAT_SME_B16B16									

SME2 Multi-vector - Multiple and Single Array Vectors (Four registers)

The encodings in this section are decoded from [SME encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	op0	1	1					0					op1							op2		op3	

Decode fields				Instruction details	Feature
op0	op1	op2	op3		
0	000	000	1	FMLALL (multiple and single vector) — Four ZA quad-vectors	FEAT_SME_F8F32
0	000	!= 000	1	UNALLOCATED	-
0	001	000		FMLALL (multiple and single vector) — One ZA quad-vector	FEAT_SME_F8F32
0	001	001		UNALLOCATED	-
0	010			SME2 single-multi long FMA four sources	-
0	011	00x		FMLAL (multiple and single vector, FP8 to FP16) — One ZA double-vector	FEAT_SME_F8F16
0	0x1	1xx		UNALLOCATED	-
0	0x1	!= 00x01x		UNALLOCATED	-
0	100			SME2 single-multi FP dot product four registers	-
0	101	x1x		SME2 single-multi mixed dot product four registers	-
1	000		1	UNALLOCATED	-
1	010			SME2 single-multi long MLA four sources	-
1	0x1			UNALLOCATED	-
1	100			UNALLOCATED	-
1	101	x1x		SME2 single-multi two-way dot product four registers	-
	000		0	SME2 single-multi long long MLA four sources	-
	101	x0x		SME2 single-multi four-way dot product four registers	-
	110	0xx		SME2 single-multi ternary FP four registers	-
	110	1xx		SME2 single-multi ternary int four registers	-
	111	0xx		SME2 single-multi ternary FP16 four registers	-
	111	1xx		UNALLOCATED	-

SME2 single-multi long FMA four sources

The encodings in this section are decoded from [SME2 Multi-vector - Multiple and Single Array Vectors \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	0	1	1	Zm			0	Rv		0	1	0	Zn				op	S	o2	off2			

Decode fields			Instruction Details	Feature
op	S	o2		
0	0	0	FMLAL (multiple and single vector, FP16 to FP32)	FEAT_SME2
0	0	1	FMLAL (multiple and single vector, FP8 to FP16)	FEAT_SME_F8F16
0	1	0	FMLSL (multiple and single vector)	FEAT_SME2
0	1	1	UNALLOCATED	-
1		1	UNALLOCATED	-
1	0	0	BFMLAL (multiple and single vector)	FEAT_SME2
1	1	0	BFMLSL (multiple and single vector)	FEAT_SME2

SME2 single-multi FP dot product four registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple and Single Array Vectors \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	0	1	1	Zm		0		Rv		1		0	0	Zn			opc			off3			

Decode fields	Instruction Details	Feature
opc		
00	FDOT (2-way, multiple and single vector, FP16 to FP32)	FEAT_SME2
01	FDOT (2-way, multiple and single vector, FP8 to FP16)	FEAT_SME_F8F16
10	BFDOT (multiple and single vector)	FEAT_SME2
11	FDOT (4-way, multiple and single vector)	FEAT_SME_F8F32

SME2 single-multi mixed dot product four registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple and Single Array Vectors \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	0	1	1	Zm			0	Rv	1	0	1	Zn			U	1	off3						

Decode fields	Instruction Details	Feature
U		
0	USDOT (4-way, multiple and single vector)	FEAT_SME2
1	SUDOT (4-way, multiple and single vector)	FEAT_SME2

SME2 single-multi long MLA four sources

The encodings in this section are decoded from [SME2 Multi-vector - Multiple and Single Array Vectors \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	1	1	1	Zm			0	Rv		0	1	0	Zn			U		S	op	off2			

Decode fields			Instruction Details	Feature
U	S	op		
		1	UNALLOCATED	-
0	0	0	SMLAL (multiple and single vector)	FEAT_SME2
0	1	0	SMLSL (multiple and single vector)	FEAT_SME2
1	0	0	UMLAL (multiple and single vector)	FEAT_SME2
1	1	0	UMLSL (multiple and single vector)	FEAT_SME2

SME2 single-multi two-way dot product four registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple and Single Array Vectors \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	1	1	1	Zm			0	Rv		1	0	1	Zn			U		1	off3				

Decode fields	Instruction Details	Feature
U		
0	SDOT (2-way, multiple and single vector)	FEAT_SME2
1	UDOT (2-way, multiple and single vector)	FEAT_SME2

SME2 single-multi long long MLA four sources

The encodings in this section are decoded from [SME2 Multi-vector - Multiple and Single Array Vectors \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	sz	1	1		Zm		0		Rv		0	0	0			Zn		U		S	op	0	o1

Decode fields				Instruction Details	Feature
sz	U	S	op		
	0	0	0	SMLALL (multiple and single vector)	FEAT_SME2
	0	1	0	SMLSLL (multiple and single vector)	FEAT_SME2
	1	0	0	UMLALL (multiple and single vector)	FEAT_SME2
	1	1	0	UMLSLL (multiple and single vector)	FEAT_SME2
0		1	1	UNALLOCATED	-
0	0	0	1	USMLALL (multiple and single vector)	FEAT_SME2
0	1	0	1	SUMLALL (multiple and single vector)	FEAT_SME2
1			1	UNALLOCATED	-

SME2 single-multi four-way dot product four registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple and Single Array Vectors \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	sz	1	1	Zm		0		Rv		1		0	1	Zn				U		0	off3		

Decode fields		Instruction Details	Feature
U			
0		SDOT (4-way, multiple and single vector)	FEAT_SME2
1		UDOT (4-way, multiple and single vector)	FEAT_SME2

SME2 single-multi ternary FP four registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple and Single Array Vectors \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	sz	1	1	Zm			0	Rv		1	1	0	Zn			0	S	off3					

Decode fields		Instruction Details	Feature
S			
0		FMLA (multiple and single vector)	FEAT_SME2
1		FMLS (multiple and single vector)	FEAT_SME2

SME2 single-multi ternary int four registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple and Single Array Vectors \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	sz	1	1	Zm			0	Rv		1	1	0	Zn			1	S	off3					

Decode fields		Instruction Details	Feature
S			
0		ADD (to array, multiple and single vector)	FEAT_SME2
1		SUB (to array, multiple and single vector)	FEAT_SME2

SME2 single-multi ternary FP16 four registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple and Single Array Vectors \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	sz	1	1	Zm		0		Rv	1	1	1	Zn			0		S	off3					

Decode fields		Instruction Details	Feature
sz	S		
0	0	FMLA (multiple and single vector)	FEAT_SME_F16F16
0	1	FMLS (multiple and single vector)	FEAT_SME_F16F16
1	0	BFMLA (multiple and single vector)	FEAT_SME_B16B16
1	1	BFMLS (multiple and single vector)	FEAT_SME_B16B16

SME2 Multi-vector - Lookup table with 6-bit indices

The encodings in this section are decoded from [SME encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0		1						1	1	1	1	op0										op1	

Decode fields		Instruction details	Feature
op0	op1		
01	xx00	LUTI6 (vector, 16-bit) — Consecutive	FEAT_SME2p3
01	xx1x	UNALLOCATED	!
01	!= xx00xx01	UNALLOCATED	-
11	00xx	LUTI6 (vector, 16-bit) — Strided	FEAT_SME2p3
11	1xxx	UNALLOCATED	!
11	!= 00xx01xx	UNALLOCATED	-
x0		UNALLOCATED	-

SME2 Multi-vector - Multiple Array Vectors (Two registers)

The encodings in this section are decoded from [SME encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	op0	1	op1	op2	0	0					op3						op4		op5		op6		

Decode fields							Instruction details	Feature
op0	op1	op2	op3	op4	op5	op6		
0			000	1	000	0	FMLALL (multiple vectors) — Two ZA quad-vectors	FEAT_SME_F8F32
0			000	1	001	0	UNALLOCATED	-
0			000	1	10x	0	UNALLOCATED	-
0			000	1	x0x	1	UNALLOCATED	-
0			010	0	xx0		SME2 multiple vectors long FMA two sources	-
0			010	1	000		FMLAL (multiple vectors, FP8 to FP16) — Two ZA double-vectors	FEAT_SME_F8F16
0			010	1	100		UNALLOCATED	-
0			010	1	x01		UNALLOCATED	-
0			0x1	1			UNALLOCATED	-
0			100		x0x		SME2 multiple vectors FP dot product two registers	-
0			101	0	01x		USDOT (4-way, multiple vectors) — Two ZA single-vectors	FEAT_SME2
0			101	0	11x		UNALLOCATED	-
0			110	1	x0x		UNALLOCATED	-
0			1x1	1			UNALLOCATED	-
0			xx0	1	x1x		UNALLOCATED	-
0			!= xx0	1			UNALLOCATED	-
1			010	0	xx0		SME2 multiple vectors long MLA two sources	-
1			100	0	x0x		UNALLOCATED	-
1			101	0	x1x		SME2 multiple vectors two-way dot product two registers	-
1				1			UNALLOCATED	-
	00	00	111	0	0xx		SME2 multiple vectors binary FP two registers	-
	00	00	111	0	1xx		SME2 multiple vectors binary int two registers	-
	00	10	111	0	0xx		SME2 multiple vectors binary FP16 two registers	-
	00	10	111	0	1xx		UNALLOCATED	-
	00	x1	111	0			UNALLOCATED	-
			000	0		0	SME2 multiple vectors long long MLA two sources	-
			000	0		1	UNALLOCATED	-
			010	0	xx1		UNALLOCATED	-
			0x1	0			UNALLOCATED	-
			100	0	x1x		SME2 multiple vectors ternary FP16 two registers	-
			101	0	x0x		SME2 multiple vectors four-way dot product two registers	-
			110	0	0xx		SME2 multiple vectors ternary FP two registers	-
			110	0	1xx		SME2 multiple vectors ternary int two registers	-
	!= 00		111	0			UNALLOCATED	-

SME2 multiple vectors long FMA two sources

The encodings in this section are decoded from [SME2 Multi-vector - Multiple Array Vectors \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	0	1	Zm			0	0	Rv		0	1	0	Zn			0	op	S	0	off2			

Decode fields		Instruction Details	Feature
op	S		
0	0	FMLAL (multiple vectors, FP16 to FP32)	FEAT_SME2
0	1	FMLSL (multiple vectors)	FEAT_SME2
1	0	BFMLAL (multiple vectors)	FEAT_SME2
1	1	BFMLSL (multiple vectors)	FEAT_SME2

SME2 multiple vectors FP dot product two registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple Array Vectors \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	0	1	Zm			0	0	Rv		1	0	0	Zn			opc		0	off3				

Decode fields		Instruction Details	Feature
opc			
00		FDOT (2-way, multiple vectors, FP16 to FP32)	FEAT_SME2
01		BFDOT (multiple vectors)	FEAT_SME2
10		FDOT (2-way, multiple vectors, FP8 to FP16)	FEAT_SME_F8F16
11		FDOT (4-way, multiple vectors)	FEAT_SME_F8F32

SME2 multiple vectors long MLA two sources

The encodings in this section are decoded from [SME2 Multi-vector - Multiple Array Vectors \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	1	1	Zm			0	0	Rv		0	1	0	Zn			0	U	S	0	off2			

Decode fields		Instruction Details	Feature
U	S		
0	0	SMLAL (multiple vectors)	FEAT_SME2
0	1	SMLSL (multiple vectors)	FEAT_SME2
1	0	UMLAL (multiple vectors)	FEAT_SME2
1	1	UMLSL (multiple vectors)	FEAT_SME2

SME2 multiple vectors two-way dot product two registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple Array Vectors \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	1	1	Zm			0	0	Rv		1	0	1	Zn			0	U	1	off3				

Decode fields		Instruction Details	Feature
U			
0		SDOT (2-way, multiple vectors)	FEAT_SME2
1		UDOT (2-way, multiple vectors)	FEAT_SME2

SME2 multiple vectors binary FP two registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple Array Vectors \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	SZ	1	0	0	0	0	0	0	Rv	1	1	1	Zm			0	0	S	off3				

Decode fields		Instruction Details	Feature
S			
0		FADD	FEAT_SME2
1		FSUB	FEAT_SME2

SME2 multiple vectors binary int two registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple Array Vectors \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	sz	1	0	0	0	0	0	0	Rv	1	1	1		Zm		0	1	S			off3		

Decode fields	Instruction Details		Feature
S			
0	ADD (to array, array and multiple vectors)		FEAT_SME2
1	SUB (to array, array and multiple vectors)		FEAT_SME2

SME2 multiple vectors binary FP16 two registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple Array Vectors \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	sz	1	0	0	1	0	0	0	Rv	1	1	1		Zm		0	0	S			off3		

Decode fields	Instruction Details		Feature
sz S			
0 0	FADD		FEAT_SME_F16F16 FEAT_SME_F8F16
0 1	FSUB		FEAT_SME_F16F16 FEAT_SME_F8F16
1 0	BFADD		FEAT_SME_B16B16
1 1	BFSUB		FEAT_SME_B16B16

SME2 multiple vectors long long MLA two sources

The encodings in this section are decoded from [SME2 Multi-vector - Multiple Array Vectors \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	sz	1		Zm		0	0	Rv	0	0	0		Zn		0	U	S	op	0	0	0	0	0

Decode fields	Instruction Details		Feature
sz U S op			
0 0 0 0	SMLALL (multiple vectors)		FEAT_SME2
0 0 1 0	SMLSLL (multiple vectors)		FEAT_SME2
1 0 0 0	UMLALL (multiple vectors)		FEAT_SME2
1 1 0 0	UMLSLL (multiple vectors)		FEAT_SME2
0 0 0 1	USMLALL (multiple vectors)		FEAT_SME2
0 0 1 1	UNALLOCATED		-
0 1 1 1	UNALLOCATED		-
1 1 1 1	UNALLOCATED		-

SME2 multiple vectors ternary FP16 two registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple Array Vectors \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	sz	1	Zm			0	0	Rv		1	0	0	Zn			0	S	1	off3				

Decode fields	Instruction Details		Feature
sz S			
0 0	FMLA (multiple vectors)		FEAT_SME_F16F16
0 1	FMLS (multiple vectors)		FEAT_SME_F16F16
1 0	BFMLA (multiple vectors)		FEAT_SME_B16B16
1 1	BFMLS (multiple vectors)		FEAT_SME_B16B16

SME2 multiple vectors four-way dot product two registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple Array Vectors \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	sz	1	Zm			0		0	Rv		1	0	1	Zn			0		U	0	off3		

Decode fields	Instruction Details	Feature
U		
0	SDOT (4-way, multiple vectors)	FEAT_SME2
1	UDOT (4-way, multiple vectors)	FEAT_SME2

SME2 multiple vectors ternary FP two registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple Array Vectors \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	sz	1	Zm			0		0	Rv		1	1	0	Zn			0		0	S	off3		

Decode fields	Instruction Details	Feature
S		
0	FMLA (multiple vectors)	FEAT_SME2
1	FMLS (multiple vectors)	FEAT_SME2

SME2 multiple vectors ternary int two registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple Array Vectors \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	sz	1	Zm			0		0	Rv		1	1	0	Zn			0		1	S	off3		

Decode fields	Instruction Details	Feature
S		
0	ADD (to array, multiple vectors)	FEAT_SME2
1	SUB (to array, multiple vectors)	FEAT_SME2

SME2 Multi-vector - Multiple Array Vectors (Four registers)

The encodings in this section are decoded from [SME encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	op0	1	op1		op2		1	0				op3						op4		op5		op6	

Decode fields							Instruction details	Feature
op0	op1	op2	op3	op4	op5	op6		
0		x0	000	01	000	0	FMLALL (multiple vectors) — Four ZA quad-vectors	FEAT_SME_F8F32
0		x0	000	01	001	0	UNALLOCATED	-
0		x0	000	01	10x	0	UNALLOCATED	-
0		x0	000	01	x0x	1	UNALLOCATED	-
0		x0	010	00	xx0		SME2 multiple vectors long FMA four sources	-
0		x0	010	01	000		FMLAL (multiple vectors, FP8 to FP16) — Four ZA double-vectors	FEAT_SME_F8F16
0		x0	010	01	100		UNALLOCATED	-
0		x0	010	01	x01		UNALLOCATED	-
0		x0	0x1	01			UNALLOCATED	-
0		x0	100	0x	x0x		SME2 multiple vectors FP dot product four registers	-
0		x0	101	00	01x		USDOT (4-way, multiple vectors) — Four ZA single-vectors	FEAT_SME2
0		x0	101	00	11x		UNALLOCATED	-
0		x0	110	01	x0x		UNALLOCATED	-
0		x0	1x1	01			UNALLOCATED	-
0		x0	xx0	01	x1x		UNALLOCATED	-
0		x0	!= xx0	01			UNALLOCATED	-
1		x0	010	00	xx0		SME2 multiple vectors long MLA four sources	-
1		x0	100	00	x0x		UNALLOCATED	-
1		x0	101	00	x1x		SME2 multiple vectors two-way dot product four registers	-
1		x0		01			UNALLOCATED	-
	00	00	111	00	0xx		SME2 multiple vectors binary FP four registers	-
	00	00	111	00	1xx		SME2 multiple vectors binary int four registers	-
	00	10	111	00	0xx		SME2 multiple vectors binary FP16 four registers	-
	00	10	111	00	1xx		UNALLOCATED	-
		x0	000	00		0	SME2 multiple vectors long long MLA four sources	-
		x0	000	00		1	UNALLOCATED	-
		x0	010	00	xx1		UNALLOCATED	-
		x0	0x1	00			UNALLOCATED	-
		x0	100	00	x1x		SME2 multiple vectors ternary FP16 four registers	-
		x0	101	00	x0x		SME2 multiple vectors four-way dot product four registers	-
		x0	110	00	0xx		SME2 multiple vectors ternary FP four registers	-
		x0	110	00	1xx		SME2 multiple vectors ternary int four registers	-
		x0		1x			UNALLOCATED	-
		x1					UNALLOCATED	-
	!= 00	x0	111	00			UNALLOCATED	-

SME2 multiple vectors long FMA four sources

The encodings in this section are decoded from [SME2 Multi-vector - Multiple Array Vectors \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	0	1	Zm	0	1	0	Rv	0	1	0	Zn	0	0	op	S	0	off2						

Decode fields		Instruction Details	Feature
op	S		
0	0	FMLAL (multiple vectors, FP16 to FP32)	FEAT_SME2
0	1	FMLSL (multiple vectors)	FEAT_SME2
1	0	BFMLAL (multiple vectors)	FEAT_SME2
1	1	BFMLSL (multiple vectors)	FEAT_SME2

SME2 multiple vectors FP dot product four registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple Array Vectors \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	0	1	Zm	0	1	0	Rv	1	0	0	Zn	0	opc	0	off3								

Decode fields		Instruction Details	Feature
opc			
00		FDOT (2-way, multiple vectors, FP16 to FP32)	FEAT_SME2
01		BFDOT (multiple vectors)	FEAT_SME2
10		FDOT (2-way, multiple vectors, FP8 to FP16)	FEAT_SME_F8F16
11		FDOT (4-way, multiple vectors)	FEAT_SME_F8F32

SME2 multiple vectors long MLA four sources

The encodings in this section are decoded from [SME2 Multi-vector - Multiple Array Vectors \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	1	1	Zm	0	1	0	Rv	0	1	0	Zn	0	0	U	S	0	off2						

Decode fields		Instruction Details	Feature
U	S		
0	0	SMLAL (multiple vectors)	FEAT_SME2
0	1	SMLSL (multiple vectors)	FEAT_SME2
1	0	UMLAL (multiple vectors)	FEAT_SME2
1	1	UMLSL (multiple vectors)	FEAT_SME2

SME2 multiple vectors two-way dot product four registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple Array Vectors \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	1	1	Zm	0	1	0	Rv	1	0	1	Zn	0	0	U	1	off3							

Decode fields		Instruction Details	Feature
U			
0		SDOT (2-way, multiple vectors)	FEAT_SME2
1		UDOT (2-way, multiple vectors)	FEAT_SME2

SME2 multiple vectors binary FP four registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple Array Vectors \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	sz	1	0	0	0	0	1	0	Rv	1	1	1	Zm	0	0	0	S	off3					

Decode fields		Instruction Details	Feature
S			
0		FADD	FEAT_SME2
1		FSUB	FEAT_SME2

SME2 multiple vectors binary int four registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple Array Vectors \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	sz	1	0	0	0	0	1	0	Rv	1	1	1	Zm	0	0	1	S	off3					

Decode fields	Instruction Details	Feature
S		
0	ADD (to array, array and multiple vectors)	FEAT_SME2
1	SUB (to array, array and multiple vectors)	FEAT_SME2

SME2 multiple vectors binary FP16 four registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple Array Vectors \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	sz	1	0	0	1	0	1	0	Rv	1	1	1	Zm	0	0	0	S	off3					

Decode fields	Instruction Details	Feature
sz S		
0 0	FADD	FEAT_SME_F16F16 FEAT_SME_F8F16
0 1	FSUB	FEAT_SME_F16F16 FEAT_SME_F8F16
1 0	BFADD	FEAT_SME_B16B16
1 1	BFSUB	FEAT_SME_B16B16

SME2 multiple vectors long long MLA four sources

The encodings in this section are decoded from [SME2 Multi-vector - Multiple Array Vectors \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	sz	1	Zm	0	1	0	Rv	0	0	0	Zn	0	0	U	S	op	0	o1					

Decode fields	Instruction Details	Feature
sz U S op		
0 0 0 0	SMLALL (multiple vectors)	FEAT_SME2
0 0 1 0	SMLSLL (multiple vectors)	FEAT_SME2
1 0 0 0	UMLALL (multiple vectors)	FEAT_SME2
1 1 0 0	UMLSLL (multiple vectors)	FEAT_SME2
0 0 0 1	USMLALL (multiple vectors)	FEAT_SME2
0 0 1 1	UNALLOCATED	-
0 1 1 1	UNALLOCATED	-
1 1 1 1	UNALLOCATED	-

SME2 multiple vectors ternary FP16 four registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple Array Vectors \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	sz	1	Zm	0	1	0	Rv	1	0	0	Zn	0	0	S	1	off3							

Decode fields	Instruction Details	Feature
sz S		
0 0	FMLA (multiple vectors)	FEAT_SME_F16F16
0 1	FMLS (multiple vectors)	FEAT_SME_F16F16
1 0	BFMLA (multiple vectors)	FEAT_SME_B16B16
1 1	BFMLS (multiple vectors)	FEAT_SME_B16B16

SME2 multiple vectors four-way dot product four registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple Array Vectors \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	sz	1		Zm		0	1	0	Rv	1	0	1		Zn		0	0	U	0		off3		

Decode fields	Instruction Details	Feature
U		
0	SDOT (4-way, multiple vectors)	FEAT_SME2
1	UDOT (4-way, multiple vectors)	FEAT_SME2

SME2 multiple vectors ternary FP four registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple Array Vectors \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	sz	1		Zm		0	1	0	Rv	1	1	0		Zn		0	0	0	S		off3		

Decode fields	Instruction Details	Feature
S		
0	FMLA (multiple vectors)	FEAT_SME2
1	FMLS (multiple vectors)	FEAT_SME2

SME2 multiple vectors ternary int four registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple Array Vectors \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	sz	1		Zm		0	1	0	Rv	1	1	0		Zn		0	0	1	S		off3		

Decode fields	Instruction Details	Feature
S		
0	ADD (to array, multiple vectors)	FEAT_SME2
1	SUB (to array, multiple vectors)	FEAT_SME2

SME2 Multi-vector - Indexed (One register)

The encodings in this section are decoded from [SME encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1		op0	0	0									op1								op2			

Decode fields	Instruction details	Feature
op0 op1 op2		
00	SME2 multi-vec indexed long long MLA one source 32-bit	-
01	FMLALL (multiple and indexed vector) — One ZA quad-vector	FEAT_SME_F8F32
01	! = 000 UNALLOCATED	-
10	0 xx0 SME2 multi-vec indexed long long MLA one source 64-bit	-
10	0 xx1 UNALLOCATED	-
10	1 SME2 multi-vec indexed long FMA one source	-
11	0 0xx FMLAL (multiple and indexed vector, FP8 to FP16) — One ZA double-vector	FEAT_SME_F8F16
11	0 1xx UNALLOCATED	-
11	1 SME2 multi-vec indexed long MLA one source	-

SME2 multi-vec indexed long long MLA one source 32-bit

The encodings in this section are decoded from [SME2 Multi-vector - Indexed \(One register\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	0	0	0	Zm		i4h		Rv		i4l				Zn			U		S	op	off2		

Decode fields			Instruction Details	Feature
U	S	op		
	1	1	UNALLOCATED	-
0	0	0	SMLALL (multiple and indexed vector)	FEAT_SME2
0	0	1	USMLALL (multiple and indexed vector)	FEAT_SME2
0	1	0	SMLSLL (multiple and indexed vector)	FEAT_SME2
1	0	0	UMLALL (multiple and indexed vector)	FEAT_SME2
1	0	1	SUMLALL (multiple and indexed vector)	FEAT_SME2
1	1	0	UMLSLL (multiple and indexed vector)	FEAT_SME2

SME2 multi-vec indexed long long MLA one source 64-bit

The encodings in this section are decoded from [SME2 Multi-vector - Indexed \(One register\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	0	0	0		Zm		i3h		Rv		0		i3l			Zn		U		S		0	off2

Decode fields		Instruction Details	Feature
U	S		
0	0	SMLALL (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64
0	1	SMLSLL (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64
1	0	UMLALL (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64
1	1	UMLSLL (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64

SME2 multi-vec indexed long FMA one source

The encodings in this section are decoded from [SME2 Multi-vector - Indexed \(One register\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	0	0	0	Zm				i3h	Rv	1	i3l	Zn				op	S	off3					

Decode fields		Instruction Details	Feature
op	S		
0	0	FMLAL (multiple and indexed vector, FP16 to FP32)	FEAT_SME2
0	1	FMLSL (multiple and indexed vector)	FEAT_SME2
1	0	BFMLAL (multiple and indexed vector)	FEAT_SME2
1	1	BFMLSL (multiple and indexed vector)	FEAT_SME2

SME2 multi-vec indexed long MLA one source

The encodings in this section are decoded from [SME2 Multi-vector - Indexed \(One register\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	1	0	0	Zm				i3h	Rv		1	i3l		Zn				U	S	off3			

Decode fields		Instruction Details	Feature
U	S		
0	0	SMLAL (multiple and indexed vector)	FEAT_SME2
0	1	SMLSL (multiple and indexed vector)	FEAT_SME2
1	0	UMLAL (multiple and indexed vector)	FEAT_SME2
1	1	UMLSL (multiple and indexed vector)	FEAT_SME2

SME2 Multi-vector - Indexed (Two registers)

The encodings in this section are decoded from [SME encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	op0		0	1					0				op1						op2		op3			

Decode fields				Instruction details	Feature
op0	op1	op2	op3		
00	0x			SME2 multi-vec indexed long long MLA two sources 32-bit	-
00	1x			SME2 multi-vec ternary indexed two registers 16-bit	-
01				SME2 multi-vec ternary indexed two registers 32-bit	-
10	00	0		SME2 multi-vec indexed long long MLA two sources 64-bit	-
10	01	0		UNALLOCATED	-
10	0x	1	00	FMLALL (multiple and indexed vector) — Two ZA quad-vectors	FEAT_SME_F8F32
10	0x	1	!= 00	UNALLOCATED	-
10	1x	0		SME2 multi-vec indexed long FMA two sources	-
10	1x	1	0x	UNALLOCATED	-
10	1x	1	1x	FMLAL (multiple and indexed vector, FP8 to FP16) — Two ZA double-vectors	FEAT_SME_F8F16
11	00	0		SME2 multi-vec ternary indexed two registers 64-bit	-
11	01	0		SME2 multi-vec indexed FP8 two-way vertical dot product to single-precision two registers	-
11	1x	0		SME2 multi-vec indexed long MLA two sources	-
11		1	0x	SME2 multi-vec indexed FP8 two-way dot product to FP16 two registers	-
11		1	1x	UNALLOCATED	-

SME2 multi-vec indexed long long MLA two sources 32-bit

The encodings in this section are decoded from [SME2 Multi-vector - Indexed \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	0	0	1	Zm				0	Rv	0	i4h	Zn				op	U	S	i4l		o1		

Decode fields			Instruction Details	Feature
op	U	S		
0	0	0	SMLALL (multiple and indexed vector)	FEAT_SME2
0	0	1	SMLSLL (multiple and indexed vector)	FEAT_SME2
0	1	0	UMLALL (multiple and indexed vector)	FEAT_SME2
0	1	1	UMLSLL (multiple and indexed vector)	FEAT_SME2
1		1	UNALLOCATED	-
1	0	0	USMLALL (multiple and indexed vector)	FEAT_SME2
1	1	0	SUMLALL (multiple and indexed vector)	FEAT_SME2

SME2 multi-vec ternary indexed two registers 16-bit

The encodings in this section are decoded from [SME2 Multi-vector - Indexed \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	0	0	1	Zm		0		Rv		1	i3h	Zn		op		S	i3l	off3					

Decode fields		Instruction Details	Feature
op	S		
0	0	FMLA (multiple and indexed vector)	FEAT_SME_F16F16
0	1	FMLS (multiple and indexed vector)	FEAT_SME_F16F16
1	0	BFMLA (multiple and indexed vector)	FEAT_SME_B16B16
1	1	BFMLS (multiple and indexed vector)	FEAT_SME_B16B16

SME2 multi-vec ternary indexed two registers 32-bit

The encodings in this section are decoded from [SME2 Multi-vector - Indexed \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	1	0	1		Zm			0		Rv	op	i2			Zn			opc2			off3		

Decode fields		Instruction Details	Feature
op	opc2		
0	000	FMLA (multiple and indexed vector)	FEAT_SME2
0	001	FVDOT (FP16 to FP32)	FEAT_SME2
0	010	FMLS (multiple and indexed vector)	FEAT_SME2
0	011	BFVDOT	FEAT_SME2
0	100	SVDOT (2-way)	FEAT_SME2
0	101	UNALLOCATED	-
0	110	UVDOT (2-way)	FEAT_SME2
0	111	FDOT (4-way, multiple and indexed vector)	FEAT_SME_F8F32
1	000	SDOT (2-way, multiple and indexed vector)	FEAT_SME2
1	001	FDOT (2-way, multiple and indexed vector, FP16 to FP32)	FEAT_SME2
1	010	UDOT (2-way, multiple and indexed vector)	FEAT_SME2
1	011	BFDOT (multiple and indexed vector)	FEAT_SME2
1	100	SDOT (4-way, multiple and indexed vector)	FEAT_SME2
1	101	USDOT (4-way, multiple and indexed vector)	FEAT_SME2
1	110	UDOT (4-way, multiple and indexed vector)	FEAT_SME2
1	111	SUDOT (4-way, multiple and indexed vector)	FEAT_SME2

SME2 multi-vec indexed long long MLA two sources 64-bit

The encodings in this section are decoded from [SME2 Multi-vector - Indexed \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	0	0	1	Zm			0	Rv	0	0	i3h	Zn			0	U	S	i3l	o1				

Decode fields		Instruction Details	Feature
U	S		
0	0	SMLALL (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64
0	1	SMLSLL (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64
1	0	UMLALL (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64
1	1	UMLSLL (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64

SME2 multi-vec indexed long FMA two sources

The encodings in this section are decoded from [SME2 Multi-vector - Indexed \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	0	0	1		Zm			0		Rv	1	i3h		Zn			0	op	S	i3l	off2		

Decode fields		Instruction Details	Feature
op	S		
0	0	FMLAL (multiple and indexed vector, FP16 to FP32)	FEAT_SME2
0	1	FMLSL (multiple and indexed vector)	FEAT_SME2
1	0	BFMLAL (multiple and indexed vector)	FEAT_SME2
1	1	BFMLSL (multiple and indexed vector)	FEAT_SME2

SME2 multi-vec ternary indexed two registers 64-bit

The encodings in this section are decoded from [SME2 Multi-vector - Indexed \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	1	0	1	Zm			0	Rv		0	0	i1	Zn			0	opc		off3				

Decode fields	Instruction Details	Feature
opc		
00	FMLA (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_F64F64
01	SDOT (4-way, multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64
10	FMLS (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_F64F64
11	UDOT (4-way, multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64

SME2 multi-vec indexed FP8 two-way vertical dot product to single-precision two registers

The encodings in this section are decoded from [SME2 Multi-vector - Indexed \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	1	0	1	Zm				0	Rv	0	1	i2h	Zn				0	T	i2l	off3			

Decode fields	Instruction Details	Feature
T		
0	FVDOTB	FEAT_SME_F8F32
1	FVDOTT	FEAT_SME_F8F32

SME2 multi-vec indexed long MLA two sources

The encodings in this section are decoded from [SME2 Multi-vector - Indexed \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	1	0	1	Zm				0	Rv	1	i3h	Zn				0	U	S	i3l	off2			

Decode fields	Instruction Details	Feature
U	S	
0	0	SMLAL (multiple and indexed vector)
0	1	SMLSL (multiple and indexed vector)
1	0	UMLAL (multiple and indexed vector)
1	1	UMLSL (multiple and indexed vector)

SME2 multi-vec indexed FP8 two-way dot product to FP16 two registers

The encodings in this section are decoded from [SME2 Multi-vector - Indexed \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	1	0	1	Zm			0	Rv	op	i3h	Zn			1	0	i3l	off3						

Decode fields	Instruction Details	Feature
op		
0	FDOT (2-way, multiple and indexed vector, FP8 to FP16)	FEAT_SME_F8F16
1	FVDOT (FP8 to FP16)	FEAT_SME_F8F16

SME2 Multi-vector - Indexed (Four registers)

The encodings in this section are decoded from [SME encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	op0		0	1					1				op1						op2		op3			

Decode fields				Instruction details	Feature
op0	op1	op2	op3		
00	0x	0xx		SME2 multi-vec indexed long long MLA four sources 32-bit	-
00	0x	100	0	FMLALL (multiple and indexed vector) — Four ZA quad-vectors	FEAT_SME_F8F32
00	0x	100	1	UNALLOCATED	-
00	1x	0xx		SME2 multi-vec ternary indexed four registers 16-bit	-
00	1x	100		FDOT (2-way, multiple and indexed vector, FP8 to FP16) — Four ZA single-vectors	FEAT_SME_F8F16
00		101		UNALLOCATED	-
00		11x		UNALLOCATED	-
01		0xx		SME2 multi-vec ternary indexed four registers 32-bit	-
10	00	00x		SME2 multi-vec indexed long long MLA four sources 64-bit	-
10	01	00x		UNALLOCATED	-
10	0x	01x		UNALLOCATED	-
10	1x	00x		SME2 multi-vec indexed long FMA four sources	-
10	1x	010		FMLAL (multiple and indexed vector, FP8 to FP16) — Four ZA double-vectors	FEAT_SME_F8F16
10	1x	011		UNALLOCATED	-
11	0x	00x		SME2 multi-vec ternary indexed four registers 64-bit	-
11	1x	00x		SME2 multi-vec indexed long MLA four sources	-
11		01x		UNALLOCATED	-
!= 00		1xx		UNALLOCATED	-

SME2 multi-vec indexed long long MLA four sources 32-bit

The encodings in this section are decoded from [SME2 Multi-vector - Indexed \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	0	0	1	Zm				1	Rv		0	i4h	Zn		0	op	U	S	i4l		o1		

Decode fields			Instruction Details	Feature
op	U	S		
0	0	0	SMLALL (multiple and indexed vector)	FEAT_SME2
0	0	1	SMLSLL (multiple and indexed vector)	FEAT_SME2
0	1	0	UMLALL (multiple and indexed vector)	FEAT_SME2
0	1	1	UMLSLL (multiple and indexed vector)	FEAT_SME2
1		1	UNALLOCATED	-
1	0	0	USMLALL (multiple and indexed vector)	FEAT_SME2
1	1	0	SUMLALL (multiple and indexed vector)	FEAT_SME2

SME2 multi-vec ternary indexed four registers 16-bit

The encodings in this section are decoded from [SME2 Multi-vector - Indexed \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	0	0	1	Zm		1		Rv		1	i3h	Zn		0	op	S	i3l	off3					

Decode fields		Instruction Details	Feature
op	S		
0	0	FMLA (multiple and indexed vector)	FEAT_SME_F16F16
0	1	FMLS (multiple and indexed vector)	FEAT_SME_F16F16
1	0	BFMLA (multiple and indexed vector)	FEAT_SME_B16B16
1	1	BFMLS (multiple and indexed vector)	FEAT_SME_B16B16

SME2 multi-vec ternary indexed four registers 32-bit

The encodings in this section are decoded from [SME2 Multi-vector - Indexed \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	1	0	1		Zm			1		Rv	op	i2		Zn		0		opc2			off3		

Decode fields		Instruction Details	Feature
op	opc2		
0	000	FMLA (multiple and indexed vector)	FEAT_SME2
0	001	FDOT (4-way, multiple and indexed vector)	FEAT_SME_F8F32
0	010	FMLS (multiple and indexed vector)	FEAT_SME2
0	011	UNALLOCATED	-
0	100	SVDOT (4-way)	FEAT_SME2
0	101	USVDOT	FEAT_SME2
0	110	UVDOT (4-way)	FEAT_SME2
0	111	SUVDOT	FEAT_SME2
1	000	SDOT (2-way, multiple and indexed vector)	FEAT_SME2
1	001	FDOT (2-way, multiple and indexed vector, FP16 to FP32)	FEAT_SME2
1	010	UDOT (2-way, multiple and indexed vector)	FEAT_SME2
1	011	BFDOT (multiple and indexed vector)	FEAT_SME2
1	100	SDOT (4-way, multiple and indexed vector)	FEAT_SME2
1	101	USDOT (4-way, multiple and indexed vector)	FEAT_SME2
1	110	UDOT (4-way, multiple and indexed vector)	FEAT_SME2
1	111	SUDOT (4-way, multiple and indexed vector)	FEAT_SME2

SME2 multi-vec indexed long long MLA four sources 64-bit

The encodings in this section are decoded from [SME2 Multi-vector - Indexed \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	0	0	1	Zm			1	Rv	0	0	i3h	Zn		0	0	U	S	i3l	o1				

Decode fields		Instruction Details	Feature
U	S		
0	0	SMLALL (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64
0	1	SMLSLL (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64
1	0	UMLALL (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64
1	1	UMLSLL (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64

SME2 multi-vec indexed long FMA four sources

The encodings in this section are decoded from [SME2 Multi-vector - Indexed \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	0	0	1	Zm			1	Rv		1	i3h		Zn		0	0	op	S	i3l	off2			

Decode fields		Instruction Details	Feature
op	S		
0	0	FMLAL (multiple and indexed vector, FP16 to FP32)	FEAT_SME2
0	1	FMLSL (multiple and indexed vector)	FEAT_SME2
1	0	BFMLAL (multiple and indexed vector)	FEAT_SME2
1	1	BFMLSL (multiple and indexed vector)	FEAT_SME2

SME2 multi-vec ternary indexed four registers 64-bit

The encodings in this section are decoded from [SME2 Multi-vector - Indexed \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	1	0	1		Zm			1		Rv	0	op	i1		Zn		0	0	opc2		off3		

Decode fields		Instruction Details	Feature
op	opc2		
0	00	FMLA (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_F64F64
0	01	SDOT (4-way, multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64
0	10	FMLS (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_F64F64
0	11	UDOT (4-way, multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64
1	×0	UNALLOCATED	-
1	01	SVDOT (4-way)	FEAT_SME2 && FEAT_SME_I16I64
1	11	UVDOT (4-way)	FEAT_SME2 && FEAT_SME_I16I64

SME2 multi-vec indexed long MLA four sources

The encodings in this section are decoded from [SME2 Multi-vector - Indexed \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	1	0	1		Zm			1		Rv	1	i3h		Zn		0	0	U	S	i3l		off2	

Decode fields		Instruction Details	Feature
U	S		
0	0	SMLAL (multiple and indexed vector)	FEAT_SME2
0	1	SMLSL (multiple and indexed vector)	FEAT_SME2
1	0	UMLAL (multiple and indexed vector)	FEAT_SME2
1	1	UMLSL (multiple and indexed vector)	FEAT_SME2

SME2 Multi-vector - Multiple and Single SVE Destructive (Two registers)

The encodings in this section are decoded from [SME encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1			1	0					1	0	1	0	0	op0	op1		op2							

Decode fields			Instruction details
op0	op1	op2	
0	00	00x	SME2 single-multi int min/max two registers
0	00	101	UNALLOCATED
0	01	00x	SME2 single-multi FP min/max two registers
0	01	100	SME2 multiple and single vector FSCALE two registers
0	01	101	UNALLOCATED
0	0x	x1x	UNALLOCATED
0	10		SME2 single-multi shift two registers
0	11	000	SME2 single-multi add two registers
0	11	001!=000	UNALLOCATED
0	11	10x	UNALLOCATED
0	!= 1x	101	UNALLOCATED
0	!= 10	x1x	UNALLOCATED
1	00	000	SME2 single-multi signed saturating doubling multiply high two registers
1	00	xx1	UNALLOCATED
1	00	!= x00x10	UNALLOCATED
1	!= 00		UNALLOCATED
	00	100	UNALLOCATED

SME2 single-multi int min/max two registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple and Single SVE Destructive \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size	1	0		Zm	1	0	1	0	0	0	0	0	0	0	0	0	op		Zdn				U

Decode fields		Instruction Details	Feature
op	U		
0	0	SMAX (multiple and single vector)	FEAT_SME2
0	1	UMAX (multiple and single vector)	FEAT_SME2
1	0	SMIN (multiple and single vector)	FEAT_SME2
1	1	UMIN (multiple and single vector)	FEAT_SME2

SME2 single-multi FP min/max two registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple and Single SVE Destructive \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size	1	0		Zm	1	0	1	0	0	0	0	0	0	0	1	0	0	op		Zdn			o2

Decode fields			Instruction Details	Feature
size	op	o2		
00	0	0	BFMAX (multiple and single vector)	FEAT_SME2 && FEAT_SVE_B16B16
00	0	1	BFMIN (multiple and single vector)	FEAT_SME2 && FEAT_SVE_B16B16
00	1	0	BFMAXNM (multiple and single vector)	FEAT_SME2 && FEAT_SVE_B16B16
00	1	1	BFMINNM (multiple and single vector)	FEAT_SME2 && FEAT_SVE_B16B16
!= 00	0	0	FMAX (multiple and single vector)	FEAT_SME2
!= 00	0	1	FMIN (multiple and single vector)	FEAT_SME2
!= 00	1	0	FMAXNM (multiple and single vector)	FEAT_SME2
!= 00	1	1	FMINNM (multiple and single vector)	FEAT_SME2

SME2 multiple and single vector FSCALE two registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple and Single SVE Destructive \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size	1	0		Zm	1	0	1	0	0	0	0	0	0	1	1	0	0		Zdn				op

Decode fields		Instruction Details										Feature									
size	op																				
	1	UNALLOCATED										-									
00	0	BFSCALE (multiple and single vector)										FEAT_SME2 && FEAT_SVE_BFSCALE									
!= 00	0	FSCALE (multiple and single vector)										FEAT_SME2 && FEAT_FP8									

SME2 single-multi shift two registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple and Single SVE Destructive \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size	1	0	Zm				1	0	1	0	0	0	1	0	opc		Zdn				U		

Decode fields		Instruction Details										Feature									
opc	U																				
001	0	SRSHL (multiple and single vector)										FEAT_SME2									
001	1	URSHL (multiple and single vector)										FEAT_SME2									
!= 001		UNALLOCATED										-									

SME2 single-multi add two registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple and Single SVE Destructive \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size	1	0		Zm	1	0	1	0	0	0	0	0	1	1	0	0	0		Zdn				op

Decode fields		Instruction Details										Feature									
op																					
0		ADD (to vector, multiple vectors)										FEAT_SME2									
1		UNALLOCATED										-									

SME2 single-multi signed saturating doubling multiply high two registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple and Single SVE Destructive \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size	1	0		Zm	1	0	1	0	0	1	0	0	1	0	0	0	0	0		Zdn			op

Decode fields		Instruction Details										Feature									
op																					
0		SQDMULH (multiple and single vector)										FEAT_SME2									
1		UNALLOCATED										-									

SME2 Multi-vector - Multiple and Single SVE Destructive (Four registers)

The encodings in this section are decoded from [SME encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1			1	0					1	0	1	0	1	op0	op1		op2						0	

Decode fields			Instruction details
op0	op1	op2	
0	00	00x	SME2 single-multi int min/max four registers
0	00	101	UNALLOCATED
0	01	00x	SME2 single-multi FP min/max four registers
0	01	100	SME2 multiple and single vector FSCALE four registers
0	01	101	UNALLOCATED
0	0x	x1x	UNALLOCATED
0	10		SME2 single-multi shift four registers
0	11	000	SME2 single-multi add four registers
0	11	001!=000	UNALLOCATED
0	11	10x	UNALLOCATED
0	!= 1x	101	UNALLOCATED
0	!= 10	x1x	UNALLOCATED
1	00	000	SME2 single-multi signed saturating doubling multiply high four registers
1	00	xx1	UNALLOCATED
1	00	!= x00x10	UNALLOCATED
1	!= 00		UNALLOCATED
	00	100	UNALLOCATED

SME2 single-multi int min/max four registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple and Single SVE Destructive \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size	1	0			Zm			1	0	1	0	1	0	0	0	0	0	op		Zdn		0	U

Decode fields		Instruction Details	Feature
op	U		
0	0	SMAX (multiple and single vector)	FEAT_SME2
0	1	UMAX (multiple and single vector)	FEAT_SME2
1	0	SMIN (multiple and single vector)	FEAT_SME2
1	1	UMIN (multiple and single vector)	FEAT_SME2

SME2 single-multi FP min/max four registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple and Single SVE Destructive \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size	1	0	Zm				1	0	1	0	1	0	0	1	0	0	op	Zdn			0	o2	

Decode fields			Instruction Details	Feature
size	op	o2		
00	0	0	BFMAX (multiple and single vector)	FEAT_SME2 && FEAT_SVE_B16B16
00	0	1	BFMIN (multiple and single vector)	FEAT_SME2 && FEAT_SVE_B16B16
00	1	0	BFMAXNM (multiple and single vector)	FEAT_SME2 && FEAT_SVE_B16B16
00	1	1	BFMINNM (multiple and single vector)	FEAT_SME2 && FEAT_SVE_B16B16
!= 00	0	0	FMAX (multiple and single vector)	FEAT_SME2
!= 00	0	1	FMIN (multiple and single vector)	FEAT_SME2
!= 00	1	0	FMAXNM (multiple and single vector)	FEAT_SME2
!= 00	1	1	FMINNM (multiple and single vector)	FEAT_SME2

SME2 multiple and single vector FSCALE four registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple and Single SVE Destructive \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size	1	0		Zm				1	0	1	0	1	0	0	1	1	0	0		Zdn		0	op

Decode fields		Instruction Details										Feature															
size	op																										
	1	UNALLOCATED										-															
00	0	BFSCALE (multiple and single vector)										FEAT_SME2 && FEAT_SVE_BFSCALE															
!= 00	0	FSCALE (multiple and single vector)										FEAT_SME2 && FEAT_FP8															

SME2 single-multi shift four registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple and Single SVE Destructive \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size	1	0		Zm				1	0	1	0	1	0	1	0	1	0	opc		Zdn		0	U

Decode fields		Instruction Details										Feature															
opc	U																										
001	0	SRSHL (multiple and single vector)										FEAT_SME2															
001	1	URSHL (multiple and single vector)										FEAT_SME2															
!= 001		UNALLOCATED										-															

SME2 single-multi add four registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple and Single SVE Destructive \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size	1	0		Zm				1	0	1	0	1	0	1	1	0	0	0		Zdn		0	op

Decode fields		Instruction Details										Feature															
op																											
0		ADD (to vector, multiple vectors)										FEAT_SME2															
1		UNALLOCATED										-															

SME2 single-multi signed saturating doubling multiply high four registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple and Single SVE Destructive \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size	1	0		Zm				1	0	1	0	1	1	0	0	0	0	0		Zdn		0	op

Decode fields		Instruction Details										Feature															
op																											
0		SQDMULH (multiple and single vector)										FEAT_SME2															
1		UNALLOCATED										-															

SME2 Multi-vector - Multiple Vectors SVE Destructive (Four registers)

The encodings in this section are decoded from [SME encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1			1				0	0	1	0	1	1	1	0		op0							0	

Decode fields

Instruction details

op0	
000	SME2 multiple vectors int min/max four registers
001	UNALLOCATED
010	SME2 multiple vectors FP min/max four registers
011	SME2 multiple vectors FSCALE four registers
10x	SME2 multiple vectors shift four registers
11x	UNALLOCATED

SME2 multiple vectors int min/max four registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple Vectors SVE Destructive \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size	1		Zm	0	0	1	0	1	1	1	0	0	0	0	opc		Zdn				0	U	

Decode fields

Instruction Details

Feature

opc	U		
00	0	SMAX (multiple vectors)	FEAT_SME2
00	1	UMAX (multiple vectors)	FEAT_SME2
01	0	SMIN (multiple vectors)	FEAT_SME2
01	1	UMIN (multiple vectors)	FEAT_SME2
1x		UNALLOCATED	-

SME2 multiple vectors FP min/max four registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple Vectors SVE Destructive \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size	1		Zm	0	0	1	0	1	1	1	0	0	1	0	opc		Zdn				0	o2	

Decode fields

Instruction Details

Feature

size	opc	o2		
	10	0	FAMAX	FEAT_SME2 && FEAT_FAMINMAX
	10	1	FAMIN	FEAT_SME2 && FEAT_FAMINMAX
	11		UNALLOCATED	-
00	00	0	BFMAX (multiple vectors)	FEAT_SME2 && FEAT_SVE_B16B16
00	00	1	BFMIN (multiple vectors)	FEAT_SME2 && FEAT_SVE_B16B16
00	01	0	BFMAXNM (multiple vectors)	FEAT_SME2 && FEAT_SVE_B16B16
00	01	1	BFMINNM (multiple vectors)	FEAT_SME2 && FEAT_SVE_B16B16
!= 00	00	0	FMAX (multiple vectors)	FEAT_SME2
!= 00	00	1	FMIN (multiple vectors)	FEAT_SME2
!= 00	01	0	FMAXNM (multiple vectors)	FEAT_SME2
!= 00	01	1	FMINNM (multiple vectors)	FEAT_SME2

SME2 multiple vectors FSCALE four registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple Vectors SVE Destructive \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size	1		Zm	0	0	1	0	1	1	1	0	0	1	1	opc		Zdn			0	o2		

Decode fields			Instruction Details	Feature
size	opc	o2		
	00	1	UNALLOCATED	-
	!= 00		UNALLOCATED	-
00	00	0	BFSCALE (multiple vectors)	FEAT_SME2 && FEAT_SVE_BFSCALE
!= 00	00	0	FSCALE (multiple vectors)	FEAT_SME2 && FEAT_FP8

SME2 multiple vectors shift four registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple Vectors SVE Destructive \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size	1		Zm	0	0	1	0	1	1	1	0	1	0		opc		Zdn	0	U				

Decode fields		U	Instruction Details	Feature
opc				
001		0	SRSHL (multiple vectors)	FEAT_SME2
001		1	URSHL (multiple vectors)	FEAT_SME2
!= 001			UNALLOCATED	-

SME2 Multi-vector - Multiple Vectors SVE Saturating Multiply (Four registers)

The encodings in this section are decoded from [SME encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1			1				0	0	1	0	1	1	1	1	op0								0	

Decode fields		Instruction details
op0		
00000		SME2 multi-vector signed saturating doubling multiply high four registers
!= 00000		UNALLOCATED

SME2 multi-vector signed saturating doubling multiply high four registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple Vectors SVE Saturating Multiply \(Four registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size	1		Zm	0	0	1	0	1	1	1	1	1	0	0	0	0	0		Zdn		0	op	

Decode fields		Instruction Details	Feature
op			
0		SQDMULH (multiple vectors)	FEAT_SME2
1		UNALLOCATED	-

SME2 Multi-vector - Multiple Vectors SVE Destructive (Two registers)

The encodings in this section are decoded from [SME encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1			1				0	1	0	1	1	0	0		op0									

Decode fields		Instruction details
op0		
000		SME2 multiple vectors int min/max two registers
001		UNALLOCATED
010		SME2 multiple vectors FP min/max two registers
011		SME2 multiple vectors FSCALE two registers
10x		SME2 multiple vectors shift two registers
11x		UNALLOCATED

SME2 multiple vectors int min/max two registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple Vectors SVE Destructive \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size	1		Zm				0	1	0	1	1	0	0	0	0	0	opc		Zdn				U

Decode fields		Instruction Details														Feature										
opc	U																									
00	0	SMAX (multiple vectors)														FEAT_SME2										
00	1	UMAX (multiple vectors)														FEAT_SME2										
01	0	SMIN (multiple vectors)														FEAT_SME2										
01	1	UMIN (multiple vectors)														FEAT_SME2										
1x		UNALLOCATED														-										

SME2 multiple vectors FP min/max two registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple Vectors SVE Destructive \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size	1		Zm				0	1	0	1	1	0	0	0	1	0	opc		Zdn				o2

Decode fields			Instruction Details														Feature										
size	opc	o2																									
	10	0	FAMAX														FEAT_SME2 && FEAT_FAMINMAX										
	10	1	FAMIN														FEAT_SME2 && FEAT_FAMINMAX										
	11		UNALLOCATED														-										
00	00	0	BFMAX (multiple vectors)														FEAT_SME2 && FEAT_SVE_B16B16										
00	00	1	BFMIN (multiple vectors)														FEAT_SME2 && FEAT_SVE_B16B16										
00	01	0	BFMAXNM (multiple vectors)														FEAT_SME2 && FEAT_SVE_B16B16										
00	01	1	BFMINNM (multiple vectors)														FEAT_SME2 && FEAT_SVE_B16B16										
!= 00	00	0	FMAX (multiple vectors)														FEAT_SME2										
!= 00	00	1	FMIN (multiple vectors)														FEAT_SME2										
!= 00	01	0	FMAXNM (multiple vectors)														FEAT_SME2										
!= 00	01	1	FMINNM (multiple vectors)														FEAT_SME2										

SME2 multiple vectors FSCALE two registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple Vectors SVE Destructive \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size	1		Zm				0	1	0	1	1	0	0	0	1	1	opc		Zdn				o2

Decode fields			Instruction Details														Feature										
size	opc	o2																									
	00	1	UNALLOCATED														-										
	!= 00		UNALLOCATED														-										
00	00	0	BFSCALE (multiple vectors)														FEAT_SME2 && FEAT_SVE_BFSCALE										
!= 00	00	0	FSCALE (multiple vectors)														FEAT_SME2 && FEAT_FP8										

SME2 multiple vectors shift two registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple Vectors SVE Destructive \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size	1		Zm				0	1	0	1	1	0	0	1	0	opc		Zdn					U

Decode fields		Instruction Details	Feature
opc	U		
001	0	SRSHL (multiple vectors)	FEAT_SME2
001	1	URSHL (multiple vectors)	FEAT_SME2
!= 001		UNALLOCATED	-

SME2 Multi-vector - Multiple Vectors SVE Saturating Multiply (Two registers)

The encodings in this section are decoded from [SME encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1			1					0	1	0	1	1	0	1			op0							

Decode fields	Instruction details
op0	
00000	SME2 multi-vector signed saturating doubling multiply high two registers
!= 00000	UNALLOCATED

SME2 multi-vector signed saturating doubling multiply high two registers

The encodings in this section are decoded from [SME2 Multi-vector - Multiple Vectors SVE Saturating Multiply \(Two registers\)](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size	1	Zm				0	1	0	1	1	0	1	0	0	0	0	0	Zdn				op	

Decode fields	Instruction Details	Feature
op		
0	SQDMULH (multiple vectors)	FEAT_SME2
1	UNALLOCATED	-

SME2 Multi-vector - SVE Select

The encodings in this section are decoded from [SME encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1			1					op0	1	0	0							op1						op2

Decode fields			Instruction details	Feature
op0	op1	op2		
01	00	00	SEL — Four registers	FEAT_SME2
01	00	10	UNALLOCATED	-
01	10	x0	UNALLOCATED	-
11	x0	x0	UNALLOCATED	-
x0	x0	x0	SEL — Two registers	FEAT_SME2
	x0	x1	UNALLOCATED	-
	x1		UNALLOCATED	-

SME2 Multi-vector - SVE Constructive Binary

The encodings in this section are decoded from [SME encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	op0	1							1	1	0		op1											op2

Decode fields			Instruction details
op0	op1	op2	
00	101		SME2 multi-vec quadwords ZIP two registers
01	101		UNALLOCATED
10	101		UNALLOCATED
11	101		SME2 multi-vec saturating shift right narrow two registers
	000		SME2 multi-vec FCLAMP two registers
	001		SME2 multi-vec CLAMP two registers
	010	0	SME2 multi-vec FCLAMP four registers
	011	0	SME2 multi-vec CLAMP four registers
	01x	1	UNALLOCATED
	100		SME2 multi-vec ZIP two registers
	11x		SME2 multi-vec saturating shift right narrow four registers

SME2 multi-vec quadwords ZIP two registers

The encodings in this section are decoded from [SME2 Multi-vector - SVE Constructive Binary](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	0	1	Zm				1	1	0	1	0	1	Zn				Zd			op			

Decode fields		Instruction Details	Feature
op			
0		ZIP (two registers)	FEAT_SME2
1		UZP (two registers)	FEAT_SME2

SME2 multi-vec saturating shift right narrow two registers

The encodings in this section are decoded from [SME2 Multi-vector - SVE Constructive Binary](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	1	1	op	imm4				1	1	0	1	0	1	Zn			U	Zd					

Decode fields		Instruction Details	Feature
op	U		
0	0	SQRSHR (two registers)	FEAT_SME2
0	1	UQRSHR (two registers)	FEAT_SME2
1	0	SQRSHRU (two registers)	FEAT_SME2
1	1	UNALLOCATED	-

SME2 multi-vec FCLAMP two registers

The encodings in this section are decoded from [SME2 Multi-vector - SVE Constructive Binary](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size	1	Zm				1	1	0	0	0	0	Zn				Zd				op			

Decode fields		Instruction Details	Feature
size	op		
	1	UNALLOCATED	-
00	0	BFCLAMP	FEAT_SME2 && FEAT_SVE_B16B16
!= 00	0	FCLAMP	FEAT_SME2

SME2 multi-vec CLAMP two registers

The encodings in this section are decoded from [SME2 Multi-vector - SVE Constructive Binary](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size	1	Zm						1	1	0	0	0	1	Zn						Zd			U

Decode fields	Instruction Details	Feature
U		
0	SCLAMP	FEAT_SME2
1	UCLAMP	FEAT_SME2

SME2 multi-vec FCLAMP four registers

The encodings in this section are decoded from [SME2 Multi-vector - SVE Constructive Binary](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	0	0	0	0	0	1	size	1	Zm						1	1	0	0	1	0	Zn						Zd			0	op

Decode fields	Instruction Details	Feature
size op		
	1	UNALLOCATED
00	0	BFCLAMP
!= 00	0	FCLAMP
		FEAT_SME2

SME2 multi-vec CLAMP four registers

The encodings in this section are decoded from [SME2 Multi-vector - SVE Constructive Binary](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	0	0	0	0	0	1	size	1	Zm						1	1	0	0	1	1	Zn						Zd			0	U

Decode fields	Instruction Details	Feature
U		
0	SCLAMP	FEAT_SME2
1	UCLAMP	FEAT_SME2

SME2 multi-vec ZIP two registers

The encodings in this section are decoded from [SME2 Multi-vector - SVE Constructive Binary](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size	1	Zm						1	1	0	1	0	0	Zn						Zd			op

Decode fields	Instruction Details	Feature
op		
0	ZIP (two registers)	FEAT_SME2
1	UZP (two registers)	FEAT_SME2

SME2 multi-vec saturating shift right narrow four registers

The encodings in this section are decoded from [SME2 Multi-vector - SVE Constructive Binary](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	tsize	1	imm5				1	1	0	1	1	N	Zn		op	U	Zd							

Decode fields			Instruction Details	Feature
N	op	U		
	1	1	UNALLOCATED	-
0	0	0	SQRSHR (four registers)	FEAT_SME2
0	0	1	UQRSHR (four registers)	FEAT_SME2
0	1	0	SQRSHRU (four registers)	FEAT_SME2
1	0	0	SQRSHRN	FEAT_SME2
1	0	1	UQRSHRN	FEAT_SME2
1	1	0	SQRSHRUN	FEAT_SME2

SME2 Multi-vector - SVE Constructive Unary

The encodings in this section are decoded from [SME encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	op0	1	op1	op2	1	1	1	0	0	0								op3					op4	

Decode fields					Instruction details
op0	op1	op2	op3	op4	
00	000	01		x0	SME2 multi-vec FP to int convert two registers
00	000	10		x0	SME2 multi-vec int to FP two registers
00	001	11			UNALLOCATED
00	100	01	0x	00	SME2 multi-vec FP to int convert four registers
00	100	10	0x	00	SME2 multi-vec int to FP four registers
00	101	00	0x		SME2 multi-vec FP8 down convert four registers
00	101	11	00	x0	SME2 multi-vec quadwords ZIP four registers
01	000	01		x0	UNALLOCATED
01	001	11			UNALLOCATED
01	101	00	0x	1x	UNALLOCATED
01	10x	00	0x	0x	UNALLOCATED
0x	000	00			SME2 multi-vec FP down convert two registers
0x	000	11			SME2 multi-vec int down convert two registers
0x	001	00	x0		SME2 multi-vec FP8 down convert two registers
0x	001	00	x1		UNALLOCATED
10	000	00			SME2 multi-vec convert two registers
10	001	00	1x		UNALLOCATED
10	x01	00	0x		UNALLOCATED
11	00x	00			UNALLOCATED
11	101	00	0x	1x	UNALLOCATED
11	10x	00	0x	0x	UNALLOCATED
1x	000	11		x1	UNALLOCATED
1x	000	x1		x0	UNALLOCATED
1x	001	11			UNALLOCATED
x0	100	00	0x	0x	UNALLOCATED
	000	01		x1	UNALLOCATED
	000	10		x1	UNALLOCATED
	001	01			SME2 multi-vec unpack two registers
	001	10			SME2 multi-vec FP8 up convert two registers
	01x		x0	x0	SME2 multi-vec FRINT two registers
	100	01	0x	01	UNALLOCATED
	100	01	1x		UNALLOCATED
	100	10	0x	01	UNALLOCATED
	100	11			SME2 multi-vec int down convert four registers
	100	!= 11	0x	1x	UNALLOCATED
	101	01	x0	0x	SME2 multi-vec unpack four registers
	101	01	x0	1x	UNALLOCATED
	101	01	x1		UNALLOCATED
	101	10	00	x0	SME2 multi-vec ZIP four registers
	101	11	1x		UNALLOCATED
	101	1x	00	x1	UNALLOCATED
	101	1x	01		UNALLOCATED
	10x	x0	1x		UNALLOCATED
	11x		00	00	SME2 multi-vec FRINT four registers
	11x		00	10	UNALLOCATED

	11x		10	x0	UNALLOCATED
	x1x		x0	x1	UNALLOCATED
	x1x		x1		UNALLOCATED
!= x0	101	00	0x	1x	UNALLOCATED
!= x0	10x	00	0x	0x	UNALLOCATED
!= 00	000	10		x0	UNALLOCATED
!= 00	100	01	0x	00	UNALLOCATED
!= 00	100	10	0x	00	UNALLOCATED
!= 00	101	11	00	x0	UNALLOCATED
!= 01	001	11			UNALLOCATED

SME2 multi-vec FP to int convert two registers

The encodings in this section are decoded from [SME2 Multi-vector - SVE Constructive Unary](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	0	1	0	0	0	0	1	1	1	1	0	0	0			Zn		U			Zd		0

Decode fields	Instruction Details		Feature
U			
0	FCVTZS		FEAT_SME2
1	FCVTZU		FEAT_SME2

SME2 multi-vec int to FP two registers

The encodings in this section are decoded from [SME2 Multi-vector - SVE Constructive Unary](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	0	1	0	0	0	1	0	1	1	1	0	0	0			Zn		U			Zd		0

Decode fields	Instruction Details		Feature
U			
0	SCVTF		FEAT_SME2
1	UCVTF		FEAT_SME2

SME2 multi-vec FP to int convert four registers

The encodings in this section are decoded from [SME2 Multi-vector - SVE Constructive Unary](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	0	1	1	0	0	0	1	1	1	1	0	0	0	Zn			0	U	Zd			0	0

Decode fields	Instruction Details		Feature
U			
0	FCVTZS		FEAT_SME2
1	FCVTZU		FEAT_SME2

SME2 multi-vec int to FP four registers

The encodings in this section are decoded from [SME2 Multi-vector - SVE Constructive Unary](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	0	1	1	0	0	1	0	1	1	1	0	0	0	Zn			0	U	Zd			0	0

Decode fields	Instruction Details		Feature
U			
0	SCVTF		FEAT_SME2
1	UCVTF		FEAT_SME2

SME2 multi-vec FP8 down convert four registers

The encodings in this section are decoded from [SME2 Multi-vector - SVE Constructive Unary](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	0	1	1	0	1	0	0	1	1	1	0	0	0		Zn		0	N			Zd		

Decode fields	Instruction Details																Feature									
N																										
0	FCVT (narrowing, FP32 to FP8)																FEAT_SME2 && FEAT_FP8									
1	FCVTN (FP32 to FP8)																FEAT_SME2 && FEAT_FP8									

SME2 multi-vec quadwords ZIP four registers

The encodings in this section are decoded from [SME2 Multi-vector - SVE Constructive Unary](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	0	1	1	0	1	1	1	1	1	1	0	0	0	Zn			0	0	Zd		op	0	

Decode fields	Instruction Details																Feature									
op																										
0	ZIP (four registers)																FEAT_SME2									
1	UZP (four registers)																FEAT_SME2									

SME2 multi-vec FP down convert two registers

The encodings in this section are decoded from [SME2 Multi-vector - SVE Constructive Unary](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	op	1	0	0	0	0	0	1	1	1	0	0	0		Zn		N			Zd			

Decode fields		Instruction Details	Feature
op	N		
0	0	FCVT (narrowing, FP32 to FP16)	FEAT_SME2
0	1	FCVTN (FP32 to FP16)	FEAT_SME2
1	0	BFCVT (single-precision to BFloat16)	FEAT_SME2
1	1	BFCVTN	FEAT_SME2

SME2 multi-vec int down convert two registers

The encodings in this section are decoded from [SME2 Multi-vector - SVE Constructive Unary](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	op	1	0	0	0	1	1	1	1	1	0	0	0		Zn		U			Zd			

Decode fields		Instruction Details	Feature
op	U		
0	0	SQCVT (two registers)	FEAT_SME2
0	1	UQCVT (two registers)	FEAT_SME2
1	0	SQCVTU (two registers)	FEAT_SME2
1	1	UNALLOCATED	-

SME2 multi-vec FP8 down convert two registers

The encodings in this section are decoded from [SME2 Multi-vector - SVE Constructive Unary](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	op	1	0	0	1	0	0	1	1	1	0	0	0		Zn		0			Zd			

Decode fields	Instruction Details	Feature
op		
0	FCVT (narrowing, FP16 to FP8)	FEAT_SME2 && FEAT_FP8
1	BFCVT (BFloat16 to 8-bit floating-point)	FEAT_SME2 && FEAT_FP8

SME2 multi-vec convert two registers

The encodings in this section are decoded from [SME2 Multi-vector - SVE Constructive Unary](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	0	1	0	0	0	0	0	1	1	1	0	0	0	Zn				Zd			L		

Decode fields	Instruction Details	Feature
L		
0	FCVT (widening)	FEAT_SME_F16F16
1	FCVTL	FEAT_SME_F16F16

SME2 multi-vec unpack two registers

The encodings in this section are decoded from [SME2 Multi-vector - SVE Constructive Unary](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size	1	0	0	1	0	1	1	1	1	0	0	0	Zn				Zd				U		

Decode fields	Instruction Details	Feature
U		
0	SUNPK	FEAT_SME2
1	UUNPK	FEAT_SME2

SME2 multi-vec FP8 up convert two registers

The encodings in this section are decoded from [SME2 Multi-vector - SVE Constructive Unary](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	opc	1	0	0	1	1	0	1	1	1	0	0	0	Zn					Zd			L		

Decode fields	Instruction Details	Feature
opc	L	
00	0	F1CVT, F2CVT — F1CVT
00	1	F1CVTL, F2CVTL — F1CVTL
01	0	BF1CVT, BF2CVT — BF1CVT
01	1	BF1CVTL, BF2CVTL — BF1CVTL
10	0	F1CVT, F2CVT — F2CVT
10	1	F1CVTL, F2CVTL — F2CVTL
11	0	BF1CVT, BF2CVT — BF2CVT
11	1	BF1CVTL, BF2CVTL — BF2CVTL

SME2 multi-vec FRINT two registers

The encodings in this section are decoded from [SME2 Multi-vector - SVE Constructive Unary](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size	1	0	1	opc			1				1	1	0	0	0	Zn			0	Zd			0

Decode fields		Instruction Details		Feature
size	opc			
10	000	FRINTN		FEAT_SME2
10	001	FRINTP		FEAT_SME2
10	010	FRINTM		FEAT_SME2
10	011	UNALLOCATED		-
10	100	FRINTA		FEAT_SME2
10	101	UNALLOCATED		-
10	11x	UNALLOCATED		-
!= 10		UNALLOCATED		-

SME2 multi-vec int down convert four registers

The encodings in this section are decoded from [SME2 Multi-vector - SVE Constructive Unary](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	sz	op	1	1	0	0	1	1	1	1	1	0	0	0		Zn		N	U			Zd		

Decode fields			Instruction Details		Feature
op	N	U			
0	0	0	SQCVT (four registers)		FEAT_SME2
0	0	1	UQCVT (four registers)		FEAT_SME2
0	1	0	SQCVTN		FEAT_SME2
0	1	1	UQCVTN		FEAT_SME2
1		1	UNALLOCATED		-
1	0	0	SQCVTU (four registers)		FEAT_SME2
1	1	0	SQCVTUN		FEAT_SME2

SME2 multi-vec unpack four registers

The encodings in this section are decoded from [SME2 Multi-vector - SVE Constructive Unary](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size	1	1	0	1	0	1	1	1	1	0	0	0	0		Zn			0		Zd		0	U

Decode fields		Instruction Details		Feature
U				
0		SUNPK		FEAT_SME2
1		UUNPK		FEAT_SME2

SME2 multi-vec ZIP four registers

The encodings in this section are decoded from [SME2 Multi-vector - SVE Constructive Unary](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size	1	1	0	1	1	0	1	1	1	0	0	0	0		Zn		0	0		Zd		op	0

Decode fields		Instruction Details		Feature
op				
0		ZIP (four registers)		FEAT_SME2
1		UZP (four registers)		FEAT_SME2

SME2 multi-vec FRINT four registers

The encodings in this section are decoded from [SME2 Multi-vector - SVE Constructive Unary](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size	1	1	1		opc		1	1	1	0	0	0	0		Zn		0	0		Zd		0	0

Decode fields		Instruction Details		Feature
size	opc			
10	000	FRINTN		FEAT_SME2
10	001	FRINTP		FEAT_SME2
10	010	FRINTM		FEAT_SME2
10	011	UNALLOCATED		-
10	100	FRINTA		FEAT_SME2
10	101	UNALLOCATED		-
10	11x	UNALLOCATED		-
!= 10		UNALLOCATED		-

SME2 Multi-vector - FP Multiply

The encodings in this section are decoded from [SME encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1			1				op0		1	1	1	0	0	1				op1	0				op2	0

Decode fields			Instruction details	
op0	op1	op2		
01	0	0	SME2 multi-vec FP multiply (four registers)	
01	0	1	UNALLOCATED	
01	1		UNALLOCATED	
11			UNALLOCATED	
x0			SME2 multi-vec FP multiply (two registers)	

SME2 multi-vec FP multiply (four registers)

The encodings in this section are decoded from [SME2 Multi-vector - FP Multiply](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size		1		Zm		0	1	1	1	1	0	0	1		Zn		0	0		Zd		0	0

Decode fields		Instruction Details		Feature
size				
00		BFMUL (multiple vectors)		FEAT_SME2 && FEAT_SVE_BFSCALE
!= 00		FMUL (multiple vectors)		FEAT_SME2p2

SME2 multi-vec FP multiply (two registers)

The encodings in this section are decoded from [SME2 Multi-vector - FP Multiply](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size	1	Zm				0	1	1	1	0	0	1	Zn				0	Zd				0	

Decode fields		Instruction Details		Feature
size				
00		BFMUL (multiple vectors)		FEAT_SME2 && FEAT_SVE_BFSCALE
!= 00		FMUL (multiple vectors)		FEAT_SME2p2

SME2 Multiple and Single Vector - FP multiply

The encodings in this section are decoded from [SME encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1			1				op0		1	1	1	0	1	0				op1	0				op2	0

Decode fields			Instruction details
op0	op1	op2	
0			SME2 multiple and single vector FP multiply (two registers)
1	0	0	SME2 multiple and single vector FP multiply (four registers)
1	0	1	UNALLOCATED
1	1		UNALLOCATED

SME2 multiple and single vector FP multiply (two registers)

The encodings in this section are decoded from [SME2 Multiple and Single Vector - FP multiply](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size	1			Zm			0	1	1	1	0	1	0			Zn		0			Zd		0

Decode fields size	Instruction Details	Feature
00	BFMUL (multiple and single vector)	FEAT_SME2 && FEAT_SVE_BFSCALE
!= 00	FMUL (multiple and single vector)	FEAT_SME2p2

SME2 multiple and single vector FP multiply (four registers)

The encodings in this section are decoded from [SME2 Multiple and Single Vector - FP multiply](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size	1	Zm					1	1	1	1	0	1	0	Zn			0	0	Zd			0	0

Decode fields size	Instruction Details	Feature
00	BFMUL (multiple and single vector)	FEAT_SME2 && FEAT_SVE_BFSCALE
!= 00	FMUL (multiple and single vector)	FEAT_SME2p2

SME Memory

The encodings in this section are decoded from [SME encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	0	0			op0				op1		op2				op3									op4			

Decode fields					Instruction details	Feature
op0	op1	op2	op3	op4		
0xx0				0xx	SME load array vector (elements)	-
0xx1				0xx	SME store array vector (elements)	-
100x	00000	0	xx000	0xx	SME save and restore array	-
100x		1	00000	000	SME2 lookup table load/store	-
100x		1	00000	001	UNALLOCATED	-
100x		1	00000	01x	UNALLOCATED	-
100x		1	01000	0xx	UNALLOCATED	-
100x		1	1x000	0xx	UNALLOCATED	-
100x			xx01x	0xx	UNALLOCATED	■
100x			xx1xx	0xx	UNALLOCATED	■
100x			!= xx000xx001	0xx	UNALLOCATED	-
100x	!= 00000	0	xx000	0xx	UNALLOCATED	-
101x				0xx	UNALLOCATED	-
110x				0xx	UNALLOCATED	-
1110				0xx	LD1Q	FEAT_SME
1111				0xx	ST1Q	FEAT_SME
				1xx	UNALLOCATED	-

SME load array vector (elements)

The encodings in this section are decoded from [SME Memory](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	0	0	0	msz	0	Rm			V	Rs	Pg		Rn			0	opc										

Decode fields		Instruction Details		Feature
msz				
00		LD1B (scalar plus scalar, tile slice)		FEAT_SME
01		LD1H (scalar plus scalar, tile slice)		FEAT_SME
10		LD1W (scalar plus scalar, tile slice)		FEAT_SME
11		LD1D (scalar plus scalar, tile slice)		FEAT_SME

SME store array vector (elements)

The encodings in this section are decoded from [SME Memory](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	0	0	0	msz	1	Rm			V	Rs	Pg		Rn			0	opc										

Decode fields		Instruction Details		Feature
msz				
00		ST1B (scalar plus scalar, tile slice)		FEAT_SME
01		ST1H (scalar plus scalar, tile slice)		FEAT_SME
10		ST1W (scalar plus scalar, tile slice)		FEAT_SME
11		ST1D (scalar plus scalar, tile slice)		FEAT_SME

SME save and restore array

The encodings in this section are decoded from [SME Memory](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	0	0	1	0	0	op	0	0	0	0	0	0		Rv	0	0	0			Rn		0				imm4	

Decode fields	Instruction Details	Feature
op		
0	LDR (array vector)	FEAT_SME
1	STR (array vector)	FEAT_SME

SME2 lookup table load/store

The encodings in this section are decoded from [SME Memory](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	0	0	1	0	0	opc						1	0	0	0	0	0	Rn					0	0	0	opc2	

Decode fields	Instruction Details	Feature
opc	opc2	
x0xxxx		UNALLOCATED
x10xxx		UNALLOCATED
x110xx		UNALLOCATED
x1110x		UNALLOCATED
x11110		UNALLOCATED
x11111	!= 00	UNALLOCATED
!= x11111		UNALLOCATED
011111	00	LDR (table)
111111	00	STR (table)
		FEAT_SME2
		FEAT_SME2

SVE encodings

The encodings in this section are decoded from [A64 instruction set encoding](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
op0				0	0	1	0	op1								op2						op3									

Decode fields				Instruction details	Feature
op0	op1	op2	op3		
000	0xx0xxxx	000xxx		SVE Integer Binary Arithmetic - Predicated	-
000	0xx0xxxx	001xxx		SVE Integer Reduction	-
000	0xx0xxxx	100xxx		SVE Bitwise Shift - Predicated	-
000	0xx0xxxx	101xxx		SVE Integer Unary Arithmetic - Predicated	-
000	0xx0xxxx	x1xxxx		SVE Integer Multiply-Add - Predicated	-
000	0xx1xxxx	001xxx		SVE Bitwise Logical - Unpredicated	-
000	0xx1xxxx	0100xx		SVE Index Generation	-
000	0xx1xxxx	0101xx		SVE Stack Allocation	-
000	0xx1xxxx	011xxx		SVE2 Integer Arithmetic - Unpredicated	-
000	0xx1xxxx	100xxx		SVE Bitwise Shift - Unpredicated	-
000	0xx1xxxx	1011xx		SVE Integer Misc - Unpredicated	-
000	0xx1xxxx	11xxxx		SVE Element Count	-
000	10x1xxxx	000xxx		SVE Permute Vector - Extract	-
000	11x1xxxx	000xxx		SVE Permute Vector - Segments	-
000	1xx00xxx			SVE Bitwise Immediate	-
000	1xx01xxx			SVE Integer Wide Immediate - Predicated	-
000	1xx1xxxx	001001		SVE Permute Vector - One Source Quadwords	-
000	1xx1xxxx	001110		SVE Permute Vector - Unpredicated	-
000	1xx1xxxx	001111		UNALLOCATED	-
000	1xx1xxxx	010xxx		SVE Permute Predicate	-
000	1xx1xxxx	10xxxx		SVE Permute Vector - Predicated	-
001	0xx0xxxx			SVE Integer Compare - Vectors	-
001	1xx00xxx	11xxxx		SVE Propagate Break	-
001	1xx01xxx	01xxxx		SVE Partition Break	-
001	1xx01xxx	11xxxx		SVE Predicate Misc	-
001	1xx100xx	10xxxx		SVE Predicate Count	-
001	1xx101xx	1000xx		SVE Inc/Dec by Predicate Count	-
001	1xx101xx	1001xx		SVE Write FFR	-
001	1xx101xx	101xxx		UNALLOCATED	-
001	1xx11xxx	10xxxx		UNALLOCATED	-
001	1xx1xxxx	00xxxx		SVE Integer Compare - Scalars	-
001	1xx1xxxx	01xxxx	1	SVE Scalar Integer Compare - Predicate-as-counter	-
001	1xx1xxxx	11xxxx		SVE Integer Wide Immediate - Unpredicated	-
010	0x10xxxx	11001x		UNALLOCATED	-
010	0xx0xxxx	0xxxxxx		SVE Integer Multiply-Add - Unpredicated	-
010	0xx0xxxx	10xxxx		SVE2 Integer - Predicated	-
010	0xx1xxxx			SVE Multiply - Indexed	-
010	1xx0xxxx	0xxxxxx		SVE2 Widening Integer Arithmetic	-
010	1xx0xxxx	10xxxx		SVE Misc	-
010	1xx0xxxx	11xxxx		SVE2 Accumulate	-
010	1xx1xxxx	0xxxxxx		SVE2 Narrowing	-
010	1xx1xxxx	101xxx		SVE2 Histogram (Segment) and Lookup Table	-
010	1xx1xxxx	111xxx		SVE2 Crypto Extensions	-
011	0x01xxxx	0111xx		UNALLOCATED	-
011	0x01xxxx	10xx10		SVE2 FP8 widening multiply-add	-

011	0x01xxxx	10xx11		UNALLOCATED	-
011	0x01xxxx	11x1xx		UNALLOCATED	-
011	0x11xxxx	10xx1x		UNALLOCATED	-
011	0x11xxxx	x1x1xx		UNALLOCATED	-
011	0xx00001	100xxx		UNALLOCATED	-
011	0xx0010x	100xxx		UNALLOCATED	-
011	0xx00x0x	11xxxx		UNALLOCATED	-
011	0xx00x1x	1xxxxx		UNALLOCATED	-
011	0xx010xx	11xxxx		UNALLOCATED	-
011	0xx011xx	1xxxxx		SVE2 floating-point unary operations - zeroing predicated	-
011	0xx1xxxx	001011		UNALLOCATED	-
011	0xx1xxxx	0011xx		UNALLOCATED	-
011	0xx1xxxx	01x0xx		SVE floating-point widening multiply-add - indexed	-
011	0xx1xxxx	10x00x		SVE floating-point widening multiply-add	-
011	0xx1xxxx	10x10x		UNALLOCATED	-
011	0xx1xxxx	11101x		UNALLOCATED	-
011	1xx001xx	0010xx		UNALLOCATED	-
011	1xx001xx	0011xx		SVE floating-point unary operations - unpredicated	-
011	1xx010xx	001xxx		SVE floating-point compare - with zero	-
011	1xx011xx	001xxx		SVE floating-point accumulating reduction	-
011	1xx0xxxx	100xxx		SVE floating-point arithmetic - predicated	-
011	1xx0xxxx	101xxx		SVE floating-point unary operations - merging predicated	-
011	1xx1xxxx			SVE floating-point multiply-add	-
100				SVE Memory - 32-bit Gather and Unsized Contiguous	-
101				SVE Memory - Contiguous Load	-
110				SVE Memory - 64-bit Gather	-
111		001xxx		SVE Memory - Non-temporal and Quadword Scatter Store	-
111		011xxx		SVE Memory - Non-temporal and Multi-register Contiguous Store	-
111		0x0xxx		SVE Memory - Contiguous Store and Unsized Contiguous	-
111		101xxx		SVE Memory - Scatter	-
111		111xxx		SVE Memory - Contiguous Store with Immediate Offset	-
111		1x0xxx		SVE Memory - Scatter with Optional Sign Extend	-
000	0xx1xxxx	000xxx		SVE integer add/subtract vectors (unpredicated)	-
000	0xx1xxxx	1010xx		SVE address generation	-
000	1xx1xxxx	001000		DUP (indexed)	FEAT_SVE FEAT_SME
000	1xx1xxxx	00101x		SVE table lookup (three sources)	-
000	1xx1xxxx	001100		TBL — Single register table	FEAT_SVE FEAT_SME
000	1xx1xxxx	001101		TBXQ	FEAT_SVE2p1 FEAT_SME2p1
000	1xx1xxxx	011xxx		SVE permute vector elements	-
000	1xx1xxxx	11xxxx		SEL (vectors)	FEAT_SVE FEAT_SME
001	0xx1xxxx			SVE integer compare with unsigned immediate	-
001	1xx00xxx	01xxxx		SVE predicate logical operations	-
001	1xx0xxxx	x0xxxx		SVE integer compare with signed immediate	-
001	1xx1xxxx	01xxxx	0	SVE broadcast predicate element	-
010	0000xxxx	11001x		SVE two-way dot product	-

010	0100xxxx	11001x		SVE two-way dot product (indexed)	-
010	0xx0xxxx	11000x		SVE integer clamp	-
010	0xx0xxxx	1101x0		SVE2 multiply-add (checked pointer)	-
010	0xx0xxxx	1101x1		SVE2 absolute difference and accumulate long	-
010	0xx0xxxx	111xxx		SVE permute vector elements (quadwords)	-
010	1xx1xxxx	100xxx		SVE2 character match	-
010	1xx1xxxx	110xxx		HISTCNT	FEAT_SVE2
011	00x1xxxx	111000		SVE2 FP8 matrix multiply-accumulate	-
011	01x1xxxx	111000		SVE2 floating-point non-widening matrix-multiply accumulate	-
011	0x01xxxx	0101xx		SVE2 FP8 multiply-add long (indexed)	-
011	0xx00000	100xxx		FCADD	FEAT_SVE FEAT_SME
011	0xx0000x	101xxx		SVE floating-point convert (top, predicated)	-
011	0xx0010x	101xxx		SVE floating-point convert precision odd elements	-
011	0xx010xx	100xxx		SVE2 floating-point pairwise operations	-
011	0xx010xx	101xxx		SVE floating-point recursive reduction (quadwords)	-
011	0xx0xxxx	0xxxxx		FCMLA (vectors)	FEAT_SVE FEAT_SME
011	0xx1xxxx	0000xx		SVE floating-point multiply-add (indexed)	-
011	0xx1xxxx	0001xx		SVE floating-point complex multiply-add (indexed)	-
011	0xx1xxxx	001001		SVE FP clamp	-
011	0xx1xxxx	0010x0		SVE floating-point multiply (indexed)	-
011	0xx1xxxx	1100xx		SVE2 FP8 multiply-add long long (indexed)	-
011	0xx1xxxx	111001		SVE floating-point matrix multiply accumulate	-
011	1xx000xx	001xxx		SVE floating-point recursive reduction	-
011	1xx0xxxx	000xxx		SVE floating-point arithmetic (unpredicated)	-
011	1xx0xxxx	x1xxxx		SVE floating-point compare vectors	-

SVE Integer Binary Arithmetic - Predicated

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0			0		op0				0	0	0													

Decode fields

op0

Instruction details

00x	SVE integer add/subtract vectors (predicated)
01x	SVE integer min/max/difference (predicated)
100	SVE integer multiply vectors (predicated)
101	SVE integer divide vectors (predicated)
11x	SVE bitwise logical operations (predicated)

SVE integer add/subtract vectors (predicated)

The encodings in this section are decoded from [SVE Integer Binary Arithmetic - Predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0	size		0	0	0	opc			0	0	0	Pg				Zm					Zdn			

Decode fields		Instruction Details	Feature
size	opc		
	000	ADD (vectors, predicated)	FEAT_SVE FEAT_SME
	001	SUB (vectors, predicated)	FEAT_SVE FEAT_SME
	010	UNALLOCATED	-
	011	SUBR (vectors)	FEAT_SVE FEAT_SME
11	100	ADDPT (predicated)	FEAT_SVE && FEAT_CPA
11	101	SUBPT (predicated)	FEAT_SVE && FEAT_CPA
11	11x	UNALLOCATED	-
!= 11	1xx	UNALLOCATED	-

SVE integer min/max/difference (predicated)

The encodings in this section are decoded from [SVE Integer Binary Arithmetic - Predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0	size	0	0	1	opc	U	0	0	0	Pg	Zm				Zdn									

Decode fields		Instruction Details	Feature
opc	U		
00	0	SMAX (vectors)	FEAT_SVE FEAT_SME
00	1	UMAX (vectors)	FEAT_SVE FEAT_SME
01	0	SMIN (vectors)	FEAT_SVE FEAT_SME
01	1	UMIN (vectors)	FEAT_SVE FEAT_SME
10	0	SABD	FEAT_SVE FEAT_SME
10	1	UABD	FEAT_SVE FEAT_SME
11		UNALLOCATED	-

SVE integer multiply vectors (predicated)

The encodings in this section are decoded from [SVE Integer Binary Arithmetic - Predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0	size	0	1	0	0	H	U	0	0	0	Pg	Zm				Zdn								

Decode fields		Instruction Details	Feature
H	U		
0	0	MUL (vectors, predicated)	FEAT_SVE FEAT_SME
0	1	UNALLOCATED	-
1	0	SMULH (predicated)	FEAT_SVE FEAT_SME
1	1	UMULH (predicated)	FEAT_SVE FEAT_SME

SVE integer divide vectors (predicated)

The encodings in this section are decoded from [SVE Integer Binary Arithmetic - Predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0	size	0	1	0	1	R	U	0	0	0	Pg	Zm				Zdn								

Decode fields		Instruction Details	Feature
R	U		
0	0	SDIV	FEAT_SVE FEAT_SME
0	1	UDIV	FEAT_SVE FEAT_SME
1	0	SDIVR	FEAT_SVE FEAT_SME
1	1	UDIVR	FEAT_SVE FEAT_SME

SVE bitwise logical operations (predicated)

The encodings in this section are decoded from [SVE Integer Binary Arithmetic - Predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0	size	0	1	1	opc	0	0	0	Pg	Zm					Zdn									

Decode fields	Instruction Details		Feature
opc			
000	ORR (vectors, predicated)		FEAT_SVE FEAT_SME
001	EOR (vectors, predicated)		FEAT_SVE FEAT_SME
010	AND (vectors, predicated)		FEAT_SVE FEAT_SME
011	BIC (vectors, predicated)		FEAT_SVE FEAT_SME
1xx	UNALLOCATED		-

SVE Integer Reduction

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0			0	op0					0	0	1													

Decode fields	Instruction details	
op0		
000	SVE integer add reduction (predicated)	
001	SVE integer add reduction (quadwords)	
010	SVE integer min/max reduction (predicated)	
011	SVE integer min/max reduction (quadwords)	
10x	SVE constructive prefix (predicated)	
110	SVE bitwise logical reduction (predicated)	
111	SVE bitwise logical reduction (quadwords)	

SVE integer add reduction (predicated)

The encodings in this section are decoded from [SVE Integer Reduction](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0			0	0	0	0	op	U	0	0	1		Pg					Zn					Vd	

Decode fields	Instruction Details		Feature
op	U		
0	0	SADDV	FEAT_SVE FEAT_SME
0	1	UADDV	FEAT_SVE FEAT_SME
1		UNALLOCATED	-

SVE integer add reduction (quadwords)

The encodings in this section are decoded from [SVE Integer Reduction](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0			0	0	0	1	op	U	0	0	1		Pg					Zn					Vd	

Decode fields	Instruction Details		Feature
op	U		
0	0	UNALLOCATED	-
0	1	ADDQV	FEAT_SVE2p1 FEAT_SME2p1
1		UNALLOCATED	-

SVE integer min/max reduction (predicated)

The encodings in this section are decoded from [SVE Integer Reduction](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0			0	0	1	0	op	U	0	0	1		Pg				Zn					Vd		

Decode fields		Instruction Details												Feature															
op	U																												
0	0	SMAXV												FEAT_SVE FEAT_SME															
0	1	UMAXV												FEAT_SVE FEAT_SME															
1	0	SMINV												FEAT_SVE FEAT_SME															
1	1	UMINV												FEAT_SVE FEAT_SME															

SVE integer min/max reduction (quadwords)

The encodings in this section are decoded from [SVE Integer Reduction](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0			0	0	1	1	op	U	0	0	1		Pg				Zn					Vd		

Decode fields		Instruction Details												Feature															
op	U																												
0	0	SMAXQV												FEAT_SVE2p1 FEAT_SME2p1															
0	1	UMAXQV												FEAT_SVE2p1 FEAT_SME2p1															
1	0	SMINQV												FEAT_SVE2p1 FEAT_SME2p1															
1	1	UMINQV												FEAT_SVE2p1 FEAT_SME2p1															

SVE constructive prefix (predicated)

The encodings in this section are decoded from [SVE Integer Reduction](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0			0	1	0		opc	M	0	0	1		Pg				Zn					Zd		

Decode fields		Instruction Details												Feature															
opc																													
00		MOVPRFX (predicated)												FEAT_SVE FEAT_SME															
!= 00		UNALLOCATED												-															

SVE bitwise logical reduction (predicated)

The encodings in this section are decoded from [SVE Integer Reduction](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0			0	1	1	0	opc		0	0	1		Pg				Zn					Vd		

Decode fields		Instruction Details												Feature															
opc																													
00		ORV												FEAT_SVE FEAT_SME															
01		EORV												FEAT_SVE FEAT_SME															
10		ANDV												FEAT_SVE FEAT_SME															
11		UNALLOCATED												-															

SVE bitwise logical reduction (quadwords)

The encodings in this section are decoded from [SVE Integer Reduction](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0			0	1	1	1	opc		0	0	1		Pg				Zn					Vd		

Decode fields opc	Instruction Details	Feature
00	ORQV	FEAT_SVE2p1 FEAT_SME2p1
01	EORQV	FEAT_SVE2p1 FEAT_SME2p1
10	ANDQV	FEAT_SVE2p1 FEAT_SME2p1
11	UNALLOCATED	-

SVE Bitwise Shift - Predicated

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0					
0	0	0	0	0	1	0	0			0	op0					1	0	0																		

Decode fields op0	Instruction details
0x	SVE bitwise shift by immediate (predicated)
10	SVE bitwise shift by vector (predicated)
11	SVE bitwise shift by wide elements (predicated)

SVE bitwise shift by immediate (predicated)

The encodings in this section are decoded from [SVE Bitwise Shift - Predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0	tszh		0	0	opc	L	U		1	0	0		Pg		tszl		imm3					Zdn		

Decode fields opc L U	Instruction Details	Feature
00 0 0	ASR (immediate, predicated)	FEAT_SVE FEAT_SME
00 0 1	LSR (immediate, predicated)	FEAT_SVE FEAT_SME
00 1 0	UNALLOCATED	-
00 1 1	LSL (immediate, predicated)	FEAT_SVE FEAT_SME
01 0 0	ASRD	FEAT_SVE FEAT_SME
01 0 1	UNALLOCATED	-
01 1 0	SQSHL (immediate)	FEAT_SVE2 FEAT_SME
01 1 1	UQSHL (immediate)	FEAT_SVE2 FEAT_SME
10	UNALLOCATED	-
11 0 0	SRSR	FEAT_SVE2 FEAT_SME
11 0 1	URSHR	FEAT_SVE2 FEAT_SME
11 1 0	UNALLOCATED	-
11 1 1	SQSHLU	FEAT_SVE2 FEAT_SME

SVE bitwise shift by vector (predicated)

The encodings in this section are decoded from [SVE Bitwise Shift - Predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0			0	1	0	R	L	U	1	0	0		Pg				Zm					Zdn		

Decode fields			Instruction Details	Feature
R	L	U		
	1	0	UNALLOCATED	-
0	0	0	ASR (vectors)	FEAT_SVE FEAT_SME
0	0	1	LSR (vectors)	FEAT_SVE FEAT_SME
0	1	1	LSL (vectors)	FEAT_SVE FEAT_SME
1	0	0	ASRR	FEAT_SVE FEAT_SME
1	0	1	LSRR	FEAT_SVE FEAT_SME
1	1	1	LSLR	FEAT_SVE FEAT_SME

SVE bitwise shift by wide elements (predicated)

The encodings in this section are decoded from [SVE Bitwise Shift - Predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0			0	1	1	R	L	U	1	0	0		Pg				Zm					Zdn		

Decode fields			Instruction Details	Feature
R	L	U		
0	0	0	ASR (wide elements, predicated)	FEAT_SVE FEAT_SME
0	0	1	LSR (wide elements, predicated)	FEAT_SVE FEAT_SME
0	1	0	UNALLOCATED	-
0	1	1	LSL (wide elements, predicated)	FEAT_SVE FEAT_SME
1			UNALLOCATED	-

SVE Integer Unary Arithmetic - Predicated

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0			0	op0					1	0	1													

Decode fields		Instruction details
op0		
x0		SVE integer unary operations (predicated)
x1		SVE bitwise unary operations (predicated)

SVE integer unary operations (predicated)

The encodings in this section are decoded from [SVE Integer Unary Arithmetic - Predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0	size	0	M	0		opc			1	0	1		Pg				Zn					Zd		

Decode fields		Instruction Details	Feature
M	opc		
0	000	SXTB, SXTH, SXTW — Byte, zeroing	FEAT_SVE2p2 FEAT_SME2p2
0	001	UXTB, UXTH, UXTW — Byte, zeroing	FEAT_SVE2p2 FEAT_SME2p2
0	010	SXTB, SXTH, SXTW — Halfword, zeroing	FEAT_SVE2p2 FEAT_SME2p2
0	011	UXTB, UXTH, UXTW — Halfword, zeroing	FEAT_SVE2p2 FEAT_SME2p2
0	100	SXTB, SXTH, SXTW — Word, zeroing	FEAT_SVE2p2 FEAT_SME2p2
0	101	UXTB, UXTH, UXTW — Word, zeroing	FEAT_SVE2p2 FEAT_SME2p2
0	110	ABS — Zeroing	FEAT_SVE2p2 FEAT_SME2p2
0	111	NEG — Zeroing	FEAT_SVE2p2 FEAT_SME2p2
1	000	SXTB, SXTH, SXTW — Byte, merging	FEAT_SVE FEAT_SME
1	001	UXTB, UXTH, UXTW — Byte, merging	FEAT_SVE FEAT_SME
1	010	SXTB, SXTH, SXTW — Halfword, merging	FEAT_SVE FEAT_SME
1	011	UXTB, UXTH, UXTW — Halfword, merging	FEAT_SVE FEAT_SME
1	100	SXTB, SXTH, SXTW — Word, merging	FEAT_SVE FEAT_SME
1	101	UXTB, UXTH, UXTW — Word, merging	FEAT_SVE FEAT_SME
1	110	ABS — Merging	FEAT_SVE FEAT_SME
1	111	NEG — Merging	FEAT_SVE FEAT_SME

SVE bitwise unary operations (predicated)

The encodings in this section are decoded from [SVE Integer Unary Arithmetic - Predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0	size	0	M	1	opc		1		0	1	Pg		Zn				Zd							

Decode fields		Instruction Details	Feature
M	opc		
	111	UNALLOCATED	-
0	000	CLS — Zeroing	FEAT_SVE2p2 FEAT_SME2p2
0	001	CLZ — Zeroing	FEAT_SVE2p2 FEAT_SME2p2
0	010	CNT — Zeroing	FEAT_SVE2p2 FEAT_SME2p2
0	011	CNOT — Zeroing	FEAT_SVE2p2 FEAT_SME2p2
0	100	FABS — Zeroing	FEAT_SVE2p2 FEAT_SME2p2
0	101	FNEG — Zeroing	FEAT_SVE2p2 FEAT_SME2p2
0	110	NOT (vector) — Zeroing	FEAT_SVE2p2 FEAT_SME2p2
1	000	CLS — Merging	FEAT_SVE FEAT_SME
1	001	CLZ — Merging	FEAT_SVE FEAT_SME
1	010	CNT — Merging	FEAT_SVE FEAT_SME
1	011	CNOT — Merging	FEAT_SVE FEAT_SME
1	100	FABS — Merging	FEAT_SVE FEAT_SME
1	101	FNEG — Merging	FEAT_SVE FEAT_SME
1	110	NOT (vector) — Merging	FEAT_SVE FEAT_SME

SVE Integer Multiply-Add - Predicated

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0			0						op0	1														

Decode fields

op0

Instruction details

0	SVE integer multiply-accumulate writing addend (predicated)
1	SVE integer multiply-add writing multiplicand (predicated)

SVE integer multiply-accumulate writing addend (predicated)

The encodings in this section are decoded from [SVE Integer Multiply-Add - Predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0	size	0			Zm				0	1	op		Pg				Zn					Zda		

Decode fields

op

Instruction Details

Feature

0	MLA (vectors)	FEAT_SVE FEAT_SME
1	MLS (vectors)	FEAT_SVE FEAT_SME

SVE integer multiply-add writing multiplicand (predicated)

The encodings in this section are decoded from [SVE Integer Multiply-Add - Predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0	size	0			Zm				1	1	op		Pg				Za					Zdn		

Decode fields

op

Instruction Details

Feature

0	MAD	FEAT_SVE FEAT_SME
1	MSB	FEAT_SVE FEAT_SME

SVE Bitwise Logical - Unpredicated

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0			1						0	0	1		op0											

Decode fields

op0

Instruction details

Feature

0xx	UNALLOCATED	-
100	SVE bitwise logical operations (unpredicated)	-
101	XAR	FEAT_SVE2 FEAT_SME
11x	SVE2 bitwise ternary operations	-

SVE bitwise logical operations (unpredicated)

The encodings in this section are decoded from [SVE Bitwise Logical - Unpredicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0	opc	1				Zm			0	0	1	1	0	0				Zn				Zd		

Decode fields

opc

Instruction Details

Feature

00	AND (vectors, unpredicated)	FEAT_SVE FEAT_SME
01	ORR (vectors, unpredicated)	FEAT_SVE FEAT_SME
10	EOR (vectors, unpredicated)	FEAT_SVE FEAT_SME
11	BIC (vectors, unpredicated)	FEAT_SVE FEAT_SME

SVE2 bitwise ternary operations

The encodings in this section are decoded from [SVE Bitwise Logical - Unpredicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0	opc	1				Zm			0	0	1	1	1	o2			Zk					Zdn		

Decode fields		Instruction Details										Feature																	
opc	o2																												
00	0	EOR3										FEAT_SVE2 FEAT_SME																	
00	1	BSL										FEAT_SVE2 FEAT_SME																	
01	0	BCAX										FEAT_SVE2 FEAT_SME																	
01	1	BSL1N										FEAT_SVE2 FEAT_SME																	
1x	0	UNALLOCATED										-																	
10	1	BSL2N										FEAT_SVE2 FEAT_SME																	
11	1	NBSL										FEAT_SVE2 FEAT_SME																	

SVE Index Generation

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0			1						0	1	0	0	op0											

Decode fields		Instruction details										Feature																	
op0																													
00		INDEX (immediates)										FEAT_SVE FEAT_SME																	
01		INDEX (scalar, immediate)										FEAT_SVE FEAT_SME																	
10		INDEX (immediate, scalar)										FEAT_SVE FEAT_SME																	
11		INDEX (scalars)										FEAT_SVE FEAT_SME																	

SVE Stack Allocation

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0	op0		1						0	1	0	1	op1											

Decode fields		Instruction details																											
op0	op1																												
0	0	SVE stack frame adjustment																											
0	1	Streaming SVE stack frame adjustment																											
1	0	SVE stack frame size																											
1	1	Streaming SVE stack frame size																											

SVE stack frame adjustment

The encodings in this section are decoded from [SVE Stack Allocation](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0	0	op	1			Rn			0	1	0	1	0				imm6					Rd		

Decode fields		Instruction Details										Feature																	
op																													
0		ADDVL										FEAT_SVE FEAT_SME																	
1		ADDPL										FEAT_SVE FEAT_SME																	

Streaming SVE stack frame adjustment

The encodings in this section are decoded from [SVE Stack Allocation](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0	0	op	1			Rn			0	1	0	1	1				imm6						Rd	

Decode fields	Instruction Details	Feature
op		
0	ADDSVL	FEAT_SME
1	ADDSPL	FEAT_SME

SVE stack frame size

The encodings in this section are decoded from [SVE Stack Allocation](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0	1	op	1			opc2			0	1	0	1	0				imm6						Rd	

Decode fields	Instruction Details	Feature
op	opc2	
0	11111	RDVL
0	!= 11111	UNALLOCATED
1		UNALLOCATED

Streaming SVE stack frame size

The encodings in this section are decoded from [SVE Stack Allocation](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0	1	op	1			opc2			0	1	0	1	1				imm6						Rd	

Decode fields	Instruction Details	Feature
op	opc2	
0	11111	RDSVL
0	!= 11111	UNALLOCATED
1		UNALLOCATED

SVE2 Integer Arithmetic - Unpredicated

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0			1						0	1	1		op0											

Decode fields	Instruction details	Feature
op0		
0xx	SVE2 integer multiply vectors (unpredicated)	-
10x	SVE2 signed saturating doubling multiply high (unpredicated)	-
110	ADDQP	FEAT_SVE2p3 FEAT_SME2p3
111	ADDSUBP	FEAT_SVE2p3 FEAT_SME2p3

SVE2 integer multiply vectors (unpredicated)

The encodings in this section are decoded from [SVE2 Integer Arithmetic - Unpredicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0	size	1				Zm			0	1	1	0	opc				Zn						Zd	

Decode fields	Instruction Details		Feature
size	opc		
	00	MUL (vectors, unpredicated)	FEAT_SVE2 FEAT_SME
	10	SMULH (unpredicated)	FEAT_SVE2 FEAT_SME
	11	UMULH (unpredicated)	FEAT_SVE2 FEAT_SME
00	01	PMUL	FEAT_SVE2 FEAT_SME
!= 00	01	UNALLOCATED	-

SVE2 signed saturating doubling multiply high (unpredicated)

The encodings in this section are decoded from [SVE2 Integer Arithmetic - Unpredicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0	size	1				Zm			0	1	1	1	0	R			Zn					Zd		

Decode fields	Instruction Details		Feature
R			
0	SQDMULH (vectors)		FEAT_SVE2 FEAT_SME
1	SQRDMULH (vectors)		FEAT_SVE2 FEAT_SME

SVE Bitwise Shift - Unpredicated

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0			1						1	0	0	op0												

Decode fields	Instruction details	
op0		
0	SVE bitwise shift by wide elements (unpredicated)	
1	SVE bitwise shift by immediate (unpredicated)	

SVE bitwise shift by wide elements (unpredicated)

The encodings in this section are decoded from [SVE Bitwise Shift - Unpredicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0			1			Zm			1	0	0	0	opc				Zn					Zd		

Decode fields	Instruction Details		Feature
opc			
00	ASR (wide elements, unpredicated)		FEAT_SVE FEAT_SME
01	LSR (wide elements, unpredicated)		FEAT_SVE FEAT_SME
10	UNALLOCATED		-
11	LSL (wide elements, unpredicated)		FEAT_SVE FEAT_SME

SVE bitwise shift by immediate (unpredicated)

The encodings in this section are decoded from [SVE Bitwise Shift - Unpredicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0	tszh	1	tszl		imm3				1	0	0	1	opc				Zn					Zd		

Decode fields	Instruction Details	Feature
opc		
00	ASR (immediate, unpredicated)	FEAT_SVE FEAT_SME
01	LSR (immediate, unpredicated)	FEAT_SVE FEAT_SME
10	UNALLOCATED	-
11	LSL (immediate, unpredicated)	FEAT_SVE FEAT_SME

SVE Integer Misc - Unpredicated

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0			1						1	0	1	1	op0											

Decode fields	Instruction details
op0	
0x	SVE floating-point trig select coefficient
10	SVE floating-point exponential accelerator
11	SVE constructive prefix (unpredicated)

SVE floating-point trig select coefficient

The encodings in this section are decoded from [SVE Integer Misc - Unpredicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0			1	Zm					1	0	1	1	0	op	Zn					Zd				

Decode fields	Instruction Details	Feature
op		
0	FTSSEL	FEAT_SVE
1	UNALLOCATED	-

SVE floating-point exponential accelerator

The encodings in this section are decoded from [SVE Integer Misc - Unpredicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0			1			opc				1	0	1	1	1	0		Zn				Zd			

Decode fields	Instruction Details	Feature
opc		
00000	FEXPA	FEAT_SVE FEAT_SSVE_FEXPA
!= 00000	UNALLOCATED	-

SVE constructive prefix (unpredicated)

The encodings in this section are decoded from [SVE Integer Misc - Unpredicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0	opc		1	opc2					1		0	1	1	1		1	Zn				Zd			

Decode fields	Instruction Details	Feature
opc	opc2	
00	00000	MOVPRFX (unpredicated)
00	!= 00000	UNALLOCATED
!= 00		UNALLOCATED

SVE Element Count

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0			1	op0					1	1		op1												

Decode fields

op0 op1

Instruction details

0	00x	SVE saturating inc/dec vector by element count
0	100	SVE element count
0	101	UNALLOCATED
1	000	SVE inc/dec vector by element count
1	100	SVE inc/dec register by element count
1	x01	UNALLOCATED
	01x	UNALLOCATED
	11x	SVE saturating inc/dec register by element count

SVE saturating inc/dec vector by element count

The encodings in this section are decoded from [SVE Element Count](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0	size	1	0	imm4			1	1	0	0	D	U	pattern					Zdn						

Decode fields

size D U

Instruction Details

Feature

00			UNALLOCATED	-
01	0	0	SQINCH (vector)	FEAT_SVE FEAT_SME
01	0	1	UQINCH (vector)	FEAT_SVE FEAT_SME
01	1	0	SQDECH (vector)	FEAT_SVE FEAT_SME
01	1	1	UQDECH (vector)	FEAT_SVE FEAT_SME
10	0	0	SQINCW (vector)	FEAT_SVE FEAT_SME
10	0	1	UQINCW (vector)	FEAT_SVE FEAT_SME
10	1	0	SQDECW (vector)	FEAT_SVE FEAT_SME
10	1	1	UQDECW (vector)	FEAT_SVE FEAT_SME
11	0	0	SQINCD (vector)	FEAT_SVE FEAT_SME
11	0	1	UQINCD (vector)	FEAT_SVE FEAT_SME
11	1	0	SQDECD (vector)	FEAT_SVE FEAT_SME
11	1	1	UQDECD (vector)	FEAT_SVE FEAT_SME

SVE element count

The encodings in this section are decoded from [SVE Element Count](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0	size	1	0	imm4			1	1	1	0	0	op	pattern					Rd						

Decode fields

size op

Instruction Details

Feature

	1	UNALLOCATED	-
00	0	CNTB, CNTD, CNTH, CNTW — Byte	FEAT_SVE FEAT_SME
01	0	CNTB, CNTD, CNTH, CNTW — Halfword	FEAT_SVE FEAT_SME
10	0	CNTB, CNTD, CNTH, CNTW — Word	FEAT_SVE FEAT_SME
11	0	CNTB, CNTD, CNTH, CNTW — Doubleword	FEAT_SVE FEAT_SME

SVE inc/dec vector by element count

The encodings in this section are decoded from [SVE Element Count](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0	size	1	1		imm4	1	1	0	0	0	D		pattern											

Decode fields size	D	Instruction Details	Feature
00		UNALLOCATED	-
01	0	INCD, INCH, INCW (vector) — Halfword	FEAT_SVE FEAT_SME
01	1	DECD, DECH, DECW (vector) — Halfword	FEAT_SVE FEAT_SME
10	0	INCD, INCH, INCW (vector) — Word	FEAT_SVE FEAT_SME
10	1	DECD, DECH, DECW (vector) — Word	FEAT_SVE FEAT_SME
11	0	INCD, INCH, INCW (vector) — Doubleword	FEAT_SVE FEAT_SME
11	1	DECD, DECH, DECW (vector) — Doubleword	FEAT_SVE FEAT_SME

SVE inc/dec register by element count

The encodings in this section are decoded from [SVE Element Count](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0	size	1	1		imm4	1	1	1	0	0	D		pattern											

Decode fields size	D	Instruction Details	Feature
00	0	INCB, INCD, INCH, INCW (scalar) — Byte	FEAT_SVE FEAT_SME
00	1	DECB, DECD, DECH, DECW (scalar) — Byte	FEAT_SVE FEAT_SME
01	0	INCB, INCD, INCH, INCW (scalar) — Halfword	FEAT_SVE FEAT_SME
01	1	DECB, DECD, DECH, DECW (scalar) — Halfword	FEAT_SVE FEAT_SME
10	0	INCB, INCD, INCH, INCW (scalar) — Word	FEAT_SVE FEAT_SME
10	1	DECB, DECD, DECH, DECW (scalar) — Word	FEAT_SVE FEAT_SME
11	0	INCB, INCD, INCH, INCW (scalar) — Doubleword	FEAT_SVE FEAT_SME
11	1	DECB, DECD, DECH, DECW (scalar) — Doubleword	FEAT_SVE FEAT_SME

SVE saturating inc/dec register by element count

The encodings in this section are decoded from [SVE Element Count](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0	size	1	sf		imm4	1	1	1	1	D	U		pattern											

Decode fields				Instruction Details	Feature
size	sf	D	U		
00	0	0	0	SQINCB — 32-bit	FEAT_SVE FEAT_SME
00	0	0	1	UQINCB — 32-bit	FEAT_SVE FEAT_SME
00	0	1	0	SQDECB — 32-bit	FEAT_SVE FEAT_SME
00	0	1	1	UQDECB — 32-bit	FEAT_SVE FEAT_SME
00	1	0	0	SQINCB — 64-bit	FEAT_SVE FEAT_SME
00	1	0	1	UQINCB — 64-bit	FEAT_SVE FEAT_SME
00	1	1	0	SQDECB — 64-bit	FEAT_SVE FEAT_SME
00	1	1	1	UQDECB — 64-bit	FEAT_SVE FEAT_SME
01	0	0	0	SQINCH (scalar) — 32-bit	FEAT_SVE FEAT_SME
01	0	0	1	UQINCH (scalar) — 32-bit	FEAT_SVE FEAT_SME
01	0	1	0	SQDECH (scalar) — 32-bit	FEAT_SVE FEAT_SME
01	0	1	1	UQDECH (scalar) — 32-bit	FEAT_SVE FEAT_SME
01	1	0	0	SQINCH (scalar) — 64-bit	FEAT_SVE FEAT_SME
01	1	0	1	UQINCH (scalar) — 64-bit	FEAT_SVE FEAT_SME
01	1	1	0	SQDECH (scalar) — 64-bit	FEAT_SVE FEAT_SME
01	1	1	1	UQDECH (scalar) — 64-bit	FEAT_SVE FEAT_SME
10	0	0	0	SQINCW (scalar) — 32-bit	FEAT_SVE FEAT_SME
10	0	0	1	UQINCW (scalar) — 32-bit	FEAT_SVE FEAT_SME
10	0	1	0	SQDECW (scalar) — 32-bit	FEAT_SVE FEAT_SME
10	0	1	1	UQDECW (scalar) — 32-bit	FEAT_SVE FEAT_SME
10	1	0	0	SQINCW (scalar) — 64-bit	FEAT_SVE FEAT_SME
10	1	0	1	UQINCW (scalar) — 64-bit	FEAT_SVE FEAT_SME
10	1	1	0	SQDECW (scalar) — 64-bit	FEAT_SVE FEAT_SME
10	1	1	1	UQDECW (scalar) — 64-bit	FEAT_SVE FEAT_SME
11	0	0	0	SQINCD (scalar) — 32-bit	FEAT_SVE FEAT_SME
11	0	0	1	UQINCD (scalar) — 32-bit	FEAT_SVE FEAT_SME
11	0	1	0	SQDECD (scalar) — 32-bit	FEAT_SVE FEAT_SME
11	0	1	1	UQDECD (scalar) — 32-bit	FEAT_SVE FEAT_SME
11	1	0	0	SQINCD (scalar) — 64-bit	FEAT_SVE FEAT_SME
11	1	0	1	UQINCD (scalar) — 64-bit	FEAT_SVE FEAT_SME
11	1	1	0	SQDECD (scalar) — 64-bit	FEAT_SVE FEAT_SME
11	1	1	1	UQDECD (scalar) — 64-bit	FEAT_SVE FEAT_SME

SVE Permute Vector - Extract

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1	0	op0	1						0	0	0													

Decode fields		Instruction details	Feature
op0			
0		EXT — Destructive	FEAT_SVE FEAT_SME
1		EXT — Constructive	FEAT_SVE2 FEAT_SME

SVE Permute Vector - Segments

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1	1	op0	1						0	0	0													

Decode fields

op0

Instruction details

0	SVE permute vector segments
1	UNALLOCATED

SVE permute vector segments

The encodings in this section are decoded from [SVE Permute Vector - Segments](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1	1	0	1			Zm			0	0	0	opc	H				Zn					Zd		

Decode fields

opc

H

Instruction Details

Feature

00	0	ZIP1, ZIP2 (vectors) — Low halves (quadwords)	FEAT_F64MM
00	1	ZIP1, ZIP2 (vectors) — High halves (quadwords)	FEAT_F64MM
01	0	UZP1, UZP2 (vectors) — Even (quadwords)	FEAT_F64MM
01	1	UZP1, UZP2 (vectors) — Odd (quadwords)	FEAT_F64MM
10		UNALLOCATED	-
11	0	TRN1, TRN2 (vectors) — Even (quadwords)	FEAT_F64MM
11	1	TRN1, TRN2 (vectors) — Odd (quadwords)	FEAT_F64MM

SVE Bitwise Immediate

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1		op0		0	0		op1																	

Decode fields

op0

op1

Instruction details

Feature

11	00	DUPM	FEAT_SVE FEAT_SME
	!= 00	UNALLOCATED	-
!= 11	00	SVE bitwise logical with immediate (unpredicated)	-

SVE bitwise logical with immediate (unpredicated)

The encodings in this section are decoded from [SVE Bitwise Immediate](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0	0	0	0	0	1	0	1	!= 11	0	0	0	0	imm13												Zdn							
opc																																

The following constraints also apply to this encoding: opc != '11'

Decode fields

opc

Instruction Details

Feature

00	ORR (immediate)	FEAT_SVE FEAT_SME
01	EOR (immediate)	FEAT_SVE FEAT_SME
10	AND (immediate)	FEAT_SVE FEAT_SME

SVE Integer Wide Immediate - Predicated

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1			0	1									op0											

Decode fields	Instruction details	Feature
op0		
0xx	SVE copy integer immediate (predicated)	-
10x	UNALLOCATED	-
110	FCPY	FEAT_SVE FEAT_SME
111	UNALLOCATED	-

SVE copy integer immediate (predicated)

The encodings in this section are decoded from [SVE Integer Wide Immediate - Predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1	size	0	1		Pg	0	M	sh																

Decode fields	Instruction Details	Feature
M		
0	CPY (immediate, zeroing)	FEAT_SVE FEAT_SME
1	CPY (immediate, merging)	FEAT_SVE FEAT_SME

SVE Permute Vector - One Source Quadwords

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1	op0	1	op1						0	0	1	0	0	1										

Decode fields	Instruction details	Feature
op0 op1		
00	DUPQ	FEAT_SVE2p1 FEAT_SME2p1
01 0	EXTQ	FEAT_SVE2p1 FEAT_SME2p1
01 1	UNALLOCATED	-
1x	UNALLOCATED	-

SVE Permute Vector - Unpredicated

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1		1	op0		op1	0	0	1	1	1	0	op2									op3			

Decode fields	Instruction details	Feature
op0 op1 op2 op3		
00 000	DUP (scalar)	FEAT_SVE FEAT_SME
00 100	INSR (scalar)	FEAT_SVE FEAT_SME
00 x1x	UNALLOCATED	-
00 != x00x01	UNALLOCATED	-
01 xx0	SVE move predicate from vector	-
01 xx0	UNALLOCATED	-
01 xx1 0	SVE move predicate into vector	-
01 xx1 1	UNALLOCATED	-
10 0xx	SVE unpack vector elements	-
10 100	INSR (SIMD&FP scalar)	FEAT_SVE FEAT_SME
10 101	UNALLOCATED	-
10 11x	UNALLOCATED	-
11 000	REV (vector)	FEAT_SVE FEAT_SME
11 != 000	UNALLOCATED	-

SVE move predicate from vector

The encodings in this section are decoded from [SVE Permute Vector - Unpredicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1	opc	1	0	1	opc2	0	0	0	1	1	1	0	Zn				0	Pd						

Decode fields		Instruction Details	Feature
opc	opc2		
00	00	UNALLOCATED	-
00	01	PMOV (to predicate) — Byte	FEAT_SVE2p1 FEAT_SME2p1
00	1x	PMOV (to predicate) — Halfword	FEAT_SVE2p1 FEAT_SME2p1
01		PMOV (to predicate) — Word	FEAT_SVE2p1 FEAT_SME2p1
1x		PMOV (to predicate) — Doubleword	FEAT_SVE2p1 FEAT_SME2p1

SVE move predicate into vector

The encodings in this section are decoded from [SVE Permute Vector - Unpredicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1	opc	1	0	1	opc2	1	0	0	1	1	1	0	0	Pn				Zd						

Decode fields		Instruction Details	Feature
opc	opc2		
00	00	UNALLOCATED	-
00	01	PMOV (to vector) — Byte	FEAT_SVE2p1 FEAT_SME2p1
00	1x	PMOV (to vector) — Halfword	FEAT_SVE2p1 FEAT_SME2p1
01		PMOV (to vector) — Word	FEAT_SVE2p1 FEAT_SME2p1
1x		PMOV (to vector) — Doubleword	FEAT_SVE2p1 FEAT_SME2p1

SVE unpack vector elements

The encodings in this section are decoded from [SVE Permute Vector - Unpredicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1	size	1	1	0	0	U	H	0	0	1	1	1	0	Zn				Zd						

Decode fields		Instruction Details	Feature
U	H		
0	0	SUNPKHI, SUNPKLO — Low half	FEAT_SVE FEAT_SME
0	1	SUNPKHI, SUNPKLO — High half	FEAT_SVE FEAT_SME
1	0	UUNPKHI, UUNPKLO — Low half	FEAT_SVE FEAT_SME
1	1	UUNPKHI, UUNPKLO — High half	FEAT_SVE FEAT_SME

SVE Permute Predicate

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1	op0	1	op1				0	1	0	op2				op3										

Decode fields				Instruction details	Feature
op0	op1	op2	op3		
00	1000x	0000	0	SVE unpack predicate elements	-
	0xxxx	xxx0	0	SVE permute predicate elements	-
	10100	0000	0	REV (predicate)	FEAT_SVE FEAT_SME
	10101	0000	0	UNALLOCATED	-
	10x0x	0010	0	UNALLOCATED	-
	10x0x	01x0	0	UNALLOCATED	-
	10x0x	1xx0	0	UNALLOCATED	-
	10x1x	xxx0	0	UNALLOCATED	-
	11xxx	xxx0	0	UNALLOCATED	-
		xxx0	1	UNALLOCATED	-
		xxx1		UNALLOCATED	-
!= 00	1000x	0000	0	UNALLOCATED	-

SVE unpack predicate elements

The encodings in this section are decoded from [SVE Permute Predicate](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1	0	0	1	1	0	0	0	H	0	1	0	0	0	0	0	Pn			0	Pd				

Decode fields		Instruction Details	Feature
H			
0		PUNPKHI, PUNPKLO — Low half	FEAT_SVE FEAT_SME
1		PUNPKHI, PUNPKLO — High half	FEAT_SVE FEAT_SME

SVE permute predicate elements

The encodings in this section are decoded from [SVE Permute Predicate](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1	size		1	0	Pm			0	1	0	opc		H	0	Pn			0	Pd					

Decode fields		Instruction Details	Feature
opc	H		
00	0	ZIP1, ZIP2 (predicates) — Low halves	FEAT_SVE FEAT_SME
00	1	ZIP1, ZIP2 (predicates) — High halves	FEAT_SVE FEAT_SME
01	0	UZP1, UZP2 (predicates) — Even	FEAT_SVE FEAT_SME
01	1	UZP1, UZP2 (predicates) — Odd	FEAT_SVE FEAT_SME
10	0	TRN1, TRN2 (predicates) — Even	FEAT_SVE FEAT_SME
10	1	TRN1, TRN2 (predicates) — Odd	FEAT_SVE FEAT_SME
11		UNALLOCATED	-

SVE Permute Vector - Predicated

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1			1	op0		op1		op2	1	0	op3													

Decode fields				Instruction details	Feature
op0	op1	op2	op3		
0	000	0	0	CPY (SIMD&FP scalar)	FEAT_SVE FEAT_SME
0	000	1	0	SVE compress Active elements	-
0	000		1	SVE extract element to general register	-
0	001	1	1	UNALLOCATED	-
0	001		0	SVE extract element to SIMD&FP scalar register	-
0	01x			SVE reverse within elements	-
0	100	0	1	CPY (scalar)	FEAT_SVE FEAT_SME
0	100		0	SVE conditionally broadcast element to vector	-
0	101		0	SVE conditionally extract element to SIMD&FP scalar	-
0	110	0	0	SPLICE — Destructive	FEAT_SVE FEAT_SME
0	110	0	1	UNALLOCATED	-
0	110	1	0	SPLICE — Constructive	FEAT_SVE2 FEAT_SME
0	111	0		SVE reverse doublewords	-
0	111	1	0	UNALLOCATED	-
0	1xx	1	1	UNALLOCATED	-
0	x01	0	1	UNALLOCATED	-
1	000	0	0	UNALLOCATED	-
1	000	1	0	EXPAND	FEAT_SVE2p2 FEAT_SME2p2
1	000		1	SVE conditionally extract element to general register	-
1	!= 000			UNALLOCATED	-

SVE compress Active elements

The encodings in this section are decoded from [SVE Permute Vector - Predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1	size	1	0	0	0	0	0	1	1	0	0	Pg					Zn						Zd	

Decode fields	Instruction Details		Feature
size			
0x	COMPACT — Byte and halfword		FEAT_SVE2p2 FEAT_SME2p2
1x	COMPACT — Word and doubleword		FEAT_SVE FEAT_SME2p2

SVE extract element to general register

The encodings in this section are decoded from [SVE Permute Vector - Predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1	size	1	0	0	0	0	0	B	1	0	1	Pg					Zn						Rd	

Decode fields	Instruction Details		Feature
B			
0	LASTA (scalar)		FEAT_SVE FEAT_SME
1	LASTB (scalar)		FEAT_SVE FEAT_SME

SVE extract element to SIMD&FP scalar register

The encodings in this section are decoded from [SVE Permute Vector - Predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1	size	1	0	0	0	0	1	B	1	0	0	Pg					Zn						Vd	

Decode fields	Instruction Details	Feature
B		
0	LASTA (SIMD&FP scalar)	FEAT_SVE FEAT_SME
1	LASTB (SIMD&FP scalar)	FEAT_SVE FEAT_SME

SVE reverse within elements

The encodings in this section are decoded from [SVE Permute Vector - Predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1	size	1	0	0	1	opc	1	0	Z	Pg	Zn	Zd												

Decode fields		Instruction Details	Feature
opc	Z		
00	0	REVB, REVH, REVW — Byte, merging	FEAT_SVE FEAT_SME
00	1	REVB, REVH, REVW — Byte, zeroing	FEAT_SVE2p2 FEAT_SME2p2
01	0	REVB, REVH, REVW — Halfword, merging	FEAT_SVE FEAT_SME
01	1	REVB, REVH, REVW — Halfword, zeroing	FEAT_SVE2p2 FEAT_SME2p2
10	0	REVB, REVH, REVW — Word, merging	FEAT_SVE FEAT_SME
10	1	REVB, REVH, REVW — Word, zeroing	FEAT_SVE2p2 FEAT_SME2p2
11	0	RBIT — Merging	FEAT_SVE FEAT_SME
11	1	RBIT — Zeroing	FEAT_SVE2p2 FEAT_SME2p2

SVE conditionally broadcast element to vector

The encodings in this section are decoded from [SVE Permute Vector - Predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0	0	0	0	0	1	0	1	size	1	0	1	0	0	B	1	0	0	Pq						Zm						Zdn		

Decode fields B	Instruction Details	Feature
0	CLASTA (vectors)	FEAT_SVE FEAT_SME
1	CLASTB (vectors)	FEAT_SVE FEAT_SME

SVE conditionally extract element to SIMD&FP scalar

The encodings in this section are decoded from [SVE Permute Vector - Predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1	size	1	0	1	0	1	B	1	0	0	Pg	Zm					Vdn							

Decode fields	Instruction Details	Feature
B		
0	CLASTA (SIMD&FP scalar)	FEAT_SVE FEAT_SME
1	CLASTB (SIMD&FP scalar)	FEAT_SVE FEAT_SME

SVE reverse doublewords

The encodings in this section are decoded from [SVE Permute Vector - Predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1	size	1	0	1	1	1	0	1	0	Z		Pg						Zn					Zd	

Decode fields size	Z	Instruction Details	Feature
00	0	REVD — Merging	FEAT_SME FEAT_SVE2p1
00	1	REVD — Zeroing	FEAT_SVE2p2 FEAT_SME2p2
!= 00		UNALLOCATED	-

SVE conditionally extract element to general register

The encodings in this section are decoded from [SVE Permute Vector - Predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1	size	1	1	0	0	0	0	B	1	0	1	Pg					Zm							Rdn

Decode fields B	Instruction Details	Feature
0	CLASTA (scalar)	FEAT_SVE FEAT_SME
1	CLASTB (scalar)	FEAT_SVE FEAT_SME

SVE Integer Compare - Vectors

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	0			0							op0														

Decode fields op0	Instruction details
0	SVE integer compare vectors
1	SVE integer compare with wide elements

SVE integer compare vectors

The encodings in this section are decoded from [SVE Integer Compare - Vectors](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	0	size	0	Zm			op		0	o2	Pg			Zn			ne		Pd						

Decode fields op o2 ne	Instruction Details	Feature
0 0 0	CMP<cc> (vectors) — Higher or same	FEAT_SVE FEAT_SME
0 0 1	CMP<cc> (vectors) — Higher	FEAT_SVE FEAT_SME
0 1 0	CMP<cc> (wide elements) — Equal	FEAT_SVE FEAT_SME
0 1 1	CMP<cc> (wide elements) — Not equal	FEAT_SVE FEAT_SME
1 0 0	CMP<cc> (vectors) — Greater than or equal	FEAT_SVE FEAT_SME
1 0 1	CMP<cc> (vectors) — Greater than	FEAT_SVE FEAT_SME
1 1 0	CMP<cc> (vectors) — Equal	FEAT_SVE FEAT_SME
1 1 1	CMP<cc> (vectors) — Not equal	FEAT_SVE FEAT_SME

SVE integer compare with wide elements

The encodings in this section are decoded from [SVE Integer Compare - Vectors](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	0	size	0	Zm			U	1	lt	Pg		Zn				ne	Pd								

Decode fields			Instruction Details	Feature
U	It	ne		
0	0	0	CMP<cc> (wide elements) — Greater than or equal	FEAT_SVE FEAT_SME
0	0	1	CMP<cc> (wide elements) — Greater than	FEAT_SVE FEAT_SME
0	1	0	CMP<cc> (wide elements) — Less than	FEAT_SVE FEAT_SME
0	1	1	CMP<cc> (wide elements) — Less than or equal	FEAT_SVE FEAT_SME
1	0	0	CMP<cc> (wide elements) — Higher or same	FEAT_SVE FEAT_SME
1	0	1	CMP<cc> (wide elements) — Higher	FEAT_SVE FEAT_SME
1	1	0	CMP<cc> (wide elements) — Lower	FEAT_SVE FEAT_SME
1	1	1	CMP<cc> (wide elements) — Lower or same	FEAT_SVE FEAT_SME

SVE Propagate Break

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1			0	0					1	1					op0									

Decode fields		Instruction details
op0		
0		SVE propagate break from previous partition
1		UNALLOCATED

SVE propagate break from previous partition

The encodings in this section are decoded from [SVE Propagate Break](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1	op	S	0	0	Pm				1	1	Pg				0	Pn				B	Pd			

Decode fields			Instruction Details	Feature
op	S	B		
0	0	0	BRKPA	FEAT_SVE FEAT_SME
0	0	1	BRKPB	FEAT_SVE FEAT_SME
0	1	0	BRKPAS	FEAT_SVE FEAT_SME
0	1	1	BRKPBS	FEAT_SVE FEAT_SME
1			UNALLOCATED	-

SVE Partition Break

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1	op0		0	1		op1			0	1					op2					op3				

Decode fields				Instruction details
op0	op1	op2	op3	
0	1000	0	0	SVE propagate break to next partition
0	1000	0	1	UNALLOCATED
1	1000	0		UNALLOCATED
	0000	0		SVE partition break condition
	x000	1		UNALLOCATED
	x01x			UNALLOCATED
	x1xx			UNALLOCATED
	!= x000x001			UNALLOCATED

SVE propagate break to next partition

The encodings in this section are decoded from [SVE Partition Break](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1	0	S	0	1	1	0	0	0	0	1	Pg			0	Pn			0	Pdm					

Decode fields			Instruction Details										Feature															
S																												
0			BRKN										FEAT_SVE FEAT_SME															
1			BRKNS										FEAT_SVE FEAT_SME															

SVE partition break condition

The encodings in this section are decoded from [SVE Partition Break](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1	B	S	0	1	0	0	0	0	0	1	Pg			0	Pn			M	Pd					

Decode fields			Instruction Details										Feature															
B	S	M																										
	1	1	UNALLOCATED										-															
0	0		BRKA										FEAT_SVE FEAT_SME															
0	1	0	BRKAS										FEAT_SVE FEAT_SME															
1	0		BRKB										FEAT_SVE FEAT_SME															
1	1	0	BRKBS										FEAT_SVE FEAT_SME															

SVE Predicate Misc

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1			0	1	op0			1	1	op1			op2			op3			op4					

Decode fields					Instruction details	Feature
op0	op1	op2	op3	op4		
0000		x0		0	SVE predicate test	-
0000		x1		0	UNALLOCATED	-
0001				0	UNALLOCATED	-
1000	000	00		0	SVE predicate first active	-
1000	000	10		0	UNALLOCATED	-
1000	100	10	0000	0	SVE predicate zero	-
1000	100	10	!= 0000	0	UNALLOCATED	-
1000	110	00		0	SVE predicate read from FFR (predicated)	-
1001	000	00		0	UNALLOCATED	-
1001	000	10		0	PNEXT	FEAT_SVE FEAT_SME
1001	100	10		0	UNALLOCATED	-
1001	110	00	0000	0	SVE predicate read from FFR (unpredicated)	-
1001	110	00	!= 0000	0	UNALLOCATED	-
100x	010	x0		0	UNALLOCATED	-
100x	0x0	x1		0	UNALLOCATED	-
100x	100	0x		0	SVE predicate initialize	-
100x	100	11		0	UNALLOCATED	-
100x	110	!= 00		0	UNALLOCATED	-
100x	xx1			0	UNALLOCATED	-
x00x				1	UNALLOCATED	-
x1xx					UNALLOCATED	
!= x00xx01x					UNALLOCATED	-

SVE predicate test

The encodings in this section are decoded from [SVE Predicate Misc](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1	op	S	0	1	0	0	0	0	1	1			Pg		0			Pn		0			opc2	

Decode fields			Instruction Details	Feature
op	S	opc2		
0	0		UNALLOCATED	-
0	1	0000	PTEST	FEAT_SVE FEAT_SME
0	1	!= 0000	UNALLOCATED	-
1			UNALLOCATED	-

SVE predicate first active

The encodings in this section are decoded from [SVE Predicate Misc](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1	op	S	0	1	1	0	0	0	1	1	0	0	0	0	0			Pg		0			Pdn	

Decode fields		Instruction Details	Feature
op	S		
0	0	UNALLOCATED	-
0	1	PFIRST	FEAT_SVE FEAT_SME
1		UNALLOCATED	-

SVE predicate zero

The encodings in this section are decoded from [SVE Predicate Misc.](#)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1	op	S	0	1	1	0	0	0	1	1	1	0	0	1	0	0	0	0	0	0	0			Pd

Decode fields		Instruction Details										Feature															
op	S																										
0	0	PFALSE										FEAT_SVE FEAT_SME															
0	1	UNALLOCATED										-															
1		UNALLOCATED										-															

SVE predicate read from FFR (predicated)

The encodings in this section are decoded from [SVE Predicate Misc.](#)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1	op	S	0	1	1	0	0	0	1	1	1	1	0	0	0	Pg			0	Pd				

Decode fields		Instruction Details										Feature															
op	S																										
0	0	RDFFR (predicated)										FEAT_SVE															
0	1	RDFFRS										FEAT_SVE															
1		UNALLOCATED										-															

SVE predicate read from FFR (unpredicated)

The encodings in this section are decoded from [SVE Predicate Misc.](#)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1	op	S	0	1	1	0	0	1	1	1	1	0	0	0	0	0	0	0	0	0	0			Pd

Decode fields		Instruction Details										Feature															
op	S																										
0	0	RDFFR (unpredicated)										FEAT_SVE															
0	1	UNALLOCATED										-															
1		UNALLOCATED										-															

SVE predicate initialize

The encodings in this section are decoded from [SVE Predicate Misc.](#)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1	size		0	1	1	0	0	S	1	1	1	0	0	0					pattern		0			Pd

Decode fields		Instruction Details										Feature															
S																											
0		PTRUE (predicate)										FEAT_SVE FEAT_SME															
1		PTRUES										FEAT_SVE FEAT_SME															

SVE Predicate Count

The encodings in this section are decoded from [SVE encodings.](#)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1			1	0	0				1	0		op0			op1									

Decode fields		Instruction details
op0	op1	
000	1	SVE predicate count (predicate-as-counter)
	0	SVE predicate count
!= 000	1	UNALLOCATED

SVE predicate count (predicate-as-counter)

The encodings in this section are decoded from [SVE Predicate Count](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1	size	1	0	0	opc			1	0	0	0	0	0	vl	1	PNn			Rd					

Decode fields	Instruction Details		Feature
opc			
000	CNTP (predicate as counter)		FEAT_SME2 FEAT_SVE2p1
!= 000	UNALLOCATED		-

SVE predicate count

The encodings in this section are decoded from [SVE Predicate Count](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1	size	1	0	0	opc	1	0	Pg		0	Pn		Rd											

Decode fields	Instruction Details		Feature
opc			
000	CNTP (predicate)		FEAT_SVE FEAT_SME
001	FIRSTP		FEAT_SVE2p2 FEAT_SME2p2
010	LASTP		FEAT_SVE2p2 FEAT_SME2p2
011	UNALLOCATED		-
1xx	UNALLOCATED		-

SVE Inc/Dec by Predicate Count

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1			1	0	1	op0			1	0	0	0	op1											

Decode fields		Instruction details
op0	op1	
0	0	SVE saturating inc/dec vector by predicate count
0	1	SVE saturating inc/dec register by predicate count
1	0	SVE inc/dec vector by predicate count
1	1	SVE inc/dec register by predicate count

SVE saturating inc/dec vector by predicate count

The encodings in this section are decoded from [SVE Inc/Dec by Predicate Count](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1	size	1	0	1	0	D	U	1	0	0	0	0	0	opc	Pm			Zdn						

Decode fields			Instruction Details	Feature
D	U	opc		
		!= 00	UNALLOCATED	-
0	0	00	SQINCP (vector)	FEAT_SVE FEAT_SME
0	1	00	UQINCP (vector)	FEAT_SVE FEAT_SME
1	0	00	SQDECP (vector)	FEAT_SVE FEAT_SME
1	1	00	UQDECP (vector)	FEAT_SVE FEAT_SME

SVE saturating inc/dec register by predicate count

The encodings in this section are decoded from [SVE Inc/Dec by Predicate Count](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1	size	1	0	1	0	D	U	1	0	0	0	1	sf	op	Pm					Rdn				

Decode fields				Instruction Details	Feature
D	U	sf	op		
			1	UNALLOCATED	-
0	0	0	0	SQINCP (scalar) — 32-bit	FEAT_SVE FEAT_SME
0	0	1	0	SQINCP (scalar) — 64-bit	FEAT_SVE FEAT_SME
0	1	0	0	UQINCP (scalar) — 32-bit	FEAT_SVE FEAT_SME
0	1	1	0	UQINCP (scalar) — 64-bit	FEAT_SVE FEAT_SME
1	0	0	0	SQDECP (scalar) — 32-bit	FEAT_SVE FEAT_SME
1	0	1	0	SQDECP (scalar) — 64-bit	FEAT_SVE FEAT_SME
1	1	0	0	UQDECP (scalar) — 32-bit	FEAT_SVE FEAT_SME
1	1	1	0	UQDECP (scalar) — 64-bit	FEAT_SVE FEAT_SME

SVE inc/dec vector by predicate count

The encodings in this section are decoded from [SVE Inc/Dec by Predicate Count](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1	size	1	0	1	1	op	D	1	0	0	0	0	0	opc2			Pm						Zdn	

Decode fields			Instruction Details	Feature
op	D	opc2		
0		!= 00	UNALLOCATED	-
0	0	00	INCP (vector)	FEAT_SVE FEAT_SME
0	1	00	DECP (vector)	FEAT_SVE FEAT_SME
1			UNALLOCATED	-

SVE inc/dec register by predicate count

The encodings in this section are decoded from [SVE Inc/Dec by Predicate Count](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1	size	1	0	1	1	op	D	1	0	0	0	1	opc2	Pm			Rdn							

Decode fields			Instruction Details	Feature
op	D	opc2		
0		!= 00	UNALLOCATED	-
0	0	00	INCP (scalar)	FEAT_SVE FEAT_SME
0	1	00	DECP (scalar)	FEAT_SVE FEAT_SME
1			UNALLOCATED	-

SVE Write FFR

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1			1	0	1	op0	op1		1	0	0	1		op2			op3					op4		

Decode fields					Instruction details
op0	op1	op2	op3	op4	
0	00	000		00000	SVE FFR write from predicate
1	00	000	0000	00000	SVE FFR initialise
1	00	000	!= 0000	00000	UNALLOCATED
	00	000		!= 00000	UNALLOCATED
	00	!= 000			UNALLOCATED
	!= 00				UNALLOCATED

SVE FFR write from predicate

The encodings in this section are decoded from **SVE Write FFR**.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1	opc		1	0	1	0	0	0	1	0	0	1	0	0	0	Pn			0	0	0	0	0	

Decode fields opc	Instruction Details	Feature
00	WRFFR	FEAT_SVE
!= 00	UNALLOCATED	-

SVE FFR initialise

The encodings in this section are decoded from [SVE Write FFR](#).

[illegible]

Decode fields opc	Instruction Details	Feature
00	SETFFR	FEAT_SVE
!= 00	UNALLOCATED	-

SVE Integer Compare - Scalars

The encodings in this section are decoded from [SVE encodings](#).

[illegible]

Decode fields			Instruction details
op0	op1	op2	
0x			SVE integer compare scalar count and limit
10	00	0000	SVE conditionally terminate scalars
10	00	!= 0000	UNALLOCATED
11	00		SVE pointer conflict compare
1x	!= 00		UNALLOCATED

SVE integer compare scalar count and limit

The encodings in this section are decoded from [SVE Integer Compare - Scalars](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1	size	1	Rm						0	0	0	sf	U	lt	Rn						eq	Pd		

Decode fields			Instruction Details	Feature
U	It	eq		
0	0	0	WHILEGE (predicate)	FEAT_SVE2 FEAT_SME
0	0	1	WHILEGT (predicate)	FEAT_SVE2 FEAT_SME
0	1	0	WHILELT (predicate)	FEAT_SVE FEAT_SME
0	1	1	WHILELE (predicate)	FEAT_SVE FEAT_SME
1	0	0	WHILEHS (predicate)	FEAT_SVE2 FEAT_SME
1	0	1	WHILEHI (predicate)	FEAT_SVE2 FEAT_SME
1	1	0	WHILELO (predicate)	FEAT_SVE FEAT_SME
1	1	1	WHILELS (predicate)	FEAT_SVE FEAT_SME

SVE conditionally terminate scalars

The encodings in this section are decoded from [SVE Integer Compare - Scalars](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1	op	sz	1			Rm			0	0	1	0	0	0			Rn		ne	0	0	0	0	0

Decode fields		Instruction Details	Feature
op	ne		
0		UNALLOCATED	-
1	0	CTERMEQ, CTERMNE — Equal	FEAT_SVE FEAT_SME
1	1	CTERMEQ, CTERMNE — Not equal	FEAT_SVE FEAT_SME

SVE pointer conflict compare

The encodings in this section are decoded from [SVE Integer Compare - Scalars](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0	0	1	0	0	1	0	1	size	1	Rm						0	0	1	1	0	0	Rn						rw	Pd			

Decode fields		Instruction Details	Feature
rw			
0		WHILEWR	FEAT_SVE2 FEAT_SME
1		WHILERW	FEAT_SVE2 FEAT_SME

SVE Scalar Integer Compare - Predicate-as-counter

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1			1			op0			0	1		op1					op2			1	op3			

Decode fields				Instruction details	Feature
op0	op1	op2	op3		
00000	110			SVE extract mask predicate from predicate-as-counter	-
00000	111	000000	0	PTRUE (predicate as counter)	FEAT_SME2 FEAT_SVE2p1
00000	111	000000	1	UNALLOCATED	-
00000	111	!= 000000		UNALLOCATED	-
	01x			SVE integer compare scalar count and limit (predicate pair)	-
	x0x			SVE integer compare scalar count and limit (predicate-as-counter)	-
!= 00000	11x			UNALLOCATED	-

SVE extract mask predicate from predicate-as-counter

The encodings in this section are decoded from [SVE Scalar Integer Compare - Predicate-as-counter](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1	size	1	0	0	0	0	0	0	0	1	1	1	0	opc			Pn		1			Pd		

Decode fields	Instruction Details	Feature
opc		
0xx	PEXT (predicate)	FEAT_SME2 FEAT_SVE2p1
10x	PEXT (predicate pair)	FEAT_SME2 FEAT_SVE2p1
11x	UNALLOCATED	-

SVE integer compare scalar count and limit (predicate pair)

The encodings in this section are decoded from [SVE Scalar Integer Compare - Predicate-as-counter](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1	size	1				Rm			0	1	0	1	U	lt			Rn			1		Pd		eq

Decode fields	Instruction Details	Feature
U lt eq		
0 0 0	WHILEGE (predicate pair)	FEAT_SME2 FEAT_SVE2p1
0 0 1	WHILEGT (predicate pair)	FEAT_SME2 FEAT_SVE2p1
0 1 0	WHILELT (predicate pair)	FEAT_SME2 FEAT_SVE2p1
0 1 1	WHILELE (predicate pair)	FEAT_SME2 FEAT_SVE2p1
1 0 0	WHILEHS (predicate pair)	FEAT_SME2 FEAT_SVE2p1
1 0 1	WHILEHI (predicate pair)	FEAT_SME2 FEAT_SVE2p1
1 1 0	WHILELO (predicate pair)	FEAT_SME2 FEAT_SVE2p1
1 1 1	WHILELS (predicate pair)	FEAT_SME2 FEAT_SVE2p1

SVE integer compare scalar count and limit (predicate-as-counter)

The encodings in this section are decoded from [SVE Scalar Integer Compare - Predicate-as-counter](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1	size	1				Rm			0	1	vl	0	U	lt			Rn			1	eq		Pnd	

Decode fields	Instruction Details	Feature
U lt eq		
0 0 0	WHILEGE (predicate as counter)	FEAT_SME2 FEAT_SVE2p1
0 0 1	WHILEGT (predicate as counter)	FEAT_SME2 FEAT_SVE2p1
0 1 0	WHILELT (predicate as counter)	FEAT_SME2 FEAT_SVE2p1
0 1 1	WHILELE (predicate as counter)	FEAT_SME2 FEAT_SVE2p1
1 0 0	WHILEHS (predicate as counter)	FEAT_SME2 FEAT_SVE2p1
1 0 1	WHILEHI (predicate as counter)	FEAT_SME2 FEAT_SVE2p1
1 1 0	WHILELO (predicate as counter)	FEAT_SME2 FEAT_SVE2p1
1 1 1	WHILELS (predicate as counter)	FEAT_SME2 FEAT_SVE2p1

SVE Integer Wide Immediate - Unpredicated

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1			1	op0			op1	1	1															

Decode fields		Instruction details
op0	op1	
00		SVE integer add/subtract immediate (unpredicated)
01		SVE integer min/max immediate (unpredicated)
10		SVE integer multiply immediate (unpredicated)
11	0	SVE broadcast integer immediate (unpredicated)
11	1	SVE broadcast floating-point immediate (unpredicated)

SVE integer add/subtract immediate (unpredicated)

The encodings in this section are decoded from [SVE Integer Wide Immediate - Unpredicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1	size	1	0	0	opc			1	1	sh	imm8								Zdn					

Decode fields		Instruction Details	Feature
opc			
000		ADD (immediate)	FEAT_SVE FEAT_SME
001		SUB (immediate)	FEAT_SVE FEAT_SME
010		UNALLOCATED	-
011		SUBR (immediate)	FEAT_SVE FEAT_SME
100		SQADD (immediate)	FEAT_SVE FEAT_SME
101		UQADD (immediate)	FEAT_SVE FEAT_SME
110		SQSUB (immediate)	FEAT_SVE FEAT_SME
111		UQSUB (immediate)	FEAT_SVE FEAT_SME

SVE integer min/max immediate (unpredicated)

The encodings in this section are decoded from [SVE Integer Wide Immediate - Unpredicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1	size	1	0	1	opc			1	1	o2	imm8								Zdn					

Decode fields		Instruction Details	Feature
opc	o2		
0xx	1	UNALLOCATED	-
000	0	SMAX (immediate)	FEAT_SVE FEAT_SME
001	0	UMAX (immediate)	FEAT_SVE FEAT_SME
010	0	SMIN (immediate)	FEAT_SVE FEAT_SME
011	0	UMIN (immediate)	FEAT_SVE FEAT_SME
1xx		UNALLOCATED	-

SVE integer multiply immediate (unpredicated)

The encodings in this section are decoded from [SVE Integer Wide Immediate - Unpredicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1	size	1	1	0	opc	1	1	o2	imm8								Zdn							

Decode fields		Instruction Details	Feature
opc	o2		
000	0	MUL (immediate)	FEAT_SVE FEAT_SME
000	1	UNALLOCATED	-
!= 000		UNALLOCATED	-

SVE broadcast integer immediate (unpredicated)

The encodings in this section are decoded from [SVE Integer Wide Immediate - Unpredicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1	size	1	1	1	opc	0	1	1	sh	imm8								Zd						

Decode fields	Instruction Details		Feature
opc			
00	DUP (immediate)	FEAT_SVE FEAT_SME	
!= 00	UNALLOCATED	-	

SVE broadcast floating-point immediate (unpredicated)

The encodings in this section are decoded from [SVE Integer Wide Immediate - Unpredicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1	size	1	1	1	opc	1	1	1	o2	imm8								Zd						

Decode fields	Instruction Details		Feature
opc	o2		
00	0	FDUP	FEAT_SVE FEAT_SME
00	1	UNALLOCATED	-
!= 00		UNALLOCATED	-

SVE Integer Multiply-Add - Unpredicated

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0			0						0	op0														

Decode fields	Instruction details		Feature
op0			
0000x	SVE integer dot product (unpredicated)	-	
0001x	SVE2 saturating multiply-add interleaved long	-	
001xx	CDOT (vectors)	FEAT_SVE2 FEAT_SME	
01xxx	SVE2 complex integer multiply-add	-	
10xxx	SVE2 integer multiply-add long	-	
110xx	SVE2 saturating multiply-add long	-	
1110x	SVE2 saturating multiply-add high	-	
11110	SVE mixed sign dot product	-	
11111	UNALLOCATED	-	

SVE integer dot product (unpredicated)

The encodings in this section are decoded from [SVE Integer Multiply-Add - Unpredicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	size	0	Zm				0	0	0	0	0	U	Zn				Zda							

Decode fields	Instruction Details		Feature
size	U		
00		UNALLOCATED	-
01	0	SDOT (2-way, vectors)	FEAT_SVE2p3 FEAT_SME2p3
01	1	UDOT (2-way, vectors)	FEAT_SVE2p3 FEAT_SME2p3
1x	0	SDOT (4-way, vectors)	FEAT_SVE FEAT_SME
1x	1	UDOT (4-way, vectors)	FEAT_SVE FEAT_SME

SVE2 saturating multiply-add interleaved long

The encodings in this section are decoded from [SVE Integer Multiply-Add - Unpredicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	size	0				Zm			0	0	0	0	1	S			Zn				Zda			

Decode fields	Instruction Details																Feature											
S																												
0	SQDMLALBT																FEAT_SVE2 FEAT_SME											
1	SQDMLSLBT																FEAT_SVE2 FEAT_SME											

SVE2 complex integer multiply-add

The encodings in this section are decoded from [SVE Integer Multiply-Add - Unpredicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	size	0				Zm			0	0	1	op	rot				Zn				Zda			

Decode fields	Instruction Details																Feature											
op																												
0	CMLA (vectors)																FEAT_SVE2 FEAT_SME											
1	SQRDCMLAH (vectors)																FEAT_SVE2 FEAT_SME											

SVE2 integer multiply-add long

The encodings in this section are decoded from [SVE Integer Multiply-Add - Unpredicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	size	0				Zm			0	1	0	S	U	T				Zn				Zda		

Decode fields			Instruction Details																Feature											
S	U	T																												
0	0	0	SMLALB (vectors)																FEAT_SVE2 FEAT_SME											
0	0	1	SMLALT (vectors)																FEAT_SVE2 FEAT_SME											
0	1	0	UMLALB (vectors)																FEAT_SVE2 FEAT_SME											
0	1	1	UMLALT (vectors)																FEAT_SVE2 FEAT_SME											
1	0	0	SMLSLB (vectors)																FEAT_SVE2 FEAT_SME											
1	0	1	SMLSLT (vectors)																FEAT_SVE2 FEAT_SME											
1	1	0	UMLSLB (vectors)																FEAT_SVE2 FEAT_SME											
1	1	1	UMLSLT (vectors)																FEAT_SVE2 FEAT_SME											

SVE2 saturating multiply-add long

The encodings in this section are decoded from [SVE Integer Multiply-Add - Unpredicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	size	0				Zm			0	1	1	0	S	T				Zn				Zda		

Decode fields		Instruction Details																Feature											
S	T																												
0	0	SQDMLALB (vectors)																FEAT_SVE2 FEAT_SME											
0	1	SQDMLALT (vectors)																FEAT_SVE2 FEAT_SME											
1	0	SQDMLSLB (vectors)																FEAT_SVE2 FEAT_SME											
1	1	SQDMLSLT (vectors)																FEAT_SVE2 FEAT_SME											

SVE2 saturating multiply-add high

The encodings in this section are decoded from [SVE Integer Multiply-Add - Unpredicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	size	0				Zm			0	1	1	1	0	S				Zn				Zda		

Decode fields	Instruction Details																Feature													
S																														
0	SQRDMLAH (vectors)																FEAT_SVE2 FEAT_SME													
1	SQRDMLSH (vectors)																FEAT_SVE2 FEAT_SME													

SVE mixed sign dot product

The encodings in this section are decoded from [SVE Integer Multiply-Add - Unpredicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	size	0				Zm			0	1	1	1	1	0				Zn				Zda		

Decode fields	Instruction Details																Feature													
size																														
10	USDOT (vectors)																(FEAT_SVE && FEAT_I8MM) (FEAT_SME && FEAT_I8MM)													
!= 10	UNALLOCATED																-													

SVE2 Integer - Predicated

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0			0			op0			1	0	op1													

Decode fields		Instruction details
op0	op1	
0010	1	SVE2 integer pairwise add and accumulate long
0011	1	UNALLOCATED
011x	1	UNALLOCATED
0x0x	1	SVE2 integer unary operations (predicated)
0xxx	0	SVE2 saturating/rounding bitwise shift left (predicated)
10xx	0	SVE2 integer halving add/subtract (predicated)
10xx	1	SVE2 integer pairwise arithmetic
11xx	0	SVE2 saturating add/subtract
11xx	1	UNALLOCATED

SVE2 integer pairwise add and accumulate long

The encodings in this section are decoded from [SVE2 Integer - Predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	size	0	0	0	0	1	0	U	1	0	1			Pg				Zn				Zda		

Decode fields	Instruction Details																Feature													
U																														
0	SADALP																FEAT_SVE2 FEAT_SME													
1	UADALP																FEAT_SVE2 FEAT_SME													

SVE2 integer unary operations (predicated)

The encodings in this section are decoded from [SVE2 Integer - Predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	size	0	0	Q	0	Z	op	1	0	1			Pg					Zn				Zd		

Decode fields			Instruction Details	Feature
Q	Z	op		
0	0	0	URECPE — Merging	FEAT_SVE2 FEAT_SME
0	0	1	URSQRTE — Merging	FEAT_SVE2 FEAT_SME
0	1	0	URECPE — Zeroing	FEAT_SVE2p2 FEAT_SME2p2
0	1	1	URSQRTE — Zeroing	FEAT_SVE2p2 FEAT_SME2p2
1	0	0	SQABS — Merging	FEAT_SVE2 FEAT_SME
1	0	1	SQNEG — Merging	FEAT_SVE2 FEAT_SME
1	1	0	SQABS — Zeroing	FEAT_SVE2p2 FEAT_SME2p2
1	1	1	SQNEG — Zeroing	FEAT_SVE2p2 FEAT_SME2p2

SVE2 saturating/rounding bitwise shift left (predicated)

The encodings in this section are decoded from [SVE2 Integer - Predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	size	0	0	Q	R	N	U	1	0	0	Pg						Zm					Zdn		

Decode fields				Instruction Details	Feature
Q	R	N	U		
0		0		UNALLOCATED	-
0	0	1	0	SRSHL	FEAT_SVE2 FEAT_SME
0	0	1	1	URSHL	FEAT_SVE2 FEAT_SME
0	1	1	0	SRSHLR	FEAT_SVE2 FEAT_SME
0	1	1	1	URSHLR	FEAT_SVE2 FEAT_SME
1	0	0	0	SQSHL (vectors)	FEAT_SVE2 FEAT_SME
1	0	0	1	UQSHL (vectors)	FEAT_SVE2 FEAT_SME
1	0	1	0	SQRSHL	FEAT_SVE2 FEAT_SME
1	0	1	1	UQRSHL	FEAT_SVE2 FEAT_SME
1	1	0	0	SQSHLR	FEAT_SVE2 FEAT_SME
1	1	0	1	UQSHLR	FEAT_SVE2 FEAT_SME
1	1	1	0	SQRSHLR	FEAT_SVE2 FEAT_SME
1	1	1	1	UQRSHLR	FEAT_SVE2 FEAT_SME

SVE2 integer halving add/subtract (predicated)

The encodings in this section are decoded from [SVE2 Integer - Predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	size	0	1	0	R	S	U	1	0	0	Pg						Zm					Zdn		

Decode fields			Instruction Details	Feature
R	S	U		
0	0	0	SHADD	FEAT_SVE2 FEAT_SME
0	0	1	UHADD	FEAT_SVE2 FEAT_SME
0	1	0	SHSUB	FEAT_SVE2 FEAT_SME
0	1	1	UHSUB	FEAT_SVE2 FEAT_SME
1	0	0	SRHADD	FEAT_SVE2 FEAT_SME
1	0	1	URHADD	FEAT_SVE2 FEAT_SME
1	1	0	SHSUBR	FEAT_SVE2 FEAT_SME
1	1	1	UHSUBR	FEAT_SVE2 FEAT_SME

SVE2 integer pairwise arithmetic

The encodings in this section are decoded from [SVE2 Integer - Predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	size		0	1	0	opc		U	1	0	1		Pg				Zm					Zdn		

Decode fields		Instruction Details												Feature											
opc	U																								
00	0	SUBP												FEAT_SVE2p3 FEAT_SME2p3											
00	1	ADDP												FEAT_SVE2 FEAT_SME											
01		UNALLOCATED												-											
10	0	SMAXP												FEAT_SVE2 FEAT_SME											
10	1	UMAXP												FEAT_SVE2 FEAT_SME											
11	0	SMINP												FEAT_SVE2 FEAT_SME											
11	1	UMINP												FEAT_SVE2 FEAT_SME											

SVE2 saturating add/subtract

The encodings in this section are decoded from [SVE2 Integer - Predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	size		0	1	1	op	S	U	1	0	0		Pg				Zm					Zdn		

Decode fields			Instruction Details												Feature											
op	S	U																								
0	0	0	SQADD (vectors, predicated)												FEAT_SVE2 FEAT_SME											
0	0	1	UQADD (vectors, predicated)												FEAT_SVE2 FEAT_SME											
0	1	0	SQSUB (vectors, predicated)												FEAT_SVE2 FEAT_SME											
0	1	1	UQSUB (vectors, predicated)												FEAT_SVE2 FEAT_SME											
1	0	0	SUQADD												FEAT_SVE2 FEAT_SME											
1	0	1	USQADD												FEAT_SVE2 FEAT_SME											
1	1	0	SQSUBR												FEAT_SVE2 FEAT_SME											
1	1	1	UQSUBR												FEAT_SVE2 FEAT_SME											

SVE Multiply - Indexed

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0			1																					

Decode fields

op0

Instruction details

00000x	SVE integer dot product (indexed)
00001x	SVE2 integer multiply-add (indexed)
00010x	SVE2 saturating multiply-add high (indexed)
00011x	SVE mixed sign dot product (indexed)
001xxx	SVE2 saturating multiply-add (indexed)
0100xx	SVE2 complex integer dot product (indexed)
0101xx	UNALLOCATED
0110xx	SVE2 complex integer multiply-add (indexed)
0111xx	SVE2 complex saturating multiply-add (indexed)
10xxxx	SVE2 integer multiply-add long (indexed)
110xxx	SVE2 integer multiply long (indexed)
1110xx	SVE2 saturating multiply (indexed)
11110x	SVE2 saturating multiply high (indexed)
111110	SVE2 integer multiply (indexed)
111111	UNALLOCATED

SVE integer dot product (indexed)

The encodings in this section are decoded from [SVE Multiply - Indexed](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	size	1		opc		0	0	0	0	0	0	0	U		Zn						Zda			

Decode fields

size U

Instruction Details

Feature

0x	0	SDOT (2-way, indexed)	FEAT_SVE2p3 FEAT_SME2p3
0x	1	UDOT (2-way, indexed)	FEAT_SVE2p3 FEAT_SME2p3
10	0	SDOT (4-way, indexed) — 8-bit to 32-bit	FEAT_SVE FEAT_SME
10	1	UDOT (4-way, indexed) — 8-bit to 32-bit	FEAT_SVE FEAT_SME
11	0	SDOT (4-way, indexed) — 16-bit to 64-bit	FEAT_SVE FEAT_SME
11	1	UDOT (4-way, indexed) — 16-bit to 64-bit	FEAT_SVE FEAT_SME

SVE2 integer multiply-add (indexed)

The encodings in this section are decoded from [SVE Multiply - Indexed](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	size	1		opc		0	0	0	0	1	S					Zn					Zda			

Decode fields

size S

Instruction Details

Feature

0x	0	MLA (indexed) — 16-bit	FEAT_SVE2 FEAT_SME
0x	1	MLS (indexed) — 16-bit	FEAT_SVE2 FEAT_SME
10	0	MLA (indexed) — 32-bit	FEAT_SVE2 FEAT_SME
10	1	MLS (indexed) — 32-bit	FEAT_SVE2 FEAT_SME
11	0	MLA (indexed) — 64-bit	FEAT_SVE2 FEAT_SME
11	1	MLS (indexed) — 64-bit	FEAT_SVE2 FEAT_SME

SVE2 saturating multiply-add high (indexed)

The encodings in this section are decoded from [SVE Multiply - Indexed](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
0	1	0	0	0	1	0	0	size	1	opc						0	0	0	1	0	S	Zn						Zda					

Decode fields size	S	Instruction Details	Feature
0x	0	SQRDMLAH (indexed) — 16-bit	FEAT_SVE2 FEAT_SME
0x	1	SQRDMLSH (indexed) — 16-bit	FEAT_SVE2 FEAT_SME
10	0	SQRDMLAH (indexed) — 32-bit	FEAT_SVE2 FEAT_SME
10	1	SQRDMLSH (indexed) — 32-bit	FEAT_SVE2 FEAT_SME
11	0	SQRDMLAH (indexed) — 64-bit	FEAT_SVE2 FEAT_SME
11	1	SQRDMLSH (indexed) — 64-bit	FEAT_SVE2 FEAT_SME

SVE mixed sign dot product (indexed)

The encodings in this section are decoded from [SVE Multiply - Indexed](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
0	1	0	0	0	1	0	0	size	1	opc						0	0	0	1	1	U	zn						zda					

Decode fields size	U	Instruction Details	Feature
10	0	USDOT (indexed)	(FEAT_SVE && FEAT_I8MM) (FEAT_SME && FEAT_I8MM)
10	1	SUDOT	(FEAT_SVE && FEAT_I8MM) (FEAT_SME && FEAT_I8MM)
!= 10		UNALLOCATED	-

SVE2 saturating multiply-add (indexed)

The encodings in this section are decoded from [SVE Multiply - Indexed](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
0	1	0	0	0	1	0	0	size	1	opc						0	0	1	S	il	T	Zn						Zda					

Decode fields size	S	T	Instruction Details	Feature
0x			UNALLOCATED	-
10	0	0	SQDMLALB (indexed) — 32-bit	FEAT_SVE2 FEAT_SME
10	0	1	SQDMLALT (indexed) — 32-bit	FEAT_SVE2 FEAT_SME
10	1	0	SQDMLSLB (indexed) — 32-bit	FEAT_SVE2 FEAT_SME
10	1	1	SQDMLSLT (indexed) — 32-bit	FEAT_SVE2 FEAT_SME
11	0	0	SQDMLALB (indexed) — 64-bit	FEAT_SVE2 FEAT_SME
11	0	1	SQDMLALT (indexed) — 64-bit	FEAT_SVE2 FEAT_SME
11	1	0	SQDMLSLB (indexed) — 64-bit	FEAT_SVE2 FEAT_SME
11	1	1	SQDMLSLT (indexed) — 64-bit	FEAT_SVE2 FEAT_SME

SVE2 complex integer dot product (indexed)

The encodings in this section are decoded from [SVE Multiply - Indexed](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0	1	0	0	0	1	0	0	size	1	opc						0	1	0	0	rot	Zn						Zda					

Decode fields size	Instruction Details	Feature
0x	UNALLOCATED	-
10	CDOT (indexed) — 8-bit to 32-bit	FEAT_SVE2 FEAT_SME
11	CDOT (indexed) — 16-bit to 64-bit	FEAT_SVE2 FEAT_SME

SVE2 complex integer multiply-add (indexed)

The encodings in this section are decoded from [SVE Multiply - Indexed](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	size	1				opc			0	1	1	0	rot				Zn					Zda		

Decode fields size	Instruction Details	Feature
0x	UNALLOCATED	-
10	CMLA (indexed) — 16-bit	FEAT_SVE2 FEAT_SME
11	CMLA (indexed) — 32-bit	FEAT_SVE2 FEAT_SME

SVE2 complex saturating multiply-add (indexed)

The encodings in this section are decoded from [SVE Multiply - Indexed](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	size	1				opc			0	1	1	1	rot				Zn					Zda		

Decode fields size	Instruction Details	Feature
0x	UNALLOCATED	-
10	SQRDCMLAH (indexed) — 16-bit	FEAT_SVE2 FEAT_SME
11	SQRDCMLAH (indexed) — 32-bit	FEAT_SVE2 FEAT_SME

SVE2 integer multiply-add long (indexed)

The encodings in this section are decoded from [SVE Multiply - Indexed](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	size	1				opc			1	0	S	U	il	T					Zn			Zda		

Decode fields size	S	U	T	Instruction Details	Feature
0x				UNALLOCATED	-
10	0	0	0	SMLALB (indexed) — 32-bit	FEAT_SVE2 FEAT_SME
10	0	0	1	SMLALT (indexed) — 32-bit	FEAT_SVE2 FEAT_SME
10	0	1	0	UMLALB (indexed) — 32-bit	FEAT_SVE2 FEAT_SME
10	0	1	1	UMLALT (indexed) — 32-bit	FEAT_SVE2 FEAT_SME
10	1	0	0	SMLSLB (indexed) — 32-bit	FEAT_SVE2 FEAT_SME
10	1	0	1	SMLSLT (indexed) — 32-bit	FEAT_SVE2 FEAT_SME
10	1	1	0	UMLSLB (indexed) — 32-bit	FEAT_SVE2 FEAT_SME
10	1	1	1	UMLSLT (indexed) — 32-bit	FEAT_SVE2 FEAT_SME
11	0	0	0	SMLALB (indexed) — 64-bit	FEAT_SVE2 FEAT_SME
11	0	0	1	SMLALT (indexed) — 64-bit	FEAT_SVE2 FEAT_SME
11	0	1	0	UMLALB (indexed) — 64-bit	FEAT_SVE2 FEAT_SME
11	0	1	1	UMLALT (indexed) — 64-bit	FEAT_SVE2 FEAT_SME
11	1	0	0	SMLSLB (indexed) — 64-bit	FEAT_SVE2 FEAT_SME
11	1	0	1	SMLSLT (indexed) — 64-bit	FEAT_SVE2 FEAT_SME
11	1	1	0	UMLSLB (indexed) — 64-bit	FEAT_SVE2 FEAT_SME
11	1	1	1	UMLSLT (indexed) — 64-bit	FEAT_SVE2 FEAT_SME

SVE2 integer multiply long (indexed)

The encodings in this section are decoded from [SVE Multiply - Indexed](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	size	1		opc		1	1	0	U	il	T						Zn						Zd	

Decode fields			Instruction Details	Feature
size	U	T		
0x			UNALLOCATED	-
10	0	0	SMULLB (indexed) — 32-bit	FEAT_SVE2 FEAT_SME
10	0	1	SMULLT (indexed) — 32-bit	FEAT_SVE2 FEAT_SME
10	1	0	UMULLB (indexed) — 32-bit	FEAT_SVE2 FEAT_SME
10	1	1	UMULLT (indexed) — 32-bit	FEAT_SVE2 FEAT_SME
11	0	0	SMULLB (indexed) — 64-bit	FEAT_SVE2 FEAT_SME
11	0	1	SMULLT (indexed) — 64-bit	FEAT_SVE2 FEAT_SME
11	1	0	UMULLB (indexed) — 64-bit	FEAT_SVE2 FEAT_SME
11	1	1	UMULLT (indexed) — 64-bit	FEAT_SVE2 FEAT_SME

SVE2 saturating multiply (indexed)

The encodings in this section are decoded from [SVE Multiply - Indexed](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	size	1		opc		1	1	1	0	il	T						Zn						Zd	

Decode fields		Instruction Details	Feature
size	T		
0x		UNALLOCATED	-
10	0	SQDMULLB (indexed) — 32-bit	FEAT_SVE2 FEAT_SME
10	1	SQDMULLT (indexed) — 32-bit	FEAT_SVE2 FEAT_SME
11	0	SQDMULLB (indexed) — 64-bit	FEAT_SVE2 FEAT_SME
11	1	SQDMULLT (indexed) — 64-bit	FEAT_SVE2 FEAT_SME

SVE2 saturating multiply high (indexed)

The encodings in this section are decoded from [SVE Multiply - Indexed](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	size	1		opc		1	1	1	1	0	R						Zn						Zd	

Decode fields		Instruction Details	Feature
size	R		
0x	0	SQDMULH (indexed) — 16-bit	FEAT_SVE2 FEAT_SME
0x	1	SQRDMULH (indexed) — 16-bit	FEAT_SVE2 FEAT_SME
10	0	SQDMULH (indexed) — 32-bit	FEAT_SVE2 FEAT_SME
10	1	SQRDMULH (indexed) — 32-bit	FEAT_SVE2 FEAT_SME
11	0	SQDMULH (indexed) — 64-bit	FEAT_SVE2 FEAT_SME
11	1	SQRDMULH (indexed) — 64-bit	FEAT_SVE2 FEAT_SME

SVE2 integer multiply (indexed)

The encodings in this section are decoded from [SVE Multiply - Indexed](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	size	1		opc		1	1	1	1	1	0						Zn						Zd	

Decode fields size	Instruction Details	Feature
0x	MUL (indexed) — 16-bit	FEAT_SVE2 FEAT_SME
10	MUL (indexed) — 32-bit	FEAT_SVE2 FEAT_SME
11	MUL (indexed) — 64-bit	FEAT_SVE2 FEAT_SME

SVE2 Widening Integer Arithmetic

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	1			0						0		op0													

Decode fields op0	Instruction details
0x	SVE2 integer add/subtract long
10	SVE2 integer add/subtract wide
11	SVE2 integer multiply long

SVE2 integer add/subtract long

The encodings in this section are decoded from [SVE2 Widening Integer Arithmetic](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0	1	0	0	0	1	0	1	size	0	Zm						0	0	op	S	U	T	Zn						opc				

Decode fields op	S	U	T	Instruction Details	Feature
0	0	0	0	SADDLB	FEAT_SVE2 FEAT_SME
0	0	0	1	SADDLT	FEAT_SVE2 FEAT_SME
0	0	1	0	UADDLB	FEAT_SVE2 FEAT_SME
0	0	1	1	UADDLT	FEAT_SVE2 FEAT_SME
0	1	0	0	SSUBLB	FEAT_SVE2 FEAT_SME
0	1	0	1	SSUBLT	FEAT_SVE2 FEAT_SME
0	1	1	0	USUBLB	FEAT_SVE2 FEAT_SME
0	1	1	1	USUBLT	FEAT_SVE2 FEAT_SME
1	0			UNALLOCATED	-
1	1	0	0	SABDLB	FEAT_SVE2 FEAT_SME
1	1	0	1	SABDLT	FEAT_SVE2 FEAT_SME
1	1	1	0	UABDLB	FEAT_SVE2 FEAT_SME
1	1	1	1	UABDLT	FEAT_SVE2 FEAT_SME

SVE2 integer add/subtract wide

The encodings in this section are decoded from [SVE2 Widening Integer Arithmetic](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	1	size	0	Zm			0	1	0	S	U	T	Zn			Zd									

Decode fields			Instruction Details	Feature
S	U	T		
0	0	0	SADDWB	FEAT_SVE2 FEAT_SME
0	0	1	SADDWT	FEAT_SVE2 FEAT_SME
0	1	0	UADDWB	FEAT_SVE2 FEAT_SME
0	1	1	UADDWT	FEAT_SVE2 FEAT_SME
1	0	0	SSUBWB	FEAT_SVE2 FEAT_SME
1	0	1	SSUBWT	FEAT_SVE2 FEAT_SME
1	1	0	USUBWB	FEAT_SVE2 FEAT_SME
1	1	1	USUBWT	FEAT_SVE2 FEAT_SME

SVE2 integer multiply long

The encodings in this section are decoded from [SVE2 Widening Integer Arithmetic](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	1	size	0				Zm		0	1	1	op	U	T				Zn						Zd	

Decode fields				Instruction Details	Feature
size	op	U	T		
	0	0	0	SQDMULLB (vectors)	FEAT_SVE2 FEAT_SME
	0	0	1	SQDMULLT (vectors)	FEAT_SVE2 FEAT_SME
	1	0	0	SMULLB (vectors)	FEAT_SVE2 FEAT_SME
	1	0	1	SMULLT (vectors)	FEAT_SVE2 FEAT_SME
	1	1	0	UMULLB (vectors)	FEAT_SVE2 FEAT_SME
	1	1	1	UMULLT (vectors)	FEAT_SVE2 FEAT_SME
00	0	1	0	PMULLB — 128-bit element	FEAT_SVE_PMULL128
00	0	1	1	PMULLT — 128-bit element	FEAT_SVE_PMULL128
!= 00	0	1	0	PMULLB — 16-bit or 64-bit elements	FEAT_SVE2 FEAT_SME
!= 00	0	1	1	PMULLT — 16-bit or 64-bit elements	FEAT_SVE2 FEAT_SME

SVE Misc

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	1	op0		0						1	0		op1												

Decode fields		Instruction details
op0	op1	
0	10xx	SVE2 bitwise shift left long
1	10xx	UNALLOCATED
	00xx	SVE2 integer add/subtract interleaved long
	010x	SVE2 bitwise exclusive-OR interleaved
	0110	SVE integer matrix multiply accumulate
	0111	UNALLOCATED
	11xx	SVE2 bitwise permute

SVE2 bitwise shift left long

The encodings in this section are decoded from [SVE Misc](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	1	0	tszh	0	tszl		imm3		1	0	1	0	U	T				Zn						Zd	

Decode fields		Instruction Details	Feature
U	T		
0	0	SSHLLB	FEAT_SVE2 FEAT_SME
0	1	SSHLLT	FEAT_SVE2 FEAT_SME
1	0	USHLLB	FEAT_SVE2 FEAT_SME
1	1	USHLLT	FEAT_SVE2 FEAT_SME

SVE2 integer add/subtract interleaved long

The encodings in this section are decoded from [SVE Misc](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	1	size	0				Zm			1	0	0	0	S	tb				Zn				Zd		

Decode fields		Instruction Details	Feature
S	tb		
0	0	SADDLBT	FEAT_SVE2 FEAT_SME
0	1	UNALLOCATED	-
1	0	SSUBLBT	FEAT_SVE2 FEAT_SME
1	1	SSUBLTB	FEAT_SVE2 FEAT_SME

SVE2 bitwise exclusive-OR interleaved

The encodings in this section are decoded from [SVE Misc](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	1	size	0				Zm			1	0	0	1	0	tb				Zn				Zd		

Decode fields		Instruction Details	Feature
tb			
0		EORBT	FEAT_SVE2 FEAT_SME
1		EORTB	FEAT_SVE2 FEAT_SME

SVE integer matrix multiply accumulate

The encodings in this section are decoded from [SVE Misc](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	1	uns	0				Zm			1	0	0	1	1	0				Zn				Zd		

Decode fields		Instruction Details	Feature
uns			
00		SMMLA	FEAT_SVE && FEAT_I8MM
01		UNALLOCATED	-
10		USMMLA	FEAT_SVE && FEAT_I8MM
11		UMMLA	FEAT_SVE && FEAT_I8MM

SVE2 bitwise permute

The encodings in this section are decoded from [SVE Misc](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	1	size	0				Zm			1	0	1	1	opc					Zn				Zd		

Decode fields	Instruction Details	Feature
opc		
00	BEXT	FEAT_SVE_BitPerm
01	BDEP	FEAT_SVE_BitPerm
10	BGRP	FEAT_SVE_BitPerm
11	UNALLOCATED	-

SVE2 Accumulate

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	1			0			op0			1	1		op1												

Decode fields	Instruction details
op0	op1
0000	011
	00x
	010
	10x
	110
	111
!= 0000	011
	UNALLOCATED

SVE2 complex integer add

The encodings in this section are decoded from [SVE2 Accumulate](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	1	size	0	0	0	0	0	op	1	1	0	1	1	rot			Zm						Zdn		

Decode fields	Instruction Details	Feature
op		
0	CADD	FEAT_SVE2 FEAT_SME
1	SQCADD	FEAT_SVE2 FEAT_SME

SVE2 integer absolute difference and accumulate long

The encodings in this section are decoded from [SVE2 Accumulate](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	1	size	0	Zm			1	1	0	0	U	T	Zn			Zda									

Decode fields	Instruction Details	Feature
U	T	
0	0	SABALB
0	1	SABALT
1	0	UABALB
1	1	UABALT

SVE2 integer add/subtract long with carry

The encodings in this section are decoded from [SVE2 Accumulate](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	1	size	0	Zm			1	1	0	1	0	T	Zn			Zda									

Decode fields		Instruction Details	Feature
size	T		
0x	0	ADCLB	FEAT_SVE2 FEAT_SME
0x	1	ADCLT	FEAT_SVE2 FEAT_SME
1x	0	SBCLB	FEAT_SVE2 FEAT_SME
1x	1	SBCLT	FEAT_SVE2 FEAT_SME

SVE2 bitwise shift right and accumulate

The encodings in this section are decoded from [SVE2 Accumulate](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	1	tszh	0	tszl	imm3	1	1	1	0	R	U														

Decode fields		Instruction Details	Feature
R	U		
0	0	SSRA	FEAT_SVE2 FEAT_SME
0	1	USRA	FEAT_SVE2 FEAT_SME
1	0	SRSRA	FEAT_SVE2 FEAT_SME
1	1	URSRA	FEAT_SVE2 FEAT_SME

SVE2 bitwise shift and insert

The encodings in this section are decoded from [SVE2 Accumulate](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	1	tszh	0	tszl	imm3	1	1	1	1	0	op														

Decode fields		Instruction Details	Feature
op			
0		SRI	FEAT_SVE2 FEAT_SME
1		SLI	FEAT_SVE2 FEAT_SME

SVE2 integer absolute difference and accumulate

The encodings in this section are decoded from [SVE2 Accumulate](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	1	size	0																						

Decode fields		Instruction Details	Feature
U			
0		SABA	FEAT_SVE2 FEAT_SME
1		UABA	FEAT_SVE2 FEAT_SME

SVE2 Narrowing

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	1	op0		1			op1	op2	0	op3				op4						op5					

Decode fields						Instruction details
op0	op1	op2	op3	op4	op5	
0	00	0	10			SVE2 saturating extract narrow
0	00	1	10	0	0	SME2 multi-vec extract narrow
0	00	1	10	0	1	UNALLOCATED
0	00	1	10	1		UNALLOCATED
0			0x			SVE2 bitwise shift right narrow
0	!= 00		10			UNALLOCATED
1			0x	0	0	SME2 multi-vec shift narrow
1			0x	0	1	UNALLOCATED
1			0x	1		UNALLOCATED
1			10			UNALLOCATED
			11			SVE2 integer add/subtract narrow high part

SVE2 saturating extract narrow

The encodings in this section are decoded from [SVE2 Narrowing](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	1	0	tszh	1		tszl	0	0	0	0	1	0	opc	T				Zn					Zd		

Decode fields		Instruction Details		Feature
opc	T			
00	0	SQXTNB		FEAT_SVE2 FEAT_SME
00	1	SQXTNT		FEAT_SVE2 FEAT_SME
01	0	UQXTNB		FEAT_SVE2 FEAT_SME
01	1	UQXTNT		FEAT_SVE2 FEAT_SME
10	0	SQXTUNB		FEAT_SVE2 FEAT_SME
10	1	SQXTUNT		FEAT_SVE2 FEAT_SME
11		UNALLOCATED		-

SME2 multi-vec extract narrow

The encodings in this section are decoded from [SVE2 Narrowing](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	1	0	tszh	1		tszl	0	0	1	0	1	0	opc	0				Zn	0				Zd		

Decode fields			Instruction Details		Feature
tszh	tszl	opc			
0	10	00	SQCVTN		FEAT_SME2 FEAT_SVE2p1
0	10	01	UQCVTN		FEAT_SME2 FEAT_SVE2p1
0	10	10	SQCVTUN		FEAT_SME2 FEAT_SVE2p1
0	10	11	UNALLOCATED		-
0	!= 10		UNALLOCATED		-
1			UNALLOCATED		-

SVE2 bitwise shift right narrow

The encodings in this section are decoded from [SVE2 Narrowing](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	1	0	tszh	1		tszl	imm3	0	0	0	op	U	R	T				Zn					Zd		

Decode fields				Instruction Details	Feature
op	U	R	T		
0	0	0	0	SQSHRUNB	FEAT_SVE2 FEAT_SME
0	0	0	1	SQSHRUNT	FEAT_SVE2 FEAT_SME
0	0	1	0	SQRSHRUNB	FEAT_SVE2 FEAT_SME
0	0	1	1	SQRSHRUNT	FEAT_SVE2 FEAT_SME
0	1	0	0	SHRNB	FEAT_SVE2 FEAT_SME
0	1	0	1	SHRNT	FEAT_SVE2 FEAT_SME
0	1	1	0	RSHRNB	FEAT_SVE2 FEAT_SME
0	1	1	1	RSHRNT	FEAT_SVE2 FEAT_SME
1	0	0	0	SQSHRNB	FEAT_SVE2 FEAT_SME
1	0	0	1	SQSHRNT	FEAT_SVE2 FEAT_SME
1	0	1	0	SQRSHRNB	FEAT_SVE2 FEAT_SME
1	0	1	1	SQRSHRNT	FEAT_SVE2 FEAT_SME
1	1	0	0	UQSHRNB	FEAT_SVE2 FEAT_SME
1	1	0	1	UQSHRNT	FEAT_SVE2 FEAT_SME
1	1	1	0	UQRSHRNB	FEAT_SVE2 FEAT_SME
1	1	1	1	UQRSHRNT	FEAT_SVE2 FEAT_SME

SME2 multi-vec shift narrow

The encodings in this section are decoded from [SVE2 Narrowing](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	1	1	op0	1	opc				0	0	op1	U	R	0	Zn			0	Zd						

Decode fields				Instruction Details		Feature
op0	opc	op1	U	R		
0		0	0	0	SQSHRN	FEAT_SVE2p3 FEAT_SME2p3
0		0	1	0	UQSHRN	FEAT_SVE2p3 FEAT_SME2p3
0		1	0	0	SQSHRUN	FEAT_SVE2p3 FEAT_SME2p3
0		1	1	0	UNALLOCATED	-
0	x1xxx	0	1	1	UNALLOCATED	-
0	00xxx			1	UNALLOCATED	-
0	01xxx	0	0	1	SQRSHRUN — 8-bit	FEAT_SVE2p3 FEAT_SME2p3
0	01xxx	1	0	1	SQRSHRN — 8-bit	FEAT_SVE2p3 FEAT_SME2p3
0	01xxx	1	1	1	UQRSHRN — 8-bit	FEAT_SVE2p3 FEAT_SME2p3
0	1xxxx	0	0	1	SQRSHRUN — 16-bit	FEAT_SME2 FEAT_SVE2p1
0	1xxxx	1	0	1	SQRSHRN — 16-bit	FEAT_SME2 FEAT_SVE2p1
0	1xxxx	1	1	1	UQRSHRN — 16-bit	FEAT_SME2 FEAT_SVE2p1
0	!= 00xxx10xxx	0	1	1	UNALLOCATED	-
1					UNALLOCATED	-

SVE2 integer add/subtract narrow high part

The encodings in this section are decoded from [SVE2 Narrowing](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
0	1	0	0	0	1	0	1	size	1	Zm				0	1	1	S	R	T	Zn			Zd										

Decode fields			Instruction Details	Feature
S	R	T		
0	0	0	ADDHNB	FEAT_SVE2 FEAT_SME
0	0	1	ADDHNT	FEAT_SVE2 FEAT_SME
0	1	0	RADDHNB	FEAT_SVE2 FEAT_SME
0	1	1	RADDHNT	FEAT_SVE2 FEAT_SME
1	0	0	SUBHNB	FEAT_SVE2 FEAT_SME
1	0	1	SUBHNT	FEAT_SVE2 FEAT_SME
1	1	0	RSUBHNB	FEAT_SVE2 FEAT_SME
1	1	1	RSUBHNT	FEAT_SVE2 FEAT_SME

SVE2 Histogram (Segment) and Lookup Table

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	1	op2	op0	1						1	0	1	op1												

Decode fields			Instruction details	Feature
op2	op0	op1		
0	0	001	UNALLOCATED	-
0	0	011	LUTi6 (8-bit)	FEAT_SVE2p3
1	0	0x1	UNALLOCATED	-
	1	001	LUTi4 (8-bit and 16-bit) — Byte, single register table	(FEAT_SVE2 && FEAT_LUT) (FEAT_SME2 && FEAT_LUT)
	1	011	LUTi6 (16-bit)	FEAT_SVE2p3 FEAT_SME2p3
		000	HISTSEG	FEAT_SVE2
		100	LUTi2 (8-bit and 16-bit) — Byte	(FEAT_SVE2 && FEAT_LUT) (FEAT_SME2 && FEAT_LUT)
		1x1	SVE2 lookup table with 4-bit indices and 16-bit element size	-
		x10	LUTi2 (8-bit and 16-bit) — Halfword	(FEAT_SVE2 && FEAT_LUT) (FEAT_SME2 && FEAT_LUT)

SVE2 lookup table with 4-bit indices and 16-bit element size

The encodings in this section are decoded from [SVE2 Histogram \(Segment\) and Lookup Table](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	1	i2		1			Zm			1	0	1	1	op	1			Zn				Zd			

Decode fields	Instruction Details		Feature
op			
0	LUTi4 (8-bit and 16-bit) — Halfword, two register table	(FEAT_SVE2 && FEAT_LUT) (FEAT_SME2 && FEAT_LUT)	
1	LUTi4 (8-bit and 16-bit) — Halfword, single register table	(FEAT_SVE2 && FEAT_LUT) (FEAT_SME2 && FEAT_LUT)	

SVE2 Crypto Extensions

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	1			1	op0		op1			1	1	1		op2			op3								op4

Decode fields					Instruction details
op0	op1	op2	op3	op4	
000	00	00x	00000		SVE2 crypto unary operations
000	00	00x	!= 00000		UNALLOCATED
000	01	00x			UNALLOCATED
000	1x	00x			SVE2 crypto destructive binary operations
xx0	1x	01x			SVE2 multi-vector AES single round (two registers)
xx1	1x	01x			SVE2 multi-vector AES single round (four registers)
	00	01x			UNALLOCATED
	01	01x			UNALLOCATED
		10x			SVE2 crypto constructive binary operations
		110		0	SVE2 Multi-vector polynomial multiply long
		111		0	SVE2 Multi-vector polynomial multiply long and accumulate vectors
		11x		1	UNALLOCATED
	!= 1x	01x			UNALLOCATED
!= 000		00x			UNALLOCATED

SVE2 crypto unary operations

The encodings in this section are decoded from [SVE2 Crypto Extensions](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	1	size	1	0	0	0	0	0	0	1	1	1	0	0	op	0	0	0	0	0	Zdn				

Decode fields		Instruction Details		Feature
size	op			
00	0	AESMC		FEAT_SVE_AES
00	1	AESIMC		FEAT_SVE_AES
!= 00		UNALLOCATED		-

SVE2 crypto destructive binary operations

The encodings in this section are decoded from [SVE2 Crypto Extensions](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	1	size	1	0	0	0	1	op	1	1	1	0	0	o2	Zm				Zdn						

Decode fields			Instruction Details	Feature
size	op	o2		
00	0	0	AESE (vectors)	FEAT_SVE_AES
00	0	1	AESD (vectors)	FEAT_SVE_AES
00	1	0	SM4E	FEAT_SVE_SM4
00	1	1	UNALLOCATED	-
!= 00			UNALLOCATED	-

SVE2 multi-vector AES single round (two registers)

The encodings in this section are decoded from [SVE2 Crypto Extensions](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	1	size	1	i2	0	1	op	1	1	1	0	1	o2	Zm				Zdn				o3			

Decode fields				Instruction Details	Feature
size	op	o2	o3		
00			1	UNALLOCATED	-
00	0	0	0	AESE (indexed)	FEAT_SVE_AES2
00	0	1	0	AESD (indexed)	FEAT_SVE_AES2
00	1	0	0	AESEMC	FEAT_SVE_AES2
00	1	1	0	AESDIMC	FEAT_SVE_AES2
!= 00				UNALLOCATED	-

SVE2 multi-vector AES single round (four registers)

The encodings in this section are decoded from [SVE2 Crypto Extensions](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	1	size	1	i2	1	1	op	1	1	1	0	1	o2						Zm				Zdn		opc3

Decode fields				Instruction Details	Feature
size	op	o2	opc3		
00			!= 00	UNALLOCATED	-
00	0	0	00	AESE (indexed)	FEAT_SVE_AES2
00	0	1	00	AESD (indexed)	FEAT_SVE_AES2
00	1	0	00	AESEMC	FEAT_SVE_AES2
00	1	1	00	AESDIMC	FEAT_SVE_AES2
!= 00				UNALLOCATED	-

SVE2 crypto constructive binary operations

The encodings in this section are decoded from [SVE2 Crypto Extensions](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	1	size	1	Zm			1	1	1	1	0	op	Zn			Zd									

Decode fields		Instruction Details	Feature
size	op		
00	0	SM4KEY	FEAT_SVE_SM4
00	1	RAX1	FEAT_SVE_SHA3
!= 00		UNALLOCATED	-

SVE2 Multi-vector polynomial multiply long

The encodings in this section are decoded from [SVE2 Crypto Extensions](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0	1	0	0	0	1	0	1	size	1	Zm						1	1	1	1	1	0	Zn						Zd				0

Decode fields		Instruction Details	Feature
size			
00		PMULL	FEAT_SVE_AES2
!= 00		UNALLOCATED	-

SVE2 Multi-vector polynomial multiply long and accumulate vectors

The encodings in this section are decoded from [SVE2 Crypto Extensions](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	1	size	1	Zm			1	1	1	1	1	1	Zn			Zda			0						

Decode fields size	Instruction Details	Feature
00	PMLAL	FEAT_SVE_AES2
!= 00	UNALLOCATED	-

SVE2 FP8 widening multiply-add

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	0	op0	0	1						1	0	op1			1	0									

Decode fields op0	Decode fields op1	Instruction details
0		SVE2 FP8 multiply-add long long
1	0	SVE2 FP8 multiply-add long
1	1	UNALLOCATED

SVE2 FP8 multiply-add long long

The encodings in this section are decoded from [SVE2 FP8 widening multiply-add](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	0	0	0	1	Zm				1	0	TT	1	0	Zn				Zda							

Decode fields TT	Instruction Details	Feature
00	FMLALLBB (vectors)	FEAT_SSVE_FP8FMA (FEAT_SVE2 && FEAT_FP8FMA)
01	FMLALLBT (vectors)	FEAT_SSVE_FP8FMA (FEAT_SVE2 && FEAT_FP8FMA)
10	FMLALLTB (vectors)	FEAT_SSVE_FP8FMA (FEAT_SVE2 && FEAT_FP8FMA)
11	FMLALLTT (vectors)	FEAT_SSVE_FP8FMA (FEAT_SVE2 && FEAT_FP8FMA)

SVE2 FP8 multiply-add long

The encodings in this section are decoded from [SVE2 FP8 widening multiply-add](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	0	1	0	1	Zm				1	0	0	T	1	0	Zn				Zda						

Decode fields T	Instruction Details	Feature
0	FMLALB (vectors, FP8 to FP16)	FEAT_SSVE_FP8FMA (FEAT_SVE2 && FEAT_FP8FMA)
1	FMLALT (vectors, FP8 to FP16)	FEAT_SSVE_FP8FMA (FEAT_SVE2 && FEAT_FP8FMA)

SVE2 floating-point unary operations - zeroing predicated

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	0			0	1	1	op0		1																

Decode fields op0	Instruction details
00x	Floating-point round to integral value (predicated)
010	Floating-point convert (predicated)
011	Floating-point square root (predicated)
10x	Floating-point round and convert from integer (predicated)
11x	Floating-point log and convert to integer

Floating-point round to integral value (predicated)

The encodings in this section are decoded from [SVE2 floating-point unary operations - zeroing predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	0	size	0	1	1	0	0	op	1	opc2		Pg						Zn					Zd		

Decode fields		Instruction Details	Feature
op	opc2		
0	00	FRINT<r> — Nearest with ties to even, zeroing	FEAT_SVE2p2 FEAT_SME2p2
0	01	FRINT<r> — Toward plus infinity, zeroing	FEAT_SVE2p2 FEAT_SME2p2
0	10	FRINT<r> — Toward minus infinity, zeroing	FEAT_SVE2p2 FEAT_SME2p2
0	11	FRINT<r> — Toward zero, zeroing	FEAT_SVE2p2 FEAT_SME2p2
1	00	FRINT<r> — Nearest with ties to away, zeroing	FEAT_SVE2p2 FEAT_SME2p2
1	01	UNALLOCATED	-
1	10	FRINT<r> — Current mode signalling inexact, zeroing	FEAT_SVE2p2 FEAT_SME2p2
1	11	FRINT<r> — Current mode, zeroing	FEAT_SVE2p2 FEAT_SME2p2

Floating-point convert (predicated)

The encodings in this section are decoded from [SVE2 floating-point unary operations - zeroing predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	0	opc	0	1	1	0	1	0	1	opc2		Pg						Zn					Zd		

Decode fields		Instruction Details	Feature
opc	opc2		
x0	11	UNALLOCATED	-
00	0x	UNALLOCATED	-
00	10	FCVTX	FEAT_SVE2p2 FEAT_SME2p2
01		UNALLOCATED	-
10	00	FCVT — Single-precision to half-precision, zeroing	FEAT_SVE2p2 FEAT_SME2p2
10	01	FCVT — Half-precision to single-precision, zeroing	FEAT_SVE2p2 FEAT_SME2p2
10	10	BFCVT	FEAT_SVE2p2 FEAT_SME2p2
11	00	FCVT — Double-precision to half-precision, zeroing	FEAT_SVE2p2 FEAT_SME2p2
11	01	FCVT — Half-precision to double-precision, zeroing	FEAT_SVE2p2 FEAT_SME2p2
11	10	FCVT — Double-precision to single-precision, zeroing	FEAT_SVE2p2 FEAT_SME2p2
11	11	FCVT — Single-precision to double-precision, zeroing	FEAT_SVE2p2 FEAT_SME2p2

Floating-point square root (predicated)

The encodings in this section are decoded from [SVE2 floating-point unary operations - zeroing predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	0	size	0	1	1	0	1	1	1	opc		Pg						Zn					Zd		

Decode fields		Instruction Details	Feature
opc			
00		FRECPX	FEAT_SVE2p2 FEAT_SME2p2
01		FSQRT	FEAT_SVE2p2 FEAT_SME2p2
1x		UNALLOCATED	-

Floating-point round and convert from integer (predicated)

The encodings in this section are decoded from [SVE2 floating-point unary operations - zeroing predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	0	opc	0	1	1	1	0	o2	1	o3	U		Pg					Zn					Zd		

Decode fields				Instruction Details	Feature
opc	o2	o3	U		
00	0		0	FRINT32Z	FEAT_SVE2p2 FEAT_SME2p2
00	0		1	FRINT32X	FEAT_SVE2p2 FEAT_SME2p2
00	1		0	FRINT64Z	FEAT_SVE2p2 FEAT_SME2p2
00	1		1	FRINT64X	FEAT_SVE2p2 FEAT_SME2p2
01	0	0		UNALLOCATED	-
01	0	1	0	SCVTF (predicated) — 16-bit to half-precision, zeroing	FEAT_SVE2p2 FEAT_SME2p2
01	0	1	1	UCVTF (predicated) — 16-bit to half-precision, zeroing	FEAT_SVE2p2 FEAT_SME2p2
01	1	0	0	SCVTF (predicated) — 32-bit to half-precision, zeroing	FEAT_SVE2p2 FEAT_SME2p2
01	1	0	1	UCVTF (predicated) — 32-bit to half-precision, zeroing	FEAT_SVE2p2 FEAT_SME2p2
01	1	1	0	SCVTF (predicated) — 64-bit to half-precision, zeroing	FEAT_SVE2p2 FEAT_SME2p2
01	1	1	1	UCVTF (predicated) — 64-bit to half-precision, zeroing	FEAT_SVE2p2 FEAT_SME2p2
10	0			UNALLOCATED	-
10	1	0	0	SCVTF (predicated) — 32-bit to single-precision, zeroing	FEAT_SVE2p2 FEAT_SME2p2
10	1	0	1	UCVTF (predicated) — 32-bit to single-precision, zeroing	FEAT_SVE2p2 FEAT_SME2p2
10	1	1		UNALLOCATED	-
11	0	0	0	SCVTF (predicated) — 32-bit to double-precision, zeroing	FEAT_SVE2p2 FEAT_SME2p2
11	0	0	1	UCVTF (predicated) — 32-bit to double-precision, zeroing	FEAT_SVE2p2 FEAT_SME2p2
11	0	1		UNALLOCATED	-
11	1	0	0	SCVTF (predicated) — 64-bit to single-precision, zeroing	FEAT_SVE2p2 FEAT_SME2p2
11	1	0	1	UCVTF (predicated) — 64-bit to single-precision, zeroing	FEAT_SVE2p2 FEAT_SME2p2
11	1	1	0	SCVTF (predicated) — 64-bit to double-precision, zeroing	FEAT_SVE2p2 FEAT_SME2p2
11	1	1	1	UCVTF (predicated) — 64-bit to double-precision, zeroing	FEAT_SVE2p2 FEAT_SME2p2

Floating-point log and convert to integer

The encodings in this section are decoded from [SVE2 floating-point unary operations - zeroing predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	0	opc	0	1	1	1	1	o2	1	o3	U		Pg					Zn					Zd		

Decode fields				Instruction Details	Feature
opc	o2	o3	U		
00	0			FLOGB	FEAT_SVE2p2 FEAT_SME2p2
00	1			UNALLOCATED	-
01	0	0		UNALLOCATED	-
01	0	1	0	FCVTZS — Half-precision to 16-bit, zeroing	FEAT_SVE2p2 FEAT_SME2p2
01	0	1	1	FCVTZU — Half-precision to 16-bit, zeroing	FEAT_SVE2p2 FEAT_SME2p2
01	1	0	0	FCVTZS — Half-precision to 32-bit, zeroing	FEAT_SVE2p2 FEAT_SME2p2
01	1	0	1	FCVTZU — Half-precision to 32-bit, zeroing	FEAT_SVE2p2 FEAT_SME2p2
01	1	1	0	FCVTZS — Half-precision to 64-bit, zeroing	FEAT_SVE2p2 FEAT_SME2p2
01	1	1	1	FCVTZU — Half-precision to 64-bit, zeroing	FEAT_SVE2p2 FEAT_SME2p2
10	0			UNALLOCATED	-
10	1	0	0	FCVTZS — Single-precision to 32-bit, zeroing	FEAT_SVE2p2 FEAT_SME2p2
10	1	0	1	FCVTZU — Single-precision to 32-bit, zeroing	FEAT_SVE2p2 FEAT_SME2p2
10	1	1		UNALLOCATED	-
11	0	0	0	FCVTZS — Double-precision to 32-bit, zeroing	FEAT_SVE2p2 FEAT_SME2p2
11	0	0	1	FCVTZU — Double-precision to 32-bit, zeroing	FEAT_SVE2p2 FEAT_SME2p2
11	0	1		UNALLOCATED	-
11	1	0	0	FCVTZS — Single-precision to 64-bit, zeroing	FEAT_SVE2p2 FEAT_SME2p2
11	1	0	1	FCVTZU — Single-precision to 64-bit, zeroing	FEAT_SVE2p2 FEAT_SME2p2
11	1	1	0	FCVTZS — Double-precision to 64-bit, zeroing	FEAT_SVE2p2 FEAT_SME2p2
11	1	1	1	FCVTZU — Double-precision to 64-bit, zeroing	FEAT_SVE2p2 FEAT_SME2p2

SVE floating-point widening multiply-add - indexed

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	0	op0		1						0	1	op1	0												

Decode fields		Instruction details
op0	op1	
0	0	SVE BFloat16 floating-point dot product (indexed)
0	1	UNALLOCATED
1		SVE floating-point multiply-add long (indexed)

SVE BFloat16 floating-point dot product (indexed)

The encodings in this section are decoded from [SVE floating-point widening multiply-add - indexed](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	0	0	op	1	i2		Zm			0	1	0	0	opc2				Zn				Zda			

Decode fields		Instruction Details	Feature
op	opc2		
0	x1	FDOT (2-way, indexed, FP8 to FP16)	FEAT_SSVE_FP8DOT2 (FEAT_SVE2 && FEAT_FP8DOT2)
0	00	FDOT (2-way, indexed, FP16 to FP32)	FEAT_SME2 FEAT_SVE2p1
0	10	UNALLOCATED	-
1	00	BFDOT (indexed)	(FEAT_SVE && FEAT_BF16) (FEAT_SME && FEAT_BF16)
1	01	FDOT (4-way, indexed)	FEAT_SSVE_FP8DOT4 (FEAT_SVE2 && FEAT_FP8DOT4)
1	1x	UNALLOCATED	-

SVE floating-point multiply-add long (indexed)

The encodings in this section are decoded from [SVE floating-point widening multiply-add - indexed](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	0	1	o2	1	i3h		Zm			0	1	op	0	i3l	T			Zn					Zda		

Decode fields			Instruction Details	Feature
o2	op	T		
0	0	0	FMLALB (indexed, FP16 to FP32)	FEAT_SVE2 FEAT_SME
0	0	1	FMLALT (indexed, FP16 to FP32)	FEAT_SVE2 FEAT_SME
0	1	0	FMLSLB (indexed)	FEAT_SVE2 FEAT_SME
0	1	1	FMLSLT (indexed)	FEAT_SVE2 FEAT_SME
1	0	0	BFMLALB (indexed)	(FEAT_SVE && FEAT_BF16) (FEAT_SME && FEAT_BF16)
1	0	1	BFMLALT (indexed)	(FEAT_SVE && FEAT_BF16) (FEAT_SME && FEAT_BF16)
1	1	0	BFMLSLB (indexed)	FEAT_SME2 FEAT_SVE2p1
1	1	1	BFMLSLT (indexed)	FEAT_SME2 FEAT_SVE2p1

SVE floating-point widening multiply-add

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	0	op0		1						1	0	op1	0	0											

Decode fields		Instruction details
op0	op1	
0	0	SVE BFloat16 floating-point dot product
0	1	UNALLOCATED
1		SVE floating-point multiply-add long

SVE BFloat16 floating-point dot product

The encodings in this section are decoded from [SVE floating-point widening multiply-add](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	0	0	op	1	Zm					1	0	0	0	0	o2	Zn					Zda				

Decode fields		Instruction Details	Feature
op	o2		
0	0	FDOT (2-way, vectors, FP16 to FP32)	FEAT_SME2 FEAT_SVE2p1
0	1	FDOT (2-way, vectors, FP8 to FP16)	FEAT_SSVE_FP8DOT2 (FEAT_SVE2 && FEAT_FP8DOT2)
1	0	BFDOT (vectors)	(FEAT_SVE && FEAT_BF16) (FEAT_SME && FEAT_BF16)
1	1	FDOT (4-way, vectors)	FEAT_SSVE_FP8DOT4 (FEAT_SVE2 && FEAT_FP8DOT4)

SVE floating-point multiply-add long

The encodings in this section are decoded from [SVE floating-point widening multiply-add](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	0	1	o2	1	Zm					1	0	op	0	0	T	Zn					Zda				

Decode fields			Instruction Details	Feature
o2	op	T		
0	0	0	FMLALB (vectors, FP16 to FP32)	FEAT_SVE2 FEAT_SME
0	0	1	FMLALT (vectors, FP16 to FP32)	FEAT_SVE2 FEAT_SME
0	1	0	FMLSLB (vectors)	FEAT_SVE2 FEAT_SME
0	1	1	FMLSLT (vectors)	FEAT_SVE2 FEAT_SME
1	0	0	BFMLALB (vectors)	(FEAT_SVE && FEAT_BF16) (FEAT_SME && FEAT_BF16)
1	0	1	BFMLALT (vectors)	(FEAT_SVE && FEAT_BF16) (FEAT_SME && FEAT_BF16)
1	1	0	BFMLSLB (vectors)	FEAT_SME2 FEAT_SVE2p1
1	1	1	BFMLSLT (vectors)	FEAT_SME2 FEAT_SVE2p1

SVE floating-point unary operations - unpredicated

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	1	op0		0	0	1	op1		0		0	1	1	op2						op3					

Decode fields				Instruction details
op0	op1	op2	op3	
00	00x			SVE2 FP8 upconverts
00	010		0	SVE2 FP8 downconverts
00	010		1	UNALLOCATED
00	011			UNALLOCATED
	100			SVE2 Int to FP upconverts
	101	0x	0	SVE2 FP to Int downconverts
	101	0x	1	UNALLOCATED
	101	1x		UNALLOCATED
	11x	00		SVE floating-point reciprocal estimate (unpredicated)
	11x	!= 00		UNALLOCATED
!= 00	0xx			UNALLOCATED

SVE2 FP8 upconverts

The encodings in this section are decoded from [SVE floating-point unary operations - unpredicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	1	0	0	0	0	1	0	0	L	0	0	1	1	opc					Zn					Zd	

Decode fields		Instruction Details	Feature
L	opc		
0	00	F1CVT, F2CVT — F1CVT	(FEAT_SVE2 && FEAT_FP8) (FEAT_SME2 && FEAT_FP8)
0	01	F1CVT, F2CVT — F2CVT	(FEAT_SVE2 && FEAT_FP8) (FEAT_SME2 && FEAT_FP8)
0	10	BF1CVT, BF2CVT — BF1CVT	(FEAT_SVE2 && FEAT_FP8) (FEAT_SME2 && FEAT_FP8)
0	11	BF1CVT, BF2CVT — BF2CVT	(FEAT_SVE2 && FEAT_FP8) (FEAT_SME2 && FEAT_FP8)
1	00	F1CVTLT, F2CVTLT — F1CVTLT	(FEAT_SVE2 && FEAT_FP8) (FEAT_SME2 && FEAT_FP8)
1	01	F1CVTLT, F2CVTLT — F2CVTLT	(FEAT_SVE2 && FEAT_FP8) (FEAT_SME2 && FEAT_FP8)
1	10	BF1CVTLT, BF2CVTLT — BF1CVTLT	(FEAT_SVE2 && FEAT_FP8) (FEAT_SME2 && FEAT_FP8)
1	11	BF1CVTLT, BF2CVTLT — BF2CVTLT	(FEAT_SVE2 && FEAT_FP8) (FEAT_SME2 && FEAT_FP8)

SVE2 FP8 downconverts

The encodings in this section are decoded from [SVE floating-point unary operations - unpredicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	1	0	0	0	0	1	0	1	0	0	0	1	1	opc			Zn		0				Zd		

Decode fields opc	Instruction Details	Feature
00	FCVTN	(FEAT_SVE2 && FEAT_FP8) (FEAT_SME2 && FEAT_FP8)
01	FCVTNB	(FEAT_SVE2 && FEAT_FP8) (FEAT_SME2 && FEAT_FP8)
10	BFCVTN	(FEAT_SVE2 && FEAT_FP8) (FEAT_SME2 && FEAT_FP8)
11	FCVTNT (unpredicated)	(FEAT_SVE2 && FEAT_FP8) (FEAT_SME2 && FEAT_FP8)

SVE2 Int to FP upconverts

The encodings in this section are decoded from [SVE floating-point unary operations - unpredicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	1	size		0	0	1	1	0	0	0	0	1	1	opc			Zn						Zd		

Decode fields opc	Instruction Details	Feature
00	SCVTF (bottom, unpredicated)	FEAT_SVE2p3 FEAT_SME2p3
01	UCVTF (bottom, unpredicated)	FEAT_SVE2p3 FEAT_SME2p3
10	SCVTFLT	FEAT_SVE2p3 FEAT_SME2p3
11	UCVTFLT	FEAT_SVE2p3 FEAT_SME2p3

SVE2 FP to Int downconverts

The encodings in this section are decoded from [SVE floating-point unary operations - unpredicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	1	size	0	0	1	1	0	1	0	0	1	1	0	op	Zn			0	Zd						

Decode fields op	Instruction Details	Feature
0	FCVTZSN	FEAT_SVE2p3 FEAT_SME2p3
1	FCVTZUN	FEAT_SVE2p3 FEAT_SME2p3

SVE floating-point reciprocal estimate (unpredicated)

The encodings in this section are decoded from [SVE floating-point unary operations - unpredicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	1	size		0	0	1	1	1	op		0	0	1	1	0	0			Zn				Zd		

Decode fields op	Instruction Details	Feature
0	FRECPE	FEAT_SVE FEAT_SME
1	FRSQRT	FEAT_SVE FEAT_SME

SVE floating-point compare - with zero

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	1			0	1	0	op0			0	0	1													

Decode fields op0	Instruction details
0	SVE floating-point compare with zero
1	UNALLOCATED

SVE floating-point compare with zero

The encodings in this section are decoded from [SVE floating-point compare - with zero](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	1	size	0	1	0	0	eq	lt	0	0	1	Pg	Zn				ne	Pd							

Decode fields			Instruction Details	Feature
eq	lt	ne		
0	0	0	FCM<cc> (zero) — Greater than or equal	FEAT_SVE FEAT_SME
0	0	1	FCM<cc> (zero) — Greater than	FEAT_SVE FEAT_SME
0	1	0	FCM<cc> (zero) — Less than	FEAT_SVE FEAT_SME
0	1	1	FCM<cc> (zero) — Less than or equal	FEAT_SVE FEAT_SME
1		1	UNALLOCATED	-
1	0	0	FCM<cc> (zero) — Equal	FEAT_SVE FEAT_SME
1	1	0	FCM<cc> (zero) — Not equal	FEAT_SVE FEAT_SME

SVE floating-point accumulating reduction

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	1			0	1	1	op0			0	0	1													

Decode fields		Instruction details
op0		
0		SVE floating-point serial reduction (predicated)
1		UNALLOCATED

SVE floating-point serial reduction (predicated)

The encodings in this section are decoded from [SVE floating-point accumulating reduction](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	1	size	0	1	1	0	opc	0	0	1	Pg							Zm						Vdn	

Decode fields		Instruction Details	Feature
opc			
00		FADDA	FEAT_SVE
!= 00		UNALLOCATED	-

SVE floating-point arithmetic - predicated

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	1			0	op0					1	0	0	op1			op2									

Decode fields			Instruction details	Feature
op0	op1	op2		
0x			SVE floating-point arithmetic (predicated)	-
10	000		FTMAD	FEAT_SVE
10	!= 000		UNALLOCATED	-
11		0000	SVE floating-point arithmetic with immediate (predicated)	-
11		!= 0000	UNALLOCATED	-

SVE floating-point arithmetic (predicated)

The encodings in this section are decoded from [SVE floating-point arithmetic - predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	1	size	0	0	opc			1	0	0	Pg			Zm				Zdn							

Decode fields		Instruction Details	Feature
size	opc		
	0011	FSUBR (vectors)	FEAT_SVE FEAT_SME
	1000	FABD	FEAT_SVE FEAT_SME
	1010	FMULX	FEAT_SVE FEAT_SME
	1011	UNALLOCATED	-
	1100	FDIVR	FEAT_SVE FEAT_SME
	1101	FDIV	FEAT_SVE FEAT_SME
	1110	FAMAX	(FEAT_SVE2 && FEAT_FAMINMAX) (FEAT_SME2 && FEAT_FAMINMAX)
	1111	FAMIN	(FEAT_SVE2 && FEAT_FAMINMAX) (FEAT_SME2 && FEAT_FAMINMAX)
00	0000	BFADD (predicated)	FEAT_SVE_B16B16
00	0001	BFSUB (predicated)	FEAT_SVE_B16B16
00	0010	BFMUL (vectors, predicated)	FEAT_SVE_B16B16
00	0100	BFMAXNM	FEAT_SVE_B16B16
00	0101	BFMINNM	FEAT_SVE_B16B16
00	0110	BFMAX	FEAT_SVE_B16B16
00	0111	BFMIN	FEAT_SVE_B16B16
00	1001	BFSCALE	FEAT_SVE_BFSCALE
!= 00	0000	FADD (vectors, predicated)	FEAT_SVE FEAT_SME
!= 00	0001	FSUB (vectors, predicated)	FEAT_SVE FEAT_SME
!= 00	0010	FMUL (vectors, predicated)	FEAT_SVE FEAT_SME
!= 00	0100	FMAXNM (vectors)	FEAT_SVE FEAT_SME
!= 00	0101	FMINNM (vectors)	FEAT_SVE FEAT_SME
!= 00	0110	FMAX (vectors)	FEAT_SVE FEAT_SME
!= 00	0111	FMIN (vectors)	FEAT_SVE FEAT_SME
!= 00	1001	FSCALE	FEAT_SVE FEAT_SME

SVE floating-point arithmetic with immediate (predicated)

The encodings in this section are decoded from [SVE floating-point arithmetic - predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	1	size	0	1	1	opc		1		0	0	Pg		0		0	0	0	i1	Zdn					

Decode fields		Instruction Details	Feature
opc			
000		FADD (immediate)	FEAT_SVE FEAT_SME
001		FSUB (immediate)	FEAT_SVE FEAT_SME
010		FMUL (immediate)	FEAT_SVE FEAT_SME
011		FSUBR (immediate)	FEAT_SVE FEAT_SME
100		FMAXNM (immediate)	FEAT_SVE FEAT_SME
101		FMINNM (immediate)	FEAT_SVE FEAT_SME
110		FMAX (immediate)	FEAT_SVE FEAT_SME
111		FMIN (immediate)	FEAT_SVE FEAT_SME

SVE floating-point unary operations - merging predicated

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	1			0	op0					1	0	1													

Decode fields

op0

Instruction details

00x	SVE floating-point round to integral value
010	SVE floating-point convert precision
011	SVE floating-point unary operations
10x	SVE integer convert to floating-point
11x	SVE floating-point convert to integer

SVE floating-point round to integral value

The encodings in this section are decoded from [SVE floating-point unary operations - merging predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	1	size	0	0	0	opc	1	0	1	Pg	Zn				Zd										

Decode fields

opc

Instruction Details

Feature

000	FRINT<r> — Nearest with ties to even, merging	FEAT_SVE FEAT_SME
001	FRINT<r> — Toward plus infinity, merging	FEAT_SVE FEAT_SME
010	FRINT<r> — Toward minus infinity, merging	FEAT_SVE FEAT_SME
011	FRINT<r> — Toward zero, merging	FEAT_SVE FEAT_SME
100	FRINT<r> — Nearest with ties to away, merging	FEAT_SVE FEAT_SME
101	UNALLOCATED	-
110	FRINT<r> — Current mode signalling inexact, merging	FEAT_SVE FEAT_SME
111	FRINT<r> — Current mode, merging	FEAT_SVE FEAT_SME

SVE floating-point convert precision

The encodings in this section are decoded from [SVE floating-point unary operations - merging predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	1	opc	0	0	1	0	opc2	1	0	1	Pg						Zn							Zd	

Decode fields

opc

opc2

Instruction Details

Feature

x0	11	UNALLOCATED	-
00	0x	UNALLOCATED	-
00	10	FCVTX	FEAT_SVE2 FEAT_SME
01		UNALLOCATED	-
10	00	FCVT — Single-precision to half-precision, merging	FEAT_SVE FEAT_SME
10	01	FCVT — Half-precision to single-precision, merging	FEAT_SVE FEAT_SME
10	10	BFCVT	(FEAT_SVE && FEAT_BF16) (FEAT_SME && FEAT_BF16)
11	00	FCVT — Double-precision to half-precision, merging	FEAT_SVE FEAT_SME
11	01	FCVT — Half-precision to double-precision, merging	FEAT_SVE FEAT_SME
11	10	FCVT — Double-precision to single-precision, merging	FEAT_SVE FEAT_SME
11	11	FCVT — Single-precision to double-precision, merging	FEAT_SVE FEAT_SME

SVE floating-point unary operations

The encodings in this section are decoded from [SVE floating-point unary operations - merging predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	1	size	0	0	1	1	opc	1	0	1	Pg							Zn						Zd	

Decode fields	Instruction Details	Feature
opc		
00	FRECPX	FEAT_SVE FEAT_SME
01	FSQRT	FEAT_SVE FEAT_SME
1x	UNALLOCATED	-

SVE integer convert to floating-point

The encodings in this section are decoded from [SVE floating-point unary operations - merging predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	1	opc	0	1	0	opc2	U	1	0	1	Pg							Zn							Zd

Decode fields	Instruction Details	Feature
opc opc2 U		
00 0x 0	FRINT32Z	FEAT_SVE2p2 FEAT_SME2p2
00 0x 1	FRINT32X	FEAT_SVE2p2 FEAT_SME2p2
00 1x 0	FRINT64Z	FEAT_SVE2p2 FEAT_SME2p2
00 1x 1	FRINT64X	FEAT_SVE2p2 FEAT_SME2p2
01 00	UNALLOCATED	-
01 01 0	SCVTF (predicated) — 16-bit to half-precision, merging	FEAT_SVE FEAT_SME
01 01 1	UCVTF (predicated) — 16-bit to half-precision, merging	FEAT_SVE FEAT_SME
01 10 0	SCVTF (predicated) — 32-bit to half-precision, merging	FEAT_SVE FEAT_SME
01 10 1	UCVTF (predicated) — 32-bit to half-precision, merging	FEAT_SVE FEAT_SME
01 11 0	SCVTF (predicated) — 64-bit to half-precision, merging	FEAT_SVE FEAT_SME
01 11 1	UCVTF (predicated) — 64-bit to half-precision, merging	FEAT_SVE FEAT_SME
10 10 0	SCVTF (predicated) — 32-bit to single-precision, merging	FEAT_SVE FEAT_SME
10 10 1	UCVTF (predicated) — 32-bit to single-precision, merging	FEAT_SVE FEAT_SME
10 != 10	UNALLOCATED	-
11 00 0	SCVTF (predicated) — 32-bit to double-precision, merging	FEAT_SVE FEAT_SME
11 00 1	UCVTF (predicated) — 32-bit to double-precision, merging	FEAT_SVE FEAT_SME
11 01	UNALLOCATED	-
11 10 0	SCVTF (predicated) — 64-bit to single-precision, merging	FEAT_SVE FEAT_SME
11 10 1	UCVTF (predicated) — 64-bit to single-precision, merging	FEAT_SVE FEAT_SME
11 11 0	SCVTF (predicated) — 64-bit to double-precision, merging	FEAT_SVE FEAT_SME
11 11 1	UCVTF (predicated) — 64-bit to double-precision, merging	FEAT_SVE FEAT_SME

SVE floating-point convert to integer

The encodings in this section are decoded from [SVE floating-point unary operations - merging predicated](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	1	opc	0	1	1	opc2	U	1	0	1	Pg							Zn							Zd

Decode fields		Instruction Details		Feature
opc	opc2	U		
00		0	FLOGB	FEAT_SVE2 FEAT_SME
00		1	UNALLOCATED	-
01	00		UNALLOCATED	-
01	01	0	FCVTZS — Half-precision to 16-bit, merging	FEAT_SVE FEAT_SME
01	01	1	FCVTZU — Half-precision to 16-bit, merging	FEAT_SVE FEAT_SME
01	10	0	FCVTZS — Half-precision to 32-bit, merging	FEAT_SVE FEAT_SME
01	10	1	FCVTZU — Half-precision to 32-bit, merging	FEAT_SVE FEAT_SME
01	11	0	FCVTZS — Half-precision to 64-bit, merging	FEAT_SVE FEAT_SME
01	11	1	FCVTZU — Half-precision to 64-bit, merging	FEAT_SVE FEAT_SME
10	10	0	FCVTZS — Single-precision to 32-bit, merging	FEAT_SVE FEAT_SME
10	10	1	FCVTZU — Single-precision to 32-bit, merging	FEAT_SVE FEAT_SME
10	!= 10		UNALLOCATED	-
11	00	0	FCVTZS — Double-precision to 32-bit, merging	FEAT_SVE FEAT_SME
11	00	1	FCVTZU — Double-precision to 32-bit, merging	FEAT_SVE FEAT_SME
11	01		UNALLOCATED	-
11	10	0	FCVTZS — Single-precision to 64-bit, merging	FEAT_SVE FEAT_SME
11	10	1	FCVTZU — Single-precision to 64-bit, merging	FEAT_SVE FEAT_SME
11	11	0	FCVTZS — Double-precision to 64-bit, merging	FEAT_SVE FEAT_SME
11	11	1	FCVTZU — Double-precision to 64-bit, merging	FEAT_SVE FEAT_SME

SVE floating-point multiply-add

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	1			1						op0															

Decode fields		Instruction details	
op0			
0		SVE floating-point multiply-accumulate writing addend	
1		SVE floating-point multiply-accumulate writing multiplicand	

SVE floating-point multiply-accumulate writing addend

The encodings in this section are decoded from [SVE floating-point multiply-add](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	1	size	1		Zm		0	opc		Pg		Zn									Zda				

Decode fields		Instruction Details		Feature
size	opc			
00	00	BFMLA (vectors)		FEAT_SVE_B16B16
00	01	BFMLS (vectors)		FEAT_SVE_B16B16
00	1x	UNALLOCATED		-
!= 00	00	FMLA (vectors)		FEAT_SVE FEAT_SME
!= 00	01	FMLS (vectors)		FEAT_SVE FEAT_SME
!= 00	10	FNMLA		FEAT_SVE FEAT_SME
!= 00	11	FNMLS		FEAT_SVE FEAT_SME

SVE floating-point multiply-accumulate writing multiplicand

The encodings in this section are decoded from [SVE floating-point multiply-add](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	1	size	1				Za			1	opc		Pg					Zm					Zdn		

Decode fields opc	Instruction Details										Feature									
00	FMAD										FEAT_SVE FEAT_SME									
01	FMSB										FEAT_SVE FEAT_SME									
10	FNMAAD										FEAT_SVE FEAT_SME									
11	FNMSB										FEAT_SVE FEAT_SME									

SVE Memory - 32-bit Gather and Unsized Contiguous

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	1	0	op0	op1								op2										op3					

Decode fields				Instruction details										Feature									
op0	op1	op2	op3																				
00	x1	0xx	0	SVE 32-bit gather prefetch (scalar plus 32-bit scaled offsets)										-									
00	x1	0xx	1	UNALLOCATED										-									
01	x1	0xx		SVE 32-bit gather load halfwords (scalar plus 32-bit scaled offsets)										-									
10	x1	0xx		SVE 32-bit gather load words (scalar plus 32-bit scaled offsets)										-									
11	0x	000	0	LDR (predicate)										FEAT_SVE FEAT_SME									
11	0x	000	1	UNALLOCATED										-									
11	0x	010		LDR (vector)										FEAT_SVE FEAT_SME									
11	0x	0x1		UNALLOCATED										-									
11	1x	0xx	0	SVE contiguous prefetch (scalar plus immediate)										-									
11	1x	0xx	1	UNALLOCATED										-									
	00	10x		SVE2 32-bit gather non-temporal load (vector plus scalar)										-									
	00	110	0	SVE contiguous prefetch (scalar plus scalar)										-									
	00	111	0	SVE 32-bit gather prefetch (vector plus immediate)										-									
	00	11x	1	UNALLOCATED										-									
	01	1xx		SVE 32-bit gather load (vector plus immediate)										-									
	1x	1xx		SVE load and broadcast element										-									
!= 11	x0	0xx		SVE 32-bit gather load (scalar plus 32-bit unscaled offsets)										-									

SVE 32-bit gather prefetch (scalar plus 32-bit scaled offsets)

The encodings in this section are decoded from [SVE Memory - 32-bit Gather and Unsized Contiguous](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	1	0	0	0	xs	1					Zm	0	msz		Pg					Rn		0			prfop		

Decode fields msz	Instruction Details										Feature									
00	PRFB (scalar plus vector)										FEAT_SVE									
01	PRFH (scalar plus vector)										FEAT_SVE									
10	PRFW (scalar plus vector)										FEAT_SVE									
11	PRFD (scalar plus vector)										FEAT_SVE									

SVE 32-bit gather load halfwords (scalar plus 32-bit scaled offsets)

The encodings in this section are decoded from [SVE Memory - 32-bit Gather and Unsized Contiguous](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	1	0	0	1	xs	1					Zm	0	U	ff		Pg					Rn				Zt		

Decode fields		Instruction Details	Feature
U	ff		
0	0	LD1SH (scalar plus vector)	FEAT_SVE
0	1	LDFF1SH (scalar plus vector)	FEAT_SVE
1	0	LD1H (scalar plus vector)	FEAT_SVE
1	1	LDFF1H (scalar plus vector)	FEAT_SVE

SVE 32-bit gather load words (scalar plus 32-bit scaled offsets)

The encodings in this section are decoded from [SVE Memory - 32-bit Gather and Unsized Contiguous](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	1	0	1	0	xs	1			Zm			0	U	ff		Pg				Rn					Zt		

Decode fields		Instruction Details	Feature
U	ff		
0		UNALLOCATED	-
1	0	LD1W (scalar plus vector)	FEAT_SVE
1	1	LDFF1W (scalar plus vector)	FEAT_SVE

SVE contiguous prefetch (scalar plus immediate)

The encodings in this section are decoded from [SVE Memory - 32-bit Gather and Unsized Contiguous](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	1	0	1	1	1				imm6			0	msz		Pg				Rn				0		prfop		

Decode fields		Instruction Details	Feature
msz			
00		PRFB (scalar plus immediate)	FEAT_SVE FEAT_SME
01		PRFH (scalar plus immediate)	FEAT_SVE FEAT_SME
10		PRFW (scalar plus immediate)	FEAT_SVE FEAT_SME
11		PRFD (scalar plus immediate)	FEAT_SVE FEAT_SME

SVE2 32-bit gather non-temporal load (vector plus scalar)

The encodings in this section are decoded from [SVE Memory - 32-bit Gather and Unsized Contiguous](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	1	0	msz		0	0			Rm			1	0	U		Pg				Zn					Zt		

Decode fields		Instruction Details	Feature
msz	U		
00	0	LDNT1SB	FEAT_SVE2
00	1	LDNT1B (vector plus scalar)	FEAT_SVE2
01	0	LDNT1SH	FEAT_SVE2
01	1	LDNT1H (vector plus scalar)	FEAT_SVE2
10	0	UNALLOCATED	-
10	1	LDNT1W (vector plus scalar)	FEAT_SVE2
11		UNALLOCATED	-

SVE contiguous prefetch (scalar plus scalar)

The encodings in this section are decoded from [SVE Memory - 32-bit Gather and Unsized Contiguous](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	1	0	msz		0	0			Rm			1	1	0		Pg				Rn			0		prfop		

Decode fields		Instruction Details	Feature
msz	Rm		
	11111	UNALLOCATED	-
00	!= 11111	PRFB (scalar plus scalar)	FEAT_SVE FEAT_SME
01	!= 11111	PRFH (scalar plus scalar)	FEAT_SVE FEAT_SME
10	!= 11111	PRFW (scalar plus scalar)	FEAT_SVE FEAT_SME
11	!= 11111	PRFD (scalar plus scalar)	FEAT_SVE FEAT_SME

SVE 32-bit gather prefetch (vector plus immediate)

The encodings in this section are decoded from [SVE Memory - 32-bit Gather and Unsized Contiguous](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	1	0	msz	0	0		imm5					1	1	1		Pg				Zn			0		prfop		

Decode fields		Instruction Details	Feature
msz			
00		PRFB (vector plus immediate)	FEAT_SVE
01		PRFH (vector plus immediate)	FEAT_SVE
10		PRFW (vector plus immediate)	FEAT_SVE
11		PRFD (vector plus immediate)	FEAT_SVE

SVE 32-bit gather load (vector plus immediate)

The encodings in this section are decoded from [SVE Memory - 32-bit Gather and Unsized Contiguous](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	1	0	msz	0	1		imm5					1	U	ff		Pg				Zn					Zt		

Decode fields			Instruction Details	Feature
msz	U	ff		
00	0	0	LD1SB (vector plus immediate)	FEAT_SVE
00	0	1	LDFF1SB (vector plus immediate)	FEAT_SVE
00	1	0	LD1B (vector plus immediate)	FEAT_SVE
00	1	1	LDFF1B (vector plus immediate)	FEAT_SVE
01	0	0	LD1SH (vector plus immediate)	FEAT_SVE
01	0	1	LDFF1SH (vector plus immediate)	FEAT_SVE
01	1	0	LD1H (vector plus immediate)	FEAT_SVE
01	1	1	LDFF1H (vector plus immediate)	FEAT_SVE
10	0		UNALLOCATED	-
10	1	0	LD1W (vector plus immediate)	FEAT_SVE
10	1	1	LDFF1W (vector plus immediate)	FEAT_SVE
11			UNALLOCATED	-

SVE load and broadcast element

The encodings in this section are decoded from [SVE Memory - 32-bit Gather and Unsized Contiguous](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	1	0	dtyph	1	imm6				1	dtypl	Pg		Rn				Zt										

Decode fields		Instruction Details	Feature
dtypeh	dtypel		
00	00	LD1RB — 8-bit element	FEAT_SVE FEAT_SME
00	01	LD1RB — 16-bit element	FEAT_SVE FEAT_SME
00	10	LD1RB — 32-bit element	FEAT_SVE FEAT_SME
00	11	LD1RB — 64-bit element	FEAT_SVE FEAT_SME
01	00	LD1RSW	FEAT_SVE FEAT_SME
01	01	LD1RH — 16-bit element	FEAT_SVE FEAT_SME
01	10	LD1RH — 32-bit element	FEAT_SVE FEAT_SME
01	11	LD1RH — 64-bit element	FEAT_SVE FEAT_SME
10	00	LD1RSH — 64-bit element	FEAT_SVE FEAT_SME
10	01	LD1RSH — 32-bit element	FEAT_SVE FEAT_SME
10	10	LD1RW — 32-bit element	FEAT_SVE FEAT_SME
10	11	LD1RW — 64-bit element	FEAT_SVE FEAT_SME
11	00	LD1RSB — 64-bit element	FEAT_SVE FEAT_SME
11	01	LD1RSB — 32-bit element	FEAT_SVE FEAT_SME
11	10	LD1RSB — 16-bit element	FEAT_SVE FEAT_SME
11	11	LD1RD	FEAT_SVE FEAT_SME

SVE 32-bit gather load (scalar plus 32-bit unscaled offsets)

The encodings in this section are decoded from [SVE Memory - 32-bit Gather and Unsized Contiguous](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	1	0	!= 11	xs	0	Zm						0	U	ff	Pg			Rn				Zt					
opc																															

The following constraints also apply to this encoding: opc != '11'

Decode fields			Instruction Details	Feature
opc	U	ff		
00	0	0	LD1SB (scalar plus vector)	FEAT_SVE
00	0	1	LDFF1SB (scalar plus vector)	FEAT_SVE
00	1	0	LD1B (scalar plus vector)	FEAT_SVE
00	1	1	LDFF1B (scalar plus vector)	FEAT_SVE
01	0	0	LD1SH (scalar plus vector)	FEAT_SVE
01	0	1	LDFF1SH (scalar plus vector)	FEAT_SVE
01	1	0	LD1H (scalar plus vector)	FEAT_SVE
01	1	1	LDFF1H (scalar plus vector)	FEAT_SVE
10	0		UNALLOCATED	-
10	1	0	LD1W (scalar plus vector)	FEAT_SVE
10	1	1	LDFF1W (scalar plus vector)	FEAT_SVE

SVE Memory - Contiguous Load

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	1	0			op0	op1							op2														

Decode fields			Instruction details
op0	op1	op2	
00	0	111	SVE contiguous non-temporal load (scalar plus immediate)
00	1	001	SVE contiguous load (quadwords, scalar plus immediate)
00	1	111	SVE load multiple structures (quadwords, scalar plus immediate)
00		100	SVE contiguous load (quadwords, scalar plus scalar)
00		110	SVE contiguous non-temporal load (scalar plus scalar)
01		100	SVE load multiple structures (quadwords, scalar plus scalar)
1x		100	UNALLOCATED
	0	001	SVE load and broadcast quadword (scalar plus immediate)
	0	101	SVE contiguous load (scalar plus immediate)
	1	101	SVE contiguous non-fault load (scalar plus immediate)
		000	SVE load and broadcast quadword (scalar plus scalar)
		010	SVE contiguous load (scalar plus scalar)
		011	SVE contiguous first-fault load (scalar plus scalar)
!= 00	0	111	SVE load multiple structures (scalar plus immediate)
!= 00	1	001	UNALLOCATED
!= 00	1	111	UNALLOCATED
!= 00		110	SVE load multiple structures (scalar plus scalar)

SVE contiguous non-temporal load (scalar plus immediate)

The encodings in this section are decoded from [SVE Memory - Contiguous Load](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	1	0	msz	0	0	0			imm4			1	1	1		Pg				Rn					Zt		

Decode fields	Instruction Details	Feature
msz		
00	LDNT1B (scalar plus immediate, single register)	FEAT_SVE FEAT_SME
01	LDNT1H (scalar plus immediate, single register)	FEAT_SVE FEAT_SME
10	LDNT1W (scalar plus immediate, single register)	FEAT_SVE FEAT_SME
11	LDNT1D (scalar plus immediate, single register)	FEAT_SVE FEAT_SME

SVE contiguous load (quadwords, scalar plus immediate)

The encodings in this section are decoded from [SVE Memory - Contiguous Load](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	1	0	dtype	0	0	1			imm4			0	0	1		Pg				Rn					Zt		

Decode fields	Instruction Details	Feature
dtype		
0x	UNALLOCATED	-
10	LD1W (scalar plus immediate, single register)	FEAT_SVE2p1
11	LD1D (scalar plus immediate, single register)	FEAT_SVE2p1

SVE load multiple structures (quadwords, scalar plus immediate)

The encodings in this section are decoded from [SVE Memory - Contiguous Load](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	1	0	num	0	0	1			imm4			1	1	1		Pg				Rn					Zt		

Decode fields num	Instruction Details	Feature
00	UNALLOCATED	-
01	LD2Q (scalar plus immediate)	FEAT_SVE2p1 FEAT_SME2p1
10	LD3Q (scalar plus immediate)	FEAT_SVE2p1 FEAT_SME2p1
11	LD4Q (scalar plus immediate)	FEAT_SVE2p1 FEAT_SME2p1

SVE contiguous load (quadwords, scalar plus scalar)

The encodings in this section are decoded from [SVE Memory - Contiguous Load](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	1	0	dtype		0	0			Rm			1	0	0		Pg				Rn					Zt		

Decode fields dtype	Rm	Instruction Details	Feature
0x		UNALLOCATED	-
1x	11111	UNALLOCATED	-
10	!= 11111	LD1W (scalar plus scalar, single register)	FEAT_SVE2p1
11	!= 11111	LD1D (scalar plus scalar, single register)	FEAT_SVE2p1

SVE contiguous non-temporal load (scalar plus scalar)

The encodings in this section are decoded from [SVE Memory - Contiguous Load](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	1	0	msz		0	0			Rm			1	1	0		Pg				Rn					Zt		

Decode fields msz	Rm	Instruction Details	Feature
	11111	UNALLOCATED	-
00	!= 11111	LDNT1B (scalar plus scalar, single register)	FEAT_SVE FEAT_SME
01	!= 11111	LDNT1H (scalar plus scalar, single register)	FEAT_SVE FEAT_SME
10	!= 11111	LDNT1W (scalar plus scalar, single register)	FEAT_SVE FEAT_SME
11	!= 11111	LDNT1D (scalar plus scalar, single register)	FEAT_SVE FEAT_SME

SVE load multiple structures (quadwords, scalar plus scalar)

The encodings in this section are decoded from [SVE Memory - Contiguous Load](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	1	0	num		0	1			Rm			1	0	0		Pg				Rn					Zt		

Decode fields num	Rm	Instruction Details	Feature
00		UNALLOCATED	-
01	!= 11111	LD2Q (scalar plus scalar)	FEAT_SVE2p1 FEAT_SME2p1
10	!= 11111	LD3Q (scalar plus scalar)	FEAT_SVE2p1 FEAT_SME2p1
11	!= 11111	LD4Q (scalar plus scalar)	FEAT_SVE2p1 FEAT_SME2p1
!= 00	11111	UNALLOCATED	-

SVE load and broadcast quadword (scalar plus immediate)

The encodings in this section are decoded from [SVE Memory - Contiguous Load](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	1	0	msz	ssz		0			imm4			0	0	1		Pg				Rn					Zt		

Decode fields		Instruction Details	Feature
msz	ssz		
	1x	UNALLOCATED	-
00	00	LD1RQB (scalar plus immediate)	FEAT_SVE FEAT_SME
00	01	LD1ROB (scalar plus immediate)	FEAT_F64MM
01	00	LD1RQH (scalar plus immediate)	FEAT_SVE FEAT_SME
01	01	LD1ROH (scalar plus immediate)	FEAT_F64MM
10	00	LD1RQW (scalar plus immediate)	FEAT_SVE FEAT_SME
10	01	LD1ROW (scalar plus immediate)	FEAT_F64MM
11	00	LD1RQD (scalar plus immediate)	FEAT_SVE FEAT_SME
11	01	LD1ROD (scalar plus immediate)	FEAT_F64MM

SVE contiguous load (scalar plus immediate)

The encodings in this section are decoded from [SVE Memory - Contiguous Load](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	0	1	0	0	1	0	dtype				0	imm4				1	0	1	Pg			Rn			Zt							

Decode fields dtype	Instruction Details	Feature
0000	LD1B (scalar plus immediate, single register) — 8-bit element	FEAT_SVE FEAT_SME
0001	LD1B (scalar plus immediate, single register) — 16-bit element	FEAT_SVE FEAT_SME
0010	LD1B (scalar plus immediate, single register) — 32-bit element	FEAT_SVE FEAT_SME
0011	LD1B (scalar plus immediate, single register) — 64-bit element	FEAT_SVE FEAT_SME
0100	LD1SW (scalar plus immediate)	FEAT_SVE FEAT_SME
0101	LD1H (scalar plus immediate, single register) — 16-bit element	FEAT_SVE FEAT_SME
0110	LD1H (scalar plus immediate, single register) — 32-bit element	FEAT_SVE FEAT_SME
0111	LD1H (scalar plus immediate, single register) — 64-bit element	FEAT_SVE FEAT_SME
1000	LD1SH (scalar plus immediate) — 64-bit element	FEAT_SVE FEAT_SME
1001	LD1SH (scalar plus immediate) — 32-bit element	FEAT_SVE FEAT_SME
1010	LD1W (scalar plus immediate, single register) — 32-bit element	FEAT_SVE FEAT_SME
1011	LD1W (scalar plus immediate, single register) — 64-bit element	FEAT_SVE FEAT_SME
1100	LD1SB (scalar plus immediate) — 64-bit element	FEAT_SVE FEAT_SME
1101	LD1SB (scalar plus immediate) — 32-bit element	FEAT_SVE FEAT_SME
1110	LD1SB (scalar plus immediate) — 16-bit element	FEAT_SVE FEAT_SME
1111	LD1D (scalar plus immediate, single register)	FEAT_SVE FEAT_SME

SVE contiguous non-fault load (scalar plus immediate)

The encodings in this section are decoded from [SVE Memory - Contiguous Load](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	0	1	0	0	1	0	dtype				1	imm4				1	0	1	Pg			Rn			Zt							

Decode fields	Instruction Details	Feature
dtype		
0000	LDNF1B — 8-bit element	FEAT_SVE
0001	LDNF1B — 16-bit element	FEAT_SVE
0010	LDNF1B — 32-bit element	FEAT_SVE
0011	LDNF1B — 64-bit element	FEAT_SVE
0100	LDNF1SW	FEAT_SVE
0101	LDNF1H — 16-bit element	FEAT_SVE
0110	LDNF1H — 32-bit element	FEAT_SVE
0111	LDNF1H — 64-bit element	FEAT_SVE
1000	LDNF1SH — 64-bit element	FEAT_SVE
1001	LDNF1SH — 32-bit element	FEAT_SVE
1010	LDNF1W — 32-bit element	FEAT_SVE
1011	LDNF1W — 64-bit element	FEAT_SVE
1100	LDNF1SB — 64-bit element	FEAT_SVE
1101	LDNF1SB — 32-bit element	FEAT_SVE
1110	LDNF1SB — 16-bit element	FEAT_SVE
1111	LDNF1D	FEAT_SVE

SVE load and broadcast quadword (scalar plus scalar)

The encodings in this section are decoded from [SVE Memory - Contiguous Load](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	1	0	msz	ssz			Rm					0	0	0	Pg			Rn						Zt			

Decode fields	Instruction Details	Feature
msz ssz Rm		
	0x 11111	UNALLOCATED
	1x	UNALLOCATED
00	00 != 11111	LD1RQB (scalar plus scalar) FEAT_SVE FEAT_SME
00	01 != 11111	LD1ROB (scalar plus scalar) FEAT_F64MM
01	00 != 11111	LD1RQH (scalar plus scalar) FEAT_SVE FEAT_SME
01	01 != 11111	LD1ROH (scalar plus scalar) FEAT_F64MM
10	00 != 11111	LD1RQW (scalar plus scalar) FEAT_SVE FEAT_SME
10	01 != 11111	LD1ROW (scalar plus scalar) FEAT_F64MM
11	00 != 11111	LD1RQD (scalar plus scalar) FEAT_SVE FEAT_SME
11	01 != 11111	LD1ROD (scalar plus scalar) FEAT_F64MM

SVE contiguous load (scalar plus scalar)

The encodings in this section are decoded from [SVE Memory - Contiguous Load](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	1	0	dtype				Rm				0			1	0	Pg		Rn				Zt					

Decode fields		Instruction Details	Feature
dtype	Rm		
	11111	UNALLOCATED	-
0000	!= 11111	LD1B (scalar plus scalar, single register) — 8-bit element	FEAT_SVE FEAT_SME
0001	!= 11111	LD1B (scalar plus scalar, single register) — 16-bit element	FEAT_SVE FEAT_SME
0010	!= 11111	LD1B (scalar plus scalar, single register) — 32-bit element	FEAT_SVE FEAT_SME
0011	!= 11111	LD1B (scalar plus scalar, single register) — 64-bit element	FEAT_SVE FEAT_SME
0100	!= 11111	LD1SW (scalar plus scalar)	FEAT_SVE FEAT_SME
0101	!= 11111	LD1H (scalar plus scalar, single register) — 16-bit element	FEAT_SVE FEAT_SME
0110	!= 11111	LD1H (scalar plus scalar, single register) — 32-bit element	FEAT_SVE FEAT_SME
0111	!= 11111	LD1H (scalar plus scalar, single register) — 64-bit element	FEAT_SVE FEAT_SME
1000	!= 11111	LD1SH (scalar plus scalar) — 64-bit element	FEAT_SVE FEAT_SME
1001	!= 11111	LD1SH (scalar plus scalar) — 32-bit element	FEAT_SVE FEAT_SME
1010	!= 11111	LD1W (scalar plus scalar, single register) — 32-bit element	FEAT_SVE FEAT_SME
1011	!= 11111	LD1W (scalar plus scalar, single register) — 64-bit element	FEAT_SVE FEAT_SME
1100	!= 11111	LD1SB (scalar plus scalar) — 64-bit element	FEAT_SVE FEAT_SME
1101	!= 11111	LD1SB (scalar plus scalar) — 32-bit element	FEAT_SVE FEAT_SME
1110	!= 11111	LD1SB (scalar plus scalar) — 16-bit element	FEAT_SVE FEAT_SME
1111	!= 11111	LD1D (scalar plus scalar, single register)	FEAT_SVE FEAT_SME

SVE contiguous first-fault load (scalar plus scalar)

The encodings in this section are decoded from [SVE Memory - Contiguous Load](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	1	0				dtype					Rm		0	1	1		Pg			Rn					Zt		

Decode fields		Instruction Details	Feature
dtype			
0000		LDFF1B (scalar plus scalar) — 8-bit element	FEAT_SVE
0001		LDFF1B (scalar plus scalar) — 16-bit element	FEAT_SVE
0010		LDFF1B (scalar plus scalar) — 32-bit element	FEAT_SVE
0011		LDFF1B (scalar plus scalar) — 64-bit element	FEAT_SVE
0100		LDFF1SW (scalar plus scalar)	FEAT_SVE
0101		LDFF1H (scalar plus scalar) — 16-bit element	FEAT_SVE
0110		LDFF1H (scalar plus scalar) — 32-bit element	FEAT_SVE
0111		LDFF1H (scalar plus scalar) — 64-bit element	FEAT_SVE
1000		LDFF1SH (scalar plus scalar) — 64-bit element	FEAT_SVE
1001		LDFF1SH (scalar plus scalar) — 32-bit element	FEAT_SVE
1010		LDFF1W (scalar plus scalar) — 32-bit element	FEAT_SVE
1011		LDFF1W (scalar plus scalar) — 64-bit element	FEAT_SVE
1100		LDFF1SB (scalar plus scalar) — 64-bit element	FEAT_SVE
1101		LDFF1SB (scalar plus scalar) — 32-bit element	FEAT_SVE
1110		LDFF1SB (scalar plus scalar) — 16-bit element	FEAT_SVE
1111		LDFF1D (scalar plus scalar)	FEAT_SVE

SVE load multiple structures (scalar plus immediate)

The encodings in this section are decoded from [SVE Memory - Contiguous Load](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	1	0			msz	!= 00	0				imm4		1	1	1		Pg			Rn					Zt		
																	opc														

The following constraints also apply to this encoding: `opc != '00'`

Decode fields		Instruction Details	Feature
msz	opc		
00	01	LD2B (scalar plus immediate)	FEAT_SVE FEAT_SME
00	10	LD3B (scalar plus immediate)	FEAT_SVE FEAT_SME
00	11	LD4B (scalar plus immediate)	FEAT_SVE FEAT_SME
01	01	LD2H (scalar plus immediate)	FEAT_SVE FEAT_SME
01	10	LD3H (scalar plus immediate)	FEAT_SVE FEAT_SME
01	11	LD4H (scalar plus immediate)	FEAT_SVE FEAT_SME
10	01	LD2W (scalar plus immediate)	FEAT_SVE FEAT_SME
10	10	LD3W (scalar plus immediate)	FEAT_SVE FEAT_SME
10	11	LD4W (scalar plus immediate)	FEAT_SVE FEAT_SME
11	01	LD2D (scalar plus immediate)	FEAT_SVE FEAT_SME
11	10	LD3D (scalar plus immediate)	FEAT_SVE FEAT_SME
11	11	LD4D (scalar plus immediate)	FEAT_SVE FEAT_SME

SVE load multiple structures (scalar plus scalar)

The encodings in this section are decoded from [SVE Memory - Contiguous Load](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	1	0	msz		!= 00		Rm				1	1	0	Pg		Rn				Zt							
opc																															

The following constraints also apply to this encoding: `opc != '00'`

Decode fields		Rm	Instruction Details	Feature
msz	opc			
	!= 00	11111	UNALLOCATED	-
00	01	!= 11111	LD2B (scalar plus scalar)	FEAT_SVE FEAT_SME
00	10	!= 11111	LD3B (scalar plus scalar)	FEAT_SVE FEAT_SME
00	11	!= 11111	LD4B (scalar plus scalar)	FEAT_SVE FEAT_SME
01	01	!= 11111	LD2H (scalar plus scalar)	FEAT_SVE FEAT_SME
01	10	!= 11111	LD3H (scalar plus scalar)	FEAT_SVE FEAT_SME
01	11	!= 11111	LD4H (scalar plus scalar)	FEAT_SVE FEAT_SME
10	01	!= 11111	LD2W (scalar plus scalar)	FEAT_SVE FEAT_SME
10	10	!= 11111	LD3W (scalar plus scalar)	FEAT_SVE FEAT_SME
10	11	!= 11111	LD4W (scalar plus scalar)	FEAT_SVE FEAT_SME
11	01	!= 11111	LD2D (scalar plus scalar)	FEAT_SVE FEAT_SME
11	10	!= 11111	LD3D (scalar plus scalar)	FEAT_SVE FEAT_SME
11	11	!= 11111	LD4D (scalar plus scalar)	FEAT_SVE FEAT_SME

SVE Memory - 64-bit Gather

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	1	0	op0		op1						op2										op3						

Decode fields				Instruction details	Feature
op0	op1	op2	op3		
00	00	101		LD1Q	FEAT_SVE2p1
00	01	0xx	1	UNALLOCATED	-
00	11	1xx	0	SVE 64-bit gather prefetch (scalar plus 64-bit scaled offsets)	-
00	11		1	UNALLOCATED	-
00	xx	0xx	0	SVE 64-bit gather prefetch (scalar plus unpacked 32-bit scaled offsets)	-
	00	111	0	SVE 64-bit gather prefetch (vector plus immediate)	-
	00	111	1	UNALLOCATED	-
	00	1x0		SVE2 64-bit gather non-temporal load (vector plus scalar)	-
	01	1xx		SVE 64-bit gather load (vector plus immediate)	-
	10	1xx		SVE 64-bit gather load (scalar plus 64-bit unscaled offsets)	-
	xx	0xx		SVE 64-bit gather load (scalar plus unpacked 32-bit unscaled offsets)	-
!= 00	00	101		UNALLOCATED	-
!= 00	11	1xx		SVE 64-bit gather load (scalar plus 64-bit scaled offsets)	-
!= 00	xx	0xx		SVE 64-bit gather load (scalar plus 32-bit unpacked scaled offsets)	-

SVE 64-bit gather prefetch (scalar plus 64-bit scaled offsets)

The encodings in this section are decoded from [SVE Memory - 64-bit Gather](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	1	0	0	0	1	1			Zm			1	msz		Pg				Rn				0			prfop	

Decode fields		Instruction Details	Feature
msz			
00		PRFB (scalar plus vector)	FEAT_SVE
01		PRFH (scalar plus vector)	FEAT_SVE
10		PRFW (scalar plus vector)	FEAT_SVE
11		PRFD (scalar plus vector)	FEAT_SVE

SVE 64-bit gather prefetch (scalar plus unpacked 32-bit scaled offsets)

The encodings in this section are decoded from [SVE Memory - 64-bit Gather](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	1	0	0	0	xs	1			Zm			0	msz		Pg				Rn				0			prfop	

Decode fields		Instruction Details	Feature
msz			
00		PRFB (scalar plus vector)	FEAT_SVE
01		PRFH (scalar plus vector)	FEAT_SVE
10		PRFW (scalar plus vector)	FEAT_SVE
11		PRFD (scalar plus vector)	FEAT_SVE

SVE 64-bit gather prefetch (vector plus immediate)

The encodings in this section are decoded from [SVE Memory - 64-bit Gather](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	1	0		msz	0	0			imm5			1	1	1		Pg				Zn			0			prfop	

Decode fields	Instruction Details	Feature
msz		
00	PRFB (vector plus immediate)	FEAT_SVE
01	PRFH (vector plus immediate)	FEAT_SVE
10	PRFW (vector plus immediate)	FEAT_SVE
11	PRFD (vector plus immediate)	FEAT_SVE

SVE2 64-bit gather non-temporal load (vector plus scalar)

The encodings in this section are decoded from [SVE Memory - 64-bit Gather](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	1	0	msz	0	0	Rm			1	U	0	Pg		Zn			Zt										

Decode fields		Instruction Details	Feature
msz	U		
00	0	LDNT1SB	FEAT_SVE2
00	1	LDNT1B (vector plus scalar)	FEAT_SVE2
01	0	LDNT1SH	FEAT_SVE2
01	1	LDNT1H (vector plus scalar)	FEAT_SVE2
10	0	LDNT1SW	FEAT_SVE2
10	1	LDNT1W (vector plus scalar)	FEAT_SVE2
11	0	UNALLOCATED	-
11	1	LDNT1D (vector plus scalar)	FEAT_SVE2

SVE 64-bit gather load (vector plus immediate)

The encodings in this section are decoded from [SVE Memory - 64-bit Gather](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	1	0	msz		0	1	imm5			1	U	ff	Pg		Zn					Zt							

Decode fields			Instruction Details	Feature
msz	U	ff		
00	0	0	LD1SB (vector plus immediate)	FEAT_SVE
00	0	1	LDFF1SB (vector plus immediate)	FEAT_SVE
00	1	0	LD1B (vector plus immediate)	FEAT_SVE
00	1	1	LDFF1B (vector plus immediate)	FEAT_SVE
01	0	0	LD1SH (vector plus immediate)	FEAT_SVE
01	0	1	LDFF1SH (vector plus immediate)	FEAT_SVE
01	1	0	LD1H (vector plus immediate)	FEAT_SVE
01	1	1	LDFF1H (vector plus immediate)	FEAT_SVE
10	0	0	LD1SW (vector plus immediate)	FEAT_SVE
10	0	1	LDFF1SW (vector plus immediate)	FEAT_SVE
10	1	0	LD1W (vector plus immediate)	FEAT_SVE
10	1	1	LDFF1W (vector plus immediate)	FEAT_SVE
11	0		UNALLOCATED	-
11	1	0	LD1D (vector plus immediate)	FEAT_SVE
11	1	1	LDFF1D (vector plus immediate)	FEAT_SVE

SVE 64-bit gather load (scalar plus 64-bit unscaled offsets)

The encodings in this section are decoded from [SVE Memory - 64-bit Gather](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	1	0	msz		1	0			Zm			1	U	ff		Pg				Rn					Zt		

Decode fields			Instruction Details	Feature
msz	U	ff		
00	0	0	LD1SB (scalar plus vector)	FEAT_SVE
00	0	1	LDFF1SB (scalar plus vector)	FEAT_SVE
00	1	0	LD1B (scalar plus vector)	FEAT_SVE
00	1	1	LDFF1B (scalar plus vector)	FEAT_SVE
01	0	0	LD1SH (scalar plus vector)	FEAT_SVE
01	0	1	LDFF1SH (scalar plus vector)	FEAT_SVE
01	1	0	LD1H (scalar plus vector)	FEAT_SVE
01	1	1	LDFF1H (scalar plus vector)	FEAT_SVE
10	0	0	LD1SW (scalar plus vector)	FEAT_SVE
10	0	1	LDFF1SW (scalar plus vector)	FEAT_SVE
10	1	0	LD1W (scalar plus vector)	FEAT_SVE
10	1	1	LDFF1W (scalar plus vector)	FEAT_SVE
11	0		UNALLOCATED	-
11	1	0	LD1D (scalar plus vector)	FEAT_SVE
11	1	1	LDFF1D (scalar plus vector)	FEAT_SVE

SVE 64-bit gather load (scalar plus unpacked 32-bit unscaled offsets)

The encodings in this section are decoded from [SVE Memory - 64-bit Gather](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	1	0	msz	xs		0			Zm			0	U	ff		Pg				Rn					Zt		

Decode fields			Instruction Details	Feature
msz	U	ff		
00	0	0	LD1SB (scalar plus vector)	FEAT_SVE
00	0	1	LDFF1SB (scalar plus vector)	FEAT_SVE
00	1	0	LD1B (scalar plus vector)	FEAT_SVE
00	1	1	LDFF1B (scalar plus vector)	FEAT_SVE
01	0	0	LD1SH (scalar plus vector)	FEAT_SVE
01	0	1	LDFF1SH (scalar plus vector)	FEAT_SVE
01	1	0	LD1H (scalar plus vector)	FEAT_SVE
01	1	1	LDFF1H (scalar plus vector)	FEAT_SVE
10	0	0	LD1SW (scalar plus vector)	FEAT_SVE
10	0	1	LDFF1SW (scalar plus vector)	FEAT_SVE
10	1	0	LD1W (scalar plus vector)	FEAT_SVE
10	1	1	LDFF1W (scalar plus vector)	FEAT_SVE
11	0		UNALLOCATED	-
11	1	0	LD1D (scalar plus vector)	FEAT_SVE
11	1	1	LDFF1D (scalar plus vector)	FEAT_SVE

SVE 64-bit gather load (scalar plus 64-bit scaled offsets)

The encodings in this section are decoded from [SVE Memory - 64-bit Gather](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	1	0	!= 00		1	1			Zm			1	U	ff		Pg				Rn					Zt		
opc																															

The following constraints also apply to this encoding: opc != '00'

Decode fields			Instruction Details	Feature
opc	U	ff		
01	0	0	LD1SH (scalar plus vector)	FEAT_SVE
01	0	1	LDFF1SH (scalar plus vector)	FEAT_SVE
01	1	0	LD1H (scalar plus vector)	FEAT_SVE
01	1	1	LDFF1H (scalar plus vector)	FEAT_SVE
10	0	0	LD1SW (scalar plus vector)	FEAT_SVE
10	0	1	LDFF1SW (scalar plus vector)	FEAT_SVE
10	1	0	LD1W (scalar plus vector)	FEAT_SVE
10	1	1	LDFF1W (scalar plus vector)	FEAT_SVE
11	0		UNALLOCATED	-
11	1	0	LD1D (scalar plus vector)	FEAT_SVE
11	1	1	LDFF1D (scalar plus vector)	FEAT_SVE

SVE 64-bit gather load (scalar plus 32-bit unpacked scaled offsets)

The encodings in this section are decoded from [SVE Memory - 64-bit Gather](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	1	0	!= 00	xs	1	Zm						0	U	ff	Pg			Rn			Zt						
opc																															

The following constraints also apply to this encoding: opc != '00'

Decode fields			Instruction Details	Feature
opc	U	ff		
01	0	0	LD1SH (scalar plus vector)	FEAT_SVE
01	0	1	LDFF1SH (scalar plus vector)	FEAT_SVE
01	1	0	LD1H (scalar plus vector)	FEAT_SVE
01	1	1	LDFF1H (scalar plus vector)	FEAT_SVE
10	0	0	LD1SW (scalar plus vector)	FEAT_SVE
10	0	1	LDFF1SW (scalar plus vector)	FEAT_SVE
10	1	0	LD1W (scalar plus vector)	FEAT_SVE
10	1	1	LDFF1W (scalar plus vector)	FEAT_SVE
11	0		UNALLOCATED	-
11	1	0	LD1D (scalar plus vector)	FEAT_SVE
11	1	1	LDFF1D (scalar plus vector)	FEAT_SVE

SVE Memory - Non-temporal and Quadword Scatter Store

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	1	0	op0		op1					0	0	1															

Decode fields		Instruction details	Feature
op0	op1		
000	1	ST1Q	FEAT_SVE2p1
xx0	0	SVE2 64-bit scatter non-temporal store (vector plus scalar)	-
xx1	0	SVE2 32-bit scatter non-temporal store (vector plus scalar)	-
!= 000	1	UNALLOCATED	-

SVE2 64-bit scatter non-temporal store (vector plus scalar)

The encodings in this section are decoded from [SVE Memory - Non-temporal and Quadword Scatter Store](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	1	0	msz	0	0		Rm					0	0	1		Pg				Zn					Zt		

Decode fields	Instruction Details																Feature									
msz																										
00	STNT1B (vector plus scalar)																FEAT_SVE2									
01	STNT1H (vector plus scalar)																FEAT_SVE2									
10	STNT1W (vector plus scalar)																FEAT_SVE2									
11	STNT1D (vector plus scalar)																FEAT_SVE2									

SVE2 32-bit scatter non-temporal store (vector plus scalar)

The encodings in this section are decoded from [SVE Memory - Non-temporal and Quadword Scatter Store](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	1	0	msz	1	0		Rm					0	0	1		Pg				Zn					Zt		

Decode fields	Instruction Details																Feature									
msz																										
00	STNT1B (vector plus scalar)																FEAT_SVE2									
01	STNT1H (vector plus scalar)																FEAT_SVE2									
10	STNT1W (vector plus scalar)																FEAT_SVE2									
11	UNALLOCATED																-									

SVE Memory - Non-temporal and Multi-register Contiguous Store

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	1	0			op0							0	1	1													

Decode fields	Instruction details																									
op0																										
00	SVE contiguous non-temporal store (scalar plus scalar)																									
!= 00	SVE store multiple structures (scalar plus scalar)																									

SVE contiguous non-temporal store (scalar plus scalar)

The encodings in this section are decoded from [SVE Memory - Non-temporal and Multi-register Contiguous Store](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	1	0	msz	0	0		Rm					0	1	1		Pg				Rn					Zt		

Decode fields		Instruction Details		Feature	
msz	Rm				
	11111	UNALLOCATED		-	
00	!= 11111	STNT1B (scalar plus scalar, single register)		FEAT_SVE FEAT_SME	
01	!= 11111	STNT1H (scalar plus scalar, single register)		FEAT_SVE FEAT_SME	
10	!= 11111	STNT1W (scalar plus scalar, single register)		FEAT_SVE FEAT_SME	
11	!= 11111	STNT1D (scalar plus scalar, single register)		FEAT_SVE FEAT_SME	

SVE store multiple structures (scalar plus scalar)

The encodings in this section are decoded from [SVE Memory - Non-temporal and Multi-register Contiguous Store](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	1	0	msz	!= 00			Rm					0	1	1		Pg				Rn					Zt		

opc

The following constraints also apply to this encoding: `opc != '00'`

Decode fields			Instruction Details	Feature
msz	opc	Rm		
	!= 00	11111	UNALLOCATED	-
00	01	!= 11111	ST2B (scalar plus scalar)	FEAT_SVE FEAT_SME
00	10	!= 11111	ST3B (scalar plus scalar)	FEAT_SVE FEAT_SME
00	11	!= 11111	ST4B (scalar plus scalar)	FEAT_SVE FEAT_SME
01	01	!= 11111	ST2H (scalar plus scalar)	FEAT_SVE FEAT_SME
01	10	!= 11111	ST3H (scalar plus scalar)	FEAT_SVE FEAT_SME
01	11	!= 11111	ST4H (scalar plus scalar)	FEAT_SVE FEAT_SME
10	01	!= 11111	ST2W (scalar plus scalar)	FEAT_SVE FEAT_SME
10	10	!= 11111	ST3W (scalar plus scalar)	FEAT_SVE FEAT_SME
10	11	!= 11111	ST4W (scalar plus scalar)	FEAT_SVE FEAT_SME
11	01	!= 11111	ST2D (scalar plus scalar)	FEAT_SVE FEAT_SME
11	10	!= 11111	ST3D (scalar plus scalar)	FEAT_SVE FEAT_SME
11	11	!= 11111	ST4D (scalar plus scalar)	FEAT_SVE FEAT_SME

SVE Memory - Contiguous Store and Unsized Contiguous

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	0	0	1	0	op0				op1						0	op2		0					op3							

Decode fields				Instruction details	Feature
op0	op1	op2	op3		
0xx	00	0		SVE store multiple structures (quadwords, scalar plus immediate)	-
0xx	01	0		UNALLOCATED	-
0xx	1x	0		SVE store multiple structures (quadwords, scalar plus scalar)	-
10x		0		UNALLOCATED	-
110		0	0	STR (predicate)	FEAT_SVE FEAT_SME
110		0	1	UNALLOCATED	-
110		1		STR (vector)	FEAT_SVE FEAT_SME
111		0		UNALLOCATED	-
!= 110		1		SVE contiguous store (scalar plus scalar)	-

SVE store multiple structures (quadwords, scalar plus immediate)

The encodings in this section are decoded from [SVE Memory - Contiguous Store and Unsized Contiguous](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	1	0	0	num		0	0	imm4				0	0	0	Pg		Rn				Zt						

Decode fields		Instruction Details	Feature
num			
00	UNALLOCATED	-	
01	ST2Q (scalar plus immediate)	FEAT_SVE2p1 FEAT_SME2p1	
10	ST3Q (scalar plus immediate)	FEAT_SVE2p1 FEAT_SME2p1	
11	ST4Q (scalar plus immediate)	FEAT_SVE2p1 FEAT_SME2p1	

SVE store multiple structures (quadwords, scalar plus scalar)

The encodings in this section are decoded from [SVE Memory - Contiguous Store and Unsized Contiguous](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	1	0	0	num	1				Rm			0	0	0		Pg				Rn					Zt		

Decode fields		Instruction Details		Feature
num	Rm			
00		UNALLOCATED		-
01	!= 11111	ST2Q (scalar plus scalar)		FEAT_SVE2p1 FEAT_SME2p1
10	!= 11111	ST3Q (scalar plus scalar)		FEAT_SVE2p1 FEAT_SME2p1
11	!= 11111	ST4Q (scalar plus scalar)		FEAT_SVE2p1 FEAT_SME2p1
!= 00	11111	UNALLOCATED		-

SVE contiguous store (scalar plus scalar)

The encodings in this section are decoded from [SVE Memory - Contiguous Store and Unsized Contiguous](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	1	0	!= 110	o2					Rm			0	1	0		Pg				Rn					Zt		
opc																															

The following constraints also apply to this encoding: opc != '110'

Decode fields		Instruction Details		Feature
opc	o2	Rm		
0xxxx1		11111	UNALLOCATED	-
0x0		11111	UNALLOCATED	!
00x		!= 11111	ST1B (scalar plus scalar, single register)	FEAT_SVE FEAT_SME
01x		!= 11111	ST1H (scalar plus scalar, single register)	FEAT_SVE FEAT_SME
10x100	0	11111	UNALLOCATED	-
100	0	!= 11111	ST1W (scalar plus scalar, single register) — SVE2	FEAT_SVE2p1
100	1		UNALLOCATED	-
101		!= 11111	ST1W (scalar plus scalar, single register) — SVE	FEAT_SVE FEAT_SME
101	1	11111	UNALLOCATED	!
111		11111	UNALLOCATED	!
111	0	!= 11111	ST1D (scalar plus scalar, single register) — SVE2	FEAT_SVE2p1
111	1	!= 11111	ST1D (scalar plus scalar, single register) — SVE	FEAT_SVE FEAT_SME

SVE Memory - Scatter

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	1	0			op0							1	0	1													

Decode fields	Instruction details
op0	
00	SVE 64-bit scatter store (scalar plus 64-bit unscaled offsets)
01	SVE 64-bit scatter store (scalar plus 64-bit scaled offsets)
10	SVE 64-bit scatter store (vector plus immediate)
11	SVE 32-bit scatter store (vector plus immediate)

SVE 64-bit scatter store (scalar plus 64-bit unscaled offsets)

The encodings in this section are decoded from [SVE Memory - Scatter](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	1	0	msz	0	0				Zm			1	0	1		Pg				Rn					Zt		

Decode fields	Instruction Details	Feature
msz		
00	ST1B (scalar plus vector)	FEAT_SVE
01	ST1H (scalar plus vector)	FEAT_SVE
10	ST1W (scalar plus vector)	FEAT_SVE
11	ST1D (scalar plus vector)	FEAT_SVE

SVE 64-bit scatter store (scalar plus 64-bit scaled offsets)

The encodings in this section are decoded from [SVE Memory - Scatter](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	1	0	msz		0	1			Zm			1	0	1		Pg				Rn					Zt		

Decode fields	Instruction Details	Feature
msz		
00	UNALLOCATED	-
01	ST1H (scalar plus vector)	FEAT_SVE
10	ST1W (scalar plus vector)	FEAT_SVE
11	ST1D (scalar plus vector)	FEAT_SVE

SVE 64-bit scatter store (vector plus immediate)

The encodings in this section are decoded from [SVE Memory - Scatter](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	1	0	msz		1	0			imm5			1	0	1		Pg				Zn					Zt		

Decode fields	Instruction Details	Feature
msz		
00	ST1B (vector plus immediate)	FEAT_SVE
01	ST1H (vector plus immediate)	FEAT_SVE
10	ST1W (vector plus immediate)	FEAT_SVE
11	ST1D (vector plus immediate)	FEAT_SVE

SVE 32-bit scatter store (vector plus immediate)

The encodings in this section are decoded from [SVE Memory - Scatter](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	1	0	msz		1	1	imm5					1	0	1		Pg		Zn					Zt				

Decode fields	Instruction Details	Feature
msz		
00	ST1B (vector plus immediate)	FEAT_SVE
01	ST1H (vector plus immediate)	FEAT_SVE
10	ST1W (vector plus immediate)	FEAT_SVE
11	UNALLOCATED	-

SVE Memory - Contiguous Store with Immediate Offset

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	1	0			op0	op1						1	1	1													

Decode fields		Instruction details
op0	op1	
00	1	SVE contiguous non-temporal store (scalar plus immediate)
	0	SVE contiguous store (scalar plus immediate)
!= 00	1	SVE store multiple structures (scalar plus immediate)

SVE contiguous non-temporal store (scalar plus immediate)

The encodings in this section are decoded from [SVE Memory - Contiguous Store with Immediate Offset](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	1	0	msz		0	0	1		imm4			1	1	1		Pg			Rn						Zt		

Decode fields		Instruction Details	Feature
msz			
00		STNT1B (scalar plus immediate, single register)	FEAT_SVE FEAT_SME
01		STNT1H (scalar plus immediate, single register)	FEAT_SVE FEAT_SME
10		STNT1W (scalar plus immediate, single register)	FEAT_SVE FEAT_SME
11		STNT1D (scalar plus immediate, single register)	FEAT_SVE FEAT_SME

SVE contiguous store (scalar plus immediate)

The encodings in this section are decoded from [SVE Memory - Contiguous Store with Immediate Offset](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	1	0	msz		opc		0		imm4			1	1	1		Pg			Rn						Zt		

Decode fields		Instruction Details	Feature
msz	opc		
00		ST1B (scalar plus immediate, single register)	FEAT_SVE FEAT_SME
01		ST1H (scalar plus immediate, single register)	FEAT_SVE FEAT_SME
10	00	ST1W (scalar plus immediate, single register) — SVE2	FEAT_SVE2p1
10	01	UNALLOCATED	-
10	1x	ST1W (scalar plus immediate, single register) — SVE	FEAT_SVE FEAT_SME
11	0x	UNALLOCATED	-
11	10	ST1D (scalar plus immediate, single register) — SVE2	FEAT_SVE2p1
11	11	ST1D (scalar plus immediate, single register) — SVE	FEAT_SVE FEAT_SME

SVE store multiple structures (scalar plus immediate)

The encodings in this section are decoded from [SVE Memory - Contiguous Store with Immediate Offset](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	1	0	msz		!= 00		1	imm4			1		1	1	Pg			Rn				Zt					
opc																															

The following constraints also apply to this encoding: opc != '00'

Decode fields		Instruction Details	Feature
msz	opc		
00	01	ST2B (scalar plus immediate)	FEAT_SVE FEAT_SME
00	10	ST3B (scalar plus immediate)	FEAT_SVE FEAT_SME
00	11	ST4B (scalar plus immediate)	FEAT_SVE FEAT_SME
01	01	ST2H (scalar plus immediate)	FEAT_SVE FEAT_SME
01	10	ST3H (scalar plus immediate)	FEAT_SVE FEAT_SME
01	11	ST4H (scalar plus immediate)	FEAT_SVE FEAT_SME
10	01	ST2W (scalar plus immediate)	FEAT_SVE FEAT_SME
10	10	ST3W (scalar plus immediate)	FEAT_SVE FEAT_SME
10	11	ST4W (scalar plus immediate)	FEAT_SVE FEAT_SME
11	01	ST2D (scalar plus immediate)	FEAT_SVE FEAT_SME
11	10	ST3D (scalar plus immediate)	FEAT_SVE FEAT_SME
11	11	ST4D (scalar plus immediate)	FEAT_SVE FEAT_SME

SVE Memory - Scatter with Optional Sign Extend

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	1	0			op0							1			0												

Decode fields		Instruction details
op0		
00		SVE 64-bit scatter store (scalar plus unpacked 32-bit unscaled offsets)
01		SVE 64-bit scatter store (scalar plus unpacked 32-bit scaled offsets)
10		SVE 32-bit scatter store (scalar plus 32-bit unscaled offsets)
11		SVE 32-bit scatter store (scalar plus 32-bit scaled offsets)

SVE 64-bit scatter store (scalar plus unpacked 32-bit unscaled offsets)

The encodings in this section are decoded from [SVE Memory - Scatter with Optional Sign Extend](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	1	0	msz	0	0	Zm						1	xs	0	Pg			Rn				Zt					

Decode fields		Instruction Details	Feature
msz			
00		ST1B (scalar plus vector)	FEAT_SVE
01		ST1H (scalar plus vector)	FEAT_SVE
10		ST1W (scalar plus vector)	FEAT_SVE
11		ST1D (scalar plus vector)	FEAT_SVE

SVE 64-bit scatter store (scalar plus unpacked 32-bit scaled offsets)

The encodings in this section are decoded from [SVE Memory - Scatter with Optional Sign Extend](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	1	0	msz	0	1	Zm			1	xs	0	Pg		Rn			Zt										

Decode fields	Instruction Details	Feature
msz		
00	UNALLOCATED	-
01	ST1H (scalar plus vector)	FEAT_SVE
10	ST1W (scalar plus vector)	FEAT_SVE
11	ST1D (scalar plus vector)	FEAT_SVE

SVE 32-bit scatter store (scalar plus 32-bit unscaled offsets)

The encodings in this section are decoded from [SVE Memory - Scatter with Optional Sign Extend](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	1	0	msz		1	0			Zm		1	xs	0		Pg					Rn					Zt		

Decode fields	Instruction Details	Feature
msz		
00	ST1B (scalar plus vector)	FEAT_SVE
01	ST1H (scalar plus vector)	FEAT_SVE
10	ST1W (scalar plus vector)	FEAT_SVE
11	UNALLOCATED	-

SVE 32-bit scatter store (scalar plus 32-bit scaled offsets)

The encodings in this section are decoded from [SVE Memory - Scatter with Optional Sign Extend](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	1	0	msz		1	1			Zm		1	xs	0		Pg					Rn					Zt		

Decode fields	Instruction Details	Feature
msz		
00	UNALLOCATED	-
01	ST1H (scalar plus vector)	FEAT_SVE
10	ST1W (scalar plus vector)	FEAT_SVE
11	UNALLOCATED	-

SVE integer add/subtract vectors (unpredicated)

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0	size		1			Zm		0	0	0		opc					Zn					Zd		

Decode fields	Instruction Details	Feature
size	opc	
	000	ADD (vectors, unpredicated)
	001	SUB (vectors, unpredicated)
	100	SQADD (vectors, unpredicated)
	101	UQADD (vectors, unpredicated)
	110	SQSUB (vectors, unpredicated)
	111	UQSUB (vectors, unpredicated)
11	010	ADDPT (unpredicated)
11	011	SUBPT (unpredicated)
!= 11	01x	UNALLOCATED
		-

SVE address generation

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0	opc	1				Zm			1	0	1	0	msz				Zn					Zd		

Decode fields	Instruction Details																Feature									
opc																										
00	ADR — Unpacked 32-bit signed offsets																FEAT_SVE									
01	ADR — Unpacked 32-bit unsigned offsets																FEAT_SVE									
1x	ADR — Packed offsets																FEAT_SVE									

SVE table lookup (three sources)

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1	size	1				Zm			0	0	1	0	1	op				Zn				Zd		

Decode fields	Instruction Details																Feature									
op																										
0	TBL																FEAT_SVE2 FEAT_SME									
1	TBX																FEAT_SVE2 FEAT_SME									

SVE permute vector elements

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1	size	1				Zm			0	1	1		opc				Zn					Zd		

Decode fields	Instruction Details																Feature									
opc																										
000	ZIP1, ZIP2 (vectors) — Low halves																FEAT_SVE FEAT_SME									
001	ZIP1, ZIP2 (vectors) — High halves																FEAT_SVE FEAT_SME									
010	UZP1, UZP2 (vectors) — Even																FEAT_SVE FEAT_SME									
011	UZP1, UZP2 (vectors) — Odd																FEAT_SVE FEAT_SME									
100	TRN1, TRN2 (vectors) — Even																FEAT_SVE FEAT_SME									
101	TRN1, TRN2 (vectors) — Odd																FEAT_SVE FEAT_SME									
11x	UNALLOCATED																-									

SVE integer compare with unsigned immediate

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	0			1				imm7				lt		Pg				Zn			ne			Pd	

Decode fields		Instruction Details	Feature
lt	ne		
0	0	CMP<cc> (immediate) — Higher or same	FEAT_SVE FEAT_SME
0	1	CMP<cc> (immediate) — Higher	FEAT_SVE FEAT_SME
1	0	CMP<cc> (immediate) — Lower	FEAT_SVE FEAT_SME
1	1	CMP<cc> (immediate) — Lower or same	FEAT_SVE FEAT_SME

SVE predicate logical operations

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1	op	S	0	0		Pm			0	1		Pg			o2		Pn			o3		Pd		

Decode fields				Instruction Details				Feature			
op	S	o2	o3								
0	0	0	0	AND (predicates)				FEAT_SVE FEAT_SME			
0	0	0	1	BIC (predicates)				FEAT_SVE FEAT_SME			
0	0	1	0	EOR (predicates)				FEAT_SVE FEAT_SME			
0	0	1	1	SEL (predicates)				FEAT_SVE FEAT_SME			
0	1	0	0	ANDS				FEAT_SVE FEAT_SME			
0	1	0	1	BICS				FEAT_SVE FEAT_SME			
0	1	1	0	EORS				FEAT_SVE FEAT_SME			
0	1	1	1	UNALLOCATED				-			
1	0	0	0	ORR (predicates)				FEAT_SVE FEAT_SME			
1	0	0	1	ORN (predicates)				FEAT_SVE FEAT_SME			
1	0	1	0	NOR				FEAT_SVE FEAT_SME			
1	0	1	1	NAND				FEAT_SVE FEAT_SME			
1	1	0	0	ORRS				FEAT_SVE FEAT_SME			
1	1	0	1	ORNS				FEAT_SVE FEAT_SME			
1	1	1	0	NORS				FEAT_SVE FEAT_SME			
1	1	1	1	NANDS				FEAT_SVE FEAT_SME			

SVE integer compare with signed immediate

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1	size	0		imm5					op	0	o2		Pg				Zn			ne		Pd		

Decode fields			Instruction Details				Feature			
op	o2	ne								
0	0	0	CMP<cc> (immediate) — Greater than or equal				FEAT_SVE FEAT_SME			
0	0	1	CMP<cc> (immediate) — Greater than				FEAT_SVE FEAT_SME			
0	1	0	CMP<cc> (immediate) — Less than				FEAT_SVE FEAT_SME			
0	1	1	CMP<cc> (immediate) — Less than or equal				FEAT_SVE FEAT_SME			
1	0	0	CMP<cc> (immediate) — Equal				FEAT_SVE FEAT_SME			
1	0	1	CMP<cc> (immediate) — Not equal				FEAT_SVE FEAT_SME			
1	1		UNALLOCATED				-			

SVE broadcast predicate element

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1	i1	tszh	1		tszl		Rv		0	1		Pn			S		Pm			0		Pd		

Decode fields		Instruction Details				Feature			
S									
0		PSEL				FEAT_SME FEAT_SVE2p1			
1		UNALLOCATED				-			

SVE two-way dot product

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	0	0	0	Zm				1	1	0	0	1	1	Zn				Zda						

Decode fields	Instruction Details										Feature
U											
0	SDOT (2-way, vectors)										FEAT_SME2 FEAT_SVE2p1
1	UDOT (2-way, vectors)										FEAT_SME2 FEAT_SVE2p1

SVE two-way dot product (indexed)

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	1	0	0	opc				1	1	0	0	1	U	Zn				Zda						

Decode fields	Instruction Details										Feature
U											
0	SDOT (2-way, indexed)										FEAT_SME2 FEAT_SVE2p1
1	UDOT (2-way, indexed)										FEAT_SME2 FEAT_SVE2p1

SVE integer clamp

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	size	0	0	Zm				1	1	0	0	0	U	Zn				Zd						

Decode fields	Instruction Details										Feature
U											
0	SCLAMP										FEAT_SME FEAT_SVE2p1
1	UCLAMP										FEAT_SME FEAT_SVE2p1

SVE2 multiply-add (checked pointer)

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	opc	0	Zm				1	1	0	1	o2	0	Zn				Zda							

Decode fields	Instruction Details										Feature
opc	o2										
11	0	MLAPT								FEAT_SVE && FEAT_CPA	
11	1	MADPT								FEAT_SVE && FEAT_CPA	
!= 11		UNALLOCATED								-	

SVE2 absolute difference and accumulate long

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	size	0	0	Zm				1	1	0	1	U	1	Zn				Zda						

Decode fields	Instruction Details										Feature
U											
0	SABAL										FEAT_SVE2p3 FEAT_SME2p3
1	UABAL										FEAT_SVE2p3 FEAT_SME2p3

SVE permute vector elements (quadwords)

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	size	0	0	Zm				1	1	1	opc			Zn				Zd						

Decode fields opc	Instruction Details	Feature
000	ZIPQ1	FEAT_SVE2p1 FEAT_SME2p1
001	ZIPQ2	FEAT_SVE2p1 FEAT_SME2p1
010	UZPQ1	FEAT_SVE2p1 FEAT_SME2p1
011	UZPQ2	FEAT_SVE2p1 FEAT_SME2p1
10x	UNALLOCATED	-
110	TBLQ	FEAT_SVE2p1 FEAT_SME2p1
111	UNALLOCATED	-

SVE2 character match

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	1	size	1				Zm			1	0	0		Pg				Zn		op			Pd		

Decode fields op	Instruction Details	Feature
0	MATCH	FEAT_SVE2
1	NMATCH	FEAT_SVE2

SVE2 FP8 matrix multiply-accumulate

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	0	0	op	1	Zm					1	1	1	0	0	0	Zn					Zda				

Decode fields op	Instruction Details	Feature
0	FMMLA (widening, FP8 to FP32)	FEAT_SVE2 && FEAT_F8F32MM
1	FMMLA (widening, FP8 to FP16)	FEAT_SVE2 && FEAT_F8F16MM

SVE2 floating-point non-widening matrix-multiply accumulate

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	0	1	op	1	Zm					1	1	1	0	0	0	Zn					Zda				

Decode fields op	Instruction Details	Feature
0	FMMLA (non-widening)	FEAT_SVE2p2 && FEAT_F16MM
1	BFMMLA (non-widening)	FEAT_SVE_B16MM

SVE2 FP8 multiply-add long (indexed)

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	0	T	0	1	i4h		Zm		0	1	0	1		i4l			Zn					Zda			

Decode fields T	Instruction Details	Feature
0	FMLALB (indexed, FP8 to FP16)	FEAT_SSVE_FP8FMA (FEAT_SVE2 && FEAT_FP8FMA)
1	FMLALT (indexed, FP8 to FP16)	FEAT_SSVE_FP8FMA (FEAT_SVE2 && FEAT_FP8FMA)

SVE floating-point convert (top, predicated)

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	0	opc		0	0	0	0	opc2		1	0	1		Pg				Zn						Zd	

Decode fields		Instruction Details	Feature
opc	opc2		
x0	11	UNALLOCATED	-
00	0x	UNALLOCATED	-
00	10	FCVTXNT	FEAT_SVE2p2 FEAT_SME2p2
01		UNALLOCATED	-
10	00	FCVTNT (predicated) — Single-precision to half-precision, zeroing	FEAT_SVE2p2 FEAT_SME2p2
10	01	FCVTLT — Half-precision to single-precision, zeroing	FEAT_SVE2p2 FEAT_SME2p2
10	10	BFCVTNT	FEAT_SVE2p2 FEAT_SME2p2
11	0x	UNALLOCATED	-
11	10	FCVTNT (predicated) — Double-precision to single-precision, zeroing	FEAT_SVE2p2 FEAT_SME2p2
11	11	FCVTLT — Single-precision to double-precision, zeroing	FEAT_SVE2p2 FEAT_SME2p2

SVE floating-point convert precision odd elements

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	0	opc		0	0	1	0	opc2		1	0	1		Pg				Zn						Zd	

Decode fields		Instruction Details	Feature
opc	opc2		
x0	11	UNALLOCATED	-
00	0x	UNALLOCATED	-
00	10	FCVTXNT	FEAT_SVE2 FEAT_SME
01		UNALLOCATED	-
10	00	FCVTNT (predicated) — Single-precision to half-precision, merging	FEAT_SVE2 FEAT_SME
10	01	FCVTLT — Half-precision to single-precision, merging	FEAT_SVE2 FEAT_SME
10	10	BFCVTNT	(FEAT_SVE && FEAT_BF16) (FEAT_SME && FEAT_BF16)
11	0x	UNALLOCATED	-
11	10	FCVTNT (predicated) — Double-precision to single-precision, merging	FEAT_SVE2 FEAT_SME
11	11	FCVTLT — Single-precision to double-precision, merging	FEAT_SVE2 FEAT_SME

SVE2 floating-point pairwise operations

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	0			0	1	0		opc		1	0	0		Pg				Zm						Zdn	

Decode fields opc	Instruction Details	Feature
000	FADDP	FEAT_SVE2 FEAT_SME
001	UNALLOCATED	-
01x	UNALLOCATED	-
100	FMAXNMP	FEAT_SVE2 FEAT_SME
101	FMINNMP	FEAT_SVE2 FEAT_SME
110	FMAXP	FEAT_SVE2 FEAT_SME
111	FMINP	FEAT_SVE2 FEAT_SME

SVE floating-point recursive reduction (quadwords)

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	0	size	0	1	0	opc	1	0	1	Pg								Zn					Vd		

Decode fields opc	Instruction Details	Feature
000	FADDQV	FEAT_SVE2p1 FEAT_SME2p1
001	UNALLOCATED	-
01x	UNALLOCATED	-
100	FMAXNMQV	FEAT_SVE2p1 FEAT_SME2p1
101	FMINNMQV	FEAT_SVE2p1 FEAT_SME2p1
110	FMAXQV	FEAT_SVE2p1 FEAT_SME2p1
111	FMINQV	FEAT_SVE2p1 FEAT_SME2p1

SVE floating-point multiply-add (indexed)

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	0	size	1	opc	0	0	0	0	o2	op									Zn				Zda		

Decode fields size o2 op	Instruction Details	Feature
0x 0 0	FMLA (indexed) — Half-precision	FEAT_SVE FEAT_SME
0x 0 1	FMLS (indexed) — Half-precision	FEAT_SVE FEAT_SME
0x 1 0	BFMLA (indexed)	FEAT_SVE_B16B16
0x 1 1	BFMLS (indexed)	FEAT_SVE_B16B16
1x 1	UNALLOCATED	-
10 0 0	FMLA (indexed) — Single-precision	FEAT_SVE FEAT_SME
10 0 1	FMLS (indexed) — Single-precision	FEAT_SVE FEAT_SME
11 0 0	FMLA (indexed) — Double-precision	FEAT_SVE FEAT_SME
11 0 1	FMLS (indexed) — Double-precision	FEAT_SVE FEAT_SME

SVE floating-point complex multiply-add (indexed)

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	0	size	1	opc			0	0	0	1	rot		Zn			Zda									

Decode fields size	Instruction Details	Feature
0x	UNALLOCATED	-
10	FCMLA (indexed) — Half-precision	FEAT_SVE FEAT_SME
11	FCMLA (indexed) — Single-precision	FEAT_SVE FEAT_SME

SVE FP clamp

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	0	size	1				Zm			0	0	1	0	0	1			Zn					Zd		

Decode fields size	Instruction Details	Feature
00	BFCLAMP	FEAT_SVE_B16B16
!= 00	FCLAMP	FEAT_SME2 FEAT_SVE2p1

SVE floating-point multiply (indexed)

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	0	size	1				opc			0	0	1	0	o2	0			Zn					Zd		

Decode fields size	o2	Instruction Details	Feature
0x	0	FMUL (indexed) — Half-precision	FEAT_SVE FEAT_SME
0x	1	BFMUL (indexed)	FEAT_SVE_B16B16
1x	1	UNALLOCATED	-
10	0	FMUL (indexed) — Single-precision	FEAT_SVE FEAT_SME
11	0	FMUL (indexed) — Double-precision	FEAT_SVE FEAT_SME

SVE2 FP8 multiply-add long long (indexed)

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	0	TT	1	i4h			Zm			1	1	0	0	i4l				Zn					Zda		

Decode fields TT	Instruction Details	Feature
00	FMLALLBB (indexed)	FEAT_SSVE_FP8FMA (FEAT_SVE2 && FEAT_FP8FMA)
01	FMLALLBT (indexed)	FEAT_SSVE_FP8FMA (FEAT_SVE2 && FEAT_FP8FMA)
10	FMLALLTB (indexed)	FEAT_SSVE_FP8FMA (FEAT_SVE2 && FEAT_FP8FMA)
11	FMLALLTT (indexed)	FEAT_SSVE_FP8FMA (FEAT_SVE2 && FEAT_FP8FMA)

SVE floating-point matrix multiply accumulate

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	0	opc	1				Zm			1	1	1	0	0	1			Zn					Zda		

Decode fields	Instruction Details	Feature
opc		
00	FMMLA (widening, FP16 to FP32)	FEAT_SVE_F16F32MM
01	BFMMLA (widening)	FEAT_SVE && FEAT_BF16
10	FMMLA (non-widening) — 32-bit element	FEAT_F32MM
11	FMMLA (non-widening) — 64-bit element	FEAT_F64MM

SVE floating-point recursive reduction

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	1	size	0	0	0	opc	0	0	1	Pg	Zn			Vd											

Decode fields	Instruction Details	Feature
opc		
000	FADDV	FEAT_SVE FEAT_SME
001	UNALLOCATED	-
01x	UNALLOCATED	-
100	FMAXNMV	FEAT_SVE FEAT_SME
101	FMINNMV	FEAT_SVE FEAT_SME
110	FMAXV	FEAT_SVE FEAT_SME
111	FMINV	FEAT_SVE FEAT_SME

SVE floating-point arithmetic (unpredicated)

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	1	size	0	Zm			0	0	0	opc		Zn			Zd										

Decode fields	Instruction Details	Feature
size	opc	
	011	FTSMUL
	10x	UNALLOCATED
	110	FRECPS
	111	FRSQRTS
00	000	BFADD (unpredicated)
00	001	BFSUB (unpredicated)
00	010	BFMUL (vectors, unpredicated)
!= 00	000	FADD (vectors, unpredicated)
!= 00	001	FSUB (vectors, unpredicated)
!= 00	010	FMUL (vectors, unpredicated)

SVE floating-point compare vectors

The encodings in this section are decoded from [SVE encodings](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	1	size	0	Zm			op	1	o2	Pg		Zn			o3	Pd									

Decode fields			Instruction Details	Feature
op	o2	o3		
0	0	0	FCM<cc> (vectors) — Greater than or equal	FEAT_SVE FEAT_SME
0	0	1	FCM<cc> (vectors) — Greater than	FEAT_SVE FEAT_SME
0	1	0	FCM<cc> (vectors) — Equal	FEAT_SVE FEAT_SME
0	1	1	FCM<cc> (vectors) — Not equal	FEAT_SVE FEAT_SME
1	0	0	FCM<cc> (vectors) — Unordered	FEAT_SVE FEAT_SME
1	0	1	FAC<cc> — Greater than or equal	FEAT_SVE FEAT_SME
1	1	0	UNALLOCATED	-
1	1	1	FAC<cc> — Greater than	FEAT_SVE FEAT_SME

Data Processing -- Immediate

The encodings in this section are decoded from [A64 instruction set encoding](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	op0	1	0	0				op1																							

Decode fields		Instruction details
op0	op1	
11	111x	Data-processing (1 source immediate)
	00xx	PC-rel. addressing
	010x	Add/subtract (immediate)
	0110	Add/subtract (immediate, with tags)
	0111	Min/max (immediate)
	100x	Logical (immediate)
	101x	Move wide (immediate)
	110x	Bitfield
!= 11	111x	Extract

Data-processing (1 source immediate)

The encodings in this section are decoded from [Data Processing -- Immediate](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sf	1	1	1	0	0	1	1	1	opc		imm16															Rd					

Decode fields			Instruction Details	Feature
sf	opc	Rd		
0			UNALLOCATED	-
1	0x	!= 11111	UNALLOCATED	-
1	00	11111	AUTIASPPC	FEAT_PAuth_LR
1	01	11111	AUTIBSPPC	FEAT_PAuth_LR
1	1x		UNALLOCATED	-

PC-rel. addressing

The encodings in this section are decoded from [Data Processing -- Immediate](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
op	immlo	1	0	0	0	0	immhi															Rd									

Decode fields										Instruction Details									
op																			
0										ADR									
1										ADRP									

Add/subtract (immediate)

The encodings in this section are decoded from [Data Processing -- Immediate](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sf	op	S	1	0	0	0	1	0	sh	imm12												Rn				Rd					

Decode fields			Instruction Details
sf	op	S	
0	0	0	ADD (immediate) — 32-bit
0	0	1	ADDS (immediate) — 32-bit
0	1	0	SUB (immediate) — 32-bit
0	1	1	SUBS (immediate) — 32-bit
1	0	0	ADD (immediate) — 64-bit
1	0	1	ADDS (immediate) — 64-bit
1	1	0	SUB (immediate) — 64-bit
1	1	1	SUBS (immediate) — 64-bit

Add/subtract (immediate, with tags)

The encodings in this section are decoded from [Data Processing -- Immediate](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sf	op	S	1	0	0	0	1	1	0	imm6						op3		imm4				Rn				Rd					

Decode fields			Instruction Details	Feature
sf	op	S		
0			UNALLOCATED	-
1		1	UNALLOCATED	-
1	0	0	ADDG	FEAT_MTE
1	1	0	SUBG	FEAT_MTE

Min/max (immediate)

The encodings in this section are decoded from [Data Processing -- Immediate](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sf	op	S	1	0	0	0	1	1	1	opc				imm8						Rn				Rd							

Decode fields				Instruction Details	Feature
sf	op	S	opc		
	0	0	1xxx	UNALLOCATED	
	0	0	!= 00xx01xx	UNALLOCATED	-
	0	1		UNALLOCATED	-
	1			UNALLOCATED	-
0	0	0	0000	SMAX (immediate) — 32-bit	FEAT_CSSC
0	0	0	0001	UMAX (immediate) — 32-bit	FEAT_CSSC
0	0	0	0010	SMIN (immediate) — 32-bit	FEAT_CSSC
0	0	0	0011	UMIN (immediate) — 32-bit	FEAT_CSSC
1	0	0	0000	SMAX (immediate) — 64-bit	FEAT_CSSC
1	0	0	0001	UMAX (immediate) — 64-bit	FEAT_CSSC
1	0	0	0010	SMIN (immediate) — 64-bit	FEAT_CSSC
1	0	0	0011	UMIN (immediate) — 64-bit	FEAT_CSSC

Logical (immediate)

The encodings in this section are decoded from [Data Processing -- Immediate](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
sf	opc	1	0	0	1	0	0	N	immr								imms								Rn				Rd			

Decode fields			Instruction Details
sf	opc	N	
0		1	UNALLOCATED
0	00	0	AND (immediate) — 32-bit
0	01	0	ORR (immediate) — 32-bit
0	10	0	EOR (immediate) — 32-bit
0	11	0	ANDS (immediate) — 32-bit
1	00		AND (immediate) — 64-bit
1	01		ORR (immediate) — 64-bit
1	10		EOR (immediate) — 64-bit
1	11		ANDS (immediate) — 64-bit

Move wide (immediate)

The encodings in this section are decoded from [Data Processing -- Immediate](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sf		opc		1	0	0	1	0	1	hw		imm16																Rd			

Decode fields			Instruction Details
sf	opc	hw	
	01	0x	UNALLOCATED
0		1x	UNALLOCATED
0	00	0x	MOVN — 32-bit
0	10	0x	MOVZ — 32-bit
0	11	0x	MOVK — 32-bit
1	00		MOVN — 64-bit
1	01	1x	UNALLOCATED
1	10		MOVZ — 64-bit
1	11		MOVK — 64-bit

Bitfield

The encodings in this section are decoded from [Data Processing -- Immediate](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sf	opc		1	0	0	1	1	0	N	immr						imms						Rn				Rd					

Decode fields									Instruction Details							
sf	opc		N													
0			1						UNALLOCATED							
0	00		0						SBFM — 32-bit							
0	01		0						BFM — 32-bit							
0	10		0						UBFM — 32-bit							
0	11		0						UNALLOCATED							
1			0						UNALLOCATED							
1	00		1						SBFM — 64-bit							
1	01		1						BFM — 64-bit							
1	10		1						UBFM — 64-bit							
1	11		1						UNALLOCATED							

Extract

The encodings in this section are decoded from [Data Processing -- Immediate](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sf	!= 11		1	0	0	1	1	1	N	o0	Rm						imms						Rn				Rd				

op21

The following constraints also apply to this encoding: op21 != '11'

Decode fields									Instruction Details							
sf	op21		N						imms							
	00								1							
	01															
	10															
0	00		0	0						0xxxxxx						EXTR — 32-bit
0	00		0	0						1xxxxxx						UNALLOCATED
0	00		1	0												UNALLOCATED
1	00		0	0												UNALLOCATED
1	00		1	0												EXTR — 64-bit

Branches, Exception Generating and System instructions

The encodings in this section are decoded from [A64 instruction set encoding](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
op0			1	0	1	op1																			op2						

Decode fields		Instruction details	
op0	op1	op2	
010	00xxxxxxxxxxxx		Conditional branch (immediate)
010	01xxxxxxxxxxxx		Miscellaneous branch (immediate)
011	00xxxxxxxx1xxx		Compare bytes/halfwords in registers and branch
01x	1xxxxxxxxxxxxx		UNALLOCATED
110	00xxxxxxxxxxxx		Exception generation
110	010000000x00xx		UNALLOCATED
110	010000001000xx		UNALLOCATED
110	01000000110000		UNALLOCATED
110	01000000110001		System instructions with register argument
110	01000000110010	11111	Hints
110	01000000110010	!= 11111	UNALLOCATED
110	01000000110011		Barriers
110	01000001xx00xx		UNALLOCATED
110	0100000xxx0100		PSTATE
110	0100000xxx0101		UNALLOCATED
110	0100000xxx011x		UNALLOCATED
110	0100000xxx1xxx		UNALLOCATED
110	0100100xxxxxxx		UNALLOCATED
110	0100x01xxxxxxx		System instructions
110	0100x1xxxxxxx		System register move
110	0101x00xxxxxxx		UNALLOCATED
110	0101x01xxxxxxx		System pair instructions
110	0101x1xxxxxxx		System register pair move
110	011xxxxxxxxxxx		UNALLOCATED
110	1xxxxxxxxxxxxx		Unconditional branch (register)
111	00xxxxxxxx1xxx		UNALLOCATED
111	1xxxxxxxxxxxxx		UNALLOCATED
x00			Unconditional branch (immediate)
x01	0xxxxxxxxxxxxx		Compare and branch (immediate)
x01	1xxxxxxxxxxxxx		Test and branch (immediate)
x11	00xxxxxxxx00xx		Compare registers and branch
x11	00xxxxxxxx01xx		UNALLOCATED
x11	01xxxxxxxx00xx		Compare register with immediate and branch
x11	01xxxxxxxx1xxx		UNALLOCATED

Conditional branch (immediate)

The encodings in this section are decoded from [Branches, Exception Generating and System instructions](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	0	1	0	0	imm19																o0	cond						

Decode fields	Instruction Details		Feature
o0			
0	B.cond		-
1	BC.cond		FEAT_HBC

Miscellaneous branch (immediate)

The encodings in this section are decoded from [Branches, Exception Generating and System instructions](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	0	1	0	1	opc				imm16																op2			

Decode fields		Instruction Details		Feature
opc	op2			
00x	!= 11111	UNALLOCATED		-
000	11111	RETAASPPC, RETABSPPC — RETAASPPC		FEAT_PAAuth_LR
001	11111	RETAASPPC, RETABSPPC — RETABSPPC		FEAT_PAAuth_LR
1xx		UNALLOCATED		
!= 00x01x		UNALLOCATED		-

Compare bytes/halfwords in registers and branch

The encodings in this section are decoded from [Branches, Exception Generating and System instructions](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	1	0	1	0	0	cc			Rm				1	H	imm9								Rt						

Decode fields		Instruction Details		Feature
cc	H			
000	0	CBB<cc> — Greater than		FEAT_CMPBR
000	1	CBH<cc> — Greater than		FEAT_CMPBR
001	0	CBB<cc> — Greater than or equal		FEAT_CMPBR
001	1	CBH<cc> — Greater than or equal		FEAT_CMPBR
010	0	CBB<cc> — Higher		FEAT_CMPBR
010	1	CBH<cc> — Higher		FEAT_CMPBR
011	0	CBB<cc> — Higher or same		FEAT_CMPBR
011	1	CBH<cc> — Higher or same		FEAT_CMPBR
10x		UNALLOCATED		-
110	0	CBB<cc> — Equal		FEAT_CMPBR
110	1	CBH<cc> — Equal		FEAT_CMPBR
111	0	CBB<cc> — Not equal		FEAT_CMPBR
111	1	CBH<cc> — Not equal		FEAT_CMPBR

Exception generation

The encodings in this section are decoded from [Branches, Exception Generating and System instructions](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	1	0	1	0	0	opc				imm16																op2		LL	

opc	Decode fields op2	LL	Instruction Details
	!= 000		UNALLOCATED
x11	000		UNALLOCATED
000	000	00	UNALLOCATED
000	000	01	SVC
000	000	10	HVC
000	000	11	SMC
001	000	00	BRK
001	000	!= 00	UNALLOCATED
010	000	00	HLT
010	000	!= 00	UNALLOCATED
1x0	000		UNALLOCATED
101	000	00	UNALLOCATED
101	000	01	DCPS1
101	000	10	DCPS2
101	000	11	DCPS3

System instructions with register argument

The encodings in this section are decoded from [Branches, Exception Generating and System instructions](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	1	0	1	0	1	0	0	0	0	0	0	1	1	0	0	0	1	CRm			op2			Rt					

CRm	Decode fields op2	Instruction Details	Feature
0000	000	WFET	FEAT_WFxT
0000	001	WFIT	FEAT_WFxT
0000	1xx	UNALLOCATED	
0000	!= 00x01x	UNALLOCATED	-
!= 0000		UNALLOCATED	-

Hints

The encodings in this section are decoded from [Branches, Exception Generating and System instructions](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	1	0	1	0	1	0	0	0	0	0	0	1	1	0	0	1	0	CRm				op2			1	1	1	1	1

Decode fields		Instruction Details	Feature
CRm	op2		
		HINT	-
0000	000	NOP	-
0000	001	YIELD	-
0000	010	WFE	-
0000	011	WFI	-
0000	100	SEV	-
0000	101	SEVL	-
0000	110	DGH	FEAT_DGH
0000	111	XPACD, XPACI, XPACLRI	FEAT_PAuth
0001	000	PACIA, PACIA1716, PACIASP, PACIAZ, PACIZA — PACIA1716	FEAT_PAuth
0001	010	PACIB, PACIB1716, PACIBSP, PACIBZ, PACIZB — PACIB1716	FEAT_PAuth
0001	100	AUTIA, AUTIA1716, AUTIASP, AUTIAZ, AUTIZA — AUTIA1716	FEAT_PAuth
0001	110	AUTIB, AUTIB1716, AUTIBSP, AUTIBZ, AUTIZB — AUTIB1716	FEAT_PAuth
0010	000	ESB	FEAT_RAS
0010	001	PSB	FEAT_SPE
0010	010	TSB	FEAT_TRF
0010	011	GCSB	FEAT_GCS
0010	100	CSDB	-
0010	110	CLRBHB	FEAT_CLRBHB
0011	000	PACIA, PACIA1716, PACIASP, PACIAZ, PACIZA — PACIAZ	FEAT_PAuth
0011	001	PACIA, PACIA1716, PACIASP, PACIAZ, PACIZA — PACIASP	FEAT_PAuth
0011	010	PACIB, PACIB1716, PACIBSP, PACIBZ, PACIZB — PACIBZ	FEAT_PAuth
0011	011	PACIB, PACIB1716, PACIBSP, PACIBZ, PACIZB — PACIBSP	FEAT_PAuth
0011	100	AUTIA, AUTIA1716, AUTIASP, AUTIAZ, AUTIZA — AUTIAZ	FEAT_PAuth
0011	101	AUTIA, AUTIA1716, AUTIASP, AUTIAZ, AUTIZA — AUTIASP	FEAT_PAuth
0011	110	AUTIB, AUTIB1716, AUTIBSP, AUTIBZ, AUTIZB — AUTIBZ	FEAT_PAuth
0011	111	AUTIB, AUTIB1716, AUTIBSP, AUTIBZ, AUTIZB — AUTIBSP	FEAT_PAuth
0100	xx0	BTI	FEAT_BTI
0100	111	PACM	FEAT_PAuth_LR
0101	000	CHKFEAT	FEAT_CHK
0110	00x	STSHH	FEAT_PCDPHINT
0110	01x	SHUH	FEAT_CMH
0110	100	STCPH	FEAT_CMH

Barriers

The encodings in this section are decoded from [Branches, Exception Generating and System instructions](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	1	0	1	0	1	0	0	0	0	0	0	1	1	0	0	1	1	CRm			op2			Rt					

Decode fields		Instruction Details			Feature
CRm	op2	Rt			
		!= 11111	UNALLOCATED		-
	000	11111	UNALLOCATED		-
	010	11111	CLREX		-
	100	11111	DSB — Memory barrier		-
	101	11111	DMB		-
	110	11111	ISB		-
	111	11111	SB		FEAT_SB
xx0x	0x1	11111	UNALLOCATED		-
xx10	001	11111	DSB — Memory nXS barrier		FEAT_XS
xx10	011	11111	UNALLOCATED		-
!= xx10xx11	0x1	11111	UNALLOCATED		-

PSTATE

The encodings in this section are decoded from [Branches, Exception Generating and System instructions](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	1	0	1	0	1	0	0	0	0	0	op1			0	1	0	0	CRm			op2			Rt					

Decode fields		Instruction Details			Feature
op1	op2	Rt			
		!= 11111	UNALLOCATED		-
000	000	11111	CFINV		FEAT_FlagM
000	001	11111	XAFLAG		FEAT_FlagM2
000	010	11111	AXFLAG		FEAT_FlagM2
000	011	11111	MSR (immediate)		-
000	1xx	11111	MSR (immediate)		-
!= 000		11111	MSR (immediate)		-

Decode fields		Instruction Details			Feature
op1:op2	Rt				
	0xxxx	UNALLOCATED		-	
	10xxx	UNALLOCATED		-	
	110xx	UNALLOCATED		-	
	1110x	UNALLOCATED		-	
	11110	UNALLOCATED		-	
000000	11111	CFINV		FEAT_FlagM	
000001	11111	XAFLAG		FEAT_FlagM2	
000010	11111	AXFLAG		FEAT_FlagM2	
!= 00000x	11111	MSR (immediate)		-	

System instructions

The encodings in this section are decoded from [Branches, Exception Generating and System instructions](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	1	0	1	0	1	0	0	L	0	1	op1			CRn			CRm			op2			Rt						

Decode fields		Instruction Details
L		
0	SYS	
1	SYSL	

System register move

The encodings in this section are decoded from [Branches, Exception Generating and System instructions](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	1	0	1	0	1	0	0	L	1	o0	op1			CRn			CRm			op2			Rt						

Decode fields

L

Instruction Details

0	MSR (register)
1	MRS

System pair instructions

The encodings in this section are decoded from [Branches, Exception Generating and System instructions](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	1	0	1	0	1	0	1	L	0	1	op1			CRn			CRm			op2			Rt						

Decode fields

L

Instruction Details

Feature

0	SYSP	FEAT_SYSINSTR128
1	UNALLOCATED	-

System register pair move

The encodings in this section are decoded from [Branches, Exception Generating and System instructions](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	1	0	1	0	1	0	1	L	1	o0	op1			CRn			CRm			op2			Rt						

Decode fields

L

Instruction Details

Feature

0	MSRR	FEAT_SYSREG128
1	MRRS	FEAT_SYSREG128

Unconditional branch (register)

The encodings in this section are decoded from [Branches, Exception Generating and System instructions](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	1	0	1	1	opc				op2				op3				Rn				op4								

opc	op2	Decode fields op3	Rn	op4	Instruction Details	Feature
	11111	001xxx			UNALLOCATED	-
	11111	01xxxx			UNALLOCATED	-
	11111	1xxxxxx			UNALLOCATED	-
	11111	!= 0000xx0001xx			UNALLOCATED	-
	!= 11111				UNALLOCATED	-
00x0	11111	000000		!= 00000	UNALLOCATED	-
00x0	11111	000001			UNALLOCATED	-
000x	11111	00001x		!= 11111	UNALLOCATED	-
0000	11111	000000		00000	BR	-
0000	11111	000010		11111	BRAA, BRAAZ, BRAB, BRABZ — Key A, zero modifier	FEAT_PAuth
0000	11111	000011		11111	BRAA, BRAAZ, BRAB, BRABZ — Key B, zero modifier	FEAT_PAuth
0001	11111	000000		00000	BLR	-
0001	11111	000000		!= 00000	UNALLOCATED	-
0001	11111	000001			UNALLOCATED	-
0001	11111	000010		11111	BLRAA, BLRAAZ, BLRAB, BLRABZ — Key A, zero modifier	FEAT_PAuth
0001	11111	000011		11111	BLRAA, BLRAAZ, BLRAB, BLRABZ — Key B, zero modifier	FEAT_PAuth
0010	11111	000000		00000	RET	-
0010	11111	00001x	!= 11111		UNALLOCATED	-
0010	11111	000010	11111	11111	RETAA, RETAB — RETAA	FEAT_PAuth
0010	11111	000010	11111	!= 11111	RETAASPPCR, RETABSPPCR — RETAASPPCR	FEAT_PAuth_LR
0010	11111	000011	11111	11111	RETAA, RETAB — RETAB	FEAT_PAuth
0010	11111	000011	11111	!= 11111	RETAASPPCR, RETABSPPCR — RETABSPPCR	FEAT_PAuth_LR
0011	11111	0000xx			UNALLOCATED	-
010x	11111	0000xx	!= 11111		UNALLOCATED	-
010x	11111	000000	11111	!= 00000	UNALLOCATED	-
010x	11111	000001	11111		UNALLOCATED	-
010x	11111	00001x	11111	xxxx0	UNALLOCATED	-
0100	11111	000000	11111	00000	ERET	-
0100	11111	00001x	11111	0xxx1	UNALLOCATED	-
0100	11111	00001x	11111	10xx1	UNALLOCATED	-
0100	11111	00001x	11111	110x1	UNALLOCATED	-
0100	11111	00001x	11111	11101	UNALLOCATED	-
0100	11111	000010	11111	11111	ERETAA, ERETAB — ERETAA	FEAT_PAuth
0100	11111	000011	11111	11111	ERETAA, ERETAB — ERETAB	FEAT_PAuth
0101	11111	000000	11111	00000	DRPS	-

Decode fields					Instruction Details	Feature
opc	op2	op3	Rn	op4		
0101	11111	00001x	11111	xxxx1	UNALLOCATED	-
011x	11111	0000xx			UNALLOCATED	-
100x	11111	00000x			UNALLOCATED	-
1000	11111	000010			BRAA, BRAAZ, BRAB, BRABZ — Key A, register modifier	FEAT_PAuth
1000	11111	000011			BRAA, BRAAZ, BRAB, BRABZ — Key B, register modifier	FEAT_PAuth
1001	11111	000010			BLRAA, BLRAAZ, BLRAB, BLRABZ — Key A, register modifier	FEAT_PAuth
1001	11111	000011			BLRAA, BLRAAZ, BLRAB, BLRABZ — Key B, register modifier	FEAT_PAuth
101x	11111	0000xx			UNALLOCATED	-
11xx	11111	0000xx			UNALLOCATED	-

Unconditional branch (immediate)

The encodings in this section are decoded from [Branches, Exception Generating and System instructions](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
op	0	0	1	0	1	imm26																									

Decode fields		Instruction Details	
op			
0		B	
1		BL	

Compare and branch (immediate)

The encodings in this section are decoded from [Branches, Exception Generating and System instructions](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sf	0	1	1	0	1	0	op	imm19																Rt							

Decode fields		Instruction Details	
sf	op		
0	0	CBZ — 32-bit	
0	1	CBNZ — 32-bit	
1	0	CBZ — 64-bit	
1	1	CBNZ — 64-bit	

Test and branch (immediate)

The encodings in this section are decoded from [Branches, Exception Generating and System instructions](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
b5	0	1	1	0	1	1	op	b40					imm14										Rt								

Decode fields		Instruction Details	
op			
0		TBZ	
1		TBNZ	

Compare registers and branch

The encodings in this section are decoded from [Branches, Exception Generating and System instructions](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sf	1	1	1	0	1	0	0	cc			Rm				0	0	imm9						Rt								

Decode fields		Instruction Details	Feature
sf	cc		
	10x	UNALLOCATED	-
0	000	CB<cc> (register) — 32-bit greater than	FEAT_CMPBR
0	001	CB<cc> (register) — 32-bit greater than or equal	FEAT_CMPBR
0	010	CB<cc> (register) — 32-bit higher	FEAT_CMPBR
0	011	CB<cc> (register) — 32-bit higher or same	FEAT_CMPBR
0	110	CB<cc> (register) — 32-bit equal	FEAT_CMPBR
0	111	CB<cc> (register) — 32-bit not equal	FEAT_CMPBR
1	000	CB<cc> (register) — 64-bit greater than	FEAT_CMPBR
1	001	CB<cc> (register) — 64-bit greater than or equal	FEAT_CMPBR
1	010	CB<cc> (register) — 64-bit higher	FEAT_CMPBR
1	011	CB<cc> (register) — 64-bit higher or same	FEAT_CMPBR
1	110	CB<cc> (register) — 64-bit equal	FEAT_CMPBR
1	111	CB<cc> (register) — 64-bit not equal	FEAT_CMPBR

Compare register with immediate and branch

The encodings in this section are decoded from [Branches, Exception Generating and System instructions](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sf	1	1	1	0	1	0	1	cc			imm6						0	imm9							Rt						

Decode fields		Instruction Details	Feature
sf	cc		
	10x	UNALLOCATED	-
0	000	CB<cc> (immediate) — 32-bit greater than	FEAT_CMPBR
0	001	CB<cc> (immediate) — 32-bit less than	FEAT_CMPBR
0	010	CB<cc> (immediate) — 32-bit higher	FEAT_CMPBR
0	011	CB<cc> (immediate) — 32-bit lower	FEAT_CMPBR
0	110	CB<cc> (immediate) — 32-bit equal	FEAT_CMPBR
0	111	CB<cc> (immediate) — 32-bit not equal	FEAT_CMPBR
1	000	CB<cc> (immediate) — 64-bit greater than	FEAT_CMPBR
1	001	CB<cc> (immediate) — 64-bit less than	FEAT_CMPBR
1	010	CB<cc> (immediate) — 64-bit higher	FEAT_CMPBR
1	011	CB<cc> (immediate) — 64-bit lower	FEAT_CMPBR
1	110	CB<cc> (immediate) — 64-bit equal	FEAT_CMPBR
1	111	CB<cc> (immediate) — 64-bit not equal	FEAT_CMPBR

Data Processing -- Register

The encodings in this section are decoded from [A64 instruction set encoding](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
op0			op1		1	0	1	op2				op3																			

		Decode fields		Instruction details
op0	op1	op2	op3	
0	1	0110		Data-processing (2 source)
1	1	0110		Data-processing (1 source)
	0	0xxx		Logical (shifted register)
	0	1xx0		Add/subtract (shifted register)
	0	1xx1		Add/subtract (extended register)
	1	0000	000000	Add/subtract (with carry)
	1	0000	001xxx	Add/subtract (checked pointer)
	1	0000	011xxx	UNALLOCATED
	1	0000	100000	UNALLOCATED
	1	0000	1x1xxx	UNALLOCATED
	1	0000	x00001	Rotate right into flags
	1	0000	x0010x	UNALLOCATED
	1	0000	x10x0x	UNALLOCATED
	1	0000	xx0010	Evaluate into flags
	1	0000	xx0011	UNALLOCATED
	1	0000	xx011x	UNALLOCATED
	1	0010	xxxx0x	Conditional compare (register)
	1	0010	xxxx1x	Conditional compare (immediate)
	1	0100		Conditional select
	1	0xx1		UNALLOCATED
	1	1xxx		Data-processing (3 source)

Data-processing (2 source)

The encodings in this section are decoded from [Data Processing -- Register](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sf	0	S	1	1	0	1	0	1	1	0	Rm				opcode				Rn				Rd								

sf	Decode fields		Instruction Details	Feature
	S	opcode		
		1xxxxx	UNALLOCATED	-
	0	00011x	UNALLOCATED	-
	1	00001x	UNALLOCATED	-
	1	0001xx	UNALLOCATED	-
	1	001xxx	UNALLOCATED	-
	1	01xxxx	UNALLOCATED	-
0		00000x	UNALLOCATED	-
0	0	0x11xx	UNALLOCATED	-
0	0	000010	UDIV — 32-bit	-
0	0	000011	SDIV — 32-bit	-
0	0	00010x	UNALLOCATED	-
0	0	001000	LSLV — 32-bit	-
0	0	001001	LSRV — 32-bit	-
0	0	001010	ASRV — 32-bit	-
0	0	001011	RORV — 32-bit	-
0	0	010x11	UNALLOCATED	-
0	0	010000	CRC32B, CRC32H, CRC32W, CRC32X — CRC32B	FEAT_CRC32
0	0	010001	CRC32B, CRC32H, CRC32W, CRC32X — CRC32H	FEAT_CRC32
0	0	010010	CRC32B, CRC32H, CRC32W, CRC32X — CRC32W	FEAT_CRC32
0	0	010100	CRC32CB, CRC32CH, CRC32CW, CRC32CX — CRC32CB	FEAT_CRC32
0	0	010101	CRC32CB, CRC32CH, CRC32CW, CRC32CX — CRC32CH	FEAT_CRC32
0	0	010110	CRC32CB, CRC32CH, CRC32CW, CRC32CX — CRC32CW	FEAT_CRC32
0	0	011000	SMAX (register) — 32-bit	FEAT_CSSC
0	0	011001	UMAX (register) — 32-bit	FEAT_CSSC
0	0	011010	SMIN (register) — 32-bit	FEAT_CSSC
0	0	011011	UMIN (register) — 32-bit	FEAT_CSSC
1		000001	UNALLOCATED	-
1	0	000000	SUBP	FEAT_MTE
1	0	000010	UDIV — 64-bit	-
1	0	000011	SDIV — 64-bit	-
1	0	000100	IRG	FEAT_MTE
1	0	000101	GMI	FEAT_MTE
1	0	001000	LSLV — 64-bit	-
1	0	001001	LSRV — 64-bit	-
1	0	001010	ASRV — 64-bit	-
1	0	001011	RORV — 64-bit	-
1	0	001100	PACGA	FEAT_PAuth
1	0	001101	UNALLOCATED	-
1	0	00111x	UNALLOCATED	-
1	0	010xx0	UNALLOCATED	-
1	0	010001	UNALLOCATED	-
1	0	010011	CRC32B, CRC32H, CRC32W, CRC32X — CRC32X	FEAT_CRC32
1	0	010101	UNALLOCATED	-
1	0	010111	CRC32CB, CRC32CH, CRC32CW, CRC32CX — CRC32CX	FEAT_CRC32
1	0	011000	SMAX (register) — 64-bit	FEAT_CSSC

Decode fields			Instruction Details	Feature
sf	S	opcode		
1	0	011001	UMAX (register) — 64-bit	FEAT_CSSC
1	0	011010	SMIN (register) — 64-bit	FEAT_CSSC
1	0	011011	UMIN (register) — 64-bit	FEAT_CSSC
1	0	0111xx	UNALLOCATED	-
1	1	000000	SUBPS	FEAT_MTE

Data-processing (1 source)

The encodings in this section are decoded from [Data Processing -- Register](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sf	1	S	1	1	0	1	0	1	1	0	opcode2				opcode				Rn				Rd								

|

		Decode fields				Instruction Details	Feature
sf	S	opcode2	opcode	Rn	Rd		
	0	00000	001001			UNALLOCATED	-
	0	00000	00101x			UNALLOCATED	-
	0	00000	0011xx			UNALLOCATED	-
	0	00000	01xxxx			UNALLOCATED	-
	0	00000	1xxxxx			UNALLOCATED	-
	0	001xx				UNALLOCATED	-
	0	01xxx				UNALLOCATED	-
	0	1xxxx				UNALLOCATED	-
	0	!= 0000x0001x				UNALLOCATED	-
	1					UNALLOCATED	-
0	0	00000	000000			RBIT — 32-bit	-
0	0	00000	000001			REV16 — 32-bit	-
0	0	00000	000010			REV — 32-bit	-
0	0	00000	000011			UNALLOCATED	-
0	0	00000	000100			CLZ — 32-bit	-
0	0	00000	000101			CLS — 32-bit	-
0	0	00000	000110			CTZ — 32-bit	FEAT_CSSC
0	0	00000	000111			CNT — 32-bit	FEAT_CSSC
0	0	00000	001000			ABS — 32-bit	FEAT_CSSC
0	0	00001				UNALLOCATED	-
1	0	00000	000000			RBIT — 64-bit	-
1	0	00000	000001			REV16 — 64-bit	-
1	0	00000	000010			REV32	-
1	0	00000	000011			REV — 64-bit	-
1	0	00000	000100			CLZ — 64-bit	-
1	0	00000	000101			CLS — 64-bit	-
1	0	00000	000110			CTZ — 64-bit	FEAT_CSSC
1	0	00000	000111			CNT — 64-bit	FEAT_CSSC
1	0	00000	001000			ABS — 64-bit	FEAT_CSSC
1	0	00001	000000			PACIA, PACIA1716, PACIASP, PACIAZ, PACIZA — PACIA	FEAT_PAuth
1	0	00001	000001			PACIB, PACIB1716, PACIBSP, PACIBZ, PACIZB — PACIB	FEAT_PAuth
1	0	00001	000010			PACDA, PACDZA — PACDA	FEAT_PAuth
1	0	00001	000011			PACDB, PACDZB — PACDB	FEAT_PAuth
1	0	00001	000100			AUTIA, AUTIA1716, AUTIASP, AUTIAZ, AUTIZA — AUTIA	FEAT_PAuth
1	0	00001	000101			AUTIB, AUTIB1716, AUTIBSP, AUTIBZ, AUTIZB — AUTIB	FEAT_PAuth
1	0	00001	000110			AUTDA, AUTDZA — AUTDA	FEAT_PAuth
1	0	00001	000111			AUTDB, AUTDZB — AUTDB	FEAT_PAuth
1	0	00001	001xxx	!= 11111		UNALLOCATED	-
1	0	00001	001000	11111		PACIA, PACIA1716, PACIASP, PACIAZ, PACIZA — PACIZA	FEAT_PAuth
1	0	00001	001001	11111		PACIB, PACIB1716, PACIBSP, PACIBZ, PACIZB — PACIZB	FEAT_PAuth
1	0	00001	001010	11111		PACDA, PACDZA — PACDZA	FEAT_PAuth

		Decode fields				Instruction Details		Feature
sf	S	opcode2	opcode	Rn	Rd			
1	0	00001	001011	11111		PACDB, PACDZB — PACDZB		FEAT_PAuth
1	0	00001	001100	11111		AUTIA, AUTIA1716, AUTIASP, AUTIAZ, AUTIZA — AUTIZA		FEAT_PAuth
1	0	00001	001101	11111		AUTIB, AUTIB1716, AUTIBSP, AUTIBZ, AUTIZB — AUTIZB		FEAT_PAuth
1	0	00001	001110	11111		AUTDA, AUTDZA — AUTDZA		FEAT_PAuth
1	0	00001	001111	11111		AUTDB, AUTDZB — AUTDZB		FEAT_PAuth
1	0	00001	01000x	!= 11111		UNALLOCATED		-
1	0	00001	010000	11111		XPACD, XPACI, XPACLRI — XPACI		FEAT_PAuth
1	0	00001	010001	11111		XPACD, XPACI, XPACLRI — XPACD		FEAT_PAuth
1	0	00001	01001x			UNALLOCATED		-
1	0	00001	0101xx			UNALLOCATED		-
1	0	00001	011xxx			UNALLOCATED		-
1	0	00001	10xxxx		!= 11110	UNALLOCATED		-
1	0	00001	1000xx	!= 11111	11110	UNALLOCATED		-
1	0	00001	100000	11111	11110	PACNBIASPPC		FEAT_PAuth_LR
1	0	00001	100001	11111	11110	PACNBIBSPPC		FEAT_PAuth_LR
1	0	00001	100010	11111	11110	PACIA171615		FEAT_PAuth_LR
1	0	00001	100011	11111	11110	PACIB171615		FEAT_PAuth_LR
1	0	00001	100100		11110	AUTIASPPCR		FEAT_PAuth_LR
1	0	00001	100101		11110	AUTIBSPPCR		FEAT_PAuth_LR
1	0	00001	10011x		11110	UNALLOCATED		-
1	0	00001	101xxx	!= 11111	11110	UNALLOCATED		-
1	0	00001	101000	11111	11110	PACIASPPC		FEAT_PAuth_LR
1	0	00001	101001	11111	11110	PACIBSPPC		FEAT_PAuth_LR
1	0	00001	10101x	11111	11110	UNALLOCATED		-
1	0	00001	10110x	11111	11110	UNALLOCATED		-
1	0	00001	101110	11111	11110	AUTIA171615		FEAT_PAuth_LR
1	0	00001	101111	11111	11110	AUTIB171615		FEAT_PAuth_LR
1	0	00001	11xxxx			UNALLOCATED		-

Logical (shifted register)

The encodings in this section are decoded from [Data Processing -- Register](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sf	opc	0	1	0	1	0	shift	N	Rm						imm6						Rn						Rd				

Decode fields				Instruction Details
sf	op	S	opt	
			!= 00	UNALLOCATED
0	0	0	00	ADD (extended register) — 32-bit
0	0	1	00	ADDS (extended register) — 32-bit
0	1	0	00	SUB (extended register) — 32-bit
0	1	1	00	SUBS (extended register) — 32-bit
1	0	0	00	ADD (extended register) — 64-bit
1	0	1	00	ADDS (extended register) — 64-bit
1	1	0	00	SUB (extended register) — 64-bit
1	1	1	00	SUBS (extended register) — 64-bit

Add/subtract (with carry)

The encodings in this section are decoded from [Data Processing -- Register](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sf	op	S	1	1	0	1	0	0	0	0	Rm				0	0	0	0	0	0	Rn				Rd						

Decode fields			Instruction Details
sf	op	S	
0	0	0	ADC — 32-bit
0	0	1	ADCS — 32-bit
0	1	0	SBC — 32-bit
0	1	1	SBCS — 32-bit
1	0	0	ADC — 64-bit
1	0	1	ADCS — 64-bit
1	1	0	SBC — 64-bit
1	1	1	SBCS — 64-bit

Add/subtract (checked pointer)

The encodings in this section are decoded from [Data Processing -- Register](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sf	op	S	1	1	0	1	0	0	0	0	Rm				0	0	1	imm3			Rn				Rd						

Decode fields			Instruction Details	Feature
sf	op	S		
0			UNALLOCATED	-
1		1	UNALLOCATED	-
1	0	0	ADDPT	FEAT_CPA
1	1	0	SUBPT	FEAT_CPA

Rotate right into flags

The encodings in this section are decoded from [Data Processing -- Register](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sf	op	S	1	1	0	1	0	0	0	0	imm6				0	0	0	0	1	Rn				o2	mask						

Decode fields				Instruction Details	Feature
sf	op	S	o2		
0				UNALLOCATED	-
1	0	0		UNALLOCATED	-
1	0	1	0	RMIF	FEAT_FlagM
1	0	1	1	UNALLOCATED	-
1	1			UNALLOCATED	-

Evaluate into flags

The encodings in this section are decoded from [Data Processing -- Register](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sf	op	S	1	1	0	1	0	0	0	0	opcode2					sz	0	0	1	0	Rn					o3	mask				

Decode fields				Instruction Details	Feature
sf	op	S	opcode2		
0	0	0		UNALLOCATED	-
0	0	1	000000	UNALLOCATED	-
0	0	1	000000	UNALLOCATED	-
0	0	1	000000	0 0 1101	SETF8, SETF16 — SETF8
0	0	1	000000	1 0 1101	SETF8, SETF16 — SETF16
0	0	1	!= 000000	UNALLOCATED	-
0	1			UNALLOCATED	-
1				UNALLOCATED	-

Conditional compare (register)

The encodings in this section are decoded from [Data Processing -- Register](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
sf	op	S	1	1	0	1	0	0	1	0	Rm					cond					0	o2	Rn					o3	nzcvc			

Decode fields				Instruction Details
sf	op	S	o2	
		0		UNALLOCATED
		1	0	1
		1	1	UNALLOCATED
0	0	1	0	0
0	1	1	0	0
1	0	1	0	0
1	1	1	0	0

Conditional compare (immediate)

The encodings in this section are decoded from [Data Processing -- Register](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
sf	op	S	1	1	0	1	0	0	1	0	imm5					cond					1	o2	Rn					o3	nzcvc			

Decode fields					Instruction Details
sf	op	S	o2	o3	
		0			UNALLOCATED
		1	0	1	UNALLOCATED
		1	1		UNALLOCATED
0	0	1	0	0	CCMN (immediate) — 32-bit
0	1	1	0	0	CCMP (immediate) — 32-bit
1	0	1	0	0	CCMN (immediate) — 64-bit
1	1	1	0	0	CCMP (immediate) — 64-bit

Conditional select

The encodings in this section are decoded from [Data Processing -- Register](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sf	op	S	1	1	0	1	0	1	0	0	Rm				cond				op2		Rn				Rd						

Decode fields				Instruction Details
sf	op	S	op2	
		0	1x	UNALLOCATED
		1		UNALLOCATED
0	0	0	00	CSEL — 32-bit
0	0	0	01	CSINC — 32-bit
0	1	0	00	CSINV — 32-bit
0	1	0	01	CSNEG — 32-bit
1	0	0	00	CSEL — 64-bit
1	0	0	01	CSINC — 64-bit
1	1	0	00	CSINV — 64-bit
1	1	0	01	CSNEG — 64-bit

Data-processing (3 source)

The encodings in this section are decoded from [Data Processing -- Register](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sf	op54		1	1	0	1	1	op31				Rm				o0	Ra				Rn				Rd						

Decode fields				Instruction Details		Feature
sf	op54	op31	o0			
	00	100		UNALLOCATED	-	
	!= 00			UNALLOCATED	-	
0	00	x1x		UNALLOCATED	-	
0	00	!= x00x01		UNALLOCATED	-	
0	00	000	0	MADD — 32-bit	-	
0	00	000	1	MSUB — 32-bit	-	
1	00	x10	1	UNALLOCATED	-	
1	00	000	0	MADD — 64-bit	-	
1	00	000	1	MSUB — 64-bit	-	
1	00	001	0	SMADDL	-	
1	00	001	1	SMSUBL	-	
1	00	010	0	SMULH	-	
1	00	011	0	MADDPT	FEAT_CPA	
1	00	011	1	MSUBPT	FEAT_CPA	
1	00	101	0	UMADDL	-	
1	00	101	1	UMSUBL	-	
1	00	110	0	UMULH	-	
1	00	111		UNALLOCATED	-	

Data Processing -- Scalar Floating-Point and Advanced SIMD

The encodings in this section are decoded from [A64 instruction set encoding](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
op0				1	1	1	op1			op2			op3																		

Decode fields				Instruction details	Feature
op0	op1	op2	op3		
00x0	0x	x101	00xxxxx10	UNALLOCATED	-
00x0	11		xxxxxxxxx1	UNALLOCATED	-
0100	0x	x101	00xxxxx10	Cryptographic AES	-
0101	0x	x0xx	xxx0xxx00	Cryptographic three-register SHA	-
0101	0x	x0xx	xxx0xxx10	UNALLOCATED	-
0101	0x	x0xx	xxx1xxx0	UNALLOCATED	-
0101	0x	x101	00xxxxx10	Cryptographic two-register SHA	-
0111	0x	x0xx	xxxxxxxxx0	UNALLOCATED	-
011x	0x	x101	00xxxxx10	UNALLOCATED	-
01x1	00	00xx	xxx0xxx1	Advanced SIMD scalar copy	-
01x1	01	00xx	xxx0xxx1	UNALLOCATED	-
01x1	0x	0111	00xxxxx10	UNALLOCATED	-
01x1	0x	10xx	xxx00xxx1	Advanced SIMD scalar three same FP16	-
01x1	0x	10xx	xxx01xxx1	UNALLOCATED	-
01x1	0x	1111	00xxxxx10	Advanced SIMD scalar two-register miscellaneous FP16	-
01x1	0x	x0xx	xxx1xxx1	Advanced SIMD scalar three same extra	-
01x1	0x	x100	00xxxxx10	Advanced SIMD scalar two-register miscellaneous	-
01x1	0x	x110	00xxxxx10	Advanced SIMD scalar pairwise	-
01x1	0x	x1xx	01xxxxx10	UNALLOCATED	-
01x1	0x	x1xx	1xxxxxx10	UNALLOCATED	-
01x1	0x	x1xx	xxxxxxxx00	Advanced SIMD scalar three different	-
01x1	0x	x1xx	xxxxxxxxx1	Advanced SIMD scalar three same	-
01x1	10		xxxxxxxxx1	Advanced SIMD scalar shift by immediate	-
01x1	1x		xxxxxxxxx0	Advanced SIMD scalar x indexed element	-
01xx	11		xxxxxxxxx1	UNALLOCATED	-
0x00	0x	x0xx	xxx0xxx00	Advanced SIMD table lookup	-
0x00	0x	x0xx	xxx0xxx10	Advanced SIMD permute	-
0x10	0x	x0xx	xxx0xxx0	Advanced SIMD extract	-
0xx0	00	00xx	xxx0xxx1	Advanced SIMD copy	-
0xx0	01	00xx	xxx0xxx1	UNALLOCATED	-
0xx0	0x	0111	00xxxxx10	UNALLOCATED	-
0xx0	0x	10xx	xxx00xxx1	Advanced SIMD three same (FP16)	-
0xx0	0x	10xx	xxx01xxx1	UNALLOCATED	-
0xx0	0x	1111	00xxxxx10	Advanced SIMD two-register miscellaneous (FP16)	-
0xx0	0x	x0xx	xxx1xxx0	UNALLOCATED	-
0xx0	0x	x0xx	xxx1xxx1	Advanced SIMD three-register extension	-
0xx0	0x	x100	00xxxxx10	Advanced SIMD two-register miscellaneous	-
0xx0	0x	x110	00xxxxx10	Advanced SIMD across lanes	-
0xx0	0x	x1xx	01xxxxx10	UNALLOCATED	-
0xx0	0x	x1xx	1xxxxxx10	UNALLOCATED	-
0xx0	0x	x1xx	xxxxxxxx00	Advanced SIMD three different	-
0xx0	0x	x1xx	xxxxxxxxx1	Advanced SIMD three same	-
0xx0	10	0000	xxxxxxxxx1	Advanced SIMD modified immediate	-
0xx0	10	!= 0000	xxxxxxxxx1	Advanced SIMD shift by immediate	-

0xx0	1x		xxxxxxxx0	Advanced SIMD vector x indexed element	-
10x0				UNALLOCATED	-
1100	00	0xxx	xxx1xxxxx	UNALLOCATED	-
1100	00	10xx	xxx10xxxx	Cryptographic three-register, imm2	-
1100	00	10xx	xxx11xxxx	UNALLOCATED	-
1100	00	11xx	xxx1x00xx	Cryptographic three-register SHA 512	-
1100	00	11xx	xxx1x01xx	UNALLOCATED	-
1100	00	11xx	xxx1x1xxx	UNALLOCATED	-
1100	00		xxx0xxxxx	Cryptographic four-register	-
1100	01	00xx		XAR	FEAT_SHA3
1100	01	1000	0000xxxxx	UNALLOCATED	-
1100	01	1000	0001000xx	Cryptographic two-register SHA 512	-
1100	01	1000	0001x01xx	UNALLOCATED	-
1100	01	1000	0001x1xxx	UNALLOCATED	-
1100	01	1000	001xxxxxx	UNALLOCATED	-
1100	01	1000	01xxxxxxx	UNALLOCATED	-
1100	01	1000	1xxxxxxx	UNALLOCATED	-
1100	01	1000	!= 0001000xx0001100xx	UNALLOCATED	-
1100	01	1001		UNALLOCATED	-
1100	01	101x		UNALLOCATED	-
1100	01	x1xx		UNALLOCATED	-
1100	1x			UNALLOCATED	-
1110				UNALLOCATED	-
11x1				UNALLOCATED	-
x0x1	0x	x0xx		Conversion between floating-point and fixed-point	-
x0x1	0x	x1xx	xxx000000	Conversion between floating-point and integer	-
x0x1	0x	x1xx	xxx100000	UNALLOCATED	-
x0x1	0x	x1xx	xxxx10000	Floating-point data-processing (1 source)	-
x0x1	0x	x1xx	xxxxx1000	Floating-point compare	-
x0x1	0x	x1xx	xxxxxx100	Floating-point immediate	-
x0x1	0x	x1xx	xxxxxxx01	Floating-point conditional compare	-
x0x1	0x	x1xx	xxxxxxx10	Floating-point data-processing (2 source)	-
x0x1	0x	x1xx	xxxxxxx11	Floating-point conditional select	-
x0x1	1x			Floating-point data-processing (3 source)	-

Cryptographic AES

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	1	1	1	0	size		1	0	1	0	0	opcode					1	0	Rn				Rd					

Decode fields		Instruction Details		Feature
size	opcode			
00	000xx	UNALLOCATED		
00	00100	AESE		FEAT_AES
00	00101	AESD		FEAT_AES
00	00110	AESMC		FEAT_AES
00	00111	AESIMC		FEAT_AES
00	1xxxx	UNALLOCATED		
00	!= 001xx01xxx	UNALLOCATED		-
!= 00		UNALLOCATED		-

Cryptographic three-register SHA

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	1	1	1	0	size	0				Rm			0		opcode		0	0				Rn					Rd	

Decode fields		Instruction Details		Feature
size	opcode			
00	000	SHA1C		FEAT_SHA1
00	001	SHA1P		FEAT_SHA1
00	010	SHA1M		FEAT_SHA1
00	011	SHA1SU0		FEAT_SHA1
00	100	SHA256H		FEAT_SHA256
00	101	SHA256H2		FEAT_SHA256
00	110	SHA256SU1		FEAT_SHA256
00	111	UNALLOCATED		-
!= 00		UNALLOCATED		-

Cryptographic two-register SHA

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	1	1	1	0	size	1	0	1	0	0	opcode						1	0	Rn					Rd				

Decode fields		Instruction Details		Feature
size	opcode			
00	00000	SHA1H		FEAT_SHA1
00	00001	SHA1SU1		FEAT_SHA1
00	00010	SHA256SU0		FEAT_SHA256
00	00011	UNALLOCATED		-
00	001xx	UNALLOCATED		-
00	01xxx	UNALLOCATED		-
00	1xxxx	UNALLOCATED		-
!= 00		UNALLOCATED		-

Advanced SIMD scalar copy

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	op	1	1	1	1	0	0	0	0					imm5		0					1				Rn				Rd	

Decode fields		Instruction Details		Feature
op	imm4			
0	0000	DUP (element)		FEAT_AdvSIMD
0	!= 0000	UNALLOCATED		-
1		UNALLOCATED		-

Advanced SIMD scalar three same FP16

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	U	1	1	1	1	0	a	1	0	Rm				0	0	opcode			1	Rn				Rd						

Decode fields			Instruction Details		Feature
U	a	opcode			
		00x	UNALLOCATED		-
	0	010	UNALLOCATED		-
0		101	UNALLOCATED		-
0	0	011	FMULX		FEAT_AdvSIMD && FEAT_FP16
0	0	100	FCMEQ (register)		FEAT_AdvSIMD && FEAT_FP16
0	0	110	UNALLOCATED		-
0	0	111	FRECPS		FEAT_AdvSIMD && FEAT_FP16
0	1	x10	UNALLOCATED		-
0	1	011	UNALLOCATED		-
0	1	100	UNALLOCATED		-
0	1	111	FRSQRTS		FEAT_AdvSIMD && FEAT_FP16
1		011	UNALLOCATED		-
1		11x	UNALLOCATED		-
1	0	100	FCMGE (register)		FEAT_AdvSIMD && FEAT_FP16
1	0	101	FACGE		FEAT_AdvSIMD && FEAT_FP16
1	1	010	FABD		FEAT_AdvSIMD && FEAT_FP16
1	1	100	FCMGT (register)		FEAT_AdvSIMD && FEAT_FP16
1	1	101	FACGT		FEAT_AdvSIMD && FEAT_FP16

Advanced SIMD scalar two-register miscellaneous FP16

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	U	1	1	1	1	0	a	1	1	1	1	0	0					opcode		1	0					Rn				Rd

Decode fields			Instruction Details	Feature
U	a	opcode		
		x0xxx	UNALLOCATED	-
		1100x	UNALLOCATED	-
	0	01xxx	UNALLOCATED	-
	1	010xx	UNALLOCATED	-
	1	11100	UNALLOCATED	-
0	0	11010	FCVTNS (vector)	FEAT_AdvSIMD && FEAT_FP16
0	0	11011	FCVTMS (vector)	FEAT_AdvSIMD && FEAT_FP16
0	0	11100	FCVTAS (vector)	FEAT_AdvSIMD && FEAT_FP16
0	0	11101	SCVTF (vector, integer)	FEAT_AdvSIMD && FEAT_FP16
0	0	1111x	UNALLOCATED	-
0	1	01100	FCMGT (zero)	FEAT_AdvSIMD && FEAT_FP16
0	1	01101	FCMEQ (zero)	FEAT_AdvSIMD && FEAT_FP16
0	1	01110	FCMLT (zero)	FEAT_AdvSIMD && FEAT_FP16
0	1	01111	UNALLOCATED	-
0	1	11010	FCVTPS (vector)	FEAT_AdvSIMD && FEAT_FP16
0	1	11011	FCVTZS (vector, integer)	FEAT_AdvSIMD && FEAT_FP16
0	1	11101	FRECPE	FEAT_AdvSIMD && FEAT_FP16
0	1	11110	UNALLOCATED	-
0	1	11111	FRECPX	FEAT_AdvSIMD && FEAT_FP16
1		1111x	UNALLOCATED	-
1	0	11010	FCVTNU (vector)	FEAT_AdvSIMD && FEAT_FP16
1	0	11011	FCVTMU (vector)	FEAT_AdvSIMD && FEAT_FP16
1	0	11100	FCVTAU (vector)	FEAT_AdvSIMD && FEAT_FP16
1	0	11101	UCVTF (vector, integer)	FEAT_AdvSIMD && FEAT_FP16
1	1	01100	FCMGE (zero)	FEAT_AdvSIMD && FEAT_FP16
1	1	01101	FCMLE (zero)	FEAT_AdvSIMD && FEAT_FP16
1	1	0111x	UNALLOCATED	-
1	1	11010	FCVTPU (vector)	FEAT_AdvSIMD && FEAT_FP16
1	1	11011	FCVTZU (vector, integer)	FEAT_AdvSIMD && FEAT_FP16
1	1	11101	FRSQRTE	FEAT_AdvSIMD && FEAT_FP16

Advanced SIMD scalar three same extra

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	U	1	1	1	1	0	size	0	Rm				1	opcode				1	Rn				Rd							

Decode fields			Instruction Details	Feature
U		opcode		
0			UNALLOCATED	-
1		0000	SQRDMLAH (vector)	FEAT_RDM
1		0001	SQRDMLSH (vector)	FEAT_RDM
1		01xx	UNALLOCATED	1
1		1xxx	UNALLOCATED	1
1		!= 000x001x	UNALLOCATED	-

Advanced SIMD scalar two-register miscellaneous

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	U	1	1	1	1	0	size	1	0	0	0	0	opcode			1	0	Rn			Rd									

U	Decode fields		Instruction Details	Feature
	size	opcode		
		00xx0	UNALLOCATED	-
		00x01	UNALLOCATED	-
		1x000	UNALLOCATED	-
		10xx1	UNALLOCATED	-
		11001	UNALLOCATED	-
	0x	01xxx	UNALLOCATED	-
	0x	1111x	UNALLOCATED	-
	1x	01111	UNALLOCATED	-
	1x	11100	UNALLOCATED	-
	10	010x1	UNALLOCATED	-
	10	01000	UNALLOCATED	-
0		00011	SUQADD	FEAT_AdvSIMD
0		00111	SQABS	FEAT_AdvSIMD
0		10010	UNALLOCATED	-
0		10100	SQXTN, SQXTN2	FEAT_AdvSIMD
0		10110	UNALLOCATED	-
0	0x	11010	FCVTNS (vector)	FEAT_AdvSIMD
0	0x	11011	FCVTMS (vector)	FEAT_AdvSIMD
0	0x	11100	FCVTAS (vector)	FEAT_AdvSIMD
0	0x	11101	SCVTF (vector, integer)	FEAT_AdvSIMD
0	1x	01100	FCMGT (zero)	FEAT_AdvSIMD
0	1x	01101	FCMEQ (zero)	FEAT_AdvSIMD
0	1x	01110	FCMLT (zero)	FEAT_AdvSIMD
0	1x	11010	FCVTPS (vector)	FEAT_AdvSIMD
0	1x	11011	FCVTZS (vector, integer)	FEAT_AdvSIMD
0	1x	11101	FRECPE	FEAT_AdvSIMD
0	1x	11110	UNALLOCATED	-
0	1x	11111	FRECPX	FEAT_AdvSIMD
0	10	01010	UNALLOCATED	-
0	11	01000	CMGT (zero)	FEAT_AdvSIMD
0	11	01001	CMEQ (zero)	FEAT_AdvSIMD
0	11	01010	CMLT (zero)	FEAT_AdvSIMD
0	11	01011	ABS	FEAT_AdvSIMD
1		00011	USQADD	FEAT_AdvSIMD
1		00111	SQNEG	FEAT_AdvSIMD
1		10010	SQXTUN, SQXTUN2	FEAT_AdvSIMD
1		10100	UQXTN, UQXTN2	FEAT_AdvSIMD
1	0x	11010	FCVTNU (vector)	FEAT_AdvSIMD
1	0x	11011	FCVTMU (vector)	FEAT_AdvSIMD
1	0x	11100	FCVTAU (vector)	FEAT_AdvSIMD
1	0x	11101	UCVTF (vector, integer)	FEAT_AdvSIMD
1	00	10110	UNALLOCATED	-
1	01	10110	FCVTXN, FCVTXN2	FEAT_AdvSIMD
1	1x	01x10	UNALLOCATED	-
1	1x	01100	FCMGE (zero)	FEAT_AdvSIMD

Decode fields			Instruction Details	Feature
U	size	opcode		
1	1x	01101	FCMLE (zero)	FEAT_AdvSIMD
1	1x	1x110	UNALLOCATED	-
1	1x	11010	FCVTPU (vector)	FEAT_AdvSIMD
1	1x	11011	FCVTZU (vector, integer)	FEAT_AdvSIMD
1	1x	11101	FRSQRTE	FEAT_AdvSIMD
1	1x	11111	UNALLOCATED	-
1	11	01000	CMGE (zero)	FEAT_AdvSIMD
1	11	01001	CMLE (zero)	FEAT_AdvSIMD
1	11	01011	NEG (vector)	FEAT_AdvSIMD

Advanced SIMD scalar pairwise

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
0	1	U	1	1	1	1	0	size	1	1	0	0	0	opcode						1	0	Rn						Rd					

Decode fields			Instruction Details	Feature
U	size	opcode		
		x0xxx	UNALLOCATED	-
		01110	UNALLOCATED	-
		111xx	UNALLOCATED	-
	10	01101	UNALLOCATED	-
0	x1	011x1	UNALLOCATED	-
0	x1	01100	UNALLOCATED	-
0	00	01100	FMAXNMP (scalar) — Half-precision	FEAT_AdvSIMD && FEAT_FP16
0	00	01101	FADDP (scalar) — Half-precision	FEAT_AdvSIMD && FEAT_FP16
0	00	01111	FMAXP (scalar) — Half-precision	FEAT_AdvSIMD && FEAT_FP16
0	10	01100	FMINNMP (scalar) — Half-precision	FEAT_AdvSIMD && FEAT_FP16
0	10	01111	FMINP (scalar) — Half-precision	FEAT_AdvSIMD && FEAT_FP16
0	11	x100x	UNALLOCATED	-
0	11	x1010	UNALLOCATED	-
0	11	01011	UNALLOCATED	-
0	11	11011	ADDP (scalar)	FEAT_AdvSIMD
0	!= 11	x10xx	UNALLOCATED	-
1		x10xx	UNALLOCATED	-
1	0x	01100	FMAXNMP (scalar) — Single-precision and double-precision	FEAT_AdvSIMD
1	0x	01101	FADDP (scalar) — Single-precision and double-precision	FEAT_AdvSIMD
1	0x	01111	FMAXP (scalar) — Single-precision and double-precision	FEAT_AdvSIMD
1	1x	01100	FMINNMP (scalar) — Single-precision and double-precision	FEAT_AdvSIMD
1	1x	01111	FMINP (scalar) — Single-precision and double-precision	FEAT_AdvSIMD
1	11	01101	UNALLOCATED	-

Advanced SIMD scalar three different

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	U	1	1	1	1	0	size	1	Rm				opcode				0	0	Rn						Rd					

Decode fields		Instruction Details	Feature
U	opcode		
0	0xxx	UNALLOCATED	-
0	1xx0	UNALLOCATED	-
0	1001	SQDMLAL, SQDMLAL2 (vector)	FEAT_AdvSIMD
0	1011	SQDMLSL, SQDMLSL2 (vector)	FEAT_AdvSIMD
0	1101	SQDMULL, SQDMULL2 (vector)	FEAT_AdvSIMD
0	1111	UNALLOCATED	-
1		UNALLOCATED	-

Advanced SIMD scalar three same

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
0	1	U	1	1	1	1	0	size	1	Rm						opcode						1	Rn						Rd					

U	Decode fields		Instruction Details	Feature
	size	opcode		
		x0011	UNALLOCATED	-
		011x1	UNALLOCATED	-
		11001	UNALLOCATED	-
		11110	UNALLOCATED	-
	x0	0x1x0	UNALLOCATED	-
	x1	00100	UNALLOCATED	-
	x1	011x0	UNALLOCATED	-
	0x	xx0x0	UNALLOCATED	-
	0x	10x01	UNALLOCATED	-
	01	00110	UNALLOCATED	-
	1x	10010	UNALLOCATED	-
	1x	11000	UNALLOCATED	-
	10	0x0x0	UNALLOCATED	-
	10	10x0x	UNALLOCATED	-
	11	00000	UNALLOCATED	-
	11	00010	UNALLOCATED	-
	11	10101	UNALLOCATED	-
	!= 10	10100	UNALLOCATED	-
	!= 11	00111	UNALLOCATED	-
0		00001	SQADD	FEAT_AdvSIMD
0		00101	SQSUB	FEAT_AdvSIMD
0		01001	SQSHL (register)	FEAT_AdvSIMD
0		01011	SQRSHL	FEAT_AdvSIMD
0		10110	SQDMULH (vector)	FEAT_AdvSIMD
0		10111	UNALLOCATED	-
0		11101	UNALLOCATED	-
0	0x	11011	FMULX	FEAT_AdvSIMD
0	0x	11100	FCMEQ (register)	FEAT_AdvSIMD
0	0x	11111	FRECPS	FEAT_AdvSIMD
0	1x	11010	UNALLOCATED	-
0	1x	11011	UNALLOCATED	-
0	1x	11100	UNALLOCATED	-
0	1x	11111	FRSQRTS	FEAT_AdvSIMD
0	11	00110	CMGT (register)	FEAT_AdvSIMD
0	11	00111	CMGE (register)	FEAT_AdvSIMD
0	11	01000	SSHL	FEAT_AdvSIMD
0	11	01010	SRSHL	FEAT_AdvSIMD
0	11	10000	ADD (vector)	FEAT_AdvSIMD
0	11	10001	CMTST	FEAT_AdvSIMD
1		00001	UQADD	FEAT_AdvSIMD
1		00101	UQSUB	FEAT_AdvSIMD
1		01001	UQSHL (register)	FEAT_AdvSIMD
1		01011	UQRSHL	FEAT_AdvSIMD
1		1x111	UNALLOCATED	-
1		10110	SQRDMULH (vector)	FEAT_AdvSIMD

Decode fields		Instruction Details		Feature
U	size	opcode		
1		11011	UNALLOCATED	-
1	0x	11100	FCMGE (register)	FEAT_AdvSIMD
1	0x	11101	FACGE	FEAT_AdvSIMD
1	1x	11010	FABD	FEAT_AdvSIMD
1	1x	11100	FCMGT (register)	FEAT_AdvSIMD
1	1x	11101	FACGT	FEAT_AdvSIMD
1	11	00110	CMHI (register)	FEAT_AdvSIMD
1	11	00111	CMHS (register)	FEAT_AdvSIMD
1	11	01000	USHL	FEAT_AdvSIMD
1	11	01010	URSHL	FEAT_AdvSIMD
1	11	10000	SUB (vector)	FEAT_AdvSIMD
1	11	10001	CMEQ (register)	FEAT_AdvSIMD

Advanced SIMD scalar shift by immediate

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	U	1	1	1	1	1	0	immh				immb				opcode				1	Rn				Rd					

U	Decode fields		Instruction Details	Feature
	immh	opcode		
		0xxx1	UNALLOCATED	-
		101xx	UNALLOCATED	-
		11101	UNALLOCATED	-
	0xxx	x10x0	UNALLOCATED	-
	0xxx	00xx0	UNALLOCATED	-
	0xxx	110x1	UNALLOCATED	-
	0000	x1110	UNALLOCATED	-
	0000	1001x	UNALLOCATED	-
	0000	11100	UNALLOCATED	-
	0000	11111	UNALLOCATED	-
	1xxx	110xx	UNALLOCATED	-
	!= 0000	11110	UNALLOCATED	-
0		01100	UNALLOCATED	-
0		1000x	UNALLOCATED	-
0	1xxx	00000	SSHR	FEAT_AdvSIMD
0	1xxx	00010	SSRA	FEAT_AdvSIMD
0	1xxx	00100	SRSR	FEAT_AdvSIMD
0	1xxx	00110	SRSRA	FEAT_AdvSIMD
0	1xxx	01000	UNALLOCATED	-
0	1xxx	01010	SHL	FEAT_AdvSIMD
0	!= 0000	01110	SQSHL (immediate)	FEAT_AdvSIMD
0	!= 0000	10010	SQSHRN, SQSHRN2	FEAT_AdvSIMD
0	!= 0000	10011	SQRSHRN, SQRSHRN2	FEAT_AdvSIMD
0	!= 0000	11100	SCVTF (vector, fixed-point)	FEAT_AdvSIMD
0	!= 0000	11111	FCVTZS (vector, fixed-point)	FEAT_AdvSIMD
1	0000	01100	UNALLOCATED	-
1	0000	1000x	UNALLOCATED	-
1	1xxx	00000	USHR	FEAT_AdvSIMD
1	1xxx	00010	USRA	FEAT_AdvSIMD
1	1xxx	00100	URSHR	FEAT_AdvSIMD
1	1xxx	00110	URSRA	FEAT_AdvSIMD
1	1xxx	01000	SRI	FEAT_AdvSIMD
1	1xxx	01010	SLI	FEAT_AdvSIMD
1	!= 0000	01100	SQSHLU	FEAT_AdvSIMD
1	!= 0000	01110	UQSHL (immediate)	FEAT_AdvSIMD
1	!= 0000	10000	SQSHRUN, SQSHRUN2	FEAT_AdvSIMD
1	!= 0000	10001	SQRSHRUN, SQRSHRUN2	FEAT_AdvSIMD
1	!= 0000	10010	UQSHRN, UQSHRN2	FEAT_AdvSIMD
1	!= 0000	10011	UQRSHRN, UQRSHRN2	FEAT_AdvSIMD
1	!= 0000	11100	UCVTF (vector, fixed-point)	FEAT_AdvSIMD
1	!= 0000	11111	FCVTZU (vector, fixed-point)	FEAT_AdvSIMD

Advanced SIMD scalar x indexed element

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	U	1	1	1	1	1	size	L	M	Rm			opcode			H	0	Rn			Rd									

Decode fields			Instruction Details		Feature
U	size	opcode			
	01	1001	UNALLOCATED		-
0		0xx0	UNALLOCATED		-
0		0011	SQDMLAL, SQDMLAL2 (by element)		FEAT_AdvSIMD
0		0111	SQDMLSL, SQDMLSL2 (by element)		FEAT_AdvSIMD
0		10x0	UNALLOCATED		-
0		1011	SQDMULL, SQDMULL2 (by element)		FEAT_AdvSIMD
0		1100	SQDMULH (by element)		FEAT_AdvSIMD
0		1101	SQRDMULH (by element)		FEAT_AdvSIMD
0		111x	UNALLOCATED		-
0	00	0001	FMLA (by element) — Scalar, half-precision		FEAT_AdvSIMD && FEAT_FP16
0	00	0101	FMLS (by element) — Scalar, half-precision		FEAT_AdvSIMD && FEAT_FP16
0	00	1001	FMUL (by element) — Scalar, half-precision		FEAT_AdvSIMD && FEAT_FP16
0	01	0x01	UNALLOCATED		-
0	1x	0001	FMLA (by element) — Scalar, single-precision and double-precision		FEAT_AdvSIMD
0	1x	0101	FMLS (by element) — Scalar, single-precision and double-precision		FEAT_AdvSIMD
0	1x	1001	FMUL (by element) — Scalar, single-precision and double-precision		FEAT_AdvSIMD
1		0xxx	UNALLOCATED		-
1		1xx0	UNALLOCATED		-
1		1011	UNALLOCATED		-
1		1101	SQRDMLAH (by element)		FEAT_RDM
1		1111	SQRDMLSH (by element)		FEAT_RDM
1	00	1001	FMULX (by element) — Scalar, half-precision		FEAT_AdvSIMD && FEAT_FP16
1	1x	1001	FMULX (by element) — Scalar, single-precision and double-precision		FEAT_AdvSIMD

Advanced SIMD table lookup

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	1	1	1	0	op2	0	Rm				0	len	op	0	0	Rn				Rd								

Decode fields				Instruction Details	Feature
Q	op2	len	op		
	00	00	0	TBL — Single register table	FEAT_AdvSIMD
	00	00	1	TBX — Single register table	FEAT_AdvSIMD
	00	01	0	TBL — Two register table	FEAT_AdvSIMD
	00	01	1	TBX — Two register table	FEAT_AdvSIMD
	00	10	0	TBL — Three register table	FEAT_AdvSIMD
	00	10	1	TBX — Three register table	FEAT_AdvSIMD
	00	11	0	TBL — Four register table	FEAT_AdvSIMD
	00	11	1	TBX — Four register table	FEAT_AdvSIMD
0	!= 00			UNALLOCATED	-
1	01		1	LUTI4 — Halfword	FEAT_AdvSIMD && FEAT_LUT
1	01	x0	0	UNALLOCATED	-
1	01	x1	0	LUTI4 — Byte	FEAT_AdvSIMD && FEAT_LUT
1	10		0	UNALLOCATED	-
1	10		1	LUTI2 — Byte	FEAT_AdvSIMD && FEAT_LUT
1	11			LUTI2 — Halfword	FEAT_AdvSIMD && FEAT_LUT

Advanced SIMD permute

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	Q	0	0	1	1	1	0	size	0				Rm			0			opcode	1	0				Rn					Rd	

Decode fields	Instruction Details		Feature
opcode			
x00	UNALLOCATED	-	
001	UZP1		FEAT_AdvSIMD
010	TRN1		FEAT_AdvSIMD
011	ZIP1		FEAT_AdvSIMD
101	UZP2		FEAT_AdvSIMD
110	TRN2		FEAT_AdvSIMD
111	ZIP2		FEAT_AdvSIMD

Advanced SIMD extract

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	Q	1	0	1	1	1	0	op2	0				Rm			0			imm4	0					Rn					Rd	

Decode fields		Instruction Details	Feature
op2			
00		EXT	FEAT_AdvSIMD
!= 00		UNALLOCATED	-

Advanced SIMD copy

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	Q	op	0	1	1	1	0	0	0	0					imm5	0			imm4	1					Rn					Rd	

		Decode fields		Instruction Details	
Q	op	imm5	imm4		Feature
	0		0000	DUP (element)	FEAT_AdvSIMD
	0		0001	DUP (general)	FEAT_AdvSIMD
	0		01x0	UNALLOCATED	-
	0		1xxx	UNALLOCATED	-
0	0		001x	UNALLOCATED	-
0	0		0101	SMOV — 32-bit	FEAT_AdvSIMD
0	0		0111	UMOV — 32-bit	FEAT_AdvSIMD
0	1			UNALLOCATED	-
1	0		0010	UNALLOCATED	-
1	0		0011	INS (general)	FEAT_AdvSIMD
1	0		0101	SMOV — 64-bit	FEAT_AdvSIMD
1	0	x0xxx	0111	UNALLOCATED	-
1	0	x1000	0111	UMOV — 64-bit	FEAT_AdvSIMD
1	0	x101x	0111	UNALLOCATED	-
1	0	x11xx	0111	UNALLOCATED	-
1	0	!= x1000x1001	0111	UNALLOCATED	-
1	1			INS (element)	FEAT_AdvSIMD

Advanced SIMD three same (FP16)

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	Q	U	0	1	1	1	0	a	1	0	Rm				0	0	opcode				1	Rn				Rd					

Decode fields			Instruction Details	Feature
U	a	opcode		
0	0	000	FMAXNM (vector)	FEAT_AdvSIMD && FEAT_FP16
0	0	001	FMLA (vector)	FEAT_AdvSIMD && FEAT_FP16
0	0	010	FADD (vector)	FEAT_AdvSIMD && FEAT_FP16
0	0	011	FMULX	FEAT_AdvSIMD && FEAT_FP16
0	0	100	FCMEQ (register)	FEAT_AdvSIMD && FEAT_FP16
0	0	101	UNALLOCATED	-
0	0	110	FMAX (vector)	FEAT_AdvSIMD && FEAT_FP16
0	0	111	FRECPS	FEAT_AdvSIMD && FEAT_FP16
0	1	000	FMINNM (vector)	FEAT_AdvSIMD && FEAT_FP16
0	1	001	FMLS (vector)	FEAT_AdvSIMD && FEAT_FP16
0	1	010	FSUB (vector)	FEAT_AdvSIMD && FEAT_FP16
0	1	011	FAMAX	FEAT_AdvSIMD && FEAT_FAMINMAX
0	1	10x	UNALLOCATED	-
0	1	110	FMIN (vector)	FEAT_AdvSIMD && FEAT_FP16
0	1	111	FRSQRTS	FEAT_AdvSIMD && FEAT_FP16
1		001	UNALLOCATED	-
1	0	000	FMAXNMP (vector)	FEAT_AdvSIMD && FEAT_FP16
1	0	010	FADDP (vector)	FEAT_AdvSIMD && FEAT_FP16
1	0	011	FMUL (vector)	FEAT_AdvSIMD && FEAT_FP16
1	0	100	FCMGE (register)	FEAT_AdvSIMD && FEAT_FP16
1	0	101	FACGE	FEAT_AdvSIMD && FEAT_FP16
1	0	110	FMAXP (vector)	FEAT_AdvSIMD && FEAT_FP16
1	0	111	FDIV (vector)	FEAT_AdvSIMD && FEAT_FP16
1	1	000	FMINNMP (vector)	FEAT_AdvSIMD && FEAT_FP16
1	1	010	FABD	FEAT_AdvSIMD && FEAT_FP16
1	1	011	FAMIN	FEAT_AdvSIMD && FEAT_FAMINMAX
1	1	100	FCMGT (register)	FEAT_AdvSIMD && FEAT_FP16
1	1	101	FACGT	FEAT_AdvSIMD && FEAT_FP16
1	1	110	FMINP (vector)	FEAT_AdvSIMD && FEAT_FP16
1	1	111	FSCALE	FEAT_FP8

Advanced SIMD two-register miscellaneous (FP16)

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	Q	U	0	1	1	1	0	a	1	1	1	1	0	0	opcode				1	0	Rn				Rd						

Decode fields			Instruction Details	Feature
U	a	opcode		
		x0xxx	UNALLOCATED	-
	0	01xxx	UNALLOCATED	-
	0	1111x	UNALLOCATED	-
	1	010xx	UNALLOCATED	-
	1	11100	UNALLOCATED	-
0	0	11000	FRINTN (vector)	FEAT_AdvSIMD && FEAT_FP16
0	0	11001	FRINTM (vector)	FEAT_AdvSIMD && FEAT_FP16
0	0	11010	FCVTNS (vector)	FEAT_AdvSIMD && FEAT_FP16
0	0	11011	FCVTMS (vector)	FEAT_AdvSIMD && FEAT_FP16
0	0	11100	FCVTAS (vector)	FEAT_AdvSIMD && FEAT_FP16
0	0	11101	SCVTF (vector, integer)	FEAT_AdvSIMD && FEAT_FP16
0	1	01100	FCMGT (zero)	FEAT_AdvSIMD && FEAT_FP16
0	1	01101	FCMEQ (zero)	FEAT_AdvSIMD && FEAT_FP16
0	1	01110	FCMLT (zero)	FEAT_AdvSIMD && FEAT_FP16
0	1	01111	FABS (vector)	FEAT_AdvSIMD && FEAT_FP16
0	1	11000	FRINTP (vector)	FEAT_AdvSIMD && FEAT_FP16
0	1	11001	FRINTZ (vector)	FEAT_AdvSIMD && FEAT_FP16
0	1	11010	FCVTPS (vector)	FEAT_AdvSIMD && FEAT_FP16
0	1	11011	FCVTZS (vector, integer)	FEAT_AdvSIMD && FEAT_FP16
0	1	11101	FRECPE	FEAT_AdvSIMD && FEAT_FP16
0	1	1111x	UNALLOCATED	-
1	0	11000	FRINTA (vector)	FEAT_AdvSIMD && FEAT_FP16
1	0	11001	FRINTX (vector)	FEAT_AdvSIMD && FEAT_FP16
1	0	11010	FCVTNU (vector)	FEAT_AdvSIMD && FEAT_FP16
1	0	11011	FCVTMU (vector)	FEAT_AdvSIMD && FEAT_FP16
1	0	11100	FCVTAU (vector)	FEAT_AdvSIMD && FEAT_FP16
1	0	11101	UCVTF (vector, integer)	FEAT_AdvSIMD && FEAT_FP16
1	1	x1110	UNALLOCATED	-
1	1	01100	FCMGE (zero)	FEAT_AdvSIMD && FEAT_FP16
1	1	01101	FCMLE (zero)	FEAT_AdvSIMD && FEAT_FP16
1	1	01111	FNEG (vector)	FEAT_AdvSIMD && FEAT_FP16
1	1	11000	UNALLOCATED	-
1	1	11001	FRINTI (vector)	FEAT_AdvSIMD && FEAT_FP16
1	1	11010	FCVTPU (vector)	FEAT_AdvSIMD && FEAT_FP16
1	1	11011	FCVTZU (vector, integer)	FEAT_AdvSIMD && FEAT_FP16
1	1	11101	FRSQRTE	FEAT_AdvSIMD && FEAT_FP16
1	1	11111	FSQRT (vector)	FEAT_AdvSIMD && FEAT_FP16

Advanced SIMD three-register extension

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	Q	U	0	1	1	1	0	size	0		Rm					1		opcode		1					Rn					Rd	

Decode fields				Instruction Details	Feature
Q	U	size	opcode		
	0		0000	UNALLOCATED	-
	0		0010	SDOT (vector)	FEAT_DotProd
	0	x0	0001	UNALLOCATED	-
	0	x0	1001	UNALLOCATED	-
	0	x0	1011	UNALLOCATED	-
	0	x1	x0x1	UNALLOCATED	-
	0	0x	1010	UNALLOCATED	-
	0	0x	1100	UNALLOCATED	-
	0	00	0011	UNALLOCATED	-
	0	00	1110	FCVTN, FCVTN2 (single-precision to 8-bit floating-point)	FEAT_FP8
	0	00	1111	FDOT (8-bit floating-point to single-precision, vector)	FEAT_FP8DOT4
	0	01	1110	FCVTN (half-precision to 8-bit floating-point)	FEAT_FP8
	0	01	1111	FDOT (8-bit floating-point to half-precision, vector)	FEAT_FP8DOT2
	0	1x	1xx0	UNALLOCATED	-
	0	10	0011	USDOT (vector)	FEAT_I8MM
	0	10	1111	FDOT (half-precision to single-precision, vector)	FEAT_F16F32DOT
	1		0000	SQRDMLAH (vector)	FEAT_RDM
	1		0001	SQRDMLSH (vector)	FEAT_RDM
	1		0010	UDOT (vector)	FEAT_DotProd
	1		0011	UNALLOCATED	-
	1		10xx	FCMLA	FEAT_FCMA
	1		11x0	FCADD	FEAT_FCMA
	1	x0	1111	UNALLOCATED	-
	1	01	1111	BFDOT (vector)	FEAT_BF16
	1	11	1111	BFMLALB, BFMLALT (vector)	FEAT_BF16
0			01xx	UNALLOCATED	-
0			1101	UNALLOCATED	-
0	0	00	1000	FMLALLBB, FMLALLBT, FMLALLTB, FMLALLTT (vector) — FMLALLBB	FEAT_FP8FMA
0	0	01	1000	FMLALLBB, FMLALLBT, FMLALLTB, FMLALLTT (vector) — FMLALLBT	FEAT_FP8FMA
0	0	11	1111	FMLALB, FMLALT (vector) — FMLALB	FEAT_FP8FMA
1		10	011x	UNALLOCATED	-
1		!= 10	01xx	UNALLOCATED	-
1	0	x0	1101	UNALLOCATED	-
1	0	00	1000	FMLALLBB, FMLALLBT, FMLALLTB, FMLALLTT (vector) — FMLALLTB	FEAT_FP8FMA
1	0	01	1000	FMLALLBB, FMLALLBT, FMLALLTB, FMLALLTT (vector) — FMLALLTT	FEAT_FP8FMA
1	0	01	1101	FMMLA (widening, half-precision to single-precision)	FEAT_F16F32MM
1	0	10	0100	SMMLA (vector)	FEAT_I8MM
1	0	10	0101	USMMLA (vector)	FEAT_I8MM
1	0	11	1101	FMMLA (non-widening)	FEAT_F16MM
1	0	11	1111	FMLALB, FMLALT (vector) — FMLALT	FEAT_FP8FMA
1	1	00	1101	FMMLA (widening, 8-bit floating-point to half-precision)	FEAT_F8F16MM
1	1	01	1101	BFMMLA (widening)	FEAT_BF16
1	1	10	0100	UMMLA (vector)	FEAT_I8MM
1	1	10	0101	UNALLOCATED	-
1	1	10	1101	FMMLA (widening, 8-bit floating-point to single-precision)	FEAT_F8F32MM

Decode fields				Instruction Details	Feature
Q	U	size	opcode		
1	1	11	1101	UNALLOCATED	-

Advanced SIMD two-register miscellaneous

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	Q	U	0	1	1	1	0	size	1	0	0	0	0	0	opcode					1	0	Rn					Rd				

U	Decode fields		Instruction Details	Feature
	size	opcode		
		1000x	UNALLOCATED	-
		10101	UNALLOCATED	-
	0x	011xx	UNALLOCATED	-
	10	11110	UNALLOCATED	-
	11	1x110	UNALLOCATED	-
0		00000	REV64	FEAT_AdvSIMD
0		00001	REV16 (vector)	FEAT_AdvSIMD
0		00010	SADDLP	FEAT_AdvSIMD
0		00011	SUQADD	FEAT_AdvSIMD
0		00100	CLS (vector)	FEAT_AdvSIMD
0		00101	CNT	FEAT_AdvSIMD
0		00110	SADALP	FEAT_AdvSIMD
0		00111	SQABS	FEAT_AdvSIMD
0		01000	CMGT (zero)	FEAT_AdvSIMD
0		01001	CMEQ (zero)	FEAT_AdvSIMD
0		01010	CMLT (zero)	FEAT_AdvSIMD
0		01011	ABS	FEAT_AdvSIMD
0		10010	XTN, XTN2	FEAT_AdvSIMD
0		10011	UNALLOCATED	-
0		10100	SQXTN, SQXTN2	FEAT_AdvSIMD
0	0x	10110	FCVTN, FCVTN2 (double to single-precision, single to half-precision)	FEAT_AdvSIMD
0	0x	10111	FCVTL, FCVTL2	FEAT_AdvSIMD
0	0x	11000	FRINTN (vector)	FEAT_AdvSIMD
0	0x	11001	FRINTM (vector)	FEAT_AdvSIMD
0	0x	11010	FCVTNS (vector)	FEAT_AdvSIMD
0	0x	11011	FCVTMS (vector)	FEAT_AdvSIMD
0	0x	11100	FCVTAS (vector)	FEAT_AdvSIMD
0	0x	11101	SCVTF (vector, integer)	FEAT_AdvSIMD
0	0x	11110	FRINT32Z (vector)	FEAT_FRINTTS
0	0x	11111	FRINT64Z (vector)	FEAT_FRINTTS
0	1x	01100	FCMGT (zero)	FEAT_AdvSIMD
0	1x	01101	FCMEQ (zero)	FEAT_AdvSIMD
0	1x	01110	FCMLT (zero)	FEAT_AdvSIMD
0	1x	01111	FABS (vector)	FEAT_AdvSIMD
0	1x	1x111	UNALLOCATED	-
0	1x	11000	FRINTP (vector)	FEAT_AdvSIMD
0	1x	11001	FRINTZ (vector)	FEAT_AdvSIMD
0	1x	11010	FCVTPS (vector)	FEAT_AdvSIMD
0	1x	11011	FCVTZS (vector, integer)	FEAT_AdvSIMD
0	1x	11100	URECPE	FEAT_AdvSIMD
0	1x	11101	FRECPE	FEAT_AdvSIMD
0	10	10110	BFCVTN, BFCVTN2	FEAT_BF16
1		00000	REV32 (vector)	FEAT_AdvSIMD
1		00010	UADDLP	FEAT_AdvSIMD
1		00011	USQADD	FEAT_AdvSIMD

Decode fields			Instruction Details	Feature
U	size	opcode		
1		00100	CLZ (vector)	FEAT_AdvSIMD
1		00110	UADALP	FEAT_AdvSIMD
1		00111	SQNEG	FEAT_AdvSIMD
1		01000	CMGE (zero)	FEAT_AdvSIMD
1		01001	CMLE (zero)	FEAT_AdvSIMD
1		01010	UNALLOCATED	-
1		01011	NEG (vector)	FEAT_AdvSIMD
1		10010	SQXTUN, SQXTUN2	FEAT_AdvSIMD
1		10011	SHLL, SHLL2	FEAT_AdvSIMD
1		10100	UQXTN, UQXTN2	FEAT_AdvSIMD
1	x0	10110	UNALLOCATED	-
1	0x	00001	UNALLOCATED	-
1	0x	11000	FRINTA (vector)	FEAT_AdvSIMD
1	0x	11001	FRINTX (vector)	FEAT_AdvSIMD
1	0x	11010	FCVTNU (vector)	FEAT_AdvSIMD
1	0x	11011	FCVTMU (vector)	FEAT_AdvSIMD
1	0x	11100	FCVTAU (vector)	FEAT_AdvSIMD
1	0x	11101	UCVTF (vector, integer)	FEAT_AdvSIMD
1	0x	11110	FRINT32X (vector)	FEAT_FRINTTS
1	0x	11111	FRINT64X (vector)	FEAT_FRINTTS
1	00	00101	NOT	FEAT_AdvSIMD
1	00	10111	F1CVTL, F1CVTL2, F2CVTL, F2CVTL2 — F1CVTL{2}	FEAT_FP8
1	01	00101	RBIT (vector)	FEAT_AdvSIMD
1	01	10110	FCVTXN, FCVTXN2	FEAT_AdvSIMD
1	01	10111	F1CVTL, F1CVTL2, F2CVTL, F2CVTL2 — F2CVTL{2}	FEAT_FP8
1	1x	00x01	UNALLOCATED	-
1	1x	01100	FCMGE (zero)	FEAT_AdvSIMD
1	1x	01101	FCMLE (zero)	FEAT_AdvSIMD
1	1x	01110	UNALLOCATED	-
1	1x	01111	FNEG (vector)	FEAT_AdvSIMD
1	1x	11000	UNALLOCATED	-
1	1x	11001	FRINTI (vector)	FEAT_AdvSIMD
1	1x	11010	FCVTPU (vector)	FEAT_AdvSIMD
1	1x	11011	FCVTZU (vector, integer)	FEAT_AdvSIMD
1	1x	11100	URSQRTE	FEAT_AdvSIMD
1	1x	11101	FRSQRTE	FEAT_AdvSIMD
1	1x	11111	FSQRT (vector)	FEAT_AdvSIMD
1	10	10111	BF1CVTL, BF1CVTL2, BF2CVTL, BF2CVTL2 — BF1CVTL{2}	FEAT_FP8
1	11	10111	BF1CVTL, BF1CVTL2, BF2CVTL, BF2CVTL2 — BF2CVTL{2}	FEAT_FP8

Advanced SIMD across lanes

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	Q	U	0	1	1	1	0	size	1	1	0	0	0	opcode			1		0	Rn			Rd								

Decode fields				Instruction Details		Feature
Q	U	size	opcode			
			x100x	UNALLOCATED	-	
			000x0	UNALLOCATED	-	
			00001	UNALLOCATED	-	
			01011	UNALLOCATED	-	
			100xx	UNALLOCATED	-	
		x0	001xx	UNALLOCATED	-	
		x0	01101	UNALLOCATED	-	
		x0	101xx	UNALLOCATED	-	
		x0	111xx	UNALLOCATED	-	
		x1	xx1xx	UNALLOCATED	-	
	0		00011	SADDLV	FEAT_AdvSIMD	
	0		01010	SMAXV	FEAT_AdvSIMD	
	0		11010	SMINV	FEAT_AdvSIMD	
	0		11011	ADDV	FEAT_AdvSIMD	
	0	x0	01110	UNALLOCATED	-	
	0	00	01100	FMAXNMV — Half-precision	FEAT_AdvSIMD && FEAT_FP16	
	0	00	01111	FMAXV — Half-precision	FEAT_AdvSIMD && FEAT_FP16	
	0	10	01100	FMINNMV — Half-precision	FEAT_AdvSIMD && FEAT_FP16	
	0	10	01111	FMINV — Half-precision	FEAT_AdvSIMD && FEAT_FP16	
	1		00011	UADDLV	FEAT_AdvSIMD	
	1		01010	UMAXV	FEAT_AdvSIMD	
	1		11010	UMINV	FEAT_AdvSIMD	
	1		11011	UNALLOCATED	-	
0	1	x0	011x0	UNALLOCATED	-	
0	1	x0	01111	UNALLOCATED	-	
1	1	x0	01110	UNALLOCATED	-	
1	1	00	01100	FMAXNMV — Single-precision	FEAT_AdvSIMD	
1	1	00	01111	FMAXV — Single-precision	FEAT_AdvSIMD	
1	1	10	01100	FMINNMV — Single-precision	FEAT_AdvSIMD	
1	1	10	01111	FMINV — Single-precision	FEAT_AdvSIMD	

Advanced SIMD three different

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	Q	U	0	1	1	1	0	size	1	Rm					opcode					0	0	Rn					Rd				

Decode fields		Instruction Details	Feature
U	opcode		
0	0000	SADDL, SADDL2	FEAT_AdvSIMD
0	0001	SADDW, SADDW2	FEAT_AdvSIMD
0	0010	SSUBL, SSUBL2	FEAT_AdvSIMD
0	0011	SSUBW, SSUBW2	FEAT_AdvSIMD
0	0100	ADDHN, ADDHN2	FEAT_AdvSIMD
0	0101	SABAL, SABAL2	FEAT_AdvSIMD
0	0110	SUBHN, SUBHN2	FEAT_AdvSIMD
0	0111	SABDL, SABDL2	FEAT_AdvSIMD
0	1000	SMLAL, SMLAL2 (vector)	FEAT_AdvSIMD
0	1001	SQDMLAL, SQDMLAL2 (vector)	FEAT_AdvSIMD
0	1010	SMLSL, SMLSL2 (vector)	FEAT_AdvSIMD
0	1011	SQDMLSL, SQDMLSL2 (vector)	FEAT_AdvSIMD
0	1100	SMULL, SMULL2 (vector)	FEAT_AdvSIMD
0	1101	SQDMULL, SQDMULL2 (vector)	FEAT_AdvSIMD
0	1110	PMULL, PMULL2	FEAT_AdvSIMD
0	1111	UNALLOCATED	-
1	0000	UADDL, UADDL2	FEAT_AdvSIMD
1	0001	UADDW, UADDW2	FEAT_AdvSIMD
1	0010	USUBL, USUBL2	FEAT_AdvSIMD
1	0011	USUBW, USUBW2	FEAT_AdvSIMD
1	0100	RADDHN, RADDHN2	FEAT_AdvSIMD
1	0101	UABAL, UABAL2	FEAT_AdvSIMD
1	0110	RSUBHN, RSUBHN2	FEAT_AdvSIMD
1	0111	UABDL, UABDL2	FEAT_AdvSIMD
1	1xx1	UNALLOCATED	-
1	1000	UMLAL, UMLAL2 (vector)	FEAT_AdvSIMD
1	1010	UMLSL, UMLSL2 (vector)	FEAT_AdvSIMD
1	1100	UMULL, UMULL2 (vector)	FEAT_AdvSIMD
1	1110	UNALLOCATED	-

Advanced SIMD three same

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	Q	U	0	1	1	1	0	size	1	Rm				opcode				1	Rn				Rd								

U	Decode fields		Instruction Details	Feature
	size	opcode		
0		00000	SHADD	FEAT_AdvSIMD
0		00001	SQADD	FEAT_AdvSIMD
0		00010	SRHADD	FEAT_AdvSIMD
0		00100	SHSUB	FEAT_AdvSIMD
0		00101	SQSUB	FEAT_AdvSIMD
0		00110	CMGT (register)	FEAT_AdvSIMD
0		00111	CMGE (register)	FEAT_AdvSIMD
0		01000	SSHL	FEAT_AdvSIMD
0		01001	SQSHL (register)	FEAT_AdvSIMD
0		01010	SRSHL	FEAT_AdvSIMD
0		01011	SQRSHL	FEAT_AdvSIMD
0		01100	SMAX	FEAT_AdvSIMD
0		01101	SMIN	FEAT_AdvSIMD
0		01110	SABD	FEAT_AdvSIMD
0		01111	SABA	FEAT_AdvSIMD
0		10000	ADD (vector)	FEAT_AdvSIMD
0		10001	CMTST	FEAT_AdvSIMD
0		10010	MLA (vector)	FEAT_AdvSIMD
0		10011	MUL (vector)	FEAT_AdvSIMD
0		10100	SMAXP	FEAT_AdvSIMD
0		10101	SMINP	FEAT_AdvSIMD
0		10110	SQDMULH (vector)	FEAT_AdvSIMD
0		10111	ADDP (vector)	FEAT_AdvSIMD
0	x1	11101	UNALLOCATED	-
0	0x	11000	FMAXNM (vector)	FEAT_AdvSIMD
0	0x	11001	FMLA (vector)	FEAT_AdvSIMD
0	0x	11010	FADD (vector)	FEAT_AdvSIMD
0	0x	11011	FMULX	FEAT_AdvSIMD
0	0x	11100	FCMEQ (register)	FEAT_AdvSIMD
0	0x	11110	FMAX (vector)	FEAT_AdvSIMD
0	0x	11111	FRECPS	FEAT_AdvSIMD
0	00	00011	AND (vector)	FEAT_AdvSIMD
0	00	11101	FMLAL, FMLAL2 (vector) — FMLAL	FEAT_FHM
0	01	00011	BIC (vector, register)	FEAT_AdvSIMD
0	1x	11000	FMINNM (vector)	FEAT_AdvSIMD
0	1x	11001	FMLS (vector)	FEAT_AdvSIMD
0	1x	11010	FSUB (vector)	FEAT_AdvSIMD
0	1x	11011	FAMAX	FEAT_AdvSIMD && FEAT_FAMINMAX
0	1x	11100	UNALLOCATED	-
0	1x	11110	FMIN (vector)	FEAT_AdvSIMD
0	1x	11111	FRSQRTS	FEAT_AdvSIMD
0	10	00011	ORR (vector, register)	FEAT_AdvSIMD
0	10	11101	FMLSL, FMLSL2 (vector) — FMLSL	FEAT_FHM
0	11	00011	ORN (vector)	FEAT_AdvSIMD
1		00000	UHADD	FEAT_AdvSIMD

U	Decode fields		Instruction Details	Feature
	size	opcode		
1		00001	UQADD	FEAT_AdvSIMD
1		00010	URHADD	FEAT_AdvSIMD
1		00100	UHSUB	FEAT_AdvSIMD
1		00101	UQSUB	FEAT_AdvSIMD
1		00110	CMHI (register)	FEAT_AdvSIMD
1		00111	CMHS (register)	FEAT_AdvSIMD
1		01000	USHL	FEAT_AdvSIMD
1		01001	UQSHL (register)	FEAT_AdvSIMD
1		01010	URSHL	FEAT_AdvSIMD
1		01011	UQRSHL	FEAT_AdvSIMD
1		01100	UMAX	FEAT_AdvSIMD
1		01101	UMIN	FEAT_AdvSIMD
1		01110	UABD	FEAT_AdvSIMD
1		01111	UABA	FEAT_AdvSIMD
1		10000	SUB (vector)	FEAT_AdvSIMD
1		10001	CMEQ (register)	FEAT_AdvSIMD
1		10010	MLS (vector)	FEAT_AdvSIMD
1		10011	PMUL	FEAT_AdvSIMD
1		10100	UMAXP	FEAT_AdvSIMD
1		10101	UMINP	FEAT_AdvSIMD
1		10110	SQRDMULH (vector)	FEAT_AdvSIMD
1		10111	UNALLOCATED	-
1	x1	11001	UNALLOCATED	-
1	0x	11000	FMAXNMP (vector)	FEAT_AdvSIMD
1	0x	11010	FADDP (vector)	FEAT_AdvSIMD
1	0x	11011	FMUL (vector)	FEAT_AdvSIMD
1	0x	11100	FCMGE (register)	FEAT_AdvSIMD
1	0x	11101	FACGE	FEAT_AdvSIMD
1	0x	11110	FMAXP (vector)	FEAT_AdvSIMD
1	0x	11111	FDIV (vector)	FEAT_AdvSIMD
1	00	00011	EOR (vector)	FEAT_AdvSIMD
1	00	11001	FMLAL, FMLAL2 (vector) — FMLAL2	FEAT_FHM
1	01	00011	BSL	FEAT_AdvSIMD
1	1x	11000	FMINNMP (vector)	FEAT_AdvSIMD
1	1x	11010	FABD	FEAT_AdvSIMD
1	1x	11011	FAMIN	FEAT_AdvSIMD && FEAT_FAMINMAX
1	1x	11100	FCMGT (register)	FEAT_AdvSIMD
1	1x	11101	FACGT	FEAT_AdvSIMD
1	1x	11110	FMINP (vector)	FEAT_AdvSIMD
1	1x	11111	FSCALE	FEAT_FP8
1	10	00011	BIT	FEAT_AdvSIMD
1	10	11001	FMLSL, FMLSL2 (vector) — FMLSL2	FEAT_FHM
1	11	00011	BIF	FEAT_AdvSIMD

Advanced SIMD modified immediate

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	Q	op	0	1	1	1	1	0	0	0	0	0	a	b	c	cmode					o2	1	d	e	f	g	h	Rd			

Decode fields				Instruction Details		Feature
Q	op	cmode	o2			
	0	0xx0	0	MOVI — 32-bit shifted immediate		FEAT_AdvSIMD
	0	0xx1	0	ORR (vector, immediate) — 32-bit		FEAT_AdvSIMD
	0	10x0	0	MOVI — 16-bit shifted immediate		FEAT_AdvSIMD
	0	10x1	0	ORR (vector, immediate) — 16-bit		FEAT_AdvSIMD
	0	110x	0	MOVI — 32-bit shifting ones		FEAT_AdvSIMD
	0	1110	0	MOVI — 8-bit		FEAT_AdvSIMD
	0	1111	0	FMOV (vector, immediate) — Single-precision		FEAT_AdvSIMD
	0	1111	1	FMOV (vector, immediate) — Half-precision		FEAT_AdvSIMD && FEAT_FP16
	0	!= 1111	1	UNALLOCATED		-
	1		1	UNALLOCATED		-
	1	0xx0	0	MVNI — 32-bit shifted immediate		FEAT_AdvSIMD
	1	0xx1	0	BIC (vector, immediate) — 32-bit		FEAT_AdvSIMD
	1	10x0	0	MVNI — 16-bit shifted immediate		FEAT_AdvSIMD
	1	10x1	0	BIC (vector, immediate) — 16-bit		FEAT_AdvSIMD
	1	110x	0	MVNI — 32-bit shifting ones		FEAT_AdvSIMD
0	1	1110	0	MOVI — 64-bit scalar		FEAT_AdvSIMD
0	1	1111	0	UNALLOCATED		-
1	1	1110	0	MOVI — 64-bit vector		FEAT_AdvSIMD
1	1	1111	0	FMOV (vector, immediate) — Double-precision		FEAT_AdvSIMD

Advanced SIMD shift by immediate

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	Q	U	0	1	1	1	1	0	!= 0000				immb				opcode				1	Rn				Rd					
immh																															

The following constraints also apply to this encoding: immh != '0000'

U	Decode fields		Instruction Details	Feature
	immh	opcode		
	!= 0000	0xxx1	UNALLOCATED	-
	!= 0000	1x110	UNALLOCATED	-
	!= 0000	101x1	UNALLOCATED	-
	!= 0000	110xx	UNALLOCATED	-
	!= 0000	11101	UNALLOCATED	-
0	!= 0000	00000	SSHR	FEAT_AdvSIMD
0	!= 0000	00010	SSRA	FEAT_AdvSIMD
0	!= 0000	00100	SRSR	FEAT_AdvSIMD
0	!= 0000	00110	SRSRA	FEAT_AdvSIMD
0	!= 0000	01x00	UNALLOCATED	-
0	!= 0000	01010	SHL	FEAT_AdvSIMD
0	!= 0000	01110	SQSHL (immediate)	FEAT_AdvSIMD
0	!= 0000	10000	SHRN, SHRN2	FEAT_AdvSIMD
0	!= 0000	10001	RSHRN, RSHRN2	FEAT_AdvSIMD
0	!= 0000	10010	SQSHRN, SQSHRN2	FEAT_AdvSIMD
0	!= 0000	10011	SQRSHRN, SQRSHRN2	FEAT_AdvSIMD
0	!= 0000	10100	SSHLL, SSHLL2	FEAT_AdvSIMD
0	!= 0000	11100	SCVTF (vector, fixed-point)	FEAT_AdvSIMD
0	!= 0000	11111	FCVTZS (vector, fixed-point)	FEAT_AdvSIMD
1	!= 0000	00000	USHR	FEAT_AdvSIMD
1	!= 0000	00010	USRA	FEAT_AdvSIMD
1	!= 0000	00100	URSHR	FEAT_AdvSIMD
1	!= 0000	00110	URSRA	FEAT_AdvSIMD
1	!= 0000	01000	SRI	FEAT_AdvSIMD
1	!= 0000	01010	SLI	FEAT_AdvSIMD
1	!= 0000	01100	SQSHLU	FEAT_AdvSIMD
1	!= 0000	01110	UQSHL (immediate)	FEAT_AdvSIMD
1	!= 0000	10000	SQSHRUN, SQSHRUN2	FEAT_AdvSIMD
1	!= 0000	10001	SQRSHRUN, SQRSHRUN2	FEAT_AdvSIMD
1	!= 0000	10010	UQSHRN, UQSHRN2	FEAT_AdvSIMD
1	!= 0000	10011	UQRSHRN, UQRSHRN2	FEAT_AdvSIMD
1	!= 0000	10100	USHLL, USHLL2	FEAT_AdvSIMD
1	!= 0000	11100	UCVTF (vector, fixed-point)	FEAT_AdvSIMD
1	!= 0000	11111	FCVTZU (vector, fixed-point)	FEAT_AdvSIMD

Advanced SIMD vector x indexed element

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	Q	U	0	1	1	1	1	size	L	M	Rm				opcode				H	0	Rn				Rd						

Decode fields				Instruction Details	Feature
Q	U	size	opcode		
	0		0010	SMLAL, SMLAL2 (by element)	FEAT_AdvSIMD
	0		0011	SQDMLAL, SQDMLAL2 (by element)	FEAT_AdvSIMD
	0		0110	SMLSL, SMLSL2 (by element)	FEAT_AdvSIMD
	0		0111	SQDMLSL, SQDMLSL2 (by element)	FEAT_AdvSIMD
	0		1000	MUL (by element)	FEAT_AdvSIMD
	0		1010	SMULL, SMULL2 (by element)	FEAT_AdvSIMD
	0		1011	SQDMULL, SQDMULL2 (by element)	FEAT_AdvSIMD
	0		1100	SQDMULH (by element)	FEAT_AdvSIMD
	0		1101	SQRDMULH (by element)	FEAT_AdvSIMD
	0		1110	SDOT (by element)	FEAT_DotProd
	0	00	0000	FDOT (8-bit floating-point to single-precision, by element)	FEAT_FP8DOT4
	0	00	0001	FMLA (by element) — Vector, half-precision	FEAT_AdvSIMD && FEAT_FP16
	0	00	0101	FMLS (by element) — Vector, half-precision	FEAT_AdvSIMD && FEAT_FP16
	0	00	1001	FMUL (by element) — Vector, half-precision	FEAT_AdvSIMD && FEAT_FP16
	0	00	1111	SUDOT (by element)	FEAT_I8MM
	0	01	0x01	UNALLOCATED	-
	0	01	0000	FDOT (8-bit floating-point to half-precision, by element)	FEAT_FP8DOT2
	0	01	1001	FDOT (half-precision to single-precision, by element)	FEAT_F16F32DOT
	0	01	1111	BFDOT (by element)	FEAT_BF16
	0	1x	0001	FMLA (by element) — Vector, single-precision and double-precision	FEAT_AdvSIMD
	0	1x	0101	FMLS (by element) — Vector, single-precision and double-precision	FEAT_AdvSIMD
	0	1x	1001	FMUL (by element) — Vector, single-precision and double-precision	FEAT_AdvSIMD
	0	10	0000	FMLAL, FMLAL2 (by element) — FMLAL	FEAT_FHM
	0	10	0100	FMLSL, FMLSL2 (by element) — FMLSL	FEAT_FHM
	0	10	1111	USDOT (by element)	FEAT_I8MM
	0	11	1111	BFMLALB, BFMLALT (by element)	FEAT_BF16
	0	!= 10	0100	UNALLOCATED	-
	1		0xx1	FCMLA (by element)	FEAT_FCMA
	1		0000	MLA (by element)	FEAT_AdvSIMD
	1		0010	UMLAL, UMLAL2 (by element)	FEAT_AdvSIMD
	1		0100	MLS (by element)	FEAT_AdvSIMD
	1		0110	UMLSL, UMLSL2 (by element)	FEAT_AdvSIMD
	1		1010	UMULL, UMULL2 (by element)	FEAT_AdvSIMD
	1		1011	UNALLOCATED	-
	1		1101	SQRDMLAH (by element)	FEAT_RDM
	1		1110	UDOT (by element)	FEAT_DotProd
	1		1111	SQRDMLSH (by element)	FEAT_RDM
	1	0x	1100	UNALLOCATED	-
	1	00	1001	FMULX (by element) — Vector, half-precision	FEAT_AdvSIMD && FEAT_FP16
	1	01	1001	UNALLOCATED	-
	1	1x	1001	FMULX (by element) — Vector, single-precision and double-precision	FEAT_AdvSIMD
	1	10	1000	FMLAL, FMLAL2 (by element) — FMLAL2	FEAT_FHM

Decode fields				Instruction Details	Feature
Q	U	size	opcode		
	1	10	1100	FMLSL, FMLSL2 (by element) — FMLSL2	FEAT_FHM
	1	11	1x00	UNALLOCATED	-
0	0	11	0000	FMLALB, FMLALT (by element) — FMLALB	FEAT_FP8FMA
0	1	00	1000	FMLALLBB, FMLALLBT, FMLALLTB, FMLALLTT (by element) — FMLALLBB	FEAT_FP8FMA
0	1	01	1000	FMLALLBB, FMLALLBT, FMLALLTB, FMLALLTT (by element) — FMLALLBT	FEAT_FP8FMA
1	0	11	0000	FMLALB, FMLALT (by element) — FMLALT	FEAT_FP8FMA
1	1	00	1000	FMLALLBB, FMLALLBT, FMLALLTB, FMLALLTT (by element) — FMLALLTB	FEAT_FP8FMA
1	1	01	1000	FMLALLBB, FMLALLBT, FMLALLTB, FMLALLTT (by element) — FMLALLTT	FEAT_FP8FMA

Cryptographic three-register, imm2

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	1	1	1	0	0	1	0	Rm					1	0	imm2	opcode	Rn					Rd						

Decode fields		Instruction Details	Feature
opcode			
00	SM3TT1A		FEAT_SM3
01	SM3TT1B		FEAT_SM3
10	SM3TT2A		FEAT_SM3
11	SM3TT2B		FEAT_SM3

Cryptographic three-register SHA 512

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	1	1	1	0	0	1	1	Rm				1	0	0	0	opcode				Rn				Rd				

Decode fields		Instruction Details	Feature
O	opcode		
0	00	SHA512H	FEAT_SHA512
0	01	SHA512H2	FEAT_SHA512
0	10	SHA512SU1	FEAT_SHA512
0	11	RAX1	FEAT_SHA3
1	00	SM3PARTW1	FEAT_SM3
1	01	SM3PARTW2	FEAT_SM3
1	10	SM4EKEY	FEAT_SM4
1	11	UNALLOCATED	-

Cryptographic four-register

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	1	1	1	0	0	Op0	Rm			0	Ra			Rn						Rd								

Decode fields	Instruction Details	Feature
Op0		
00	EOR3	FEAT_SHA3
01	BCAX	FEAT_SHA3
10	SM3SS1	FEAT_SM3
11	UNALLOCATED	-

Cryptographic two-register SHA 512

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	1	1	1	0	1	1	0	0	0	0	0	0	1	0	0	0	opcode	Rn					Rd					

Decode fields	Instruction Details	Feature
opcode		
00	SHA512SU0	FEAT_SHA512
01	SM4E	FEAT_SM4
1x	UNALLOCATED	-

Conversion between floating-point and fixed-point

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sf	0	S	1	1	1	1	0	ftype	0	rmode	opcode																				

sf	S	Decode fields			Instruction Details	Feature
		ftype	rmode	opcode		
	0			1xx	UNALLOCATED	-
	0	10		0xx	UNALLOCATED	-
	0	!= 10	x1	01x	UNALLOCATED	-
	0	!= 10	0x	00x	UNALLOCATED	-
	0	!= 10	10	0xx	UNALLOCATED	-
	1				UNALLOCATED	-
0	0	00	00	010	SCVTF (scalar, fixed-point) — 32-bit to single-precision	FEAT_FP
0	0	00	00	011	UCVTF (scalar, fixed-point) — 32-bit to single-precision	FEAT_FP
0	0	00	11	000	FCVTZS (scalar, fixed-point) — Single-precision to 32-bit	FEAT_FP
0	0	00	11	001	FCVTZU (scalar, fixed-point) — Single-precision to 32-bit	FEAT_FP
0	0	01	00	010	SCVTF (scalar, fixed-point) — 32-bit to double-precision	FEAT_FP
0	0	01	00	011	UCVTF (scalar, fixed-point) — 32-bit to double-precision	FEAT_FP
0	0	01	11	000	FCVTZS (scalar, fixed-point) — Double-precision to 32-bit	FEAT_FP
0	0	01	11	001	FCVTZU (scalar, fixed-point) — Double-precision to 32-bit	FEAT_FP
0	0	11	00	010	SCVTF (scalar, fixed-point) — 32-bit to half-precision	FEAT_FP16
0	0	11	00	011	UCVTF (scalar, fixed-point) — 32-bit to half-precision	FEAT_FP16
0	0	11	11	000	FCVTZS (scalar, fixed-point) — Half-precision to 32-bit	FEAT_FP16
0	0	11	11	001	FCVTZU (scalar, fixed-point) — Half-precision to 32-bit	FEAT_FP16
1	0	00	00	010	SCVTF (scalar, fixed-point) — 64-bit to single-precision	FEAT_FP
1	0	00	00	011	UCVTF (scalar, fixed-point) — 64-bit to single-precision	FEAT_FP
1	0	00	11	000	FCVTZS (scalar, fixed-point) — Single-precision to 64-bit	FEAT_FP
1	0	00	11	001	FCVTZU (scalar, fixed-point) — Single-precision to 64-bit	FEAT_FP
1	0	01	00	010	SCVTF (scalar, fixed-point) — 64-bit to double-precision	FEAT_FP
1	0	01	00	011	UCVTF (scalar, fixed-point) — 64-bit to double-precision	FEAT_FP
1	0	01	11	000	FCVTZS (scalar, fixed-point) — Double-precision to 64-bit	FEAT_FP
1	0	01	11	001	FCVTZU (scalar, fixed-point) — Double-precision to 64-bit	FEAT_FP
1	0	11	00	010	SCVTF (scalar, fixed-point) — 64-bit to half-precision	FEAT_FP16
1	0	11	00	011	UCVTF (scalar, fixed-point) — 64-bit to half-precision	FEAT_FP16
1	0	11	11	000	FCVTZS (scalar, fixed-point) — Half-precision to 64-bit	FEAT_FP16
1	0	11	11	001	FCVTZU (scalar, fixed-point) — Half-precision to 64-bit	FEAT_FP16

Conversion between floating-point and integer

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sf	0	S	1	1	1	1	0	ftype		1	rmode		opcode			0	0	0	0	0	0	Rn						Rd			

sf	Decode fields				Instruction Details	Feature
	S	f _{type}	r _{mode}	opcode		
	0	10	x0		UNALLOCATED	-
	0	10	x1	!= 11x0xx	UNALLOCATED	-
	0	10	x1	10x	UNALLOCATED	-
	0	11	x1	11x	UNALLOCATED	-
	0	!= 10	01	10x	UNALLOCATED	-
	1				UNALLOCATED	-
0	0	x0	01	11x	UNALLOCATED	-
0	0	00	00	000	FCVTNS (scalar) — Single-precision to 32-bit	FEAT_FP
0	0	00	00	001	FCVTNU (scalar) — Single-precision to 32-bit	FEAT_FP
0	0	00	00	010	SCVTF (scalar, integer) — 32-bit to single-precision	FEAT_FP
0	0	00	00	011	UCVTF (scalar, integer) — 32-bit to single-precision	FEAT_FP
0	0	00	00	100	FCVTAS (scalar) — Single-precision to 32-bit	FEAT_FP
0	0	00	00	101	FCVTAU (scalar) — Single-precision to 32-bit	FEAT_FP
0	0	00	00	110	FMOV (general) — Single-precision to 32-bit	FEAT_FP
0	0	00	00	111	FMOV (general) — 32-bit to single-precision	FEAT_FP
0	0	00	01	000	FCVTPS (scalar) — Single-precision to 32-bit	FEAT_FP
0	0	00	01	001	FCVTPU (scalar) — Single-precision to 32-bit	FEAT_FP
0	0	00	1x	1xx	UNALLOCATED	-
0	0	00	10	000	FCVTMS (scalar) — Single-precision to 32-bit	FEAT_FP
0	0	00	10	001	FCVTMU (scalar) — Single-precision to 32-bit	FEAT_FP
0	0	00	11	000	FCVTZS (scalar, integer) — Single-precision to 32-bit	FEAT_FP
0	0	00	11	001	FCVTZU (scalar, integer) — Single-precision to 32-bit	FEAT_FP
0	0	00	!= 00	01x	UNALLOCATED	-
0	0	01	0x	11x	UNALLOCATED	-
0	0	01	00	000	FCVTNS (scalar) — Double-precision to 32-bit	FEAT_FP
0	0	01	00	001	FCVTNU (scalar) — Double-precision to 32-bit	FEAT_FP
0	0	01	00	010	SCVTF (scalar, integer) — 32-bit to double-precision	FEAT_FP
0	0	01	00	011	UCVTF (scalar, integer) — 32-bit to double-precision	FEAT_FP
0	0	01	00	100	FCVTAS (scalar) — Double-precision to 32-bit	FEAT_FP
0	0	01	00	101	FCVTAU (scalar) — Double-precision to 32-bit	FEAT_FP
0	0	01	01	000	FCVTPS (scalar) — Double-precision to 32-bit	FEAT_FP
0	0	01	01	001	FCVTPU (scalar) — Double-precision to 32-bit	FEAT_FP
0	0	01	01	010	FCVTNS (scalar SIMD&FP) — Double-precision to 32-bit	FEAT_FPRCVT
0	0	01	01	011	FCVTNU (scalar SIMD&FP) — Double-precision to 32-bit	FEAT_FPRCVT
0	0	01	10	000	FCVTMS (scalar) — Double-precision to 32-bit	FEAT_FP
0	0	01	10	001	FCVTMU (scalar) — Double-precision to 32-bit	FEAT_FP
0	0	01	10	010	FCVTPS (scalar SIMD&FP) — Double-precision to 32-bit	FEAT_FPRCVT
0	0	01	10	011	FCVTPU (scalar SIMD&FP) — Double-precision to 32-bit	FEAT_FPRCVT
0	0	01	10	100	FCVTMS (scalar SIMD&FP) — Double-precision to 32-bit	FEAT_FPRCVT
0	0	01	10	101	FCVTMU (scalar SIMD&FP) — Double-precision to 32-bit	FEAT_FPRCVT
0	0	01	10	110	FCVTZS (scalar SIMD&FP) — Double-precision to 32-bit	FEAT_FPRCVT
0	0	01	10	111	FCVTZU (scalar SIMD&FP) — Double-precision to 32-bit	FEAT_FPRCVT
0	0	01	11	000	FCVTZS (scalar, integer) — Double-precision to 32-bit	FEAT_FP
0	0	01	11	001	FCVTZU (scalar, integer) — Double-precision to 32-bit	FEAT_FP
0	0	01	11	010	FCVTAS (scalar SIMD&FP) — Double-precision to 32-bit	FEAT_FPRCVT
0	0	01	11	011	FCVTAU (scalar SIMD&FP) — Double-precision to 32-bit	FEAT_FPRCVT

sf	S	Decode fields			Instruction Details	Feature
		f _{type}	r _{mode}	opcode		
0	0	01	11	100	SCVTF (scalar SIMD&FP) — 32-bit to double-precision	FEAT_FPRCVT
0	0	01	11	101	UCVTF (scalar SIMD&FP) — 32-bit to double-precision	FEAT_FPRCVT
0	0	01	11	110	FJCVTZS	FEAT_JSCVT
0	0	01	11	111	UNALLOCATED	-
0	0	10	11	11x	UNALLOCATED	-
0	0	11	00	000	FCVTNS (scalar) — Half-precision to 32-bit	FEAT_FP16
0	0	11	00	001	FCVTNU (scalar) — Half-precision to 32-bit	FEAT_FP16
0	0	11	00	010	SCVTF (scalar, integer) — 32-bit to half-precision	FEAT_FP16
0	0	11	00	011	UCVTF (scalar, integer) — 32-bit to half-precision	FEAT_FP16
0	0	11	00	100	FCVTAS (scalar) — Half-precision to 32-bit	FEAT_FP16
0	0	11	00	101	FCVTAU (scalar) — Half-precision to 32-bit	FEAT_FP16
0	0	11	00	110	FMOV (general) — Half-precision to 32-bit	FEAT_FP16
0	0	11	00	111	FMOV (general) — 32-bit to half-precision	FEAT_FP16
0	0	11	01	000	FCVTPS (scalar) — Half-precision to 32-bit	FEAT_FP16
0	0	11	01	001	FCVTPU (scalar) — Half-precision to 32-bit	FEAT_FP16
0	0	11	01	010	FCVTNS (scalar SIMD&FP) — Half-precision to 32-bit	FEAT_FPRCVT
0	0	11	01	011	FCVTNU (scalar SIMD&FP) — Half-precision to 32-bit	FEAT_FPRCVT
0	0	11	10	000	FCVTMS (scalar) — Half-precision to 32-bit	FEAT_FP16
0	0	11	10	001	FCVTMU (scalar) — Half-precision to 32-bit	FEAT_FP16
0	0	11	10	010	FCVTPS (scalar SIMD&FP) — Half-precision to 32-bit	FEAT_FPRCVT
0	0	11	10	011	FCVTPU (scalar SIMD&FP) — Half-precision to 32-bit	FEAT_FPRCVT
0	0	11	10	100	FCVTMS (scalar SIMD&FP) — Half-precision to 32-bit	FEAT_FPRCVT
0	0	11	10	101	FCVTMU (scalar SIMD&FP) — Half-precision to 32-bit	FEAT_FPRCVT
0	0	11	10	110	FCVTZS (scalar SIMD&FP) — Half-precision to 32-bit	FEAT_FPRCVT
0	0	11	10	111	FCVTZU (scalar SIMD&FP) — Half-precision to 32-bit	FEAT_FPRCVT
0	0	11	11	000	FCVTZS (scalar, integer) — Half-precision to 32-bit	FEAT_FP16
0	0	11	11	001	FCVTZU (scalar, integer) — Half-precision to 32-bit	FEAT_FP16
0	0	11	11	010	FCVTAS (scalar SIMD&FP) — Half-precision to 32-bit	FEAT_FPRCVT
0	0	11	11	011	FCVTAU (scalar SIMD&FP) — Half-precision to 32-bit	FEAT_FPRCVT
0	0	11	11	100	SCVTF (scalar SIMD&FP) — 32-bit to half-precision	FEAT_FPRCVT
0	0	11	11	101	UCVTF (scalar SIMD&FP) — 32-bit to half-precision	FEAT_FPRCVT
1	0	x0	11	11x	UNALLOCATED	-
1	0	00	0x	11x	UNALLOCATED	-
1	0	00	00	000	FCVTNS (scalar) — Single-precision to 64-bit	FEAT_FP
1	0	00	00	001	FCVTNU (scalar) — Single-precision to 64-bit	FEAT_FP
1	0	00	00	010	SCVTF (scalar, integer) — 64-bit to single-precision	FEAT_FP
1	0	00	00	011	UCVTF (scalar, integer) — 64-bit to single-precision	FEAT_FP
1	0	00	00	100	FCVTAS (scalar) — Single-precision to 64-bit	FEAT_FP
1	0	00	00	101	FCVTAU (scalar) — Single-precision to 64-bit	FEAT_FP
1	0	00	01	000	FCVTPS (scalar) — Single-precision to 64-bit	FEAT_FP
1	0	00	01	001	FCVTPU (scalar) — Single-precision to 64-bit	FEAT_FP
1	0	00	01	010	FCVTNS (scalar SIMD&FP) — Single-precision to 64-bit	FEAT_FPRCVT
1	0	00	01	011	FCVTNU (scalar SIMD&FP) — Single-precision to 64-bit	FEAT_FPRCVT
1	0	00	10	000	FCVTMS (scalar) — Single-precision to 64-bit	FEAT_FP
1	0	00	10	001	FCVTMU (scalar) — Single-precision to 64-bit	FEAT_FP
1	0	00	10	010	FCVTPS (scalar SIMD&FP) — Single-precision to 64-bit	FEAT_FPRCVT

Decode fields					Instruction Details	Feature
sf	S	f _{type}	r _{mode}	opcode		
1	0	00	10	011	FCVTPU (scalar SIMD&FP) — Single-precision to 64-bit	FEAT_FPRCVT
1	0	00	10	100	FCVTMS (scalar SIMD&FP) — Single-precision to 64-bit	FEAT_FPRCVT
1	0	00	10	101	FCVTMU (scalar SIMD&FP) — Single-precision to 64-bit	FEAT_FPRCVT
1	0	00	10	110	FCVTZS (scalar SIMD&FP) — Single-precision to 64-bit	FEAT_FPRCVT
1	0	00	10	111	FCVTZU (scalar SIMD&FP) — Single-precision to 64-bit	FEAT_FPRCVT
1	0	00	11	000	FCVTZS (scalar, integer) — Single-precision to 64-bit	FEAT_FP
1	0	00	11	001	FCVTZU (scalar, integer) — Single-precision to 64-bit	FEAT_FP
1	0	00	11	010	FCVTAS (scalar SIMD&FP) — Single-precision to 64-bit	FEAT_FPRCVT
1	0	00	11	011	FCVTAU (scalar SIMD&FP) — Single-precision to 64-bit	FEAT_FPRCVT
1	0	00	11	100	SCVTF (scalar SIMD&FP) — 64-bit to single-precision	FEAT_FPRCVT
1	0	00	11	101	UCVTF (scalar SIMD&FP) — 64-bit to single-precision	FEAT_FPRCVT
1	0	01	00	000	FCVTNS (scalar) — Double-precision to 64-bit	FEAT_FP
1	0	01	00	001	FCVTNU (scalar) — Double-precision to 64-bit	FEAT_FP
1	0	01	00	010	SCVTF (scalar, integer) — 64-bit to double-precision	FEAT_FP
1	0	01	00	011	UCVTF (scalar, integer) — 64-bit to double-precision	FEAT_FP
1	0	01	00	100	FCVTAS (scalar) — Double-precision to 64-bit	FEAT_FP
1	0	01	00	101	FCVTAU (scalar) — Double-precision to 64-bit	FEAT_FP
1	0	01	00	110	FMOV (general) — Double-precision to 64-bit	FEAT_FP
1	0	01	00	111	FMOV (general) — 64-bit to double-precision	FEAT_FP
1	0	01	01	x1x	UNALLOCATED	-
1	0	01	01	000	FCVTPS (scalar) — Double-precision to 64-bit	FEAT_FP
1	0	01	01	001	FCVTPU (scalar) — Double-precision to 64-bit	FEAT_FP
1	0	01	1x	1xx	UNALLOCATED	-
1	0	01	1x	!= 00x01x	UNALLOCATED	-
1	0	01	10	000	FCVTMS (scalar) — Double-precision to 64-bit	FEAT_FP
1	0	01	10	001	FCVTMU (scalar) — Double-precision to 64-bit	FEAT_FP
1	0	01	11	000	FCVTZS (scalar, integer) — Double-precision to 64-bit	FEAT_FP
1	0	01	11	001	FCVTZU (scalar, integer) — Double-precision to 64-bit	FEAT_FP
1	0	10	01	110	FMOV (general) — Top half of 128-bit to 64-bit	FEAT_FP
1	0	10	01	111	FMOV (general) — 64-bit to top half of 128-bit	FEAT_FP
1	0	11	00	000	FCVTNS (scalar) — Half-precision to 64-bit	FEAT_FP16
1	0	11	00	001	FCVTNU (scalar) — Half-precision to 64-bit	FEAT_FP16
1	0	11	00	010	SCVTF (scalar, integer) — 64-bit to half-precision	FEAT_FP16
1	0	11	00	011	UCVTF (scalar, integer) — 64-bit to half-precision	FEAT_FP16
1	0	11	00	100	FCVTAS (scalar) — Half-precision to 64-bit	FEAT_FP16
1	0	11	00	101	FCVTAU (scalar) — Half-precision to 64-bit	FEAT_FP16
1	0	11	00	110	FMOV (general) — Half-precision to 64-bit	FEAT_FP16
1	0	11	00	111	FMOV (general) — 64-bit to half-precision	FEAT_FP16
1	0	11	01	000	FCVTPS (scalar) — Half-precision to 64-bit	FEAT_FP16
1	0	11	01	001	FCVTPU (scalar) — Half-precision to 64-bit	FEAT_FP16
1	0	11	01	010	FCVTNS (scalar SIMD&FP) — Half-precision to 64-bit	FEAT_FPRCVT
1	0	11	01	011	FCVTNU (scalar SIMD&FP) — Half-precision to 64-bit	FEAT_FPRCVT
1	0	11	10	000	FCVTMS (scalar) — Half-precision to 64-bit	FEAT_FP16
1	0	11	10	001	FCVTMU (scalar) — Half-precision to 64-bit	FEAT_FP16
1	0	11	10	010	FCVTPS (scalar SIMD&FP) — Half-precision to 64-bit	FEAT_FPRCVT
1	0	11	10	011	FCVTPU (scalar SIMD&FP) — Half-precision to 64-bit	FEAT_FPRCVT

Decode fields					Instruction Details	Feature
sf	S	ftype	rmode	opcode		
1	0	11	10	100	FCVTMS (scalar SIMD&FP) — Half-precision to 64-bit	FEAT_FPRCVT
1	0	11	10	101	FCVTMU (scalar SIMD&FP) — Half-precision to 64-bit	FEAT_FPRCVT
1	0	11	10	110	FCVTZS (scalar SIMD&FP) — Half-precision to 64-bit	FEAT_FPRCVT
1	0	11	10	111	FCVTZU (scalar SIMD&FP) — Half-precision to 64-bit	FEAT_FPRCVT
1	0	11	11	000	FCVTZS (scalar, integer) — Half-precision to 64-bit	FEAT_FP16
1	0	11	11	001	FCVTZU (scalar, integer) — Half-precision to 64-bit	FEAT_FP16
1	0	11	11	010	FCVTAS (scalar SIMD&FP) — Half-precision to 64-bit	FEAT_FPRCVT
1	0	11	11	011	FCVTAU (scalar SIMD&FP) — Half-precision to 64-bit	FEAT_FPRCVT
1	0	11	11	100	SCVTF (scalar SIMD&FP) — 64-bit to half-precision	FEAT_FPRCVT
1	0	11	11	101	UCVTF (scalar SIMD&FP) — 64-bit to half-precision	FEAT_FPRCVT

Floating-point data-processing (1 source)

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
M	0	S	1	1	1	1	0	ftype	1	opcode							1	0	0	0	0	Rn					Rd				

M	S	Decode fields		Instruction Details	Feature
		f _{type}	opcode		
0	0		1xxxxx	UNALLOCATED	-
0	0	0x	0101xx	UNALLOCATED	-
0	0	0x	011xxx	UNALLOCATED	-
0	0	00	000000	FMOV (register) — Single-precision	FEAT_FP
0	0	00	000001	FABS (scalar) — Single-precision	FEAT_FP
0	0	00	000010	FNEG (scalar) — Single-precision	FEAT_FP
0	0	00	000011	FSQRT (scalar) — Single-precision	FEAT_FP
0	0	00	0001x0	UNALLOCATED	-
0	0	00	000101	FCVT — Single-precision to double-precision	FEAT_FP
0	0	00	000111	FCVT — Single-precision to half-precision	FEAT_FP
0	0	00	001000	FRINTN (scalar) — Single-precision	FEAT_FP
0	0	00	001001	FRINTP (scalar) — Single-precision	FEAT_FP
0	0	00	001010	FRINTM (scalar) — Single-precision	FEAT_FP
0	0	00	001011	FRINTZ (scalar) — Single-precision	FEAT_FP
0	0	00	001100	FRINTA (scalar) — Single-precision	FEAT_FP
0	0	00	001110	FRINTX (scalar) — Single-precision	FEAT_FP
0	0	00	001111	FRINTI (scalar) — Single-precision	FEAT_FP
0	0	00	010000	FRINT32Z (scalar) — Single-precision	FEAT_FRINTTS
0	0	00	010001	FRINT32X (scalar) — Single-precision	FEAT_FRINTTS
0	0	00	010010	FRINT64Z (scalar) — Single-precision	FEAT_FRINTTS
0	0	00	010011	FRINT64X (scalar) — Single-precision	FEAT_FRINTTS
0	0	01	000000	FMOV (register) — Double-precision	FEAT_FP
0	0	01	000001	FABS (scalar) — Double-precision	FEAT_FP
0	0	01	000010	FNEG (scalar) — Double-precision	FEAT_FP
0	0	01	000011	FSQRT (scalar) — Double-precision	FEAT_FP
0	0	01	000100	FCVT — Double-precision to single-precision	FEAT_FP
0	0	01	000101	UNALLOCATED	-
0	0	01	000110	BFCVT	FEAT_BF16
0	0	01	000111	FCVT — Double-precision to half-precision	FEAT_FP
0	0	01	001000	FRINTN (scalar) — Double-precision	FEAT_FP
0	0	01	001001	FRINTP (scalar) — Double-precision	FEAT_FP
0	0	01	001010	FRINTM (scalar) — Double-precision	FEAT_FP
0	0	01	001011	FRINTZ (scalar) — Double-precision	FEAT_FP
0	0	01	001100	FRINTA (scalar) — Double-precision	FEAT_FP
0	0	01	001110	FRINTX (scalar) — Double-precision	FEAT_FP
0	0	01	001111	FRINTI (scalar) — Double-precision	FEAT_FP
0	0	01	010000	FRINT32Z (scalar) — Double-precision	FEAT_FRINTTS
0	0	01	010001	FRINT32X (scalar) — Double-precision	FEAT_FRINTTS
0	0	01	010010	FRINT64Z (scalar) — Double-precision	FEAT_FRINTTS
0	0	01	010011	FRINT64X (scalar) — Double-precision	FEAT_FRINTTS
0	0	10	0xxxxx	UNALLOCATED	-
0	0	11	000000	FMOV (register) — Half-precision	FEAT_FP16
0	0	11	000001	FABS (scalar) — Half-precision	FEAT_FP16
0	0	11	000010	FNEG (scalar) — Half-precision	FEAT_FP16
0	0	11	000011	FSQRT (scalar) — Half-precision	FEAT_FP16

Decode fields				Instruction Details		Feature
M	S	fctype	opcode			
0	0	11	000100	FCVT — Half-precision to single-precision		FEAT_FP
0	0	11	000101	FCVT — Half-precision to double-precision		FEAT_FP
0	0	11	00011x	UNALLOCATED		-
0	0	11	001000	FRINTN (scalar) — Half-precision		FEAT_FP16
0	0	11	001001	FRINTP (scalar) — Half-precision		FEAT_FP16
0	0	11	001010	FRINTM (scalar) — Half-precision		FEAT_FP16
0	0	11	001011	FRINTZ (scalar) — Half-precision		FEAT_FP16
0	0	11	001100	FRINTA (scalar) — Half-precision		FEAT_FP16
0	0	11	001110	FRINTX (scalar) — Half-precision		FEAT_FP16
0	0	11	001111	FRINTI (scalar) — Half-precision		FEAT_FP16
0	0	11	01xxxx	UNALLOCATED		-
0	0	!= 10	001101	UNALLOCATED		-
0	1			UNALLOCATED		-
1				UNALLOCATED		-

Floating-point compare

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
M	0	S	1	1	1	1	0	fctype	1				Rm				op	1	0	0	0				Rn					opcode2	

Decode fields				Instruction Details		Feature
M	S	fctype	op	opcode2		
0	0		00	xx01x	UNALLOCATED	
0	0		00	xx1xx	UNALLOCATED	
0	0		00	!= xx000xx001	UNALLOCATED	
0	0		!= 00		UNALLOCATED	
0	0	00	00	00000	FCMP — Single-precision	
0	0	00	00	01000	FCMP — Single-precision, zero	
0	0	00	00	10000	FCMPE — Single-precision	
0	0	00	00	11000	FCMPE — Single-precision, zero	
0	0	01	00	00000	FCMP — Double-precision	
0	0	01	00	01000	FCMP — Double-precision, zero	
0	0	01	00	10000	FCMPE — Double-precision	
0	0	01	00	11000	FCMPE — Double-precision, zero	
0	0	10	00	xx000	UNALLOCATED	
0	0	11	00	00000	FCMP — Half-precision	
0	0	11	00	01000	FCMP — Half-precision, zero	
0	0	11	00	10000	FCMPE — Half-precision	
0	0	11	00	11000	FCMPE — Half-precision, zero	
0	1				UNALLOCATED	
1					UNALLOCATED	

Floating-point immediate

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
M	0	S	1	1	1	1	0	fctype	1											1	0	0									Rd

Decode fields				Instruction Details	Feature
M	S	ftype	imm5		
0	0		!= 00000	UNALLOCATED	-
0	0	00	00000	FMOV (scalar, immediate) — Single-precision	FEAT_FP
0	0	01	00000	FMOV (scalar, immediate) — Double-precision	FEAT_FP
0	0	10	00000	UNALLOCATED	-
0	0	11	00000	FMOV (scalar, immediate) — Half-precision	FEAT_FP16
0	1			UNALLOCATED	-
1				UNALLOCATED	-

Floating-point conditional compare

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
M	0	S	1	1	1	1	0	ftype	1				Rm						cond		0	1			Rn		op			nzcv	

Decode fields				Instruction Details	Feature
M	S	ftype	op		
0	0	00	0	FCCMP — Single-precision	FEAT_FP
0	0	00	1	FCCMPE — Single-precision	FEAT_FP
0	0	01	0	FCCMP — Double-precision	FEAT_FP
0	0	01	1	FCCMPE — Double-precision	FEAT_FP
0	0	10		UNALLOCATED	-
0	0	11	0	FCCMP — Half-precision	FEAT_FP16
0	0	11	1	FCCMPE — Half-precision	FEAT_FP16
0	1			UNALLOCATED	-
1				UNALLOCATED	-

Floating-point data-processing (2 source)

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
M	0	S	1	1	1	1	0	ftype	1				Rm						opcode		1	0			Rn					Rd	

Decode fields				Instruction Details	Feature
M	S	f _{type}	opcode		
0	0	00	0000	FMUL (scalar) — Single-precision	FEAT_FP
0	0	00	0001	FDIV (scalar) — Single-precision	FEAT_FP
0	0	00	0010	FADD (scalar) — Single-precision	FEAT_FP
0	0	00	0011	FSUB (scalar) — Single-precision	FEAT_FP
0	0	00	0100	FMAX (scalar) — Single-precision	FEAT_FP
0	0	00	0101	FMIN (scalar) — Single-precision	FEAT_FP
0	0	00	0110	FMAXNM (scalar) — Single-precision	FEAT_FP
0	0	00	0111	FMINNM (scalar) — Single-precision	FEAT_FP
0	0	00	1000	FNMUL (scalar) — Single-precision	FEAT_FP
0	0	01	0000	FMUL (scalar) — Double-precision	FEAT_FP
0	0	01	0001	FDIV (scalar) — Double-precision	FEAT_FP
0	0	01	0010	FADD (scalar) — Double-precision	FEAT_FP
0	0	01	0011	FSUB (scalar) — Double-precision	FEAT_FP
0	0	01	0100	FMAX (scalar) — Double-precision	FEAT_FP
0	0	01	0101	FMIN (scalar) — Double-precision	FEAT_FP
0	0	01	0110	FMAXNM (scalar) — Double-precision	FEAT_FP
0	0	01	0111	FMINNM (scalar) — Double-precision	FEAT_FP
0	0	01	1000	FNMUL (scalar) — Double-precision	FEAT_FP
0	0	10		UNALLOCATED	-
0	0	11	0000	FMUL (scalar) — Half-precision	FEAT_FP16
0	0	11	0001	FDIV (scalar) — Half-precision	FEAT_FP16
0	0	11	0010	FADD (scalar) — Half-precision	FEAT_FP16
0	0	11	0011	FSUB (scalar) — Half-precision	FEAT_FP16
0	0	11	0100	FMAX (scalar) — Half-precision	FEAT_FP16
0	0	11	0101	FMIN (scalar) — Half-precision	FEAT_FP16
0	0	11	0110	FMAXNM (scalar) — Half-precision	FEAT_FP16
0	0	11	0111	FMINNM (scalar) — Half-precision	FEAT_FP16
0	0	11	1000	FNMUL (scalar) — Half-precision	FEAT_FP16
0	0	!= 10	1001	UNALLOCATED	-
0	0	!= 10	101x	UNALLOCATED	-
0	0	!= 10	11xx	UNALLOCATED	-
0	1			UNALLOCATED	-
1				UNALLOCATED	-

Floating-point conditional select

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
M	0	S	1	1	1	1	0	f _{type}	1	R _m				cond				1	1	R _n				R _d							

Decode fields			Instruction Details	Feature
M	S	ftype		
0	0	00	FCSEL — Single-precision	FEAT_FP
0	0	01	FCSEL — Double-precision	FEAT_FP
0	0	10	UNALLOCATED	-
0	0	11	FCSEL — Half-precision	FEAT_FP16
0	1		UNALLOCATED	-
1			UNALLOCATED	-

Floating-point data-processing (3 source)

The encodings in this section are decoded from [Data Processing -- Scalar Floating-Point and Advanced SIMD](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
M	0	S	1	1	1	1	1	ftype	o1				Rm			o0			Ra					Rn					Rd		

Decode fields					Instruction Details	Feature
M	S	ftype	o1	o0		
0	0	00	0	0	FMADD — Single-precision	FEAT_FP
0	0	00	0	1	FMSUB — Single-precision	FEAT_FP
0	0	00	1	0	FNMADD — Single-precision	FEAT_FP
0	0	00	1	1	FNMSUB — Single-precision	FEAT_FP
0	0	01	0	0	FMADD — Double-precision	FEAT_FP
0	0	01	0	1	FMSUB — Double-precision	FEAT_FP
0	0	01	1	0	FNMADD — Double-precision	FEAT_FP
0	0	01	1	1	FNMSUB — Double-precision	FEAT_FP
0	0	10			UNALLOCATED	-
0	0	11	0	0	FMADD — Half-precision	FEAT_FP16
0	0	11	0	1	FMSUB — Half-precision	FEAT_FP16
0	0	11	1	0	FNMADD — Half-precision	FEAT_FP16
0	0	11	1	1	FNMSUB — Half-precision	FEAT_FP16
0	1				UNALLOCATED	-
1					UNALLOCATED	-

Loads and Stores

The encodings in this section are decoded from [A64 instruction set encoding](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	op0			1	op1	0																									

Decode fields			Instruction details
op0	op1	op2	
0000	0	1xx1xxxxxxxxxxxx	UNALLOCATED
0001	1	1xx1xxxxxxxxxxxx	UNALLOCATED
0x00	0	00x1xxxxxxxxxxxx	Compare and swap pair
0x00	0	10x0xxxxxxxxxxxx	UNALLOCATED
0x00	0	11x0xxxxxxxxxxxx	Compare and swap pair (unprivileged)
0x00	1	00x000000xxxxxx	Advanced SIMD load/store multiple structures
0x00	1	00x000001xxxxxx	UNALLOCATED
0x00	1	01x0xxxxxxxxxxxx	Advanced SIMD load/store multiple structures (post-indexed)
0x00	1	0xx1xxxxxxxxxxxx	UNALLOCATED
0x00	1	10x10001xxxxxxx	UNALLOCATED
0x00	1	10x1001xxxxxxx	UNALLOCATED
0x00	1	10x101xxxxxxxxxx	UNALLOCATED
0x00	1	10x11xxxxxxxxxxx	UNALLOCATED
0x00	1	10xx0000xxxxxxx	Advanced SIMD load/store single structure
0x00	1	11xxxxxxxxxxxxxx	Advanced SIMD load/store single structure (post-indexed)
0x00	1	x0x00001xxxxxxx	UNALLOCATED
0x00	1	x0x0001xxxxxxx	UNALLOCATED
0x00	1	x0x001xxxxxxx	UNALLOCATED
0x00	1	x0x01xxxxxxxxxx	UNALLOCATED
0x01	0	1xx0xxxxxxxxxx11	UNALLOCATED
0x01	0	1xx1xxxxxx000010	RCW compare and swap
0x01	0	1xx1xxxxxx000011	RCW compare and swap pair
0x01	0	1xx1xxxxxx00011x	UNALLOCATED
0x01	0	1xx1xxxxxx001x1x	UNALLOCATED
0x01	0	1xx1xxxxxx01xx1x	UNALLOCATED
0x01	0	1xx1xxxxxx1xxx1x	UNALLOCATED
0x01	0	1xx1xxxxxxxxxx00	128-bit atomic memory operations
0x01	0	1xx1xxxxxxxxxx01	Atomic memory operations (unprivileged)
1001	0	1xx0xxxxxxxxxx11	UNALLOCATED
1001	1	1xx1xxxxxxxxxxxx	UNALLOCATED
100x	0	1xx1xxxxxxxxxxxx	UNALLOCATED
1101	0	1000111110xxx11	GCS load/store
1101	0	1100111110xxx11	UNALLOCATED
1101	0	1x000xxxxxxxxx11	UNALLOCATED
1101	0	1x0010xxxxxxxx11	UNALLOCATED
1101	0	1x00110xxxxxxxx11	UNALLOCATED
1101	0	1x001110xxxxxx11	UNALLOCATED
1101	0	1x00111110xxx11	UNALLOCATED
1101	0	1x00111111xxx11	UNALLOCATED
1101	0	1x10xxxxxxxxxx11	UNALLOCATED
1101	0	1xx1xxxxxxxxxxxx	Load/store memory tags
1x00	0	00x1xxxxxxxxxxxx	Load/store exclusive pair
1x00	0	10x0xxxxxxxxxxxx	Load/store exclusive register (unprivileged)
1x00	0	11x0xxxxxxxxxxxx	Compare and swap (unprivileged)
1x00	1		UNALLOCATED

x100	0	1xx1xxxxxxxxxxxx	UNALLOCATED
x101	1	1xx1xxxxxxxxxxxx	UNALLOCATED
xx00	0	00x0xxxxxxxxxxxx	Load/store exclusive register
xx00	0	01x0xxxxxxxxxxxx	Load/store ordered
xx00	0	01x1xxxxxxxxxxxx	Compare and swap
xx01	0	10x0xxxxxxxxxx10	Load/store ordered register pair
xx01	0	11x000000000010	Load/store ordered (writeback)
xx01	0	11x000000000110	UNALLOCATED
xx01	0	11x000000001x10	UNALLOCATED
xx01	0	11x00000001xx10	UNALLOCATED
xx01	0	11x0000001xxx10	UNALLOCATED
xx01	0	11x000001xxx10	UNALLOCATED
xx01	0	11x00001xxxx10	UNALLOCATED
xx01	0	11x0001xxxxxx10	UNALLOCATED
xx01	0	11x001xxxxxx10	UNALLOCATED
xx01	0	11x01xxxxxx10	UNALLOCATED
xx01	0	1xx0xxxxxxxxxx00	Load/store ordered (unscaled immediate)
xx01	1	1xx0xxxxxxxxxx00	UNALLOCATED
xx01	1	1xx0xxxxxxxxxx10	Load/store ordered (SIMD&FP)
xx01	1	1xx0xxxxxxxxxx11	UNALLOCATED
xx01		0xxxxxxxxxxxxxx	Load register (literal)
xx01		1xx0xxxxxxxxxx01	Memory Copy and Memory Set
xx10		00xxxxxxxxxxxxxx	Load/store no-allocate pair (offset)
xx10		01xxxxxxxxxxxxxx	Load/store register pair (post-indexed)
xx10		10xxxxxxxxxxxxxx	Load/store register pair (offset)
xx10		11xxxxxxxxxxxxxx	Load/store register pair (pre-indexed)
xx11		0xx0xxxxxxxxxx00	Load/store register (unscaled immediate)
xx11		0xx0xxxxxxxxxx01	Load/store register (immediate post-indexed)
xx11		0xx0xxxxxxxxxx10	Load/store register (unprivileged)
xx11		0xx0xxxxxxxxxx11	Load/store register (immediate pre-indexed)
xx11		0xx1xxxxxxxxxx00	Atomic memory operations
xx11		0xx1xxxxxxxxxx10	Load/store register (register offset)
xx11		0xx1xxxxxxxxxx1	Load/store register (pac)
xx11		1xxxxxxxxxxxxxx	Load/store register (unsigned immediate)

Compare and swap pair

The encodings in this section are decoded from [Loads and Stores](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	SZ	0	0	1	0	0	0	0	L	1	Rs				o0		Rt2				Rn				Rt						

Decode fields				Instruction Details	Feature
sz	L	o0	Rt2		
			!= 11111	UNALLOCATED	-
0	0	0	11111	CASP, CASPA, CASPAL, CASPL — 32-bit CASP	FEAT_LSE
0	0	1	11111	CASP, CASPA, CASPAL, CASPL — 32-bit CASPL	FEAT_LSE
0	1	0	11111	CASP, CASPA, CASPAL, CASPL — 32-bit CASPA	FEAT_LSE
0	1	1	11111	CASP, CASPA, CASPAL, CASPL — 32-bit CASPAL	FEAT_LSE
1	0	0	11111	CASP, CASPA, CASPAL, CASPL — 64-bit CASP	FEAT_LSE
1	0	1	11111	CASP, CASPA, CASPAL, CASPL — 64-bit CASPL	FEAT_LSE
1	1	0	11111	CASP, CASPA, CASPAL, CASPL — 64-bit CASPA	FEAT_LSE
1	1	1	11111	CASP, CASPA, CASPAL, CASPL — 64-bit CASPAL	FEAT_LSE

Compare and swap pair (unprivileged)

The encodings in this section are decoded from [Loads and Stores](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	sz	0	0	1	0	0	1	1	L	0	Rs				o0	Rt2				Rn				Rt							

Decode fields				Instruction Details	Feature
sz	L	o0	Rt2		
0				UNALLOCATED	-
1			!= 11111	UNALLOCATED	-
1	0	0	11111	CASPT, CASPAT, CASPALT, CASPLT — CASPT	FEAT_LSUI
1	0	1	11111	CASPT, CASPAT, CASPALT, CASPLT — CASPLT	FEAT_LSUI
1	1	0	11111	CASPT, CASPAT, CASPALT, CASPLT — CASPAT	FEAT_LSUI
1	1	1	11111	CASPT, CASPAT, CASPALT, CASPLT — CASPALT	FEAT_LSUI

Advanced SIMD load/store multiple structures

The encodings in this section are decoded from [Loads and Stores](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	Q	0	0	1	1	0	0	0	L	0	0	0	0	0	0	opcode		size		Rn				Rt							

Decode fields		Instruction Details	Feature
L	opcode		
	x0x1	UNALLOCATED	-
	0101	UNALLOCATED	-
	11xx	UNALLOCATED	-
0	0000	ST4 (multiple structures)	FEAT_AdvSIMD
0	0010	ST1 (multiple structures) — Four registers	FEAT_AdvSIMD
0	0100	ST3 (multiple structures)	FEAT_AdvSIMD
0	0110	ST1 (multiple structures) — Three registers	FEAT_AdvSIMD
0	0111	ST1 (multiple structures) — One register	FEAT_AdvSIMD
0	1000	ST2 (multiple structures)	FEAT_AdvSIMD
0	1010	ST1 (multiple structures) — Two registers	FEAT_AdvSIMD
1	0000	LD4 (multiple structures)	FEAT_AdvSIMD
1	0010	LD1 (multiple structures) — Four registers	FEAT_AdvSIMD
1	0100	LD3 (multiple structures)	FEAT_AdvSIMD
1	0110	LD1 (multiple structures) — Three registers	FEAT_AdvSIMD
1	0111	LD1 (multiple structures) — One register	FEAT_AdvSIMD
1	1000	LD2 (multiple structures)	FEAT_AdvSIMD
1	1010	LD1 (multiple structures) — Two registers	FEAT_AdvSIMD

Advanced SIMD load/store multiple structures (post-indexed)

The encodings in this section are decoded from [Loads and Stores](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	Q	0	0	1	1	0	0	1	L	0	Rm				opcode				size		Rn				Rt						

Decode fields		Instruction Details		Feature
L	Rm	opcode		
		x0x1	UNALLOCATED	-
		0101	UNALLOCATED	-
		11xx	UNALLOCATED	-
0	11111	0000	ST4 (multiple structures) — Immediate offset	FEAT_AdvSIMD
0	11111	0010	ST1 (multiple structures) — Four registers, immediate offset	FEAT_AdvSIMD
0	11111	0100	ST3 (multiple structures) — Immediate offset	FEAT_AdvSIMD
0	11111	0110	ST1 (multiple structures) — Three registers, immediate offset	FEAT_AdvSIMD
0	11111	0111	ST1 (multiple structures) — One register, immediate offset	FEAT_AdvSIMD
0	11111	1000	ST2 (multiple structures) — Immediate offset	FEAT_AdvSIMD
0	11111	1010	ST1 (multiple structures) — Two registers, immediate offset	FEAT_AdvSIMD
0	!= 11111	0000	ST4 (multiple structures) — Register offset	FEAT_AdvSIMD
0	!= 11111	0010	ST1 (multiple structures) — Four registers, register offset	FEAT_AdvSIMD
0	!= 11111	0100	ST3 (multiple structures) — Register offset	FEAT_AdvSIMD
0	!= 11111	0110	ST1 (multiple structures) — Three registers, register offset	FEAT_AdvSIMD
0	!= 11111	0111	ST1 (multiple structures) — One register, register offset	FEAT_AdvSIMD
0	!= 11111	1000	ST2 (multiple structures) — Register offset	FEAT_AdvSIMD
0	!= 11111	1010	ST1 (multiple structures) — Two registers, register offset	FEAT_AdvSIMD
1	11111	0000	LD4 (multiple structures) — Immediate offset	FEAT_AdvSIMD
1	11111	0010	LD1 (multiple structures) — Four registers, immediate offset	FEAT_AdvSIMD
1	11111	0100	LD3 (multiple structures) — Immediate offset	FEAT_AdvSIMD
1	11111	0110	LD1 (multiple structures) — Three registers, immediate offset	FEAT_AdvSIMD
1	11111	0111	LD1 (multiple structures) — One register, immediate offset	FEAT_AdvSIMD
1	11111	1000	LD2 (multiple structures) — Immediate offset	FEAT_AdvSIMD
1	11111	1010	LD1 (multiple structures) — Two registers, immediate offset	FEAT_AdvSIMD
1	!= 11111	0000	LD4 (multiple structures) — Register offset	FEAT_AdvSIMD
1	!= 11111	0010	LD1 (multiple structures) — Four registers, register offset	FEAT_AdvSIMD
1	!= 11111	0100	LD3 (multiple structures) — Register offset	FEAT_AdvSIMD
1	!= 11111	0110	LD1 (multiple structures) — Three registers, register offset	FEAT_AdvSIMD
1	!= 11111	0111	LD1 (multiple structures) — One register, register offset	FEAT_AdvSIMD
1	!= 11111	1000	LD2 (multiple structures) — Register offset	FEAT_AdvSIMD
1	!= 11111	1010	LD1 (multiple structures) — Two registers, register offset	FEAT_AdvSIMD

Advanced SIMD load/store single structure

The encodings in this section are decoded from [Loads and Stores](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	Q	0	0	1	1	0	1	0	L	R	0	0	0	0	o2	opcode	S	size	Rn				Rt								

Decode fields						Instruction Details	Feature
L	R	o2	opcode	S	size		
		0	01x		x1	UNALLOCATED	-
		0	10x		1x	UNALLOCATED	-
		0	10x	1	01	UNALLOCATED	-
	0	1	100		!= 01	UNALLOCATED	-
	0	1	100	1	01	UNALLOCATED	-
	0	1	!= 100			UNALLOCATED	-
	1	1				UNALLOCATED	-
0		0	11x			UNALLOCATED	-
0	0	0	000			ST1 (single structure) — 8-bit	FEAT_AdvSIMD
0	0	0	001			ST3 (single structure) — 8-bit	FEAT_AdvSIMD
0	0	0	010		x0	ST1 (single structure) — 16-bit	FEAT_AdvSIMD
0	0	0	011		x0	ST3 (single structure) — 16-bit	FEAT_AdvSIMD
0	0	0	100		00	ST1 (single structure) — 32-bit	FEAT_AdvSIMD
0	0	0	100	0	01	ST1 (single structure) — 64-bit	FEAT_AdvSIMD
0	0	0	101		00	ST3 (single structure) — 32-bit	FEAT_AdvSIMD
0	0	0	101	0	01	ST3 (single structure) — 64-bit	FEAT_AdvSIMD
0	0	1	100	0	01	STL1 (SIMD&FP)	FEAT_AdvSIMD && FEAT_LRCPC3
0	1	0	000			ST2 (single structure) — 8-bit	FEAT_AdvSIMD
0	1	0	001			ST4 (single structure) — 8-bit	FEAT_AdvSIMD
0	1	0	010		x0	ST2 (single structure) — 16-bit	FEAT_AdvSIMD
0	1	0	011		x0	ST4 (single structure) — 16-bit	FEAT_AdvSIMD
0	1	0	100		00	ST2 (single structure) — 32-bit	FEAT_AdvSIMD
0	1	0	100	0	01	ST2 (single structure) — 64-bit	FEAT_AdvSIMD
0	1	0	101		00	ST4 (single structure) — 32-bit	FEAT_AdvSIMD
0	1	0	101	0	01	ST4 (single structure) — 64-bit	FEAT_AdvSIMD
1		0	11x	1		UNALLOCATED	-
1	0	0	000			LD1 (single structure) — 8-bit	FEAT_AdvSIMD
1	0	0	001			LD3 (single structure) — 8-bit	FEAT_AdvSIMD
1	0	0	010		x0	LD1 (single structure) — 16-bit	FEAT_AdvSIMD
1	0	0	011		x0	LD3 (single structure) — 16-bit	FEAT_AdvSIMD
1	0	0	100		00	LD1 (single structure) — 32-bit	FEAT_AdvSIMD
1	0	0	100	0	01	LD1 (single structure) — 64-bit	FEAT_AdvSIMD
1	0	0	101		00	LD3 (single structure) — 32-bit	FEAT_AdvSIMD
1	0	0	101	0	01	LD3 (single structure) — 64-bit	FEAT_AdvSIMD
1	0	0	110	0		LD1R	FEAT_AdvSIMD
1	0	0	111	0		LD3R	FEAT_AdvSIMD
1	0	1	100	0	01	LDAP1 (SIMD&FP)	FEAT_AdvSIMD && FEAT_LRCPC3
1	1	0	000			LD2 (single structure) — 8-bit	FEAT_AdvSIMD
1	1	0	001			LD4 (single structure) — 8-bit	FEAT_AdvSIMD
1	1	0	010		x0	LD2 (single structure) — 16-bit	FEAT_AdvSIMD
1	1	0	011		x0	LD4 (single structure) — 16-bit	FEAT_AdvSIMD
1	1	0	100		00	LD2 (single structure) — 32-bit	FEAT_AdvSIMD
1	1	0	100	0	01	LD2 (single structure) — 64-bit	FEAT_AdvSIMD
1	1	0	101		00	LD4 (single structure) — 32-bit	FEAT_AdvSIMD
1	1	0	101	0	01	LD4 (single structure) — 64-bit	FEAT_AdvSIMD

Decode fields					Instruction Details		Feature
L	R	o2	opcode	S	size		
1	1	0	110	0		LD2R	FEAT_AdvSIMD
1	1	0	111	0		LD4R	FEAT_AdvSIMD

Advanced SIMD load/store single structure (post-indexed)

The encodings in this section are decoded from [Loads and Stores](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	Q	0	0	1	1	0	1	1	L	R	Rm				opcode				S	size		Rn				Rt					

Decode fields			Instruction Details				Feature
L	R	Rm	opcode	S	size		
			01x		x1	UNALLOCATED	-
			10x		1x	UNALLOCATED	-
			10x	1	01	UNALLOCATED	-
0			11x			UNALLOCATED	-
0	0	11111	000			ST1 (single structure) — 8-bit, immediate offset	FEAT_AdvSIMD
0	0	11111	001			ST3 (single structure) — 8-bit, immediate offset	FEAT_AdvSIMD
0	0	11111	010		x0	ST1 (single structure) — 16-bit, immediate offset	FEAT_AdvSIMD
0	0	11111	011		x0	ST3 (single structure) — 16-bit, immediate offset	FEAT_AdvSIMD
0	0	11111	100		00	ST1 (single structure) — 32-bit, immediate offset	FEAT_AdvSIMD
0	0	11111	100	0	01	ST1 (single structure) — 64-bit, immediate offset	FEAT_AdvSIMD
0	0	11111	101		00	ST3 (single structure) — 32-bit, immediate offset	FEAT_AdvSIMD
0	0	11111	101	0	01	ST3 (single structure) — 64-bit, immediate offset	FEAT_AdvSIMD
0	0	!= 11111	000			ST1 (single structure) — 8-bit, register offset	FEAT_AdvSIMD
0	0	!= 11111	001			ST3 (single structure) — 8-bit, register offset	FEAT_AdvSIMD
0	0	!= 11111	010		x0	ST1 (single structure) — 16-bit, register offset	FEAT_AdvSIMD
0	0	!= 11111	011		x0	ST3 (single structure) — 16-bit, register offset	FEAT_AdvSIMD
0	0	!= 11111	100		00	ST1 (single structure) — 32-bit, register offset	FEAT_AdvSIMD
0	0	!= 11111	100	0	01	ST1 (single structure) — 64-bit, register offset	FEAT_AdvSIMD
0	0	!= 11111	101		00	ST3 (single structure) — 32-bit, register offset	FEAT_AdvSIMD
0	0	!= 11111	101	0	01	ST3 (single structure) — 64-bit, register offset	FEAT_AdvSIMD
0	1	11111	000			ST2 (single structure) — 8-bit, immediate offset	FEAT_AdvSIMD
0	1	11111	001			ST4 (single structure) — 8-bit, immediate offset	FEAT_AdvSIMD
0	1	11111	010		x0	ST2 (single structure) — 16-bit, immediate offset	FEAT_AdvSIMD
0	1	11111	011		x0	ST4 (single structure) — 16-bit, immediate offset	FEAT_AdvSIMD
0	1	11111	100		00	ST2 (single structure) — 32-bit, immediate offset	FEAT_AdvSIMD
0	1	11111	100	0	01	ST2 (single structure) — 64-bit, immediate offset	FEAT_AdvSIMD
0	1	11111	101		00	ST4 (single structure) — 32-bit, immediate offset	FEAT_AdvSIMD
0	1	11111	101	0	01	ST4 (single structure) — 64-bit, immediate offset	FEAT_AdvSIMD
0	1	!= 11111	000			ST2 (single structure) — 8-bit, register offset	FEAT_AdvSIMD
0	1	!= 11111	001			ST4 (single structure) — 8-bit, register offset	FEAT_AdvSIMD
0	1	!= 11111	010		x0	ST2 (single structure) — 16-bit, register offset	FEAT_AdvSIMD
0	1	!= 11111	011		x0	ST4 (single structure) — 16-bit, register offset	FEAT_AdvSIMD
0	1	!= 11111	100		00	ST2 (single structure) — 32-bit, register offset	FEAT_AdvSIMD
0	1	!= 11111	100	0	01	ST2 (single structure) — 64-bit, register offset	FEAT_AdvSIMD
0	1	!= 11111	101		00	ST4 (single structure) — 32-bit, register offset	FEAT_AdvSIMD
0	1	!= 11111	101	0	01	ST4 (single structure) — 64-bit, register offset	FEAT_AdvSIMD
1			11x	1		UNALLOCATED	-
1	0	11111	000			LD1 (single structure) — 8-bit, immediate offset	FEAT_AdvSIMD
1	0	11111	001			LD3 (single structure) — 8-bit, immediate offset	FEAT_AdvSIMD
1	0	11111	010		x0	LD1 (single structure) — 16-bit, immediate offset	FEAT_AdvSIMD
1	0	11111	011		x0	LD3 (single structure) — 16-bit, immediate offset	FEAT_AdvSIMD
1	0	11111	100		00	LD1 (single structure) — 32-bit, immediate offset	FEAT_AdvSIMD
1	0	11111	100	0	01	LD1 (single structure) — 64-bit, immediate offset	FEAT_AdvSIMD
1	0	11111	101		00	LD3 (single structure) — 32-bit, immediate offset	FEAT_AdvSIMD
1	0	11111	101	0	01	LD3 (single structure) — 64-bit, immediate offset	FEAT_AdvSIMD

Decode fields						Instruction Details	Feature
L	R	Rm	opcode	S	size		
1	0	11111	110	0		LD1R — Immediate offset	FEAT_AdvSIMD
1	0	11111	111	0		LD3R — Immediate offset	FEAT_AdvSIMD
1	0	!= 11111	000			LD1 (single structure) — 8-bit, register offset	FEAT_AdvSIMD
1	0	!= 11111	001			LD3 (single structure) — 8-bit, register offset	FEAT_AdvSIMD
1	0	!= 11111	010		×0	LD1 (single structure) — 16-bit, register offset	FEAT_AdvSIMD
1	0	!= 11111	011		×0	LD3 (single structure) — 16-bit, register offset	FEAT_AdvSIMD
1	0	!= 11111	100		00	LD1 (single structure) — 32-bit, register offset	FEAT_AdvSIMD
1	0	!= 11111	100	0	01	LD1 (single structure) — 64-bit, register offset	FEAT_AdvSIMD
1	0	!= 11111	101		00	LD3 (single structure) — 32-bit, register offset	FEAT_AdvSIMD
1	0	!= 11111	101	0	01	LD3 (single structure) — 64-bit, register offset	FEAT_AdvSIMD
1	0	!= 11111	110	0		LD1R — Register offset	FEAT_AdvSIMD
1	0	!= 11111	111	0		LD3R — Register offset	FEAT_AdvSIMD
1	1	11111	000			LD2 (single structure) — 8-bit, immediate offset	FEAT_AdvSIMD
1	1	11111	001			LD4 (single structure) — 8-bit, immediate offset	FEAT_AdvSIMD
1	1	11111	010		×0	LD2 (single structure) — 16-bit, immediate offset	FEAT_AdvSIMD
1	1	11111	011		×0	LD4 (single structure) — 16-bit, immediate offset	FEAT_AdvSIMD
1	1	11111	100		00	LD2 (single structure) — 32-bit, immediate offset	FEAT_AdvSIMD
1	1	11111	100	0	01	LD2 (single structure) — 64-bit, immediate offset	FEAT_AdvSIMD
1	1	11111	101		00	LD4 (single structure) — 32-bit, immediate offset	FEAT_AdvSIMD
1	1	11111	101	0	01	LD4 (single structure) — 64-bit, immediate offset	FEAT_AdvSIMD
1	1	11111	110	0		LD2R — Immediate offset	FEAT_AdvSIMD
1	1	11111	111	0		LD4R — Immediate offset	FEAT_AdvSIMD
1	1	!= 11111	000			LD2 (single structure) — 8-bit, register offset	FEAT_AdvSIMD
1	1	!= 11111	001			LD4 (single structure) — 8-bit, register offset	FEAT_AdvSIMD
1	1	!= 11111	010		×0	LD2 (single structure) — 16-bit, register offset	FEAT_AdvSIMD
1	1	!= 11111	011		×0	LD4 (single structure) — 16-bit, register offset	FEAT_AdvSIMD
1	1	!= 11111	100		00	LD2 (single structure) — 32-bit, register offset	FEAT_AdvSIMD
1	1	!= 11111	100	0	01	LD2 (single structure) — 64-bit, register offset	FEAT_AdvSIMD
1	1	!= 11111	101		00	LD4 (single structure) — 32-bit, register offset	FEAT_AdvSIMD
1	1	!= 11111	101	0	01	LD4 (single structure) — 64-bit, register offset	FEAT_AdvSIMD
1	1	!= 11111	110	0		LD2R — Register offset	FEAT_AdvSIMD
1	1	!= 11111	111	0		LD4R — Register offset	FEAT_AdvSIMD

RCW compare and swap

The encodings in this section are decoded from [Loads and Stores](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	S	0	1	1	0	0	1	A	R	1	Rs				0	0	0	0	1	0	Rn				Rt						

Decode fields			Instruction Details	Feature
S	A	R		
0	0	0	RCWCAS, RCWCASA, RCWCASAL, RCWCASL — RCWCAS	FEAT_THE
0	0	1	RCWCAS, RCWCASA, RCWCASAL, RCWCASL — RCWCASL	FEAT_THE
0	1	0	RCWCAS, RCWCASA, RCWCASAL, RCWCASL — RCWCASA	FEAT_THE
0	1	1	RCWCAS, RCWCASA, RCWCASAL, RCWCASL — RCWCASAL	FEAT_THE
1	0	0	RCWSCAS, RCWSCASA, RCWSCASAL, RCWSCASL — RCWSCAS	FEAT_THE
1	0	1	RCWSCAS, RCWSCASA, RCWSCASAL, RCWSCASL — RCWSCASL	FEAT_THE
1	1	0	RCWSCAS, RCWSCASA, RCWSCASAL, RCWSCASL — RCWSCASA	FEAT_THE
1	1	1	RCWSCAS, RCWSCASA, RCWSCASAL, RCWSCASL — RCWSCASAL	FEAT_THE

RCW compare and swap pair

The encodings in this section are decoded from [Loads and Stores](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	S	0	1	1	0	0	1	A	R	1	Rs				0	0	0	0	1	1	Rn				Rt						

Decode fields			Instruction Details	Feature
S	A	R		
0	0	0	RCWCASP, RCWCASPA, RCWCASPAL, RCWCASPL — RCWCASP	FEAT_D128 && FEAT_THE
0	0	1	RCWCASP, RCWCASPA, RCWCASPAL, RCWCASPL — RCWCASPL	FEAT_D128 && FEAT_THE
0	1	0	RCWCASP, RCWCASPA, RCWCASPAL, RCWCASPL — RCWCASPA	FEAT_D128 && FEAT_THE
0	1	1	RCWCASP, RCWCASPA, RCWCASPAL, RCWCASPL — RCWCASPAL	FEAT_D128 && FEAT_THE
1	0	0	RCWSCASP, RCWSCASPA, RCWSCASPAL, RCWSCASPL — RCWSCASP	FEAT_D128 && FEAT_THE
1	0	1	RCWSCASP, RCWSCASPA, RCWSCASPAL, RCWSCASPL — RCWSCASPL	FEAT_D128 && FEAT_THE
1	1	0	RCWSCASP, RCWSCASPA, RCWSCASPAL, RCWSCASPL — RCWSCASPA	FEAT_D128 && FEAT_THE
1	1	1	RCWSCASP, RCWSCASPA, RCWSCASPAL, RCWSCASPL — RCWSCASPAL	FEAT_D128 && FEAT_THE

128-bit atomic memory operations

The encodings in this section are decoded from [Loads and Stores](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	S	0	1	1	0	0	1	A	R	1	Rt2				o3	opc				0	0	Rn				Rt					

Decode fields					Instruction Details	Feature
S	A	R	o3	opc		
				1xx	UNALLOCATED	-
0			0	0x0	UNALLOCATED	-
0	0	0	0	001	LDCLRP, LDCLRPA, LDCLRPAL, LDCLRPL — LDCLRP	FEAT_LSE128
0	0	0	0	011	LDSETP, LDSETPA, LDSETPAL, LDSETPL — LDSETP	FEAT_LSE128
0	0	0	1	000	SWPP, SWPPA, SWPPAL, SWPPL — SWPP	FEAT_LSE128
0	0	0	1	001	RCWCLRP, RCWCLRPA, RCWCLRPAL, RCWCLRPL — RCWCLRP	FEAT_D128 && FEAT_THE
0	0	0	1	010	RCWSWPP, RCWSWPPA, RCWSWPPAL, RCWSWPPL — RCWSWPP	FEAT_D128 && FEAT_THE
0	0	0	1	011	RCWSETP, RCWSETPA, RCWSETPAL, RCWSETPL — RCWSETP	FEAT_D128 && FEAT_THE
0	0	1	0	001	LDCLRP, LDCLRPA, LDCLRPAL, LDCLRPL — LDCLRPL	FEAT_LSE128
0	0	1	0	011	LDSETP, LDSETPA, LDSETPAL, LDSETPL — LDSETPL	FEAT_LSE128
0	0	1	1	000	SWPP, SWPPA, SWPPAL, SWPPL — SWPPL	FEAT_LSE128
0	0	1	1	001	RCWCLRP, RCWCLRPA, RCWCLRPAL, RCWCLRPL — RCWCLRPL	FEAT_D128 && FEAT_THE
0	0	1	1	010	RCWSWPP, RCWSWPPA, RCWSWPPAL, RCWSWPPL — RCWSWPPL	FEAT_D128 && FEAT_THE
0	0	1	1	011	RCWSETP, RCWSETPA, RCWSETPAL, RCWSETPL — RCWSETPL	FEAT_D128 && FEAT_THE
0	1	0	0	001	LDCLRP, LDCLRPA, LDCLRPAL, LDCLRPL — LDCLRPA	FEAT_LSE128
0	1	0	0	011	LDSETP, LDSETPA, LDSETPAL, LDSETPL — LDSETPA	FEAT_LSE128
0	1	0	1	000	SWPP, SWPPA, SWPPAL, SWPPL — SWPPA	FEAT_LSE128
0	1	0	1	001	RCWCLRP, RCWCLRPA, RCWCLRPAL, RCWCLRPL — RCWCLRPA	FEAT_D128 && FEAT_THE
0	1	0	1	010	RCWSWPP, RCWSWPPA, RCWSWPPAL, RCWSWPPL — RCWSWPPA	FEAT_D128 && FEAT_THE
0	1	0	1	011	RCWSETP, RCWSETPA, RCWSETPAL, RCWSETPL — RCWSETPA	FEAT_D128 && FEAT_THE
0	1	1	0	001	LDCLRP, LDCLRPA, LDCLRPAL, LDCLRPL — LDCLRPAL	FEAT_LSE128
0	1	1	0	011	LDSETP, LDSETPA, LDSETPAL, LDSETPL — LDSETPAL	FEAT_LSE128
0	1	1	1	000	SWPP, SWPPA, SWPPAL, SWPPL — SWPPAL	FEAT_LSE128
0	1	1	1	001	RCWCLRP, RCWCLRPA, RCWCLRPAL, RCWCLRPL — RCWCLRPAL	FEAT_D128 && FEAT_THE
0	1	1	1	010	RCWSWPP, RCWSWPPA, RCWSWPPAL, RCWSWPPL — RCWSWPPAL	FEAT_D128 && FEAT_THE
0	1	1	1	011	RCWSETP, RCWSETPA, RCWSETPAL, RCWSETPL — RCWSETPAL	FEAT_D128 && FEAT_THE
1			0	0xx	UNALLOCATED	-
1			1	000	UNALLOCATED	-
1	0	0	1	001	RCWSCLRP, RCWSCLRPA, RCWSCLRPAL, RCWSCLRPL — RCWSCLRP	FEAT_D128 && FEAT_THE
1	0	0	1	010	RCWSSWPP, RCWSSWPPA, RCWSSWPPAL, RCWSSWPPL — RCWSSWPP	FEAT_D128 && FEAT_THE
1	0	0	1	011	RCWSSETP, RCWSSETPA, RCWSSETPAL, RCWSSETPL — RCWSSETP	FEAT_D128 && FEAT_THE
1	0	1	1	001	RCWSCLRP, RCWSCLRPA, RCWSCLRPAL, RCWSCLRPL — RCWSCLRPL	FEAT_D128 && FEAT_THE
1	0	1	1	010	RCWSSWPP, RCWSSWPPA, RCWSSWPPAL, RCWSSWPPL — RCWSSWPPL	FEAT_D128 && FEAT_THE
1	0	1	1	011	RCWSSETP, RCWSSETPA, RCWSSETPAL, RCWSSETPL — RCWSSETPL	FEAT_D128 && FEAT_THE
1	1	0	1	001	RCWSCLRP, RCWSCLRPA, RCWSCLRPAL, RCWSCLRPL — RCWSCLRPA	FEAT_D128 && FEAT_THE

Decode fields					Instruction Details	Feature
S	A	R	o3	opc		
1	1	0	1	010	RCWSSWPP, RCWSSWPPA, RCWSSWPPAL, RCWSSWPPL — RCWSSWPPA	FEAT_D128 && FEAT_THE
1	1	0	1	011	RCWSSETP, RCWSSETPA, RCWSSETPAL, RCWSSETPL — RCWSSETPA	FEAT_D128 && FEAT_THE
1	1	1	1	001	RCWSCLRP, RCWSCLRPA, RCWSCLRPAL, RCWSCLRPL — RCWSCLRPAL	FEAT_D128 && FEAT_THE
1	1	1	1	010	RCWSSWPP, RCWSSWPPA, RCWSSWPPAL, RCWSSWPPL — RCWSSWPPAL	FEAT_D128 && FEAT_THE
1	1	1	1	011	RCWSSETP, RCWSSETPA, RCWSSETPAL, RCWSSETPL — RCWSSETPAL	FEAT_D128 && FEAT_THE

Atomic memory operations (unprivileged)

The encodings in this section are decoded from [Loads and Stores](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	sz	0	1	1	0	0	1	A	R	1	Rs				o3		opc				0	1	Rn				Rt				

Decode fields					Instruction Details	Feature
sz	A	R	o3	opc		
				010	UNALLOCATED	-
				1xx	UNALLOCATED	-
			1	0x1	UNALLOCATED	-
0	0	0	0	000	LDTADD, LDTADDA, LDTADDAL, LDTADDL — 32-bit no memory ordering	FEAT_LSUI
0	0	0	0	001	LDTCLR, LDTCLRA, LDTCLRAL, LDTCLRL — 32-bit no memory ordering	FEAT_LSUI
0	0	0	0	011	LDTSET, LDTSETA, LDTSETAL, LDTSETL — 32-bit no memory ordering	FEAT_LSUI
0	0	0	1	000	SWPT, SWPTA, SWPTAL, SWPTL — 32-bit SWPT	FEAT_LSUI
0	0	1	0	000	LDTADD, LDTADDA, LDTADDAL, LDTADDL — 32-bit release	FEAT_LSUI
0	0	1	0	001	LDTCLR, LDTCLRA, LDTCLRAL, LDTCLRL — 32-bit release	FEAT_LSUI
0	0	1	0	011	LDTSET, LDTSETA, LDTSETAL, LDTSETL — 32-bit release	FEAT_LSUI
0	0	1	1	000	SWPT, SWPTA, SWPTAL, SWPTL — 32-bit SWPTL	FEAT_LSUI
0	1	0	0	000	LDTADD, LDTADDA, LDTADDAL, LDTADDL — 32-bit acquire	FEAT_LSUI
0	1	0	0	001	LDTCLR, LDTCLRA, LDTCLRAL, LDTCLRL — 32-bit acquire	FEAT_LSUI
0	1	0	0	011	LDTSET, LDTSETA, LDTSETAL, LDTSETL — 32-bit acquire	FEAT_LSUI
0	1	0	1	000	SWPT, SWPTA, SWPTAL, SWPTL — 32-bit SWPTA	FEAT_LSUI
0	1	1	0	000	LDTADD, LDTADDA, LDTADDAL, LDTADDL — 32-bit acquire-release	FEAT_LSUI
0	1	1	0	001	LDTCLR, LDTCLRA, LDTCLRAL, LDTCLRL — 32-bit acquire-release	FEAT_LSUI
0	1	1	0	011	LDTSET, LDTSETA, LDTSETAL, LDTSETL — 32-bit acquire-release	FEAT_LSUI
0	1	1	1	000	SWPT, SWPTA, SWPTAL, SWPTL — 32-bit SWPTAL	FEAT_LSUI
1	0	0	0	000	LDTADD, LDTADDA, LDTADDAL, LDTADDL — 64-bit no memory ordering	FEAT_LSUI
1	0	0	0	001	LDTCLR, LDTCLRA, LDTCLRAL, LDTCLRL — 64-bit no memory ordering	FEAT_LSUI
1	0	0	0	011	LDTSET, LDTSETA, LDTSETAL, LDTSETL — 64-bit no memory ordering	FEAT_LSUI
1	0	0	1	000	SWPT, SWPTA, SWPTAL, SWPTL — 64-bit SWPT	FEAT_LSUI
1	0	1	0	000	LDTADD, LDTADDA, LDTADDAL, LDTADDL — 64-bit release	FEAT_LSUI
1	0	1	0	001	LDTCLR, LDTCLRA, LDTCLRAL, LDTCLRL — 64-bit release	FEAT_LSUI
1	0	1	0	011	LDTSET, LDTSETA, LDTSETAL, LDTSETL — 64-bit release	FEAT_LSUI
1	0	1	1	000	SWPT, SWPTA, SWPTAL, SWPTL — 64-bit SWPTL	FEAT_LSUI
1	1	0	0	000	LDTADD, LDTADDA, LDTADDAL, LDTADDL — 64-bit acquire	FEAT_LSUI
1	1	0	0	001	LDTCLR, LDTCLRA, LDTCLRAL, LDTCLRL — 64-bit acquire	FEAT_LSUI
1	1	0	0	011	LDTSET, LDTSETA, LDTSETAL, LDTSETL — 64-bit acquire	FEAT_LSUI
1	1	0	1	000	SWPT, SWPTA, SWPTAL, SWPTL — 64-bit SWPTA	FEAT_LSUI
1	1	1	0	000	LDTADD, LDTADDA, LDTADDAL, LDTADDL — 64-bit acquire-release	FEAT_LSUI
1	1	1	0	001	LDTCLR, LDTCLRA, LDTCLRAL, LDTCLRL — 64-bit acquire-release	FEAT_LSUI
1	1	1	0	011	LDTSET, LDTSETA, LDTSETAL, LDTSETL — 64-bit acquire-release	FEAT_LSUI
1	1	1	1	000	SWPT, SWPTA, SWPTAL, SWPTL — 64-bit SWPTAL	FEAT_LSUI

GCS load/store

The encodings in this section are decoded from [Loads and Stores](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	1	1	0	0	1	0	0	0	1	1	1	1	1	0	opc			1	1	Rn				Rt					

Decode fields	Instruction Details	Feature
opc		
000	GCSSTR	FEAT_GCS
001	GCSSTTR	FEAT_GCS
1xx	UNALLOCATED	-
!= 00x01x	UNALLOCATED	-

Load/store memory tags

The encodings in this section are decoded from [Loads and Stores](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	1	1	0	0	1	opc	1											op2											

Decode fields			Instruction Details	Feature
opc	imm9	op2		
00		01	STG — Post-index	FEAT_MTE
00		10	STG — Signed offset	FEAT_MTE
00		11	STG — Pre-index	FEAT_MTE
00	000000000	00	STZGM	FEAT_MTE2
01		00	LDG	FEAT_MTE
01		01	STZG — Post-index	FEAT_MTE
01		10	STZG — Signed offset	FEAT_MTE
01		11	STZG — Pre-index	FEAT_MTE
10		01	ST2G — Post-index	FEAT_MTE
10		10	ST2G — Signed offset	FEAT_MTE
10		11	ST2G — Pre-index	FEAT_MTE
10	000000000	00	STGM	FEAT_MTE2
11		01	STZ2G — Post-index	FEAT_MTE
11		10	STZ2G — Signed offset	FEAT_MTE
11		11	STZ2G — Pre-index	FEAT_MTE
11	000000000	00	LDGM	FEAT_MTE2
!= 01	!= 000000000	00	UNALLOCATED	-

Load/store exclusive pair

The encodings in this section are decoded from [Loads and Stores](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	sz	0	0	1	0	0	0	0	L	1																					

Decode fields			Instruction Details
sz	L	o0	
0	0	0	STXP — 32-bit
0	0	1	STLXP — 32-bit
0	1	0	LDXP — 32-bit
0	1	1	LDAXP — 32-bit
1	0	0	STXP — 64-bit
1	0	1	STLXP — 64-bit
1	1	0	LDXP — 64-bit
1	1	1	LDAXP — 64-bit

Load/store exclusive register (unprivileged)

The encodings in this section are decoded from [Loads and Stores](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	sz	0	0	1	0	0	1	0	L	0	Rs				o0	Rt2				Rn				Rt							

Decode fields			Instruction Details		Feature
sz	L	o0			
0	0	0	STTXR — 32-bit		FEAT_LSUI
0	0	1	STLTXR — 32-bit		FEAT_LSUI
0	1	0	LDTXR — 32-bit		FEAT_LSUI
0	1	1	LDATXR — 32-bit		FEAT_LSUI
1	0	0	STTXR — 64-bit		FEAT_LSUI
1	0	1	STLTXR — 64-bit		FEAT_LSUI
1	1	0	LDTXR — 64-bit		FEAT_LSUI
1	1	1	LDATXR — 64-bit		FEAT_LSUI

Compare and swap (unprivileged)

The encodings in this section are decoded from [Loads and Stores](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	sz	0	0	1	0	0	1	1	L	0	Rs				o0	Rt2				Rn				Rt							

Decode fields				Instruction Details		Feature
sz	L	o0	Rt2			
0				UNALLOCATED		-
1			!= 11111	UNALLOCATED		-
1	0	0	11111	CAST, CASAT, CASALT, CASLT — CAST		FEAT_LSUI
1	0	1	11111	CAST, CASAT, CASALT, CASLT — CASLT		FEAT_LSUI
1	1	0	11111	CAST, CASAT, CASALT, CASLT — CASAT		FEAT_LSUI
1	1	1	11111	CAST, CASAT, CASALT, CASLT — CASALT		FEAT_LSUI

Load/store exclusive register

The encodings in this section are decoded from [Loads and Stores](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
size	0	0	1	0	0	0	0	L	0	Rs				o0	Rt2				Rn				Rt								

Decode fields			Instruction Details
size	L	o0	
00	0	0	STXRB
00	0	1	STLXRB
00	1	0	LDXRB
00	1	1	LDAXRB
01	0	0	STXRH
01	0	1	STLXRH
01	1	0	LDXRH
01	1	1	LDAXRH
10	0	0	STXR — 32-bit
10	0	1	STLXR — 32-bit
10	1	0	LDXR — 32-bit
10	1	1	LDAXR — 32-bit
11	0	0	STXR — 64-bit
11	0	1	STLXR — 64-bit
11	1	0	LDXR — 64-bit
11	1	1	LDAXR — 64-bit

Load/store ordered

The encodings in this section are decoded from [Loads and Stores](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
size	0	0	1	0	0	0	1	L	0	Rs			o0			Rt2			Rn			Rt									

Decode fields			Instruction Details	Feature
size	L	o0		
00	0	0	STLLRB	FEAT_LOR
00	0	1	STLRB	-
00	1	0	LDLARB	FEAT_LOR
00	1	1	LDARB	-
01	0	0	STLLRH	FEAT_LOR
01	0	1	STLRH	-
01	1	0	LDLARH	FEAT_LOR
01	1	1	LDARH	-
10	0	0	STLLR — 32-bit	FEAT_LOR
10	0	1	STLR — 32-bit	-
10	1	0	LDLAR — 32-bit	FEAT_LOR
10	1	1	LDAR — 32-bit	-
11	0	0	STLLR — 64-bit	FEAT_LOR
11	0	1	STLR — 64-bit	-
11	1	0	LDLAR — 64-bit	FEAT_LOR
11	1	1	LDAR — 64-bit	-

Compare and swap

The encodings in this section are decoded from [Loads and Stores](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
size	0	0	1	0	0	0	1	L	1				Rs			o0			Rt2					Rn						Rt	

Decode fields				Instruction Details	Feature
size	L	o0	Rt2		
			!= 11111	UNALLOCATED	-
00	0	0	11111	CASB, CASAB, CASALB, CASLB — CASB	FEAT_LSE
00	0	1	11111	CASB, CASAB, CASALB, CASLB — CASLB	FEAT_LSE
00	1	0	11111	CASB, CASAB, CASALB, CASLB — CASAB	FEAT_LSE
00	1	1	11111	CASB, CASAB, CASALB, CASLB — CASALB	FEAT_LSE
01	0	0	11111	CASH, CASHA, CASALH, CASLH — CASH	FEAT_LSE
01	0	1	11111	CASH, CASHA, CASALH, CASLH — CASLH	FEAT_LSE
01	1	0	11111	CASH, CASHA, CASALH, CASLH — CASHA	FEAT_LSE
01	1	1	11111	CASH, CASHA, CASALH, CASLH — CASALH	FEAT_LSE
10	0	0	11111	CAS, CASA, CASAL, CASL — 32-bit CAS	FEAT_LSE
10	0	1	11111	CAS, CASA, CASAL, CASL — 32-bit CASL	FEAT_LSE
10	1	0	11111	CAS, CASA, CASAL, CASL — 32-bit CASA	FEAT_LSE
10	1	1	11111	CAS, CASA, CASAL, CASL — 32-bit CASAL	FEAT_LSE
11	0	0	11111	CAS, CASA, CASAL, CASL — 64-bit CAS	FEAT_LSE
11	0	1	11111	CAS, CASA, CASAL, CASL — 64-bit CASL	FEAT_LSE
11	1	0	11111	CAS, CASA, CASAL, CASL — 64-bit CASA	FEAT_LSE
11	1	1	11111	CAS, CASA, CASAL, CASL — 64-bit CASAL	FEAT_LSE

Load/store ordered register pair

The encodings in this section are decoded from [Loads and Stores](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
size	0	1	1	0	0	1	0	L	0		Rt2					opc2				1	0		Rn					Rt			

Decode fields			Instruction Details	Feature
size	L	opc2		
0x			UNALLOCATED	-
1x		1xxx	UNALLOCATED	-
1x	0	001x	UNALLOCATED	-
1x	1	001x	UNALLOCATED	-
10		01xx	UNALLOCATED	-
10	0	0000	STILP — 32-bit pre-index	FEAT_LRCPC3
10	0	0001	STILP — 32-bit	FEAT_LRCPC3
10	1	0000	LDIAPP — 32-bit post-index	FEAT_LRCPC3
10	1	0001	LDIAPP — 32-bit	FEAT_LRCPC3
11		0100	UNALLOCATED	-
11	0	0000	STILP — 64-bit pre-index	FEAT_LRCPC3
11	0	0001	STILP — 64-bit	FEAT_LRCPC3
11	0	0101	STLP	FEAT_LSCP
11	0	011x	UNALLOCATED	-
11	1	0000	LDIAPP — 64-bit post-index	FEAT_LRCPC3
11	1	0001	LDIAPP — 64-bit	FEAT_LRCPC3
11	1	0101	LDAP	FEAT_LSCP
11	1	0110	UNALLOCATED	-
11	1	0111	LDAPP	FEAT_LSCP

Load/store ordered (writeback)

The encodings in this section are decoded from [Loads and Stores](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
size	0	1	1	0	0	1	1	L	0	0	0	0	0	0	0	0	0	0	0	1	0	Rn				Rt					

Decode fields		Instruction Details		Feature
size	L			
0x		UNALLOCATED		-
10	0	STLR — 32-bit		FEAT_LRCPC3
10	1	LDAPR — 32-bit		FEAT_LRCPC3
11	0	STLR — 64-bit		FEAT_LRCPC3
11	1	LDAPR — 64-bit		FEAT_LRCPC3

Load/store ordered (unscaled immediate)

The encodings in this section are decoded from [Loads and Stores](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
size		0	1	1	0	0	1	opc		0	imm9										0	0	Rn				Rt				

Decode fields		Instruction Details		Feature
size	opc			
00	00	STLURB		FEAT_LRCPC2
00	01	LDAPURB		FEAT_LRCPC2
00	10	LDAPURSB — 64-bit		FEAT_LRCPC2
00	11	LDAPURSB — 32-bit		FEAT_LRCPC2
01	00	STLURH		FEAT_LRCPC2
01	01	LDAPURH		FEAT_LRCPC2
01	10	LDAPURSH — 64-bit		FEAT_LRCPC2
01	11	LDAPURSH — 32-bit		FEAT_LRCPC2
10	00	STLUR — 32-bit		FEAT_LRCPC2
10	01	LDAPUR — 32-bit		FEAT_LRCPC2
10	10	LDAPURSW		FEAT_LRCPC2
10	11	UNALLOCATED		-
11	00	STLUR — 64-bit		FEAT_LRCPC2
11	01	LDAPUR — 64-bit		FEAT_LRCPC2
11	1x	UNALLOCATED		-

Load/store ordered (SIMD&FP)

The encodings in this section are decoded from [Loads and Stores](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
size	0	1	1	1	0	1	opc	0	imm9												1	0	Rn				Rt				

Decode fields		Instruction Details	Feature
size	opc		
00	00	STLUR (SIMD&FP) — 8-bit	FEAT_FP && FEAT_LRCPC3
00	01	LDAPUR (SIMD&FP) — 8-bit	FEAT_FP && FEAT_LRCPC3
00	10	STLUR (SIMD&FP) — 128-bit	FEAT_FP && FEAT_LRCPC3
00	11	LDAPUR (SIMD&FP) — 128-bit	FEAT_FP && FEAT_LRCPC3
01	00	STLUR (SIMD&FP) — 16-bit	FEAT_FP && FEAT_LRCPC3
01	01	LDAPUR (SIMD&FP) — 16-bit	FEAT_FP && FEAT_LRCPC3
10	00	STLUR (SIMD&FP) — 32-bit	FEAT_FP && FEAT_LRCPC3
10	01	LDAPUR (SIMD&FP) — 32-bit	FEAT_FP && FEAT_LRCPC3
11	00	STLUR (SIMD&FP) — 64-bit	FEAT_FP && FEAT_LRCPC3
11	01	LDAPUR (SIMD&FP) — 64-bit	FEAT_FP && FEAT_LRCPC3
!= 00	1x	UNALLOCATED	-

Load register (literal)

The encodings in this section are decoded from [Loads and Stores](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opc		0	1	1	VR	0	0	imm19																	Rt						

Decode fields		Instruction Details	Feature
opc	VR		
00	0	LDR (literal) — 32-bit	-
00	1	LDR (literal, SIMD&FP) — 32-bit	FEAT_FP
01	0	LDR (literal) — 64-bit	-
01	1	LDR (literal, SIMD&FP) — 64-bit	FEAT_FP
10	0	LDRSW (literal)	-
10	1	LDR (literal, SIMD&FP) — 128-bit	FEAT_FP
11	0	PRFM (literal)	-
11	1	UNALLOCATED	-

Memory Copy and Memory Set

The encodings in this section are decoded from [Loads and Stores](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
size	0	1	1	o0	0	1	op1	0	Rs					op2				0	1	Rn				Rd							

Decode fields			Instruction Details	Feature
o0	op1	op2		
	11	11xx	UNALLOCATED	-
0	00	0000	CPYFP, CPYFM, CPYFE — Prologue	FEAT_MOPS
0	00	0001	CPYFPWT, CPYFMWT, CPYFEWT — Prologue	FEAT_MOPS
0	00	0010	CPYFPRT, CPYFMRT, CPYFERT — Prologue	FEAT_MOPS
0	00	0011	CPYFPT, CPYFMT, CPYFET — Prologue	FEAT_MOPS
0	00	0100	CPYFPWN, CPYFMWN, CPYFEWN — Prologue	FEAT_MOPS
0	00	0101	CPYFPWTWN, CPYFMWTWN, CPYFEWTWN — Prologue	FEAT_MOPS
0	00	0110	CPYFPRTWN, CPYFMRTWN, CPYFERTWN — Prologue	FEAT_MOPS
0	00	0111	CPYFPTWN, CPYFMTWN, CPYFETWN — Prologue	FEAT_MOPS
0	00	1000	CPYFPRN, CPYFMRN, CPYFERN — Prologue	FEAT_MOPS
0	00	1001	CPYFPWTRN, CPYFMWTRN, CPYFEWTRN — Prologue	FEAT_MOPS
0	00	1010	CPYFPRTRN, CPYFMRTRN, CPYFERTRN — Prologue	FEAT_MOPS
0	00	1011	CPYFPTRN, CPYFMTRN, CPYFETRN — Prologue	FEAT_MOPS
0	00	1100	CPYFPN, CPYFMN, CPYFEN — Prologue	FEAT_MOPS
0	00	1101	CPYFPWTN, CPYFMWTN, CPYFEWTN — Prologue	FEAT_MOPS
0	00	1110	CPYFPRTN, CPYFMRTN, CPYFERTN — Prologue	FEAT_MOPS
0	00	1111	CPYFPTN, CPYFMTN, CPYFETN — Prologue	FEAT_MOPS
0	01	0000	CPYFP, CPYFM, CPYFE — Main	FEAT_MOPS
0	01	0001	CPYFPWT, CPYFMWT, CPYFEWT — Main	FEAT_MOPS
0	01	0010	CPYFPRT, CPYFMRT, CPYFERT — Main	FEAT_MOPS
0	01	0011	CPYFPT, CPYFMT, CPYFET — Main	FEAT_MOPS
0	01	0100	CPYFPWN, CPYFMWN, CPYFEWN — Main	FEAT_MOPS
0	01	0101	CPYFPWTWN, CPYFMWTWN, CPYFEWTWN — Main	FEAT_MOPS
0	01	0110	CPYFPRTWN, CPYFMRTWN, CPYFERTWN — Main	FEAT_MOPS
0	01	0111	CPYFPTWN, CPYFMTWN, CPYFETWN — Main	FEAT_MOPS
0	01	1000	CPYFPRN, CPYFMRN, CPYFERN — Main	FEAT_MOPS
0	01	1001	CPYFPWTRN, CPYFMWTRN, CPYFEWTRN — Main	FEAT_MOPS
0	01	1010	CPYFPRTRN, CPYFMRTRN, CPYFERTRN — Main	FEAT_MOPS
0	01	1011	CPYFPTRN, CPYFMTRN, CPYFETRN — Main	FEAT_MOPS
0	01	1100	CPYFPN, CPYFMN, CPYFEN — Main	FEAT_MOPS
0	01	1101	CPYFPWTN, CPYFMWTN, CPYFEWTN — Main	FEAT_MOPS
0	01	1110	CPYFPRTN, CPYFMRTN, CPYFERTN — Main	FEAT_MOPS
0	01	1111	CPYFPTN, CPYFMTN, CPYFETN — Main	FEAT_MOPS
0	10	0000	CPYFP, CPYFM, CPYFE — Epilogue	FEAT_MOPS
0	10	0001	CPYFPWT, CPYFMWT, CPYFEWT — Epilogue	FEAT_MOPS
0	10	0010	CPYFPRT, CPYFMRT, CPYFERT — Epilogue	FEAT_MOPS
0	10	0011	CPYFPT, CPYFMT, CPYFET — Epilogue	FEAT_MOPS
0	10	0100	CPYFPWN, CPYFMWN, CPYFEWN — Epilogue	FEAT_MOPS
0	10	0101	CPYFPWTWN, CPYFMWTWN, CPYFEWTWN — Epilogue	FEAT_MOPS
0	10	0110	CPYFPRTWN, CPYFMRTWN, CPYFERTWN — Epilogue	FEAT_MOPS
0	10	0111	CPYFPTWN, CPYFMTWN, CPYFETWN — Epilogue	FEAT_MOPS
0	10	1000	CPYFPRN, CPYFMRN, CPYFERN — Epilogue	FEAT_MOPS
0	10	1001	CPYFPWTRN, CPYFMWTRN, CPYFEWTRN — Epilogue	FEAT_MOPS
0	10	1010	CPYFPRTRN, CPYFMRTRN, CPYFERTRN — Epilogue	FEAT_MOPS
0	10	1011	CPYFPTRN, CPYFMTRN, CPYFETRN — Epilogue	FEAT_MOPS

Decode fields			Instruction Details	Feature
o0	op1	op2		
0	10	1100	CPYFPN, CPYFMN, CPYFEN — Epilogue	FEAT_MOPS
0	10	1101	CPYFPWTN, CPYFMWTN, CPYFEWTN — Epilogue	FEAT_MOPS
0	10	1110	CPYFPRTN, CPYFMRTN, CPYFERTN — Epilogue	FEAT_MOPS
0	10	1111	CPYFPTN, CPYFMTN, CPYFETN — Epilogue	FEAT_MOPS
0	11	0000	SETP, SETM, SETE — Prologue	FEAT_MOPS
0	11	0001	SETPT, SETMT, SETET — Prologue	FEAT_MOPS
0	11	0010	SETPN, SETMN, SETEN — Prologue	FEAT_MOPS
0	11	0011	SETPTN, SETMTN, SETETN — Prologue	FEAT_MOPS
0	11	0100	SETP, SETM, SETE — Main	FEAT_MOPS
0	11	0101	SETPT, SETMT, SETET — Main	FEAT_MOPS
0	11	0110	SETPN, SETMN, SETEN — Main	FEAT_MOPS
0	11	0111	SETPTN, SETMTN, SETETN — Main	FEAT_MOPS
0	11	1000	SETP, SETM, SETE — Epilogue	FEAT_MOPS
0	11	1001	SETPT, SETMT, SETET — Epilogue	FEAT_MOPS
0	11	1010	SETPN, SETMN, SETEN — Epilogue	FEAT_MOPS
0	11	1011	SETPTN, SETMTN, SETETN — Epilogue	FEAT_MOPS
1	00	0000	CPYP, CPYM, CPYE — Prologue	FEAT_MOPS
1	00	0001	CPYPWT, CPYMW, CPYEWT — Prologue	FEAT_MOPS
1	00	0010	CPYPRT, CPYMRT, CPYERT — Prologue	FEAT_MOPS
1	00	0011	CPYPT, CPYMT, CPYET — Prologue	FEAT_MOPS
1	00	0100	CPYPWN, CPYMW, CPYEWN — Prologue	FEAT_MOPS
1	00	0101	CPYPWTWN, CPYMWWTWN, CPYEWTWN — Prologue	FEAT_MOPS
1	00	0110	CPYPRTWN, CPYMRTWN, CPYERTWN — Prologue	FEAT_MOPS
1	00	0111	CPYPTWN, CPYMTWN, CPYETWN — Prologue	FEAT_MOPS
1	00	1000	CPYPRN, CPYMRN, CPYERN — Prologue	FEAT_MOPS
1	00	1001	CPYPWTRN, CPYMWTRN, CPYEWTRN — Prologue	FEAT_MOPS
1	00	1010	CPYPRTRN, CPYMRTRN, CPYERTRN — Prologue	FEAT_MOPS
1	00	1011	CPYPTRN, CPYMTRN, CPYETRN — Prologue	FEAT_MOPS
1	00	1100	CPYPN, CPYMN, CPYEN — Prologue	FEAT_MOPS
1	00	1101	CPYPWTN, CPYMW, CPYEWTN — Prologue	FEAT_MOPS
1	00	1110	CPYPRTN, CPYMRTN, CPYERTN — Prologue	FEAT_MOPS
1	00	1111	CPYPTN, CPYMTN, CPYETN — Prologue	FEAT_MOPS
1	01	0000	CPYP, CPYM, CPYE — Main	FEAT_MOPS
1	01	0001	CPYPWT, CPYMW, CPYEWT — Main	FEAT_MOPS
1	01	0010	CPYPRT, CPYMRT, CPYERT — Main	FEAT_MOPS
1	01	0011	CPYPT, CPYMT, CPYET — Main	FEAT_MOPS
1	01	0100	CPYPWN, CPYMW, CPYEWN — Main	FEAT_MOPS
1	01	0101	CPYPWTWN, CPYMWWTWN, CPYEWTWN — Main	FEAT_MOPS
1	01	0110	CPYPRTWN, CPYMRTWN, CPYERTWN — Main	FEAT_MOPS
1	01	0111	CPYPTWN, CPYMTWN, CPYETWN — Main	FEAT_MOPS
1	01	1000	CPYPRN, CPYMRN, CPYERN — Main	FEAT_MOPS
1	01	1001	CPYPWTRN, CPYMWTRN, CPYEWTRN — Main	FEAT_MOPS
1	01	1010	CPYPRTRN, CPYMRTRN, CPYERTRN — Main	FEAT_MOPS
1	01	1011	CPYPTRN, CPYMTRN, CPYETRN — Main	FEAT_MOPS
1	01	1100	CPYPN, CPYMN, CPYEN — Main	FEAT_MOPS
1	01	1101	CPYPWTN, CPYMW, CPYEWTN — Main	FEAT_MOPS

Decode fields			Instruction Details	Feature
o0	op1	op2		
1	01	1110	CPYPRTN, CPYMRTN, CPYERTN — Main	FEAT_MOPS
1	01	1111	CPYPTN, CPYMTN, CPYETN — Main	FEAT_MOPS
1	10	0000	CPYP, CPYM, CPYE — Epilogue	FEAT_MOPS
1	10	0001	CPYPWT, CPYMW, CPYEWT — Epilogue	FEAT_MOPS
1	10	0010	CPYPRT, CPYMRT, CPYERT — Epilogue	FEAT_MOPS
1	10	0011	CPYPT, CPYMT, CPYET — Epilogue	FEAT_MOPS
1	10	0100	CPYPWN, CPYMW, CPYEWN — Epilogue	FEAT_MOPS
1	10	0101	CPYPWTWN, CPYMW, CPYEWTWN — Epilogue	FEAT_MOPS
1	10	0110	CPYPRTWN, CPYMRTWN, CPYERTWN — Epilogue	FEAT_MOPS
1	10	0111	CPYPTWN, CPYMTWN, CPYETWN — Epilogue	FEAT_MOPS
1	10	1000	CPYPRN, CPYMRN, CPYERN — Epilogue	FEAT_MOPS
1	10	1001	CPYPWTRN, CPYMWTRN, CPYEWTRN — Epilogue	FEAT_MOPS
1	10	1010	CPYPRTRN, CPYMRTRN, CPYERTRN — Epilogue	FEAT_MOPS
1	10	1011	CPYPTRN, CPYMTRN, CPYETRN — Epilogue	FEAT_MOPS
1	10	1100	CPYPN, CPYMN, CPYEN — Epilogue	FEAT_MOPS
1	10	1101	CPYPWTN, CPYMW, CPYEW — Epilogue	FEAT_MOPS
1	10	1110	CPYPRTN, CPYMRTN, CPYERTN — Epilogue	FEAT_MOPS
1	10	1111	CPYPTN, CPYMTN, CPYETN — Epilogue	FEAT_MOPS
1	11	0000	SETGP, SETGM, SETGE — Prologue	FEAT_MOPS && FEAT_MTE
1	11	0001	SETGPT, SETGMT, SETGET — Prologue	FEAT_MOPS && FEAT_MTE
1	11	0010	SETGPN, SETGMN, SETGEN — Prologue	FEAT_MOPS && FEAT_MTE
1	11	0011	SETGPTN, SETGMTN, SETGETN — Prologue	FEAT_MOPS && FEAT_MTE
1	11	0100	SETGP, SETGM, SETGE — Main	FEAT_MOPS && FEAT_MTE
1	11	0101	SETGPT, SETGMT, SETGET — Main	FEAT_MOPS && FEAT_MTE
1	11	0110	SETGPN, SETGMN, SETGEN — Main	FEAT_MOPS && FEAT_MTE
1	11	0111	SETGPTN, SETGMTN, SETGETN — Main	FEAT_MOPS && FEAT_MTE
1	11	1000	SETGP, SETGM, SETGE — Epilogue	FEAT_MOPS && FEAT_MTE
1	11	1001	SETGPT, SETGMT, SETGET — Epilogue	FEAT_MOPS && FEAT_MTE
1	11	1010	SETGPN, SETGMN, SETGEN — Epilogue	FEAT_MOPS && FEAT_MTE
1	11	1011	SETGPTN, SETGMTN, SETGETN — Epilogue	FEAT_MOPS && FEAT_MTE

Load/store no-allocate pair (offset)

The encodings in this section are decoded from [Loads and Stores](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opc		1	0	1	VR		0	0	0	L	imm7							Rt2					Rn				Rt				

Decode fields			Instruction Details	Feature
opc	VR	L		
00	0	0	STNP — 32-bit	-
00	0	1	LDNP — 32-bit	-
00	1	0	STNP (SIMD&FP) — 32-bit	FEAT_FP
00	1	1	LDNP (SIMD&FP) — 32-bit	FEAT_FP
01	0		UNALLOCATED	-
01	1	0	STNP (SIMD&FP) — 64-bit	FEAT_FP
01	1	1	LDNP (SIMD&FP) — 64-bit	FEAT_FP
10	0	0	STNP — 64-bit	-
10	0	1	LDNP — 64-bit	-
10	1	0	STNP (SIMD&FP) — 128-bit	FEAT_FP
10	1	1	LDNP (SIMD&FP) — 128-bit	FEAT_FP
11	0	0	STTNP	FEAT_LSUI
11	0	1	LDTNP	FEAT_LSUI
11	1	0	STTNP (SIMD&FP)	FEAT_FP && FEAT_LSUI
11	1	1	LDTNP (SIMD&FP)	FEAT_FP && FEAT_LSUI

Load/store register pair (post-indexed)

The encodings in this section are decoded from [Loads and Stores](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opc		1	0	1	VR	0	0	1	L	imm7							Rt2					Rn					Rt				

Decode fields			Instruction Details	Feature
opc	VR	L		
00	0	0	STP — 32-bit	-
00	0	1	LDP — 32-bit	-
00	1	0	STP (SIMD&FP) — 32-bit	FEAT_FP
00	1	1	LDP (SIMD&FP) — 32-bit	FEAT_FP
01	0	0	STGP	FEAT_MTE
01	0	1	LDPSW	-
01	1	0	STP (SIMD&FP) — 64-bit	FEAT_FP
01	1	1	LDP (SIMD&FP) — 64-bit	FEAT_FP
10	0	0	STP — 64-bit	-
10	0	1	LDP — 64-bit	-
10	1	0	STP (SIMD&FP) — 128-bit	FEAT_FP
10	1	1	LDP (SIMD&FP) — 128-bit	FEAT_FP
11	0	0	STTP	FEAT_LSUI
11	0	1	LDTP	FEAT_LSUI
11	1	0	STTP (SIMD&FP)	FEAT_FP && FEAT_LSUI
11	1	1	LDTP (SIMD&FP)	FEAT_FP && FEAT_LSUI

Load/store register pair (offset)

The encodings in this section are decoded from [Loads and Stores](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opc		1	0	1	VR	0	1	0	L	imm7							Rt2					Rn					Rt				

Decode fields			Instruction Details	Feature
opc	VR	L		
00	0	0	STP — 32-bit	-
00	0	1	LDP — 32-bit	-
00	1	0	STP (SIMD&FP) — 32-bit	FEAT_FP
00	1	1	LDP (SIMD&FP) — 32-bit	FEAT_FP
01	0	0	STGP	FEAT_MTE
01	0	1	LDPSW	-
01	1	0	STP (SIMD&FP) — 64-bit	FEAT_FP
01	1	1	LDP (SIMD&FP) — 64-bit	FEAT_FP
10	0	0	STP — 64-bit	-
10	0	1	LDP — 64-bit	-
10	1	0	STP (SIMD&FP) — 128-bit	FEAT_FP
10	1	1	LDP (SIMD&FP) — 128-bit	FEAT_FP
11	0	0	STTP	FEAT_LSUI
11	0	1	LDTP	FEAT_LSUI
11	1	0	STTP (SIMD&FP)	FEAT_FP && FEAT_LSUI
11	1	1	LDTP (SIMD&FP)	FEAT_FP && FEAT_LSUI

Load/store register pair (pre-indexed)

The encodings in this section are decoded from [Loads and Stores](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opc		1	0	1	VR	0	1	1	L	imm7							Rt2					Rn					Rt				

Decode fields			Instruction Details	Feature
opc	VR	L		
00	0	0	STP — 32-bit	-
00	0	1	LDP — 32-bit	-
00	1	0	STP (SIMD&FP) — 32-bit	FEAT_FP
00	1	1	LDP (SIMD&FP) — 32-bit	FEAT_FP
01	0	0	STGP	FEAT_MTE
01	0	1	LDPSW	-
01	1	0	STP (SIMD&FP) — 64-bit	FEAT_FP
01	1	1	LDP (SIMD&FP) — 64-bit	FEAT_FP
10	0	0	STP — 64-bit	-
10	0	1	LDP — 64-bit	-
10	1	0	STP (SIMD&FP) — 128-bit	FEAT_FP
10	1	1	LDP (SIMD&FP) — 128-bit	FEAT_FP
11	0	0	STTP	FEAT_LSUI
11	0	1	LDTP	FEAT_LSUI
11	1	0	STTP (SIMD&FP)	FEAT_FP && FEAT_LSUI
11	1	1	LDTP (SIMD&FP)	FEAT_FP && FEAT_LSUI

Load/store register (unscaled immediate)

The encodings in this section are decoded from [Loads and Stores](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
size		1	1	1	VR	0	0	opc		0	imm9										0		0	Rn					Rt			

Decode fields			Instruction Details	Feature
size	VR	opc		
00	0	00	STURB	-
00	0	01	LDURB	-
00	0	10	LDURSB — 64-bit	-
00	0	11	LDURSB — 32-bit	-
00	1	00	STUR (SIMD&FP) — 8-bit	FEAT_FP
00	1	01	LDUR (SIMD&FP) — 8-bit	FEAT_FP
00	1	10	STUR (SIMD&FP) — 128-bit	FEAT_FP
00	1	11	LDUR (SIMD&FP) — 128-bit	FEAT_FP
01	0	00	STURH	-
01	0	01	LDURH	-
01	0	10	LDURSH — 64-bit	-
01	0	11	LDURSH — 32-bit	-
01	1	00	STUR (SIMD&FP) — 16-bit	FEAT_FP
01	1	01	LDUR (SIMD&FP) — 16-bit	FEAT_FP
1x	0	11	UNALLOCATED	-
10	0	00	STUR — 32-bit	-
10	0	01	LDUR — 32-bit	-
10	0	10	LDURSW	-
10	1	00	STUR (SIMD&FP) — 32-bit	FEAT_FP
10	1	01	LDUR (SIMD&FP) — 32-bit	FEAT_FP
11	0	00	STUR — 64-bit	-
11	0	01	LDUR — 64-bit	-
11	0	10	PRFUM	-
11	1	00	STUR (SIMD&FP) — 64-bit	FEAT_FP
11	1	01	LDUR (SIMD&FP) — 64-bit	FEAT_FP
!= 00	1	1x	UNALLOCATED	-

Load/store register (immediate post-indexed)

The encodings in this section are decoded from [Loads and Stores](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
size		1	1	1	VR	0	0	opc		0	imm9										0	1	Rn				Rt				

Decode fields			Instruction Details	Feature
size	VR	opc		
00	0	00	STRB (immediate)	-
00	0	01	LDRB (immediate)	-
00	0	10	LDRSB (immediate) — 64-bit	-
00	0	11	LDRSB (immediate) — 32-bit	-
00	1	00	STR (immediate, SIMD&FP) — 8-bit	FEAT_FP
00	1	01	LDR (immediate, SIMD&FP) — 8-bit	FEAT_FP
00	1	10	STR (immediate, SIMD&FP) — 128-bit	FEAT_FP
00	1	11	LDR (immediate, SIMD&FP) — 128-bit	FEAT_FP
01	0	00	STRH (immediate)	-
01	0	01	LDRH (immediate)	-
01	0	10	LDRSH (immediate) — 64-bit	-
01	0	11	LDRSH (immediate) — 32-bit	-
01	1	00	STR (immediate, SIMD&FP) — 16-bit	FEAT_FP
01	1	01	LDR (immediate, SIMD&FP) — 16-bit	FEAT_FP
01	1	1x	UNALLOCATED	-
10	0	00	STR (immediate) — 32-bit	-
10	0	01	LDR (immediate) — 32-bit	-
10	0	10	LDRSW (immediate)	-
10	0	11	UNALLOCATED	-
10	1	00	STR (immediate, SIMD&FP) — 32-bit	FEAT_FP
10	1	01	LDR (immediate, SIMD&FP) — 32-bit	FEAT_FP
10	1	1x	UNALLOCATED	-
11		1x	UNALLOCATED	-
11	0	00	STR (immediate) — 64-bit	-
11	0	01	LDR (immediate) — 64-bit	-
11	1	00	STR (immediate, SIMD&FP) — 64-bit	FEAT_FP
11	1	01	LDR (immediate, SIMD&FP) — 64-bit	FEAT_FP

Load/store register (unprivileged)

The encodings in this section are decoded from [Loads and Stores](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
size		1	1	1	VR	0	0	opc		0	imm9										1	0	Rn					Rt				

Decode fields		Instruction Details	
size	VR	opc	
	1		UNALLOCATED
00	0	00	STTRB
00	0	01	LDTRB
00	0	10	LDTRSB — 64-bit
00	0	11	LDTRSB — 32-bit
01	0	00	STTRH
01	0	01	LDTRH
01	0	10	LDTRSH — 64-bit
01	0	11	LDTRSH — 32-bit
10	0	00	STTR — 32-bit
10	0	01	LDTR — 32-bit
10	0	10	LDTRSW
10	0	11	UNALLOCATED
11	0	00	STTR — 64-bit
11	0	01	LDTR — 64-bit
11	0	1x	UNALLOCATED

Load/store register (immediate pre-indexed)

The encodings in this section are decoded from [Loads and Stores](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
size	1	1	1	VR	0	0		opc	0	imm9											1	1	Rn					Rt			

Decode fields			Instruction Details	Feature
size	VR	opc		
00	0	00	STRB (immediate)	-
00	0	01	LDRB (immediate)	-
00	0	10	LDRSB (immediate) — 64-bit	-
00	0	11	LDRSB (immediate) — 32-bit	-
00	1	00	STR (immediate, SIMD&FP) — 8-bit	FEAT_FP
00	1	01	LDR (immediate, SIMD&FP) — 8-bit	FEAT_FP
00	1	10	STR (immediate, SIMD&FP) — 128-bit	FEAT_FP
00	1	11	LDR (immediate, SIMD&FP) — 128-bit	FEAT_FP
01	0	00	STRH (immediate)	-
01	0	01	LDRH (immediate)	-
01	0	10	LDRSH (immediate) — 64-bit	-
01	0	11	LDRSH (immediate) — 32-bit	-
01	1	00	STR (immediate, SIMD&FP) — 16-bit	FEAT_FP
01	1	01	LDR (immediate, SIMD&FP) — 16-bit	FEAT_FP
01	1	1x	UNALLOCATED	-
10	0	00	STR (immediate) — 32-bit	-
10	0	01	LDR (immediate) — 32-bit	-
10	0	10	LDRSW (immediate)	-
10	0	11	UNALLOCATED	-
10	1	00	STR (immediate, SIMD&FP) — 32-bit	FEAT_FP
10	1	01	LDR (immediate, SIMD&FP) — 32-bit	FEAT_FP
10	1	1x	UNALLOCATED	-
11		1x	UNALLOCATED	-
11	0	00	STR (immediate) — 64-bit	-
11	0	01	LDR (immediate) — 64-bit	-
11	1	00	STR (immediate, SIMD&FP) — 64-bit	FEAT_FP
11	1	01	LDR (immediate, SIMD&FP) — 64-bit	FEAT_FP

Atomic memory operations

The encodings in this section are decoded from [Loads and Stores](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
size		1	1	1	VR	0	0	A	R	1	Rs					o3	opc			0	0	Rn				Rt					

size	VR	A	R	Decode fields			Rt	Instruction Details	Feature
				Rs	o3	opc			
	0	0			1	100		UNALLOCATED	-
	0	0			1	11x		UNALLOCATED	-
	0	1	1		1	100		UNALLOCATED	-
	1				0	001		UNALLOCATED	-
	1				0	01x		UNALLOCATED	-
	1	0			1		!= 11111	UNALLOCATED	-
	1	0			1	001	11111	UNALLOCATED	-
	1	0			1	01x	11111	UNALLOCATED	-
	1	1			1			UNALLOCATED	-
0x	0		0		1	101		UNALLOCATED	-
0x	0	0	1		1	101		UNALLOCATED	-
0x	0	1			1	11x		UNALLOCATED	-
0x	0	1	1		1	101		UNALLOCATED	-
00	0	0	0		0	000		LDADDB, LDADDAB, LDADDALB, LDADDLB — No memory ordering	FEAT_LSE
00	0	0	0		0	001		LDCLRB, LDCLRAB, LDCLRALB, LDCLRLB — No memory ordering	FEAT_LSE
00	0	0	0		0	010		LDEORB, LDEORAB, LDEORALB, LDEORLB — No memory ordering	FEAT_LSE
00	0	0	0		0	011		LDSETB, LDSETAB, LDSETALB, LDSETLB — No memory ordering	FEAT_LSE
00	0	0	0		0	100		LDSMAXB, LDSMAXAB, LDSMAXALB, LDSMAXLB — No memory ordering	FEAT_LSE
00	0	0	0		0	101		LDSMINB, LDSMINAB, LDSMINALB, LDSMINLB — No memory ordering	FEAT_LSE
00	0	0	0		0	110		LDUMAXB, LDUMAXAB, LDUMAXALB, LDUMAXLB — No memory ordering	FEAT_LSE
00	0	0	0		0	111		LDUMINB, LDUMINAB, LDUMINALB, LDUMINLB — No memory ordering	FEAT_LSE
00	0	0	0		1	000		SWPB, SWPAB, SWPALB, SWPLB — SWPB	FEAT_LSE
00	0	0	0		1	001		RCWCLR, RCWCLRA, RCWCLRAL, RCWCLRL — RCWCLR	FEAT_THE
00	0	0	0		1	010		RCWSWP, RCWSWPA, RCWSWPAL, RCWSWPL — RCWSWP	FEAT_THE
00	0	0	0		1	011		RCWSET , RCWSETA , RCWSETAL , RCWSETL — RCWSET	FEAT_THE
00	0	0	1		0	000		LDADDB, LDADDAB, LDADDALB, LDADDLB — Release	FEAT_LSE
00	0	0	1		0	001		LDCLRB, LDCLRAB, LDCLRALB, LDCLRLB — Release	FEAT_LSE
00	0	0	1		0	010		LDEORB, LDEORAB, LDEORALB, LDEORLB — Release	FEAT_LSE
00	0	0	1		0	011		LDSETB, LDSETAB, LDSETALB, LDSETLB — Release	FEAT_LSE
00	0	0	1		0	100		LDSMAXB, LDSMAXAB, LDSMAXALB, LDSMAXLB — Release	FEAT_LSE
00	0	0	1		0	101		LDSMINB, LDSMINAB, LDSMINALB, LDSMINLB — Release	FEAT_LSE
00	0	0	1		0	110		LDUMAXB, LDUMAXAB, LDUMAXALB, LDUMAXLB — Release	FEAT_LSE

size	VR	A	R	Decode fields			Rt	Instruction Details	Feature
				Rs	o3	opc			
00	0	0	1		0	111		LDUMINB, LDUMINAB, LDUMINALB, LDUMINLB — Release	FEAT_LSE
00	0	0	1		1	000		SWPB, SWPAB, SWPALB, SWPLB — SWPLB	FEAT_LSE
00	0	0	1		1	001		RCWCLR, RCWCLRA, RCWCLRAL, RCWCLRL — RCWCLRL	FEAT_THE
00	0	0	1		1	010		RCWSWP, RCWSWPA, RCWSWPAL, RCWSWPL — RCWSWPL	FEAT_THE
00	0	0	1		1	011		RCWSET, RCWSETA, RCWSETAL, RCWSETL — RCWSETL	FEAT_THE
00	0	1	0		0	000		LDADDB, LDADDAB, LDADDALB, LDADDLB — Acquire	FEAT_LSE
00	0	1	0		0	001		LDCLRB, LDCLRAB, LDCLRALB, LDCLRLB — Acquire	FEAT_LSE
00	0	1	0		0	010		LDEORB, LDEORAB, LDEORALB, LDEORLB — Acquire	FEAT_LSE
00	0	1	0		0	011		LDSETB, LDSETAB, LDSETALB, LDSETLB — Acquire	FEAT_LSE
00	0	1	0		0	100		LDSMAXB, LDSMAXAB, LDSMAXALB, LDSMAXLB — Acquire	FEAT_LSE
00	0	1	0		0	101		LDSMINB, LDSMINAB, LDSMINALB, LDSMINLB — Acquire	FEAT_LSE
00	0	1	0		0	110		LDUMAXB, LDUMAXAB, LDUMAXALB, LDUMAXLB — Acquire	FEAT_LSE
00	0	1	0		0	111		LDUMINB, LDUMINAB, LDUMINALB, LDUMINLB — Acquire	FEAT_LSE
00	0	1	0		1	000		SWPB, SWPAB, SWPALB, SWPLB — SWPAB	FEAT_LSE
00	0	1	0		1	001		RCWCLR, RCWCLRA, RCWCLRAL, RCWCLRL — RCWCLRA	FEAT_THE
00	0	1	0		1	010		RCWSWP, RCWSWPA, RCWSWPAL, RCWSWPL — RCWSWPA	FEAT_THE
00	0	1	0		1	011		RCWSET, RCWSETA, RCWSETAL, RCWSETL — RCWSETA	FEAT_THE
00	0	1	0		1	100		LDAPRB	FEAT_LRCPC
00	0	1	1		0	000		LDADDB, LDADDAB, LDADDALB, LDADDLB — Acquire-release	FEAT_LSE
00	0	1	1		0	001		LDCLRB, LDCLRAB, LDCLRALB, LDCLRLB — Acquire-release	FEAT_LSE
00	0	1	1		0	010		LDEORB, LDEORAB, LDEORALB, LDEORLB — Acquire-release	FEAT_LSE
00	0	1	1		0	011		LDSETB, LDSETAB, LDSETALB, LDSETLB — Acquire-release	FEAT_LSE
00	0	1	1		0	100		LDSMAXB, LDSMAXAB, LDSMAXALB, LDSMAXLB — Acquire-release	FEAT_LSE
00	0	1	1		0	101		LDSMINB, LDSMINAB, LDSMINALB, LDSMINLB — Acquire-release	FEAT_LSE
00	0	1	1		0	110		LDUMAXB, LDUMAXAB, LDUMAXALB, LDUMAXLB — Acquire-release	FEAT_LSE
00	0	1	1		0	111		LDUMINB, LDUMINAB, LDUMINALB, LDUMINLB — Acquire-release	FEAT_LSE
00	0	1	1		1	000		SWPB, SWPAB, SWPALB, SWPLB — SWPALB	FEAT_LSE

size	VR	A	R	Decode fields			Rt	Instruction Details	Feature
				Rs	o3	opc			
00	0	1	1		1	001		RCWCLR, RCWCLRA, RCWCLRAL, RCWCLRL — RCWCLRAL	FEAT_THE
00	0	1	1		1	010		RCWSWP, RCWSWPA, RCWSWPAL, RCWSWPL — RCWSWPAL	FEAT_THE
00	0	1	1		1	011		RCWSET , RCWSETA , RCWSETAL , RCWSETL — RCWSETAL	FEAT_THE
00	1	0	0		0	000		LDBFADD, LDBFADDA, LDBFADDAL, LDBFADDL — No memory ordering	FEAT_LSFE
00	1	0	0		0	100		LDBFMAX, LDBFMAXA, LDBFMAXAL, LDBFMAXL — No memory ordering	FEAT_LSFE
00	1	0	0		0	101		LDBFMIN, LDBFMINA, LDBFMINAL, LDBFMINL — No memory ordering	FEAT_LSFE
00	1	0	0		0	110		LDBFMAXNM, LDBFMAXNMA, LDBFMAXNMAL, LDBFMAXNML — No memory ordering	FEAT_LSFE
00	1	0	0		0	111		LDBFMINNM, LDBFMINNMA, LDBFMINNMAL, LDBFMINNML — No memory ordering	FEAT_LSFE
00	1	0	0		1	000	11111	STBFADD, STBFADDL — No memory ordering	FEAT_LSFE
00	1	0	0		1	100	11111	STBFMAX, STBFMAXL — No memory ordering	FEAT_LSFE
00	1	0	0		1	101	11111	STBFMIN, STBFMINL — No memory ordering	FEAT_LSFE
00	1	0	0		1	110	11111	STBFMAXNM, STBFMAXNML — No memory ordering	FEAT_LSFE
00	1	0	0		1	111	11111	STBFMINNM, STBFMINNML — No memory ordering	FEAT_LSFE
00	1	0	1		0	000		LDBFADD, LDBFADDA, LDBFADDAL, LDBFADDL — Release	FEAT_LSFE
00	1	0	1		0	100		LDBFMAX, LDBFMAXA, LDBFMAXAL, LDBFMAXL — Release	FEAT_LSFE
00	1	0	1		0	101		LDBFMIN, LDBFMINA, LDBFMINAL, LDBFMINL — Release	FEAT_LSFE
00	1	0	1		0	110		LDBFMAXNM, LDBFMAXNMA, LDBFMAXNMAL, LDBFMAXNML — Release	FEAT_LSFE
00	1	0	1		0	111		LDBFMINNM, LDBFMINNMA, LDBFMINNMAL, LDBFMINNML — Release	FEAT_LSFE
00	1	0	1		1	000	11111	STBFADD, STBFADDL — Release	FEAT_LSFE
00	1	0	1		1	100	11111	STBFMAX, STBFMAXL — Release	FEAT_LSFE
00	1	0	1		1	101	11111	STBFMIN, STBFMINL — Release	FEAT_LSFE
00	1	0	1		1	110	11111	STBFMAXNM, STBFMAXNML — Release	FEAT_LSFE
00	1	0	1		1	111	11111	STBFMINNM, STBFMINNML — Release	FEAT_LSFE
00	1	1	0		0	000		LDBFADD, LDBFADDA, LDBFADDAL, LDBFADDL — Acquire	FEAT_LSFE
00	1	1	0		0	100		LDBFMAX, LDBFMAXA, LDBFMAXAL, LDBFMAXL — Acquire	FEAT_LSFE
00	1	1	0		0	101		LDBFMIN, LDBFMINA, LDBFMINAL, LDBFMINL — Acquire	FEAT_LSFE

size	VR	A	R	Decode fields			Rt	Instruction Details	Feature
				Rs	o3	opc			
00	1	1	0		0	110		LDBFMAXNM, LDBFMAXNMA, LDBFMAXNMAL, LDBFMAXNML — Acquire	FEAT_LSFE
00	1	1	0		0	111		LDBFMINNM, LDBFMINNMA, LDBFMINNMAL, LDBFMINNML — Acquire	FEAT_LSFE
00	1	1	1		0	000		LDBFADD, LDBFADDA, LDBFADDAL, LDBFADDL — Acquire-release	FEAT_LSFE
00	1	1	1		0	100		LDBFMAX, LDBFMAXA, LDBFMAXAL, LDBFMAXL — Acquire-release	FEAT_LSFE
00	1	1	1		0	101		LDBFMIN, LDBFMINA, LDBFMINAL, LDBFMINL — Acquire-release	FEAT_LSFE
00	1	1	1		0	110		LDBFMAXNM, LDBFMAXNMA, LDBFMAXNMAL, LDBFMAXNML — Acquire-release	FEAT_LSFE
00	1	1	1		0	111		LDBFMINNM, LDBFMINNMA, LDBFMINNMAL, LDBFMINNML — Acquire-release	FEAT_LSFE
01	0	0	0		0	000		LDADDH, LDADDAH, LDADDALH, LDADDLH — No memory ordering	FEAT_LSE
01	0	0	0		0	001		LDCLRH, LDCLRAH, LDCLRALH, LDCLRLH — No memory ordering	FEAT_LSE
01	0	0	0		0	010		LDEORH, LDEORAH, LDEORALH, LDEORLH — No memory ordering	FEAT_LSE
01	0	0	0		0	011		LDSETH, LDSETAH, LDSETALH, LDSETLH — No memory ordering	FEAT_LSE
01	0	0	0		0	100		LDSMAXH, LDSMAXAH, LDSMAXALH, LDSMAXLH — No memory ordering	FEAT_LSE
01	0	0	0		0	101		LDSMINH, LDSMINAH, LDSMINALH, LDSMINLH — No memory ordering	FEAT_LSE
01	0	0	0		0	110		LDUMAXH, LDUMAXAH, LDUMAXALH, LDUMAXLH — No memory ordering	FEAT_LSE
01	0	0	0		0	111		LDUMINH, LDUMINAH, LDUMINALH, LDUMINLH — No memory ordering	FEAT_LSE
01	0	0	0		1	000		SWPH, SWPAH, SWPALH, SWPLH — SWPH	FEAT_LSE
01	0	0	0		1	001		RCWSCLR, RCWSCLRA, RCWSCLRAL, RCWSCLRL — RCWSCLR	FEAT_THE
01	0	0	0		1	010		RCWSSWP, RCWSSWPA, RCWSSWPAL, RCWSSWPL — RCWSSWP	FEAT_THE
01	0	0	0		1	011		RCWSSET , RCWSSETA , RCWSSETAL , RCWSSETL — RCWSSET	FEAT_THE
01	0	0	1		0	000		LDADDH, LDADDAH, LDADDALH, LDADDLH — Release	FEAT_LSE
01	0	0	1		0	001		LDCLRH, LDCLRAH, LDCLRALH, LDCLRLH — Release	FEAT_LSE
01	0	0	1		0	010		LDEORH, LDEORAH, LDEORALH, LDEORLH — Release	FEAT_LSE
01	0	0	1		0	011		LDSETH, LDSETAH, LDSETALH, LDSETLH — Release	FEAT_LSE
01	0	0	1		0	100		LDSMAXH, LDSMAXAH, LDSMAXALH, LDSMAXLH — Release	FEAT_LSE

size	VR	A	R	Decode fields			Rt	Instruction Details	Feature
				Rs	o3	opc			
01	0	0	1		0	101		LDSMINH, LDSMINAH, LDSMINALH, LDSMINLH — Release	FEAT_LSE
01	0	0	1		0	110		LDUMAXH, LDUMAXAH, LDUMAXALH, LDUMAXLH — Release	FEAT_LSE
01	0	0	1		0	111		LDUMINH, LDUMINAH, LDUMINALH, LDUMINLH — Release	FEAT_LSE
01	0	0	1		1	000		SWPH, SWPAH, SWPALH, SWPLH — SWPLH	FEAT_LSE
01	0	0	1		1	001		RCWSCLR, RCWSCLRA, RCWSCLRAL, RCWSCLRL — RCWSCLRL	FEAT_THE
01	0	0	1		1	010		RCWSSWP, RCWSSWA, RCWSSWPAL, RCWSSWPL — RCWSSWPL	FEAT_THE
01	0	0	1		1	011		RCWSSET, RCWSSETA, RCWSSETAL, RCWSSETL — RCWSSETL	FEAT_THE
01	0	1	0		0	000		LDADDH, LDADDAH, LDADDALH, LDADDLH — Acquire	FEAT_LSE
01	0	1	0		0	001		LDCLRH, LDCLRAH, LDCLRALH, LDCLRLH — Acquire	FEAT_LSE
01	0	1	0		0	010		LDEORH, LDEORAH, LDEORALH, LDEORLH — Acquire	FEAT_LSE
01	0	1	0		0	011		LDSETH, LDSETAH, LDSETALH, LDSETLH — Acquire	FEAT_LSE
01	0	1	0		0	100		LDSMAXH, LDSMAXAH, LDSMAXALH, LDSMAXLH — Acquire	FEAT_LSE
01	0	1	0		0	101		LDSMINH, LDSMINAH, LDSMINALH, LDSMINLH — Acquire	FEAT_LSE
01	0	1	0		0	110		LDUMAXH, LDUMAXAH, LDUMAXALH, LDUMAXLH — Acquire	FEAT_LSE
01	0	1	0		0	111		LDUMINH, LDUMINAH, LDUMINALH, LDUMINLH — Acquire	FEAT_LSE
01	0	1	0		1	000		SWPH, SWPAH, SWPALH, SWPLH — SWPAH	FEAT_LSE
01	0	1	0		1	001		RCWSCLR, RCWSCLRA, RCWSCLRAL, RCWSCLRL — RCWSCLRA	FEAT_THE
01	0	1	0		1	010		RCWSSWP, RCWSSWA, RCWSSWPAL, RCWSSWPL — RCWSSWA	FEAT_THE
01	0	1	0		1	011		RCWSSET, RCWSSETA, RCWSSETAL, RCWSSETL — RCWSSETA	FEAT_THE
01	0	1	0		1	100		LDAPRH	FEAT_LRCPC
01	0	1	1		0	000		LDADDH, LDADDAH, LDADDALH, LDADDLH — Acquire-release	FEAT_LSE
01	0	1	1		0	001		LDCLRH, LDCLRAH, LDCLRALH, LDCLRLH — Acquire-release	FEAT_LSE
01	0	1	1		0	010		LDEORH, LDEORAH, LDEORALH, LDEORLH — Acquire-release	FEAT_LSE
01	0	1	1		0	011		LDSETH, LDSETAH, LDSETALH, LDSETLH — Acquire-release	FEAT_LSE
01	0	1	1		0	100		LDSMAXH, LDSMAXAH, LDSMAXALH, LDSMAXLH — Acquire-release	FEAT_LSE

size	VR	A	R	Decode fields			Rt	Instruction Details	Feature
				Rs	o3	opc			
01	0	1	1		0	101		LDSMINH, LDSMINAH, LDSMINALH, LDSMINLH — Acquire-release	FEAT_LSE
01	0	1	1		0	110		LDUMAXH, LDUMAXAH, LDUMAXALH, LDUMAXLH — Acquire-release	FEAT_LSE
01	0	1	1		0	111		LDUMINH, LDUMINAH, LDUMINALH, LDUMINLH — Acquire-release	FEAT_LSE
01	0	1	1		1	000		SWPH, SWPAH, SWPALH, SWPLH — SWPALH	FEAT_LSE
01	0	1	1		1	001		RCWSCLR, RCWSCLRA, RCWSCLRAL, RCWSCLRL — RCWSCLRAL	FEAT_THE
01	0	1	1		1	010		RCWSSWP, RCWSSWPA, RCWSSWPAL, RCWSSWPL — RCWSSWPAL	FEAT_THE
01	0	1	1		1	011		RCWSSET, RCWSSETA, RCWSSETAL, RCWSSETL — RCWSSETAL	FEAT_THE
01	1	0	0		0	000		LDFADD, LDFADDA, LDFADDAL, LDFADDL — Half-precision no memory ordering	FEAT_LSFE
01	1	0	0		0	100		LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL — Half-precision no memory ordering	FEAT_LSFE
01	1	0	0		0	101		LDFMIN, LDFMINA, LDFMINAL, LDFMINL — Half-precision no memory ordering	FEAT_LSFE
01	1	0	0		0	110		LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML — Half-precision no memory ordering	FEAT_LSFE
01	1	0	0		0	111		LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML — Half-precision no memory ordering	FEAT_LSFE
01	1	0	0		1	000	11111	STFADD, STFADDL — Half-precision no memory ordering	FEAT_LSFE
01	1	0	0		1	100	11111	STFMAX, STFMAXL — Half- precision no memory ordering	FEAT_LSFE
01	1	0	0		1	101	11111	STFMIN, STFMINL — Half-precision no memory ordering	FEAT_LSFE
01	1	0	0		1	110	11111	STFMAXNM, STFMAXNML — Half- precision no memory ordering	FEAT_LSFE
01	1	0	0		1	111	11111	STFMINNM, STFMINNML — Half- precision no memory ordering	FEAT_LSFE
01	1	0	1		0	000		LDFADD, LDFADDA, LDFADDAL, LDFADDL — Half-precision release	FEAT_LSFE
01	1	0	1		0	100		LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL — Half-precision release	FEAT_LSFE
01	1	0	1		0	101		LDFMIN, LDFMINA, LDFMINAL, LDFMINL — Half-precision release	FEAT_LSFE
01	1	0	1		0	110		LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML — Half-precision release	FEAT_LSFE
01	1	0	1		0	111		LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML — Half-precision release	FEAT_LSFE
01	1	0	1		1	000	11111	STFADD, STFADDL — Half-precision release	FEAT_LSFE
01	1	0	1		1	100	11111	STFMAX, STFMAXL — Half- precision release	FEAT_LSFE
01	1	0	1		1	101	11111	STFMIN, STFMINL — Half-precision release	FEAT_LSFE

size	VR	A	R	Decode fields			Rt	Instruction Details	Feature
				Rs	o3	opc			
01	1	0	1		1	110	11111	STFMAXNM, STFMAXNML — Half-precision release	FEAT_LSFE
01	1	0	1		1	111	11111	STFMINNM, STFMINNML — Half-precision release	FEAT_LSFE
01	1	1	0		0	000		LDFADD, LDFADDA, LDFADDAL, LDFADDL — Half-precision acquire	FEAT_LSFE
01	1	1	0		0	100		LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL — Half-precision acquire	FEAT_LSFE
01	1	1	0		0	101		LDFMIN, LDFMINA, LDFMINAL, LDFMINL — Half-precision acquire	FEAT_LSFE
01	1	1	0		0	110		LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML — Half-precision acquire	FEAT_LSFE
01	1	1	0		0	111		LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML — Half-precision acquire	FEAT_LSFE
01	1	1	1		0	000		LDFADD, LDFADDA, LDFADDAL, LDFADDL — Half-precision acquire-release	FEAT_LSFE
01	1	1	1		0	100		LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL — Half-precision acquire-release	FEAT_LSFE
01	1	1	1		0	101		LDFMIN, LDFMINA, LDFMINAL, LDFMINL — Half-precision acquire-release	FEAT_LSFE
01	1	1	1		0	110		LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML — Half-precision acquire-release	FEAT_LSFE
01	1	1	1		0	111		LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML — Half-precision acquire-release	FEAT_LSFE
1x	0	1			1	x1x		UNALLOCATED	!
1x	0	1			1	!= x00x01		UNALLOCATED	-
10	0	0			1	x01		UNALLOCATED	-
10	0	0			1	010		UNALLOCATED	-
10	0	0			1	011		UNALLOCATED	-
10	0	0	0		0	000		LDADD, LDADDA, LDADDAL, LDADDL — 32-bit no memory ordering	FEAT_LSE
10	0	0	0		0	001		LDCLR, LDCLRA, LDCLRAL, LDCLRL — 32-bit no memory ordering	FEAT_LSE
10	0	0	0		0	010		LDEOR, LDEORA, LDEORAL, LDEORL — 32-bit no memory ordering	FEAT_LSE
10	0	0	0		0	011		LDSET, LDSETA, LDSETAL, LDSETL — 32-bit no memory ordering	FEAT_LSE
10	0	0	0		0	100		LDSMAX, LDSMAXA, LDSMAXAL, LDSMAXL — 32-bit no memory ordering	FEAT_LSE
10	0	0	0		0	101		LDSMIN, LDSMINA, LDSMINAL, LDSMINL — 32-bit no memory ordering	FEAT_LSE
10	0	0	0		0	110		LDUMAX, LDUMAXA, LDUMAXAL, LDUMAXL — 32-bit no memory ordering	FEAT_LSE
10	0	0	0		0	111		LDUMIN, LDUMINA, LDUMINAL, LDUMINL — 32-bit no memory ordering	FEAT_LSE

size	VR	A	R	Decode fields			Rt	Instruction Details	Feature
				Rs	o3	opc			
10	0	0	0		1	000		SWP, SWPA, SWPAL, SWPL — 32-bit SWP	FEAT_LSE
10	0	0	1		0	000		LDADD, LDADDA, LDADDAL, LDADDL — 32-bit release	FEAT_LSE
10	0	0	1		0	001		LDCLR, LDCLRA, LDCLRAL, LDCLRL — 32-bit release	FEAT_LSE
10	0	0	1		0	010		LDEOR, LDEORA, LDEORAL, LDEORL — 32-bit release	FEAT_LSE
10	0	0	1		0	011		LDSET, LDSETA, LDSETAL, LDSETL — 32-bit release	FEAT_LSE
10	0	0	1		0	100		LDSMAX, LDSMAXA, LDSMAXAL, LDSMAXL — 32-bit release	FEAT_LSE
10	0	0	1		0	101		LDSMIN, LDSMINA, LDSMINAL, LDSMINL — 32-bit release	FEAT_LSE
10	0	0	1		0	110		LDUMAX, LDUMAXA, LDUMAXAL, LDUMAXL — 32-bit release	FEAT_LSE
10	0	0	1		0	111		LDUMIN, LDUMINA, LDUMINAL, LDUMINL — 32-bit release	FEAT_LSE
10	0	0	1		1	000		SWP, SWPA, SWPAL, SWPL — 32-bit SWPL	FEAT_LSE
10	0	1	0		0	000		LDADD, LDADDA, LDADDAL, LDADDL — 32-bit acquire	FEAT_LSE
10	0	1	0		0	001		LDCLR, LDCLRA, LDCLRAL, LDCLRL — 32-bit acquire	FEAT_LSE
10	0	1	0		0	010		LDEOR, LDEORA, LDEORAL, LDEORL — 32-bit acquire	FEAT_LSE
10	0	1	0		0	011		LDSET, LDSETA, LDSETAL, LDSETL — 32-bit acquire	FEAT_LSE
10	0	1	0		0	100		LDSMAX, LDSMAXA, LDSMAXAL, LDSMAXL — 32-bit acquire	FEAT_LSE
10	0	1	0		0	101		LDSMIN, LDSMINA, LDSMINAL, LDSMINL — 32-bit acquire	FEAT_LSE
10	0	1	0		0	110		LDUMAX, LDUMAXA, LDUMAXAL, LDUMAXL — 32-bit acquire	FEAT_LSE
10	0	1	0		0	111		LDUMIN, LDUMINA, LDUMINAL, LDUMINL — 32-bit acquire	FEAT_LSE
10	0	1	0		1	000		SWP, SWPA, SWPAL, SWPL — 32-bit SWPA	FEAT_LSE
10	0	1	0		1	100		LDAPR — 32-bit	FEAT_LRCPC
10	0	1	1		0	000		LDADD, LDADDA, LDADDAL, LDADDL — 32-bit acquire-release	FEAT_LSE
10	0	1	1		0	001		LDCLR, LDCLRA, LDCLRAL, LDCLRL — 32-bit acquire-release	FEAT_LSE
10	0	1	1		0	010		LDEOR, LDEORA, LDEORAL, LDEORL — 32-bit acquire-release	FEAT_LSE
10	0	1	1		0	011		LDSET, LDSETA, LDSETAL, LDSETL — 32-bit acquire-release	FEAT_LSE
10	0	1	1		0	100		LDSMAX, LDSMAXA, LDSMAXAL, LDSMAXL — 32-bit acquire-release	FEAT_LSE
10	0	1	1		0	101		LDSMIN, LDSMINA, LDSMINAL, LDSMINL — 32-bit acquire-release	FEAT_LSE
10	0	1	1		0	110		LDUMAX, LDUMAXA, LDUMAXAL, LDUMAXL — 32-bit acquire-release	FEAT_LSE
10	0	1	1		0	111		LDUMIN, LDUMINA, LDUMINAL, LDUMINL — 32-bit acquire-release	FEAT_LSE
10	0	1	1		1	000		SWP, SWPA, SWPAL, SWPL — 32-bit SWPAL	FEAT_LSE

size	VR	A	R	Decode fields			Rt	Instruction Details	Feature
				Rs	o3	opc			
10	1	0	0		0	000		LDFADD, LDFADDA, LDFADDAL, LDFADDL — Single-precision no memory ordering	FEAT_LSFE
10	1	0	0		0	100		LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL — Single-precision no memory ordering	FEAT_LSFE
10	1	0	0		0	101		LDFMIN, LDFMINA, LDFMINAL, LDFMINL — Single-precision no memory ordering	FEAT_LSFE
10	1	0	0		0	110		LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML — Single-precision no memory ordering	FEAT_LSFE
10	1	0	0		0	111		LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML — Single-precision no memory ordering	FEAT_LSFE
10	1	0	0		1	000	11111	STFADD, STFADDL — Single-precision no memory ordering	FEAT_LSFE
10	1	0	0		1	100	11111	STFMAX, STFMAXL — Single-precision no memory ordering	FEAT_LSFE
10	1	0	0		1	101	11111	STFMIN, STFMINL — Single-precision no memory ordering	FEAT_LSFE
10	1	0	0		1	110	11111	STFMAXNM, STFMAXNML — Single-precision no memory ordering	FEAT_LSFE
10	1	0	0		1	111	11111	STFMINNM, STFMINNML — Single-precision no memory ordering	FEAT_LSFE
10	1	0	1		0	000		LDFADD, LDFADDA, LDFADDAL, LDFADDL — Single-precision release	FEAT_LSFE
10	1	0	1		0	100		LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL — Single-precision release	FEAT_LSFE
10	1	0	1		0	101		LDFMIN, LDFMINA, LDFMINAL, LDFMINL — Single-precision release	FEAT_LSFE
10	1	0	1		0	110		LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML — Single-precision release	FEAT_LSFE
10	1	0	1		0	111		LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML — Single-precision release	FEAT_LSFE
10	1	0	1		1	000	11111	STFADD, STFADDL — Single-precision release	FEAT_LSFE
10	1	0	1		1	100	11111	STFMAX, STFMAXL — Single-precision release	FEAT_LSFE
10	1	0	1		1	101	11111	STFMIN, STFMINL — Single-precision release	FEAT_LSFE
10	1	0	1		1	110	11111	STFMAXNM, STFMAXNML — Single-precision release	FEAT_LSFE
10	1	0	1		1	111	11111	STFMINNM, STFMINNML — Single-precision release	FEAT_LSFE
10	1	1	0		0	000		LDFADD, LDFADDA, LDFADDAL, LDFADDL — Single-precision acquire	FEAT_LSFE
10	1	1	0		0	100		LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL — Single-precision acquire	FEAT_LSFE
10	1	1	0		0	101		LDFMIN, LDFMINA, LDFMINAL, LDFMINL — Single-precision acquire	FEAT_LSFE
10	1	1	0		0	110		LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML — Single-precision acquire	FEAT_LSFE
10	1	1	0		0	111		LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML — Single-precision acquire	FEAT_LSFE
10	1	1	1		0	000		LDFADD, LDFADDA, LDFADDAL, LDFADDL — Single-precision acquire-release	FEAT_LSFE

size	VR	A	R	Decode fields			Rt	Instruction Details	Feature
				Rs	o3	opc			
10	1	1	1		0	100		LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL — Single-precision acquire-release	FEAT_LSFE
10	1	1	1		0	101		LDFMIN, LDFMINA, LDFMINAL, LDFMINL — Single-precision acquire-release	FEAT_LSFE
10	1	1	1		0	110		LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML — Single-precision acquire-release	FEAT_LSFE
10	1	1	1		0	111		LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML — Single-precision acquire-release	FEAT_LSFE
11	0	0	0		0	000		LDADD, LDADDA, LDADDAL, LDADDL — 64-bit no memory ordering	FEAT_LSE
11	0	0	0		0	001		LDCLR, LDCLRA, LDCLRAL, LDCLRL — 64-bit no memory ordering	FEAT_LSE
11	0	0	0		0	010		LDEOR, LDEORA, LDEORAL, LDEORL — 64-bit no memory ordering	FEAT_LSE
11	0	0	0		0	011		LDSET, LDSETA, LDSETAL, LDSETL — 64-bit no memory ordering	FEAT_LSE
11	0	0	0		0	100		LDSMAX, LDSMAXA, LDSMAXAL, LDSMAXL — 64-bit no memory ordering	FEAT_LSE
11	0	0	0		0	101		LDSMIN, LDSMINA, LDSMINAL, LDSMINL — 64-bit no memory ordering	FEAT_LSE
11	0	0	0		0	110		LDUMAX, LDUMAXA, LDUMAXAL, LDUMAXL — 64-bit no memory ordering	FEAT_LSE
11	0	0	0		0	111		LDUMIN, LDUMINA, LDUMINAL, LDUMINL — 64-bit no memory ordering	FEAT_LSE
11	0	0	0		1	000		SWP, SWPA, SWPAL, SWPL — 64-bit SWP	FEAT_LSE
11	0	0	0		1	010		ST64BV0	FEAT_LS64_ACCDATA
11	0	0	0		1	011		ST64BV	FEAT_LS64_V
11	0	0	0	11111	1	001		ST64B	FEAT_LS64
11	0	0	0	11111	1	101		LD64B	FEAT_LS64
11	0	0	0	!= 11111	1	x01		UNALLOCATED	-
11	0	0	1		0	000		LDADD, LDADDA, LDADDAL, LDADDL — 64-bit release	FEAT_LSE
11	0	0	1		0	001		LDCLR, LDCLRA, LDCLRAL, LDCLRL — 64-bit release	FEAT_LSE
11	0	0	1		0	010		LDEOR, LDEORA, LDEORAL, LDEORL — 64-bit release	FEAT_LSE
11	0	0	1		0	011		LDSET, LDSETA, LDSETAL, LDSETL — 64-bit release	FEAT_LSE
11	0	0	1		0	100		LDSMAX, LDSMAXA, LDSMAXAL, LDSMAXL — 64-bit release	FEAT_LSE
11	0	0	1		0	101		LDSMIN, LDSMINA, LDSMINAL, LDSMINL — 64-bit release	FEAT_LSE
11	0	0	1		0	110		LDUMAX, LDUMAXA, LDUMAXAL, LDUMAXL — 64-bit release	FEAT_LSE
11	0	0	1		0	111		LDUMIN, LDUMINA, LDUMINAL, LDUMINL — 64-bit release	FEAT_LSE
11	0	0	1		1	x01		UNALLOCATED	-

size	VR	A	R	Decode fields			Rt	Instruction Details	Feature
				Rs	o3	opc			
11	0	0	1		1	000		SWP, SWPA, SWPAL, SWPL — 64-bit SWPL	FEAT_LSE
11	0	0	1		1	010		UNALLOCATED	-
11	0	0	1		1	011		UNALLOCATED	-
11	0	1	0		0	000		LDADD, LDADDA, LDADDAL, LDADDL — 64-bit acquire	FEAT_LSE
11	0	1	0		0	001		LDCLR, LDCLRA, LDCLRAL, LDCLRL — 64-bit acquire	FEAT_LSE
11	0	1	0		0	010		LDEOR, LDEORA, LDEORAL, LDEORL — 64-bit acquire	FEAT_LSE
11	0	1	0		0	011		LDSET, LDSETA, LDSETAL, LDSETL — 64-bit acquire	FEAT_LSE
11	0	1	0		0	100		LDSMAX, LDSMAXA, LDSMAXAL, LDSMAXL — 64-bit acquire	FEAT_LSE
11	0	1	0		0	101		LDSMIN, LDSMINA, LDSMINAL, LDSMINL — 64-bit acquire	FEAT_LSE
11	0	1	0		0	110		LDUMAX, LDUMAXA, LDUMAXAL, LDUMAXL — 64-bit acquire	FEAT_LSE
11	0	1	0		0	111		LDUMIN, LDUMINA, LDUMINAL, LDUMINL — 64-bit acquire	FEAT_LSE
11	0	1	0		1	000		SWP, SWPA, SWPAL, SWPL — 64-bit SWPA	FEAT_LSE
11	0	1	0		1	100		LDAPR — 64-bit	FEAT_LRCPC
11	0	1	1		0	000		LDADD, LDADDA, LDADDAL, LDADDL — 64-bit acquire-release	FEAT_LSE
11	0	1	1		0	001		LDCLR, LDCLRA, LDCLRAL, LDCLRL — 64-bit acquire-release	FEAT_LSE
11	0	1	1		0	010		LDEOR, LDEORA, LDEORAL, LDEORL — 64-bit acquire-release	FEAT_LSE
11	0	1	1		0	011		LDSET, LDSETA, LDSETAL, LDSETL — 64-bit acquire-release	FEAT_LSE
11	0	1	1		0	100		LDSMAX, LDSMAXA, LDSMAXAL, LDSMAXL — 64-bit acquire-release	FEAT_LSE
11	0	1	1		0	101		LDSMIN, LDSMINA, LDSMINAL, LDSMINL — 64-bit acquire-release	FEAT_LSE
11	0	1	1		0	110		LDUMAX, LDUMAXA, LDUMAXAL, LDUMAXL — 64-bit acquire-release	FEAT_LSE
11	0	1	1		0	111		LDUMIN, LDUMINA, LDUMINAL, LDUMINL — 64-bit acquire-release	FEAT_LSE
11	0	1	1		1	000		SWP, SWPA, SWPAL, SWPL — 64-bit SWPAL	FEAT_LSE
11	1	0	0		0	000		LDFADD, LDFADDA, LDFADDAL, LDFADDL — Double-precision no memory ordering	FEAT_LSFE
11	1	0	0		0	100		LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL — Double-precision no memory ordering	FEAT_LSFE
11	1	0	0		0	101		LDFMIN, LDFMINA, LDFMINAL, LDFMINL — Double-precision no memory ordering	FEAT_LSFE
11	1	0	0		0	110		LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML — Double-precision no memory ordering	FEAT_LSFE
11	1	0	0		0	111		LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML — Double-precision no memory ordering	FEAT_LSFE
11	1	0	0		1	000	11111	STFADD, STFADDL — Double-precision no memory ordering	FEAT_LSFE

size	VR	A	R	Decode fields			Rt	Instruction Details	Feature
				Rs	o3	opc			
11	1	0	0		1	100	11111	STFMAX, STFMAXL — Double-precision no memory ordering	FEAT_LSFE
11	1	0	0		1	101	11111	STFMIN, STFMINL — Double-precision no memory ordering	FEAT_LSFE
11	1	0	0		1	110	11111	STFMAXNM, STFMAXNML — Double-precision no memory ordering	FEAT_LSFE
11	1	0	0		1	111	11111	STFMINNM, STFMINNML — Double-precision no memory ordering	FEAT_LSFE
11	1	0	1		0	000		LDFADD, LDFADDA, LDFADDAL, LDFADDL — Double-precision release	FEAT_LSFE
11	1	0	1		0	100		LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL — Double-precision release	FEAT_LSFE
11	1	0	1		0	101		LDFMIN, LDFMINA, LDFMINAL, LDFMINL — Double-precision release	FEAT_LSFE
11	1	0	1		0	110		LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML — Double-precision release	FEAT_LSFE
11	1	0	1		0	111		LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML — Double-precision release	FEAT_LSFE
11	1	0	1		1	000	11111	STFADD, STFADDL — Double-precision release	FEAT_LSFE
11	1	0	1		1	100	11111	STFMAX, STFMAXL — Double-precision release	FEAT_LSFE
11	1	0	1		1	101	11111	STFMIN, STFMINL — Double-precision release	FEAT_LSFE
11	1	0	1		1	110	11111	STFMAXNM, STFMAXNML — Double-precision release	FEAT_LSFE
11	1	0	1		1	111	11111	STFMINNM, STFMINNML — Double-precision release	FEAT_LSFE
11	1	1	0		0	000		LDFADD, LDFADDA, LDFADDAL, LDFADDL — Double-precision acquire	FEAT_LSFE
11	1	1	0		0	100		LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL — Double-precision acquire	FEAT_LSFE
11	1	1	0		0	101		LDFMIN, LDFMINA, LDFMINAL, LDFMINL — Double-precision acquire	FEAT_LSFE
11	1	1	0		0	110		LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML — Double-precision acquire	FEAT_LSFE
11	1	1	0		0	111		LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML — Double-precision acquire	FEAT_LSFE
11	1	1	1		0	000		LDFADD, LDFADDA, LDFADDAL, LDFADDL — Double-precision acquire-release	FEAT_LSFE
11	1	1	1		0	100		LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL — Double-precision acquire-release	FEAT_LSFE
11	1	1	1		0	101		LDFMIN, LDFMINA, LDFMINAL, LDFMINL — Double-precision acquire-release	FEAT_LSFE
11	1	1	1		0	110		LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML — Double-precision acquire-release	FEAT_LSFE
11	1	1	1		0	111		LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML — Double-precision acquire-release	FEAT_LSFE

Load/store register (register offset)

The encodings in this section are decoded from [Loads and Stores](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
size	1	1	1	VR	0	0	opc	1	Rm			option			S	1	0	Rn			Rt										

Decode fields				Rt		Instruction Details		Feature
size	VR	opc	option					
00	0	00	011			STRB (register) — Shifted register		-
00	0	00	!= 011			STRB (register) — Extended register		-
00	0	01	011			LDRB (register) — Shifted register		-
00	0	01	!= 011			LDRB (register) — Extended register		-
00	0	10	011			LDRSB (register) — 64-bit with shifted register offset		-
00	0	10	!= 011			LDRSB (register) — 64-bit with extended register offset		-
00	0	11	011			LDRSB (register) — 32-bit with shifted register offset		-
00	0	11	!= 011			LDRSB (register) — 32-bit with extended register offset		-
00	1	00	011			STR (register, SIMD&FP) — 8-bit with shifted register offset		FEAT_FP
00	1	00	!= 011			STR (register, SIMD&FP) — 8-bit with extended register offset		FEAT_FP
00	1	01	011			LDR (register, SIMD&FP) — 8-bit with shifted register offset		FEAT_FP
00	1	01	!= 011			LDR (register, SIMD&FP) — 8-bit with extended register offset		FEAT_FP
00	1	10				STR (register, SIMD&FP) — 128-bit		FEAT_FP
00	1	11				LDR (register, SIMD&FP) — 128-bit		FEAT_FP
01	0	00				STRH (register)		-
01	0	01				LDRH (register)		-
01	0	10				LDRSH (register) — 64-bit		-
01	0	11				LDRSH (register) — 32-bit		-
01	1	00				STR (register, SIMD&FP) — 16-bit		FEAT_FP
01	1	01				LDR (register, SIMD&FP) — 16-bit		FEAT_FP
1x	0	11				UNALLOCATED		-
10	0	00				STR (register) — 32-bit		-
10	0	01				LDR (register) — 32-bit		-
10	0	10				LDRSW (register)		-
10	1	00				STR (register, SIMD&FP) — 32-bit		FEAT_FP
10	1	01				LDR (register, SIMD&FP) — 32-bit		FEAT_FP
11	0	00				STR (register) — 64-bit		-
11	0	01				LDR (register) — 64-bit		-
11	0	10	x0x			UNALLOCATED		-
11	0	10	x1x	11xxx		RPRFM		FEAT_RPRFM
11	0	10	x1x	!= 11xxx		PRFM (register)		-
11	1	00				STR (register, SIMD&FP) — 64-bit		FEAT_FP
11	1	01				LDR (register, SIMD&FP) — 64-bit		FEAT_FP
!= 00	1	1x				UNALLOCATED		-

Load/store register (pac)

The encodings in this section are decoded from [Loads and Stores](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
size	1	1	1	VR	0	0	M	S	1	imm9										W	1	Rn					Rt				

Decode fields				Instruction Details		Feature
size	VR	M	W			
11	0	0	0	LDRAA, LDRAB — Key A, offset		FEAT_PAuth
11	0	0	1	LDRAA, LDRAB — Key A, pre-indexed		FEAT_PAuth
11	0	1	0	LDRAA, LDRAB — Key B, offset		FEAT_PAuth
11	0	1	1	LDRAA, LDRAB — Key B, pre-indexed		FEAT_PAuth
11	1			UNALLOCATED		-
!= 11				UNALLOCATED		-

Load/store register (unsigned immediate)

The encodings in this section are decoded from [Loads and Stores](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0					
size		1		1		1		VR		0		1		opc		imm12												Rn					Rt			

Decode fields			Instruction Details		Feature
size	VR	opc			
00	0	00	STRB (immediate)		-
00	0	01	LDRB (immediate)		-
00	0	10	LDRSB (immediate) — 64-bit		-
00	0	11	LDRSB (immediate) — 32-bit		-
00	1	00	STR (immediate, SIMD&FP) — 8-bit		FEAT_FP
00	1	01	LDR (immediate, SIMD&FP) — 8-bit		FEAT_FP
00	1	10	STR (immediate, SIMD&FP) — 128-bit		FEAT_FP
00	1	11	LDR (immediate, SIMD&FP) — 128-bit		FEAT_FP
01	0	00	STRH (immediate)		-
01	0	01	LDRH (immediate)		-
01	0	10	LDRSH (immediate) — 64-bit		-
01	0	11	LDRSH (immediate) — 32-bit		-
01	1	00	STR (immediate, SIMD&FP) — 16-bit		FEAT_FP
01	1	01	LDR (immediate, SIMD&FP) — 16-bit		FEAT_FP
1x	0	11	UNALLOCATED		-
10	0	00	STR (immediate) — 32-bit		-
10	0	01	LDR (immediate) — 32-bit		-
10	0	10	LDRSW (immediate)		-
10	1	00	STR (immediate, SIMD&FP) — 32-bit		FEAT_FP
10	1	01	LDR (immediate, SIMD&FP) — 32-bit		FEAT_FP
11	0	00	STR (immediate) — 64-bit		-
11	0	01	LDR (immediate) — 64-bit		-
11	0	10	PRFM (immediate)		-
11	1	00	STR (immediate, SIMD&FP) — 64-bit		FEAT_FP
11	1	01	LDR (immediate, SIMD&FP) — 64-bit		FEAT_FP
!= 00	1	1x	UNALLOCATED		-

Shared Pseudocode Functions

This page displays common pseudocode functions shared by many pages

PseudocodePseudocodes

Library pseudocode for aarch32/at/AArch32_EncodePARLD

```
// AArch32_EncodePARLD()
// =====
// Returns 64-bit format PAR on address translation instruction.

func AArch32_EncodePARLD(addrdesc : AddressDescriptor, ss : SecurityState)
begin
    PAR()[63:0] = Zeros{64};

    if !IsFault(addrdesc) then
        var ns : bit;
        if ss == SS\_NonSecure then
            ns = ARBITRARY : bit;
        elsif addrdesc.paddress.paspace == PAS\_Secure then
            ns = '0';
        else
            ns = '1';
        end;
        PAR().F      = '0';
        PAR().SH     = ReportedPARShareability(PAREncodeShareability(addrdesc.memattrs));
        PAR().NS     = ns;
        PAR()[10]    = ImpDefBit("Non-Faulting PAR");           // IMPDEF
        PAR().LPAE   = '1';
        PAR().PA     = addrdesc.paddress.address[39:12];
        PAR().ATTR   = ReportedPARAttrs(EncodePARAttrs(addrdesc.memattrs));
    else
        PAR().F      = '1';
        PAR().FST    = AArch32\_PARFaultStatusLD(addrdesc.fault);
        PAR().S2WLK  = if addrdesc.fault.s2fslwalk then '1' else '0';
        PAR().FSTAGE = if addrdesc.fault.secondstage then '1' else '0';
        PAR().LPAE   = '1';
        PAR()[63:48] = ImpDefBits{16}("Faulting PAR");         // IMPDEF
    end;
    return;
end;
```


Library pseudocode for aarch32/at/AArch32_EncodePARSD

```
// AArch32_EncodePARSD()
// =====
// Returns 32-bit format PAR on address translation instruction.

func AArch32_EncodePARSD(addrdesc_in : AddressDescriptor, supersection : bit, ss : SecurityState)
begin
    PAR\(\)[31:0] = Zeros{32};
    var addrdesc : AddressDescriptor = addrdesc_in;
    if !IsFault(addrdesc) then
        if (addrdesc.memattrs.memtype == MemType\_Device ||
            (addrdesc.memattrs.inner.attrs == MemAttr\_NC &&
             addrdesc.memattrs.outer.attrs == MemAttr\_NC)) then
            addrdesc.memattrs.shareability = Shareability\_OSH;
        end;
        var ns : bit;
        if ss == SS\_NonSecure then
            ns = ARBITRARY : bit;
        elsif addrdesc.paddress.paspace == PAS\_Secure then
            ns = '0';
        else
            ns = '1';
        end;
        let sh : bits(2) = (if addrdesc.memattrs.shareability != Shareability\_NSH then '01'
                           else '00');
        PAR\(\).F      = '0';
        PAR\(\).SS     = supersection;
        PAR\(\).Outer  = AArch32\_ReportedOuterAttrs(AArch32\_PAROuterAttrs(addrdesc.memattrs));
        PAR\(\).Inner  = AArch32\_ReportedInnerAttrs(AArch32\_PARInnerAttrs(addrdesc.memattrs));
        PAR\(\).SH     = ReportedPARShareability(sh);
        PAR\(\)\[8\]      = ImpDefBit("Non-Faulting PAR"); // IMPDEF
        PAR\(\).NS     = ns;
        PAR\(\).NOS    = if addrdesc.memattrs.shareability == Shareability\_OSH then '0' else '1';
        PAR\(\).LPAE   = '0';
        PAR\(\).PA     = addrdesc.paddress.address[39:12];
    else
        PAR\(\).F      = '1';
        PAR\(\).FST    = AArch32\_PARFaultStatusSD(addrdesc.fault);
        PAR\(\).LPAE   = '0';
        PAR\(\)\[31:16\] = ImpDefBits{16}("Faulting PAR"); // IMPDEF
    end;
    return;
end;
```

Library pseudocode for aarch32/debug/VCRMatch/AArch32_VCRMatch

```
// AArch32_VCRMatch()
// =====

func AArch32_VCRMatch(vaddress : bits(32)) => boolean
begin
    var match : boolean;
    if UsingAArch32() && ELUsingAArch32(EL1) && PSTATE.EL != PSTATE.EL != EL2 then
        // Each bit position in this string corresponds to a bit in DBGVCR and an exception vector.
        var match_word : bits(32) = Zeros{};

        let ss : SecurityState = CurrentSecurityState();
        if vaddress[31:5] == ExcVectorBase()[31:5] then
            if HaveEL(EL3) && ss == SS_NonSecure then
                match_word[UInt(vaddress[4:2]) + 24] = '1'; // Non-secure vectors
            else
                match_word[UInt(vaddress[4:2]) + 0] = '1'; // Secure vectors (or no EL3)
            end;
        end;
        if (HaveEL(EL3) && ELUsingAArch32(EL3) && vaddress[31:5] == MVBAR()[31:5] &&
            ss == SS_Secure) then
            match_word[UInt(vaddress[4:2]) + 8] = '1'; // Monitor vectors
        end;
        // Mask out bits not corresponding to vectors.
        var mask : bits(32);
        if !HaveEL(EL3) then
            mask = '00000000'::'00000000'::'00000000'::'11011110'; // DBGVCR[31:8] are RES0
        elsif !ELUsingAArch32(EL3) then
            mask = '11011110'::'00000000'::'00000000'::'11011110'; // DBGVCR[15:8] are RES0
        else
            mask = '11011110'::'00000000'::'11011100'::'11011110';
        end;

        match_word = match_word AND DBGVCR() AND mask;
        match = !IsZero(match_word);

        // Check for UNPREDICTABLE case - match on Prefetch Abort and Data Abort vectors
        if !IsZero(match_word[28:27,12:11,4:3]) && DebugTarget() == PSTATE.EL() == PSTATE.EL then
            match = ConstrainUnpredictableBool(Unpredictable_VCMATCHDAPA);
        end;

        if !IsZero(vaddress[1:0]) && match then
            match = ConstrainUnpredictableBool(Unpredictable_VCMATCHHALF);
        end;
    else
        match = FALSE;
    end;

    return match;
end;
```

Library pseudocode for aarch32/debug/watchpoint/AArch32_WatchpointByteMatch

```
// AArch32_WatchpointByteMatch()
// =====

func AArch32_WatchpointByteMatch(n : integer, vaddress : bits(32)) => boolean
begin
    let dbgtop : integer{} = 31;
    let cmpbottom : integer{} = if DBGWVR(n)[2] == '1' then 2 else 3; // Word or doubleword
    var bottom = cmpbottom;
    let select : integer{} = UInt(vaddress[cmpbottom-1:0]);
    var byte_select_match : boolean = (DBGWCR(n).BAS[select] != '0');
    var mask : integer{} = UInt(DBGWCR(n).MASK);

    // If DBGWCR(n).MASK is a nonzero value and DBGWCR(n).BAS is not set to '11111111', or
    // DBGWCR(n).BAS specifies a non-contiguous set of bytes behavior is CONSTRAINED
    // UNPREDICTABLE.
    if mask > 0 && !IsOnes(DBGWCR(n).BAS) then
        byte_select_match = ConstrainUnpredictableBool(Unpredictable WPMASKANDBAS);
    else
        let LSB : bits(8) = (DBGWCR(n).BAS AND NOT(DBGWCR(n).BAS - 1));
        let MSB : bits(8) = (DBGWCR(n).BAS + LSB);
        if !IsZero(MSB AND (MSB - 1)) then // Not contiguous
            byte_select_match = ConstrainUnpredictableBool(Unpredictable WPBASCONTIGUOUS);
            bottom = 3; // For the whole doubleword
        end;
    end;

    // If the address mask is set to a reserved value, the behavior is CONSTRAINED UNPREDICTABLE.
    if mask > 0 && mask <= 2 then
        var c : Constraint;
        var unpred_mask : integer;
        (c, unpred_mask) = (ConstrainUnpredictableInteger(3, 31, Unpredictable RESWPMASK);
        assert);
        as c IN ({Constraint, integer(0..31)});
        assert c IN ({Constraint DISABLED, Constraint NONE, Constraint UNKNOWN});
        case c of
            when Constraint DISABLED => return FALSE; // Disabled
            when Constraint NONE => mask = 0; // No masking
            // Otherwise the value returned by ConstrainUnpredictableInteger is a not-reserved value
            otherwise => mask = unpred_mask as integer(3..31);
        end;
    end;

    let cmpmsb : integer{} = dbgtop;
    let cmpls : integer{} = if mask > bottom then mask else bottom;
    let bottombit : integer{} = bottom;
    var WVR_match : boolean = (vaddress[cmpmsb:cmpls] == DBGWVR(n)[cmpmsb:cmpls]);
    if mask > bottom then
        // If masked bits of DBGWVR(n) are not zero, the behavior is CONSTRAINED UNPREDICTABLE.
        if WVR_match && !IsZero(DBGWVR(n)[cmpls-1:bottombit]) then
            WVR_match = ConstrainUnpredictableBool(Unpredictable WPMASKEDBITS);
        end;
    end;

    return (WVR_match && byte_select_match);
end;
```

Library pseudocode for aarch32/exceptions/aborts/AArch32_ReportDataAbort

```
// AArch32_ReportDataAbort()
// =====
// Report syndrome information for aborts taken to modes other than Hyp mode.

func AArch32_ReportDataAbort(route_to_monitor : boolean, fault : FaultRecord)
begin
    var long_format : boolean;
    if route_to_monitor && CurrentSecurityState() != SS\_Secure then
        long_format = ((TTBCR_S().EAE == '1') ||
            (IsExternalSyncAbort(fault) && ((PSTATE.EL == PSTATE.EL == EL2 || TTBCR().EAE == '1') ||
                (fault.secondstage && ImpDefBool(
                    "Report abort using Long-descriptor format"))));
    else
        long_format = TTBCR().EAE == '1';
    end;
    var syndrome : bits(32) = AArch32\_CommonFaultStatus(fault, long_format);

    // bits of syndrome that are not common to I and D side
    if fault.accessdesc.acctype IN {AccessType\_DC, AccessType\_IC, AccessType\_AT} then
        syndrome[13] = '1'; // CM
        syndrome[11] = '1'; // WnR
    else
        syndrome[11] = if fault.write then '1' else '0'; // WnR
    end;

    if !long_format then
        syndrome[7:4] = fault.domain; // Domain
    end;

    if fault.accessdesc.acctype == AccessType\_IC then
        var i_syndrome : bits(32);
        if (!long_format &&
            ImpDefBool("Report I-cache maintenance fault in IFSR")) then
            i_syndrome = syndrome;
            syndrome[10,3:0] = EncodeSDFSC(Fault\_ICacheMaint, 1);
        else
            i_syndrome = ARBITRARY : bits(32);
        end;
        if route_to_monitor then
            IFSR_S() = i_syndrome;
        else
            IFSR() = i_syndrome;
        end;
    end;

    if route_to_monitor then
        DFSR_S() = syndrome;
        DFAR_S() = fault.vaddress[31:0];
    else
        DFSR() = syndrome;
        DFAR() = fault.vaddress[31:0];
    end;

    return;
end;
```

Library pseudocode for aarch32/exceptions/aborts/AArch32_ReportPrefetchAbort

```
// AArch32_ReportPrefetchAbort()
// =====
// Report syndrome information for aborts taken to modes other than Hyp mode.

func AArch32_ReportPrefetchAbort(route_to_monitor : boolean, fault : FaultRecord)
begin
    // The encoding used in the IFSR can be Long-descriptor format or Short-descriptor format.
    // Normally, the current translation table format determines the format. For an abort from
    // Non-secure state to Monitor mode, the IFSR uses the Long-descriptor format if any of the
    // following applies:
    // * The Secure TTBCR.EAE is set to 1.
    // * It is taken from Hyp mode.
    // * It is taken from EL1 or EL0, and the Non-secure TTBCR.EAE is set to 1.
    var long_format : boolean = FALSE;
    if route_to_monitor && CurrentSecurityState() != SS Secure then
        long_format = TTBCR_S().EAE == '1' || PSTATE.EL == PSTATE.EL == EL2 || TTBCR().EAE == '1';
    else
        long_format = TTBCR().EAE == '1';
    end;

    let fsr : bits(32) = AArch32\_CommonFaultStatus(fault, long_format);
    if route_to_monitor then
        IFSR_S() = fsr;
        IFAR_S() = fault.vaddress[31:0];
    else
        IFSR() = fsr;
        IFAR() = fault.vaddress[31:0];
    end;
    return;
end;
```

Library pseudocode for aarch32/exceptions/async/AArch32_TakePhysicalFIQException

```
// AArch32_TakePhysicalFIQException()
// =====

func AArch32_TakePhysicalFIQException()
begin
    // Check if routed to AArch64 state
    var route_to_aarch64 : boolean = PSTATE.EL == PSTATE.EL == EL0 && !ELUsingAArch32\(EL1\);
    if !route_to_aarch64 && EL2Enabled() && !ELUsingAArch32\(EL2\) then
        route_to_aarch64 = HCR_EL2().TGE == '1' || (HCR_EL2().FMO == '1' && !IsInHost());
    end;

    if !route_to_aarch64 then
        route_to_aarch64 = && EffectiveSCR\_EL3\_FIQHaveEL() == '1';
    end;

    if route_to_aarch64 then AArch64_TakePhysicalFIQException(); end;
    let route_to_monitor : boolean = (EL3) && !ELUsingAArch32\(EL3\) then
        route_to_aarch64 = SCR_EL3().FIQ == '1';
    end;

    if route_to_aarch64 then AArch64_TakePhysicalFIQException(); end;
    let route_to_monitor : boolean = HaveEL\(EL3\) && SCR().FIQ == '1';
    let route_to_hyp : boolean = (PSTATE.EL IN {PSTATE.EL IN {EL0, EL1} && EL2Enabled() &&
        (HCR().TGE == '1' || HCR().FMO == '1')));
    let preferred_exception_return : bits(32) = ThisInstrAddr();
    let vect_offset : integer = 0x1C;
    let lr_offset : integer = 4;
    if route_to_monitor then
        AArch32_EnterMonitorMode(preferred_exception_return, lr_offset, vect_offset);
    elseif PSTATE.EL() &&
        (HCR().TGE == '1' || HCR().FMO == '1')) then
        let preferred_exception_return : bits(32) = ThisInstrAddr();
        let vect_offset : integer = 0x1C;
        let lr_offset : integer = 4;
        if route_to_monitor then
            AArch32_EnterMonitorMode(preferred_exception_return, lr_offset, vect_offset);
        elseif PSTATE.EL == EL2 || route_to_hyp then
            let except : ExceptionRecord = ExceptionSyndrome(Exception FIQ);
            AArch32_EnterHypMode(except, preferred_exception_return, vect_offset);
        else
            AArch32_EnterMode(M32\_FIQ, preferred_exception_return, lr_offset, vect_offset);
        end;
    end;
end;
```

Library pseudocode for aarch32/exceptions/async/AArch32_TakePhysicalIRQException

```
// AArch32_TakePhysicalIRQException()
// =====
// Take an enabled physical IRQ exception.

func AArch32_TakePhysicalIRQException()
begin
    // Check if routed to AArch64 state
    var route_to_aarch64 : boolean = PSTATE.EL == PSTATE.EL == EL0 && !ELUsingAArch32(EL1);
    if !route_to_aarch64 && EL2Enabled() && !ELUsingAArch32(EL2) then
        route_to_aarch64 = HCR_EL2().TGE == '1' || (HCR_EL2().IMO == '1' && !IsInHost());
    end;
    if !route_to_aarch64 then
        route_to_aarch64 = && EffectiveSCR_EL3_IRQHaveEL() EffectiveSCR_EL3_IRQHaveEL() == '1';
    end;

    if route_to_aarch64 then AArch64_TakePhysicalIRQException(); end;

    let route_to_monitor : boolean = (EL3) && !ELUsingAArch32(EL3) then
        route_to_monitor = SCR_EL3().IRQ == '1';
    end;

    if route_to_aarch64 then AArch64_TakePhysicalIRQException(); end;

    let route_to_monitor : boolean = HaveEL(EL3) && SCR().IRQ == '1';
    let route_to_hyp : boolean = (PSTATE.EL IN {PSTATE.EL IN {EL0, EL1} && EL2Enabled() &&
        (HCR().TGE == '1' || HCR().IMO == '1')});
    let preferred_exception_return : bits(32) = ThisInstrAddr{}();
    let vect_offset : integer = 0x18;
    let lr_offset : integer = 4;
    if route_to_monitor then
        AArch32_EnterMonitorMode(preferred_exception_return, lr_offset, vect_offset);
    elseif PSTATE.EL() &&
        (HCR().TGE == '1' || HCR().IMO == '1') then
        let preferred_exception_return : bits(32) = ThisInstrAddr{}();
        let vect_offset : integer = 0x18;
        let lr_offset : integer = 4;
        if route_to_monitor then
            AArch32_EnterMonitorMode(preferred_exception_return, lr_offset, vect_offset);
        elseif PSTATE.EL == EL2 || route_to_hyp then
            let except : ExceptionRecord = ExceptionSyndrome(Exception_IRQ);
            AArch32_EnterHypMode(except, preferred_exception_return, vect_offset);
        else
            AArch32_EnterMode(M32_IRQ, preferred_exception_return, lr_offset, vect_offset);
        end;
    end;
end;
```

Library pseudocode for aarch32/exceptions/async/AArch32_TakeVirtualFIQException

```
// AArch32_TakeVirtualFIQException()
// =====

func AArch32_TakeVirtualFIQException()
begin
    assert PSTATE.EL IN { PSTATE.EL IN {EL0, EL1} && EL2Enabled();
    if ELUsingAArch32(EL2) then // Virtual IRQ enabled if TGE==0 and FMO==1
        assert HCR().TGE == '0' && HCR().FMO == '1';
    else
        assert HCR_EL2().TGE == '0' && HCR_EL2().FMO == '1';
    end;
    // Check if routed to AArch64 state
    if PSTATE.EL PSTATE.EL then // Virtual IRQ enabled if TGE==0 and FMO==1
        assert HCR().TGE == '0' && HCR().FMO == '1';
    else
        assert HCR_EL2().TGE == '0' && HCR_EL2().FMO == '1';
    end;
    // Check if routed to AArch64 state
    if PSTATE.EL == EL0 && !ELUsingAArch32(EL1) then AArch64_TakeVirtualFIQException(); end;

    let preferred_exception_return : bits(32) = ThisInstrAddr{}() ThisInstrAddr{}();
    let vect_offset : integer = 0x1C;
    let lr_offset : integer = 4;

    AArch32_EnterMode(M32_FIQ, preferred_exception_return, lr_offset, vect_offset);
end;
```

Library pseudocode for aarch32/exceptions/exceptions/AArch32_ExceptionClass

```
// AArch32_ExceptionClass()
// =====
// Returns the Exception Class and Instruction Length fields to be reported in HSR

func AArch32_ExceptionClass(exceptype : Exception) => (integer,bit)
begin
    var il_is_valid : boolean = TRUE;
    var ec : integer;
    case exceptype of
        when Exception Uncategorized =>          ec = 0x00; il_is_valid = FALSE;
        when Exception WFXTrap =>                ec = 0x01;
        when Exception CP15RTTTrap =>            ec = 0x03;
        when Exception CP15RRTTrap =>            ec = 0x04;
        when Exception CP14RTTTrap =>            ec = 0x05;
        when Exception CP14DTTTrap =>            ec = 0x06;
        when Exception AdvSIMDFPAccessTrap =>    ec = 0x07;
        when Exception FPIDTrap =>                ec = 0x08;
        when Exception PACTrap =>                ec = 0x09;
        when Exception GPC =>                    ec = 0x1E;
        when Exception CP14RRTTrap =>            ec = 0x0C;
        when Exception BranchTarget =>           ec = 0x0D;
        when Exception IllegalState =>           ec = 0x0E; il_is_valid = FALSE;
        when Exception SupervisorCall =>         ec = 0x11;
        when Exception HypervisorCall =>         ec = 0x12;
        when Exception MonitorCall =>           ec = 0x13;
        when Exception InstructionAbort =>
            ec = if PSTATE.EL == EL2 then 0x21 else 0x20;
            il_is_valid = FALSE;
        when Exception PCAlignment =>           ec = 0x22; il_is_valid = FALSE;
        when Exception DataAbort =>             ec = if PSTATE.EL == EL2 then 0x25 else 0x24;
        when EL2Exception NV2DataAbort then =>  ec = 0x25 else 0x24;
        when Exception FPTrappedException =>    ec = 0x28;
        when Exception Profiling =>             ec = 0x3D;
        otherwise =>                            unreachable;
    end;

    var il : bit;
    if il_is_valid then
        il = if ThisInstrLength() == 32 then '1' else '0';
    else
        il = '1';
    end;

    return (ec,il);
end;
```



```

// AArch32_TakeReset()
// =====
// Reset into AArch32 state

func AArch32_TakeReset(cold_reset : boolean)
begin
    assert !HaveAArch64();

    // Enter the highest implemented Exception level in AArch32 state
    if HaveEL(EL3) then
        AArch32_WriteMode(M32_Svc);
        SCR().NS = '0'; // Secure state
    elseif HaveEL(EL2) then
        AArch32_WriteMode(M32_Hyp);
    else
        AArch32_WriteMode(M32_Svc);
    end;

    // Reset System registers in the coproc=0b111x encoding space
    // and other system components
    AArch32_ResetControlRegisters(cold_reset);
    FPExc().EN = '0';

    // Reset all other PSTATE fields, including instruction set and endianness according to the
    // SCTLr values produced by the above call to ResetControlRegisters()
    PSTATE.[A,I,F] = '111'; // All asynchronous exceptions masked
    PSTATE.IT = '00000000'; // IT block state reset
    if
PSTATE.IT = '00000000'; // IT block state reset
    if HaveEL(EL2) && !HaveEL(EL3) then
        PSTATE.T = HSCTLR().TE; // Instruction set: TE=0:A32, TE=1:T32. PSTATE.J is RES0.
        PSTATE.E = HSCTLR().EE; // Endianness: EE=0: little-endian, EE=1: big-endian.
    else
        PSTATE.T = SCTLr().TE; // Instruction set: TE=0:A32, TE=1:T32. PSTATE.J is RES0.
        PSTATE.E = SCTLr().EE; // Endianness: EE=0: little-endian, EE=1: big-endian.
    end;
    PSTATE.IL = '0'; // Clear Illegal Execution state bit

    // All registers, bits and fields not reset by the above pseudocode or by the BranchTo() call
    // below are UNKNOWN bitstrings after reset. In particular, the return information registers
    // R14 or ELR_hyp and SPSR have UNKNOWN values, so that it
    // is impossible to return from a reset in an architecturally defined way.
    AArch32_ResetGeneralRegisters();
    if IsFeatureImplemented(FEAT_SME) || IsFeatureImplemented(FEAT_SVE) then
        ResetSVERegisters();
    else
        AArch32_ResetSIMDFFPRegisters();
    end;
    AArch32_ResetSpecialRegisters();
    ResetExternalDebugRegisters(cold_reset);

    var rv : bits(32); // IMPLEMENTATION DEFINED reset vector

    if
PSTATE.T = HSCTLR().TE; // Instruction set: TE=0:A32, TE=1:T32. PSTATE.J is RES0.
    PSTATE.E = HSCTLR().EE; // Endianness: EE=0: little-endian, EE=1: big-endian.
    else
        PSTATE.T = SCTLr().TE; // Instruction set: TE=0:A32, TE=1:T32. PSTATE.J is RES0.
        PSTATE.E = SCTLr().EE; // Endianness: EE=0: little-endian, EE=1: big-endian.
    end;
    PSTATE.IL = '0'; // Clear Illegal Execution state bit

    // All registers, bits and fields not reset by the above pseudocode or by the BranchTo() call
    // below are UNKNOWN bitstrings after reset. In particular, the return information registers
    // R14 or ELR_hyp and SPSR have UNKNOWN values, so that it
    // is impossible to return from a reset in an architecturally defined way.
    AArch32_ResetGeneralRegisters();
    if IsFeatureImplemented(FEAT_SME) || IsFeatureImplemented(FEAT_SVE) then
        ResetSVERegisters();
    else
        AArch32_ResetSIMDFFPRegisters();
    end;
    AArch32_ResetSpecialRegisters();
    ResetExternalDebugRegisters(cold_reset);

    var rv : bits(32); // IMPLEMENTATION DEFINED reset vector
    if HaveEL(EL3) then
        if MVBAR()[0] == '1' then // Reset vector in MVBAR
            rv = MVBAR()[31:1]::'0';
        else
            rv = ImpDefBits{32}("reset vector address");
        end;
    else
        rv = RVBAR()[31:1]::'0';
    end;

    // The reset vector must be correctly aligned
    assert rv[0] == '0' && (PSTATE.T) then
        if MVBAR()[0] == '1' then // Reset vector in MVBAR

```

```

        rv = MVBAR()[31:1]::'0';
    else
        rv = ImpDefBits(32)("reset-vector-address");
    end;
else
    rv = RVBAR()[31:1]::'0';
end;

// The reset vector must be correctly aligned
assert rv[0] == '0' && (PSTATE.T == '1' || rv[1] == '0');

let branch_conditional : boolean = FALSE;
EDPRSR().R = '0'; // Leaving Reset State.
BranchTo{32}(rv, BranchType\_RESET, branch_conditional);
end;

```

Library pseudocode for aarch32/exceptions/syscalls/AArch32_TakeSMCException

```

// AArch32_TakeSMCException()
// =====

func AArch32_TakeSMCException()
begin
    assert HaveEL\(EL3\) && ELUsingAArch32\(EL3\);
    AArch32_ITAdvance();
    HSAdvance();
    SSAdvance();
    let preferred_exception_return : bits(32) = NextInstrAddr{}();
    let vect_offset : integer{+} = 0x08;
    let lr_offset : integer{+} = 0;
    AArch32_EnterMonitorMode(preferred_exception_return, lr_offset, vect_offset);
end;

```

Library pseudocode for aarch32/exceptions/takeexception/AArch32_EnterHypMode

```
// AArch32_EnterHypMode()
// =====
// Take an exception to Hyp mode.

noreturn func AArch32_EnterHypMode(except : ExceptionRecord, preferred_exception_return : bits(32),
                                   vect_offset : integer)
begin
    SynchronizeContext();
    assert HaveEL\(EL2\) && CurrentSecurityState\(\) == SS\_NonSecure && ELUsingAArch32\(EL2\);

    if Halted\(\) then
        AArch32_EnterHypModeInDebugState(except);
    end;
    let spsr : bits(32) = GetPSRFromPSTATE{}(AArch32\_NonDebugState);
    if ! except.excepttype IN {Exception\_IRQ, Exception\_FIQ} then
        AArch32_ReportHypEntry(except);
    end;
    AArch32_WriteMode(M32\_Hyp);
    SPSR\_curr() = spsr;
    ELR\_hyp() = preferred_exception_return;
    PSTATE.T = HSCTLR\(\).TE; // PSTATE.J is RES0
    PSTATE.SS = '0';
    if !
    PSTATE.T = HSCTLR\(\).TE; // PSTATE.J is RES0
    PSTATE.SS = '0';
    if !HaveEL\(EL3\) then
        PSTATE.A = '1';
        PSTATE.I = '1';
        PSTATE.F = '1';
    else
        if
        PSTATE.A = '1';
        PSTATE.I = '1';
        PSTATE.F = '1';
    else
        if ELUsingAArch32\(EL3\) then
            if SCR\(\).EA == '0' then PSTATE.A = '1'; end;
            if SCR\(\).IRQ == '0' then PSTATE.I = '1'; end;
            if SCR\(\).FIQ == '0' then PSTATE.F = '1'; end;
        else
            if SCR\_EL3\(\).EA == '0' then PSTATE.A = '1'; end;
            if EffectiveSCR\_EL3\_IRQ\(\) == '0' then PSTATE.I = '1'; end;
            if EffectiveSCR\_EL3\_FIQ\(\) == '0' then PSTATE.F = '1'; end;
        end;
    end;
    PSTATE.E = HSCTLR\(\).EE;
    PSTATE.IL = '0';
    PSTATE.IT = '00000000';
    if IsFeatureImplemented\(FEAT\_SSBS\) then PSTATE.SSBS then
        if SCR\(\).EA == '0' then PSTATE.A = '1'; end;
        if SCR\(\).IRQ == '0' then PSTATE.I = '1'; end;
        if SCR\(\).FIQ == '0' then PSTATE.F = '1'; end;
    else
        if SCR\_EL3\(\).EA == '0' then PSTATE.A = '1'; end;
        if SCR\_EL3\(\).IRQ == '0' then PSTATE.I = '1'; end;
        if SCR\_EL3\(\).FIQ == '0' then PSTATE.F = '1'; end;
    end;
    end;
    PSTATE.E = HSCTLR\(\).EE;
    PSTATE.IL = '0';
    PSTATE.IT = '00000000';
    if IsFeatureImplemented\(FEAT\_SSBS\) then PSTATE.SSBS = HSCTLR\(\).DSSBS; end;
    let branch_conditional : boolean = FALSE;
    BranchTo{32}(HVBAR() [31:5]::vect_offset[4:0], BranchType\_EXCEPTION, branch_conditional);

    CheckExceptionCatch(TRUE); // Check for debug event on exception entry

    EndOfInstruction();
end;
```



```

// Mem_with_type
// =====

accessor Mem_with_type{size : integer{8, 16, 32, 64}}(address : bits(32),
                                                    accdesc_in : AccessDescriptor
                                                    ) <=> value_in : bits(size)

begin
  // Perform a read of 'size' bits. The access byte order is reversed for a big-endian access.
  // Instruction fetches would call AArch32_MemSingle directly.
  getter
    let bytes : integer{} = size DIV 8;
    var accdesc : AccessDescriptor = accdesc_in;
    var value : bits(size);

    // Check alignment on size of element accessed, not overall access size
    var aligned : boolean;
    if accdesc.ispair then
      let halfalignment : integer{} = (size if DIV accdesc.ispair 2 then asbytes integer{32};
      aligned DIV 2 IsAlignedSize(address, else halfbytes;
    var DIV 8);
    else
      aligned : boolean = IsAlignedSize(address, bytesalignment);
    end;

    if !aligned && AArch32\_UnalignedAccessFaults(accdesc) then
      let fault : FaultRecord = AlignmentFault(accdesc, ZeroExtend{64}(address));
      AArch32_Abort(fault);
    end;
    if aligned then
      value = AArch32\_MemSingle{size}(address, accdesc, aligned);
    else
      assert bytes > 1;
      value[7:0] = AArch32\_MemSingle{8}(address, accdesc, aligned);

      // For subsequent bytes, if they cross to a new translation page which assigns
      // Device memory type, it is CONSTRAINED UNPREDICTABLE whether an unaligned access
      // will generate an Alignment Fault.
      let c : Constraint = ConstrainUnpredictable(Unpredictable\_DEVPAGE2);
      assert c IN {Constraint\_FAULT, Constraint\_NONE};
      if c == Constraint\_NONE then aligned = TRUE; end;

      for i = 1 to bytes-1 do
        value[i*8] = AArch32\_MemSingle{8}(address+i, accdesc, aligned);
      end;
    end;

    if BigEndian(accdesc.acctype) then
      value = BigEndianReverse{size}(value);
    end;

    return value;
  end;

  // Perform a write of 'size' bits. The byte order is reversed for a big-endian access.
  setter
    let bytes : integer{} = size DIV 8;
    var value : bits(size) = value_in;
    var accdesc : AccessDescriptor = accdesc_in;

    // Check alignment on size of element accessed, not overall access size
    var aligned : boolean;
    if accdesc.ispair then
      let halfalignment : integer{} = (size if DIV accdesc.ispair 2 then asbytes integer{32};
      aligned DIV 2 IsAlignedSize(address, else halfbytes;
    var DIV 8);
    else
      aligned : boolean = IsAlignedSize(address, bytesalignment);
    end;

    if !aligned && AArch32\_UnalignedAccessFaults(accdesc) then
      let fault : FaultRecord = AlignmentFault(accdesc, ZeroExtend{64}(address));
      AArch32_Abort(fault);
    end;

    if BigEndian(accdesc.acctype) then
      value = BigEndianReverse{size}(value);
    end;
    if aligned then
      AArch32\_MemSingle{size}(address, accdesc, aligned) = value;
    else
      assert bytes > 1;
      AArch32\_MemSingle{8}(address, accdesc, aligned) = value[7:0];

      // For subsequent bytes, if they cross to a new translation page which assigns
      // Device memory type, it is CONSTRAINED UNPREDICTABLE whether an unaligned access
      // will generate an Alignment Fault.

      let c : Constraint = ConstrainUnpredictable(Unpredictable\_DEVPAGE2);
      assert c IN {Constraint\_FAULT, Constraint\_NONE};

```

```

        if c == Constraint NONE then aligned = TRUE; end;

        for i = 1 to bytes-1 do
            AArch32 MemSingle{8}(address+i, accdesc, aligned) = value[i*:8];
        end;
    end;
    return;
end;
end;

```

Library pseudocode for aarch32/functions/system/AArch32_WriteModeByInstr

```

// AArch32_WriteModeByInstr()
// =====
// Function for dealing with writes to PSTATE.M from an AArch32 instruction, and ensuring that
// illegal state changes are correctly flagged in PSTATE.II.

func AArch32_WriteModeByInstr(mode : bits(5))
begin
    var (valid,el) : (boolean, bits(2)) = ELFromM32(mode);

    // 'valid' is set to FALSE if 'mode' is invalid for this implementation or the current value
    // of SCR.NS/SCR_EL3.NS. Additionally, it is illegal for an instruction to write 'mode' to
    // PSTATE.EL if it would result in any of:
    // * A change to a mode that would cause entry to a higher Exception level.
    if UInt(el) > UInt(PSTATE.EL) then
        valid = FALSE;
    end;

    // * A change to or from Hyp mode.
    if (PSTATE.M == PSTATE.EL) then
        valid = FALSE;
    end;

    // * A change to or from Hyp mode.
    if (PSTATE.M == M32 Hyp || mode == M32 Hyp) && PSTATE.M != mode then
        valid = FALSE;
    end;

    // * When EL2 is implemented, the value of HCR.TGE is '1', a change to a Non-secure EL1 mode.
    if (PSTATE.M == PSTATE.M != mode) then
        valid = FALSE;
    end;

    // * When EL2 is implemented, the value of HCR.TGE is '1', a change to a Non-secure EL1 mode.
    if (PSTATE.M == M32 Monitor && HaveEL(EL2) && el == EL1 && SCR().NS == '1'
        && HCR().TGE == '1') then
        valid = FALSE;
    end;

    if !valid then
        PSTATE.II && SCR().NS == '1'
        && HCR().TGE == '1') then
            valid = FALSE;
        end;

    if !valid then
        PSTATE.II = '1';
    else
        AArch32_WriteMode(mode);
    end;
end;

```

Library pseudocode for aarch32/functions/tlbi/AArch32_DTLBI_ALL

```
// AArch32_DTLBI_ALL()
// =====
// Invalidate all data TLB entries for the indicated translation regime with the
// the indicated security state for all TLBs within the indicated broadcast domain.
// Invalidation applies to all applicable stage 1 and stage 2 entries.

func AArch32_DTLBI_ALL(security : SecurityState, regime : Regime, broadcast_in broadcast : Broadcast,
    attr_in attr : TLBIMemAttr)
begin
    assert PSTATE.EL IN { PSTATE.EL IN {EL3, EL2, EL1}};
    var attr : TLBIMemAttr = attr_in;

    let broadcast : Broadcast = AArch32_EffectiveBroadcast(broadcast_in);
    if ExcludeXS() then
        attr = TLBI_ExcludeXS;
    end;

    var r : TLBIRecord;
    r.op = TLBIOp_DALL;
    r.from_aarch64 = FALSE;
    r.security = security;
    r.regime = regime;
    r.level = TLBILevel_Any;
    r.attr = attr;

    TLBI(r);
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;
    return;
end;
```

Library pseudocode for aarch32/functions/tlbi/AArch32_DTLBI_ASID

```
// AArch32_DTLBI_ASID()
// =====
// Invalidate all data TLB stage 1 entries matching the indicated VMID (where regime supports)
// and ASID in the parameter Rt in the indicated translation regime with the
// indicated security state for all TLBs within the indicated broadcast domain.
// Note: stage 1 and stage 2 combined entries are in the scope of this operation.

func AArch32_DTLBI_ASID(security : SecurityState, regime : Regime, vmid : bits(16),
    broadcast_in broadcast : Broadcast, attr_in attr : TLBIMemAttr, Rt : bits(32))
begin
    assert PSTATE.EL IN { PSTATE.EL IN {EL3, EL2, EL1}};
    var attr : TLBIMemAttr = attr_in;

    let broadcast : Broadcast = AArch32_EffectiveBroadcast(broadcast_in);
    if ExcludeXS() then
        attr = TLBI_ExcludeXS;
    end;

    var r : TLBIRecord;
    r.op = TLBIOp_DASID;
    r.from_aarch64 = FALSE;
    r.security = security;
    r.regime = regime;
    r.vmid = vmid;
    r.use_vmid = UseVMID(regime);
    r.level = TLBILevel_Any;
    r.attr = attr;
    r.asid = Zeros{8} :: Rt[7:0];

    TLBI(r);
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;
    return;
end;
```

Library pseudocode for aarch32/functions/tlbi/AArch32_DTLBI_VA

```
// AArch32_DTLBI_VA()
// =====
// Invalidate by VA all stage 1 data TLB entries in the indicated broadcast domain
// matching the indicated VMID and ASID (where regime supports VMID, ASID) in the indicated regime
// with the indicated security state.
// ASID, VA and related parameters are derived from Rt.
// Note: stage 1 and stage 2 combined entries are in the scope of this operation.

func AArch32_DTLBI_VA(security : SecurityState, regime : Regime, vmid : bits(16),
    broadcast_in broadcast : Broadcast, level : TLBILevel, attr_in attr : TLBIMemAttr,
    Rt : bits(32))
begin
    assert PSTATE.EL IN { PSTATE.EL IN {EL3, EL2, EL1}};
    var attr : TLBIMemAttr = attr_in;

    let broadcast : Broadcast = AArch32_EffectiveBroadcast(broadcast_in);
    if ExcludeXS() then
        attr = TLBI_ExcludeXS;
    end;

    var r : TLBIRecord;
    r.op = TLBIOp_DVA;
    r.from_aarch64 = FALSE;
    r.security = security;
    r.regime = regime;
    r.vmid = vmid;
    r.use_vmid = UseVMID(regime);
    r.level = level;
    r.attr = attr;
    r.asid = Zeros{8} :: Rt[7:0];
    r.address = Zeros{32} :: Rt[31:12] :: Zeros{12};

    TLBI(r);
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;
    return;
end;
```

Library pseudocode for aarch32/functions/tlbi/AArch32_EffectiveBroadcastAArch32_ITLBI_ALL

```
// AArch32_EffectiveBroadcastAArch32_ITLBI_ALL()
// =====
// Invalidate all instruction TLB entries for the indicated translation regime with the
// the indicated security state for all TLBs within the indicated broadcast domain.
// Invalidation applies to all applicable stage 1 and stage 2 entries.

func AArch32_ITLBI_ALL(security : AArch32_EffectiveBroadcast(broadcast_in, regime : BroadcastRegime), => broadcast : Broadcast
begin
    var broadcast : Broadcast;
    attr : BroadcastTLBIMemAttr =>
begin
    assert broadcast_in;

    if PSTATE.EL IN { PSTATE.EL3, EL2, EL1 } then return broadcast; end;

    if var (broadcast : Broadcast_NSH, TLBIRecord r) && ((
        r.op = ELUsingAArch32TLBIOp_IALL;
        r.from_aarch64 = FALSE;
        r.security = security;
        r.regime = regime;
        r.level = EL2TLBILevel_Any;
        r.attr = attr;
    ) then
        TLBI(r);
        if HCR().FBbroadcast == '1' ||
            (!ELUsingAArch32Broadcast_NSH(EL2) && HCR_EL2().FB == '1')) then
            broadcast = Broadcast_ForcedISH; then BroadcastTLBI(broadcast, r); end;
        end;
    end;

    return broadcast;
end;
```


Library pseudocode for aarch32/functions/tlbi/AArch32_ITLBI_ALLAArch32_ITLBI_ASID

```
// AArch32_ITLBI_ALLAArch32_ITLBI_ASID()
// =====
// Invalidate all instruction TLB stage 1 entries for matching the indicated translation VMID
// (where regime with supports) and ASID in the parameter Rt in the indicated translation
// regime with the indicated security state for all TLBs within the indicated broadcast domain.
// InvalidationNote: applies to all applicable stage 1 and stage 2 entries. combined entries are in the scope of this
// operation.

func AArch32_ITLBI_ALLAArch32_ITLBI_ASID(security : SecurityState, regime : Regime, broadcast_in vmid : bits(16),
    broadcast : Broadcast,
    attr_in attr : TLBIMemAttr, Rt : bits(32))
begin
    assert PSTATE.EL IN { PSTATE.EL IN {EL3, EL2, EL1};
    var attr : TLBIMemAttr TLBIRecord ;
    r.op = attr_in;

    let broadcast : Broadcast TLBIOp_IASID ;
    r.from_aarch64 = FALSE;
    r.security = security;
    r.regime = regime;
    r.vmid = vmid;
    r.use_vmid = AArch32_EffectiveBroadcastUseVMID(broadcast_in);
    if ExcludeXS() then
        attr = TLBI ExcludeXS;
    end;

    var r : TLBIRecord;
    r.op = TLBIOp_IALL;
    r.from_aarch64 = FALSE;
    r.security = security;
    r.regime = regime;
    r.level = TLBILevel Any;
    r.attr = attr;
    r.asid = Zeros(8) :: Rt[7:0];

    TLBI(r);
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;
    return;
end;
```

Library pseudocode for aarch32/functions/tlbi/AArch32_ITLBI_ASIDAArch32_ITLBI_VA

```
// AArch32_ITLBI_ASIDAArch32_ITLBI_VA()
// =====
// Invalidate allby instructionVA TLBAll stage 1 instruction TLB entries matching in the indicated VMID broadcast domain
// (where matching regime the supports) indicated VMID and ASID in (where the regime parameters supports Rt VMID, ASID) in the
// indicated translation regime
// regime with the indicated security statestate.
// forASID, allVA TLBs and within related the parameters indicated are broadcast derived domain, from Rt.
// Note: stage 1 and stage 2 combined entries are in the scope of this operation.

func AArch32_ITLBI_ASIDAArch32_ITLBI_VA(security : SecurityState, regime : Regime, vmid : bits(16),
    broadcast_in
    broadcast : Broadcast, attr_in level : TLBILevel, attr : TLBIMemAttr, Rt : bits(32))
begin
    assert PSTATE.EL IN { PSTATE.EL IN {EL3, EL2, EL1};
    var attr : TLBIMemAttr = attr_in;

    let broadcast : Broadcast = AArch32_EffectiveBroadcast(broadcast_in);
    if ExcludeXS() then
        attr = TLBI ExcludeXS;
    end;

    var r : TLBIRecord;
    r.op = TLBIOp_IASID TLBIOp_IVA;
    r.from_aarch64 = FALSE;
    r.security = security;
    r.regime = regime;
    r.vmid = vmid;
    r.use_vmid = UseVMID(regime);
    r.level = TLBILevel Any (regime);
    r.level = level;
    r.attr = attr;
    r.asid = Zeros(8) :: Rt[7:0];
    r.address = Zeros(32) :: Rt[31:12] :: Zeros(12);

    TLBI(r);
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;
    return;
end;
```

Library pseudocode for aarch32/functions/tlbi/AArch32_ITLBI_VA

```

// AArch32_ITLBI_VA()
// =====
// Invalidate by VA all stage 1 instruction TLB entries in the indicated broadcast domain
// matching the indicated VMID and ASID (where regime supports VMID, ASID) in the indicated regime
// with the indicated security state.
// ASID, VA and related parameters are derived from Rt.
// Note: stage 1 and stage 2 combined entries are in the scope of this operation.

func AArch32_ITLBI_VA(security : SecurityState, regime : Regime, vmid : bits(16),
    broadcast_in : Broadcast, level : TLBILevel, attr_in : TLBIMemAttr,
    Rt : bits(32))
begin
    assert PSTATE.EL IN {EL3, EL2, EL1};
    var attr : TLBIMemAttr = attr_in;

    let broadcast : Broadcast = AArch32_EffectiveBroadcast(broadcast_in);
    if ExcludeXS() then
        attr = TLBI_ExcludeXS;
    end;

    var r : TLBIRecord;
    r.op = TLBIOp_IVA;
    r.from_aarch64 = FALSE;
    r.security = security;
    r.regime = regime;
    r.vmid = vmid;
    r.use_vmid = UseVMID(regime);
    r.level = level;
    r.attr = attr;
    r.asid = Zeros{8} :: Rt[7:0];
    r.address = Zeros{32} :: Rt[31:12] :: Zeros{12};

    TLBI(r);
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;
    return;
end;

```

Library pseudocode for aarch32/functions/tlbi/AArch32_TLBI_ASID

```

// AArch32_TLBI_ASID()
// =====
// Invalidate all stage 1 entries matching the indicated VMID (where regime supports)
// and ASID in the parameter Rt in the indicated translation regime with the
// indicated security state for all TLBs within the indicated broadcast domain.
// Note: stage 1 and stage 2 combined entries are in the scope of this operation.

func AArch32_TLBI_ASID(security : SecurityState, regime : Regime, vmid : bits(16),
    broadcast_in : Broadcast, attr_in : TLBIMemAttr, Rt : bits(32))
begin
    assert PSTATE.EL IN {EL3, EL2, EL1};
    var attr : TLBIMemAttr = attr_in;

    let broadcast : Broadcast = AArch32_EffectiveBroadcast(broadcast_in);
    if ExcludeXS() then
        attr = TLBI_ExcludeXS;
    end;

    var r : TLBIRecord;
    r.op = TLBIOp_ASID;
    r.from_aarch64 = FALSE;
    r.security = security;
    r.regime = regime;
    r.vmid = vmid;
    r.use_vmid = UseVMID(regime);
    r.level = TLBILevel_Any;
    r.attr = attr;
    r.asid = Zeros{8} :: Rt[7:0];

    TLBI(r);
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;
    return;
end;

```

Library pseudocode for aarch32/functions/tlbi/AArch32_TLBI_VA

```
// AArch32_TLBI_VA()
// =====
// Invalidate by VA all stage 1 TLB entries in the indicated broadcast domain
// matching the indicated VMID and ASID (where regime supports VMID, ASID) in the indicated regime
// with the indicated security state.
// ASID, VA and related parameters are derived from Rt.
// Note: stage 1 and stage 2 combined entries are in the scope of this operation.

func AArch32_TLBI_VA(security : SecurityState, regime : Regime, vmid : bits(16),
    broadcast_inbroadcast : Broadcast, level : TLBILevel, attr_inattr : TLBIMemAttr,
    Rt : bits(32))
begin
    assert PSTATE.EL IN { PSTATE.EL IN {EL3, EL2, EL1};
    var attr : TLBIMemAttr = attr_in;

    let broadcast : Broadcast = AArch32_EffectiveBroadcast(broadcast_in);
    if ExcludeXS() then
        attr = TLBI_ExcludeXS;
    end;

    var r : TLBIRecord;
    r.op = TLBIOp_VA;
    r.from_aarch64 = FALSE;
    r.security = security;
    r.regime = regime;
    r.vmid = vmid;
    r.use_vmid = UseVMID(regime);
    r.level = level;
    r.attr = attr;
    r.asid = Zeros{8} :: Rt[7:0];
    r.address = Zeros{32} :: Rt[31:12] :: Zeros{12};

    TLBI(r);
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;
    return;
end;
```

Library pseudocode for aarch32/functions/tlbi/AArch32_TLBI_VAA

```
// AArch32_TLBI_VAA()
// =====
// Invalidate by VA all stage 1 TLB entries in the indicated broadcast domain
// matching the indicated VMID (where regime supports VMID) and all ASID in the indicated regime
// with the indicated security state.
// VA and related parameters are derived from Rt.
// Note: stage 1 and stage 2 combined entries are in the scope of this operation.

func AArch32_TLBI_VAA(security : SecurityState, regime : Regime, vmid : bits(16),
    broadcast_inbroadcast : Broadcast, level : TLBILevel, attr_inattr : TLBIMemAttr,
    Rt : bits(32))
begin
    assert PSTATE.EL IN { PSTATE.EL IN {EL3, EL2, EL1};
    var attr : TLBIMemAttr = attr_in;

    let broadcast : Broadcast = AArch32_EffectiveBroadcast(broadcast_in);
    if ExcludeXS() then
        attr = TLBI_ExcludeXS;
    end;

    var r : TLBIRecord;
    r.op = TLBIOp_VAA;
    r.from_aarch64 = FALSE;
    r.security = security;
    r.regime = regime;
    r.vmid = vmid;
    r.use_vmid = UseVMID(regime);
    r.level = level;
    r.attr = attr;
    r.address = Zeros{32} :: Rt[31:12] :: Zeros{12};

    TLBI(r);
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;
    return;
end;
```

Library pseudocode for aarch32/functions/tlbi/AArch32_TLBI_VMALL

```
// AArch32_TLBI_VMALL()
// =====
// Invalidate all stage 1 entries for the indicated translation regime with the
// the indicated security state for all TLBs within the indicated broadcast
// domain that match the indicated VMID (where applicable).
// Note: stage 1 and stage 2 combined entries are in the scope of this operation.
// Note: stage 2 only entries are not in the scope of this operation.

func AArch32_TLBI_VMALL(security : SecurityState, regime : Regime, vmid : bits(16),
    broadcast_inbroadcast : Broadcast, attr_inattr : TLBIMemAttr)
begin
    assert PSTATE.EL IN { PSTATE.EL IN {EL3, EL2, EL1}};
    var attr : TLBIMemAttr = attr_in;

    let broadcast : Broadcast = AArch32_EffectiveBroadcast(broadcast_in);
    if ExcludeXS() then
        attr = TLBI_ExcludeXS;
    end;

    var r : TLBIRecord;
    r.op = TLBIOp_VMALL;
    r.from_aarch64 = FALSE;
    r.security = security;
    r.regime = regime;
    r.level = TLBILevel Any;
    r.vmid = vmid;
    r.use_vmid = UseVMID(regime);
    r.attr = attr;

    TLBI(r);
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;
    return;
end;
```

Library pseudocode for aarch32/ic/AArch32_IC

```
// AArch32_IC()
// =====
// Perform Instruction Cache Operation.

func AArch32_IC(opscope : CacheOpScope)
begin
    let regval : bits(32) = ARBITRARY : bits(32);
    AArch32_IC(regval, opscope);
end;

// AArch32_IC()
// =====
// Perform Instruction Cache Operation.

func AArch32_IC(regval : bits(32), opscope : CacheOpScope)
begin
    var cache : CacheRecord;

    cache.acctype = AccessType\_IC;
    cache.cachetype = CacheType\_Instruction;
    cache.cacheop = CacheOp\_Invalidate;
    cache.opscope = opscope;
    cache.security = SecurityStateAtEL(PSTATE.EL);

if opscope IN {PSTATE.EL};

    if opscope IN {CacheOpScope\_ALLU, CacheOpScope\_ALLUIS} then
        if (opscope == CacheOpScope\_ALLUIS || then
cache.shareability =
AArch32\_EffectiveBroadcast(Broadcast\_NSH) == Broadcast\_ForcedISH) then
            cache.shareability = Shareability\_ISH;
        else
            cache.shareability = Shareability\_NSH;
        end;
        cache.regval = ZeroExtend{64}(regval);
        CACHE_OP(cache);
    else
        assert opscope == CacheOpScope\_PoU;

        if EL2Enabled() then
            if PSTATE.EL IN { PSTATE.EL IN {EL0, EL1} then
                cache.is_vmid_valid = TRUE;
                cache.vmid = VMID();
            else
                cache.is_vmid_valid = FALSE;
            end;
        else
            cache.is_vmid_valid = FALSE;
        end;

if PSTATE.EL();
else
cache.is_vmid_valid = FALSE;
end;
else
cache.is_vmid_valid = FALSE;
end;

        if PSTATE.EL == EL0 then
            cache.is_asid_valid = TRUE;
            cache.asid = ASID();
        else
            cache.is_asid_valid = FALSE;
        end;

        cache.shareability = Shareability\_NSH;
        cache.vaddress = ZeroExtend{64}(regval);

        let size : integer = 0;
        let aligned : boolean = TRUE;
        let accdesc : AccessDescriptor = CreateAccDescIC(cache);
        let memaddrdesc : AddressDescriptor = AArch32\_TranslateAddress(regval, accdesc, aligned,
                                                                    size);

        if IsFault(memaddrdesc) then
            AArch32_Abort(memaddrdesc.fault);
        end;

        cache.paddress = memaddrdesc.paddress;
        CACHE_OP(cache);
    end;
    return;
end;
```

Library pseudocode for aarch32/translation/attrs/AArch32_MAIRAttr

```
// AArch32_MAIRAttr()
// =====
// Retrieve the memory attribute encoding indexed in the given MAIR

func AArch32_MAIRAttr(index : integer, mair : bits(64) MAIRType) => bits(8)
begin
    assert (index < 8);
    return mair[index*8];
end;
```

Library pseudocode for aarch64/debug/brbe/BRBEDebugStateExit

```
// BRBEDebugStateExit()
// =====
// Called to write Debug state exit branch record when BRBE is active.

func BRBEDebugStateExit(target_address : bits(64))
begin
    if BranchRecordAllowed(PSTATE.EL) then
        // Debug state is a prohibited region, therefore ccu=1, cc=0, source_address=0
        let branch_type : bits(6) = '111001';
        let ccu : bit = '1';
        let cc : bits(14) = Zeros();
        let el : bits(2) = PSTATE.EL;
        let mispredict : bit = '0';

        UpdateBranchRecordBuffer(ccu, cc, branch_type, el, mispredict,
                                '01', Zeros(64), target_address);

        PMUEvent(PMU_EVENT_BRB_FILTRATE);
    end;

    return;
end; PSTATE.EL) then
    // Debug state is a prohibited region, therefore ccu=1, cc=0, source_address=0
    let branch_type : bits(6) = '111001';
    let ccu : bit = '1';
    let cc : bits(14) = Zeros(14);
    let el : bits(2) = PSTATE.EL;
    let mispredict : bit = '0';

    UpdateBranchRecordBuffer(ccu, cc, branch_type, el, mispredict,
                            '01', Zeros(64), target_address);

    PMUEvent(PMU_EVENT_BRB_FILTRATE);
end;

return;
end;
```

Library pseudocode for aarch64/debug/brbe/BranchEncCycleCount

```
// BranchEncCycleCount()
// =====

// The first return result is '1' if either of the following is true, and '0' otherwise:
// - This is the first Branch record after the PE exited a Prohibited Region.
// - This is the first Branch record after cycle counting has been enabled.
// If the first return result is '0', the second return result is the encoded cycle count
// since the last branch.
// The format of this field uses a mantissa and exponent to express the cycle count value.
// - bits[7:0] indicate the mantissa M.
// - bits[13:8] indicate the exponent E.
// The cycle count is expressed using the following function:
//   cycle_count = (if IsZero(E) then UInt(M) else UInt('1'::M::Zeros((UInt(E)-1))))))
// A value of all ones in both the mantissa and exponent indicates the cycle count value
// exceeded the size of the cycle counter.
// If the cycle count is not known, the second return result is zero.

func BranchEncCycleCount() => (bit, bits(14))
begin
    var cc : integer = BranchRawCycleCount();
    if !BRBCycleCountingEnabled() || FirstBranchAfterProhibited() then
        return ('1', Zeros{14});
    end;

    // The format of this field uses a mantissa and exponent to express the cycle count value.
    // - bits[7:0] indicate the mantissa M.
    // - bits[13:8] indicate the exponent E.
    // The cycle count is expressed using the following function:
    //   cycle_count = (if IsZero(E) then UInt(M) else UInt('1'::M::Zeros((UInt(E)-1))))))
    // A value of all ones in both the mantissa and exponent indicates the cycle count value
    // exceeded the size of the cycle counter.

    var E : bits(6);
    var M : bits(8);

    if cc < 2^8 then
        E = Zeros{6};
        M = cc[7:0];
    elsif cc >= 2^20 then
        E = Ones{6};
        M = Ones{8};
    else
        E = 1[5:0];
        while cc >= 2^9 looplimit 11 do
            E = E + 1;
            cc = cc DIVRM 2;
        end;
        M = cc[7:0];
    end;

    return ('0', E::M);
end;
```



```

// AArch64_BreakpointValueMatch()
// =====
// Returns breakpoint type to indicate the type of breakpoint and a boolean to indicate
// whether the breakpoint matched successfully. For Address Mismatch breakpoints, the
// returned boolean is the inverted result. If the breakpoint type return value is Inactive,
// then the boolean result is FALSE.

func AArch64_BreakpointValueMatch(n_in : integer, vaddress : bits(64),
                                  linked_to : boolean, isbreakpnt : boolean,
                                  from_linking_enabled : boolean
                                ) => (BreakpointType, boolean) recurselimit 2

begin

    // "n_in" is the identity of the breakpoint unit to match against.
    // "vaddress" is the current instruction address, ignored if linked_to is TRUE and for Context
    // matching breakpoints.
    // "linked_to" is TRUE if this is a call from StateMatch for linking.
    // "isbreakpnt" TRUE is this is a call from BreakpointMatch or from StateMatch for a
    // linked breakpoint or from BreakpointValueMatch for a linked breakpoint with linking enabled.
    // "from_linking_enabled" is TRUE if this is a call from BreakpointValueMatch for a linked
    // breakpoint with linking enabled.
    var n : integer = n_in;
    var c : Constraint;
    var dbgtype : bits(5);

    // If a non-existent breakpoint then it is CONSTRAINED UNPREDICTABLE whether this gives
    // no match or the breakpoint is mapped to another UNKNOWN implemented breakpoint.
    if n >= NumBreakpointsImplemented() then
        (c, n) = ConstrainUnpredictableInteger(0, NumBreakpointsImplemented() - 1,
                                              Unpredictable_BPNOTIMPL);
        assert c IN {Constraint_DISABLED, Constraint_UNKNOWN};
        if c == Constraint_DISABLED then return (BreakpointType_Inactive, FALSE); end;
    end;

    // If this breakpoint is not enabled, it cannot generate a match.
    // (This could also happen on a call from StateMatch for linking).
    if !IsBreakpointEnabled(n) then return (BreakpointType_Inactive, FALSE); end;

    // If BT is set to a reserved type, behaves either as disabled or as a not-reserved type.
    if IsFeatureImplemented(FEAT_ABLE) then
        dbgtype = DBGBCR_EL1(n).[BT2,BT];
    else
        dbgtype = '0' :: DBGBCR_EL1(n).BT;
    end;

    (c, dbgtype) = AArch64_ReservedBreakpointType(n, dbgtype);
    if c == Constraint_DISABLED then return (BreakpointType_Inactive, FALSE); end;
    // Otherwise the value returned by ConstrainUnpredictableBits must be a not-reserved value

    // Determine what to compare against.
    let match_addr : boolean = (dbgtype == 'x0x0x');
    let mismatch : boolean = (dbgtype == 'x010x');
    let match_vmid : boolean = (dbgtype == 'x10xx');
    let match_cid : boolean = (dbgtype == 'x001x');
    let match_cid1 : boolean = (dbgtype IN {'x101x', 'xx11x'});
    let match_cid2 : boolean = (dbgtype == 'x11xx');
    let linking_enabled : boolean = (dbgtype IN {'xxx11', 'x1xx1', '1xxxx'});

    // If this is a call from StateMatch, return FALSE if the breakpoint is not
    // programmed with linking enabled.
    if linked_to && !linking_enabled then
        return (BreakpointType_Inactive, FALSE);
    end;

    // If called from BreakpointMatch return FALSE for breakpoint with linking enabled.
    if !linked_to && linking_enabled then
        return (BreakpointType_Inactive, FALSE);
    end;

    let linked : boolean = (dbgtype == 'x0x01');
    if from_linking_enabled then // A breakpoint with linking enabled has called this function.
        assert linked_to && isbreakpnt;
        if linked then
            // A breakpoint with linking enabled is linked to a linked breakpoint. This is
            // architecturally UNPREDICTABLE, but treated as disabled in the pseudo code to
            // avoid potential recursion in BreakpointValueMatch().
            return (BreakpointType_Inactive, FALSE);
        end;
    end;

    // If a linked breakpoint is linked to an address matching breakpoint,
    // the behavior is CONSTRAINED UNPREDICTABLE.
    if linked_to && match_addr && isbreakpnt then
        if !ConstrainUnpredictableBool(Unpredictable_BPLINKEDADDRMATCH) then
            return (BreakpointType_Inactive, FALSE);
        end;
    end;

    // A breakpoint programmed for address mismatch does not match in AArch32 state.

```

```

if mismatch && UsingAArch32() then
    return (BreakpointType_Inactive, FALSE);
end;

var bvr_match : boolean = FALSE;
var bxvr_match : boolean = FALSE;
var bp_type : BreakpointType;
var mask : integer{0..31};

if IsFeatureImplemented(FEAT_BWE) then
    mask = UInt(DBGBCR_EL1(n).MASK);

    // If the mask is set to a reserved value, the behavior is CONSTRAINED UNPREDICTABLE.
    if mask IN {1, 2} then
        var unpred_mask : integer;
        (c, unpred_maskmask) = (ConstrainUnpredictableInteger(3, 31, Unpredictable_RESBPMASK);)
        as (Constraint, integer{3..31}));
        assert c IN {Constraint_DISABLED, Constraint_NONE, Constraint_UNKNOWN};
        case c of
            when Constraint_DISABLED => return (BreakpointType_Inactive, FALSE); // Disabled
            when Constraint_NONE => mask = 0; // No masking
            // Otherwise the value returned by ConstrainUnpredictableBits must
            // be a not-reserved value.
            otherwise => mask = unpred_mask as integer{3..31};
        end;
    end;

if mask != 0 then
    // When DBGBCR_EL1(n).MASK is a valid nonzero value, the behavior is
    // CONSTRAINED UNPREDICTABLE if any of the following are true:
    // - DBGBCR_EL1(n).[BT2,BT] is programmed for a Context matching breakpoint.
    // - DBGBCR_EL1(n).BAS is not '1111' and AArch32 is supported at EL0.
    if ((match_cid || match_cid1 || match_cid2) ||
        (DBGBCR_EL1(n).BAS != '1111' && HaveAArch32())) then
        if !ConstrainUnpredictableBool(Unpredictable_BPMASK) then
            return (BreakpointType_Inactive, FALSE);
        end;
    end;
else
    // A stand-alone mismatch of a single address is not supported.
    if mismatch then
        return (BreakpointType_Inactive, FALSE);
    end;
end;

else
    mask = 0;
end;

// Do the comparison.
if match_addr then
    var byte_select_match : boolean;
    let byte : integer = UInt(vaddress[1:0]);

    if HaveAArch32() then
        // T32 instructions can be executed at EL0 in an AArch64 translation regime.
        assert byte IN {0,2}; // "vaddress" is halfword aligned
        byte_select_match = (DBGBCR_EL1(n).BAS[byte] == '1');
    else
        assert byte == 0; // "vaddress" is word aligned
        byte_select_match = TRUE; // DBGBCR_EL1(n).BAS[byte] is RES1
    end;

    // When FEAT_LVA3 is not implemented, if the DBGBCR_EL1(n).RESS field bits are not a
    // sign extension of the MSB of DBGBCR_EL1(n).VA, it is UNPREDICTABLE whether they
    // appear to be included in the match.
    // If 'vaddress' is outside of the current virtual address space, then the access
    // generates a Translation fault.
    let dbgtop : AddressSize = DebugAddrTop();
    let unpredictable_ress : boolean = (dbgtop < 55 && !IsOnes(DBGBCR_EL1(n)[63:dbgtop]) &&
        !IsZero(DBGBCR_EL1(n)[63:dbgtop]) &&
        ConstrainUnpredictableBool(Unpredictable_DBGxVR_RESS));
    let cmpmsb : integer{} = if unpredictable_ress then 63 else dbgtop;
    let cmpls : integer{} = if mask > 2 then mask else 2;
    bvr_match = ((vaddress[cmpmsb:cmpls] == DBGBCR_EL1(n)[cmpmsb:cmpls]) &&
        byte_select_match);
    if mask > 2 then
        // If masked bits of DBGBCR_EL1(n) are not zero, the behavior
        // is CONSTRAINED UNPREDICTABLE.
        let masktop : integer{} = mask - 1;
        if bvr_match && !IsZero(DBGBCR_EL1(n)[masktop:2]) then
            bvr_match = ConstrainUnpredictableBool(Unpredictable_BPMAKEDBITS);
        end;
    end;

elseif match_cid then
    if IsInHost() then
        bvr_match = (CONTEXTIDR_EL2()[31:0] == DBGBCR_EL1(n)[31:0]);
    else

```

```

        bvr_match = (PSTATE.EL IN {EL0, EL1} && CONTEXTIDR_EL1() [31:0] == DBGEBR_EL1(n) [31:0]);
    end;

    elsif match_cid1 then
        bvr_match = (PSTATE.EL IN {EL0, EL1} && !IsInHost() &&
            CONTEXTIDR_EL1() [31:0] == DBGEBR_EL1(n) [31:0]);
    end;

    if match_vmid then
        let vmid : bits(16) = VMID();
        var bvr_vmid : bits(16);

        if !IsFeatureImplemented(FEAT_VMID16) || VTCR_EL2().VS == '0' then
            bvr_vmid = ZeroExtend{16}(DBGEBR_EL1(n) [39:32]);
        else
            bvr_vmid = DBGEBR_EL1(n) [47:32];
        end;

        bxvr_match = (PSTATE.EL IN {EL0, EL1} && EL2Enabled() && !IsInHost() && vmid == bvr_vmid);

    elsif match_cid2 then
        bxvr_match = (PSTATE.EL != EL3 && EL2Enabled() &&
            DBGEBR_EL1(n) [63:32] == CONTEXTIDR_EL2() [31:0]);
    end;

    let bvr_match_valid : boolean = (match_addr || match_cid || match_cid1);
    let bxvr_match_valid : boolean = (match_vmid || match_cid2);

    var value_match : boolean = ((!bxvr_match_valid || bxvr_match) &&
        (!bvr_match_valid || bvr_match));

    // A watchpoint might be linked to a linked address matching breakpoint with linking enabled,
    // which is in turn linked to a context matching breakpoint.
    if linked_to && linked then
        // If called from StateMatch and breakpoint is a linked breakpoint then it must be a
        // watchpoint that is linked to an address matching breakpoint which is linked to a
        // context matching breakpoint.
        assert !isbreakpnt && match_addr && IsFeatureImplemented(FEAT_ABLE);
        let lbnx : bits(2) = (if IsFeatureImplemented(FEAT_Debugv8p9) then DBGBCR_EL1(n).LBNX
            else '00');

        let linked_linked_n : integer{} = UInt(lbnx :: DBGBCR_EL1(n).LBN);
        var linked_value_match : boolean;
        let linked_vaddress = ARBITRARY : bits(64);
        let linked_linked_to = TRUE;
        let linked_isbreakpnt = TRUE;
        let linked_from_linking_enabled = TRUE;
        (bp_type, linked_value_match) = AArch64_BreakpointValueMatch(linked_linked_n,
            linked_vaddress,
            linked_linked_to,
            linked_isbreakpnt,
            linked_from_linking_enabled);

        value_match = value_match && linked_value_match;
    end;

    if match_addr && !mismatch then
        bp_type = BreakpointType_AddrMatch;
    elsif match_addr && mismatch then
        bp_type = BreakpointType_AddrMismatch;
    elsif match_vmid || match_cid || match_cid1 || match_cid2 then
        bp_type = BreakpointType_CtxtMatch;
    else
        unreachable;
    end;

    return (bp_type, value_match);
end;

```

Library pseudocode for aarch64/debug/ebep

```

var inst_addr_executed, sync_counter_overflowed : bits(64); boolean;
var inst_addr_executed : bits(64);

```

Library pseudocode for aarch64/debug/statisticalprofiling/CheckMDCR_EL3_NSPBTrap

```
// CheckMDCR_EL3_NSPBTrap()
// =====
// Check if the register access is trappable by MDCR_EL3.[MDCR_EL3().NSPBE, NSPB]

func CheckMDCR_EL3_NSPBTrap() => boolean
begin
    var state_bits : bits(3);
    var reserved : boolean;
    (state_bits, reserved) = EffectiveMDCR_EL3_NSPB();
    return ((reserved && ConstrainUnpredictableBool(Unpredictable RESERVEDNSxB_Trap)) ||
        state_bits[0] == '0' || state_bits[1] != SCR_EL3().NS ||
        (IsFeatureImplemented(FEAT_RME) && state_bits[2] != SCR_EL3().NSE));
end; EffectiveSCR_EL3_NS() ||
    (IsFeatureImplemented(FEAT_RME) && state_bits[2] != EffectiveSCR_EL3_NSE());
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/EffectiveMDCR_EL3_NSPB

```
// EffectiveMDCR_EL3_NSPB()
// =====
// Return the Effective value of MDCR_EL3.[MDCR_EL3().NSPBE, NSPB] field and whether it is a reserved value.

readonly func EffectiveMDCR_EL3_NSPB() => (bits(3), boolean)
begin
    var state_bits : bits(3);
    var reserved : boolean = FALSE;

    if IsFeatureImplemented(FEAT_RME) then
        state_bits = MDCR_EL3().[NSPBE, NSPB];
        if state_bits == '10x' || (!IsFeatureImplemented(FEAT_Secure) && state_bits == '00x') then
            // Reserved value
            reserved = TRUE;
            (-, state_bits) = ConstrainUnpredictableBits{3}(Unpredictable RESERVEDNSxB);
        end;
    else
        state_bits = '0' :: MDCR_EL3().NSPB;
    end;

    return (state_bits, reserved);
end;
```



```

// SPEEvent()
// =====
// Called when a PMU event is generated by the sampled structure.
// Sets appropriate bit in SPESampleStorage.events.

func SPEEvent(pmuevent : bits(16))
begin
    if !SPESampleInFlight then
        return;
    end;
    case pmuevent of
        when PMU_EVENT_DSNP_HIT_RD =>
            if IsFeatureImplemented(FEAT_SPEvlp4) then
                SPESampleEvents[23] = '1'; // Snoop hit
            end;
        when {PMU_EVENT_L1D_LFB_HIT_RD, PMU_EVENT_L2D_LFB_HIT_RD,
            PMU_EVENT_L3D_LFB_HIT_RD, PMU_EVENT_LL_LFB_HIT_RD} =>
            if IsFeatureImplemented(FEAT_SPEvlp4) then
                SPESampleEvents[22] = '1'; // Recent fetch
            end;
        when {PMU_EVENT_L1D_CACHE_HITM_RD, PMU_EVENT_L2D_CACHE_HITM_RD,
            PMU_EVENT_L3D_CACHE_HITM_RD, PMU_EVENT_LL_CACHE_HITM_RD} PMU_EVENT_L2D_LFB_HIT_RD =>
            if IsFeatureImplemented(FEAT_SPEvlp4) then
                SPESampleEvents[21:22] = '1'; // ModifiedRecent fetch
            end;
        when PMU_EVENT_L2D_CACHE_LMISS_RD PMU_EVENT_L3D_LFB_HIT_RD =>
            if IsFeatureImplemented(FEAT_SPEvlp4) then
                SPESampleEvents[20:22] = '1'; // L2Recent miss fetch
            end;
        when PMU_EVENT_L2D_CACHE_RD PMU_EVENT_LL_LFB_HIT_RD =>
            if IsFeatureImplemented(FEAT_SPEvlp4) then
                SPESampleEvents[19:22] = '1'; // L2Recent access fetch
            end;
        when PMU_EVENT_SVE_PRED_EMPTY_SPEC PMU_EVENT_L1D_CACHE_HITM_RD =>
            if IsFeatureImplemented(FEAT_SPEvlp1 FEAT_SPEvlp4) then
                SPESampleEvents[18:21] = '1'; // Empty predicate Modified
            end;
        when PMU_EVENT_SVE_PRED_NOT_FULL_SPEC PMU_EVENT_L2D_CACHE_HITM_RD =>
            if IsFeatureImplemented(FEAT_SPEvlp1 FEAT_SPEvlp4) then
                SPESampleEvents[17:21] = '1'; // Partial predicate Modified
            end;
        when PMU_EVENT_LDST_ALIGN_LAT PMU_EVENT_L3D_CACHE_HITM_RD =>
            if IsFeatureImplemented(FEAT_SPEvlp1 FEAT_SPEvlp4) then
                SPESampleEvents[11:21] = '1'; // Misaligned Modified
            end;
        when PMU_EVENT_REMOTE_ACCESS PMU_EVENT_LL_CACHE_HITM_RD =>
            if IsFeatureImplemented(FEAT_SPEvlp4) then SPESampleEvents[10:21] = '1'; // Remote access Modified
            end;
        when PMU_EVENT_LL_CACHE_MISS PMU_EVENT_L2D_CACHE_LMISS_RD =>
            if IsFeatureImplemented(FEAT_SPEvlp4) then SPESampleEvents[9:20] = '1'; // LLC L2 miss
            end;
        when PMU_EVENT_LL_CACHE PMU_EVENT_L2D_CACHE_RD =>
            if IsFeatureImplemented(FEAT_SPEvlp4) then SPESampleEvents[8:19] = '1'; // LLC L2 access
            end;
        when PMU_EVENT_BR_MIS_PRED PMU_EVENT_SVE_PRED_EMPTY_SPEC =>
            if IsFeatureImplemented(FEAT_SPEvlp1) then SPESampleEvents[7:18] = '1'; // Mispredicted Empty predicate
            end;
        when PMU_EVENT_BR_MIS_PRED RETIRED PMU_EVENT_SVE_PRED NOT_FULL_SPEC =>
            if IsFeatureImplemented(FEAT_SPEvlp1) then SPESampleEvents[7:17] = '1'; // Not Partial taken predicate
            end;
        when PMU_EVENT_DTLB_WALK PMU_EVENT_LDST_ALIGN_LAT =>
            if IsFeatureImplemented(FEAT_SPEvlp1) then SPESampleEvents[5:11] = '1'; // TLB walk Misaligned
            end;
        when PMU_EVENT_L1D_TLB PMU_EVENT_REMOTE_ACCESS =>
            SPESampleEvents[4:10] = '1'; // TLB Remote
    access
        when PMU_EVENT_L1D_CACHE_REFILL PMU_EVENT_LL_CACHE_MISS =>
            if !IsFeatureImplemented(FEAT_SPEvlp4) then
                SPESampleEvents[3:9] = '1'; // L1 LLC refill
            end; miss
        when PMU_EVENT_L1D_CACHE_LMISS_RD PMU_EVENT_LL_CACHE =>
            if IsFeatureImplemented(FEAT_SPEvlp4) then
                SPESampleEvents[3:8] = '1'; // L1 LLC miss
            end; access
        when PMU_EVENT_L1D_CACHE PMU_EVENT_BR_MIS_PRED =>
            SPESampleEvents[2:7] = '1'; // L1
    access Mispredicted
        when PMU_EVENT_INST_RETIRED PMU_EVENT_BR_MIS_PRED RETIRED =>
            SPESampleEvents[1:7] = '1'; //
    Retire Not taken
        when PMU_EVENT_EXC TAKEN PMU_EVENT_DTLB_WALK =>
            SPESampleEvents[0:5] = '1'; // Exception TLB walk
    otherwise when PMU_EVENT_L1D_TLB => return;
    end;
    return;
end; SPESampleEvents[4] = '1'; // TLB access
    when PMU_EVENT_L1D_CACHE_REFILL =>
        if !IsFeatureImplemented(FEAT_SPEvlp4) then
            SPESampleEvents[3] = '1'; // L1 refill
        end;
    when PMU_EVENT_L1D_CACHE_LMISS_RD =>
        if IsFeatureImplemented(FEAT_SPEvlp4) then
            SPESampleEvents[3] = '1'; // L1 miss
        end;

```

```

end;
when PMU_EVENT_L1D_CACHE => SPESampleEvents[2] = '1'; // L1 access
when PMU_EVENT_INST_RETIRED => SPESampleEvents[1] = '1'; // Retire
when PMU_EVENT_EXC_TAKEN => SPESampleEvents[0] = '1'; // Exception
otherwise => return;
end;
return;
end;

```



```

// AArch64_TakeExceptionInDebugState()
// =====
// Take an exception in Debug state to an Exception level using AArch64.

noreturn func AArch64_TakeExceptionInDebugState(target_el : bits(2), exception_in : ExceptionRecord)
begin
    assert HaveEL(target_el) && !ELUsingAArch32(target_el) && UInt(target_el) >= UInt(PSTATE.EL);
    assert target_el != PSTATE.EL;
    assert target_el != EL3 || EDSCR().SDD == '0';
    var except : ExceptionRecord = exception_in;
    var sync_errors : boolean;
    if IsFeatureImplemented(FEAT_IESB) then
        sync_errors = SCTLR_EL(target_el).IESB == '1';
        if IsFeatureImplemented(FEAT_DoubleFault) then
            sync_errors = sync_errors || (SCR_EL3().[EA,NMEA] == '11' && target_el == EL3);
        end;
        // The Effective value of SCTLR[].IESB might be zero in Debug state.
        if !ConstrainUnpredictableBool(Unpredictable_IESBinDebug) then
            sync_errors = FALSE;
        end;
    else
        sync_errors = FALSE;
    end;

    if !IsFeatureImplemented(FEAT_ExS) || SCTLR_EL(target_el).EIS == '1' then
        // Synchronize the context, including Instruction Fetch Barrier effect
        SynchronizeContext();
    end;

    // If coming from AArch32 state, the top parts of the X() registers might be set to zero
    let from_32 : boolean = UsingAArch32();
    if from_32 then AArch64_MaybeZeroRegisterUppers(); end;
    if from_32 && IsFeatureImplemented(FEAT_SME) && PSTATE.SM == '1' then
        ResetSVEState();
    else
        MaybeZeroSVEUppers(target_el);
    end;

    AArch64_ReportException(except, target_el);

    if IsFeatureImplemented(FEAT_GCS) then
        PSTATE.EXLOCK = '0'; // Effective value of GCSCR_ELx.EXLOCKEN is 0 in Debug state
    end;

    PSTATE.EL = target_el;
    PSTATE.nRW = '0';
    PSTATE.SP = '1'; PSTATE.SM == '1' then
        ResetSVEState();
    else
        MaybeZeroSVEUppers(target_el);
    end;

    AArch64_ReportException(except, target_el);

    if IsFeatureImplemented(FEAT_GCS) then
        PSTATE.EXLOCK = '0'; // Effective value of GCSCR_ELx.EXLOCKEN is 0 in Debug state
    end;

    PSTATE.EL = target_el;
    PSTATE.nRW = '0';
    PSTATE.SP = '1';

    SPSR_ELx() = ARBITRARY : bits(64);
    ELR_ELx() = ARBITRARY : bits(64);
    // PSTATE.[SS,D,A,I,F] are not observable and ignored in Debug state, so behave as if UNKNOWN.
    PSTATE.[SS,D,A,I,F] = ARBITRARY : bits(5);
    if IsFeatureImplemented(FEAT_SSBS) then PSTATE.SSBS = ARBITRARY : bit; end;
    if IsFeatureImplemented(FEAT_EBEP) then PSTATE.PM = ARBITRARY : bit; end;
    if IsFeatureImplemented(FEAT_SEBEP) then PSTATE.PPEND = '0'; end;
    if IsFeatureImplemented(FEAT_MTE) then PSTATE.TCO = '1'; end;
    PSTATE.IL = '0';
    if IsFeatureImplemented(FEAT_UAO) then PSTATE.UAO = '0'; end;
    if IsFeatureImplemented(FEAT_UINJ) then PSTATE.UINJ = '0'; end;
    if IsFeatureImplemented(FEAT_PAuth_LR) then PSTATE.PACM = '0'; end;
    if (IsFeatureImplemented(FEAT_PAN) && (PSTATE.EL == PSTATE.SSBS = ARBITRARY : bit; end;
    if IsFeatureImplemented(FEAT_EBEP) then PSTATE.PM = ARBITRARY : bit; end;
    if IsFeatureImplemented(FEAT_SEBEP) then PSTATE.PPEND = '0'; end;
    if IsFeatureImplemented(FEAT_MTE) then PSTATE.TCO = '1'; end;
    PSTATE.IL = '0';
    if IsFeatureImplemented(FEAT_UAO) then PSTATE.UAO = '0'; end;
    if IsFeatureImplemented(FEAT_UINJ) then PSTATE.UINJ = '0'; end;
    if IsFeatureImplemented(FEAT_PAuth_LR) then PSTATE.PACM = '0'; end;
    if (IsFeatureImplemented(FEAT_PAN) && (PSTATE.EL == EL1 ||
        (PSTATE.EL == PSTATE.EL == EL2 && ELIsInHost(EL0))) &&
        SCTLR_ELx().SPAN == '0') then
        PSTATE.PAN = '1';
    end;
    if from_32 then // Coming from AArch32
        PSTATE.IT = '00000000';

```

```

PSTATE.T = '0'; // PSTATE.J is RES0
end;
if IsFeatureImplemented(FEAT_BTI) then PSTATE.BTYPE().SPAN == '0' then
PSTATE.PAN = '1';
end;
if from 32 then // Coming from AArch32
PSTATE.IT = '00000000';
PSTATE.T = '0'; // PSTATE.J is RES0
end;
if IsFeatureImplemented(FEAT_BTI) then PSTATE.BTYPE = '00'; end;
DLR_EL0() = ARBITRARY : bits(64);
DSPSR_EL0() = ARBITRARY : bits(64);

EDSCR().ERR = '1';
UpdateEDSCRFields(); // Update EDSCR PE state flags.

if sync_errors then
SynchronizeErrors();
end;

EndOfInstruction();
end;

```

Library pseudocode for aarch64/debug/watchpoint/AArch64_WatchpointByteMatch

```

// AArch64_WatchpointByteMatch()
// =====

func AArch64_WatchpointByteMatch(n : integer, vaddress : bits(64)) => boolean
begin
let dbgtop : AddressSize = DebugAddrTop();
let cmpbottom : integer{} = if DBGWVR_EL1(n)[2] == '1' then 2 else 3; // Word or doubleword
var bottom : integer{} = cmpbottom;
let select : integer{} = UInt(vaddress[cmpbottom-1:0]);
var byte_select_match : boolean = (DBGWCR_EL1(n).BAS[select] != '0');
var mask : integer{} = UInt(DBGWCR_EL1(n).MASK);

// If DBGWCR_EL1(n).MASK is a nonzero value and DBGWCR_EL1(n).BAS is not set to '11111111', or
// DBGWCR_EL1(n).BAS specifies a non-contiguous set of bytes behavior is CONSTRAINED
// UNPREDICTABLE.
if mask > 0 && !IsOnes(DBGWCR_EL1(n).BAS) then
byte_select_match = ConstrainUnpredictableBool(Unpredictable_WPMASKANDBAS);
else
let LSB : bits(8) = (DBGWCR_EL1(n).BAS AND NOT(DBGWCR_EL1(n).BAS - 1));
let MSB : bits(8) = (DBGWCR_EL1(n).BAS + LSB);
if !IsZero(MSB AND (MSB - 1)) then // Not contiguous
byte_select_match = ConstrainUnpredictableBool(Unpredictable_WPBASCONTIGUOUS);
bottom = 3; // For the whole doubleword
end;
end;

// If the address mask is set to a reserved value, the behavior is CONSTRAINED UNPREDICTABLE.
if mask > 0 && mask <= 2 then
var c : Constraint;
var unpred_mask : integer;
(c, unpred_maskmask) = (ConstrainUnpredictableInteger(3, 31, Unpredictable_RESWPMASK););
as (Constraint, integer{0..31});
assert c IN {Constraint_DISABLED, Constraint_NONE, Constraint_UNKNOWN};
case c of
when Constraint_DISABLED => return FALSE; // Disabled
when Constraint_NONE => mask = 0; // No masking
// Otherwise the value returned by ConstrainUnpredictableInteger is a not-reserved value
otherwise => mask = unpred_mask as integer{3..31};
end;
end;

// When FEAT_LVA3 is not implemented, if the DBGWVR_EL1(n).RESS field bits are not a
// sign extension of the MSB of DBGWVR_EL1(n).VA, it is UNPREDICTABLE whether they
// appear to be included in the match.
let unpredictable_ress : boolean = (dbgtop < 55 && !IsOnes(DBGWVR_EL1(n)[63:dbgtop]) &&
!IsZero(DBGWVR_EL1(n)[63:dbgtop]) &&
ConstrainUnpredictableBool(Unpredictable_DBGxVR_RESS));
let cmpmsb : integer{} = if unpredictable_ress then 63 else dbgtop;
let cmpls : integer{} = if mask > bottom then mask else bottom;
let bottombit : integer{} = bottom;
var WVR_match : boolean = (vaddress[cmpmsb:cmpls] == DBGWVR_EL1(n)[cmpmsb:cmpls]);
if mask > bottom then
// If masked bits of DBGWVR_EL1(n) are not zero, the behavior is CONSTRAINED UNPREDICTABLE.
if WVR_match && !IsZero(DBGWVR_EL1(n)[cmpls-1:bottombit]) then
WVR_match = ConstrainUnpredictableBool(Unpredictable_WPMASKEDBITS);
end;
end;

return (WVR_match && byte_select_match);
end;

```

Library pseudocode for aarch64/exceptions/async/AArch64_TakePhysicalFIQException

```
// AArch64_TakePhysicalFIQException()
// =====

func AArch64_TakePhysicalFIQException()
begin

    let route_to_el3 : boolean = EffectiveSCR_EL3_FIQHaveEH() == '1';
    let route_to_el2 : boolean = (PSTATE.EL3 == EL3) && SCR_EL3().FIQ == '1';
    let route_to_el2 : boolean = (PSTATE.EL IN {EL0, EL1} && EL2Enabled() &&
        (HCR_EL2().TGE == '1' || HCR_EL2().FMO == '1'));
    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    let vect_offset : integer = 0x100;
    let except : ExceptionRecord = ExceptionSyndrome(Exception_FIQ);

    if route_to_el3 then
        AArch64_TakeException(EL3, except, preferred_exception_return, vect_offset);
    elseif PSTATE.EL == EL2 || route_to_el2 then
        assert PSTATE.EL != EL3;
        AArch64_TakeException(EL2, except, preferred_exception_return, vect_offset);
    else
        assert PSTATE.EL, except, preferred_exception_return, vect_offset;
    else
        assert PSTATE.EL IN {EL0, EL1};
        AArch64_TakeException(EL1, except, preferred_exception_return, vect_offset);
    end;
end;
```

Library pseudocode for aarch64/exceptions/async/AArch64_TakePhysicalIRQException

```
// AArch64_TakePhysicalIRQException()
// =====
// Take an enabled physical IRQ exception.

func AArch64_TakePhysicalIRQException()
begin

    let route_to_el3 : boolean = EffectiveSCR_EL3_IRQHaveEH() == '1';
    let route_to_el2 : boolean = (PSTATE.EL3 == EL3) && SCR_EL3().IRQ == '1';
    let route_to_el2 : boolean = (PSTATE.EL IN {EL0, EL1} && EL2Enabled() &&
        (HCR_EL2().TGE == '1' || HCR_EL2().IMO == '1'));
    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    let vect_offset : integer = 0x80;

    let except : ExceptionRecord = ExceptionSyndrome(Exception_IRQ);

    if route_to_el3 then
        AArch64_TakeException(EL3, except, preferred_exception_return, vect_offset);
    elseif PSTATE.EL == EL2 || route_to_el2 then
        assert PSTATE.EL != EL3;
        AArch64_TakeException(EL2, except, preferred_exception_return, vect_offset);
    else
        assert PSTATE.EL, except, preferred_exception_return, vect_offset;
    else
        assert PSTATE.EL IN {EL0, EL1};
        AArch64_TakeException(EL1, except, preferred_exception_return, vect_offset);
    end;
end;
```



```

// AArch64_TakeException()
// =====
// Take an exception to an Exception level using AArch64.

noreturn func AArch64_TakeException(target_el : bits(2), exception_in : ExceptionRecord,
                                   preferred_exception_return : bits(64), vect_offset_in : integer)
begin
    assert HaveEL(target_el) && !ELUsingAArch32(target_el) && UInt(target_el) >= UInt(PSTATE.EL);
    if PSTATE.EL;
    if Halted() then
        AArch64_TakeExceptionInDebugState(target_el, exception_in);
    end;
    var except : ExceptionRecord = exception_in;
    var sync_errors : boolean;
    var iesb_req : boolean;
    if IsFeatureImplemented(FEAT_IESB) then
        sync_errors = SCTLRL_EL(target_el).IESB == '1';
        if IsFeatureImplemented(FEAT_DoubleFault) then
            sync_errors = sync_errors || (SCR_EL3().[EA,NMEA] == '11' && target_el == EL3);
        end;
        if sync_errors && InsertIESBBeforeException(target_el) then
            SynchronizeErrors();
            if except.exceptype != Exception_SError then
                iesb_req = FALSE;
                sync_errors = FALSE;
                TakeUnmaskedPhysicalSErrorInterrupts(iesb_req);
            end;
        end;
    else
        sync_errors = FALSE;
    end;

    var brbe_source_allowed : boolean = FALSE;
    var brbe_source_address : bits(64) = Zeros{};
    if IsFeatureImplemented(FEAT_BRBE) then
        brbe_source_allowed = BranchRecordAllowed(PSTATE.EL);
        brbe_source_address = preferred_exception_return;
    end;

    if !IsFeatureImplemented(FEAT_ExS) || PSTATE.EL;
    brbe_source_address = preferred_exception_return;
    end;

    if !IsFeatureImplemented(FEAT_ExS) || SCTLRL_EL(target_el).EIS == '1' then
        // Synchronize the context, including Instruction Fetch Barrier effect
        SynchronizeContext();
    elseif !(except.exceptype == Exception_SoftwareBreakpoint ||
            (except.exceptype IN {Exception_SupervisorCall,
                                Exception_HypervisorCall,
                                Exception_MonitorCall} &&
             !except.trappedsyscallinst)) then
        InstructionFetchBarrier();
    end;

    // If coming from AArch32 state, the top parts of the X() registers might be set to zero
    let from_32 : boolean = UsingAArch32();
    if from_32 then AArch64_MaybeZeroRegisterUppers(); end;
    if from_32 && IsFeatureImplemented(FEAT_SME) && PSTATE.SM == '1' then
        ResetSVEState();
    else
        MaybeZeroSVEUppers(target_el);
    end;

    var vect_offset : integer = vect_offset_in;
    if UInt(target_el) > UInt(PSTATE.EL) then
        var lower_32 : boolean;
        if target_el == PSTATE.SM == '1' then
            ResetSVEState();
        else
            MaybeZeroSVEUppers(target_el);
        end;

        var vect_offset : integer = vect_offset_in;
        if UInt(target_el) > UInt(PSTATE.EL) then
            var lower_32 : boolean;
            if target_el == EL3 then
                if EL2Enabled() then
                    lower_32 = ELUsingAArch32(EL2);
                else
                    lower_32 = ELUsingAArch32(EL1);
                end;
            elseif IsInHost() && PSTATE.EL == PSTATE.EL == EL0 && target_el == EL2 then
                lower_32 = ELUsingAArch32(EL0);
            else
                lower_32 = ELUsingAArch32(target_el - 1);
            end;
            vect_offset = vect_offset + (if lower_32 then 0x600 else 0x400);
        elseif PSTATE.SP == '1' then

```

```

    vect_offset = vect_offset + 0x200;
end;

var spsr : bits(64) = GetPSRFromPSTATE(){ PSTATE.SP == '1' then
    vect_offset = vect_offset + 0x200;
end;

assert vect_offset < 2048;
let vect_base : bits(64) = VBAR_EL(target_el)[63:11]::Zeros{11};
let target_vector : bits(64) = vect_base + vect_offset;

var spsr : bits(64) = GetPSRFromPSTATE(){(AArch64_NonDebugState);

if PSTATE.EL == PSTATE.EL == EL1 && target_el == EL1 && EL2Enabled() then
    if EffectiveHCR_EL2_NVx() IN {'x01', '111'} then
        spsr[3:2] = '10';
    end;
end;

if IsFeatureImplemented(FEAT_BTI) && !UsingAArch32() then
    var zero_btype : boolean;
    // SPSR_ELx[].BTYPE is only guaranteed valid for these exception types
    if except.exceptype IN {Exception_Error, Exception_IRQ, Exception_FIQ,
        Exception_SoftwareStep, Exception_PCAIgment,
        Exception_InstructionAbort, Exception_Breakpoint,
        Exception_VectorCatch, Exception_SoftwareBreakpoint,
        Exception_IllegalState, Exception_BranchTarget} then
        zero_btype = FALSE;
    else
        zero_btype = ConstrainUnpredictableBool(Unpredictable_ZEROBTYPE);
    end;
    if zero_btype then spsr[11:10] = '00'; end;
end;

if (IsFeatureImplemented(FEAT_NV2) &&
    except.exceptype == Exception_NV2DataAbort && target_el == EL3) then
    // External aborts are configured to be taken to EL3
    except.exceptype = Exception_DataAbort;
end;
if ! except.exceptype IN {Exception_IRQ, Exception_FIQ} then
    AArch64_ReportException(except, target_el);
end;

if IsFeatureImplemented(FEAT_BRBE) then
    BRBEException(except, let brbe_source_allowed, brbe_target_address brbe_source_address,
        target_vector, target_el, except.trappedsyscallinst);
end;

if IsFeatureImplementedbits(FEAT_GCS64) then
    if PSTATE.VBAR_EL(target_el)[63:11]::vect_offset[10:0];
    BRBEException(except, brbe_source_allowed, brbe_source_address,
        brbe_target_address, target_el,
        except.trappedsyscallinst);
end;

if IsFeatureImplemented(FEAT_GCS) then
    if PSTATE.EL == target_el then
        if GetCurrentEXLOCKEN() then
            PSTATE.EXLOCK = '1';
        else
            PSTATE.EXLOCK = '0';
        end;
    else
        PSTATE.EXLOCK = '0';
    end;
end;

PSTATE.EL = target_el;
PSTATE.nRW = '0';
PSTATE.SP = '1';
PSTATE.EXLOCK = '1';
else
    PSTATE.EXLOCK = '0';
end;
else
    PSTATE.EXLOCK = '0';
end;
end;

PSTATE.EL = target_el;
PSTATE.nRW = '0';
PSTATE.SP = '1';

SPSR_ELx() = spsr;
ELR_ELx() = preferred_exception_return;
PSTATE.SS = '0';
if IsFeatureImplemented(FEAT_SSBS) then PSTATE.SSBS =
PSTATE.SS = '0';
if IsFeatureImplemented(FEAT_SSBS) then PSTATE.SSBS = SCTLRL_ELx().DSSBS; end;

```

```

    if IsFeatureImplemented(FEAT_EBEP) then PSTATE.PM = '1'; end;
    if IsFeatureImplemented(FEAT_SEBEP) then
        PSTATE.PPEND = '0'; PSTATE.PM = '1'; end;
    if IsFeatureImplemented(FEAT_SEBEP) then
        PSTATE.PPEND = '0';
        ShouldSetPPEND = FALSE;
    end;
    if IsFeatureImplemented(FEAT_NMI) then
        PSTATE.ALLINT = NOT
PSTATE.ALLINT = NOT SCTLRL_ELx().SPINTMASK;
    end;
    PSTATE.[D,A,I,F] = '1111';
    if IsFeatureImplemented(FEAT_MTE) then PSTATE.TCO = '1'; end;
    PSTATE.IL = '0';
    if IsFeatureImplemented(FEAT_UAO) then PSTATE.UAO = '0'; end;
    if IsFeatureImplemented(FEAT_UINJ) then PSTATE.UINJ = '0'; end;
    if IsFeatureImplemented(FEAT_PAuth_LR) then PSTATE.PACM = '0'; end;
    if (IsFeatureImplemented(FEAT_PAN) && (PSTATE.EL == PSTATE.TCO = '1'; end;
PSTATE.IL = '0';
    if IsFeatureImplemented(FEAT_UAO) then PSTATE.UAO = '0'; end;
    if IsFeatureImplemented(FEAT_UINJ) then PSTATE.UINJ = '0'; end;
    if IsFeatureImplemented(FEAT_PAuth_LR) then PSTATE.PACM = '0'; end;
    if (IsFeatureImplemented(FEAT_PAN) && (PSTATE.EL == EL1 ||
        (PSTATE.EL == PSTATE.EL == EL2 && ELIsInHost(EL0))) &&
        SCTLRL_ELx().SPAN == '0') then
        PSTATE.PAN = '1';
    end;
    if from_32 then // Coming from AArch32
        PSTATE.IT = '00000000';
        PSTATE.T = '0'; // PSTATE.J is RES0
    end;
    if IsFeatureImplemented(FEAT_BTI) then PSTATE.BTYPE = '00'; end;
    let branch_conditional : boolean = FALSE;
    BranchTo{64} {
PSTATEVBAR_ELx.PAN = '1';
    end;
    if from_32 then // Coming from AArch32
        PSTATE.IT = '00000000';
        PSTATE.T = '0'; // PSTATE.J is RES0
    end;
    if IsFeatureImplemented(FEAT_BTI) then PSTATE.BTYPE = '00'; end;
    let branch_conditional {}{63}: boolean = FALSE;
    BranchTo{64}(target_vector, 11)::veet_offset{10:0}, BranchType EXCEPTION, branch_conditional);
    CheckExceptionCatch(TRUE); // Check for debug event on exception entry

    if sync_errors then
        SynchronizeErrors();
        iesb_req = TRUE;
        TakeUnmaskedPhysicalSErrorInterrupts(iesb_req);
    end;

    EndOfInstruction();
end;

```

Library pseudocode for aarch64/exceptions/traps/WaitForEventUntilDelay

```

// WaitForEventUntilDelay()
// =====
// Returns TRUE if WaitForEvent() returns before WFE trap delay expires, including potentially
// some IMPLEMENTATION SPECIFIC additional delay, FALSE otherwise.

impdef func WaitForEventUntilDelay(delay_enabled : boolean, delay : integer) => boolean
begin
    return FALSE;
end;

```



```

// AltDecodeBitMasks()
// =====
// Alternative but logically equivalent implementation of DecodeBitMasks() that
// uses simpler primitives to compute tmask and wmask.

func AltDecodeBitMasks{M}(immN : bit, imms : bits(6), immr : bits(6),
                        immediate : boolean) => (bits(M), bits(M))
begin
    var tmask, wmask : bits(64);
    var tmask_and, wmask_and : bits(6);
    var tmask_or, wmask_or : bits(6);
    var levels : bits(6);

    // Compute log2 of element size
    // 2^len must be in range [2, M]
    if immN::NOT(imms) == '000000x' then Undefined(); end;
    let len : integer{} = HighestSetBitNZ(immN::NOT(imms));
    assert 2 <= (2^len) && (2^len) <= M;

    // Determine s, r and s - r parameters
    levels = ZeroExtend{6}(Ones{len});

    // For logical immediates an all-ones value of s is reserved
    // since it would generate a useless all-ones result (many times)
    if immediate && (imms AND levels) == levels then Undefined(); end;

    let s : integer{} = UInt(imms AND levels);
    let r : integer{} = UInt(immr AND levels);
    let diff : integer{} = s - r;    // 6-bit subtract with borrow

    // Compute "top mask"
    tmask_and = diff[5:0] OR NOT(levels);
    tmask_or = diff[5:0] AND levels;

    tmask = Ones{64};
    tmask = ((tmask
        AND Replicate{64, 2}(Replicate{1}(tmask_and[0]) :: Ones{1}))
        OR Replicate{64, 2}(Zeros{1} :: Replicate{1}(tmask_or[0])));
    // optimization of first step:
    // tmask = Replicate{64}(tmask_and[0] :: '1');
    tmask = ((tmask
        AND Replicate{64, 4}(Replicate{2}(tmask_and[1]) :: Ones{2}))
        OR Replicate{64, 4}(Zeros{2} :: Replicate{2}(tmask_or[1])));
    tmask = ((tmask
        AND Replicate{64, 8}(Replicate{4}(tmask_and[2]) :: Ones{4}))
        OR Replicate{64, 8}(Zeros{4} :: Replicate{4}(tmask_or[2])));
    tmask = ((tmask
        AND Replicate{64, 16}(Replicate{8}(tmask_and[3]) :: Ones{8}))
        OR Replicate{64, 16}(Zeros{8} :: Replicate{8}(tmask_or[3])));
    tmask = ((tmask
        AND Replicate{64, 32}(Replicate{16}(tmask_and[4]) :: Ones{16}))
        OR Replicate{64, 32}(Zeros{16} :: Replicate{16}(tmask_or[4])));
    tmask = ((tmask
        AND Replicate{64, 64}(Replicate{32}(tmask_and[5]) :: Ones{32}))
        OR Replicate{64, 64}(Zeros{32} :: Replicate{32}(tmask_or[5])));

    // Compute "wraparound mask"
    wmask_and = immr OR NOT(levels);
    wmask_or = immr AND levels;

    wmask = Zeros{64};
    wmask = ((wmask
        AND Replicate{64, 2}(Ones{1} :: Replicate{1}(wmask_and[0]))
        OR Replicate{64, 2}(Replicate{1}(wmask_or[0]) :: Zeros{1}));
    // wmask = Replicate{64}(wmask_or[0] :: '0');
    wmask = ((wmask
        AND Replicate{64, 4}(Ones{2} :: Replicate{2}(wmask_and[1]))
        OR Replicate{64, 4}(Replicate{2}(wmask_or[1]) :: Zeros{2}));
    wmask = ((wmask
        AND Replicate{64, 8}(Ones{4} :: Replicate{4}(wmask_and[2]))
        OR Replicate{64, 8}(Replicate{4}(wmask_or[2]) :: Zeros{4}));
    wmask = ((wmask
        AND Replicate{64, 16}(Ones{8} :: Replicate{8}(wmask_and[3]))
        OR Replicate{64, 16}(Replicate{8}(wmask_or[3]) :: Zeros{8}));
    wmask = ((wmask
        AND Replicate{64, 32}(Ones{16} :: Replicate{16}(wmask_and[4]))
        OR Replicate{64, 32}(Replicate{16}(wmask_or[4]) :: Zeros{16}));
    wmask = ((wmask
        AND Replicate{64, 64}(Ones{32} :: Replicate{32}(wmask_and[5]))
        OR Replicate{64, 64}(Replicate{32}(wmask_or[5]) :: Zeros{32}));

    if diff[6] != '0' then // borrow from s - r
        wmask = wmask AND tmask;
    else
        wmask = wmask OR tmask;
    end;

    return (wmask[M-1:0], tmask[M-1:0]);
end;

```

Library pseudocode for aarch64/functions/cache/AArch64_WriteTagMem

```
// AArch64_WriteTagMem()
// =====
// Write to tag memory.

func AArch64_WriteTagMem(regval : bits(64), cachetype : bits(64),
    accdesc_in : CacheType,
    accdesc_in : AccessDescriptor, size : integer)
begin
    assert accdesc_in.tagaccess && !accdesc_in.tagchecked;

    var accdesc : AccessDescriptor = accdesc_in;
    let vaddress : bits(64) = AlignDownSize{64}(regval, size as integer{16..2048});
    let count : integer = size >> LOG2_TAG_GRANULE;
    var tag : bits(4);
    if cachetype == CacheType.TagZero then
        *
    else
        tag : bits(4) = AArch64_AllocationTagFromAddress(vaddress);
    end;
    let aligned : boolean = IsAlignedSize{64}(vaddress, TAG_GRANULE);
    assert aligned;

    accdesc.tagaccess = IsMTEEnabled(accdesc.el);

    var (memtagtype, memaddrdesc) = AArch64_TranslateTagAddress(vaddress, accdesc, aligned, size);

    // Check for aborts or debug exceptions
    if IsFault(memaddrdesc) then
        if IsDebugException(memaddrdesc.fault) then
            AArch64_Abort(memaddrdesc.fault);
        else
            memaddrdesc.fault.vaddress = regval;
            AArch64_Abort(memaddrdesc.fault);
        end;
    end;

    if !accdesc.tagaccess || memtagtype != MemTag_AllocationTagged then
        return;
    end;

    var memstatus : PhysMemRetStatus;

    for i = 0 to count-1 do
        memstatus = PhysMemTagWrite(memaddrdesc, accdesc, tag);
        if IsFault(memstatus) then
            HandleExternalWriteAbort(memstatus, memaddrdesc, 1, accdesc);
        end;

        memaddrdesc.paddress.address = memaddrdesc.paddress.address + TAG_GRANULE;
        memaddrdesc.vaddress = memaddrdesc.vaddress + TAG_GRANULE;
    end;

    return;
end;
```

Library pseudocode for aarch64/functions/dc/AArch64_MemZero

```
// AArch64_MemZero()
// =====

func AArch64_MemZero(regval : bits(64), cachetype : CacheType)
begin
    let size : integer{} = (if cachetype IN {CacheType\_TagWrite, CacheType\_TagZero}
                            then 4*(2^(UInt(DCZID_EL0().TBS)))
                            else 4*(2^(UInt(DCZID_EL0().BS))));
    assert size <= MAX\_ZERO\_BLOCK\_SIZE;
    if IsFeatureImplemented(FEAT_MTE2) then
        assert size >= TAG\_GRANULE;
    end;
    let vaddress : bits(64) = AlignDownSize{}(regval, size);

    let accdesc : AccessDescriptor = CreateAccDescDCZero(cachetype);

    if cachetype !=IN { } then
        AArch64_WriteTagMem(zeros{64}, vaddress, accdesc, size);
    end;

    if cachetype IN {CacheType\_Tag, CacheType\_TagWrite, CacheType\_Data\_TagCacheType\_DataCacheType\_TagZero } then
        AArch64_WriteTagMem(regval, cachetypevaddress, accdesc, size);
    end;

    if cachetype IN {CacheType\_Data, CacheType\_Data\_Tag} then
        AArch64_DataMemZero(regval, vaddress, accdesc, size);
    end;
    return;
end;
```



```

// AArch64_EffectiveOpScopeAArch64_IC()
// =====
// Perform Instruction Cache Operation.

func AArch64_IC(opscope : AArch64_EffectiveOpScope(opscope_in : CacheOpScope)|
begin
    let regval : bits(64) = ARBITRARY : bits(64);
    AArch64_IC(regval, opscope);
end;

// AArch64_IC()
// =====
// Perform Instruction Cache Operation.

func AArch64_IC(regval : bits(64), opscope : CacheOpScope
)
begin
    var opscopecache : CacheOpScopeCacheRecord = opscope_in;

    if opscopecache.acetype == PSTATEAccessType_IC.EL;
    cache.cachetype != EL1CacheType_Instruction then return opscope; end;

    if IsFeatureImplemented(FEAT_TLBIID) then
        var fnb : bit;
        if
            cache.cacheop == IsHCRXEL2EnabledCacheOp_Invalidate();
            cache.opscope == &&opscope;
        if (!opscope IN {HaveEL(EL3) || SCR_EL3().VTLBIDEn == '1'}) then
            fnb = HCRX_EL2().FNB;
        else
            fnb = '0';
        end;

        case fnb::HCR_EL2().FB of
            when '00' =>
                pass;
            when '01' =>
                if opscope == CacheOpScope_ALLU then
                    // The operation is broadcast within the Inner Shareable shareability domain.
                    opscope = CacheOpScope_ALLUISCacheOpScope_ALLUIS_CnP;
                end;
            when '10' =>
                if opscope == CacheOpScope_ALLUIS then
                    //
                    let This operation only affects the PE on which the instruction is executed.
                    opscope = SecurityState = SecurityStateAtEL(PSTATE.EL);
                    cache.cpas = CPASAtSecurityState(ss);

                    case opscope of
                        when CacheOpScope_ALLU;
                        end;
                    when '11' =>
                        if opscope.shareability opscope == Shareability_NSH;
                        when CacheOpScope_ALLUIS then
                            // The operation is broadcast to PEs which would share TLB entries with the
                            // current PE if CnP were 1.
                            opscope=>
                            cache.shareability = Shareability_ISH;
                            // When executed at EL1, if the Effective value of HCRX_EL2.VTLBIDEn is '1',
                            // then the operation is broadcast to the set of PEs selected by VTLBID0_EL2.TD0.
                            // Otherwise, the operation is broadcast to all PEs in the Shareability domain.
                            cache.domains = TLBIDomains(Broadcast_ISH, Zeros{16});
                        when CacheOpScope_ALLUIS CnP;=>
                    end;
                end;
            else
                if opscope.shareability opscope == CacheOpScope_ALLUShareability_ISH &&
                cache.domains = DEFAULT_TLBI_DOMAIN;
                otherwise =>
                    unreachable;
                end;
            end;

            cache.regval = regval;
            CACHE_OP(cache);
        else
            assert opscope == CacheOpScope_PoU;

            if EL2Enabled() && HCR_EL2().FB == '1' then
                opscope =! () then
                    if PSTATE.EL IN {EL0, EL1} then
                        cache.is_vmid_valid = TRUE;
                        cache.vmid = VMID();
                    else
                        cache.is_vmid_valid = FALSE;
                    end;
                else
                    cache.is_vmid_valid = FALSE;
                end;
            end;
        end;
    end;
end;

```

```

    if PSTATE.EL == ELO then
        cache.is_asid_valid = TRUE;
        cache.asid = ASID();
    else
        cache.is_asid_valid = FALSE;
    end;

    let vaddress : bits(64) = regval;
    cache.vaddress = regval;

    let accdesc : AccessDescriptor = CreateAccDescIC(cache);
    let aligned : boolean = TRUE;
    let size : integer = 0;
    var memaddrdesc : AddressDescriptor = AArch64_TranslateAddress(vaddress, accdesc,
                                                                    aligned, size);

    if IsFault(memaddrdesc) then
        memaddrdesc.fault.vaddress = regval;
        AArch64_Abort(memaddrdesc.fault);
    end;

    cache.cpas = CPASatPAS(memaddrdesc.paddress.paspace);
    cache.paddress = memaddrdesc.paddress;
    cache.shareability = memaddrdesc.memattrs.shareability;
    if memaddrdesc.memattrs.shareability == Shareability_OSH then
        cache.shareability = Shareability_ISHCacheOpScope_ALLUISIsInHost;
    end;
    CACHE_OP(cache);
end;

return opscope;
end;

```

Library pseudocode for aarch64/functions/ic/AArch64_IC

```

// AArch64_IC()
// =====
// Perform Instruction Cache Operation.

func AArch64_IC(opscope : CacheOpScope)
begin
    let regval : bits(64) = ARBITRARY : bits(64);
    AArch64_IC(regval, opscope);
end;

// AArch64_IC()
// =====
// Perform Instruction Cache Operation.

func AArch64_IC(regval : bits(64), opscope_in : CacheOpScope)
begin
    var cache : CacheRecord;
    var opscope : CacheOpScope = opscope_in;

    cache.acctype = AccessType IC;
    cache.cachetype = CacheType Instruction;
    cache.cacheop = CacheOp Invalidate;
    cache.opscope = opscope;

    if opscope IN {CacheOpScope ALLU,
                  CacheOpScope ALLUIS CnP,
                  CacheOpScope ALLUIS} then
        let ss : SecurityState = SecurityStateAtEL(PSTATE.EL);
        cache.cpas = CPASAtSecurityState(ss);
        opscope = AArch64_EffectiveOpScope(opscope);

        case opscope of
            when CacheOpScope ALLU =>
                cache.shareability = Shareability NSH;
            when CacheOpScope ALLUIS =>
                cache.shareability = Shareability ISH;
                // When executed at EL1, if the Effective value of HCRX_EL2.VTLBIDEn is '1',
                // then the operation is broadcast to the set of PEs selected by VTLBID0_EL2.TD0.
                // Otherwise, the operation is broadcast to all PEs in the Shareability domain.
                cache.domains = TLBIDomains(Broadcast ISH, Zeros{16});
            when CacheOpScope ALLUIS CnP =>
                cache.shareability = Shareability ISH;
                cache.domains = DEFAULT_TLBI_DOMAIN;
            otherwise =>
                unreachable;
        end;

        cache.regval = regval;
        CACHE_OP(cache);
    else
        assert opscope == CacheOpScope PoU;

        if EL2Enabled() && !IsInHost() then
            if PSTATE.EL IN {EL0, EL1} then
                cache.is_vmid_valid = TRUE;
                cache.vmid = VMID();
            else
                cache.is_vmid_valid = FALSE;
            end;
        else
            cache.is_vmid_valid = FALSE;
        end;

        if PSTATE.EL == EL0 then
            cache.is_asid_valid = TRUE;
            cache.asid = ASID();
        else
            cache.is_asid_valid = FALSE;
        end;

        let vaddress : bits(64) = regval;
        cache.vaddress = regval;

        let accdesc : AccessDescriptor = CreateAccDescIC(cache);
        let aligned : boolean = TRUE;
        let size : integer = 0;
        var memaddrdesc : AddressDescriptor = AArch64_TranslateAddress(vaddress, accdesc,
                                                                    aligned, size);

        if IsFault(memaddrdesc) then
            memaddrdesc.fault.vaddress = regval;
            AArch64_Abort(memaddrdesc.fault);
        end;

        cache.cpas = CPASAtPAS(memaddrdesc.paddress.paspace);
        cache.paddress = memaddrdesc.paddress;
        cache.shareability = memaddrdesc.memattrs.shareability;
        if memaddrdesc.memattrs.shareability == Shareability OSH then
            cache.shareability = Shareability ISH;
        end;
    end;
end;

```

```

end;
|
    if memaddrdesc.memattrs.shareability != Shareability NSH then
        // If the Effective value of HCRX_EL2().VTLBIDEn is '1', the invalidation is broadcast
        // to the set of PEs in the Inner shareable domain selected by VTLBID0_EL2().TD0 when
        // executed at EL1, or at EL0 when HCR_EL2().TGE is 0.
        cache.domains = TLBIDomains(Broadcast ISH, Zeros{16});
    end;
|
    CACHE_OP(cache);
end;
return;
end;

```



```

// AArch64_MemSingleRead()
// =====
// Perform an atomic, little-endian read of 'size' bits.

func AArch64_MemSingleRead(size : integer{8, 16, 32, 64, 128, 256})(address : bits(64),
                                                                    accdesc in : AccessDescriptor,
                                                                    aligned : boolean
                                                                    ) => (bits(size),
                                                                    AddressDescriptor,
                                                                    PhysMemRetStatus)

begin
    var value : bits(size) = ARBITRARY : bits(size);
    var memstatus : PhysMemRetStatus = ARBITRARY : PhysMemRetStatus;
    let bytes : integer{} = size DIV 8;
    memstatus.statuscode = Fault\_None;
    var accdesc : AccessDescriptor = accdesc_in;
    if IsFeatureImplemented(FEAT_LSE2) then
        let quantity : integer = MemSingleGranule();
        assert ((IsFeatureImplemented(FEAT_LS64WB) &&
                bytes == 32 && accdesc.acctype == AccessType\_ASIMD) ||
                AllInAlignedQuantity(address, bytes, quantity));
    else
        assert IsAlignedSize(address, bytes);
    end;

    // If the instruction encoding permits tag checking, confer with system register configuration
    // which may override this.
    if accdesc.tagchecked then
        accdesc.tagchecked = AArch64\_AccessIsTagChecked(address, accdesc);
    end;

    var memaddrdesc : AddressDescriptor;
    memaddrdesc = AArch64\_TranslateAddress(address, accdesc, aligned, bytes);

    // Check for aborts or debug exceptions
    if IsFault(memaddrdesc) then
        return (value, memaddrdesc, memstatus);
    end;

    // Memory array access
    if accdesc.tagchecked then
        let ltag : bits(4) = AArch64\_LogicalAddressTag(address);
        let fault : FaultRecord = AArch64\_CheckTag(memaddrdesc, accdesc, bytes, ltag);
        if fault.statuscode != Fault\_None then
            memaddrdesc.fault = fault;
            return (value, memaddrdesc, memstatus);
        end;
    end;

    if accdesc.acctype != AccessType\_IFETCH && SPESampleInFlight then
        let is_load : boolean = TRUE;
        SPESampleLoadStore(is_load, accdesc, memaddrdesc);
    end;

    var atomic : boolean;
    if IsWBShareable(memaddrdesc.memattrs) then
        atomic = TRUE;
    elseif accdesc.exclusive then
        atomic = TRUE;
    elseif (accdesc.acctype == AccessType\_SVE && accdesc.predicated &&
            bytes == 8 && IsAlignedSize(address, 8)) then
        // An SVE predicated load of a 128-bit element that is 64-bit aligned
        // is treated as a pair of 64-bit single-copy atomic accesses.
        // This is one of the 64-bit single-copy atomic access.
        atomic = TRUE;
    elseif aligned then
        atomic = !accdesc.ispair;
    else
        // Misaligned accesses within MemSingleGranule() byte aligned memory but
        // not Normal Cacheable Writeback are Atomic
        atomic = ImpDefBool("FEAT_LSE2: access is atomic");
    end;

    if atomic then
        (memstatus, value) = PhysMemRead{size}(memaddrdesc, accdesc);
    elseif accdesc.acctype == AccessType\_ASIMD && bytes == 32 && accdesc.ispair then
        // A 32 byte LDP (SIMD&FP) that does not target Normal Inner Write-Back, Outer
        // Write-Back cacheable, Shareable memory is treated as four 8 byte atomic accesses.
        // As this access was not split in Mem(), it must be aligned to 32 bytes.
        assert IsAlignedSize(address, 32);
        accdesc.ispair = FALSE;
        for i = 0 to 3 do
            if IsFault(memstatus) then // Do not continue past a fault
                (memstatus, value[i*64+:64]) = PhysMemRead{64}(memaddrdesc, accdesc);
            end;

            if IsFault(memstatus) then
                memaddrdesc.paddress.address = memaddrdesc.paddress.address + 8;
                memaddrdesc.vaddress = memaddrdesc.vaddress + 8;
            end;
        end;
    end;
end;

```

```

        end;
    end;

    elsif aligned && accdesc.ispair then
        let half : integer{} = (size DIV 2) as integer{32, 64, 128};
        (memstatus, value[0*:half]) = PhysMemRead{half}(memaddrdesc, accdesc);

        if !IsFault(memstatus) then
            memaddrdesc.paddress.address = memaddrdesc.paddress.address + (halfbytes DIV 82);
            memaddrdesc.vaddress = memaddrdesc.vaddress + (halfbytes DIV 82);
            (memstatus, value[1*:half]) = PhysMemRead{half}(memaddrdesc, accdesc);
        end;

    else
        for i = 0 to bytes-1 do
            if !IsFault(memstatus) then // Do not continue past a fault
                (memstatus, value[i*:8]) = PhysMemRead{8}(memaddrdesc, accdesc);
            end;

            if !IsFault(memstatus) then
                memaddrdesc.paddress.address = memaddrdesc.paddress.address + 1;
                memaddrdesc.vaddress = memaddrdesc.vaddress + 1;
            end;

        end;
    end;

    if IsFault(memstatus) then
        return (value, memaddrdesc, memstatus);
    end;

    if accdesc.acctype == AccessType IFETCH then
        memaddrdesc.fault = AArch64 CheckDebug(address, accdesc, bytes);
    end;

    return (value, memaddrdesc, memstatus);
end;

```



```

// AArch64_MemSingleWrite()
// =====
// Perform an atomic, little-endian write of 'size' bits.

func AArch64_MemSingleWrite{size : integer{8, 16, 32, 64, 128, 256}}(address : bits(64),
                                                                    accdesc_in : AccessDescriptor,
                                                                    aligned : boolean,
                                                                    value : bits(size)
                                                                    ) => (AddressDescriptor, PhysMemRetStatus)

begin
    var accdesc : AccessDescriptor = accdesc_in;
    let bytes : integer{} = size DIV 8;
    if IsFeatureImplemented(FEAT_LSE2) then
        let quantity : integer = MemSingleGranule();
        assert ((IsFeatureImplemented(FEAT_LS64WB) &&
            bytes == 32 && accdesc.acctype == AccessType\_ASIMD) ||
            AllInAlignedQuantity(address, bytes, quantity));
    else
        assert IsAlignedSize(address, bytes);
    end;

    // If the instruction encoding permits tag checking, confer with system register configuration
    // which may override this.
    if accdesc.tagchecked then
        accdesc.tagchecked = AArch64\_AccessIsTagChecked(address, accdesc);
    end;

    var memaddrdesc : AddressDescriptor;
    var memstatus : PhysMemRetStatus = ARBITRARY : PhysMemRetStatus;
    memaddrdesc = AArch64\_TranslateAddress(address, accdesc, aligned, bytes);

    // Check for aborts or debug exceptions
    if IsFault(memaddrdesc) then
        return (memaddrdesc, memstatus);
    end;

    // Effect on exclusives
    if memaddrdesc.memattrs.shareability != Shareability\_NSH then
        ClearExclusiveByAddress(memaddrdesc.paddress, ProcessorID(), bytes);
    end;

    if accdesc.tagchecked then
        let ltag : bits(4) = AArch64\_LogicalAddressTag(address);
        let fault : FaultRecord = AArch64_CheckTag(memaddrdesc, accdesc, bytes, ltag);
        if fault.statuscode != Fault\_None then
            memaddrdesc.fault = fault;
            return (memaddrdesc, memstatus);
        end;
    end;

    if SPESampleInFlight then
        let is_load : boolean = FALSE;
        SPESampleLoadStore(is_load, accdesc, memaddrdesc);
    end;

    var atomic : boolean;
    if IsWBShareable(memaddrdesc.memattrs) then
        atomic = TRUE;
    elsif accdesc.exclusive then
        atomic = TRUE;
    elsif (accdesc.acctype == AccessType\_SVE && accdesc.predicated &&
        bytes == 8 && IsAlignedSize(address, 8)) then
        // An SVE predicated load of a 128-bit element that is 64-bit aligned
        // is treated as a pair of 64-bit single-copy atomic accesses.
        // This is one of the 64-bit single-copy atomic access.
        atomic = TRUE;
    elsif aligned then
        atomic = !accdesc.ispair;
    else
        // Misaligned accesses within MemSingleGranule() byte aligned memory but
        // not Normal Cacheable Writeback are Atomic
        atomic = ImpDefBool("FEAT_LSE2: access is atomic");
    end;

    if atomic then
        memstatus = PhysMemWrite(size)(memaddrdesc, accdesc, value);
        if IsFault(memstatus) then
            return (memaddrdesc, memstatus);
        end;
    end;

    elsif accdesc.acctype == AccessType\_ASIMD && bytes == 32 && accdesc.ispair then
        // A 32 byte STP (SIMD&FP) that does not target Normal Inner Write-Back, Outer
        // Write-Back cacheable, Shareable memory is treated as four 8 byte atomic accesses.
        // As this access was not split in Mem(), it must be aligned to 32 bytes.
        assert IsAlignedSize(address, 32);
        accdesc.ispair = FALSE;
        for i = 0 to 3 do
            memstatus = PhysMemWrite{64}(memaddrdesc, accdesc, value[64*i+:64]);
            if IsFault(memstatus) then
                return (memaddrdesc, memstatus);
            end;
        end;
    end;
end

```

```

        end;
        memaddrdesc.paddress.address = memaddrdesc.paddress.address + 8;
        memaddrdesc.vaddress        = memaddrdesc.vaddress + 8;
    end;

    elsif aligned && accdesc.ispair then
        let half : integer{} = (size DIV 2) as integer{32, 64};
        memstatus = PhysMemWrite{half}(memaddrdesc, accdesc, value[0*:half]);
        if IsFault(memstatus) then
            return (memaddrdesc, memstatus);
        end;

        memaddrdesc.paddress.address = memaddrdesc.paddress.address + (halfbytes DIV 82);
        memaddrdesc.vaddress        = memaddrdesc.vaddress + (halfbytes DIV 82);
        memstatus = PhysMemWrite{half}(memaddrdesc, accdesc, value[1*:half]);
        if IsFault(memstatus) then
            return (memaddrdesc, memstatus);
        end;

    else
        for i = 0 to bytes-1 do
            memstatus = PhysMemWrite{8}(memaddrdesc, accdesc, value[i*:8]);
            if IsFault(memstatus) then
                return (memaddrdesc, memstatus);
            end;
            memaddrdesc.paddress.address = memaddrdesc.paddress.address + 1;
            memaddrdesc.vaddress        = memaddrdesc.vaddress + 1;
        end;
    end;

    return (memaddrdesc, memstatus);
end;

```

Library pseudocode for aarch64/functions/memory/AArch64_MemTagWrite

```

// AArch64_MemTagWrite()
// =====
// Store an Allocation Tag to memory.

func AArch64_MemTagWrite(address : bits(64),
                        accdesc_in : AccessDescriptor,
                        value : bits(4)) => (AddressDescriptor, PhysMemRetStatus)
begin
    assert accdesc_in.tagaccess && !accdesc_in.tagchecked;

    var accdesc : AccessDescriptor = accdesc_in;
    var memaddrdesc : AddressDescriptor = ARBITRARY : AddressDescriptor;
    var memtagtype : MemTagType = ARBITRARY : MemTagType;
    var memstatus : PhysMemRetStatus = ARBITRARY : PhysMemRetStatus;

    let aligned : boolean = IsAlignedSize(address, TAG\_GRANULE);

    // Stores of allocation tags must be aligned
    if !aligned then
        memaddrdesc.fault = AlignmentFault(accdesc, address);
        return (memaddrdesc, memstatus);
    end;

    accdesc.tagaccess = IsMTEEnabled(accdesc.el);

    (memtagtype, memaddrdesc) = AArch64\_TranslateTagAddress(address, accdesc, aligned, TAG\_GRANULE);

    // Check for aborts or debug exceptions
    if IsFault(memaddrdesc) then
        memaddrdesc.fault.vaddress = memaddrdesc.vaddress;
        return (memaddrdesc, memstatus);
    end;

    if accdesc.tagaccess && memtagtype == MemTag\_AllocationTagged then
        memstatus = PhysMemTagWrite(memaddrdesc, accdesc, value);
    end;
    return (memaddrdesc, memstatus);
end;

```



```

// Mem - accessor
// =====

accessor Mem{size : integer{8, 16, 32, 64, 128, 256}}(address : bits(64),
                                                    accdesc_in : AccessDescriptor
                                                    ) <=> value_in : bits(size)

begin
  // Perform a read of 'size' bits. The access byte order is reversed for a big-endian access.
  // Instruction fetches would call AArch64_MemSingle directly.
  getter
    var accdesc : AccessDescriptor = accdesc_in;
    let bytes : integer{} = size DIV 8;
    var value : bits(size);

    // Check alignment on size of element accessed, not overall access size
    varlet alignedalignment : boolean;
    integer{} = (if accdesc.ispair && !accdesc.exclusive then
      let halfbytes : integer{} = (size DIV 2)
      else asbytes);
    var integer{32, 64, 128};
    aligned = IsAlignedSize(address, half DIV 8);
    else
      aligned boolean = IsAlignedSize(address, bytesalignment);
    end;

    let quantity : integer = MemSingleGranule();

    if !aligned && AArch64 UnalignedAccessFaults(accdesc, address, bytes) then
      let fault : FaultRecord = AlignmentFault(accdesc, address);
      AArch64\_Abort(fault);
    end;
    if accdesc.acctype == AccessType ASIMD && bytes == 16 && IsAlignedSize(address, 8) then
      // If 128-bit SIMD&FP ordered access are treated as a pair of
      // 64-bit single-copy atomic accesses, then these single copy atomic
      // access can be observed in any order.
      let half : integer{} = (size DIV 2) as integer{64};
      let highaddress : bits(64) = AddressIncrement(address, halfbytes DIV 8, accdesc);
      value[0*:half] = AArch64 MemSingle{half}(address, accdesc, aligned);
      value[1*:half] = AArch64 MemSingle{half}(highaddress, accdesc, aligned);
    elsif (accdesc.acctype == AccessType ASIMD && bytes == 32 &&
      accdesc.ispair && IsAlignedSize(address, 32)) then
      value = AArch64 MemSingle{size}(address, accdesc, aligned);
    elsif accdesc.acctype == AccessType ASIMD && bytes == 32 && IsAlignedSize(address, 8) then
      // If a 32 byte LDP (SIMD&FP) access is not aligned to 32 bytes but aligned to
      // 8 bytes, it is treated as four 8 byte single-copy atomic accesses.
      accdesc.ispair = FALSE;
      aligned = TRUE;
      for i = 0 to 3 do
        let blockaddress : bits(64) = AddressIncrement(address, i*8, accdesc);
        value[64*i+:64] = AArch64 MemSingle{64}(blockaddress, accdesc, aligned);
      end;
    elsif (IsFeatureImplemented(FEAT_LSE2) &&
      AllInAlignedQuantity(address, bytes, quantity)) then
      value = AArch64 MemSingle{size}(address, accdesc, aligned);
    elsif ((aligned && accdesc.ispair) ||
      (accdesc.acctype == AccessType SVE && accdesc.predicated &&
        bytes == 16 && IsAlignedSize(address, 8))) then
      // Either: an aligned pair access, OR
      // an SVE predicated load of a 128-bit element that is 64-bit aligned,
      // which is treated as two 64-bit single-copy atomic accesses.
      accdesc.ispair = FALSE;
      let half : integer{} = (size DIV 2) as integer{32, 64, 128};
      let highaddress : bits(64) = AddressIncrement(address, halfbytes DIV 8, accdesc);
      if IsFeatureImplemented(FEAT_LRCP3) && accdesc.highestaddressfirst then
        value[1*:half] = AArch64 MemSingle{half}(highaddress, accdesc, aligned);
        value[0*:half] = AArch64 MemSingle{half}(address, accdesc, aligned);
      else
        value[0*:half] = AArch64 MemSingle{half}(address, accdesc, aligned);
        value[1*:half] = AArch64 MemSingle{half}(highaddress, accdesc, aligned);
      end;
    elsif aligned then
      value = AArch64 MemSingle{size}(address, accdesc, aligned);
    else
      assert bytes > 1;
      if (IsFeatureImplemented(FEAT_LRCP3) && accdesc.ispair &&
        accdesc.highestaddressfirst) then
        let half : integer{} = (size DIV 2) as integer{32, 64, 128};
        var lowhalf, highhalf : bits(half);

        for i = 0 to (halfbytes DIV 8)-1 do
          let byteaddress : bits(64) = AddressIncrement(address,
            (halfbytes DIV 8) + i, accdesc);
          // Individual byte access can be observed in any order
          highhalf[i*:8] = AArch64 MemSingle{8}(byteaddress, accdesc, aligned);
        end;
        for i = 0 to (halfbytes DIV 8)-1 do
          let byteaddress : bits(64) = AddressIncrement(address, i, accdesc);
          // Individual byte access can be observed in any order
          lowhalf[i*:8] = AArch64 MemSingle{8}(byteaddress, accdesc, aligned);
        end;
      end;
    end;
  end;
end

```



```

end;

value = highhalf::lowhalf;

else
    value[7:0] = AArch64 MemSingle{8}(address, accdesc, aligned);
    accdesc.lowestaddress = FALSE;

    // For subsequent bytes, if they cross to a new translation page which assigns
    // Device memory type, it is CONSTRAINED UNPREDICTABLE whether an unaligned access
    // will generate an Alignment Fault.
    let c : Constraint = ConstrainUnpredictable(Unpredictable_DEVPAGE2);
    assert c IN {Constraint FAULT, Constraint NONE};
    if c == Constraint NONE then aligned = TRUE; end;

    for i = 1 to bytes-1 do
        let byteaddress : bits(64) = AddressIncrement(address, i, accdesc);
        value[i*8] = AArch64 MemSingle{8}(byteaddress, accdesc, aligned);
    end;
end;

end;

if BigEndian(accdesc.acctype) then
    value = BigEndianReverse(size)(value);
end;

return value;
end;

// Perform a write of 'size' bits. The byte order is reversed for a big-endian access.
setter
    var value : bits(size) = value_in;
    let bytes : integer{} = size DIV 8;
    var accdesc : AccessDescriptor = accdesc_in;

    // Check alignment on size of element accessed, not overall access size
    var let alignedalignment : boolean;
    integer{} = (if accdesc.ispair && !accdesc.exclusive then
        let halfbytes : integer{} = (size DIV 2)
        else asbytes);
    var integer{32, 64, 128};
    aligned = IsAlignedSize(address, half DIV 8);
    else
        aligned boolean = IsAlignedSize(address, bytesalignment);
    end;
    let quantity : integer = MemSingleGranule();

    if !aligned && AArch64 UnalignedAccessFaults(accdesc, address, bytes) then
        let fault : FaultRecord = AlignmentFault(accdesc, address);
        AArch64_Abort(fault);
    end;

    if BigEndian(accdesc.acctype) then
        value = BigEndianReverse(size)(value);
    end;

    if accdesc.acctype == AccessType ASIMD && bytes == 16 && IsAlignedSize(address, 8) then
        let half : integer{} = (size DIV 2) as integer{64};
        // 128-bit SIMD&FP stores are treated as a pair of 64-bit single-copy atomic accesses
        // 64-bit aligned.
        let highaddress : bits(64) = AddressIncrement(address, half(bytes DIV 8, 2), accdesc);
        AArch64 MemSingle(half)(address, accdesc, aligned) = value[0+:half];
        AArch64 MemSingle(half)(highaddress, accdesc, aligned) = value[half+:half];
    elsif (accdesc.acctype == AccessType ASIMD && bytes == 32 &&
        accdesc.ispair && IsAlignedSize(address, 32)) then
        AArch64 MemSingle(size)(address, accdesc, aligned) = value;
    elsif accdesc.acctype == AccessType ASIMD && bytes == 32 && IsAlignedSize(address, 8) then
        // If a 32 byte STP (SIMD&FP) access is not aligned to 32 bytes but aligned to
        // 8 bytes, it is treated as four 8 byte single-copy atomic accesses.
        accdesc.ispair = FALSE;
        aligned = TRUE;
        for i = 0 to 3 do
            let blockaddress : bits(64) = AddressIncrement(address, i*8, accdesc);
            AArch64 MemSingle{64}(blockaddress, accdesc, aligned) = value[64*i+:64];
        end;
    elsif (IsFeatureImplemented(FEAT_LSE2) &&
        AllInAlignedQuantity(address, bytes, quantity)) then
        AArch64 MemSingle(size)(address, accdesc, aligned) = value;
    elsif ((aligned && accdesc.ispair) ||
        (accdesc.acctype == AccessType SVE && accdesc.predicated &&
        bytes == 16 && IsAlignedSize(address, 8))) then
        // Either: an aligned pair access, OR
        // an SVE predicated load of a 128-bit element that is 64-bit aligned,
        // which is treated as two 64-bit single-copy atomic accesses.
        let half : integer{} = (size DIV 2) as integer{32, 64, 128};
        accdesc.ispair = FALSE;
        let highaddress : bits(64) = AddressIncrement(address, half(bytes DIV 8, 2), accdesc);
        if IsFeatureImplemented(FEAT_LRCPC3) && accdesc.highestaddressfirst then
            AArch64 MemSingle(half)(highaddress, accdesc, aligned) = value[half+:half];
            AArch64 MemSingle(half)(address, accdesc, aligned) = value[0+:half];
        end;
    end;
end;

```

```

else
    AArch64 MemSingle{half}(address, accdesc, aligned) = value[0+:half];
    AArch64 MemSingle{half}(highaddress, accdesc, aligned) = value[half+:half];
end;
elsif aligned then
    AArch64 MemSingle{size}(address, accdesc, aligned) = value;
else
    assert bytes > 1;
    if (IsFeatureImplemented(FEAT_LRCPC3) && accdesc.ispair &&
        accdesc.highestaddressfirst) then
        let half : integer{} = (size DIV 2) as integer{32, 64};
        var lowhalf, highhalf : bits(half);
        (highhalf, lowhalf) = (value[half+:half], value[0+:half]);
        for i = 0 to (halfbytes DIV 8)-1 do
            let byteaddress : bits(64) = AddressIncrement(address,
                (halfbytes DIV 8) + i, accdesc);
            // Individual byte access can be observed in any order
            AArch64 MemSingle{8}(byteaddress, accdesc, aligned) = highhalf[i*:8];
        end;
        for i = 0 to (halfbytes DIV 8)-1 do
            let byteaddress : bits(64) = AddressIncrement(address,
                (halfbytes DIV 8) + i, accdesc);
            // Individual byte access can be observed in any order,
            // but implies observability of highhalf
            AArch64 MemSingle{8}(byteaddress, accdesc, aligned) = lowhalf[i*:8];
        end;
    else
        AArch64 MemSingle{8}(address, accdesc, aligned) = value[7:0];
        accdesc.lowestaddress = FALSE;

        // For subsequent bytes, if they cross to a new translation page which assigns
        // Device memory type, it is CONSTRAINED UNPREDICTABLE whether an unaligned access
        // will generate an Alignment Fault.

        let c : Constraint = ConstrainUnpredictable(Unpredictable DEVPAGE2);
        assert c IN {Constraint FAULT, Constraint NONE};
        if c == Constraint NONE then aligned = TRUE; end;

        for i = 1 to bytes-1 do
            let byteaddress : bits(64) = AddressIncrement(address, i, accdesc);
            AArch64 MemSingle{8}(byteaddress, accdesc, aligned) = value[i*:8];
        end;
    end;
end;
end;
end;
end;

```

Library pseudocode for aarch64/functions/memory/NVMem

```
// NVMem - accessor
// =====

accessor NVMem(offset : integer) <=> value : bits(64)
begin
    // This function is the load memory access for the transformed System register read access
    // when Enhanced Nested Virtualization is enabled with HCR_EL2.NV2 = 1.
    // The address for the load memory access is calculated using
    // the formula SignExtend{64}({VNCR_EL2.BADDR :: Offset[11:0]}, 64) where,
    // * VNCR_EL2.BADDR holds the base address of the memory location, and
    // * Offset is the unique offset value defined architecturally for each System register that
    // supports transformation of register access to memory access.
    getter
        assert offset > 0 && offset MOD 8 == 0;
        let directread : boolean = FALSE;
        let baddr : bits(64) = EffectiveBADDR(VNCR_EL2().BADDR :: Zeros{12}, directread);
        let address : bits(64) = baddr + offset;
        let accdesc : AccessDescriptor = CreateAccDescNV2(MemOp_LOAD);
        return Mem{64}(address, accdesc);
    end;

    // This function is the store memory access for the transformed System register write access
    // when Enhanced Nested Virtualization is enabled with HCR_EL2.NV2 = 1.
    // The address for the store memory access is calculated using
    // the formula SignExtend{64}({VNCR_EL2.BADDR :: Offset[11:0]}, 64) where,
    // * VNCR_EL2.BADDR holds the base address of the memory location, and
    // * Offset is the unique offset value defined architecturally for each System register that
    // supports transformation of register access to memory access.
    setter
        assert offset > 0 && offset MOD 8 == 0;
        let directread : boolean = FALSE;
        let baddr : bits(64) = EffectiveBADDR(VNCR_EL2().BADDR :: Zeros{12}, directread);
        let address : bits(64) = baddr + offset;
        let accdesc : AccessDescriptor = CreateAccDescNV2(MemOp_STORE);
        Mem{64}(address, accdesc) = value;
        return;
    end;
end;

accessor NVMem128(offset : integer) <=> value : bits(128)
begin
    getter
        assert offset > 0 && offset MOD 16 == 0;
        let directread : boolean = FALSE;
        let baddr : bits(64) = EffectiveBADDR(VNCR_EL2().BADDR :: Zeros{12}, directread);
        let address : bits(64) = baddr + offset;
        let accdesc : AccessDescriptor = CreateAccDescNV2(MemOp_LOAD);
        return Mem{128}(address, accdesc);
    end;

    setter
        assert offset > 0 && offset MOD 16 == 0;
        let directread : boolean = FALSE;
        let baddr : bits(64) = EffectiveBADDR(VNCR_EL2().BADDR :: Zeros{12}, directread);
        let address : bits(64) = baddr + offset;
        let accdesc : AccessDescriptor = CreateAccDescNV2(MemOp_STORE);
        Mem{128}(address, accdesc) = value;
    end;
end;
```

Library pseudocode for aarch64/functions/mops/CheckSETConstrainedUnpredictable

```
// CheckSETConstrainedUnpredictable()
// =====
// Check for CONSTRAINED UNPREDICTABLE behaviour in the SET* and SETG* instructions.

func CheckSETConstrainedUnpredictable(memsetn : SETParams, integer, d : integer, s : integer)
begin
    if (memsetn.ss == memsetn.nn || memsetn.ss == memsetn.de || memsetn.nn == memsetn.de) then
        let c : Constraint = ConstrainUnpredictable(Unpredictable MOPSOVERLAP);
        assert c IN {Constraint UNDEF, Constraint NOP};
        case c of
            when Constraint UNDEF => Undefined();
            when Constraint NOP => ExecuteAsNOP();
        end;
    end;

    if (memsetn.de == 31 || memsetn.nn == 31) then
        let c : Constraint = ConstrainUnpredictable(Unpredictable MOPS_R31);
        assert c IN {Constraint UNDEF, Constraint NOP};
        case c of
            when Constraint UNDEF => Undefined();
            when Constraint NOP => ExecuteAsNOP();
        end;
    end;
end;
```



```

// InsertPAC()
// =====
// Calculates the pointer authentication code for a 64-bit quantity and then
// inserts that into pointer authentication code field of that 64-bit quantity.

func InsertPAC(ptr : bits(64), modifier : bits(64), modifier2 : bits(64), use_modifier2 : boolean,
               K : bits(128), data : boolean) => bits(64)
begin
    var PAC : bits(64);
    var result : bits(64);
    var ext_ptr : bits(64);
    var extfield : bits(64);
    var selbit : bit;
    var bit55 : bit;
    let tbi : boolean = EffectiveTBI(ptr, !data, PSTATE.EL) == '1';
    let mtX : boolean = (IsFeatureImplemented(FEAT_MTE_NO_ADDRESS_TAGS) &&
                        EffectiveMTX(ptr, !data, PSTATE.EL) == '1');
    let top_bit : integer{} = if tbi then 55 else 63;
    let EL3_using_lva3 : boolean = (IsFeatureImplemented(FEAT_LVA3) &&
                                    TranslationRegime(PSTATE.EL) == Regime_EL3 &&
                                    AArch64_IASize(TCR_EL3().T0SZ) > 52);
    let is_VA_56bit : boolean = (TranslationRegime(PSTATE.EL) == Regime_EL3 &&
                                   AArch64_IASize(TCR_EL3().T0SZ) == 56);

    // If tagged pointers are in use for a regime with two TTBRs, use bit[55] of
    // the pointer to select between upper and lower ranges, and preserve this.
    // This handles the awkward case where there is apparently no correct choice between
    // the upper and lower address range - ie an addr of 1xxxxxxx0... with TBI0=0 and TBI1=1
    // and 0xxxxxxx1 with TBI1=0 and TBI0=1:
    if PtrHasUpperAndLowerAddRanges() then
        assert S1TranslationRegime() IN {EL1, EL2};
        if S1TranslationRegime() == EL1 then
            // EL1 translation regime registers
            if data then
                if TCR_EL1().TBI1 == '1' || TCR_EL1().TBI0 == '1' then
                    selbit = ptr[55];
                else
                    selbit = ptr[63];
                end;
            else
                if ((TCR_EL1().TBI1 == '1' && TCR_EL1().TBID1 == '0') ||
                    (TCR_EL1().TBI0 == '1' && TCR_EL1().TBID0 == '0')) then
                    selbit = ptr[55];
                else
                    selbit = ptr[63];
                end;
            end;
        else
            // EL2 translation regime registers
            if data then
                if TCR_EL2().TBI1 == '1' || TCR_EL2().TBI0 == '1' then
                    selbit = ptr[55];
                else
                    selbit = ptr[63];
                end;
            else
                if ((TCR_EL2().TBI1 == '1' && TCR_EL2().TBID1 == '0') ||
                    (TCR_EL2().TBI0 == '1' && TCR_EL2().TBID0 == '0')) then
                    selbit = ptr[55];
                else
                    selbit = ptr[63];
                end;
            end;
        end;
    else selbit = if tbi then ptr[55] else ptr[63];
    end;

    if IsFeatureImplemented(FEAT_PAuth2) && IsFeatureImplemented(FEAT_CONSTPACFIELD) then
        selbit = ptr[55];
    end;
    let bottom_PAC_bit : AddressSize = CalculateBottomPACBit(selbit);

    if EL3_using_lva3 then
        extfield = Replicate{64}('0');
    else
        extfield = Replicate{64}(selbit);
    end;

    // Compute the pointer authentication code for a ptr with good extension bits
    if tbi then
        if bottom_PAC_bit <= 55 then
            ext_ptr = (ptr[63:56] ::
                      extfield[55:bottom_PAC_bit] :: ptr[bottom_PAC_bit-1:0]);
        else
            ext_ptr = ptr[63:56] :: ptr[55:0];
        end;
    elseif mtX then
        if bottom_PAC_bit <= 55 then
            ext_ptr = (extfield[63:60] :: ptr[59:56] ::
                      ptr[55:0]);
        else
            ext_ptr = (extfield[63:60] :: ptr[59:56] ::
                      ptr[55:0]);
        end;
    end;
end;

```

```

        extfield[55:bottom_PAC_bit] :: ptr[bottom_PAC_bit-1:0]);
    else
        ext_ptr = extfield[63:60] :: ptr[59:56] :: ptr[55:0];
    end;
else
    ext_ptr = extfield[63:bottom_PAC_bit] :: ptr[bottom_PAC_bit-1:0];
end;

if use_modifier2 then
    assert IsFeatureImplemented(FEAT_PAuth_LR);
    PAC = ComputePAC2(ext_ptr, modifier, modifier2, K[127:64], K[63:0]);
else
    PAC = ComputePAC(ext_ptr, modifier, K[127:64], K[63:0]);
end;

if !IsFeatureImplemented(FEAT_PAuth2) then
    // If FEAT_PAuth2 is not implemented, the PAC is corrupted if the pointer does not have
    // a canonical VA.
    assert bottom_PAC_bit <= 52;
    if !IsZero(ptr[top_bit:bottom_PAC_bit]) && !IsOnes(ptr[top_bit:bottom_PAC_bit]) then
        if IsFeatureImplemented(FEAT_EPAC) then
            PAC = 0x0000000000000000[63:0];
        else
            PAC[top_bit-1] = NOT(PAC[top_bit-1]);
        end;
    end;
end;

// Preserve the determination between upper and lower address at bit[55] and insert PAC into
// bits that are not used for the address or the tag(s).
if !IsFeatureImplemented(FEAT_PAuth2) then
    assert (bottom_PAC_bit <= 52);
    if tbi then
        result = ptr[63:56]::selbit::PAC[54:bottom_PAC_bit]::ptr[bottom_PAC_bit-1:0];
    else
        result = PAC[63:56]::selbit::PAC[54:bottom_PAC_bit]::ptr[bottom_PAC_bit-1:0];
    end;
else
    if EL3_using_lva3 then
        // Bit 55 is an address bit (when VA size is 56-bits) or
        // used to store PAC (when VA size is less than 56-bits)
        if is_VA_56bit then
            bit55 = ptr[55];
        else
            bit55 = ptr[55] XOR PAC[55];
        end;
    else
        bit55 = selbit;
    end;
    if tbi then
        if bottom_PAC_bit < 55 then
            result = (ptr[63:56] :: bit55 ::
                (ptr[54:bottom_PAC_bit] XOR PAC[54:bottom_PAC_bit]) ::
                ptr[bottom_PAC_bit-1:0]);
        else
            result = (ptr[63:56] :: bit55 :: ptr[54:0]);
        end;
    elsif mtx then
        if bottom_PAC_bit < 55 then
            result = ((ptr[63:60] XOR PAC[63:60]) :: ptr[59:56] :: bit55 ::
                (ptr[54:bottom_PAC_bit] XOR PAC[54:bottom_PAC_bit]) ::
                ptr[bottom_PAC_bit-1:0]);
        else
            result = ((ptr[63:60] XOR PAC[63:60]) :: ptr[59:56] :: bit55 ::
                ptr[54:0]);
        end;
    else
        if bottom_PAC_bit < 55 then
            result = ((ptr[63:56] XOR PAC[63:56]) :: bit55 ::
                (ptr[54:bottom_PAC_bit] XOR PAC[54:bottom_PAC_bit]) ::
                ptr[bottom_PAC_bit-1:0]);
        else
            result = ((ptr[63:56] XOR PAC[63:56]) :: bit55 ::
                ptr[54:0]);
        end;
    end;
end;
return result;
end;

```

Library pseudocode for aarch64/functions/pac/pac/IsAPDAKeyEnabled

```
// IsAPDAKeyEnabled()
// =====
// Returns TRUE if authentication using the APDAKey_EL1 key is enabled.
// Otherwise, depending on the state of the PE, generate a trap, or return FALSE.

func IsAPDAKeyEnabled() => boolean
begin
    var TrapEL2 : boolean;
    var TrapEL3 : boolean;
    var Enable : bits(1);

    case PSTATE.EL of
        when PSTATE.EL of
            when EL0 =>
                let IsEL1Regime : boolean = S1TranslationRegime() == EL1;
                Enable = if IsEL1Regime then SCTL2_EL1().EnDA else SCTL2_EL2().EnDA;
                if IsFeatureImplemented(FEAT_PAuth_EnhCtl) then
                    var enda2 : bit;
                    if IsEL1Regime then
                        enda2 = if IsSCTL2_EL1Enabled() then SCTL2_EL1().EnDA2 else '0';
                    else
                        enda2 = if IsSCTL2_EL2Enabled() then
                            SCTL2_EL2.SCTL2_EL1().EnDA2EnDA else '0';
                        endSCTL2_EL2().EnDA;
                    if IsFeatureImplemented(FEAT_PAuth_EnhCtl) then
                        Enable = Enable XOR enda2;
                    end;
                end;
                TrapEL2 = EL2Enabled() && HCR_EL2().API == '0' && !IsInHost();
                TrapEL3 = HaveEL(EL3) && SCR_EL3().API == '0';
            when EL1 =>
                Enable = SCTL2_EL1().EnDA;
                TrapEL2 = EL2Enabled() && HCR_EL2().API == '0';
                TrapEL3 = HaveEL(EL3) && SCR_EL3().API == '0';
            when EL2 =>
                Enable = SCTL2_EL2().EnDA;
                TrapEL2 = FALSE;
                TrapEL3 = HaveEL(EL3) && SCR_EL3().API == '0';
            when EL3 =>
                Enable = SCTL2_EL3().EnDA;
                TrapEL2 = FALSE;
                TrapEL3 = FALSE;
    end;

    if Enable == '0' then
        return FALSE;
    elsif TrapEL3 && EL3SDDUndefPriority() then
        Undefined();
    elsif TrapEL2 then
        TrapPACUse(EL2);
    elsif TrapEL3 then
        if EL3SDDUndef() then
            Undefined();
        else
            TrapPACUse(EL3);
        end;
    else
        return TRUE;
    end;
end;
```


Library pseudocode for aarch64/functions/pac/pac/IsAPDBKeyEnabled

```
// IsAPDBKeyEnabled()
// =====
// Returns TRUE if authentication using the APDBKey_EL1 key is enabled.
// Otherwise, depending on the state of the PE, generate a trap, or return FALSE.

func IsAPDBKeyEnabled() => boolean
begin
    var TrapEL2 : boolean;
    var TrapEL3 : boolean;
    var Enable : bits(1);

    case PSTATE.EL of
        when PSTATE.EL of
            when EL0 =>
                let IsEL1Regime : boolean = S1TranslationRegime() == EL1;
                Enable = if IsEL1Regime then SCTL2_EL1().EnDB else SCTL2_EL2().EnDB;
                if IsFeatureImplemented(FEAT_PAuth_EnhCtl) then
                    var endb2 : bit;
                    if IsEL1Regime then
                        endb2 = if IsSCTL2_EL1Enabled() then SCTL2_EL1().EnDB2 else '0';
                    else
                        endb2 = if IsSCTL2_EL2Enabled() then
                            SCTL2_EL2().EnDB2 else '0';
                    end
                    Enable = Enable XOR endb2;
                if IsFeatureImplemented(FEAT_PAuth_EnhCtl) then
                    Enable = Enable XOR endb2;
                end;
                TrapEL2 = EL2Enabled() && HCR_EL2().API == '0' && !IsInHost();
                TrapEL3 = HaveEL(EL3) && SCR_EL3().API == '0';
            when EL1 =>
                Enable = SCTL2_EL1().EnDB;
                TrapEL2 = EL2Enabled() && HCR_EL2().API == '0';
                TrapEL3 = HaveEL(EL3) && SCR_EL3().API == '0';
            when EL2 =>
                Enable = SCTL2_EL2().EnDB;
                TrapEL2 = FALSE;
                TrapEL3 = HaveEL(EL3) && SCR_EL3().API == '0';
            when EL3 =>
                Enable = SCTL2_EL3().EnDB;
                TrapEL2 = FALSE;
                TrapEL3 = FALSE;
    end;

    if Enable == '0' then
        return FALSE;
    elsif TrapEL3 && EL3SDDUndefPriority() then
        Undefined();
    elsif TrapEL2 then
        TrapPACUse(EL2);
    elsif TrapEL3 then
        if EL3SDDUndef() then
            Undefined();
        else
            TrapPACUse(EL3);
        end;
    else
        return TRUE;
    end;
end;
```

Library pseudocode for aarch64/functions/pac/pac/IsAPIAKeyEnabled

```
// IsAPIAKeyEnabled()
// =====
// Returns TRUE if authentication using the APIAKey_EL1 key is enabled.
// Otherwise, depending on the state of the PE, generate a trap, or return FALSE.

func IsAPIAKeyEnabled() => boolean
begin
    var TrapEL2 : boolean;
    var TrapEL3 : boolean;
    var Enable : bits(1);

    case PSTATE.EL of
        when PSTATE.EL of
            when EL0 =>
                let IsEL1Regime : boolean = S1TranslationRegime() == EL1;
                Enable = if IsEL1Regime then SCTL2_EL1().EnIA else SCTL2_EL2().EnIA;
                if IsFeatureImplemented(FEAT_PAuth_EnhCtl) then
                    var enia2 : bit;
                    if IsEL1Regime then
                        enia2 = if IsSCTL2_EL1Enabled() then SCTL2_EL1().EnIA2 else '0';
                    else
                        enia2 = if IsSCTL2_EL2Enabled() then
                            SCTL2_EL2().EnIA2 else '0';
                    end
                end
                if IsFeatureImplemented(FEAT_PAuth_EnhCtl) then
                    Enable = Enable XOR enia2;
                end
            when EL1 =>
                Enable = SCTL2_EL1().EnIA;
                TrapEL2 = EL2Enabled() && HCR_EL2().API == '0' && !IsInHost();
                TrapEL3 = HaveEL(EL3) && SCR_EL3().API == '0';
            when EL2 =>
                Enable = SCTL2_EL2().EnIA;
                TrapEL2 = FALSE;
                TrapEL3 = HaveEL(EL3) && SCR_EL3().API == '0';
            when EL3 =>
                Enable = SCTL2_EL3().EnIA;
                TrapEL2 = FALSE;
                TrapEL3 = FALSE;
    end;

    if Enable == '0' then
        return FALSE;
    elsif TrapEL3 && EL3SDDUndefPriority() then
        Undefined();
    elsif TrapEL2 then
        TrapPACUse(EL2);
    elsif TrapEL3 then
        if EL3SDDUndef() then
            Undefined();
        else
            TrapPACUse(EL3);
        end;
    else
        return TRUE;
    end;
end;
```

Library pseudocode for aarch64/functions/pac/pac/IsAPIBKeyEnabled

```
// IsAPIBKeyEnabled()
// =====
// Returns TRUE if authentication using the APIBKey_EL1 key is enabled.
// Otherwise, depending on the state of the PE, generate a trap, or return FALSE.

func IsAPIBKeyEnabled() => boolean
begin
    var TrapEL2 : boolean;
    var TrapEL3 : boolean;
    var Enable : bits(1);

    case PSTATE.EL of
    when PSTATE.EL of
    when EL0 =>
        let IsEL1Regime : boolean = SiTranslationRegime() == EL1;
        Enable = if IsEL1Regime then SCTL2_EL1().EnIB else SCTL2_EL2().EnIB;
        if IsFeatureImplemented(FEAT_PAAuth_EnhCtl) then
            var enib2 : bit;
            if IsEL1Regime then
                enib2 = if IsSCTL2_EL1Enabled() then SCTL2_EL1().EnIB2 else '0';
            else
                enib2 = if IsSCTL2_EL2Enabled();
            Enable = if IsEL1Regime then SCTL2_EL2SCTL2_EL1().EnIB2EnIB else '0';
            endSCTL2_EL2().EnIB;
            if IsFeatureImplemented(FEAT_PAAuth_EnhCtl) then
                Enable = Enable XOR enib2; (if IsEL1Regime then SCTL2_EL1().EnIB2
                else SCTL2_EL2().EnIB2);
        end;
        TrapEL2 = EL2Enabled() && HCR_EL2().API == '0' && !IsInHost();
        TrapEL3 = HaveEL(EL3) && SCR_EL3().API == '0';
    when EL1 =>
        Enable = SCTL2_EL1().EnIB;
        TrapEL2 = EL2Enabled() && HCR_EL2().API == '0';
        TrapEL3 = HaveEL(EL3) && SCR_EL3().API == '0';
    when EL2 =>
        Enable = SCTL2_EL2().EnIB;
        TrapEL2 = FALSE;
        TrapEL3 = HaveEL(EL3) && SCR_EL3().API == '0';
    when EL3 =>
        Enable = SCTL2_EL3().EnIB;
        TrapEL2 = FALSE;
        TrapEL3 = FALSE;
    end;

    if Enable == '0' then
        return FALSE;
    elsif TrapEL3 && EL3SDDUndefPriority() then
        Undefined();
    elsif TrapEL2 then
        TrapPACUse(EL2);
    elsif TrapEL3 then
        if EL3SDDUndef() then
            Undefined();
        else
            TrapPACUse(EL3);
        end;
    else
        return TRUE;
    end;
end;
```

Library pseudocode for aarch64/functions/ras/AArch64_dESBOperation

```
// AArch64_dESBOperation()
// =====
// Perform the AArch64 ESB operation for a pending delegated SError exception.

func AArch64_dESBOperation()
begin
    assert (IsFeatureImplemented(FEAT_E3DSE) && !ELUsingAArch32(EL3) && PSTATE.EL != PSTATE.EL != EL3);
    // When FEAT_E3DSE is implemented, SCR_EL3.DSE might inject a delegated SError exception.
    var dsei_pending, dsei_masked : boolean;
    dsei_pending = EffectiveSCR_EL3_EnDSE();
    // ==When '1' FEAT_E3DSE && is implemented, SCR_EL3.DSE might inject a delegated SError exception.
    var dsei_pending, dsei_masked : boolean;
    dsei_pending = SCR_EL3().{EnDSE, DSE} == '11';
    (dsei_masked, -) = AArch64_DelegatedSErrorTarget();
    if dsei_pending && dsei_masked then
        var target : bits(64) = Zeros();
        target[31] = '1'; // A
        target[24:0] = VESR_EL3()[24:0];
        VDISR_EL3() = target;
        ClearPendingDelegatedSError();
    end;
    return;
end;
```

Library pseudocode for aarch64/functions/ras/EffectiveSCR_EL3_EndSE

```
// EffectiveSCR_EL3_EndSE()
// =====
// Return the Effective value of SCR_EL3().EndSE.
func EffectiveSCR_EL3_EndSE() => bit
begin
    if !IsFeatureImplemented(FEAT_E3DSE) then
        return '0';
    end;
    if SCR_EL3().EA == '0' && EffectiveSCR_EL3_TMEA() == '0' then
        return '0';
    end;
    return SCR_EL3().EndSE;
end;
```



```

// SysOp()
// =====

func SysOp(op1 : bits(3), CRn : bits(4), CRm : bits(4), op2 : bits(3)) => SystemOp
begin
  case op1::CRn::CRm::op2 of
    when '000 0111 1000 000' => return Sys AT; // S1E1R
    when '000 0111 1000 001' => return Sys AT; // S1E1W
    when '000 0111 1000 010' => return Sys AT; // S1E0R
    when '000 0111 1000 011' => return Sys AT; // S1E0W
    when '000 0111 1001 000' => return Sys AT; // S1E1RP
    when '000 0111 1001 001' => return Sys AT; // S1E1WP
    when '000 0111 1001 010' => return Sys AT; // S1E1A
    when '100 0111 1000 000' => return Sys AT; // S1E2R
    when '100 0111 1000 001' => return Sys AT; // S1E2W
    when '100 0111 1001 010' => return Sys AT; // S1E2A
    when '100 0111 1000 100' => return Sys AT; // S1E21R
    when '100 0111 1000 101' => return Sys AT; // S1E21W
    when '100 0111 1000 110' => return Sys AT; // S1E20R
    when '100 0111 1000 111' => return Sys AT; // S1E20W
    when '110 0111 1000 000' => return Sys AT; // S1E3R
    when '110 0111 1000 001' => return Sys AT; // S1E3W
    when '110 0111 1001 010' => return Sys AT; // S1E3A
    when '001 0111 0010 100' => return Sys BRB; // IALL
    when '001 0111 0010 101' => return Sys BRB; // INJ
    when '000 0111 0110 001' => return Sys DC; // IVAC
    when '000 0111 0110 010' => return Sys DC; // ISW
    when '000 0111 0110 011' => return Sys DC; // IGVAC
    when '000 0111 0110 100' => return Sys DC; // IGSW
    when '000 0111 0110 101' => return Sys DC; // IGDVAC
    when '000 0111 0110 110' => return Sys DC; // IGDW
    when '000 0111 1010 010' => return Sys DC; // CSW
    when '000 0111 1010 100' => return Sys DC; // CGSW
    when '000 0111 1010 110' => return Sys DC; // CGDSW
    when '000 0111 1110 010' => return Sys DC; // CISW
    when '000 0111 1110 100' => return Sys DC; // CIGSW
    when '000 0111 1110 110' => return Sys DC; // CIGDSW
    when '011 0111 0100 001' => return Sys DC; // ZVA
    when '011 0111 0100 011' => return Sys DC; // GVA
    when '011 0111 0100 100' => return Sys DC; // GZVA
    when '011 0111 1010 001' => return Sys DC; // CVAC
    when '011 0111 1010 011' => return Sys DC; // CGVAC
    when '011 0111 1010 101' => return Sys DC; // CGDVAC
    when '011 0111 1011 001' => return Sys DC; // CVAU
    when '011 0111 1100 001' => return Sys DC; // CVAP
    when '011 0111 1100 011' => return Sys DC; // CGVAP
    when '011 0111 1100 101' => return Sys DC; // CGDVAP
    when '011 0111 1101 001' => return Sys DC; // CVADP
    when '011 0111 1101 011' => return Sys DC; // CGVADP
    when '011 0111 1101 101' => return Sys DC; // CGDVADP
    when '011 0111 1110 001' => return Sys DC; // CIVAC
    when '011 0111 1110 011' => return Sys DC; // CIGVAC
    when '011 0111 1110 101' => return Sys DC; // CIGDVAC
    when '100 0111 00001110 100000' => return Sys MLBI; // ALLE1
    when '100 0111 0000 101' => return Sys MLBI; // VMALLE1
    when '100 0111 0000 110' => return Sys MLBI; // VPIDE1
    when '100 0111 0000 111' => return Sys MLBI; // VPMGE1
    when '100 0111 1110 000' => return Sys DC; // CIPAE
    when '100 0111 1110 111' => return Sys DC; // CIGDPAE
    when '110 0111 1110 001' => return Sys DC; // CIPAPA
    when '110 0111 1110 101' => return Sys DC; // CIGDPAPA
    when '000 0111 1111 001' => return Sys DC; // CIVAPS
    when '000 0111 1111 101' => return Sys DC; // CIGDVAPS
    when '000 0111 0001 000' => return Sys IC; // IALLUIS
    when '000 0111 0101 000' => return Sys IC; // IALLU
    when '011 0111 0101 001' => return Sys IC; // IVAU
    when '000 1000 0001 000' => return Sys TLBI; // VMALLE10S
    when '000 1000 0001 001' => return Sys TLBI; // VAE10S
    when '000 1000 0001 010' => return Sys TLBI; // ASIDE10S
    when '000 1000 0001 011' => return Sys TLBI; // VAAE10S
    when '000 1000 0001 101' => return Sys TLBI; // VALE10S
    when '000 1000 0001 111' => return Sys TLBI; // VAALE10S
    when '000 1000 0010 001' => return Sys TLBI; // RVAE1IS
    when '000 1000 0010 011' => return Sys TLBI; // RVAE1IS
    when '000 1000 0010 101' => return Sys TLBI; // RVALE1IS
    when '000 1000 0010 111' => return Sys TLBI; // RVALE1IS
    when '000 1000 0011 000' => return Sys TLBI; // VMALLE1IS
    when '000 1000 0011 001' => return Sys TLBI; // VAE1IS
    when '000 1000 0011 010' => return Sys TLBI; // ASIDE1IS
    when '000 1000 0011 011' => return Sys TLBI; // VAAE1IS
    when '000 1000 0011 101' => return Sys TLBI; // VALE1IS
    when '000 1000 0011 111' => return Sys TLBI; // VAALE1IS
    when '000 1000 0101 001' => return Sys TLBI; // RVAE10S
    when '000 1000 0101 011' => return Sys TLBI; // RVAE10S
    when '000 1000 0101 101' => return Sys TLBI; // RVALE10S
    when '000 1000 0101 111' => return Sys TLBI; // RVALE10S
    when '000 1000 0110 001' => return Sys TLBI; // RVAE1
    when '000 1000 0110 011' => return Sys TLBI; // RVAE1
    when '000 1000 0110 101' => return Sys TLBI; // RVALE1
  end case
end

```

```

when '000 1000 0110 111' => return Sys TLBI; // RVAALE1
when '000 1000 0111 000' => return Sys TLBI; // VMALLE1
when '000 1000 0111 001' => return Sys TLBI; // VAE1
when '000 1000 0111 010' => return Sys TLBI; // ASIDE1
when '000 1000 0111 011' => return Sys TLBI; // VAAE1
when '000 1000 0111 101' => return Sys TLBI; // VALE1
when '000 1000 0111 111' => return Sys TLBI; // VAALE1
when '000 1001 0001 000' => return Sys TLBI; // VMALLE1OSNXS
when '000 1001 0001 001' => return Sys TLBI; // VAE1OSNXS
when '000 1001 0001 010' => return Sys TLBI; // ASIDE1OSNXS
when '000 1001 0001 011' => return Sys TLBI; // VAAE1OSNXS
when '000 1001 0001 101' => return Sys TLBI; // VALE1OSNXS
when '000 1001 0001 111' => return Sys TLBI; // VAALE1OSNXS
when '000 1001 0010 001' => return Sys TLBI; // RVAE1ISNXS
when '000 1001 0010 011' => return Sys TLBI; // RVAAE1ISNXS
when '000 1001 0010 101' => return Sys TLBI; // RVALE1ISNXS
when '000 1001 0010 111' => return Sys TLBI; // RVAALE1ISNXS
when '000 1001 0011 000' => return Sys TLBI; // VMALLE1ISNXS
when '000 1001 0011 001' => return Sys TLBI; // VAE1ISNXS
when '000 1001 0011 010' => return Sys TLBI; // ASIDE1ISNXS
when '000 1001 0011 011' => return Sys TLBI; // VAAE1ISNXS
when '000 1001 0011 101' => return Sys TLBI; // VALE1ISNXS
when '000 1001 0011 111' => return Sys TLBI; // VAALE1ISNXS
when '000 1001 0101 001' => return Sys TLBI; // RVAE1OSNXS
when '000 1001 0101 011' => return Sys TLBI; // RVAAE1OSNXS
when '000 1001 0101 101' => return Sys TLBI; // RVALE1OSNXS
when '000 1001 0101 111' => return Sys TLBI; // RVAALE1OSNXS
when '000 1001 0110 001' => return Sys TLBI; // RVAE1NXS
when '000 1001 0110 011' => return Sys TLBI; // RVAAE1NXS
when '000 1001 0110 101' => return Sys TLBI; // RVALE1NXS
when '000 1001 0110 111' => return Sys TLBI; // RVAALE1NXS
when '000 1001 0111 000' => return Sys TLBI; // VMALLE1NXS
when '000 1001 0111 001' => return Sys TLBI; // VAE1NXS
when '000 1001 0111 010' => return Sys TLBI; // ASIDE1NXS
when '000 1001 0111 011' => return Sys TLBI; // VAAE1NXS
when '000 1001 0111 101' => return Sys TLBI; // VALE1NXS
when '000 1001 0111 111' => return Sys TLBI; // VAALE1NXS
when '100 1000 0000 001' => return Sys TLBI; // IPAS2E1IS
when '100 1000 0000 010' => return Sys TLBI; // RIPAS2E1IS
when '100 1000 0000 101' => return Sys TLBI; // IPAS2LE1IS
when '100 1000 0000 110' => return Sys TLBI; // RIPAS2LE1IS
when '100 1000 0001 000' => return Sys TLBI; // ALLE2OS
when '100 1000 0001 001' => return Sys TLBI; // VAE2OS
when '100 1000 0001 100' => return Sys TLBI; // ALLE1OS
when '100 1000 0001 101' => return Sys TLBI; // VALE2OS
when '100 1000 0001 110' => return Sys TLBI; // VMALLS12E1OS
when '100 1000 0010 001' => return Sys TLBI; // RVAE2IS
when '100 1000 0010 101' => return Sys TLBI; // RVALE2IS
when '100 1000 0011 000' => return Sys TLBI; // ALLE2IS
when '100 1000 0011 001' => return Sys TLBI; // VAE2IS
when '100 1000 0011 100' => return Sys TLBI; // ALLE1IS
when '100 1000 0011 101' => return Sys TLBI; // VALE2IS
when '100 1000 0011 110' => return Sys TLBI; // VMALLS12E1IS
when '100 1000 0100 000' => return Sys TLBI; // IPAS2E1OS
when '100 1000 0100 001' => return Sys TLBI; // IPAS2E1
when '100 1000 0100 010' => return Sys TLBI; // RIPAS2E1
when '100 1000 0100 011' => return Sys TLBI; // RIPAS2E1OS
when '100 1000 0100 100' => return Sys TLBI; // IPAS2LE1OS
when '100 1000 0100 101' => return Sys TLBI; // IPAS2LE1
when '100 1000 0100 110' => return Sys TLBI; // RIPAS2LE1
when '100 1000 0100 111' => return Sys TLBI; // RIPAS2LE1OS
when '100 1000 0101 001' => return Sys TLBI; // RVAE2OS
when '100 1000 0101 101' => return Sys TLBI; // RVALE2OS
when '100 1000 0110 001' => return Sys TLBI; // RVAE2
when '100 1000 0110 101' => return Sys TLBI; // RVALE2
when '100 1000 0111 000' => return Sys TLBI; // ALLE2
when '100 1000 0111 001' => return Sys TLBI; // VAE2
when '100 1000 0111 100' => return Sys TLBI; // ALLE1
when '100 1000 0111 101' => return Sys TLBI; // VALE2
when '100 1000 0111 110' => return Sys TLBI; // VMALLS12E1
when '100 1001 0000 001' => return Sys TLBI; // IPAS2E1ISNXS
when '100 1001 0000 010' => return Sys TLBI; // RIPAS2E1ISNXS
when '100 1001 0000 101' => return Sys TLBI; // IPAS2LE1ISNXS
when '100 1001 0000 110' => return Sys TLBI; // RIPAS2LE1ISNXS
when '100 1001 0001 000' => return Sys TLBI; // ALLE2OSNXS
when '100 1001 0001 001' => return Sys TLBI; // VAE2OSNXS
when '100 1001 0001 100' => return Sys TLBI; // ALLE1OSNXS
when '100 1001 0001 101' => return Sys TLBI; // VALE2OSNXS
when '100 1001 0001 110' => return Sys TLBI; // VMALLS12E1OSNXS
when '100 1001 0010 001' => return Sys TLBI; // RVAE2ISNXS
when '100 1001 0010 101' => return Sys TLBI; // RVALE2ISNXS
when '100 1001 0011 000' => return Sys TLBI; // ALLE2ISNXS
when '100 1001 0011 001' => return Sys TLBI; // VAE2ISNXS
when '100 1001 0011 100' => return Sys TLBI; // ALLE1ISNXS
when '100 1001 0011 101' => return Sys TLBI; // VALE2ISNXS
when '100 1001 0011 110' => return Sys TLBI; // VMALLS12E1ISNXS
when '100 1001 0100 000' => return Sys TLBI; // IPAS2E1OSNXS
when '100 1001 0100 001' => return Sys TLBI; // IPAS2E1NXS
when '100 1001 0100 010' => return Sys TLBI; // RIPAS2E1NXS

```

```

when '100 1001 0100 011' => return Sys TLBI; // RIPAS2E1OSNXS
when '100 1001 0100 100' => return Sys TLBI; // IPAS2LE1OSNXS
when '100 1001 0100 101' => return Sys TLBI; // IPAS2LE1NXS
when '100 1001 0100 110' => return Sys TLBI; // RIPAS2LE1NXS
when '100 1001 0100 111' => return Sys TLBI; // RIPAS2LE1OSNXS
when '100 1001 0101 001' => return Sys TLBI; // RVAE2OSNXS
when '100 1001 0101 101' => return Sys TLBI; // RVALE2OSNXS
when '100 1001 0110 001' => return Sys TLBI; // RVAE2NXS
when '100 1001 0110 101' => return Sys TLBI; // RVALE2NXS
when '100 1001 0111 000' => return Sys TLBI; // ALLE2NXS
when '100 1001 0111 001' => return Sys TLBI; // VAE2NXS
when '100 1001 0111 100' => return Sys TLBI; // ALLE1NXS
when '100 1001 0111 101' => return Sys TLBI; // VALE2NXS
when '100 1001 0111 110' => return Sys TLBI; // VMALLS12E1NXS
when '110 1000 0001 000' => return Sys TLBI; // ALLE3OS
when '110 1000 0001 001' => return Sys TLBI; // VAE3OS
when '110 1000 0001 100' => return Sys TLBI; // PAALLOS
when '110 1000 0001 101' => return Sys TLBI; // VALE3OS
when '110 1000 0010 001' => return Sys TLBI; // RVAE3IS
when '110 1000 0010 101' => return Sys TLBI; // RVALE3IS
when '110 1000 0011 000' => return Sys TLBI; // ALLE3IS
when '110 1000 0011 001' => return Sys TLBI; // VAE3IS
when '110 1000 0011 101' => return Sys TLBI; // VALE3IS
when '110 1000 0100 011' => return Sys TLBI; // RPAOS
when '110 1000 0100 111' => return Sys TLBI; // RPALOS
when '110 1000 0101 001' => return Sys TLBI; // RVAE3OS
when '110 1000 0101 101' => return Sys TLBI; // RVALE3OS
when '110 1000 0110 001' => return Sys TLBI; // RVAE3
when '110 1000 0110 101' => return Sys TLBI; // RVALE3
when '110 1000 0111 000' => return Sys TLBI; // ALLE3
when '110 1000 0111 001' => return Sys TLBI; // VAE3
when '110 1000 0111 100' => return Sys TLBI; // PAALL
when '110 1000 0111 101' => return Sys TLBI; // VALE3
when '110 1001 0001 000' => return Sys TLBI; // ALLE3OSNXS
when '110 1001 0001 001' => return Sys TLBI; // VAE3OSNXS
when '110 1001 0001 101' => return Sys TLBI; // VALE3OSNXS
when '110 1001 0010 001' => return Sys TLBI; // RVAE3ISNXS
when '110 1001 0010 101' => return Sys TLBI; // RVALE3ISNXS
when '110 1001 0011 000' => return Sys TLBI; // ALLE3ISNXS
when '110 1001 0011 001' => return Sys TLBI; // VAE3ISNXS
when '110 1001 0011 101' => return Sys TLBI; // VALE3ISNXS
when '110 1001 0101 001' => return Sys TLBI; // RVAE3OSNXS
when '110 1001 0101 101' => return Sys TLBI; // RVALE3OSNXS
when '110 1001 0110 001' => return Sys TLBI; // RVAE3NXS
when '110 1001 0110 101' => return Sys TLBI; // RVALE3NXS
when '110 1001 0111 000' => return Sys TLBI; // ALLE3NXS
when '110 1001 0111 001' => return Sys TLBI; // VAE3NXS
when '110 1001 0111 101' => return Sys TLBI; // VALE3NXS
when '000 1100 0000 000' => return Sys GSB; // SYS
when '000 1100 0000 001' => return Sys GSB; // ACK
when '000 1100 0001 000' => return Sys GIC; // CDDIS
when '000 1100 0001 001' => return Sys GIC; // CDEN
when '000 1100 0001 010' => return Sys GIC; // CDPRI
when '000 1100 0001 011' => return Sys GIC; // CDAFF
when '000 1100 0001 100' => return Sys GIC; // CDPEND
when '000 1100 0001 101' => return Sys GIC; // CDRCFG
when '000 1100 0001 111' => return Sys GIC; // CDEOI
when '000 1100 0010 000' => return Sys GIC; // CDDI
when '000 1100 0010 001' => return Sys GIC; // CDHM
when '100 1100 0001 000' => return Sys GIC; // VDDIS
when '100 1100 0001 001' => return Sys GIC; // VDEN
when '100 1100 0001 010' => return Sys GIC; // VDPRI
when '100 1100 0001 011' => return Sys GIC; // VDAFF
when '100 1100 0001 100' => return Sys GIC; // VDPEND
when '100 1100 0001 101' => return Sys GIC; // VDRCFG
when '100 1100 0010 000' => return Sys GIC; // VDDI
when '100 1100 0010 001' => return Sys GIC; // VDHM
when '110 1100 0001 000' => return Sys GIC; // LDDIS
when '110 1100 0001 001' => return Sys GIC; // LDEN
when '110 1100 0001 010' => return Sys GIC; // LDPRI
when '110 1100 0001 011' => return Sys GIC; // LDAFF
when '110 1100 0001 100' => return Sys GIC; // LDPEND
when '110 1100 0001 101' => return Sys GIC; // LDRCFG
when '110 1100 0010 000' => return Sys GIC; // LDDI
when '110 1100 0010 001' => return Sys GIC; // LDHM
otherwise => return Sys SYS;
end;
end;

```


Library pseudocode for aarch64/functions/sysop/SystemOp

```
// SystemOp
// =====
// System instruction types.

type SystemOp of enumeration {
    Sys_AT,
    Sys_BRB,
    Sys_DC,
    Sys_GIC,
    Sys_GSB,
    Sys_IC,
    Sys_MLBI, Sys_TLBI,
    Sys_TLBI, Sys_SYS
};
```

Library pseudocode for aarch64/functions/system/AArch64_InterruptPending

```
// AArch64_InterruptPending()
// =====
// Returns TRUE if there are any pending physical, virtual, or delegated
// interrupts, and FALSE otherwise.

func AArch64_InterruptPending() => boolean
begin
    let (irq_pending, -) = IRQPending();
    let (fiq_pending, -) = FIQPending();
    let pending_physical_interrupt : boolean = (irq_pending || fiq_pending ||
                                                IsPhysicalErrorPending());

    var pending_virtual_interrupt : boolean = FALSE;
    if EL2Enabled() && PSTATE.EL IN { PSTATE.EL IN {EL0, EL1} && HCR_EL2().TGE == '0' then
        let virq_pending : boolean = (HCR_EL2().IMO == '1' && (VirtualIRQPending() ||
                                                                HCR_EL2().VI == '1'));
        let vfiq_pending : boolean = (HCR_EL2().FMO == '1' && (VirtualFIQPending() ||
                                                                HCR_EL2().VF == '1'));
        let vsei_pending : boolean = ((HCR_EL2().AMO == '1' ||
                                        (IsFeatureImplemented(FEAT_DoubleFault2) &&
                                         IsHCRXEL2Enabled() && HCRX_EL2().TMEA == '1')) &&
                                        (IsVirtualSErrorPending() || HCR_EL2().VSE == '1'));

        pending_virtual_interrupt = vsei_pending || virq_pending || vfiq_pending;
    end;

    let pending_delegated_interrupt : boolean = (IsFeatureImplemented(FEAT_E3DSE) &&
                                                PSTATE.EL != PSTATE.EL != EL3 &&
                                                EffectiveSCR_EL3_EndSE() == '1' &&
                                                SCR_EL3().{EndSE, DSE} == '11');

    return pending_physical_interrupt || pending_virtual_interrupt || pending_delegated_interrupt;
end;
```

Library pseudocode for aarch64/functions/system/BTypeCompatible_PAC

```
// BTypeCompatible_PAC()
// =====
// Returns TRUE if PACIxSP or PACIxSPPC instructions are implicitly compatible with PSTATE.BTYPE,
// FALSE otherwise.

func BTypeCompatible_PAC(pacinst : PACInstType) => boolean
begin
    // For only PACIxSP instructions, check if BTI behavior should be disabled
    if IsFeatureImplemented(FEAT_PAuth_EnhCtl) && pacinst == PACIxSP then
        var btd : bit;
        case PSTATE.EL of
            when PSTATE.EL of
            when EL0 =>
btd = if ELIsInHost(EL0) then
            btd = SCTLR2_EL2().BTD0; if else SCTLR2_EL1().BTD0;
            when IsSCTLR2EL2Enabled() then SCTLR2_EL2().BTD0 else '0';
            else
                btd = if IsSCTLR2EL1Enabled() then SCTLR2_EL1().BTD0 else '0';
            end;
            when EL1 => btd = if SCTLR2_EL1().BTD;
            when IsSCTLR2EL1Enabled() then SCTLR2_EL1().BTD else '0';
            when EL2 => btd = if SCTLR2_EL2().BTD;
            when IsSCTLR2EL2Enabled() then SCTLR2_EL2().BTD else '0';
            when EL3 => btd = SCTLR2_EL3().BTD;
            end;
            // When BTI compatibility is disabled,
            // these instructions are not compatible when PSTATE.BTYPE is non-zero
            if btd == '1' then return IsZero(PSTATE.BTYPE); end;
        end;

        if PSTATE.BTYPE != '11' then
            return TRUE;
        else
            let index : integer = if PSTATE.EL => btd = SCTLR2_EL3().BTD;
            end;
            // When BTI compatibility is disabled,
            // these instructions are not compatible when PSTATE.BTYPE is non-zero
            if btd == '1' then return IsZero(PSTATE.BTYPE); end;
        end;

        if PSTATE.BTYPE != '11' then
            return TRUE;
        else
            let index : integer = if PSTATE.EL == EL0 then 35 else 36;
            return SCTLR_ELx()[index] == '0';
        end;
    end;
end;
```

Library pseudocode for aarch64/functions/system/IsHCRXEL2Enabled

```
// IsHCRXEL2Enabled()
// =====
// Returns TRUE if access to HCRX_EL2 register is enabled, and FALSE otherwise.
// Indirect read of HCRX_EL2 returns 0 when access is not enabled.

readonly func IsHCRXEL2Enabled() => boolean
begin
    if !IsFeatureImplemented(FEAT_HCX) then return FALSE; end;
    if ELUsingAArch32(EL2) then return FALSE; end;
    if HaveEL(EL3) && SCR_EL3().HXEn == '0' then
        return FALSE;
    end;

    return EL2Enabled();
end;
```

Library pseudocode for aarch64/functions/system/IsSystemRegisterMaskingEnabled

```
// IsSystemRegisterMaskingEnabled()
// =====
// Returns TRUE if access to the MASK registers introduced by FEAT_SRMASK is enabled and the
// values in the register are not treated as 0.

func IsSystemRegisterMaskingEnabled(el : bits(2)) => boolean
begin
    assert IsFeatureImplemented(FEAT_SRMASK) && el IN {EL1, EL2};

    if HaveEL(EL3) && SCR_EL3().SRMASKE == '0' then
        return FALSE;
    end;

    return el == EL2 || !EL2Enabled() || (IsHCRXEL2Enabled() && HCRX_EL2().SRMASKE == '1');
end;
```



```

// AArch64_EffectiveBroadcast AArch64_TLBIP_IPAS2()
// =====
// Invalidate by IPA all stage 2 only TLB entries in the indicated broadcast
// domain matching the indicated VMID in the indicated regime with the indicated security state.
// Note: stage 1 and stage 2 combined entries are not in the scope of this operation.
// IPA and related parameters of the are derived from Xt.

func AArch64_TLBIP_IPAS2(security : AArch64_EffectiveBroadcast(broadcast_in, regime : BroadcastRegime), =>vmid :
bits(16),
broadcast_in : Broadcast
begin
    var broadcast_level : TLBILevel, attr : TLBIMemAttr,
        Xt : bits(128))
begin
    assert PSTATE.EL IN {EL3, EL2};
    var broadcast : Broadcast = broadcast_in;

    if var r : PSTATE.TLBIRecord.EL != ,
        r.op = EL1TLBIOp_IPAS2 then,
        r.from_aarch64 return broadcast TRUE;
        r.security = endsecurity;

        r.regime = regime;
        r.vmid = vmid;
        r.use_vmid = TRUE;
        r.level = level;
        r.attr = attr;
        r.ttl = Xt[47:44];
        r.address = ZeroExtend{64}(Xt[107:64] :: Zeros{12});
        if IsFeatureImplemented(FEAT_TLBID) && Xt[32] == '1' then
            var r.d64 = fnbTRUE;
            r.d128 = :r.ttl bit == '00xx';
        else
            r.d64 = r.ttl == '00xx';
            r.d128 = TRUE;
        end;

        case security of
        if when IsHCRXEL2EnabledSS_NonSecure() =>
            r.ipaspace && (!HaveELPAS_NonSecure(,
                when EL3SS_Secure) || SCR_EL3().VTLBIDEn == '1') then =>
            fnb r.ipaspace = HCRX_EL2().FNB;
            else
                fnb if = Xt[63] '0';
            end;

            case == fnb::HCR_EL2().FB of
            when '001' =>
                pass;
            when '01' =>
                if broadcast == then Broadcast NSHPAS_NonSecure then
                    // The operation is broadcast within the Inner Shareable shareability domain.
                    broadcast = else Broadcast ForcedISHPAS_Secure;
                end;
            when '10' =>
                if broadcast == Broadcast ISHSS_Realm then
                    // The operation only affects the PE on which the instruction is executed.
                    broadcast =>
                    r.ipaspace = Broadcast NSHPAS_Realm;

                    otherwise =>
                        // Root security state does not have stage 2 translation
                        unreachable;
                end;

                when

                TLBI(r);
                let '11' domains =>
                    if broadcast bits(16) == Broadcast ISHTLBIDomains then
                        // The operation is broadcast to PEs which would share TLB entries with the
                        // current PE Xt[15:0]);
                    if CnP were 1.
                        broadcast == Broadcast ISH CnP Broadcast_OSH;
                    end;
                end;
            else
                if broadcast == &&
                    ( Broadcast NSHIsBroadcast_OSHnISH && (regime) || EL2EnabledOSHDomainExceedsNIS() (domains))) && HCR_EL2().FB
                    == '1' then
                        broadcast = ;
                    end;

                    if broadcast != Broadcast NSHBroadcast_ForcedISHHBroadcast_OSHnISH;
                    then Broadcast TLBI(broadcast, r, domains); end;
                end;

                return broadcast;
            end;
end;

```

Library pseudocode for aarch64/functions/tlbi/~~AArch64_TLBIP_IPAS2~~~~AArch64_TLBIP_RIPAS2~~

```
// AArch64_TLBIP_IPAS2AArch64_TLBIP_RIPAS2()
// =====
// InvalidateRange invalidate by IPA all stage 2 only TLB entries in the indicated broadcast
// broadcast domain matching the indicated VMID in the indicated regime with the indicated
// security state.
// Note: stage 1 and stage 2 combined entries are not in the scope of this operation.
// The range of IPA and related parameters of the are derived from Xt.

func AArch64_TLBIP_IPAS2AArch64_TLBIP_RIPAS2(security : SecurityState, regime : Regime, vmid : bits(16),
    broadcast_in : Broadcast, level : TLBILevel, attr : TLBIMemAttr,
    Xt : bits(128))

begin
    assert PSTATE.EL IN { PSTATE.EL IN {EL3, EL2}};
    var broadcast : EL1;
    var broadcast : Broadcast = broadcast_in;

    var r : TLBIRecord;
    r.op = TLBIOp_IPAS2TLBIOp_RIPAS2;
    r.from_aarch64 = TRUE;
    r.security = security;
    r.regime = regime;
    r.vmid = vmid;
    r.use_vmid = TRUE;
    r.level = level;
    r.attr = attr;
    r.ttl = Xt[47:44];
    r.address = 0 = ZeroExtend{64}(Xt[107:64] :: Zeros{12});37;
    if IsFeatureImplemented(FEAT_TLBID) && Xt[32] == '1' then
        r.d64 = TRUE;
        r.d128 = r.ttl[1:0] == '00xx00';
    else
        r.d64 = r.ttl[1:0] == '00xx00';
        r.d128 = TRUE;
    end;

    var valid : boolean;
    (valid, r.tg, r.address, r.end_address) = TLBIPRange(regime, Xt);

    if !valid then return; end;

    case security of
        when SS_NonSecure =>
            r.ipaspace = PAS_NonSecure;
        when SS_Secure =>
            r.ipaspace = if Xt[63] == '1' then PAS_NonSecure else PAS_Secure;
        when SS_Realm =>
            r.ipaspace = PAS_Realm;
        otherwise =>
            // Root security state does not have stage 2 translation
            unreachable;
    end;

    TLBI(r);
    let domains : bits(16) = TLBIDomains(broadcast, Xt[15:0]);
    if (broadcast == Broadcast_OSH &&
        (IsBroadcast_OSHnISH()(regime) || OSHDomainExceedsNIS(domains))) then
        broadcast = Broadcast_OSHnISH;
    end;

    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r, domains); end;

    return;
end;
```

Library pseudocode for aarch64/functions/tlbi/AArch64_TLBIP_RIPAS2AArch64_TLBIP_RVA

```
// AArch64_TLBIP_RIPAS2AArch64_TLBIP_RVA()
// =====
// Range invalidate by IPA VA range all stage 2; only TLB entries in the indicated
// broadcast domain matching the indicated VMID and ASID (where regime
// supports VMID, ASID) in the indicated regime with the indicated
// security state.
// ASID, and range related parameters are derived from Xt.
// Note: stage 1 and stage 2 combined entries are not in the scope of this operation.
// The range of IPA and related parameters of the are derived from Xt.

func AArch64_TLBIP_RIPAS2AArch64_TLBIP_RVA(security : SecurityState, regime : Regime, vmid : bits(16),
      broadcast_in : Broadcast, level : TLBILevel, attr : TLBIMemAttr,
      Xt : bits(128))
begin
    assert PSTATE.EL IN { PSTATE.EL IN {EL3, EL2}};
    var broadcast : Broadcast;
    var broadcast : Broadcast = broadcast_in;

    var r : TLBIRecord;
    r.op = TLBIOp_RIPAS2TLBIOp_RVA;
    r.from_aarch64 = TRUE;
    r.security = security;
    r.regime = regime;
    r.vmid = vmid;
    r.use_vmid = TRUE;
    r.level = level;
    r.attr = attr;
    r.ttl[1:0] = Xt[38:37];
    if IsFeatureImplemented(FEAT_TLBIID) && Xt[32] == '1' then
        r.d64 = TRUE;
        r.d128 = r.ttl[1:0] == '00';
    else
        r.d64 = r.ttl[1:0] == '00';
        r.d128 = TRUE;
    end;

    var valid : boolean;
    (valid, r.tg, r.address, r.end_address) = TLBIPRange(regime, Xt);

    if !valid then return; end;

    case security of
        when SS_NonSecureUseVMID =>
            r.ipaspace = PAS_NonSecure;
        when SS_Secure =>
            r.ipaspace = if Xt[63] == '1' then PAS_NonSecure else PAS_Secure;
        when SS_Realm =>
            r.ipaspace = PAS_Realm(regime);
    end;
    r.level = level;
    r.attr = attr;
    r.asid = Xt[63:48];
    r.ttl[1:0] = Xt[38:37];
    if IsFeatureImplemented(FEAT_TLBIID) && Xt[32] == '1' then
        r.d64 = TRUE;
        r.d128 = Root(r.ttl[1:0], security, state'00';
    else
        r.d64 = does(r.ttl[1:0], not== have'00';
        r.d128 = stageTRUE;
    end;

    var 2valid translation
    unreachable: boolean;
    (valid, r.tg, r.address, r.end_address) = TLBIPRange(regime, Xt);

    if !valid then return; end;

    TLBI(r);
    let domains : bits(16) = TLBIDomains(broadcast, Xt[15:0]);
    if (broadcast == Broadcast_OSH &&
        (IsBroadcast_OSHnISH() (regime) || OSHDomainExceedsNIS(domains))) then
        broadcast = Broadcast_OSHnISH;
    end;

    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r, domains); end;

    return;
end;
```

Library pseudocode for aarch64/functions/tlbi/AArch64_TLBIP_RVA

```
// AArch64_TLBIP_RVA
// =====
// Range invalidate by VA range all stage 1 TLB entries in the indicated
// broadcast domain matching the indicated VMID and ASID (where regime supports VMID)
// supports VMID, all ASID, in the indicated regime with the indicated security state.
// ASID, VA and range related parameters are derived from Xt.
// Note: stage 1 and stage 2 combined entries are in the scope of this operation.

func AArch64_TLBIP_RVA(security : SecurityState, regime_in : Regime, vmid_in : bits(16),
    broadcast_in : Broadcast, level : TLBIlevel, attr_in : TLBIMemAttr,
    Xt : bits(128))
begin
    assert PSTATE.EL IN { EL3, EL2, EL1 };
    var broadcast : Broadcast = broadcast_in;

    var r : AArch64_EffectiveBroadcast(broadcast_in);
    var attr : TLBIMemAttr = attr_in;

    if attr == TLBI_AllAttr && ExcludeXS() then
        attr = TLBI_ExcludeXS;
    end;

    var regime : Regime = regime_in;
    var vmid : bits(16) = vmid_in;

    if ELIsInHost(EL0) then
        regime = Regime_EL20;
        vmid = VMID_NONE;
    elsif regime == Regime_EL2 && ELIsInHost(EL2) then
        regime = Regime_EL20;
    end;

    var r : TLBIRecord;
    r.op = TLBIOp_RVA;
    r.from_aarch64 = TRUE;
    r.security = security;
    r.regime = regime;
    r.vmid = vmid;
    r.use_vmid = UseVMID(regime);
    r.level = level;
    r.attr = attr;
    r.asid = Xt[63:48];
    r.ttl[1:0] = Xt[38:37];
    if IsFeatureImplemented(FEAT_TLBID) && Xt[32] == '1' then
        r.d64 = TRUE;
        r.d128 = r.ttl[1:0] == '00';
    else
        r.d64 = r.ttl[1:0] == '00';
        r.d128 = TRUE;
    end;

    var valid : boolean;
    (valid, r.tg, r.address, r.end_address) = TLBIPRange(regime, Xt);

    if !valid then return; end;

    TLBI(r);
    let domains : bits(16) = TLBIDomains(broadcast, Xt[15:0]);
    if (broadcast == Broadcast_OSH &&
        (IsBroadcast_OSHnISH(regime) || OSHDomainExceedsNIS(domains))) then
        broadcast = Broadcast_OSHnISH;
    end;

    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r, domains); end;
    return;
end;
```

Library pseudocode for aarch64/functions/tlbi/AArch64_TLBIP_RVAAArch64_TLBIP_VA

```
// AArch64_TLBIP_RVAAArch64_TLBIP_VA()
// =====
// RangeInvalidate invalidate by VA range all stage 1 TLB entries in the indicated
// broadcast domain
// matching the indicated VMID and ASID (where regimesupportsregime supports VMID)
// and all ASID in the indicated regime
// with the indicated security state.
// ASID, VA rangeand related parameters are derived from Xt.
// Note: stage 1 and stage 2 combined entries are in the scope of this operation.

func AArch64_TLBIP_RVAAArch64_TLBIP_VA(security : SecurityState, regime_inregime : Regime, vmid_invmid : bits(16),
    broadcast_in : Broadcast, level : TLBIlevel, attr_inattr : TLBIMemAttr,
    Xt : bits(128))
begin
    assert PSTATE.EL IN { PSTATE.EL IN {EL3, EL2, EL1}};
    var broadcast : Broadcast = broadcast_in;

    var r : AArch64_EffectiveBroadcast(broadcast_in);
    var attr : TLBIMemAttr = attr_in;

    if attr == TLBI_AllAttr && ExcludeXS() then
        attr = TLBI_ExcludeXS;
    end;

    var regime : Regime = regime_in;
    var vmid : bits(16) = vmid_in;

    if ELIsInHost(EL0) then
        regime = Regime_EL20;
        vmid = VMID_NONE;
    end;

    var r : TLBIRecord;
    r.op = TLBIOp_RVAAArch64_TLBIP_VA;
    r.from_aarch64 = TRUE;
    r.security = security;
    r.regime = regime;
    r.vmid = vmid;
    r.use_vmid = UseVMID(regime);
    r.level = level;
    r.attr = attr;
    r.ttlr.asid = Xt[163:0] 48;
    r.ttl = Xt[3847:3744];
    r.address = ZeroExtend(64)(Xt[107:64] :: Zeros(12));
    if IsFeatureImplemented(FEAT_TLBID) && Xt[32] == '1' then
        r.d64 = TRUE;
        r.d128 = r.ttl[1:0] == '0000xx';
    else
        r.d64 = r.ttl[1:0] == '0000xx';
        r.d128 = TRUE;
    end;

    var valid : boolean;
    (valid, r.tg, r.address, r.end_address) = TLBIPRange(regime, Xt);

    if !valid then return; end;

    TLBI(r);
    let domains : bits(16) = TLBIDomains(broadcast, Xt[15:0]);
    if (broadcast == Broadcast_OSH &&
        (IsBroadcast_OSHnISH() || OSHDomainExceedsNIS(domains))) regime then
        broadcast = Broadcast_OSHnISH;
    end;

    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r, domains); end;

    return;
end;
```


Library pseudocode for aarch64/functions/tlbi/AArch64_TLBIP_VAAArch64_TLBIP_VAA

```
// AArch64_TLBIP_VAAArch64_TLBIP_VAA()
// =====
// Invalidate by VA all stage 1 TLB entries in the indicated broadcast domain
// matching the indicated VMID and ASID (where regime supports VMID,) and all ASID in the indicated regime
// with the indicated security state.
// ASID, VA and related parameters are derived from Xt.
// Note: stage 1 and stage 2 combined entries are in the scope of this operation.

func AArch64_TLBIP_VAAArch64_TLBIP_VAA(security : SecurityState, regime_in : Regime, vmid_in : bits(16),
    broadcast_in : Broadcast, level : TLBILevel, attr_in : TLBIMemAttr,
    Xt : bits(128))

begin
    assert PSTATE.EL IN { EL3, EL2, EL1 };
    var broadcast : Broadcast = broadcast_in;

    var r : AArch64_EffectiveBroadcast(broadcast_in);
    var attr : TLBIMemAttr = attr_in;

    if attr == TLBI_AllAttr && ExcludeXS() then
        attr = TLBI_ExcludeXS;
    end;

    var regime : Regime = regime_in;
    var vmid : bits(16) = vmid_in;

    if regime == Regime_EL10 && ELIsInHost(EL0) then
        regime = Regime_EL20;
        vmid = VMID_NONE;
    elsif regime == Regime_EL2 && ELIsInHost(EL2) then
        regime = Regime_EL20;
    end;

    var r : TLBIRecord;
    r.op = TLBIOp_VATLBIP_VAA;
    r.from_aarch64 = TRUE;
    r.security = security;
    r.regime = regime;
    r.vmid = vmid;
    r.use_vmid = UseVMID(regime);
    r.level = level;
    r.attr = attr;
    r.asid = Xt[63:48];
    r.ttl = Xt[47:44];
    r.address = ZeroExtend{64}(Xt[107:64] :: Zeros{12});
    if IsFeatureImplemented(FEAT_TLBID) && Xt[32] == '1' then
        r.d64 = TRUE;
        r.d128 = r.ttl == '00xx';
    else
        r.d64 = r.ttl == '00xx';
        r.d128 = TRUE;
    end;

    TLBI(r);
    let domains : bits(16) = TLBIDomains(broadcast, Xt[15:0]);
    if (broadcast == Broadcast_OSH &&
        (IsBroadcast_OSHnISH() || OSHDomainExceedsNIS(domains))) regime then
        broadcast = Broadcast_OSHnISH;
    end;

    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r, domains); end;

    return;
end;
```

Library pseudocode for aarch64/functions/tlbi/AArch64_TLBIP_VAAArch64_TLBI_ALL

```
// AArch64_TLBIP_VAAArch64_TLBI_ALL()
// =====
// Invalidate by VA all stage 1 TLB entries infor the indicated broadcasttranslation domainregime with the
// matching the indicated VMIDsecurity (wherestate regimefor supports VMID) and all ASIDTLBs inwithin the indicated
// regime
//broadcast with the indicated security state.domain.
// VAInvalidation andapplies relatedto parametersall areapplicable derived from Xt.
// Note: stage 1 and stage 2 combined entries are in the scope of this operation.entries.

func AArch64_TLBIP_VAAArch64_TLBI_ALL(security : SecurityState, regime_inregime : Regime, vmid_in : bits(16),
broadcast_in : Broadcast, levelattr : TLBILevel, attr_in : TLBIMemAttr,
Xt : bits(12864))
begin
    assert PSTATE.EL IN { PSTATE.EL IN {EL3, EL2, 1, 0}
    var broadcast : EL1;
    var broadcast : Broadcast = broadcast_in;
    var r : AArch64_EffectiveBroadcast(broadcast_in);
    var attr : TLBIMemAttr = attr_in;
    if attr == TLBI_AllAttr && ExcludeXS() then
        attr = TLBI_ExcludeXS;
    end;
    var regime : Regime = regime_in;
    var vmid : bits(16) = vmid_in;
    if ELIsInHost(EL0) then
        regime = Regime_EL20;
        vmid = VMID_NONE;
    end;
    var r : TLBIRecord;
    r.op = TLBIOp_VAATLBIOp_ALL;
    r.from_aarch64 = TRUE;
    r.security = security;
    r.regime = regime;
    r.vmid = vmid;
    r.use_vmid r.level = UseVMIDTLBIlevel_Any(regime);
    r.level = level;
    r.attr = attr;
    r.ttl = Xt[47:44];
    r.address = ZeroExtend{64}(Xt[107:64] :: Zeros{12});
    if IsFeatureImplemented(FEAT_TLBID) && Xt[32] == '1' then
        r.d64 = TRUE;
        r.d128 = r.ttl == '00xx';
    else
        r.d64 = r.ttl == '00xx';
        r.d128 = TRUE;
    end;
    TLBI(r);
    let domains : bits(16) = TLBIDomains(broadcast, Xt[15:0]);
    if (broadcast == Broadcast_OSH &&
        (IsBroadcast_OSHnISH() (regime) || OSHDomainExceedsNIS(domains))) then
        broadcast = Broadcast_OSHnISH;
    end;
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r, domains); end;
    return;
end;
```

Library pseudocode for aarch64/functions/tlbi/AArch64_TLBI_ALLAArch64_TLBI_ASID

```
// AArch64_TLBI_ALLAArch64_TLBI_ASID()
// =====
// Invalidate all stage 1 entries for matching the indicated VMID (where regime supports)
// and ASID in the parameter Xt in the indicated translation regime with the
// the indicated security state for all TLBs within the indicated broadcast domain.
// InvalidationNote: applies to all applicable stage 1 and stage 2 entries. combined entries are in the scope of this
// operation.

func AArch64_TLBI_ALLAArch64_TLBI_ASID(security : SecurityState, regime : Regime,
vmid : bits(16),
broadcast_in : Broadcast, attr : TLBIMemAttr, Xt : bits(64))
begin
    assert PSTATE.EL IN { PSTATE.EL IN {EL3, EL2}};
    var broadcast : EL1;
    var broadcast : Broadcast = broadcast_in;

    var r : TLBIRecord;
    r.op = ;
    r.from_aarch64 = TRUE;
    r.security = security;
    r.regime = regime;
    r.vmid = vmid;
    r.use_vmid = UseVMIDTLBIOp ALLTLBIOp_ASID;
    r.from_aarch64 = TRUE;
    r.security = security;
    r.regime = regime;
    r.level = TLBIlevel Any;
    r.attr = attr;
    r.asid = Xt[63:48];

    TLBI(r);
    let domains : bits(16) = TLBIDomains(broadcast, Xt[15:0]);
    if (broadcast == Broadcast_OSH &&
        (IsBroadcast_OSHnISH() (regime) || OSHDomainExceedsNIS(domains))) then
        broadcast = Broadcast_OSHnISH;
    end;

    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r, domains); end;
    return;
end;
```

Library pseudocode for aarch64/functions/tlbi/AArch64_TLBI_ASIDAArch64_TLBI_IPAS2

```
// AArch64_TLBI_ASIDAArch64_TLBI_IPAS2()
// =====
// Invalidate by IPA all stage 12 only TLB entries matching in the indicated VMID (where regime supports) broadcast
// and domain ASID matching in the parameter indicated XtVMID in the indicated translation regime with the
// indicated security state for all TLBs within the indicated broadcast domain.state.
// Note: stage 1 and stage 2 combined entries are not in the scope of this operation.
// IPA and related parameters of the are derived from Xt.

func AArch64_TLBI_ASIDAArch64_TLBI_IPAS2(security : SecurityState, regime_inregime : Regime, vmid_invmid : bits(16),
    broadcast_in : Broadcast, attr_inlevel : TLBIlevel, attr : TLBIMemAttr,
    Xt : bits(64))

begin
    assert PSTATE.EL IN { PSTATE.EL IN {EL3, EL2,},
    var broadcast : EL1};
    var broadcast : Broadcast = broadcast_in;

    var r : AArch64_EffectiveBroadcastTLBIRecord(broadcast_in);
    var attr : r.op = TLBIMemAttrTLBIOp_IPAS2;
    r.from_aarch64 = attr_inTRUE;

    if
    r.security = security;
    r.regime = regime;
    r.vmid = vmid;
    r.use_vmid = TRUE;
    r.level = level;
    r.attr = attr;
    r.ttl = Xt[47:44];
    r.address = ZeroExtend{64}(Xt[39:0] :: Zeros{12});
    r.d64 = TRUE;
    r.d128 = r.ttl == '00xx';

    case security of
    when TLBI_AllAttrSS_NonSecure &&=>
        r.ipaspace = ExcludeXSPAS_NonSecure() then;
    attr =when TLBI_ExcludeXSS_Secure;
    end;

    var=>
        r.ipaspace regime = :if Xt[63] == '1' then RegimePAS_NonSecure = regime_in;
    var vmid : bits(16) = vmid_in;

    ifelse ELIsInHostPAS_Secure(
    when EL0SS_Realm) then
        regime=>
            r.ipaspace = Regime_EL20PAS_Realm;
            vmid =otherwise =>
                // Root security state does not have stage 2 translation
                unreachable;
    end;

    TLBI(r);
    if broadcast == VMID_NONE;
    end;

    var r : TLBIRecord;
    r.op = TLBIOp_ASID;
    r.from_aarch64 = TRUE;
    r.security = security;
    r.regime = regime;
    r.vmid = vmid;
    r.use_vmid = UseVMID(regime);
    r.level = TLBIlevel_Any;
    r.attr = attr;
    r.asid = Xt[63:48];

    TLBI(r);
    let domains : bits(16) = TLBIDomains(broadcast, Xt[15:0]);
    if (broadcast == Broadcast_OSH &&
        (IsBroadcast_OSHnISH() || OSHDomainExceedsNIS(domains))) regime) then
        broadcast = Broadcast_OSHnISH;
    end;

    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r, domains); end;

    return;
end;
```

Library pseudocode for aarch64/functions/tlbi/AArch64_TLBI_IPAS2

```

// AArch64_TLBI_IPAS2()
// =====
// Invalidate by IPA all stage 2 only TLB entries in the indicated broadcast
// domain matching the indicated VMID in the indicated regime with the indicated security state.
// Note: stage 1 and stage 2 combined entries are not in the scope of this operation.
// IPA and related parameters of the are derived from Xt.

func AArch64_TLBI_IPAS2(security : SecurityState, regime : Regime, vmid : bits(16),
                        broadcast_in : Broadcast, level : TLBILevel, attr : TLBIMemAttr,
                        Xt : bits(64))
begin
    assert PSTATE.EL IN {EL3, EL2};
    var broadcast : Broadcast = broadcast_in;

    var r : TLBIRecord;
    r.op = TLBIOp_IPAS2;
    r.from_aarch64 = TRUE;
    r.security = security;
    r.regime = regime;
    r.vmid = vmid;
    r.use_vmid = TRUE;
    r.level = level;
    r.attr = attr;
    r.ttl = Xt[47:44];
    r.address = ZeroExtend{64}(Xt[39:0] :: Zeros{12});
    r.d64 = TRUE;
    r.d128 = r.ttl == '00xx';

    case security of
        when SS_NonSecure =>
            r.ipaspace = PAS_NonSecure;
        when SS_Secure =>
            r.ipaspace = if Xt[63] == '1' then PAS_NonSecure else PAS_Secure;
        when SS_Realm =>
            r.ipaspace = PAS_Realm;
        otherwise =>
            // Root security state does not have stage 2 translation
            unreachable;
    end;

    TLBI(r);
    if broadcast == Broadcast_OSH && IsBroadcast_OSHnISH() then
        broadcast = Broadcast_OSHnISH;
    end;
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;

    return;
end;

```

Library pseudocode for aarch64/functions/tlbi/AArch64_TLBI_RIPAS2

```
// AArch64_TLBI_RIPAS2()
// =====
// Range invalidate by IPA all stage 2 only TLB entries in the indicated
// broadcast domain matching the indicated VMID in the indicated regime with the indicated
// security state.
// Note: stage 1 and stage 2 combined entries are not in the scope of this operation.
// The range of IPA and related parameters of the are derived from Xt.

func AArch64_TLBI_RIPAS2(security : SecurityState, regime : Regime, vmid : bits(16),
                        broadcast_in : Broadcast, level : TLBILevel, attr : TLBIMemAttr,
                        Xt : bits(64))

begin
    assert PSTATE.EL IN { PSTATE.EL IN {EL3, EL2, EL1};
    var broadcast : Broadcast = broadcast_in;

    var r : TLBIRecord;
    r.op = TLBIOp\_RIPAS2;
    r.from_aarch64 = TRUE;
    r.security = security;
    r.regime = regime;
    r.vmid = vmid;
    r.use_vmid = TRUE;
    r.level = level;
    r.attr = attr;
    r.ttl[1:0] = Xt[38:37];
    r.d64 = TRUE;
    r.d128 = r.ttl[1:0] == '00';

    var valid : boolean;
    (valid, r.tg, r.address, r.end_address) = TLBIRange(regime, Xt);

    if !valid then return; end;

    case security of
        when SS\_NonSecure =>
            r.ipaspace = PAS\_NonSecure;
        when SS\_Secure =>
            r.ipaspace = if Xt[63] == '1' then PAS\_NonSecure else PAS\_Secure;
        when SS\_Realm =>
            r.ipaspace = PAS\_Realm;
        otherwise =>
            // Root security state does not have stage 2 translation
            unreachable;
    end;

    TLBI(r);
    if broadcast == Broadcast\_OSH && IsBroadcast\_OSHnISH(regime) then
        broadcast = Broadcast\_OSHnISH;
    end;
    if broadcast != Broadcast\_NSH then BroadcastTLBI(broadcast, r); end;
    return;
end;
```

Library pseudocode for aarch64/functions/tlbi/AArch64_TLBI_RPA

```
// AArch64_TLBI_RPA()
// =====
// TLB Range Invalidate GPT Information by PA.
// Invalidates cached copies of GPT entries from TLBs in the indicated
// Shareability domain.
// The invalidation applies to TLB entries containing GPT information relating
// to the indicated physical address range.
// When the indicated level is
//     TLBILevel_Any : this applies to TLB entries containing GPT information
//                     from all levels of the GPT walk
//     TLBILevel_Last : this applies to TLB entries containing GPT information
//                     from the last level of the GPT walk

func AArch64_TLBI_RPA(level : TLBILevel, Xt : bits(64), broadcast : Broadcast)
begin
    assert IsFeatureImplemented(FEAT_RME) && PSTATE.EL == EL3;

    var r : TLBIRecord;
    var range_bits : AddressSize;
    var p : AddressSize;

    // r.security and r.regime do not apply for TLBI by PA operations
    r.op = TLBIOp_RPA;
    r.level = level;
    r.attr = TLBI_AllAttr;

    // SIZE field
    case Xt[47:44] of
        when '0000' => range_bits = 12; // 4KB
        when '0001' => range_bits = 14; // 16KB
        when '0010' => range_bits = 16; // 64KB
        when '0011' => range_bits = 21; // 2MB
        when '0100' => range_bits = 25; // 32MB
        when '0101' => range_bits = 29; // 512MB
        when '0110' => range_bits = 30; // 1GB
        when '0111' => range_bits = 34; // 16GB
        when '1000' => range_bits = 36; // 64GB
        when '1001' => range_bits = 39; // 512GB
        otherwise => return; // Reserved encoding, no TLB entries are required to be invalidated
    end;

    // If SIZE selects a range smaller than PGS, then PGS is used instead
    case DecodePGS(GPCCR_EL3().PGS) of
        when PGS_4KB => p = 12;
        when PGS_16KB => p = 14;
        when PGS_64KB => p = 16;
        otherwise => return; // Reserved encoding, no TLB entries are required to be invalidated
    end;

    if range_bits < p then
        range_bits = p;
    end;

    var BaseADDR : bits(5652) = Zeros{};
    case GPCCR_EL3().PGS of
        when '00' => BaseADDR[51:12] = Xt[39:0]; // 4KB
        when '10' => BaseADDR[51:14] = Xt[39:2]; // 16KB
        when '01' => BaseADDR[51:16] = Xt[39:4]; // 64KB
    end;

    if IsFeatureImplemented(FEAT_D128) && AArch64_PAMax() == 56 then
        BaseADDR[55:52] = Xt[43:40]; // Extension to Address
    end;

    // The calculation here automatically aligns BaseADDR to the size of
    // the region specified in SIZE. However, the architecture does not
    // require this alignment and if BaseADDR is not aligned to the region
    // specified by SIZE then no entries are required to be invalidated.
    let range_pbits : integer{} = range_bits;
    let start_addr : bits(5652) = BaseADDR AND NOT ZeroExtend(5652)(Ones{range_pbits});
    let end_addr : bits(5652) = start_addr + ZeroExtend(5652)(Ones{range_pbits});

    // PASpace is not considered in TLBI by PA operations
    r.address = ZeroExtend(64)(start_addr);
    r.end_address = ZeroExtend(64)(end_addr);

    TLBI(r);
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;
    return;
end;
```

Library pseudocode for aarch64/functions/tlbi/AArch64_TLBI_RVA

```
// AArch64_TLBI_RVA()
// =====
// Range invalidate by VA range all stage 1 TLB entries in the indicated
// broadcast domain matching the indicated VMID and ASID (where regime
// supports VMID, ASID) in the indicated regime with the indicated security state.
// ASID, and range related parameters are derived from Xt.
// Note: stage 1 and stage 2 combined entries are in the scope of this operation.

func AArch64_TLBI_RVA(security : SecurityState, regime_in : Regime, vmid_in : bits(16),
    broadcast_in : Broadcast, level : TLBILevel, attr_in : TLBIMemAttr,
    Xt : bits(64))
begin
    assert PSTATE.EL IN { EL3, EL2, EL1 };
    var broadcast : Broadcast = AArch64_EffectiveBroadcast(broadcast_in);
    var attr : TLBIMemAttr = attr_in;

    if attr == TLBI_AllAttr && ExcludeXS() then
        attr = TLBI_ExcludeXS;
    end;

    var regime : Regime = regime_in;
    var vmid : bits(16) = vmid_in;

    if regime == Regime_EL10 && ELIsInHost(EL0) then
        regime = Regime_EL20;
        vmid = VMID_NONE;
    elsif regime == Regime_EL2 && ELIsInHost(EL2) then
        regime = Regime_EL20;
    end;

    var r : TLBIRecord;
    r.op = TLBIOp_RVA;
    r.from_aarch64 = TRUE;
    r.security = security;
    r.regime = regime;
    r.vmid = vmid;
    r.use_vmid = UseVMID(regime);
    r.level = level;
    r.attr = attr;
    r.asid = Xt[63:48];
    r.ttl[1:0] = Xt[38:37];
    r.d64 = TRUE;
    r.d128 = r.ttl[1:0] == '00';

    var valid : boolean;
    (valid, r.tg, r.address, r.end_address) = TLBIRange(regime, Xt);

    if !valid then return; end;

    TLBI(r);
    if broadcast == Broadcast_OSH && IsBroadcast_OSHnISH() then
        broadcast = Broadcast_OSHnISH;
    end;
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;
    return;
end;
```


Library pseudocode for aarch64/functions/tlbi/AArch64_TLBI_RVAA

```
// AArch64_TLBI_RVAA()
// =====
// Range invalidate by VA range all stage 1 TLB entries in the indicated
// broadcast domain matching the indicated VMID (where regimesupports VMID)
// and all ASID in the indicated regime with the indicated security state.
// VA range related parameters are derived from Xt.
// Note: stage 1 and stage 2 combined entries are in the scope of this operation.

func AArch64_TLBI_RVAA(security : SecurityState, regime_inregime : Regime, vmid_invmid : bits(16),
    broadcast_inbroadcast : Broadcast, level : TLBILevel, attr_inattr : TLBIMemAttr,
    Xt : bits(64))
begin
    assert PSTATE.EL IN { PSTATE.EL IN {EL3, EL2, EL1};
    var broadcast : Broadcast = AArch64_EffectiveBroadcast(broadcast_in);
    var attr : TLBIMemAttr = attr_in;

    if attr == TLBI_AllAttr && ExcludeXS() then
        attr = TLBI_ExcludeXS;
    end;

    var regime : Regime = regime_in;
    var vmid : bits(16) = vmid_in;

    if ELIsInHost(EL0) then
        regime = Regime_EL20;
        vmid = VMID_NONE;
    end;

    var r : TLBIRecord;
    r.op = TLBIOp_RVAA;
    r.from_aarch64 = TRUE;
    r.security = security;
    r.regime = regime;
    r.vmid = vmid;
    r.use_vmid = UseVMID(regime);
    r.level = level;
    r.attr = attr;
    r.ttl[1:0] = Xt[38:37];
    r.d64 = TRUE;
    r.d128 = r.ttl[1:0] == '00';

    var valid : boolean;
    (valid, r.tg, r.address, r.end_address) = TLBIRange(regime, Xt);

    if !valid then return; end;

    TLBI(r);
    if broadcast == Broadcast_OSH && IsBroadcast_OSHnISH() then
        broadcast = Broadcast_OSHnISH(regime, Xt);
    end;

    if !valid then return;
    end;

    TLBI(r);
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;
    return;
end;
```

Library pseudocode for aarch64/functions/tlbi/AArch64_TLBI_VA

```
// AArch64_TLBI_VA()
// =====
// Invalidate by VA all stage 1 TLB entries in the indicated broadcast domain
// matching the indicated VMID and ASID (where regime supports VMID, ASID) in the indicated regime
// with the indicated security state.
// ASID, VA and related parameters are derived from Xt.
// Note: stage 1 and stage 2 combined entries are in the scope of this operation.

func AArch64_TLBI_VA(security : SecurityState, regime_in : Regime, vmid_in : bits(16),
    broadcast_in : Broadcast, level : TLBILevel, attr_in : TLBIMemAttr,
    Xt : bits(64))
begin
    assert PSTATE.EL IN { EL3, EL2, EL1 };
    var broadcast : Broadcast = AArch64_EffectiveBroadcast(broadcast_in);
    var attr : TLBIMemAttr = attr_in;

    if attr == TLBI_AllAttr && ExcludeXS() then
        attr = TLBI_ExcludeXS;
    end;

    var regime : Regime = regime_in;
    var vmid : bits(16) = vmid_in;
    if regime == Regime_EL10 && ELIsInHost(EL0) then
        regime = Regime_EL20;
        vmid = VMID_NONE;
    elsif regime == Regime_EL2 && ELIsInHost(EL2) then
        regime = Regime_EL20;
    end;

    var r : TLBIRecord;
    r.op = TLBIOp_VA;
    r.from_aarch64 = TRUE;
    r.security = security;
    r.regime = regime;
    r.vmid = vmid;
    r.use_vmid = UseVMID(regime);
    r.level = level;
    r.attr = attr;
    r.asid = Xt[63:48];
    r.ttl = Xt[47:44];
    r.address = ZeroExtend{64}(Xt[43:0] :: Zeros{12});
    r.d64 = TRUE;
    r.d128 = r.ttl == '00xx';

    TLBI(r);
    if broadcast == Broadcast_OSH && IsBroadcast_OSHnISH(regime) then
        broadcast = Broadcast_OSHnISH;
    end;
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;

    return;
end;
```

Library pseudocode for aarch64/functions/tlbi/AArch64_TLBI_VAA

```
// AArch64_TLBI_VAA()
// =====
// Invalidate by VA all stage 1 TLB entries in the indicated broadcast domain
// matching the indicated VMID (where regime supports VMID) and all ASID in the indicated regime
// with the indicated security state.
// VA and related parameters are derived from Xt.
// Note: stage 1 and stage 2 combined entries are in the scope of this operation.

func AArch64_TLBI_VAA(security : SecurityState, regime_in : Regime, vmid_in : bits(16),
    broadcast_in : Broadcast, level : TLBILevel, attr_in : TLBIMemAttr,
    Xt : bits(64))
begin
    assert PSTATE.EL IN { EL3, EL2, EL1 };
    var broadcast : Broadcast = AArch64_EffectiveBroadcast(broadcast_in);
    var attr : TLBIMemAttr = attr_in;

    if attr == TLBI_AllAttr && ExcludeXS() then
        attr = TLBI_ExcludeXS;
    end;

    var regime : Regime = regime_in;
    var vmid : bits(16) = vmid_in;
    if ELIsInHost(EL0) then
        regime = Regime_EL20;
        vmid = VMID_NONE;
    end;

    var r : TLBIRecord;
    r.op = TLBIOp_VAA;
    r.from_aarch64 = TRUE;
    r.security = security;
    r.regime = regime;
    r.vmid = vmid;
    r.use_vmid = UseVMID(regime);
    r.level = level;
    r.attr = attr;
    r.ttl = Xt[47:44];
    r.address = ZeroExtend{64}(Xt[43:0] :: Zeros{12});
    r.d64 = TRUE;
    r.d128 = r.ttl == '00xx';

    TLBI(r);
    if broadcast == Broadcast_OSH && IsBroadcast_OSHnISH(regime) then
        broadcast = Broadcast_OSHnISH;
    end;
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;

    return;
end;
```

Library pseudocode for aarch64/functions/tlbi/AArch64_TLBI_VMALL

```
// AArch64_TLBI_VMALL()
// =====
// Invalidate all stage 1 entries for the indicated translation regime with the
// the indicated security state for all TLBs within the indicated broadcast
// domain that match the indicated VMID (where applicable).
// Note: stage 1 and stage 2 combined entries are in the scope of this operation.
// Note: stage 2 only entries are not in the scope of this operation.

func AArch64_TLBI_VMALL(security : SecurityState, regime_in : Regime,
                        vmid_in : bits(16), broadcast_in : Broadcast,
                        attr_in : TLBIMemAttr, Xt : bits(64))

begin
    assert PSTATE.EL IN { EL3, EL2, EL1 };
    var broadcast : Broadcast = AArch64_EffectiveBroadcast(broadcast_in);
    var attr : TLBIMemAttr = attr_in;

    if attr == TLBI_AllAttr && ExcludeXS() then
        attr = TLBI_ExcludeXS;
    end;

    var regime : Regime = regime_in;
    var vmid : bits(16) = vmid_in;

    if ELIsInHost(EL0) then
        regime = Regime_EL20;
        vmid = VMID_NONE;
    end;

    var r : TLBIRecord;
    r.op = TLBIOp_VMALL;
    r.from_aarch64 = TRUE;
    r.security = security;
    r.regime = regime;
    r.level = TLBIlevel_Any;
    r.vmid = vmid;
    r.use_vmid = UseVMID(regime);
    r.attr = attr;

    TLBI(r);
    let domains : bits(16) = TLBIDomains(broadcast, Xt[15:0]);
    if (broadcast == Broadcast_OSH &&
        (IsBroadcast_OSHnISH(0)(regime) || OSHDomainExceedsNIS(domains))) then
        broadcast = Broadcast_OSHnISH;
    end;

    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r, domains); end;

    return;
end;
```

Library pseudocode for aarch64/functions/tlbi/AArch64_TLBI_VMALLS12

```
// AArch64_TLBI_VMALLS12()
// =====
// Invalidate all stage 1 and stage 2 entries for the indicated translation
// regime with the indicated security state for all TLBs within the indicated
// broadcast domain that match the indicated VMID.

func AArch64_TLBI_VMALLS12(security : SecurityState, regime : Regime,
                           vmid : bits(16), broadcast_in : Broadcast,
                           attr : TLBIMemAttr, Xt : bits(64))

begin
    assert PSTATE.EL IN { EL3, EL2 };
    var broadcast : Broadcast = AArch64_EffectiveBroadcast(broadcast_in);

    var r : TLBIRecord;
    r.op = TLBIOp_VMALLS12;
    r.from_aarch64 = TRUE;
    r.security = security;
    r.regime = regime;
    r.level = TLBIlevel_Any;
    r.vmid = vmid;
    r.use_vmid = TRUE;
    r.attr = attr;

    TLBI(r);
    let domains : bits(16) = TLBIDomains(broadcast, Xt[15:0]);
    if (broadcast == Broadcast_OSH &&
        (IsBroadcast_OSHnISH(0)(regime) || OSHDomainExceedsNIS(domains))) then
        broadcast = Broadcast_OSHnISH;
    end;

    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r, domains); end;

    return;
end;
```

Library pseudocode for aarch64/functions/tlbi/AArch64_TLBI_VMALLWS2

```
// AArch64_TLBI_VMALLWS2()
// =====
// Remove stage 2 dirty state from entries for the indicated translation regime
// with the indicated security state for all TLBs within the indicated broadcast
// domain that match the indicated VMID.

func AArch64_TLBI_VMALLWS2(security : SecurityState, regime : Regime, vmid : bits(16),
                           broadcast_in : Broadcast, attr : TLBIMemAttr,
                           Xt : bits(64))

begin
  assert PSTATE.EL IN { PSTATE.EL IN {EL3, EL2}};
  assert regime == Regime_EL10;

  if security == SS_Secure && HaveEL(EL3) && SCR_EL3().EEL2 == '0' then
    return;
  end;

  var broadcast : Broadcast = broadcast_in;

  var r : TLBIRecord;
  r.op = TLBIOp_VMALLWS2;
  r.from_aarch64 = TRUE;
  r.security = security;
  r.regime = regime;
  r.level = TLBILevel_Any;
  r.vmid = vmid;
  r.use_vmid = TRUE;
  r.attr = attr;

  TLBI(r);
  let domains : bits(16) = TLBIDomains(broadcast, Xt[15:0]);
  if (broadcast == Broadcast_OSH &&
      (IsBroadcast_OSHnISH(0) {regime} || OSHDomainExceedsNIS(domains))) then
    broadcast = Broadcast_OSHnISH;
  end;

  if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r, domains); end;

  return;
end;
```

Library pseudocode for aarch64/functions/tlbi/GPTTLBIMatch

```
// GPTTLBIMatch()
// =====
// Determine whether the GPT TLB entry lies within the scope of invalidation

func GPTTLBIMatch(tlbi : TLBIRecord, gpt_entry : GPTEntry) => boolean
begin
  assert tlbi.op IN {TLBIOp_RPA, TLBIOp_PAALL};

  var match : boolean;
  let entry_size_mask : bits(64) = ZeroExtend{}(Ones{gpt_entry.size});
  let entry_end_address : bits(64) = (ZeroExtend{64}(gpt_entry.pa[55:0] OR
    entry_size_mask[55:0]));
  let entry_start_address : bits(64) = (ZeroExtend{64}(gpt_entry.pa[55:0] AND NOT
    entry_size_mask[55:0]));

  case tlbi.op of
    when TLBIOp_RPA =>
      match = (UInt(tlbi.address[55:0]) <= UInt(entry_end_address[55:0]) &&
        UInt(tlbi.end_address[55:0]) > UInt(entry_start_address[55:0]) &&
        (tlbi.level == TLBILevel_Any || !gpt_entry.isstable {gpt_entry.level == 1}));
    when TLBIOp_PAALL =>
      match = TRUE;
  end;

  return match;
end;
```

Library pseudocode for aarch64/functions/tlbi/IsBroadcast_OSHnISH

```
// IsBroadcast_OSHnISH()
// =====
// Returns TRUE if a TLBI *OS* instruction affects TLBs in the Outer shareability domain,
// but not in the Inner shareability domain.

func IsBroadcast_OSHnISH() (regime => boolean
begin
  case: PSTATERegime.EL => boolean
begin
  case regime of
    when EL3Regime_EL3 => return FALSE;
    when EL2Regime_EL20 => return IsSCTLR2EL2Enabled() && SCTLR2_EL2().TLBOSNIS == '1';
    when EL1Regime_EL10 => return IsSCTLR2EL1Enabled() && SCTLR2_EL1().TLBOSNIS == '1';
    otherwise => unreachable;
  end;
end;
```



```

// TLBIDomains()
// =====
// Determine the domains the TLBI operation is broadcast to

func TLBIDomains(broadcast_in : Broadcast, tlbid_in : bits(16)) => bits(16)
begin
    assert broadcast_in != Broadcast_OSHnISH;

    var broadcast : Broadcast = broadcast_in;
    var tlbid : bits(16) = tlbid_in;

    if !IsFeatureImplemented(FEAT_TLBIID) then return DEFAULT_TLBI_DOMAIN; end;
    if broadcast == Broadcast_NSH then return DEFAULT_TLBI_DOMAIN; end;
    if broadcast == Broadcast_ISH_CnP then return DEFAULT_TLBI_DOMAIN; end;
    if PSTATE.EL == PSTATE.EL == EL3 then return DEFAULT_TLBI_DOMAIN; end;

    let nos : integer{} = UInt(TLBIDIDR_EL1().NOS);
    assert nos <= 16;
    if broadcast == PSTATE.EL == EL0 && HCR_EL2().TGE == '1' then return DEFAULT_TLBI_DOMAIN; end;

    let nos : integer{} = UInt(TLBIDIDR_EL1().NOS);
    assert nos <= 16;
    if broadcast == Broadcast_OSH && nos == 0 then
        return DEFAULT_TLBI_DOMAIN;
    end;

    let nis : integer{} = UInt(TLBIDIDR_EL1().NIS);
    assert nis <= 16;
    if broadcast == Broadcast_ISH && nis == 0 then
        return DEFAULT_TLBI_DOMAIN;
    end;

    if PSTATE.EL == PSTATE.EL == EL2 then
        if broadcast == Broadcast_OSH then
            return ZeroExtend{16}(tlbid[nos-1:0]);
        else // Broadcast_ISH
            return ZeroExtend{16}(tlbid[nis-1:0]);
        end;
    end;

    assert PSTATE.EL == PSTATE.EL == EL1 || (PSTATE.EL == EL0 && HCR_EL2().TGE == '0');

    if broadcast == Broadcast_ForcedISH then
        // For a TLBI instruction without an IS or OS qualifier executed at EL1, if HCR_EL2.FB
        // is 1 then the process for a TLBI IS instruction is followed as though TLBIID is zero.
        broadcast = Broadcast_ISH;
        tlbid = Zeros{16};
    end;

    assert broadcast IN {Broadcast_ISH, Broadcast_OSH};

    // If the Effective value of VTLBIID(OS)En is 0 the TLBIID field is not transformed.
    if ((HaveEL(EL3) && SCR_EL3().VTLBIIDEn == '0') || !IsHCRXEL2Enabled() ||
        (broadcast == Broadcast_OSH && HCRX_EL2().VTLBIIDOSEn == '0') ||
        (broadcast == Broadcast_ISH && HCRX_EL2().VTLBIIDEn == '0')) then
        if broadcast == Broadcast_OSH then
            return ZeroExtend{16}(tlbid[nos-1:0]);
        else // Broadcast_ISH
            return ZeroExtend{16}(tlbid[nis-1:0]);
        end;
    end;

    if broadcast == Broadcast_OSH then
        let nvos : integer{} = UInt(TLBIDIDR_EL1().NVOS);
        assert (nos <= 8 && nvos <= 5) || (nos <= 16 && nvos <= 4);

        // If NVOS is 0, the TLBIID field is ignored as only VTLBIIDOS[VTLBIIDOS_EL2{n}_EL2.TD0].TD0 is implemented.
        let d : integer{} = if nvos == 0 then 0 else UInt(tlbid[nvos-1:0]);
        let s : integer{} = if nos <= 8 then 8 else 4;
        let n : integer = d DIVRM s;
        let m : integer = d MOD s;

        var vtlbid : bits(16);
        if nos <= 8 then
            vtlbid = ZeroExtend{16}(VTLBIIDOS_EL2(n)[8*m+8]);
        else
            vtlbid = VTLBIIDOS_EL2(n)[16*m+16];
        end;

        return ZeroExtend{16}(vtlbid[nos-1:0]);
    else // Broadcast_ISH
        let nviss : integer{} = UInt(TLBIDIDR_EL1().NVIS);
        assert (nis <= 8 && nviss <= 5) || (nis <= 16 && nviss <= 4);

        // If NVIS is 0, the TLBIID field is ignored as only VTLBIID[VTLBIID_EL2{n}_EL2.TD0].TD0 is implemented.
        let d : integer{} = if nviss == 0 then 0 else UInt(tlbid[nviss-1:0]);
        let s : integer{} = if nis <= 8 then 8 else 4;
        let n : integer = d DIVRM s;
        let m : integer = d MOD s;
    end
end

```



```

var vtlbid : bits(16);
if nis <= 8 then
    vtlbid = ZeroExtend{16}(VTLBID_EL2(n)[8*m+:8]);
else
    vtlbid = VTLBID_EL2(n)[16*m+:16];
end;

return ZeroExtend{16}(vtlbid[nis-1:0]);
end;
end;

```



```

// AArch64_S1DirectBasePermissions()
// =====
// Computes the stage 1 direct base permissions

func AArch64_S1DirectBasePermissions(regime : Regime, walkstate : TTWState,
                                     walkparams : S1TTWParams,
                                     accdesc : AccessDescriptor) => S1AccessControls
begin
    var r, w, x : bit;
    var pr, pw, px : bit;
    var ur, uw, ux : bit;
    var permissions : Permissions = walkstate.permissions;
    var slperms : S1AccessControls;
    // Descriptors marked with DBM set have the effective value of AP[2] cleared.
    // This implies no Permission faults caused by lack of write permissions are
    // reported, and the Dirty bit can be set.
    if permissions.dbm == '1' && walkparams.hd == '1' then
        permissions.ap[2] = '0';
    end;

    if HasUnprivileged(regime) then
        // Apply leaf permissions
        case permissions.ap[2:1] of
            when '00' => (pr,pw,ur,uw) = ('1','1','0','0'); // Privileged access
            when '01' => (pr,pw,ur,uw) = ('1','1','1','1'); // No effect
            when '10' => (pr,pw,ur,uw) = ('1','0','0','0'); // Read-only, privileged access
            when '11' => (pr,pw,ur,uw) = ('1','0','1','0'); // Read-only
        end;

        // Apply hierarchical permissions
        case permissions.ap_table of
            when '00' => (pr,pw,ur,uw) = ( pr, pw, ur, uw); // No effect
            when '01' => (pr,pw,ur,uw) = ( pr, pw,'0','0'); // Privileged access
            when '10' => (pr,pw,ur,uw) = ( pr,'0', ur,'0'); // Read-only
            when '11' => (pr,pw,ur,uw) = ( pr,'0','0','0'); // Read-only, privileged access
        end;

        // Locations writable by unprivileged cannot be executed by privileged
        px = NOT(permissions.pxn OR permissions.pxn_table OR uw);
        ux = NOT(permissions.uxn OR permissions.uxn_table);

        if (IsFeatureImplemented(FEAT_PAN) && accdesc.pan && !(regime == Regime\_EL10 &&
            walkparams.nv1 == '1')) then
            var pan : bit;
            if (ImpDefBool("SCR_EL3.SIF affects EPAN") && accdesc.ss == SS\_Secure &&
                walkstate.baseaddress.paspace == PAS\_NonSecure && walkparams.sif == '1') then
                ux = '0';
            end;

            if (ImpDefBool("Realm EL2&0 regime affects EPAN") && accdesc.ss == SS\_Realm &&
                regime == Regime\_EL20 && walkstate.baseaddress.paspace != PAS\_Realm) then
                ux = '0';
            end;

            let pan = PSTATE.PAN AND (ur OR uw OR (walkparams.epan AND ux));
            pr = pr AND NOT(pan);
            pw := bitpw AND NOT(pan);
            end;

        else
            // Apply leaf permissions
            case permissions.ap[2] of
                when '0' => (pr,pw) = ('1','1'); // No effect
                when '1' => (pr,pw) = ('1','0'); // Read-only
            end;

            // Apply hierarchical permissions
            case permissions.ap_table[1] of
                when '0' => (pr,pw) = ( pr, pw); // No effect
                when '1' => (pr,pw) = ( pr,'0'); // Read-only
            end;

            px = NOT(permissions.xn OR permissions.xn_table);
            end;

            (r,w,x) = if accdesc.el == PSTATE.PAN AND (ur OR uw OR (walkparams.epan AND ux));
            pr = pr AND NOT(pan);
            pw = pw AND NOT(pan);
            end;

        else
            // Apply leaf permissions
            case permissions.ap[2] of
                when '0' => (pr,pw) = ('1','1'); // No effect
                when '1' => (pr,pw) = ('1','0'); // Read-only
            end;

            // Apply hierarchical permissions
            case permissions.ap_table[1] of

```

```

        when '0' => (pr,pw) = ( pr, pw); // No effect
        when '1' => (pr,pw) = ( pr, '0'); // Read-only
    end;

    px = NOT(permissions.xn OR permissions.xn_table);
end;

(r,w,x) = if accdesc.el == ELO then (ur,uw,ux) else (pr,pw,px);

// Compute WXN value
let wxn : bit = walkparams.wxn AND w AND x;

// Prevent execution from Non-secure space by PE in secure state if SIF is set
if accdesc.ss == SS\_Secure && walkstate.baseaddress.paspace == PAS\_NonSecure then
    x = x AND NOT(walkparams.sif);
end;
// Prevent execution from non-Root space by Root
if accdesc.ss == SS\_Root && walkstate.baseaddress.paspace != PAS\_Root then
    x = '0';
end;
// Prevent execution from non-Realm space by Realm EL2 and Realm EL2&0
if (accdesc.ss == SS\_Realm && regime IN {Regime\_EL2, Regime\_EL20} &&
    walkstate.baseaddress.paspace != PAS\_Realm) then
    x = '0';
end;

slperms.r    = r;
slperms.w    = w;
slperms.x    = x;
slperms.gcs  = '0';
slperms.wxn  = wxn;
slperms.overlay = TRUE;

return slperms;
end;

```

Library pseudocode for aarch64/translation/vmsa_faults/AArch64_S1HasAlignmentFaultDueToMemType

```

// AArch64_S1HasAlignmentFaultDueToMemType()
// =====
// Returns whether stage 1 output fails alignment requirement on data accesses due to memory type

func AArch64_S1HasAlignmentFaultDueToMemType(regime : Regime, accdesc : AccessDescriptor,
    aligned : boolean, ntlsmd : bit,
    memattr : MemoryAttributes) => boolean

begin
    if accdesc.exclusive || accdesc.atomicop || accdesc.acqsc || accdesc.acqpc || accdesc.relsc then
        if (!aligned && !(IsWBShareable(memattr) && AArch64\_S1DCacheEnabled(regime)) &&
            ConstrainUnpredictableBool(Unpredictable\_LSE2\_ALIGNMENT\_FAULT)) then
            return TRUE;
        end;
    end;

    if let memattr.memtype is TagWrite != MemType_Device boolean then
        return FALSE;
    elsif accdesc.cachetype ((accdesc.acctype IN != AccessType\_DCZero &&
        accdesc.cachetype IN {CacheType\_Tag, CacheType\_TagWrite, CacheType\_TagZero})
    if memattr.memtype != MemType_Device then
        return FALSE;
    elsif ((accdesc.acctype == AccessType\_DCZero) && isTagWrite) ||
        accdesc.stzgm then
        return ConstrainUnpredictable(Unpredictable\_DEVICE\_TAGSTORE) == Constraint\_FAULT;
    elsif accdesc.a32lsmd && ntlsmd == '0' then
        return memattr.device != DeviceType\_GRE;
    elsif accdesc.acctype == AccessType\_DCZero then
        return TRUE;
    elsif !aligned then
        return !(ImpDefBool("Device location supports unaligned access"));
    else
        return FALSE;
    end;
end;
end;

```



```

// AArch64_S1IndirectBasePermissions()
// =====
// Computes the stage 1 indirect base permissions

func AArch64_S1IndirectBasePermissions(regime : Regime, walkstate : TTWState,
                                       walkparams : S1TTWParams,
                                       accdesc : AccessDescriptor) => S1AccessControls
begin
    var r, w, x, gcs, wxn, overlay : bit;
    var pr, pw, px, pgcs, pwxn, p_overlay : bit;
    var ur, uw, ux, ugcs, uwxn, u_overlay : bit;
    let permissions : Permissions = walkstate.permissions;
    var slperms : S1AccessControls;

    // Apply privileged indirect permissions
    case permissions.ppi of
        when '0000' => (pr,pw,px,pgcs) = ('0','0','0','0'); // No access
        when '0001' => (pr,pw,px,pgcs) = ('1','0','0','0'); // Privileged read
        when '0010' => (pr,pw,px,pgcs) = ('0','0','1','0'); // Privileged execute
        when '0011' => (pr,pw,px,pgcs) = ('1','0','1','0'); // Privileged read and execute
        when '0100' => (pr,pw,px,pgcs) = ('0','0','0','0'); // Reserved
        when '0101' => (pr,pw,px,pgcs) = ('1','1','0','0'); // Privileged read and write
        when '0110' => (pr,pw,px,pgcs) = ('1','1','1','0'); // Privileged read, write and execute
        when '0111' => (pr,pw,px,pgcs) = ('1','1','1','1'); // Privileged read, write and execute
        when '1000' => (pr,pw,px,pgcs) = ('1','0','0','0'); // Privileged read
        when '1001' => (pr,pw,px,pgcs) = ('1','0','0','1'); // Privileged read and gcs
        when '1010' => (pr,pw,px,pgcs) = ('1','0','1','0'); // Privileged read and execute
        when '1011' => (pr,pw,px,pgcs) = ('0','0','0','0'); // Reserved
        when '1100' => (pr,pw,px,pgcs) = ('1','1','0','0'); // Privileged read and write
        when '1101' => (pr,pw,px,pgcs) = ('0','0','0','0'); // Reserved
        when '1110' => (pr,pw,px,pgcs) = ('1','1','1','0'); // Privileged read, write and execute
        when '1111' => (pr,pw,px,pgcs) = ('0','0','0','0'); // Reserved
    end;

    p_overlay = NOT(permissions.ppi[3]);
    pwxn = if permissions.ppi == '0110' then '1' else '0';

    if HasUnprivileged(regime) then
        // Apply unprivileged indirect permissions
        case permissions.upi of
            when '0000' => (ur,uw,ux,ugcs) = ('0','0','0','0'); // No access
            when '0001' => (ur,uw,ux,ugcs) = ('1','0','0','0'); // Unprivileged read
            when '0010' => (ur,uw,ux,ugcs) = ('0','0','1','0'); // Unprivileged execute
            when '0011' => (ur,uw,ux,ugcs) = ('1','0','1','0'); // Unprivileged read and execute
            when '0100' => (ur,uw,ux,ugcs) = ('0','0','0','0'); // Reserved
            when '0101' => (ur,uw,ux,ugcs) = ('1','1','0','0'); // Unprivileged read and write
            when '0110' => (ur,uw,ux,ugcs) = ('1','1','1','0'); // Unprivileged read, write and execute
            when '0111' => (ur,uw,ux,ugcs) = ('1','1','1','1'); // Unprivileged read, write and execute
            when '1000' => (ur,uw,ux,ugcs) = ('1','0','0','0'); // Unprivileged read
            when '1001' => (ur,uw,ux,ugcs) = ('1','0','0','1'); // Unprivileged read and gcs
            when '1010' => (ur,uw,ux,ugcs) = ('1','0','1','0'); // Unprivileged read and execute
            when '1011' => (ur,uw,ux,ugcs) = ('0','0','0','0'); // Reserved
            when '1100' => (ur,uw,ux,ugcs) = ('1','1','0','0'); // Unprivileged read and write
            when '1101' => (ur,uw,ux,ugcs) = ('0','0','0','0'); // Reserved
            when '1110' => (ur,uw,ux,ugcs) = ('1','1','1','0'); // Unprivileged read, write and execute
            when '1111' => (ur,uw,ux,ugcs) = ('0','0','0','0'); // Reserved
        end;

        u_overlay = NOT(permissions.upi[3]);
        uwxn = if permissions.upi == '0110' then '1' else '0';

        // If the decoded permissions has either px or pgcs along with either uw or ugcs,
        // then all effective Stage 1 Base Permissions are set to 0
        if ((px == '1' || pgcs == '1') && (uw == '1' || ugcs == '1')) then
            (pr,pw,px,pgcs) = ('0','0','0','0');
            (ur,uw,ux,ugcs) = ('0','0','0','0');
        elseif end;

        if (IsFeatureImplemented(FEAT_PAN) && accdesc.pan && !(regime == Regime_EL10 &&
            walkparams.nvl == '1')) then
            if PSTATE.PAN == '1' && (ImpDefBool("Realmpermissions.upi EL2&0!= regime'0000'") affects then
                (pr,pw) EPAN") = &&('0','0');
            end;
        end;
    end;

    if accdesc.ss == SS Realm &&
        regime == Regime_EL20 && walkstate.baseaddress.paspace != PAS Realm) then
        ux = '0';
    end;

    if (ImpDefBool("SCR_EL3.SIF affects EPAN") && accdesc.ss == SS Secure &&
        walkstate.baseaddress.paspace == PAS NonSecure && walkparams.sif == '1') then
        ux = '0';
    end;

    if PSTATE.PAN == '1' && (!IsZero(ur::uw::ux::ugcs) || (!IsZero(permissions.upi) &&
        ImpDefBool("PAN applies when S1UnprivBasePerm is reserved"))) then

```

```

        (pr,pw) = ('0','0');
    end;
end;
end;

if accdesc.el == EL0 then
    (r,w,x,gcs,wxn,overlay) = (ur,uw,ux,ugcs,uwxn,u_overlay);
else
    (r,w,x,gcs,wxn,overlay) = (pr,pw,px,pgcs,pwxn,p_overlay);
end;

// Prevent execution from Non-secure space by PE in secure state if SIF is set
if accdesc.ss == SS_Secure && walkstate.baseaddress.paspace == PAS_NonSecure then
    x = x AND NOT(walkparams.sif);
    gcs = '0';
end;
// Prevent execution from non-Root space by Root
if accdesc.ss == SS_Root && walkstate.baseaddress.paspace != PAS_Root then
    x = '0';
    gcs = '0';
end;
// Prevent execution from non-Realm space by Realm EL2 and Realm EL2&0
if (accdesc.ss == SS_Realm && regime IN {Regime_EL2, Regime_EL20} &&
    walkstate.baseaddress.paspace != PAS_Realm) then
    x = '0';
    gcs = '0';
end;

slperms.r      = r;
slperms.w      = w;
slperms.x      = x;
slperms.gcs    = gcs;
slperms.wxn    = wxn;
slperms.overlay = overlay == '1';

return slperms;
end;

```



```

// AArch64_S2CheckPermissions()
// =====
// Verifies memory access with available permissions.

func AArch64_S2CheckPermissions(fault_in : FaultRecord, walkstate : TTWState,
                                walkparams : S2TTWParams, ipa : AddressDescriptor,
                                accdesc : AccessDescriptor) => (FaultRecord, boolean)
begin
    let memtype : MemType = walkstate.memattrs.memtype;
    let permissions : Permissions = walkstate.permissions;
    var fault : FaultRecord = fault_in;
    let s2perms : S2AccessControls = AArch64_S2ComputePermissions(permissions, walkparams,
                                                                    accdesc);

    var r, w : bit;
    var or, ow : bit;

    if accdesc.acctype == AccessType_TTW then
        r = s2perms.r_mmu;
        w = s2perms.w_mmu;
        or = s2perms.or_mmu;
        ow = s2perms.ow_mmu;
    elseif accdesc.rcw then
        r = s2perms.r_rcw;
        w = s2perms.w_rcw;
        or = s2perms.or_rcw;
        ow = s2perms.ow_rcw;
    else
        r = s2perms.r;
        w = s2perms.w;
        or = s2perms.or;
        ow = s2perms.ow;
    end;

    if accdesc.acctype == AccessType_TTW then
        if (accdesc.toplevel && accdesc.varange == VRange_LOWER &&
            ((walkparams.tl0 == '1' && s2perms.toplevel0 == '0') ||
             (walkparams.tl1 == '1' && s2perms.[toplevel1,toplevel0] == '10'))) then
            fault.statuscode = Fault_Permission;
            fault.toplevel = TRUE;
        elseif (accdesc.toplevel && accdesc.varange == VRange_UPPER &&
            ((walkparams.tl1 == '1' && s2perms.toplevel1 == '0') ||
             (walkparams.tl0 == '1' && s2perms.[toplevel1,toplevel0] == '01'))) then
            fault.statuscode = Fault_Permission;
            fault.toplevel = TRUE;
        // Stage 2 Permission fault due to AssuredOnly check
        elseif (walkstate.s2assuredonly == '1' && !ipa.slassured) then
            fault.statuscode = Fault_Permission;
            fault.assuredonly = TRUE;

        elseif s2perms.overlay && or == '0' then
            fault.statuscode = Fault_Permission;
            fault.overlay = TRUE;
        elseif accdesc.write && s2perms.overlay && ow == '0' then
            fault.statuscode = Fault_Permission;
            fault.overlay = TRUE;

        elseif walkparams.ptw == '1' && memtype == MemType_Device then
            fault.statuscode = Fault_Permission;
        // Prevent translation table walks in Non-secure space by Realm state
        elseif accdesc.ss == SS_Realm && walkstate.baseaddress.paspace != PAS_Realm then
            fault.statuscode = Fault_Permission;
        elseif r == '0' then
            fault.statuscode = Fault_Permission;
        elseif accdesc.write && w == '0' then
            fault.statuscode = Fault_Permission;
            fault.hdbssf = walkparams.hdbss == '1' && !CanAppendToHDBSS() && permissions.dbm == '1';
        elseif (accdesc.write &&
            (walkparams.hd && walkparams.s2pie != '1' && !permissions.s2dirty(walkparams.hdbss == '01' &&
            (walkparams.hd != !CanAppendToHDBSS())) '1' ||
            (accdesc.acctype != AccessType_AT &&
            (walkparams.hdbss && walkparams.s2pie == '1' && !CanAppendToHDBSS())) permissions.s2dirty == '0')) then
            fault.statuscode = Fault_Permission;
            fault.dirtybit = TRUE;
            fault.hdbssf = walkparams.hdbss == '1' && !CanAppendToHDBSS();

        end;

    // Stage 2 Permission fault due to AssuredOnly check
    elseif ((walkstate.s2assuredonly == '1' && !ipa.slassured) ||
            (walkstate.s2assuredonly != '1' && IsFeatureImplemented(FEAT_GCS) &&
             VTCR_EL2().GCSH == '1' && accdesc.acctype == AccessType_GCS && accdesc.el != EL0)) then
        fault.statuscode = Fault_Permission;
        fault.assuredonly = TRUE;

    elseif accdesc.acctype == AccessType_IFETCH then
        if s2perms.overlay && s2perms.ox == '0' then
            fault.statuscode = Fault_Permission;
            fault.overlay = TRUE;

```

```

elseif (memtype == MemType_Device &&
        ConstrainUnpredictable(Unpredictable_INSTRDEVICE) == Constraint_FAULT) then
    fault.statuscode = Fault_Permission;

// Prevent execution from Non-secure space by Realm state
elseif accdesc.ss == SS_Realm && walkstate.baseaddress.paspace != PAS_Realm then
    fault.statuscode = Fault_Permission;
elseif s2perms.x == '0' then
    fault.statuscode = Fault_Permission;
end;

elseif accdesc.acctype == AccessType_DC then
    if accdesc.cacheop == CacheOp_Invalidate then
        if !ELUsingAArch32(EL1) && s2perms.overlay && ow == '0' then
            fault.statuscode = Fault_Permission;
            fault.overlay = TRUE;
        elseif !ELUsingAArch32(EL1) && w == '0' then
            fault.statuscode = Fault_Permission;
        elseif (walkparams.hd != '1' && walkparams.s2pie == '1' &&
                permissions.s2dirty == '0') then
            fault.statuscode = Fault_Permission;
            fault.dirtybit = TRUE;
        end;
    elseif !ELUsingAArch32(EL1) && accdesc.el == EL0 && s2perms.overlay && or == '0' then
        fault.statuscode = Fault_Permission;
        fault.overlay = TRUE;
    elseif (walkparams.cmw == '1' && accdesc.cacheop == CacheOp_CleanInvalidate &&
            s2perms.overlay && ow == '0') then
        fault.statuscode = Fault_Permission;
        fault.overlay = TRUE;
    elseif !ELUsingAArch32(EL1) && accdesc.el == EL0 && r == '0' then
        fault.statuscode = Fault_Permission;
    elseif (walkparams.cmw == '1' && accdesc.cacheop == CacheOp_CleanInvalidate &&
            w == '0') then
        fault.statuscode = Fault_Permission;
    elseif (walkparams.cmw == '1' && accdesc.cacheop == CacheOp_CleanInvalidate &&
            walkparams.hd != '1' && walkparams.s2pie == '1' && permissions.s2dirty == '0') then
        fault.statuscode = Fault_Permission;
        fault.dirtybit = TRUE;
    end;

elseif accdesc.acctype == AccessType_IC then
    if (!ELUsingAArch32(EL1) && accdesc.el == EL0 && s2perms.overlay && or == '0' &&
        ImpDefBool("Permission fault on EL0 IC_IVAU execution")) then
        fault.statuscode = Fault_Permission;
        fault.overlay = TRUE;
    elseif walkparams.cmw == '1' && s2perms.overlay && ow == '0' then
        fault.statuscode = Fault_Permission;
        fault.overlay = TRUE;
    elseif (!ELUsingAArch32(EL1) && accdesc.el == EL0 && r == '0' &&
        ImpDefBool("Permission fault on EL0 IC_IVAU execution")) then
        fault.statuscode = Fault_Permission;
    elseif walkparams.cmw == '1' && w == '0' then
        fault.statuscode = Fault_Permission;
    elseif (walkparams.cmw == '1' && walkparams.hd != '1' && walkparams.s2pie == '1' &&
            permissions.s2dirty == '0') then
        fault.statuscode = Fault_Permission;
        fault.dirtybit = TRUE;
    end;

elseif accdesc.read && s2perms.overlay && or == '0' then
    fault.statuscode = Fault_Permission;
    fault.overlay = TRUE;
    fault.write = FALSE;
elseif accdesc.write && s2perms.overlay && ow == '0' then
    fault.statuscode = Fault_Permission;
    fault.overlay = TRUE;
    fault.write = TRUE;
elseif accdesc.read && r == '0' then
    fault.statuscode = Fault_Permission;
    fault.write = FALSE;
elseif accdesc.write && w == '0' then
    fault.statuscode = Fault_Permission;
    fault.write = TRUE;
    fault.hdbssf = walkparams.hdbss == '1' && !CanAppendToHDBSS() && permissions.dbm == '1';
elseif (IsFeatureImplemented(FEAT_MTE_PERM) &&
        ((accdesc.tagchecked &&
            AArch64_EffectiveTCF(accdesc.el, accdesc.read) != TCFTType_Ignore) ||
            accdesc.tagaccess) &&
            ipa.memattrs.tags == MemTag_AllocationTagged &&
            permissions.s2tag_na == '1' && S2DCacheEnabled()) then
    fault.statuscode = Fault_Permission;
    fault.tagaccess = TRUE;
    fault.write = accdesc.tagaccess && accdesc.write;
elseif (accdesc.write &&
        (walkparams.hd != '1' || (walkparams.hdbss == '1' && !CanAppendToHDBSS())) &&
            walkparams.s2pie == '1' && permissions.s2dirty == '0') then
    fault.statuscode = Fault_Permission;
    fault.dirtybit = TRUE;

```

```

        fault.write      = TRUE;
        fault.hdbssf     = walkparams.hdbss == '1' && !CanAppendToHDBSS();
    end;
    // MRO* allows only RCW and MMU writes
    var mro : boolean;
    if s2perms.overlay then
        mro = (s2perms.[w,w_rcw,w_mmu] AND s2perms.[ow,ow_rcw,ow_mmu]) == '011';
    else
        mro = s2perms.[w,w_rcw,w_mmu] == '011';
    end;

    return (fault, mro);
end;

```

Library pseudocode for aarch64/translation/vmsa_faults/AArch64_S2HasAlignmentFaultDueToMemType

```

// AArch64_S2HasAlignmentFaultDueToMemType()
// =====
// Returns whether stage 2 output fails alignment requirement on data accesses due to memory type

func AArch64_S2HasAlignmentFaultDueToMemType(accdesc : AccessDescriptor, aligned : boolean,
                                              memattrs : MemoryAttributes) => boolean
begin
    if accdesc.exclusive || accdesc.atomicop || accdesc.acqsc || accdesc.acqpc || accdesc.relsc then
        if (!aligned && !(IsWBShareable(memattrs) && S2DCacheEnabled())) &&
            ConstrainUnpredictableBool(Unpredictable\_LSE2\_ALIGNMENT\_FAULT)) then
            return TRUE;
        end;
    end;

    if memattrs.memtype != MemType_Device then
        return FALSE;
    elsif ((accdesc.acctype == AccessType\_DCZero &&
            accdesc.cachetype IN (CacheType_Tag,)) ++
            accdesc.stzgm) then
        return ConstrainUnpredictable(CacheType\_TagWrite, CacheType\_TagZero)) ||
            accdesc.stzgm) then
        return ConstrainUnpredictable(Unpredictable\_DEVICETAGSTORE) == Constraint\_FAULT;
    elsif accdesc.acctype == AccessType\_DCZero then
        return TRUE;
    elsif !aligned then
        return !(ImpDefBool("Device location supports unaligned access"));
    else
        return FALSE;
    end;
end;

```



```

// AArch64_S1Translate()
// =====
// Translate VA to IPA/PA depending on the regime

func AArch64_S1Translate(fault_in : FaultRecord, regime : Regime,
    va : bits(64), size : integer, aligned : boolean,
    accdesc : AccessDescriptor) => (FaultRecord, AddressDescriptor)
    recurselimit Unbounded_DescriptorUpdate

begin
    var fault : FaultRecord = fault_in;
    // Prepare fault fields in case a fault is detected
    fault.secondstage = FALSE;
    fault.s2fslwalk = FALSE;

    if !AArch64\_S1Enabled(regime, accdesc.acctype) then
        return AArch64_S1DisabledOutput(fault, regime, va, accdesc, aligned);
    end;

    var walkparams : S1TTWParams = AArch64\_GetS1TTWParams(regime, accdesc.el, accdesc.ss, va);
    let slmintxsz : integer = AArch64\_S1MinTxSZ(regime, walkparams);
    let slmaxtxsz : integer = AArch64\_MaxTxSZ(walkparams.tgx);
    if AArch64\_S1TxSZFaults(regime, walkparams) then
        fault.statuscode = Fault\_Translation;
        fault.level = 0;
        return (fault, ARBITRARY : AddressDescriptor);
    elsif UInt(walkparams.txsz) < slmintxsz then
        walkparams.txsz = slmintxsz[5:0];
    elsif UInt(walkparams.txsz) > slmaxtxsz then
        walkparams.txsz = slmaxtxsz[5:0];
    end;

    if AArch64_VAIsOutOfRange(va, accdesc.acctype, regime, walkparams) then
        fault.statuscode = Fault\_Translation;
        fault.level = 0;
        return (fault, ARBITRARY : AddressDescriptor);
    end;

    if accdesc.el == EL0 && walkparams.e0pd == '1' then
        fault.statuscode = Fault\_Translation;
        fault.level = 0;
        return (fault, ARBITRARY : AddressDescriptor);
    end;

    if (IsFeatureImplemented(FEAT_SVE) && accdesc.el == EL0 && walkparams.nfd == '1' &&
        ((accdesc.nonfault && accdesc.contiguous) ||
        (accdesc.firstfault && !accdesc.first && !accdesc.contiguous))) then
        fault.statuscode = Fault\_Translation;
        fault.level = 0;
        return (fault, ARBITRARY : AddressDescriptor);
    end;

    var descipaddr : AddressDescriptor;
    var walkstate : TWState;
    var descriptor : bits(128);
    if walkparams.dl28 == '1' then
        (fault, descipaddr, walkstate, descriptor) = AArch64\_S1Walk{128}(fault, walkparams, va,
            regime, accdesc);
    else
        (fault, descipaddr, walkstate, descriptor[63:0]) = AArch64\_S1Walk{64}(fault, walkparams, va,
            regime, accdesc);
        descriptor[127:64] = Zeros{64};
    end;
    if fault.statuscode != Fault\_None then
        return (fault, ARBITRARY : AddressDescriptor);
    end;
    if AArch64\_S1HasAlignmentFaultDueToMemType(regime, accdesc, aligned, walkparams.ntlsm,
        walkstate.memattrs) then
        fault.statuscode = Fault\_Alignment;
    end;

    let fault_perm : FaultRecord = AArch64\_S1CheckPermissions(fault, va, size, regime,
        walkstate, walkparams, accdesc);

    var mem_desc : bits(128);
    var new_desc : bits(128) = descriptor;

    if AArch64\_SetAccessFlag(walkparams.ha, accdesc, fault) then
        // Set descriptor AF bit
        new_desc[10] = '1';
    end;

    // If HW update of dirty bit is enabled, the walk state permissions
    // will already reflect a configuration permitting writes.
    // The update of the descriptor occurs only if the descriptor bits in
    // memory do not reflect that and the access instigates a write.

    if AArch64\_SetDirtyState(walkparams.hd, (walkparams.pie OR descriptor[51]),
        accdesc, fault, fault_perm) then
        // Clear descriptor AP[2]/nDirty bit permitting stage 1 writes

```

```

    new_desc[7] = '0';
end;

if fault.statuscode == Fault\_None && fault_perm.statuscode != Fault\_None then
    fault = fault_perm;
end;

// Either the access flag was clear or AP[2]/nDirty was set
if new_desc != descriptor then
    if var (!descpaddr : IsWBSHareable(descipaddr.memattr) &&
        ConstrainUnpredictableBool(Unpredictable\_Unsupported\_Atomic\_HW\_Update)) then
        fault.statuscode = Fault\_HWUpdateAccessFlag;
        return (fault, ARBITRARY : AddressDescriptor);
    end;

    var descpaddr : AddressDescriptor;
    let descaccess : AccessDescriptor = CreateAccDescTTEUpdate(accdesc);

    if regime == Regime\_EL10 && EL2Enabled() then
        var s2fault : FaultRecord;
        let slaarch64 : boolean = TRUE;
        let s2aligned : boolean = TRUE;
        (s2fault, descpaddr) = AArch64\_S2Translate(fault, descipaddr, slaarch64, s2aligned,
            descaccess);

        if s2fault.statuscode != Fault\_None then
            return (s2fault, ARBITRARY : AddressDescriptor);
        end;

    else
        descpaddr = descipaddr;
    end;

    if walkparams.d128 == '1' then
        (fault, mem_desc) = AArch64\_MemSwapTableDesc{128}(fault, descriptor,
            new_desc, walkparams.ee,
            descaccess, descpaddr);

    else
        (fault, mem_desc[63:0]) = AArch64\_MemSwapTableDesc{64}(fault, descriptor[63:0],
            new_desc[63:0], walkparams.ee,
            descaccess, descpaddr);

        mem_desc[127:64] = Zeros{64};
    end;

    if fault.statuscode != Fault\_None then
        if (accdesc.acctype == AccessType\_AT &&
            !(ImpDefBool("AT reports the HW update fault"))) then
            // Mask the fault
            fault.statuscode = Fault\_None;
        else
            return (fault, ARBITRARY : AddressDescriptor);
        end;
    elseif new_desc != descriptor && mem_desc != new_desc then
        // HW update of Dirty state or AF was not successful due to the descriptor being updated
        // not matching the descriptor used for translation. Due to this, the walk is restarted.
        return AArch64\_S1Translate(fault_in, regime, va, size, aligned, accdesc);
    end;
end;

if fault.statuscode != Fault\_None then
    return (fault, ARBITRARY : AddressDescriptor);
end;

// Output Address
let oa : FullAddress = StageOA(va, walkparams.d128, walkparams.tgx, walkstate);
var memattr : MemoryAttributes;
if AArch64\_S1TreatAsNormalNC(walkstate, regime, accdesc) then
    // Treat memory attributes as Normal Non-Cacheable
    memattr = NormalNCMemAttr();
    memattr.xs = walkstate.memattr.xs;

    // The effect of SCTLr_ELx.C when '0' is Constrained UNPREDICTABLE on the Tagged attribute
    // when the memory region is Allocation Tagged.
    if (IsFeatureImplemented(FEAT_MTE2) &&
        walkstate.memattr.tags == MemTag\_AllocationTagged &&
        ConstrainUnpredictableBool(Unpredictable\_S1CTAGGED)) then
        memattr.tags = MemTag\_AllocationTagged;
    // SCTLr_ELx.C has no effect on whether the memory region is treated as Canonically Tagged.
    elseif (IsFeatureImplemented(FEAT_MTE_CANONICAL_TAGS) &&
        walkstate.memattr.tags == MemTag\_CanonicallyTagged) then
        memattr.tags = MemTag\_CanonicallyTagged;
    end;
else
    memattr = walkstate.memattr;
end;

// Shareability value of stage 1 translation subject to stage 2 is IMPLEMENTATION DEFINED
// to be either effective value or descriptor value
if (regime == Regime\_EL10 && EL2Enabled() && HCR_EL2().VM == '1' &&
    !(ImpDefBool("Apply effective shareability at stage 1"))) then
    memattr.shareability = walkstate.memattr.shareability;
end;

```

```

else
    memattrs.shareability = EffectiveShareability(memattrs);
end;

var ipa : AddressDescriptor = CreateAddressDescriptor(va, oa, memattrs, accdesc);
ipa.slassured = walkstate.slassured;
let varange : VARange = AArch64\_GetVARange(va);
ipa.mecid = AArch64\_SlOutputMECID{128}(walkparams, regime, varange, ipa.paddress.paspace,
                                     descriptor);

if (accdesc.atomicop && !IsWBShareable(memattrs) &&
    ConstrainUnpredictableBool(Unpredictable\_Atomic\_MMU\_IMPDEF\_FAULT)) then
    fault.statuscode = Fault\_Exclusive;
    return (fault, ipa);
end;

if accdesc.ls64 && memattrs.memtype == MemType_Normal then
    if IsFeatureImplemented(FEAT_LS64WB) && !accdesc.withstatus then
        if (!IsWBShareable(memattrs) &&
            !(memattrs.inner.attrs == MemAttr\_NC &&
              memattrs.outer.attrs == MemAttr\_NC) &&
            (ImpDefBool(
                "LD64B or ST64B faults to cacheable non-iWBowB memory"))) then
            fault.statuscode = Fault\_Exclusive;
            return (fault, ipa);
        end;
    elseif !(memattrs.inner.attrs == MemAttr\_NC && memattrs.outer.attrs == MemAttr\_NC) then
        fault.statuscode = Fault\_Exclusive;
        return (fault, ipa);
    end;
end;

return (fault, ipa);
end;

```



```

// AArch64_S2Translate()
// =====
// Translate stage 1 IPA to PA and combine memory attributes

func AArch64_S2Translate(fault_in : FaultRecord, ipa : AddressDescriptor,
    slaarch64 : boolean, aligned : boolean,
    accdesc : AccessDescriptor) => (FaultRecord, AddressDescriptor)
    recurselimit Unbounded_DescriptorUpdate

begin
    var walkparams : S2TTWParams = AArch64\_GetS2TTWParams(accdesc.ss, ipa.paddress.paspace,
        slaarch64);

    var fault : FaultRecord = fault_in;
    var s2fslmro : boolean;
    // Prepare fault fields in case a fault is detected
    fault.statuscode = Fault\_None; // Ignore any faults from stage 1
    fault.dirtybit    = FALSE;
    fault.overlay     = FALSE;
    fault.tagaccess   = FALSE;
    fault.s1tagnotdata = FALSE;
    fault.secondstage  = TRUE;
    fault.s2fslwalk   = accdesc.acctype == AccessType\_TTW;
    fault.ipaddress    = ipa.paddress;

    if walkparams.vm != '1' then
        // Stage 2 translation is disabled
        return (fault, ipa);
    end;

    let s2mintxsx : integer = AArch64\_S2MinTxSZ(walkparams, slaarch64);
    let s2maxtxsz : integer = AArch64\_MaxTxSZ(walkparams.tgx);
    if AArch64\_S2TxSZFaults(walkparams, slaarch64) then
        fault.statuscode = Fault\_Translation;
        fault.level      = 0;
        return (fault, ARBITRARY : AddressDescriptor);
    elseif UInt(walkparams.txsz) < s2mintxsx then
        walkparams.txsz = s2mintxsx[5:0];
    elseif UInt(walkparams.txsz) > s2maxtxsz then
        walkparams.txsz = s2maxtxsz[5:0];
    end;

    if (walkparams.dl28 == '0' &&
        (AArch64\_S2InvalidSL(walkparams) || AArch64\_S2InconsistentSL(walkparams))) then
        fault.statuscode = Fault\_Translation;
        fault.level      = 0;
        return (fault, ARBITRARY : AddressDescriptor);
    end;

    if AArch64\_IPAIsOutOfRange(ipa.paddress.address, walkparams) then
        fault.statuscode = Fault\_Translation;
        fault.level      = 0;
        return (fault, ARBITRARY : AddressDescriptor);
    end;

    var descaddr : AddressDescriptor;
    var walkstate : TTWState;
    var descriptor : bits(128);
    if walkparams.dl28 == '1' then
        (fault, descaddr, walkstate, descriptor) = AArch64\_S2Walk{128}(fault, ipa, walkparams,
            accdesc);
    else
        (fault, descaddr, walkstate, descriptor[63:0]) = AArch64\_S2Walk{64}(fault, ipa, walkparams,
            accdesc);
        descriptor[127:64] = Zeros{64};
    end;
    if fault.statuscode != Fault\_None then
        return (fault, ARBITRARY : AddressDescriptor);
    end;

    if AArch64\_S2HasAlignmentFaultDueToMemType(accdesc, aligned, walkstate.memattrs) then
        fault.statuscode = Fault\_Alignment;
    end;

    var fault_perm : FaultRecord;
    (fault_perm, s2fslmro) = AArch64\_S2CheckPermissions(fault, walkstate, walkparams, ipa, accdesc);

    var mem_desc : bits(128);
    var new_desc : bits(128) = descriptor;

    if AArch64\_SetAccessFlag(walkparams.ha, accdesc, fault) then
        // Set descriptor AF bit
        new_desc[10] = '1';
    end;

    // If HW update of dirty bit is enabled, the walk state permissions
    // will already reflect a configuration permitting writes.
    // The update of the descriptor occurs only if the descriptor bits in
    // memory do not reflect that and the access instigates a write.

    if AArch64\_SetDirtyState(walkparams.hd, (walkparams.s2pie OR descriptor[51]),

```

```

        accdesc, fault, fault_perm) then
    // Set descriptor S2AP[1]/Dirty bit permitting stage 2 writes
    new_desc[7] = '1';
end;

if fault.statuscode == Fault None && fault_perm.statuscode != Fault None then
    fault = fault_perm;
end;

// Either the access flag was clear or S2AP[1]/Dirty was is clear
if new_desc != descriptor then
    if walkparams.hdbss == '1' && descriptor[7] == '0' && new_desc[7] == '1' then
        fault = AppendToHDBSS(fault, ipa.paddress, accdesc, walkparams, walkstate.level);
    end;

    // If an error, other than a synchronous External abort, occurred on the HDBSS update,
    // stage 2 hardware update of dirty state is not permitted.
    if (HDBSSPROD_EL2().FSC != '101000' &&
        (!fault.hdbssf || IsWBShareable(descpaddr.memattrs) &&
        ConstrainUnpredictableBool(Unpredictable\_UnsupportedAtomicHWUpdate)) then
        fault.statuscode = Fault\_HWUpdateAccessFlag;
        return (fault, ARBITRARY : AddressDescriptor);
    end;

    if walkparams.hdbss == '1' && descriptor[7] == '0' && new_desc[7] == '1' then
        fault = AppendToHDBSS(fault, ipa.paddress, accdesc, walkparams, walkstate.level);
    end;

    // If an error, other than a synchronous External abort, occurred on the HDBSS update,
    // stage 2 hardware update of dirty state is not permitted.
    if (!(walkparams.hdbss == '1' && descriptor[7] != new_desc[7]) ||
        (HDBSSPROD_EL2().FSC != '101000' &&
        (!fault.hdbssf || IsExternalAbort(fault.statuscode)))) then
        let descaccess : AccessDescriptor = CreateAccDescTTEUpdate(accdesc);
        if walkparams.d128 == '1' then
            (fault, mem_desc) = AArch64\_MemSwapTableDesc{128}(fault, descriptor, new_desc,
                walkparams.ee, descaccess,
                descpaddr);
        else
            (fault, mem_desc[63:0]) = AArch64\_MemSwapTableDesc{64}(fault, descriptor[63:0],
                new_desc[63:0], walkparams.ee,
                descaccess, descpaddr);
            mem_desc[127:64] = Zeros{64};
        end;
    end;

    if fault.statuscode != Fault None then
        if (accdesc.acctype == AccessType\_AT &&
            !(ImpDefBool("AT reports the HW update fault"))) then
            // Mask the fault
            fault.statuscode = Fault None;
        else
            return (fault, ARBITRARY : AddressDescriptor);
        end;
    elsif new_desc != descriptor && mem_desc != new_desc then
        // HW update of Dirty state or AF was not successful due to the descriptor being updated
        // not matching the descriptor used for translation. Due to this, the walk is restarted.
        return AArch64\_S2Translate(fault_in, ipa, slaarch64, aligned, accdesc);
    end;
end;

if fault.statuscode != Fault None then
    return (fault, ARBITRARY : AddressDescriptor);
end;

let ipa_64 : bits(64) = ZeroExtend{}(ipa.paddress.address);
// Output Address
let oa : FullAddress = StageOA(ipa_64, walkparams.d128, walkparams.tgx, walkstate);
var s2_memattrs : MemoryAttributes;

if AArch64\_S2TreatAsNormalNC(walkstate, walkparams, accdesc) then
    // Treat memory attributes as Normal Non-Cacheable
    s2_memattrs = NormalNCMemAttr();
    s2_memattrs.xs = walkstate.memattrs.xs;
    if walkstate.memattrs.tags == MemTag\_CanonicallyTagged then
        s2_memattrs.tags = MemTag\_CanonicallyTagged;
    end;
else
    s2_memattrs = walkstate.memattrs;
end;

let s2aarch64 : boolean = TRUE;
var memattrs : MemoryAttributes;
if walkparams.fwb == '0' then
    memattrs = S2CombinesS1MemAttrs(ipa.memattrs, s2_memattrs, s2aarch64);
else
    memattrs = s2_memattrs;
end;

```

```

var pa : AddressDescriptor = CreateAddressDescriptor(ipa.vaddress, oa, memattrs, accdesc);
pa.s2fslmro = s2fslmro;
pa.mecid = AArch64_S2OutputMECID{128}(walkparams, pa.paddress.paspace, descriptor);

if (accdesc.acctype == AccessType TTW && accdesc.atomicop && !IsWBShareable(pa.memattrs) &&
    ConstrainUnpredictableBool(Unpredictable_Unsupported_Atomic_HW_Update)) then
    fault.statuscode = Fault_HWUpdateAccessFlag;
    return (fault, pa);
end;

if (accdesc.acctype != AccessType TTW && {128}(walkparams, pa.paddress.paspace, descriptor);

if (accdesc.atomicop && !IsWBShareable(pa.memattrs s2_memattrs) &&
    ConstrainUnpredictableBool(Unpredictable_Atomic_MMU_IMPDEF_FAULT)) then
    fault.statuscode = Fault_Exclusive;
    return (fault, pa);
end;

if accdesc.ls64 && s2_memattrs.memtype == MemType_Normal then
    if IsFeatureImplemented(FEAT_LS64WB) && !accdesc.withstatus then
        if (!IsWBShareable(s2_memattrs) &&
            !(s2_memattrs.inner.attrs == MemAttr_NC &&
                s2_memattrs.outer.attrs == MemAttr_NC) &&
            (ImpDefBool(
                "LD64B or ST64B faults to cacheable non-iWBowB memory"))) then
            fault.statuscode = Fault_Exclusive;
            return (fault, ipa);
        end;
    elsif !(s2_memattrs.inner.attrs == MemAttr_NC && s2_memattrs.outer.attrs == MemAttr_NC) then
        fault.statuscode = Fault_Exclusive;
        return (fault, ipa);
    end;
end;

return (fault, pa);
end;

```



```

// AArch64_SlWalk()
// =====
// Traverse stage 1 translation tables obtaining the final descriptor
// as well as the address leading to that descriptor

func AArch64_SlWalk(N : integer{64, 128})(fault_in : FaultRecord,
                                         walkparams : SlTTWParams,
                                         va : bits(64), regime : Regime,
                                         accdesc : AccessDescriptor) => (FaultRecord, AddressDescriptor, TTWState, bits(N))

begin
    var fault : FaultRecord = fault_in;
    var aligned : boolean;

    if HasUnprivileged(regime) && AArch64\_SlEPD(regime, va) == '1' then
        fault.statuscode = Fault\_Translation;
        fault.level = 0;
        return (fault, ARBITRARY : AddressDescriptor, ARBITRARY : TTWState, ARBITRARY : bits(N));
    end;

    var walkstate : TTWState = AArch64\_SlInitialTTWState(walkparams, va, regime, accdesc.ss);
    let startlevel : integer = walkstate.level;

    if startlevel > 3 then
        fault.statuscode = Fault\_Translation;
        fault.level = 0;
        return (fault, ARBITRARY : AddressDescriptor, ARBITRARY : TTWState, ARBITRARY : bits(N));
    end;

    var descriptor : bits(N);
    var walkaddress : AddressDescriptor;
    var skl : bits(2) = '00';
    walkaddress.vaddress = va;
    walkaddress.mecid = AArch64\_SlTTWalkMECID(walkparams.emec, regime, accdesc.ss);

    if !AArch64\_SlDCacheEnabled(regime) then
        walkaddress.memattrs = NormalNCMemAttr();
        walkaddress.memattrs.xs = walkstate.memattrs.xs;
    else
        walkaddress.memattrs = walkstate.memattrs;
    end;

    // Shareability value of stage 1 translation subject to stage 2 is IMPLEMENTATION DEFINED
    // to be either effective value or descriptor value
    if (regime == Regime\_EL10 && EL2Enabled() && HCR_EL2().VM == '1' &&
        !(ImpDefBool("Apply effective shareability at stage 1"))) then
        walkaddress.memattrs.shareability = walkstate.memattrs.shareability;
    else
        walkaddress.memattrs.shareability = EffectiveShareability(walkaddress.memattrs);
    end;

    var s2fslmro : boolean = FALSE;

    var desctype : DescriptorType;
    var descaddress : FullAddress = AArch64\_SlSLTTEEntryAddress(walkstate.level, walkparams, va,
                                                             walkstate.baseaddress);

    // Detect Address Size Fault by Descriptor Address
    if AArch64\_Sl0AOutOfRange(descaddress.address, walkparams) then
        fault.statuscode = Fault\_AddressSize;
        fault.level = 0;
        return (fault, ARBITRARY : AddressDescriptor, ARBITRARY : TTWState, ARBITRARY : bits(N));
    end;

    repeat
        fault.level = walkstate.level;
        walkaddress.paddress = descaddress;
        walkaddress.slassured = walkstate.slassured;

        let toplevel : boolean = walkstate.level == startlevel;
        let varange : VARange = AArch64\_GetVARange(va);
        let walkaccess : AccessDescriptor = CreateAccDescSlTTW(toplevel, varange, accdesc);
        var s2fault : FaultRecord;
        var s2walkaddress : AddressDescriptor;
        if regime == Regime\_EL10 && EL2Enabled() then
            let slaarch64 : boolean = TRUE;
            aligned = TRUE;
            (s2fault, s2walkaddress) = AArch64\_S2Translate(fault, walkaddress, slaarch64, aligned,
                                                         walkaccess);

            if s2fault.statuscode != Fault\_None then
                return (s2fault, ARBITRARY : AddressDescriptor, ARBITRARY : TTWState,
                        ARBITRARY : bits(N));
            end;

            s2fslmro = s2walkaddress.s2fslmro;
            (fault, descriptor) = FetchDescriptor{N}(walkparams.ee, s2walkaddress, walkaccess,
                                                    fault);
        else

```

```

(fault, descriptor) = FetchDescriptor{N}(walkparams.ee, walkaddress, walkaccess,
                                         fault);

end;

if fault.statuscode != Fault None then
    return (fault, ARBITRARY : AddressDescriptor, ARBITRARY : TTWState,
           ARBITRARY : bits(N));
end;

var new_descriptor : bits(N);
repeat
    new_descriptor = descriptor;
    descctype = AArch64 DecodeDescriptorType{N}(descriptor, walkparams.dl28, walkparams.ds,
                                                walkparams.tgx, walkstate.level);

    case descctype of
        when DescriptorType Table =>
            walkstate = AArch64 S1NextWalkStateTable{N}(walkstate, s2fslmro,
                                                         regime, walkparams, descriptor);
            skl = if walkparams.dl28 == '1' then descriptor[110:109] else '00';
            descaddress = AArch64 S1TTEntryAddress{N}(walkstate.level, walkparams, skl, va,
                                                    walkstate.baseaddress, descriptor);

            // Detect Address Size Fault by Descriptor Address
            if AArch64 S1OAOuOfRange(descaddress.address, walkparams) then
                fault.statuscode = Fault AddressSize;
                return (fault, ARBITRARY : AddressDescriptor, ARBITRARY : TTWState,
                       ARBITRARY : bits(N));
            end;

            if walkparams.haft == '1' then
                new_descriptor[10] = '1';
            end;

            if (walkparams.dl28 == '1' && skl != '00' &&
                AArch64 nTFaults{N}(walkparams.dl28, descriptor)) then
                fault.statuscode = Fault Translation;
                return (fault, ARBITRARY : AddressDescriptor, ARBITRARY : TTWState,
                       ARBITRARY : bits(N));
            end;

        when DescriptorType Leaf =>
            walkstate = AArch64 S1NextWalkStateLeaf{N}(walkstate, s2fslmro, regime, accdesc,
                                                         walkparams, descriptor);

        when DescriptorType Invalid =>
            fault.statuscode = Fault Translation;
            return (fault, ARBITRARY : AddressDescriptor, ARBITRARY : TTWState,
                   ARBITRARY : bits(N));

        otherwise =>
            unreachable;
    end;

    if new_descriptor != descriptor then
        if
            var (!descpaddr : IsWBShareable(walkaddress.memattrs) &&
                ConstrainUnpredictableBool(Unpredictable Unsupported Atomic HW Update)) then
            fault.statuscode = Fault HWUpdateAccessFlag;
            return (fault, ARBITRARY : AddressDescriptor, ARBITRARY : TTWState,
                   ARBITRARY : bits(N));
        end;

        var descpaddr : AddressDescriptor;
        let descaccess : AccessDescriptor = CreateAccDescTTEUpdate(accdesc);
        if regime == Regime EL10 && EL2Enabled() then
            let slaarch64 : boolean = TRUE;
            aligned = TRUE;
            (s2fault, descpaddr) = AArch64 S2Translate(fault, walkaddress,
                                                         slaarch64, aligned,
                                                         descaccess);

            if s2fault.statuscode != Fault None then
                return (s2fault, ARBITRARY : AddressDescriptor,
                       ARBITRARY : TTWState, ARBITRARY : bits(N));
            end;
        else
            descpaddr = walkaddress;
        end;

        (fault, descriptor) = AArch64 MemSwapTableDesc{N}(fault, descriptor, new_descriptor,
                                                         walkparams.ee, descaccess,
                                                         descpaddr);

        if fault.statuscode != Fault None then
            return (fault, ARBITRARY : AddressDescriptor,
                   ARBITRARY : TTWState, ARBITRARY : bits(N));
        end;
    end;

    until new_descriptor == descriptor looplimit Unbounded\_DescriptorUpdate;
until descctype == DescriptorType Leaf looplimit MAX\_WALK\_LEVELS;

let oa : FullAddress = StageOA(va, walkparams.dl28, walkparams.tgx, walkstate);
if (walkstate.contiguous == '1' &&

```

```

    AArch64_ContiguousBitFaults(walkparams.d128, walkparams.txsz, walkparams.tgx,
                                walkstate.level)) then
        fault.statuscode = Fault Translation;
    elsif walkstate.level < FINAL LEVEL && AArch64 nTFaults{N}(walkparams.d128, descriptor) then
        fault.statuscode = Fault Translation;
    elsif AArch64 SlAMECFault{N}(walkparams, walkstate.baseaddress.paspace, regime, descriptor) then
        fault.statuscode = Fault Translation;
    // Detect Address Size Fault by final output
    elsif AArch64 SlOAOutOfRange(oa.address, walkparams) then
        fault.statuscode = Fault AddressSize;
    // Check descriptor AF bit
    elsif (descriptor[10] == '0' && walkparams.ha == '0' &&
           (!accdesc.acctype IN {AccessType DC, AccessType IC} ||
            ImpDefBool("Generate access flag fault on IC/DC operations"))) then
        fault.statuscode = Fault AccessFlag;
    end;

    if fault.statuscode != Fault None then
        return (fault, ARBITRARY : AddressDescriptor, ARBITRARY : TTWState, ARBITRARY : bits(N));
    end;

    return (fault, walkaddress, walkstate, descriptor);
end;

```



```

// AArch64_S2Walk()
// =====
// Traverse stage 2 translation tables obtaining the final descriptor
// as well as the address leading to that descriptor

func AArch64_S2Walk(N : integer{64, 128})(fault_in : FaultRecord,
                                           ipa : AddressDescriptor,
                                           walkparams : S2TTWParams,
                                           accdesc : AccessDescriptor) => (FaultRecord,
                                                                    AddressDescriptor,
                                                                    TTWState,
                                                                    bits(N))

begin

    var fault : FaultRecord = fault_in;
    let ipa_64 : bits(64) = ZeroExtend{}(ipa.paddress.address);

    var walkstate : TTWState;
    if accdesc.ss == SS\_Secure then
        walkstate = AArch64\_SS2InitialTTWState(walkparams, ipa.paddress.paspace);
    else
        walkstate = AArch64\_S2InitialTTWState(accdesc.ss, walkparams);
    end;
    let startlevel : integer = walkstate.level;

    if startlevel > 3 then
        fault.statuscode = Fault\_Translation;
        fault.level = 0;
        return (fault, ARBITRARY : AddressDescriptor, ARBITRARY : TTWState, ARBITRARY : bits(N));
    end;

    var descriptor : bits(N);
    let walkaccess : AccessDescriptor = CreateAccDescS2TTW(accdesc);
    var walkaddress : AddressDescriptor;
    var skl : bits(2) = '00';

    walkaddress.vaddress = ipa.vaddress;
    walkaddress.mecid = AArch64\_S2TTWalkMECID(walkparams.emec, accdesc.ss);

    if !S2DCacheEnabled() then
        walkaddress.memattrs = NormalNCMemAttr();
        walkaddress.memattrs.xs = walkstate.memattrs.xs;
    else
        walkaddress.memattrs = walkstate.memattrs;
    end;

    walkaddress.memattrs.shareability = EffectiveShareability(walkaddress.memattrs);

    var desctype : DescriptorType;

    // Initial lookup might index into concatenated tables
    var descaddress : FullAddress = AArch64\_S2SLTTEnterAddress(walkparams, ipa.paddress.address,
                                                            walkstate.baseaddress);

    // Detect Address Size Fault by Descriptor Address
    if AArch64\_S2OAOutOfRange(descaddress.address, walkparams) then
        fault.statuscode = Fault\_AddressSize;
        fault.level = 0;
        return (fault, ARBITRARY : AddressDescriptor, ARBITRARY : TTWState, ARBITRARY : bits(N));
    end;

    repeat
        fault.level = walkstate.level;
        walkaddress.paddress = descaddress;
        (fault, descriptor) = FetchDescriptor{N}(walkparams.ee, walkaddress, walkaccess, fault);

        if fault.statuscode != Fault\_None then
            return (fault, ARBITRARY : AddressDescriptor,
                    ARBITRARY : TTWState, ARBITRARY : bits(N));
        end;

        var new_descriptor : bits(N);
        repeat
            new_descriptor = descriptor;
            desctype = AArch64\_DecodeDescriptorType{N}(descriptor, walkparams.dl28, walkparams.ds,
                                                         walkparams.tgx, walkstate.level);

            case desctype of
                when DescriptorType\_Table =>
                    walkstate = AArch64\_S2NextWalkStateTable{N}(walkstate, walkparams, descriptor);
                    skl = if walkparams.dl28 == '1' then descriptor[110:109] else '00';
                    descaddress = AArch64\_S2TTEnterAddress(walkstate.level, walkparams, skl,
                                                            ipa.paddress.address,
                                                            walkstate.baseaddress);

                    // Detect Address Size Fault by table descriptor
                    if AArch64\_S2OAOutOfRange(descaddress.address, walkparams) then
                        fault.statuscode = Fault\_AddressSize;
                        return (fault, ARBITRARY : AddressDescriptor, ARBITRARY : TTWState,
                                ARBITRARY : bits(N));
                    end;
            end;
        repeat

```

```

end;

if walkparams.haft == '1' then
    new_descriptor[10] = '1';
end;

if (walkparams.d128 == '1' && skl != '00' &&
    AArch64_nTFaults{N}(walkparams.d128, descriptor)) then
    fault.statuscode = Fault_Translation;
    return (fault, ARBITRARY : AddressDescriptor, ARBITRARY : TTWState,
        ARBITRARY : bits(N));
end;

when DescriptorType_Leaf =>
    walkstate = AArch64_S2NextWalkStateLeaf{N}(walkstate, accdesc.ss, walkparams,
        ipa, descriptor);

when DescriptorType_Invalid =>
    fault.statuscode = Fault_Translation;
    return (fault, ARBITRARY : AddressDescriptor,
        ARBITRARY : TTWState, ARBITRARY : bits(N));

otherwise =>
    unreachable;
end;

if new_descriptor != descriptor then
    if
        let (!descaccess : IsWBShareable(walkaddress.memattrs) &&
            ConstrainUnpredictableBool(Unpredictable_Unsupported_Atomic_HW_Update)) then
            fault.statuscode = Fault_HWUpdateAccessFlag;
            return (fault, ARBITRARY : AddressDescriptor, ARBITRARY : TTWState,
                ARBITRARY : bits(N));
        end;

    let descaccess : AccessDescriptor = CreateAccDescTTEUpdate(accdesc);
    (fault, descriptor) = AArch64_MemSwapTableDesc{N}(fault, descriptor, new_descriptor,
        walkparams.ee, descaccess,
        walkaddress);

    if fault.statuscode != Fault_None then
        return (fault, ARBITRARY : AddressDescriptor,
            ARBITRARY : TTWState, ARBITRARY : bits(N));
    end;

    end;
    until new_descriptor == descriptor looplimit Unbounded_DescriptorUpdate;
until desctype == DescriptorType_Leaf looplimit MAX_WALK_LEVELS;

let oa : FullAddress = StageOA(ipa_64, walkparams.d128, walkparams.tgx, walkstate);
if (walkstate.contiguous == '1' &&
    AArch64_ContiguousBitFaults(walkparams.d128, walkparams.txsz, walkparams.tgx,
        walkstate.level)) then
    fault.statuscode = Fault_Translation;
elseif walkstate.level < FINAL_LEVEL && AArch64_nTFaults{N}(walkparams.d128, descriptor) then
    fault.statuscode = Fault_Translation;
// Detect Address Size Fault by final output
elseif AArch64_S2OAOutOfRange(oa.address, walkparams) then
    fault.statuscode = Fault_AddressSize;
// Check descriptor AF bit
elseif (descriptor[10] == '0' && walkparams.ha == '0' &&
    (!accdesc.acctype IN {AccessType_DC, AccessType_IC} ||
        ImpDefBool("Generate access flag fault on IC/DC operations"))) then
    fault.statuscode = Fault_AccessFlag;
end;

return (fault, walkaddress, walkstate, descriptor);
end;

```

Library pseudocode for shared/debug/SoftwareLockStatus/Component

```

// Component
// =====
// Component Types.

type Component of enumeration {
    Component_AMUComponent_ETE,
    Component_ETEComponent_TRBE,
    Component_TRBEComponent_RAS,
    Component_RASComponent_GIC,
    Component_GICComponent_PMU,
    Component_PMUComponent_Debug,
    Component_Debug, Component_CTI
};
Component_CTI
};

```

Library pseudocode for shared/debug/SoftwareLockStatus/GetAccessComponent

```
// GetAccessComponent()
// =====
// Determines>Returns the component accessed by decoding the physical address in the address descriptor.component.

impdef func GetAccessComponent(addrdesc() => AddressDescriptor) => Component
begin
    return Component_Debug;
end;
```

Library pseudocode for shared/debug/SoftwareLockStatus/SoftwareLockStatus

```
// SoftwareLockStatus()
// =====
// Returns the state of the Software Lock.

func SoftwareLockStatus(addrdesc() => boolean
begin
    let component : AddressDescriptor => boolean
begin
    let component : Component = GetAccessComponent(addrdesc);()
    if !HaveSoftwareLock(component) then
        return FALSE;
    end;
    case component of
        when Component_ETE => return TRCLSR().SLK == '1';
        when Component_Debug => return EDLSR().SLK == '1';
        when Component_PMU => return PMLSR().SLK == '1';
        when Component_CTI => return CTILSR().SLK == '1';
        otherwise => return FALSE;
    end;
end;
```

Library pseudocode for shared/debug/authentication/AllowExternalDebugAccess

```
// AllowExternalDebugAccess()
// =====
// Returns TRUE if an external debug interface access to the External DBGBCR<DBGBCR_EL1(n>_EL1),
// DBGBCR<DBGBCR_EL1(n>_EL1), DBGWVR<DBGWVR_EL1(n>_EL1), DBGWCR<DBGWCR_EL1(n>_EL1) registers, and, from Armv8.2, the
// OSLAR_EL1 register is allowed for the access. Returns FALSE otherwise.

func AllowExternalDebugAccess(addrdesc : AddressDescriptor) => boolean
begin
    // The access may also be subject to OS Lock, power-down, etc.
    if IsFeatureImplemented(FEAT_RME) then
        case MDCR_EL3().[EDADE,EDAD] of
            when '00' => return TRUE;
            when '01' => return addrdesc.paddress.paspace IN {PAS_Root, PAS_Secure};
            when '10' => return addrdesc.paddress.paspace IN {PAS_Root, PAS_Realm};
            when '11' => return addrdesc.paddress.paspace == PAS_Root;
        end;
    end;

    if IsFeatureImplemented(FEAT_Debugv8p4) then
        if addrdesc.paddress.paspace == PAS_Secure then return TRUE; end;
    else
        if !ExternalInvasiveDebugEnabled() then return FALSE; end;
        if ExternalSecureInvasiveDebugEnabled() then return TRUE; end;
    end;

    if HaveEL(EL3) then
        let EDAD_bit : bit = if ELUsingAArch32(EL3) then SDCR().EDAD else MDCR_EL3().EDAD;
        return EDAD_bit == '0';
    else
        return NonSecureOnlyImplementation();
    end;
end;
```

Library pseudocode for shared/debug/authentication/ExternalInvasiveDebugEnabled

```
// ExternalInvasiveDebugEnabled()
// =====
// The definition of this function is IMPLEMENTATION DEFINED.
// In the recommended interface, this function returns the state of the DBGEN signal.

readonly func ExternalInvasiveDebugEnabled() => boolean
begin
    return DBGEN == HIGH;
end;
```

Library pseudocode for shared/debug/authentication/ExternalNoninvasiveDebugEnabled

```
// ExternalNoninvasiveDebugEnabled()
// =====
// This function returns TRUE if the FEAT_Debugv8p4 is implemented.
// Otherwise, this function is IMPLEMENTATION DEFINED, and, in the
// recommended interface, ExternalNoninvasiveDebugEnabled returns
// the state of the (DBGEN OR NIDEN) signal.

readonly func ExternalNoninvasiveDebugEnabled() => boolean
begin
    return (IsFeatureImplemented(FEAT_Debugv8p4) || ExternalInvasiveDebugEnabled() ||
            NIDEN == HIGH);
end;
```

Library pseudocode for shared/debug/authentication/ExternalRealmInvasiveDebugEnabled

```
// ExternalRealmInvasiveDebugEnabled()
// =====
// The definition of this function is IMPLEMENTATION DEFINED.
// In the recommended interface, this function returns the state of the
// (DBGEN AND RLPIDEN) signal.

readonly func ExternalRealmInvasiveDebugEnabled() => boolean
begin
    if !IsFeatureImplemented(FEAT_RME) then return FALSE; end;
    return ExternalInvasiveDebugEnabled() && RLPIDEN == HIGH;
end;
```

Library pseudocode for shared/debug/authentication/ExternalRealmNoninvasiveDebugEnabled

```
// ExternalRealmNoninvasiveDebugEnabled()
// =====
// The definition of this function is IMPLEMENTATION DEFINED.
// In the recommended interface, this function returns the state of the
// (DBGEN AND RLPIDEN) signal.

readonly func ExternalRealmNoninvasiveDebugEnabled() => boolean
begin
    if !IsFeatureImplemented(FEAT_RME) then return FALSE; end;
    return ExternalRealmInvasiveDebugEnabled();
end;
```

Library pseudocode for shared/debug/authentication/ExternalSecureInvasiveDebugEnabled

```
// ExternalSecureInvasiveDebugEnabled()
// =====
// The definition of this function is IMPLEMENTATION DEFINED.
// In the recommended interface, this function returns the state of the (DBGEN AND SPIDEN) signal.
// CoreSight allows asserting SPIDEN without also asserting DBGEN, but this is not recommended.

readonly func ExternalSecureInvasiveDebugEnabled() => boolean
begin
    if !HaveSecureState() then
        return (!IsFeatureImplemented(FEAT_RME) FALSE, || RTPIDEN == HIGH);
    end;
    return ExternalInvasiveDebugEnabled() && SPIDEN == HIGH;
end;
```

Library pseudocode for shared/debug/authentication/ExternalSecureNoninvasiveDebugEnabled

```
// ExternalSecureNoninvasiveDebugEnabled()
// =====
// This function returns the value of ExternalSecureInvasiveDebugEnabled() when FEAT_Debugv8p4
// is implemented. Otherwise, the definition of this function is IMPLEMENTATION DEFINED.
// In the recommended interface, this function returns the state of the (DBGEN OR NIDEN) AND
// (SPIDEN OR SPNIDEN) signal.

readonly func ExternalSecureNoninvasiveDebugEnabled() => boolean
begin
    if !HaveSecureState() then return FALSE; end;
    if !IsFeatureImplemented(FEAT_Debugv8p4) then
        return (ExternalNoninvasiveDebugEnabled() &&
                (SPIDEN == HIGH || SPNIDEN == HIGH));
    else
        return ExternalSecureInvasiveDebugEnabled();
    end;
end;
```

Library pseudocode for shared/debug/halting/ExitDebugState

```
// ExitDebugState()
// =====

func ExitDebugState()
begin
    assert Halted();
    SynchronizeContext();

    // Although EDSCR.STATUS signals that the PE is restarting, debuggers must use EDPRSR.SDR to
    // detect that the PE has restarted.
    EDSCR().STATUS = '000001'; // Signal restarting
    // Clear any pending Halting debug events
    if IsFeatureImplemented(FEAT_Debugv8p8) then
        EDESR()[3:0] = '0000';
    else
        EDESR()[2:0] = '000';
    end;

    var new_pc : bits(64);
    var spsr : bits(64);

    if UsingAArch32() then
        new_pc = ZeroExtend{64}(DLR());
        if IsFeatureImplemented(FEAT_Debugv8p9) then
            spsr = DSPSR2() :: DSPSR();
        else
            spsr = ZeroExtend{64}(DSPSR());
        end;
    else
        new_pc = DLR_EL0();
        spsr = DSPSR_EL0();
    end;

    let illegal_psr_state : boolean = IllegalExceptionReturn{64}(spsr);
    // If this is an illegal return, SetPSTATEFromPSR() will set PSTATE.IL.
    SetPSTATEFromPSR{64}(spsr); // Can update privileged bits, even at EL0

    let branch_conditional : boolean = FALSE;
    if UsingAArch32() then
        if ConstrainUnpredictableBool(Unpredictable_RESTARTALIGNPC) then new_pc[0] = '0'; end;
        // AArch32 branch
        BranchTo{32}(new_pc[31:0], BranchType_DBGEXIT, branch_conditional);
    else
        // If targeting AArch32 then PC[63:32,1:0] might be set to UNKNOWN.
        if illegal_psr_state && spsr[4] == '1' then
            new_pc[63:32] = ARBITRARY : bits(32);
            new_pc[1:0] = ARBITRARY : bits(2);
        end;
        if IsFeatureImplemented(FEAT_BRBE) then
            BRBEDebugStateExit(new_pc);
        end;
        // A type of branch that is never predicted
        BranchTo{64}(new_pc, AArch64_BranchAddr(new_pc, PSTATE.EL));
    end;
    // A type of branch that is never predicted
    BranchTo{64}(new_pc, BranchType_DBGEXIT, branch_conditional);
end;

    // Atomically signal restarted
    EDSCR().STATUS = '000010';
    EDPRSR().SDR = '1';
    // End of atomically signal
    EDPRSR().HALTED = '0';
    UpdateEDSCRFields(); // Stop signalling PE state
    DisableITRAndResumeInstructionPrefetch();

    return;
end;
```



```

// Halt()
// =====

func Halt(reason : bits(6))
begin
    let is_async : boolean = FALSE;
    let fault : FaultRecord = NoFault();
    Halt(reason, is_async, fault);
end;

// Halt()
// =====

func Halt(reason : bits(6), is_async : boolean, fault : FaultRecord)
begin
    CTI_SignalEvent(CrossTriggerIn CrossHalt); // Trigger other cores to halt

    let preferred_restart_address : bits(64) = ThisInstrAddr{}();
    var spsr : bits(64) = GetPSRFromPSTATE{}() GetPSRFromPSTATE{64}(DebugState);

    if (IsFeatureImplemented(FEAT_BTI) && !is_async &&
        ! reason IN {DebugHalt Step Normal, DebugHalt Step Exclusive,
                     DebugHalt Step NoSyndrome, DebugHalt Breakpoint,
                     DebugHalt HaltInstruction} &&
        ConstrainUnpredictableBool(Unpredictable ZEROBTYPE)) then
        spsr[11:10] = '00';
    end;

    if UsingAArch32() then
        DLR() = preferred_restart_address[31:0];
        DSPSR() = spsr[31:0];
        if IsFeatureImplemented(FEAT_Debugv8p9) then
            DSPSR2() = spsr[63:32];
        end;
    else
        DLR_EL0() = preferred_restart_address;
        DSPSR_EL0() = spsr;
    end;
    EDSCR().ITE = '1';
    EDSCR().ITO = '0';
    if IsFeatureImplemented(FEAT_RME) then
        if PSTATE.EL == PSTATE.EL == EL3 then
            EDSCR().SDD = '0';
        else
            EDSCR().SDD = if ExternalRootInvasiveDebugEnabled() then '0' else '1';
        end;
    elseif CurrentSecurityState() == SS Secure then
        EDSCR().SDD = '0'; // If entered in Secure state, allow debug
    elseif HaveEL(EL3) then
        EDSCR().SDD = if ExternalSecureInvasiveDebugEnabled() then '0' else '1';
    else
        EDSCR().SDD = '1'; // Otherwise EDSCR.SDD is RES1
    end;
    EDSCR().MA = '0';

    // In Debug state:
    // * PSTATE.[SS,SSBS,D,A,I,F] are not observable and ignored so behave-as-if UNKNOWN.
    // * PSTATE.[N,Z,C,V,Q,GE,E,M,nRW,EL,SP,DIT] are also not observable, but since these
    //   are not changed on exception entry, this function also leaves them unchanged.
    // * PSTATE.[IT,T] are ignored.
    // * PSTATE.IL is ignored and behave-as-if 0.
    // * PSTATE.BTYPE is ignored and behave-as-if 0.
    // * PSTATE.TCO is set 1.
    // * PSTATE.PACM is ignored and behave-as-if 0.
    // * PSTATE.[UAO,PAN] are observable and not changed on entry into Debug state.
    // * PSTATE.UINJ is set to 0.

    if UsingAArch32() then
        PSTATE.\[IT,SS,SSBS,A,I,F,T\] = ARBITRARY : bits\(14\);
    else
        PSTATE.\[SS,SSBS,D,A,I,F\] = ARBITRARY : bits\(6\);
    end;

    if IsFeatureImplemented(FEAT_MTE) then PSTATE.TCO = '1'; end;
    if IsFeatureImplemented(FEAT_BTI) then PSTATE.BTYPE = '00'; end;
    if IsFeatureImplemented(FEAT_PAuth_LR) then PSTATE.PACM = '0'; end;
    PSTATE.IL = '0';
    if IsFeatureImplemented(FEAT_UINJ) then PSTATE.UINJ\(\) then
        PSTATE.\[IT,SS,SSBS,A,I,F,T\] = ARBITRARY : bits\(14\);
    else
        PSTATE.\[SS,SSBS,D,A,I,F\] = ARBITRARY : bits\(6\);
    end;

    if IsFeatureImplemented(FEAT_MTE) then PSTATE.TCO = '1'; end;
    if IsFeatureImplemented(FEAT_BTI) then PSTATE.BTYPE = '00'; end;
    if IsFeatureImplemented(FEAT_PAuth_LR) then PSTATE.PACM = '0'; end;
    PSTATE.IL = '0';
    if IsFeatureImplemented(FEAT_UINJ) then PSTATE.UINJ = '0'; end;
    if IsFeatureImplemented(FEAT_BRBE) then

```

```

    BRBEDebugStateEntry(preferred_restart_address);
end;
StopInstructionPrefetchAndEnableITR();
// atomic write
EDSCR().STATUS = reason;
EDPRSR().HALTED = '1';
// end of atomic write
UpdateEDSCRFields(); // Update EDSCR PE state flags.
if IsFeatureImplemented(FEAT_EDHSR) then
    UpdateEDHSR(reason, fault); // Update EDHSR fields.
end;
if !is_async then EndOfInstruction(); end;
return;
end;

```


Library pseudocode for shared/debug/halting/UpdateEDSCRFields

```
// UpdateEDSCRFields()
// =====
// Update EDSCR PE state fields

func UpdateEDSCRFields()
begin
    if !Halted() then
        EDSCR().EL = '00';
        if IsFeatureImplemented(FEAT_RME) then
            // SDD bit.
            EDSCR().SDD = if ExternalRootInvasiveDebugEnabled() then '0' else '1';
            EDSCR().[NSE,NS] = ARBITRARY : bits(2);
        else
            // SDD bit.
            EDSCR().SDD = if ExternalSecureInvasiveDebugEnabled() then '0' else '1';
            EDSCR().NS = ARBITRARY : bit;
        end;

        EDSCR().RW = '1111';
    else
        EDSCR().EL = PSTATE.EL;
        // SError Pending.
        if PSTATE.EL;
        // SError Pending.
        if EL2Enabled() && HCR_EL2().[AMO,TGE] == '10' && PSTATE.EL IN { PSTATE.EL IN {EL0,EL1} then
            EDSCR().A = if IsVirtualErrorPending() then '1' else '0';
        elsif (IsFeatureImplemented(FEAT_E3DSE) && SCR_EL3().EndSE == '1' &&
            PSTATE.EL IN {EffectiveSCR_EL3_EndSE() == '1' && PSTATE.EL IN {EL0,EL1,EL2}} then
            EDSCR().A = if IsDelegatedSErrorPending() then '1' else '0';
        else
            EDSCR().A = if IsPhysicalErrorPending() then '1' else '0';
        end;

        let ss : SecurityState = CurrentSecurityState();
        if IsFeatureImplemented(FEAT_RME) then
            case ss of
                when SS_Secure => EDSCR().[NSE,NS] = '00';
                when SS_NonSecure => EDSCR().[NSE,NS] = '01';
                when SS_Root => EDSCR().[NSE,NS] = '10';
                when SS_Realm => EDSCR().[NSE,NS] = '11';
            end;
        else
            EDSCR().NS = if ss == SS_Secure then '0' else '1';
        end;

        var RW : bits(4);
        RW[1] = if ELUsingAArch32(EL1) then '0' else '1';
        if PSTATE.EL then '0' else '1';
        if PSTATE.EL != EL0 then
            RW[0] = RW[1];
        else
            RW[0] = if UsingAArch32() then '0' else '1';
        end;
        if !EL2Enabled() then
            RW[2] = RW[1];
        else
            RW[2] = if ELUsingAArch32(EL2) then '0' else '1';
        end;
        if !HaveEL(EL3) then
            RW[3] = RW[2];
        else
            RW[3] = if ELUsingAArch32(EL3) then '0' else '1';
        end;

        // The least-significant bits of EDSCR.RW are UNKNOWN if any higher EL is using AArch32. AArch32
        if RW[3] == '0' then RW[2:0] = ARBITRARY : bits(3);
        elsif RW[2] == '0' then RW[1:0] = ARBITRARY : bits(2);
        elsif RW[1] == '0' then RW[0] = ARBITRARY : bit; end;
        EDSCR().RW = RW;
    end;
    return;
end;
```

Library pseudocode for shared/debug/pmu

```
var PMUEventAccumulator : array [[31]] of integer; // Accumulates PMU events for a cycle

var PMULastThresholdValue : array [[31]] of boolean; // A record of the threshold result for each

// Constant used in PMU functions to represent actions on the cycle counter.
constant CYCLE_COUNTER_ID : integer = 31;

// Constant used in PMU functions to represent actions on the instruction counter.
constant INSTRUCTION_COUNTER_ID : integer = 32;
```

Library pseudocode for shared/functions/aborts/EncodeLDFSC

```
// EncodeLDFSC()
// =====
// Function that gives the Long-descriptor FSC code for types of Fault

func EncodeLDFSC(statuscode : Fault, level : integer) => bits(6)
begin
    var result : bits(6);

    // 128-bit descriptors will start from level -2 for 4KB to resolve bits IA[55:51]
    if level == -2 then
        assert IsFeatureImplemented(FEAT_D128);
        case statuscode of
            when Fault AddressSize => result = '101100';
            when Fault Translation => result = '101010';
            when Fault SyncExternalOnWalk => result = '010010';
            when Fault SyncParityOnWalk =>
                result = '011010';
                assert !IsFeatureImplemented(FEAT_RAS);
            when Fault GPCFOnWalk => result = '100010';
            otherwise => unreachable;
        end;
        return result;
    end;

    if level == -1 then
        assert IsFeatureImplemented(FEAT_LPA2);
        case statuscode of
            when Fault AddressSize => result = '101001';
            when Fault Translation => result = '101011';
            when Fault SyncExternalOnWalk => result = '010011';
            when Fault SyncParityOnWalk =>
                result = '011011';
                assert !IsFeatureImplemented(FEAT_RAS);
            when Fault GPCFOnWalk => result = '100011';
            otherwise => unreachable;
        end;

        return result;
    end;

    case statuscode of
        when Fault AddressSize => result = '0000'::level[1:0]; assert level IN {0,1,2,3};
        when Fault AccessFlag => result = '0010'::level[1:0]; assert level IN {0,1,2,3};
        when Fault Permission => result = '0011'::level[1:0]; assert level IN {0,1,2,3};
        when Fault Translation => result = '0001'::level[1:0]; assert level IN {0,1,2,3};
        when Fault AsyncExternal => result = '010001'; assert UsingAArch32();
        when Fault SyncExternal => result = '010000';
        when Fault SyncExternalOnWalk => result = '0101'::level[1:0]; assert level IN {0,1,2,3};
        when Fault SyncParity => result = '011000';
        when Fault SyncParityOnWalk => result = '0111'::level[1:0]; assert level IN {0,1,2,3};
        when Fault AsyncParity => result = '011001';
        when Fault AsyncExternal => result = '010001'; assert UsingAArch32 => result = '011001'; ();
        when Fault TagCheck => result = '010001'; assert IsFeatureImplemented(FEAT_MTE2);
        when Fault Alignment => result = '100001';
        when Fault Debug => result = '100010';
        when Fault GPCFOnWalk => result = '1001'::level[1:0]; assert level IN {0,1,2,3};
        when Fault GPCFOnOutput => result = '101000';
        when Fault TLBConflict => result = '110000';
        when Fault HWUpdateAccessFlag => result = '110001';
        when Fault Lockdown => result = '110100'; // IMPLEMENTATION DEFINED
        when Fault Exclusive => result = '110101'; // IMPLEMENTATION DEFINED
        otherwise => unreachable;
    end;

    return result;
end;
```

Library pseudocode for shared/functions/extension/HaveTraceExt

```
// HaveTraceExt()
// =====
// Returns TRUE if Trace functionality as described by the Trace Architecture
// is implemented.

readonly func HaveTraceExt() => boolean
begin
    return IsFeatureImplemented(FEAT_ETE) || IsFeatureImplemented(FEAT_ETMv4);
end;
```

Library pseudocode for shared/functions/memory/CreateAccDescDCZero

```
// CreateAccDescDCZero()
// =====
// Access descriptor for data cache zero operations

func CreateAccDescDCZero(cachetype : CacheType) => AccessDescriptor
begin
    var accdesc : AccessDescriptor = NewAccDesc(AccessType\_DCZero);

    accdesc.write           = TRUE;
    accdesc.pan             = TRUE;
    accdesc.tagchecked      = cachetype == CacheType\_Data;
    accdesc.tagaccess       = cachetype IN {CacheType\_Tag,
CacheType\_TagWrite,
CacheType\_TagZero,
CacheType\_Data\_Tag};

    accdesc.cachetype       = cachetype;

    return accdesc;
end;
```

Library pseudocode for shared/functions/memory/IsContiguousSVEAccess

```
// IsContiguousSVEAccess()
// =====
// Return TRUE if memory access is contiguous load/stores in an SVE mode.

func IsContiguousSVEAccess(accdesc : AccessDescriptor) => boolean
begin
    return (IsFeatureImplemented(FEAT_SVE) &&
accdesc.acctype == AccessType\_SVE && accdesc.contiguous;);
end;
```

Library pseudocode for shared/functions/memory/IsSMEAccess

```
// IsSMEAccess()
// =====
// Return TRUE if access is of SME load/stores.

func IsSMEAccess(accdesc : AccessDescriptor) => boolean
begin
    return IsFeatureImplemented(FEAT_SME) && accdesc.acctype == AccessType\_SME;
end;
```

Library pseudocode for shared/functions/mpam/ReportVIDFault

```
// ReportVIDFault()
// =====
// Report a GPCF or External abort on an MVMS or MITT access.

func ReportVIDFault(mpamdata : MPAMdata, el : bits(2),
                    fault_address : bits(56), fsc : bits(2))
begin
    assert el == '0x';
    MPAMVIDSR_EL2().FADDR = fault_address;
    MPAMVIDSR_EL2().FSC   = fsc;
MPAMVIDSR_EL2().FIR = if mpamdata.sm then '10' else el;
end;
```

Library pseudocode for shared/functions/sysregisters/SPSR_ELx

```
// SPSR_ELx - accessor
// =====

accessor SPSR_ELx() <=> value : bits(64)
begin
  getter
    var result : bits(64);
    case PSTATE.EL of
      when EL1 => result = SPSR_EL1()[63:0];
      when EL2 => result = SPSR_EL2()[63:0];
      when EL3 => result = SPSR_EL3()[63:0];
      otherwise => unreachable;
    end;
    return result;
  end;

  setter
    case PSTATE.EL => result = SPSR_EL3()[63:0];
    otherwise => unreachable;
    end;
    return result;
  end;
  setter
    case PSTATE.EL of
      when EL1 => SPSR_EL1()[63:0] = value[63:0];
      when EL2 => SPSR_EL2()[63:0] = value[63:0];
      when EL3 => SPSR_EL3()[63:0] = value[63:0];
      otherwise => unreachable;
    end;
    return;
  end;
end;
```

Library pseudocode for shared/functions/system/ELStateUsingAArch32K

```
// ELStateUsingAArch32K()
// =====
// Returns (known, aarch32):
//   'known' is FALSE for EL0 if the current Exception level is not EL0 and EL1 is
//   using AArch64, since it cannot determine the state of EL0; TRUE otherwise.
//   'aarch32' is TRUE if the specified Exception level is using AArch32; FALSE otherwise.

readonly func ELStateUsingAArch32K(el : bits(2), secure : boolean) => (boolean, boolean)
begin
    assert HaveEL(el);

    if !HaveAArch32EL(el) then
        return (TRUE, FALSE); // Exception level is using AArch64
    elsif secure && el == EL2 then
        return (TRUE, FALSE); // Secure EL2 is using AArch64
    elsif !HaveAArch64() then
        return (TRUE, TRUE); // Highest Exception level, therefore all levels are using AArch32
    end;

    // Remainder of function deals with the interprocessing cases when highest
    // Exception level is using AArch64.AArch64

    if el == EL3 then
        return (TRUE, FALSE);
    end;

    if (HaveEL(EL3) && SCR_EL3().RW == '0' &&
        (!secure || !IsFeatureImplemented(FEAT_SEL2) || SCR_EL3().EEL2 == '0')) then
        // AArch32 below EL3.
        return (TRUE, TRUE);
    end;

    if el == EffectiveSCR_EL3_RW() == '0' &&
        (!secure || !IsFeatureImplemented(FEAT_SEL2) || SCR_EL3().EEL2 == '0')) then
        // AArch32 below EL3.
        return (TRUE, TRUE);
    end;

    if el == EL2 then
        return (TRUE, FALSE);
    end;

    if (HaveEL(EL2) && !ELIsInHost(EL0) && HCR_EL2().RW == '0' &&
        (!secure || (IsFeatureImplemented(FEAT_SEL2) && SCR_EL3().EEL2 == '1')))) then
        // AArch32 below EL2.
        return (TRUE, TRUE);
    end;

    if el == EL1 then
        return (TRUE, FALSE);
    end;

    // The execution state of EL0 is only known from PSTATE.nRW when executing at EL0.
    if PSTATE.EL == PSTATE.EL == EL0 then
        return (TRUE, PSTATE.nRW);
    else
        return (TRUE, PSTATE.nRW == '1');
    end;
end;
```

Library pseudocode for shared/functions/system/EffectiveEA

```
// EffectiveEA()
// =====
// Returns effective SCR_EL3.EA value

readonly func EffectiveEA() => bit
begin
    if !HaveEL(EL3) || Halted() then
        return '0';
    else
        return if HaveAArch64() then SCR_EL3().EA else SCR().EA;
    end;
end;
```

Library pseudocode for shared/functions/system/EffectiveHCR_EL2_NVx

```
// EffectiveHCR_EL2_NVx()
// =====
// Return the Effective value of HCR_EL2.{HCR_EL2().NV2,NV1,NV}.

func EffectiveHCR_EL2_NVx() => bits(3)
begin
    if !EL2Enabled() || !IsFeatureImplemented(FEAT_NV) then
        return '000';
    end;

    var nv1 : bit = HCR_EL2().NV1;
    if (!IsFeatureImplemented(FEAT_E2H0) &&
        ImpDefBool("HCR_EL2.NV1 is implemented as RAZ")) then
        nv1 = '0';
    end;

    if HCR_EL2().NV == '0' then
        if nv1 == '1' then
            case ConstrainUnpredictable(Unpredictable NVNV1) of
                when Constraint NVNV1_00 => return '000';
                when Constraint NVNV1_01 => return '010';
                when Constraint NVNV1_11 => return '011';
            end;
        else
            return '000';
        end;
    end;

    if !IsFeatureImplemented(FEAT_NV2) then
        return '0' :: nv1 :: '1';
    end;

    var nv2 : bit = HCR_EL2().NV2;
    if (nv2 == '0' && ImpDefBool(
        "Programming HCR_EL2.<NV,NV2> to '10' behaves as '11'")) then
        nv2 = '1';
    end;

    return nv2 :: nv1 :: '1';
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_FIQ

```
// EffectiveSCR_EL3_FIQ()
// =====
// Returns effective SCR_EL3().FIQ value

readonly func EffectiveSCR_EL3_FIQ() => bit
begin
    if !HaveEL(EL3) || ELUsingAArch32(EL3) then
        return '0';
    end;
    if IsFeatureImplemented(FEAT_GCIE) && IsFeatureImplemented(FEAT_EL3) then
        return '1';
    end;
    return SCR_EL3().FIQ;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_IRQ

```
// EffectiveSCR_EL3_IRQ()
// =====
// Returns effective SCR_EL3().IRQ value

readonly func EffectiveSCR_EL3_IRQ() => bit
begin
    if !HaveEL(EL3) || ELUsingAArch32(EL3) then
        return '0';
    end;
    if IsFeatureImplemented(FEAT_GCIE) then
        return '0';
    end;
    return SCR_EL3().IRQ;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_RW

```
// EffectiveSCR_EL3_RW()
// =====
// Returns effective SCR_EL3.RW value

readonly func EffectiveSCR_EL3_RW() => bit
begin
    if !HaveAArch64() then
        return '0';
    end;
    if !HaveAArch32EL(EL2) && !HaveAArch32EL(EL1) then
        return '1';
    end;
    if HaveAArch32EL(EL1) then
        if !HaveAArch32EL(EL2) && EffectiveSCR_EL3_NS() == '1' then
            return '1';
        end;
        if (IsFeatureImplemented(FEAT_SEL2) && SCR_EL3().EEL2 == '1' &&
            EffectiveSCR_EL3_NS() == '0') then
            return '1';
        end;
    end;
    return SCR_EL3().RW;
end;
```

Library pseudocode for shared/functions/system/IllegalExceptionReturn

```
// IllegalExceptionReturn()
// =====

func IllegalExceptionReturn(N)(spsr : bits(N)) => boolean
begin
    // Check for illegal return:
    // * To an unimplemented Exception level.
    // * To EL2 in Secure state, when SecureEL2 is not enabled.
    // * To EL0 using AArch64 state, with SPSR.M[0]==1.
    // * To AArch64 state with SPSR.M[1]==1.
    // * To AArch32 state with an illegal value of SPSR.M.
    let (valid, target) : (boolean, bits(2)) = ELFromSPSR{N}(spsr);
    if !valid then return TRUE; end;

    // Check for return to higher Exception level.
    if UInt(target) > UInt(PSTATE.EL) then return TRUE; end;

    let spsr_mode_is_aarch32 : boolean = (spsr[4] == '1');

    // Check for illegal return:
    // * To EL1, EL2 or EL3 with register width specified in the SPSR different from the
    //   Execution state used in the Exception level being returned to, as determined by
    //   the SCR_EL3.RW or HCR_EL2.RW bits, or as configured from reset.
    // * To EL0 using AArch64 state when EL1 is using AArch32 state as determined by the
    //   SCR_EL3.RW or HCR_EL2.RW bits or as configured from reset.
    // * To AArch64 state from AArch32 state (should be caught by above).
    let (known, target_el_is_aarch32) : (boolean, boolean) = PSTATE.EL then return TRUE; end;

    let spsr_mode_is_aarch32 : boolean = (spsr[4] == '1');

    // Check for illegal return:
    // * To EL1, EL2 or EL3 with register width specified in the SPSR different from the
    //   Execution state used in the Exception level being returned to, as determined by
    //   the SCR_EL3.RW or HCR_EL2.RW bits, or as configured from reset.
    // * To EL0 using AArch64 state when EL1 is using AArch32 state as determined by the
    //   SCR_EL3.RW or HCR_EL2.RW bits or as configured from reset.
    // * To AArch64 state from AArch32 state (should be caught by above).
    let (known, target_el_is_aarch32) : (boolean, boolean) = ELUsingAArch32K(target);
    assert known || (target == EL0 && !ELUsingAArch32(EL1));
    if known && spsr_mode_is_aarch32 != target_el_is_aarch32 then return TRUE; end;

    // Check for illegal return from AArch32 to AArch64.AArch64
    if UsingAArch32() && !spsr_mode_is_aarch32 then return TRUE; end;
    // Check for illegal return to EL1 when HCR_EL2.TGE is set and when either of
    // * SecureEL2 is enabled.
    // * SecureEL2 is not enabled and EL1 is in Non-secure state.
    if EL2Enabled() && target == EL1 && HCR_EL2().TGE == '1' then
        if (!IsSecureBelowEL3() || IsSecureEL2Enabled()) then return TRUE; end;
    end;

    // Check for an illegal return to EL1 by a Guest Hypervisor at EL1 when HCRX_EL2.NVTGE is set.
    if (IsFeatureImplemented(FEAT_NV3) && PSTATE.EL == PSTATE.EL == EL1 && spsr[3:2] == '01' &&
        EffectiveHCR\_EL2\_NVx() == '101' && IsHCRXEL2Enabled() && HCRX_EL2().NVTGE == '1' &&
        NVHCR_EL2().TGE == '1') then
        return TRUE;
    end;

    if (IsFeatureImplemented(FEAT_GCS) && PSTATE.EXLOCK == '0' &&
        PSTATE.EL() && HCRX_EL2().NVTGE == '1' &&
        NVHCR_EL2().TGE == '1') then
        return TRUE;
    end;

    if (IsFeatureImplemented(FEAT_GCS) && PSTATE.EXLOCK == '0' &&
        PSTATE.EL == target && GetCurrentEXLOCKEN()) then
        return TRUE;
    end;

    return FALSE;
end;
```


Library pseudocode for shared/functions/system/PSTATE

```
// PSTATE
// =====
// PE state bits.
// There is no significance to the field order.

var PSTATE : collection {
    N : bits (1), // Negative condition flag
    Z : bits (1), // Zero condition flag
    C : bits (1), // Carry condition flag
    V : bits (1), // Overflow condition flag
    D : bits (1), // Debug mask bit [AArch64 only]
    A : bits (1), // SError interrupt mask bit
    I : bits (1), // IRQ mask bit
    F : bits (1), // FIQ mask bit
    EXLOCK : bits (1), // Lock exception return state
    PAN : bits (1), // Privileged Access Never Bit [v8.1]
    UAO : bits (1), // User Access Override [v8.2]
    DIT : bits (1), // Data Independent Timing [v8.4]
    TCO : bits (1), // Tag Check Override [v8.5, AArch64 only]
    PM : bits (1), // PMU exception Mask
    PPEND : bits (1), // synchronous PMU exception to be observed
    BTYPE : bits (2), // Branch Type [v8.5]
    PACM : bits (1), // PAC instruction modifier
    ZA : bits (1), // Accumulation array enabled [SME]
    SM : bits (1), // Streaming SVE mode enabled [SME]
    ALLINT : bits (1), // Interrupt mask bit
    UINJ : bits (1), // Undefined Exception Injection
    SS : bits (1), // Software step bit
    IL : bits (1), // Illegal Execution state bit
    EL : bits (2), // Exception level
    nRW : bits (1), // Execution state: 0=AArch64, 1=AArch32
    SP : bits (1), // Stack pointer select: 0=SP0, 1=SPx [AArch64 only]
    Q : bits (1), // Cumulative saturation flag [AArch32 only]
    GE : bits (4), // Greater than or Equal flags [AArch32 only]
    SSBS : bits (1), // Speculative Store Bypass Safe
    IT : bits (8), // If-then bits, RES0 in CPSR [AArch32 only]
    J : bits (1), // J bit, RES0 [AArch32 only, RES0 in SPSR and CPSR]
    T : bits (1), // T32 bit, RES0 in CPSR [AArch32 only]
    E : bits (1), // Endianness bit [AArch32 only]
    M : bits (5), // Mode field [AArch32 only]
};

PSTATE : collection {
    N : bits (1), // Negative condition flag
    Z : bits (1), // Zero condition flag
    C : bits (1), // Carry condition flag
    V : bits (1), // Overflow condition flag
    D : bits (1), // Debug mask bit [AArch64 only]
    A : bits (1), // SError interrupt mask bit
    I : bits (1), // IRQ mask bit
    F : bits (1), // FIQ mask bit
    EXLOCK : bits (1), // Lock exception return state
    PAN : bits (1), // Privileged Access Never Bit [v8.1]
    UAO : bits (1), // User Access Override [v8.2]
    DIT : bits (1), // Data Independent Timing [v8.4]
    TCO : bits (1), // Tag Check Override [v8.5, AArch64 only]
    PM : bits (1), // PMU exception Mask
    PPEND : bits (1), // synchronous PMU exception to be observed
    BTYPE : bits (2), // Branch Type [v8.5]
    PACM : bits (1), // PAC instruction modifier
    ZA : bits (1), // Accumulation array enabled [SME]
    SM : bits (1), // Streaming SVE mode enabled [SME]
    ALLINT : bits (1), // Interrupt mask bit
    UINJ : bits (1), // Undefined Exception Injection
    SS : bits (1), // Software step bit
    IL : bits (1), // Illegal Execution state bit
    EL : bits (2), // Exception level
    nRW : bits (1), // Execution state: 0=AArch64, 1=AArch32
    SP : bits (1), // Stack pointer select: 0=SP0, 1=SPx [AArch64 only]
    Q : bits (1), // Cumulative saturation flag [AArch32 only]
    GE : bits (4), // Greater than or Equal flags [AArch32 only]
    SSBS : bits (1), // Speculative Store Bypass Safe
    IT : bits (8), // If-then bits, RES0 in CPSR [AArch32 only]
    J : bits (1), // J bit, RES0 [AArch32 only, RES0 in SPSR and CPSR]
    T : bits (1), // T32 bit, RES0 in CPSR [AArch32 only]
    E : bits (1), // Endianness bit [AArch32 only]
    M : bits (5) // Mode field [AArch32 only]
};
```

Library pseudocode for shared/functions/tlbi/ExcludeXS

```

// ExcludeXS()
// =====
// Returns TRUE if a TLBI maintenance instruction without the nXS qualifier executed at EL1
// behaves in the same way as the corresponding TLBI maintenance instruction with the nXS qualifier.
func ExcludeXS() => boolean
begin
    return (IsFeatureImplemented(FEAT_XS) && PSTATE.EL == EL1 &&
            IsHCRXEL2Enabled() && HCRX_EL2().FnXS == '1');
end;

```



```

// ConstrainUnpredictable()
// =====
// Return the appropriate Constraint result to control the caller's behavior.
// The return value is IMPLEMENTATION DEFINED within a permitted list for each
// UNPREDICTABLE case.
// (The permitted list is determined by an assert or case statement at the call site.)

readonly impdef func ConstrainUnpredictable(which : Unpredictable) => Constraint
begin
  case which of
    when Unpredictable VMSR =>
      return Constraint UNDEF;
    when Unpredictable WBOVERLAPLD =>
      return Constraint WBSUPPRESS; // return loaded value
    when Unpredictable WBOVERLAPST =>
      return Constraint NONE; // store pre-writeback value
    when Unpredictable LDPOVERLAP =>
      return Constraint UNDEF; // instruction is UNDEFINED
    when Unpredictable BASEOVERLAP =>
      return Constraint UNKNOWN; // use UNKNOWN address
    when Unpredictable DATAOVERLAP =>
      return Constraint UNKNOWN; // store UNKNOWN value
    when Unpredictable DEVPAGE2 =>
      return Constraint FAULT; // take an alignment fault
    when Unpredictable RESTCF =>
      return Constraint UNKNOWN; // Do not take a fault
    when Unpredictable DEVICETAGSTORE =>
      return Constraint NONE; // Do not take a fault
    when Unpredictable INSTRDEVICE =>
      return Constraint NONE; // Do not take a fault
    when Unpredictable RESCPACR =>
      return Constraint TRUE; // Map to UNKNOWN value
    when Unpredictable RESMAIR =>
      return Constraint UNKNOWN; // Map to UNKNOWN value
    when Unpredictable S1CTAGGED =>
      return Constraint FALSE; // SCTLRL_ELx.C == '0' marks address as untagged
    when Unpredictable S2RESMEMATTR =>
      return Constraint NC; // Map to Noncacheable value
    when Unpredictable RESTEXCB =>
      return Constraint UNKNOWN; // Map to UNKNOWN value
    when Unpredictable RESDACR =>
      return Constraint UNKNOWN; // Map to UNKNOWN value
    when Unpredictable RESPRRR =>
      return Constraint UNKNOWN; // Map to UNKNOWN value
    when Unpredictable RESVTCRS =>
      return Constraint UNKNOWN; // Map to UNKNOWN value
    when Unpredictable RESTnSZ =>
      return Constraint FORCE; // Map to the limit value
    when Unpredictable OORTnSZ =>
      return Constraint FORCE; // Map to the limit value
    when Unpredictable LARGEIPA =>
      return Constraint FORCE; // Restrict the IA size to the PAMax value
    when Unpredictable ESRCONDPASS =>
      return Constraint FALSE; // Report as "AL"
    when Unpredictable ILZEROIT =>
      return Constraint FALSE; // Do not zero PSTATE.IT
    when Unpredictable ILZEROT =>
      return Constraint FALSE; // Do not zero PSTATE.T
    when Unpredictable BPVECTORCATCHPRI =>
      return Constraint TRUE; // Debug Vector Catch: match on 2nd halfword
    when Unpredictable VCMATCHHALF =>
      return Constraint FALSE; // No match
    when Unpredictable VCMATCHDAPA =>
      return Constraint FALSE; // No match on Data Abort or Prefetch abort
    when Unpredictable WPMASKANDBAS =>
      return Constraint FALSE; // Watchpoint disabled
    when Unpredictable WPBASCONTIGUOUS =>
      return Constraint FALSE; // Watchpoint disabled
    when Unpredictable RESWPMASK =>
      return Constraint DISABLED; // Watchpoint disabled
    when Unpredictable WPMASKEDBITS =>
      return Constraint FALSE; // Watchpoint disabled
    when Unpredictable RESBPWPCTRL =>
      return Constraint DISABLED; // Breakpoint/watchpoint disabled
    when Unpredictable BPNOTIMPL =>
      return Constraint DISABLED; // Breakpoint disabled
    when Unpredictable RESBPTYPE =>
      return Constraint DISABLED; // Breakpoint disabled
    when Unpredictable BPNOTCTXCMP =>
      return Constraint DISABLED; // Breakpoint disabled
    when Unpredictable RESMDSELR =>
      return Constraint UNKNOWN; // Map to UNKNOWN value
    when Unpredictable BPMATCHHALF =>
      return Constraint FALSE; // No match
    when Unpredictable BPMISMATCHHALF =>
      return Constraint FALSE; // No match
    when Unpredictable BPLINKINGDISABLED =>
      return Constraint FALSE; // Breakpoint does not match
    when Unpredictable RESTARTALIGNPC =>

```

```

    return Constraint FALSE; // Do not force alignment
when Unpredictable RESTARTZEROUPPERPC =>
    return Constraint TRUE; // Force zero extension
when Unpredictable ZEROUPPER =>
    return Constraint TRUE; // zero top halves of X registers
when Unpredictable ERETZEROUPPERPC =>
    return Constraint TRUE; // zero top half of PC
when Unpredictable A32FORCEALIGNPC =>
    return Constraint FALSE; // Do not force alignment
when Unpredictable SMD =>
    return Constraint UNDEF; // disabled SMC is Unallocated
when Unpredictable NONFAULT =>
    return Constraint FALSE; // Speculation enabled
when Unpredictable SVEZEROUPPER =>
    return Constraint TRUE; // zero top bits of Z registers
when Unpredictable SVELDNFDATA =>
    return Constraint TRUE; // Load mem data in NF loads
when Unpredictable SVELDNFZERO =>
    return Constraint TRUE; // Write zeros in NF loads
when Unpredictable CHECKSPNONEACTIVE =>
    return Constraint TRUE; // Check SP alignment
when Unpredictable SMEZEROUPPER =>
    return Constraint TRUE; // zero top bits of ZA registers
when Unpredictable NVNV1 =>
    return Constraint NVNV1_00; // Map unpredictable configuration of HCR_EL2[NV,NV1]
                                // to NV = 0 and NV1 = 0
when Unpredictable BADDR RESS =>
    return Constraint RESS; // Values behave as RESS
when Unpredictable Shareability =>
    return Constraint OSH; // Map reserved encoding of shareability to Outer Shareable
when Unpredictable AFUPDATE => // AF update for alignment or Permission fault
    return Constraint TRUE;
when Unpredictable DBUPDATE => // DB State update for alignment fault
    return Constraint TRUE;
when Unpredictable Unsupported Atomic HW Update =>
    return Constraint FALSE; // Do not raise Unsupported Atomic HW Update fault
                                // when encountering unsupported memory attributes
when Unpredictable IESBinDebug => // Use SCTLr_ELx[].IESB in Debug state
    return Constraint TRUE;
when Unpredictable BADPMSFCR => // Bad settings for PMSFCR_EL1/PMSEVFR_EL1/PMSLATFR_EL1
    return Constraint TRUE;
when Unpredictable RES PMU VS => // Bad setting for SVE Streaming mode filter
    return Constraint FALSE;
when Unpredictable COUNT CHAIN => // Chained PMU counters idx, idx+1 are not in same range
    return Constraint FALSE;
when Unpredictable EL1TIMESTAMP => // Bad settings for TRFCR_EL1
    return Constraint EL1TIMESTAMP;
when Unpredictable EL2TIMESTAMP => // Bad settings for TRFCR_EL2
    return Constraint EL2TIMESTAMP;
when Unpredictable ZEROBTYP =>
    return Constraint TRUE; // Save BTYPE in SPSR_ELx/DPSR_ELO as '00'
when Unpredictable CLEARERRITEZERO => // Clearing sticky errors when instruction in flight
    return Constraint FALSE;
when Unpredictable ALUEXCEPTIONRETURN =>
    return Constraint UNDEF;
when Unpredictable DBGxVR RESS =>
    return Constraint FALSE;
when Unpredictable PMSCR PCT =>
    return Constraint PMSCR_PCT_VIRT;
when Unpredictable WFXTDEBUG =>
    return Constraint FALSE; // WFXT in Debug state does not execute as a NOP
// Accesses are not single-copy atomic above the byte level
when Unpredictable LS64UNSUPPORTED =>
    return Constraint LIMITED ATOMICITY;
// Unaligned exclusives, atomics, acquire/release
// to a region that is not to Normal inner write-back
// outer write-back shareable generate an Alignment fault.
when Unpredictable LSE2 ALIGNMENT FAULT =>
    return Constraint FALSE;
when Unpredictable LSE128OVERLAP =>
    return Constraint UNDEF; // instruction is UNDEFINED
when Unpredictable IGNORETRAPINDEBUG =>
    return Constraint FALSE; // Trap to register access in debug state is enabled
when Unpredictable PMUEVENTCOUNTER =>
    return Constraint UNDEF; // Accesses to the register are UNDEFINED
when Unpredictable RES HPMN =>
    return Constraint UNKNOWN; // The counter is reserved for EL2 use
when Unpredictable RES EPMN =>
    return Constraint UNKNOWN; // The counter is reserved for external use
when Unpredictable BRBFILTRATE =>
    return Constraint FALSE; // BRB_FILTRATE event not generated on BRB injection
when Unpredictable PMUSNAPSHOTEVENT => // PMU_SNAPSHOT event not generated in Debug state
    return Constraint FALSE;
when Unpredictable RESEPMSSAD =>
    // External access to PMUv3 Snapshot extension allowed in Secure state
    // when FEAT_RME is not implemented or Root state otherwise
    return Constraint UNKNOWN;
when Unpredictable RESPMSSSE =>
    return Constraint DISABLED; // PMU capture events disabled

```

```

when Unpredictable RESPMEE =>
    return Constraint DISABLED; // PMU Profiling exception disabled, PMUIRQ enabled
when Unpredictable RESPMSEE =>
    return Constraint DISABLED; // SPE Profiling exception disabled
when Unpredictable RESTRFEE =>
    return Constraint DISABLED; // TRBE Profiling exception disabled
when Unpredictable RESTC =>
    return Constraint DISABLED; // Threshold features disabled
when Unpredictable MOPSOVERLAP => // Instruction is UNDEFINED
    return Constraint UNDEF;
when Unpredictable MOPS R31 => // Instruction is UNDEFINED
    return Constraint UNDEF;
when Unpredictable CASRETURNOLDVALUE =>
    return Constraint TRUE;
when Unpredictable WRITEFAILED CAS =>
    return Constraint FALSE;
when Unpredictable STRONLYTAGCHECKED CAS =>
    return Constraint TRUE; // CAS with compare fail does a Tag Check
when Unpredictable STRONLYTAGCHECKED RCWSCAS =>
    return Constraint TRUE; // RCW(S)CAS with RCW(S) fail/compare fail does a Tag Check
when Unpredictable RESERVEDNSxB =>
    return Constraint MAPTOALLOCATED;
when Unpredictable RESERVEDNSxB Trap =>
    return Constraint TRUE;
when Unpredictable RES ETBAD =>
    // ExternalTracebufferaccess disabled for res values
    return Constraint DISABLED;
when Unpredictable RESBPMASK =>
    return Constraint DISABLED; // Mask set to 0
when Unpredictable BPMASK =>
    return Constraint FALSE; // Breakpoint match will fail
when Unpredictable BPMASKEDBITS =>
    return Constraint FALSE; // Breakpoint match will fail
when Unpredictable BPLINKEDADDRMATCH =>
    return Constraint FALSE; // Breakpoint match will fail
when Unpredictable 16BYTEROUNDEDUPACCESS =>
    // Watchpoint match of 16 byte rounded range in case of SVE access
    return Constraint TRUE;
when Unpredictable 16BYTEROUNDEDOWNACCESS =>
    // Watchpoint match of 16 byte rounded range in case of SVE access
    return Constraint TRUE;
when Unpredictable NODTRTAGCHK =>
    // The load or store operation does not perform the Tag check in Debug state
    return Constraint TRUE;
when Unpredictable CPACHECK => // Do not perform Checked Pointer Arithmetic
    return Constraint FALSE;
when Unpredictable Atomic SYNC ABORT =>
    return Constraint FALSE;
when Unpredictable Atomic ERROR =>
    return Constraint FALSE;
when Unpredictable Atomic NOP =>
    return Constraint FALSE;
when Unpredictable Atomic MMU IMPDEF FAULT =>
    return Constraint FALSE;
when Unpredictable MPAM TRUNCATE VID =>
    return Constraint TRUE;
when Unpredictable USE DEFAULT PMG =>
    return Constraint TRUE;
when Unpredictable BankedRegister =>
    return Constraint NOP;
when Unpredictable UnimplementedRegister =>
    return Constraint NOP;
otherwise =>
    unreachable;
end;
end;

```



```

// Unpredictable
// =====
// List of Constrained Unpredictable situations.

type Unpredictable of enumeration {
    // VMSR on MVFR
    Unpredictable_VMSR,
    // Writeback/transfer register overlap (load)
    Unpredictable_WBOVERLAPLD,
    // Writeback/transfer register overlap (store)
    Unpredictable_WBOVERLAPST,
    // Load Pair transfer register overlap
    Unpredictable_LDPOVERLAP,
    // Store-exclusive base/status register overlap
    Unpredictable_BASEOVERLAP,
    // Store-exclusive data/status register overlap
    Unpredictable_DATAOVERLAP,
    // Load-store alignment checks
    Unpredictable_DEVPAGE2,
    // Instruction fetch from Device memory
    Unpredictable_INSTRDEVICE,
    // Reserved CPACR value
    Unpredictable_RESCPACR,
    // Reserved MAIR value
    Unpredictable_RESMAIR,
    // Effect of SCTLr_ELx.C on Tagged attribute
    Unpredictable_S1CTAGGED,
    // Reserved Stage 2 MemAttr value
    Unpredictable_S2RESMEMATTR,
    // Reserved TEX:::C:::B value
    Unpredictable_RESTEXCB,
    // Reserved PRRR value
    Unpredictable_RESPRRR,
    // Reserved DCCR field
    Unpredictable_RESDACR,
    // Reserved VTCR.S value
    Unpredictable_RESVTCRS,
    // Reserved TCR.TnSZ value
    Unpredictable_RESTnSZ,
    // Reserved SCTLr_ELx.TCF value
    Unpredictable_RESTCF,
    // Tag stored to Device memory
    Unpredictable_DEVICETAGSTORE,
    // Out-of-range TCR.TnSZ value
    Unpredictable_OORTnSZ,

    // IPA size exceeds PA size
    Unpredictable_LARGEIPA,
    // Syndrome for a known-passing conditional A32 instruction
    Unpredictable_ESRCONDPASS,
    // Illegal State exception: zero PSTATE.IT
    Unpredictable_ILZEROIT,
    // Illegal State exception: zero PSTATE.T
    Unpredictable_ILZEROT,
    // Debug: prioritization of Vector Catch
    Unpredictable_BPVECTORCATCHPRI,
    // Debug Vector Catch: match on 2nd halfword
    Unpredictable_VCMATCHHALF,
    // Debug Vector Catch: match on Data Abort
    // or Prefetch abort
    Unpredictable_VCMATCHDAPA,
    // Debug watchpoints: nonzero MASK and non-ones BAS
    Unpredictable_WPMASKANDBAS,
    // Debug watchpoints: non-contiguous BAS
    Unpredictable_WPBASCONTIGUOUS,
    // Debug watchpoints: reserved MASK
    Unpredictable_RESWPMASK,
    // Debug watchpoints: nonzero MASKed bits of address
    Unpredictable_WPMASKEDBITS,
    // Debug breakpoints and watchpoints: reserved control bits
    Unpredictable_RESBPWPCTRL,
    // Debug breakpoints: not implemented
    Unpredictable_BPNOTIMPL,
    // Debug breakpoints: reserved type
    Unpredictable_RESBPTYPE,
    // Debug breakpoints and watchpoints: reserved MDSELR_EL1.BANK
    Unpredictable_RESMDSELR,
    // Debug breakpoints: not-context-aware breakpoint
    Unpredictable_BPNOTCTXCMP,
    // Debug breakpoints: match on 2nd halfword of instruction
    Unpredictable_BPMATCHHALF,
    // Debug breakpoints: mismatch on 2nd halfword of instruction
    Unpredictable_BPMISMATCHHALF,
    // Debug breakpoints: a breakpoint is linked to that is not
    // programmed with linking enabled
    Unpredictable_BPLINKINGDISABLED,
    // Debug breakpoints: reserved MASK
    Unpredictable_RESBPMASK,
    // Debug breakpoints: MASK is set for a Context matching

```



```

// breakpoint or when DBGBCR_EL1[4:n].1-BAS != '1111'
Unpredictable_BPMASK,
// Debug breakpoints: nonzero MASKED bits of address
Unpredictable_BPMASKEDBITS,
// Debug breakpoints: A linked breakpoint is
// linked to an address matching breakpoint
Unpredictable_BPLINKEDADDRMATCH,
// Debug: restart to a misaligned AArch32 PC value
Unpredictable_RESTARTALIGNPC,
// Debug: restart to a not-zero-extended AArch32 PC value
Unpredictable_RESTARTZEROUPPERPC,
// Zero top 32 bits of X registers in AArch32 state
Unpredictable_ZEROUPPER,
// Zero top 32 bits of PC on illegal return to
// AArch32 state
Unpredictable_ERETZEROUPPERPC,
// Force address to be aligned when interworking
// branch to A32 state
Unpredictable_A32FORCEALIGNPC,
// SMC disabled
Unpredictable_SMD,
// FF speculation
Unpredictable_NONFAULT,
// Zero top bits of Z registers in EL change
Unpredictable_SVEZEROUPPER,
// Load mem data in NF loads
Unpredictable_SVELDNFDATA,
// Write zeros in NF loads
Unpredictable_SVELDNFZERO,
// SP alignment fault when predicate is all zero
Unpredictable_CHECKSPNONEACTIVE,
// Zero top bits of ZA registers in EL change
Unpredictable_SMEZEROUPPER,
// Watchpoint match of last rounded up memory access in case of
// 16 byte rounding
Unpredictable_16BYTEROUNDEDUPACCESS,
// Watchpoint match of first rounded down memory access in case of
// 16 byte rounding
Unpredictable_16BYTEROUNDEDOWNACCESS,
// HCR_EL2.[HCR_EL2().NV,NV1] == '01'
Unpredictable_NVNV1,
// Upper bits of a BADDR are not RESS
Unpredictable_BADDR_RESS,
// Reserved shareability encoding
Unpredictable_Shareability,
// Access Flag Update by HW
Unpredictable_AFUPDATE,
// Dirty Bit State Update by HW
Unpredictable_DBUPDATE,
// UnsupportedConsider memorySCTLR_ELx[].IESB attributes in forDebug HW updatestate
Unpredictable_Unsupported_Atomic_HW_UpdateUnpredictable_IESBinDebug,
// ConsiderBad SCTLR_ELx().IESBsettings infor Debug statePMSFCR_EL1/PMSEVFR_EL1/PMSLATFR_EL1
Unpredictable_IESBinDebugUnpredictable_BADPMSFCR,
// TheZero HCR_EL2.TERRsaved /BType HCR2.TERRvalue andin SCR_EL3.TERR SPSR_ELx/ SCR.TERR
// trap control can be treated as RES0.DPSR_EL0
Unpredictable_MINIMALRASv2Unpredictable_ZEROBTYPE,
// BadTimestamp settingsconstrained forto PMSFCR_EL1/PMSEVFR_EL1/PMSLATFR_EL1virtual-or-
physical
Unpredictable_BADPMSFCRUnpredictable_EL2TIMESTAMP,
// Zero saved BType value in SPSR_ELx/DPSR_EL0
Unpredictable_ZEROBTYPEUnpredictable_EL1TIMESTAMP,
// TimestampReserved constrainedMDCR_EL3().[NSTBE,NSTB] tooor virtualMDCR_EL3().[NSPBE,NSPB]
or physicalvalue
Unpredictable_EL2TIMESTAMPUnpredictable_RESERVEDNSxB,
Unpredictable_EL1TIMESTAMPUnpredictable_RESERVEDNSxB_Trap,
// ReservedWFET MDCR_EL3.[NSTBE,NSTB] or MDCR_EL3.[NSPBE,NSPB]WFET valueinstruction in-
Debug-state
Unpredictable_RESERVEDNSxBUnpredictable_WFXTDEBUG,
// Address does not support LS64 instructions
Unpredictable_RESERVEDNSxB_TrapUnpredictable_LS64UNSUPPORTED,
// WFETUnalignedorexclusives, WFITatomics, instructionacquire/release
// into Debuga stateregion that is not to Normal inner write-back
// outer write-back shareable generate an Alignment fault.
Unpredictable_WFXTDEBUGUnpredictable_LSE2_ALIGNMENT_FAULT,
// Address128-bit doesAtomic notor support128-bit LS64RCW(S) instructionstransfer register
overlap
Unpredictable_LS64UNSUPPORTEDUnpredictable_LSE128OVERLAP,
// UnalignedClearing exclusives, atomics, acquireDCC/release
// ITR tosticky aflags regionwhen thatinstruction is notin to Normal inner write-back
// outer write-back shareable generate an Alignment fault.flight
Unpredictable_LSE2_ALIGNMENT_FAULTUnpredictable_CLEARERRITEZERO,
// 128-bitALUEXCEPTIONRETURN Atomicwhen orin 128-bituser/system RCW(S)mode transferin
// registerA32 overlapinstructions
Unpredictable_LSE128OVERLAPUnpredictable_ALUEXCEPTIONRETURN,
// ClearingTrap DCC/ITRto stickyregister flagsin whendebug instructionstate isare in
flightignored
Unpredictable_CLEARERRITEZEROUnpredictable_IGNORETRAPINDEBUG,
// ALUEXCEPTIONRETURNCompare whenDBGBVR.RESS infor userBF/system mode in
// A32 instructionsWF

```

```

Unpredictable_ALUExceptionReturnUnpredictable_DBGxVR_RESS,
// TrapInaccessible to event register in debug state are ignoredcounter
Unpredictable_IGNORETRAPINDEBUGUnpredictable_PMEVENTCOUNTER,
// CompareReserved DBGxVR.RESSPMSCR.PCT for BP/WPbehavior
Unpredictable_DBGxVR_RESSUnpredictable_PMSCR_PCT,
// InaccessibleMDCR_EL2.HPMN eventor counterHDCR.HPMN is larger than PMCR.N or
// FEAT_HPMN0 is not implemented and HPMN is 0.
Unpredictable_PMEVENTCOUNTERUnpredictable_RES_HPMN,
// ReservedChained PMSCR.PCTPMU behaviorcounters idx and idx+1 are not in same range
Unpredictable_PMSCR_PCTUnpredictable_COUNT_CHAIN,
// MDCR_EL2.HPMNPMCCR.EPMN or HDCR.HPMN is larger than PMCR.N or
// FEAT_HPMN0 is not implemented and HPMN is 0.
Unpredictable_RES_HPMNUnpredictable_RES_EPMN,
// ChainedGenerate PMUBRB_FILTRATE countersevent idxon andBRB idx+1 are not in same
rangeinjection
Unpredictable_COUNT_CHAINUnpredictable_BRBFILTRATE,
// PMCCR.EPMNGenerate isPMU_SNAPSHOT larger event thanin PMCR.NDebug state
Unpredictable_RES_EPMNUnpredictable_PMUSNAPSHOTEVENT,
// GenerateReserved BRB_FILTRATEMDCR_EL3.EPMSSAD event on BRB injectionvalue
Unpredictable_BRBFILTRATEUnpredictable_RESEPMSSAD,
// GenerateReserved PMU_SNAPSHOTPMECR_EL1.SSE event in Debug statevalue
Unpredictable_PMUSNAPSHOTEVENTUnpredictable_RESPMSSE,
// ReservedEnable MDCR_EL3.EPMSSADfor valuePMU Profiling exception and PMUIRQ
Unpredictable_RESEPMSSADUnpredictable_RESPMEE,
// ReservedEnables PMECR_EL1.SSEfor valuesPE Profiling exceptions and PMSIRQ
Unpredictable_RESPMSSEUnpredictable_RESPMGEE,
// EnableEnables for PMUTRBE Profiling exceptionexceptions and PMUIRQPMSIRQ
Unpredictable_RESPMEEUnpredictable_RESTRFEE,
// EnablesOperands for SPECPY*/SET* Profilinginstructions exceptions and PMSIRQoverlap
Unpredictable_RESPMSEEUnpredictable_MOPSOVERLAP,
// EnablesOperands for TRBECOPY*/SET* Profilinginstructions exceptionsuse and0b11111
// PMSIRQas a register specifier
Unpredictable_RESTRFEEUnpredictable_MOPS_R31,
// OperandsChooses forwhich CPY*/SET*value instructions to overlapreturn in a non failed
Atomic Compare and Swap
Unpredictable_MOPSOVERLAPUnpredictable_CASRETURNOLDVALUE,
// OperandsEnables forwrite CPY*/SET*of instructions the use newvalue 0b11111
// in as a register failed specifierAtomic Compare and Swap
Unpredictable_MOPS_R31Unpredictable_WRITEFAILEDDCAS,
// ChoosesStore-only whichTag valuechecking toon return in a non failed Atomic Compare and
Swap
Unpredictable_CASRETURNOLDVALUEUnpredictable_STRONLYTAGCHECKEDCAS,
// EnablesStore-only writeTag ofchecking theon newvalues in failed aRCW(S) failedchecks
// or RCW(S) Atomic Compare and Swap
Unpredictable_WRITEFAILEDDCASUnpredictable_STRONLYTAGCHECKEDRCWSCAS,
// Store-onlyReserved TagMDCR_EL3.ETBAD checking on a failed Atomic Compare and Swapvalue
Unpredictable_STRONLYTAGCHECKEDCASUnpredictable_RES_ETBAD,
// Store-onlyInvalid TagStreaming checkingMode onfilter a failed RCW(S) checks
// or RCW(S) Atomic Compare and Swapbits
Unpredictable_STRONLYTAGCHECKEDRCWSCASUnpredictable_RES_PMU_VS,
// ReservedApply MDCR_EL3.ETBADChecked valuePointer Arithmetic on a sequential access to
bytes
// that cross the 0xFFFF_FFFF_FFFF_FFFF boundary.
Unpredictable_RES_ETBADUnpredictable_CPACHECK,
// InvalidReserved StreamingPMEVTYPER_ELO(n).[TC,TE,TC2] Mode filter bitsvalues
Unpredictable_RES_PMU_VSUnpredictable_RESTC,
// ApplyWhen CheckedFEAT_MTE Pointeris Arithmeticimplemented, onif aMemory sequential access
to mode bytesis enabled
// thatand crossPSTATE.TCO is 0, Reads and writes to the 0xFFFF_FFFF_FFFF_FFFFexternal
boundary debug
// interface DTR registers are CONSTRAINED UNPREDICTABLE for tagecheck
Unpredictable_CPACHECKUnpredictable_NODTRTAGCHK,
// ReservedUse PMEVTYPER[n]_ELO.[TC,TE,TC2] the valuesdefault PMG when the default PARTID is
generated
// due to MPAM error
Unpredictable_RESTCUnpredictable_USE_DEFAULT_PMG,
// WhenIf FEAT_MTEthe isatomic implemented, instructions ifare Memorynot accessatomic modein
is regard enabled to other
// andagents PSTATE.TCOthat isaccess 0memory, Reads and then writes the to instruction the can
external have debug one or
// interface more DTR of registers the are following CONSTRAINED UNPREDICTABLE for
tagecheck effects
Unpredictable_NODTRTAGCHKUnpredictable_Atomic_SYNC_ABORT,
// Use the default PMG when the default PARTID is generated
// due to MPAM error
Unpredictable_USE_DEFAULT_PMGUnpredictable_Atomic_ERROR,
// If the atomic instructions are not atomic in regard to other
// agents that access memory, then the instruction can have one or
// more of the following effects
Unpredictable_Atomic_SYNC_ABORTUnpredictable_Atomic_NOF,
Unpredictable_Atomic_ERRORUnpredictable_Atomic_MMU_IMPDEF_FAULT,
// Truncate virtual identifier to bottom 12 bits if it exceeds 2^12
Unpredictable_Atomic_NOPUnpredictable_MPAM_TRUNCATE_VID,
// Accessing DBGDSCRint via MRC in debug state
Unpredictable_Atomic_MMU_IMPDEF_FAULTUnpredictable_MRC_APSR_TARGET,
// TruncateAccessing virtualBanked identifierregister to not bottom accessible 12 from bits the
if PE it exceeds 2^12 mode
Unpredictable_MPAM_TRUNCATE_VIDUnpredictable_BankedRegister,
// Accessing DBGDSCRintunimplemented viaBanked MRC in debug state register

```

```

Unpredictable_MRC_APSR_TARGETUnpredictable_UnimplementedRegister,
// Accessing Banked register not accessible from the PE mode
+;
Unpredictable_BankedRegister,
// Accessing unimplemented Banked register
Unpredictable_UnimplementedRegister,
};

```

Library pseudocode for shared/trace/TraceBuffer/CheckMDCR_EL3_NSTBTrap

```

// CheckMDCR_EL3_NSTBTrap()
// =====
// Check if the register access is trappable by MDCR_EL3.[NSTBE, NSTB]

func CheckMDCR_EL3_NSTBTrap() => boolean
begin
    var state_bits : bits(3);
    var reserved    : boolean;
    (state_bits, reserved) = EffectiveMDCR_EL3_NSTB();
    return ((reserved && ConstrainUnpredictableBool(Unpredictable_RESERVEDNSxB_Trap)) ||
            state_bits[0] == '0' || state_bits[1] != SCR_EL3().NS ||
            (IsFeatureImplemented(FEAT_RME) && state_bits[2] != SCR_EL3().NSE));
end; EffectiveSCR_EL3_NS() ||
        (IsFeatureImplemented(FEAT_RME) && state_bits[2] != EffectiveSCR_EL3_NSE());
end;

```

Library pseudocode for shared/trace/selfhosted/SelfHostedTraceEnabled

```

// SelfHostedTraceEnabled()
// =====
// Returns TRUE if Self-hosted Trace is enabled.

readonly func SelfHostedTraceEnabled() => boolean
begin
    var secure_trace_enable : bit = '0';
    if !(HaveTraceExt() && IsFeatureImplemented(FEAT_TRF)) then return FALSE; end;
    if EDSCR().TFO == '0' then return TRUE; end;
    if IsFeatureImplemented(FEAT_RME) then
        secure_trace_enable = if IsFeatureImplemented(FEAT_SEL2) then MDCR_EL3().STE else '0';
        return ((secure_trace_enable == '1' && !ExternalSecureNoninvasiveDebugEnabled()) ||
                (MDCR_EL3().RLTE == '1' && !ExternalRealmNoninvasiveDebugEnabled()));
    end;
    if HaveEL(EL3) then
        secure_trace_enable = if ELUsingAArch32(EL3) then SDCR().STE else MDCR_EL3().STE;
    else
        secure_trace_enable = if SecureOnlyImplementation() then '1' else '0';
    end;

    if secure_trace_enable == '1' && !ExternalSecureNoninvasiveDebugEnabled() then
        return TRUE;
    end;

    return FALSE;
end;

```

Library pseudocode for shared/translation/gpc/DecodeGPTBlock

```

// DecodeGPTBlock()
// =====
// Decode a GPT Block descriptor.

func DecodeGPTBlock(pgs : PGSe, gpt_entry : bits(64)) => GPTEntry
begin
    assert gpt_entry[3:0] == GPT_Block;
    var result : GPTEntry;
    result.gpi = gpt_entry[7:4];
    result.level = 0;
    result.istable = FALSE;

    // GPT information from a level 0 GPT Block descriptor is permitted
    // to be cached in a TLB as though the Block is a contiguous region
    // of granules each of the size configured in GPCCR_EL3.PGS.
    case pgs of
        when PGS_4KB => result.size = GPTRange_4KB;
        when PGS_16KB => result.size = GPTRange_16KB;
        when PGS_64KB => result.size = GPTRange_64KB;
        otherwise => unreachable;
    end;
    result.contig_size = GPTL0Size();

    return result;
end;

```

Library pseudocode for shared/translation/gpc/DecodeGPTContiguous

```
// DecodeGPTContiguous()
// =====
// Decode a GPT Contiguous descriptor.

func DecodeGPTContiguous(pgs : PGSe, gpt_entry : bits(64)) => GPTEntry
begin
  assert gpt_entry[3:0] == GPT_Contig;
  var result : GPTEntry;
  result.gpi = gpt_entry[7:4];

  case pgs of
    when PGS_4KB => result.size = GPTRange_4KB;
    when PGS_16KB => result.size = GPTRange_16KB;
    when PGS_64KB => result.size = GPTRange_64KB;
    otherwise => unreachable;
  end;

  case gpt_entry[9:8] of
    when '01' => result.contig_size = GPTRange_2MB;
    when '10' => result.contig_size = GPTRange_32MB;
    when '11' => result.contig_size = GPTRange_512MB;
    otherwise => unreachable;
  end;

  result.level = 1;
  result.istable = FALSE;

  return result;
end;
```

Library pseudocode for shared/translation/gpc/DecodeGPTGranules

```
// DecodeGPTGranules()
// =====
// Decode a GPT Granules descriptor.

func DecodeGPTGranules(pgs : PGSe, index : integer, gpt_entry : bits(64)) => GPTEntry
begin
  var result : GPTEntry;
  result.gpi = gpt_entry[index*4 +:4];

  case pgs of
    when PGS_4KB => result.size = GPTRange_4KB;
    when PGS_16KB => result.size = GPTRange_16KB;
    when PGS_64KB => result.size = GPTRange_64KB;
    otherwise => unreachable;
  end;

  result.contig_size = result.size; // No contiguity
  result.level = 1;
  result.istable = FALSE;

  return result;
end;
```

Library pseudocode for shared/translation/gpc/DecodeGPTTable

```
// DecodeGPTTable()
// =====
// Decode a GPT Table descriptor.

func DecodeGPTTable(pgs : PGSe, gpt_entry : bits(64)) => GPTTable
begin
    assert gpt_entry[3:0] == GPT_Table;
    var result : GPTTable;

    // Descriptor bits [LOGPTSZ-PGS-2:12] are RES0
    var s : integer{30, 34, 36, 39};
    case GPTL0Size() of
        when GPTRange_1GB => s = 30;
        when GPTRange_16GB => s = 34;
        when GPTRange_64GB => s = 36;
        when GPTRange_512GB => s = 39;
        otherwise => unreachable;
    end;

    var p : integer{12, 14, 16};
    case pgs of
        when PGS_4KB => result.address = 12; gpt_entry[55:17]::Zeros(17);
        when PGS_16KB => result.address = 14; gpt_entry[55:15]::Zeros(15);
        when PGS_64KB => result.address = 16; gpt_entry[55:13]::Zeros(13);
        otherwise => unreachable;
    end;

    let align_bits : integer{} = (s - p) - 1;
    result.address = AlignDownP2(gpt_entry[55:0], align_bits);

    return result;
end;
```

Library pseudocode for shared/translation/gpc/GPTEntry

```
// GPTEntry
// =====

type GPTEntry of record {
    gpi : bits(4), // GPI value for this region
    size : AddressSize, // Region size
    contig_size : AddressSize, // Contiguous region size
    istable : level boolean, // Flag a table entry
    level : integer, // Level of GPT lookup
    pa : bits(56) // PA uniquely identifying the GPT entry
};
```

Library pseudocode for shared/translation/gpc/GPTTableDescriptorValid

```
// GPTTableDescriptorValid()
// =====
// Returns TRUE if the given GPT Table descriptor is valid, and FALSE otherwise.

func GPTTableDescriptorValid(level_0_entry : bits(64)) => boolean
begin
    assert level_0_entry[3:0] == GPT_Table;
    let l0sz : integer{} = GPTL0Size();
    let pgs : PGSe = DecodePGS(GPCCR_EL3().PGS);
    let p : integer{} = DecodePGSRange(pgs);
    let top : integer{} = if DecodePPS() == 56 then 56 else 52;
    return IsZero(level_0_entry[63:top52, 11:4]) && IsZero(level_0_entry[(l0sz-p)-2:12]);
end;
```

Library pseudocode for srmask/EffectiveACTLRMASK_EL1

```
// EffectiveACTLRMASK_EL1()
// =====
// Return the effective value of ACTLRMASK_EL1.

func EffectiveACTLRMASK_EL1() => ACTLR_EL1_Type
begin
    if !IsFeatureImplemented(FEAT_SRMASK) then return Zeros(64); end;
    if HaveEL(EL3) && SCR_EL3().SRMASKEn == '0' then return Zeros(64); end;
    if EL2Enabled() && (!IsHCRXEL2Enabled() || HCRX_EL2().SRMASKEn == '0') then
        return Zeros(64);
    end;
    let mask : ACTLR_EL1_Type = ImpDefBits(64)("ACTLR_EL1 layout");

    return mask;
end;
```

Library pseudocode for srmask/EffectiveACTLRMASK_EL2

```
// EffectiveACTLRMASK_EL2()
// =====
// Return the effective value of ACTLRMASK_EL2.

func EffectiveACTLRMASK_EL2() => ACTLR_EL2_Type
begin
    if !IsFeatureImplemented(FEAT_SRMASK) then return Zeros{64}; end;
    if HaveEL(EL3) && SCR_EL3().SRMASKEn == '0' then return Zeros{64}; end;
    let mask : ACTLR_EL2_Type = ImpDefBits{64}("ACTLR_EL2 layout");

    return mask;
end;
```

Library pseudocode for srmask/EffectiveCPACRMASK_EL1

```
// EffectiveCPACRMASK_EL1()
// =====
// Return the effective value of CPACRMASK_EL1.

func EffectiveCPACRMASK_EL1() => CPACR_EL1_Type
begin
    if !IsFeatureImplemented(FEAT_SRMASK) then return Zeros{64}; end;
    if HaveEL(EL3) && SCR_EL3().SRMASKEn == '0' then return Zeros{64}; end;
    if EL2Enabled() && (!IsHCRXEL2Enabled() || HCRX_EL2().SRMASKEn == '0') then
        return Zeros{64};
    end;
    var mask : CPACR_EL1_Type = Ones{64};
    let mask_reg : CPACRMASK_EL1_Type = CPACRMASK_EL1();

    mask.TCPAC = mask_reg.TCPAC;
    mask.TAM = mask_reg.TAM;
    mask.E0POE = mask_reg.E0POE;
    mask.TTA = mask_reg.TTA;
    mask.SMEN = SignExtend{2}(mask_reg.SMEN);
    mask.FPEN = SignExtend{2}(mask_reg.FPEN);
    mask.ZEN = SignExtend{2}(mask_reg.ZEN);
    mask[32+: 32] = Zeros{32};
    mask[26+: 2] = Zeros{2};
    mask[22+: 2] = Zeros{2};
    mask[18+: 2] = Zeros{2};
    mask[0+: 16] = Zeros{16};
    return mask;
end;
```

Library pseudocode for srmask/EffectiveCPTRMASK_EL2

```
// EffectiveCPTRMASK_EL2()
// =====
// Return the effective value of CPTRMASK_EL2.

func EffectiveCPTRMASK_EL2() => CPTR_EL2_Type
begin
    if !IsFeatureImplemented(FEAT_SRMASK) then return Zeros(64); end;
    if HaveEL(EL3) && SCR_EL3().SRMASKEn == '0' then return Zeros(64); end;
    var mask : CPTR_EL2_Type = Ones(64);
    let mask_reg : CPTRMASK_EL2_Type = CPTRMASK_EL2();

    if ELIsInHost(EL2) then
        mask.TCPAC = mask_reg.TCPAC;
        mask.TAM = mask_reg.TAM;
        mask.E0POE = mask_reg.E0POE;
        mask.TTA = mask_reg.TTA;
        mask.SMEN = SignExtend(2)(mask_reg.SMEN);
        mask.FPEN = SignExtend(2)(mask_reg.FPEN);
        mask.ZEN = SignExtend(2)(mask_reg.ZEN);
        mask[32+: 32] = Zeros(32);
        mask[26+: 2] = Zeros(2);
        mask[22+: 2] = Zeros(2);
        mask[18+: 2] = Zeros(2);
        mask[0+: 16] = Zeros(16);
    else
        mask.TCPAC = mask_reg.TCPAC;
        mask.TAM = mask_reg.TAM;
        mask.TTA = mask_reg.TTA;
        mask.TSM = mask_reg.TSM;
        mask.TFP = mask_reg.TFP;
        mask.TZ = mask_reg.TZ;
        mask[32+: 32] = Zeros(32);
        mask[21+: 9] = Zeros(9);
        mask[14+: 6] = Zeros(6);
        mask[13+: 1] = '0';
        mask[11+: 1] = '0';
        mask[9+: 1] = '0';
        mask[0+: 8] = Zeros(8);
    end;
    return mask;
end;
```

Library pseudocode for srmask/EffectiveHCRMASK_EL2

```
// EffectiveHCRMASK_EL2()
// =====
// Return the effective value of HCRMASK_EL2.

func EffectiveHCRMASK_EL2() => HCR_EL2_Type
begin
    if !IsFeatureImplemented(FEAT_SRMASK2) then return Zeros{64}; end;
    if HaveEL(EL3) && SCR_EL3().SRMASKEn == '0' then return Zeros{64}; end;
    var mask : HCR_EL2_Type = Ones{64};
    let mask_reg : HCRMASK_EL2_Type = HCRMASK_EL2();

    mask.TWEDEL = SignExtend{4}(mask_reg.TWEDEL);
    mask.TWEDEn = mask_reg.TWEDEn;
    mask.TID5 = mask_reg.TID5;
    mask.DCT = mask_reg.DCT;
    mask.ATA = mask_reg.ATA;
    mask.TTLBOS = mask_reg.TTLBOS;
    mask.TTLBIS = mask_reg.TTLBIS;
    mask.EnSCXT = mask_reg.EnSCXT;
    mask.TOCU = mask_reg.TOCU;
    mask.AMVOFFEN = mask_reg.AMVOFFEN;
    mask.TICAB = mask_reg.TICAB;
    mask.TID4 = mask_reg.TID4;
    mask.GPF = mask_reg.GPF;
    mask.FIEN = mask_reg.FIEN;
    mask.FWB = mask_reg.FWB;
    mask.NV2 = mask_reg.NV2;
    mask.AT = mask_reg.AT;
    mask.NV1 = mask_reg.NV1;
    mask.NV = mask_reg.NV;
    mask.API = mask_reg.API;
    mask.APK = mask_reg.APK;
    mask.TEA = mask_reg.TEA;
    mask.TERR = mask_reg.TERR;
    mask.TLOR = mask_reg.TLOR;
    mask.E2H = mask_reg.E2H;
    mask.ID = mask_reg.ID;
    mask.CD = mask_reg.CD;
    mask.RW = mask_reg.RW;
    mask.TRVM = mask_reg.TRVM;
    mask.HCD = mask_reg.HCD;
    mask.TDZ = mask_reg.TDZ;
    mask.TGE = mask_reg.TGE;
    mask.TVM = mask_reg.TVM;
    mask.TTLB = mask_reg.TTLB;
    mask.TPU = mask_reg.TPU;
    mask.TPCP = mask_reg.TPCP;
    mask.TSW = mask_reg.TSW;
    mask.TACR = mask_reg.TACR;
    mask.TIDCP = mask_reg.TIDCP;
    mask.TSC = mask_reg.TSC;
    mask.TID3 = mask_reg.TID3;
    mask.TID2 = mask_reg.TID2;
    mask.TID1 = mask_reg.TID1;
    mask.TID0 = mask_reg.TID0;
    mask.TWE = mask_reg.TWE;
    mask.TWI = mask_reg.TWI;
    mask.DC = mask_reg.DC;
    mask.BSU = SignExtend{2}(mask_reg.BSU);
    mask.FB = mask_reg.FB;
    mask.VSE = mask_reg.VSE;
    mask.VI = mask_reg.VI;
    mask.VF = mask_reg.VF;
    mask.AMO = mask_reg.AMO;
    mask.IMO = mask_reg.IMO;
    mask.FMO = mask_reg.FMO;
    mask.PTW = mask_reg.PTW;
    mask.SWIO = mask_reg.SWIO;
    mask.VM = mask_reg.VM;
    mask[38+: 2] = Zeros{2};
    return mask;
end;
```


Library pseudocode for srmask/EffectiveHCRXMASK_EL2

```
// EffectiveHCRXMASK_EL2()
// =====
// Return the effective value of HCRXMASK_EL2.

func EffectiveHCRXMASK_EL2() => HCRX_EL2_Type
begin
    if !IsFeatureImplemented(FEAT_SRMASK2) then return Zeros(64); end;
    if HaveEL(EL3) && SCR_EL3().SRMASKEn == '0' then return Zeros(64); end;
    var mask : HCRX_EL2_Type = Ones(64);
    let mask_reg : HCRXMASK_EL2_Type = HCRXMASK_EL2();

    mask.FNB = mask_reg.FNB;
    mask.VTLBIDSEn = mask_reg.VTLBIDSEn;
    mask.NVnTTLBOS = mask_reg.NVnTTLBOS;
    mask.NVnTTLBIS = mask_reg.NVnTTLBIS;
    mask.NVnTLTB = mask_reg.NVnTLTB;
    mask.FDIT = mask_reg.FDIT;
    mask.NVTGE = mask_reg.NVTGE;
    mask.SRMASKEn = mask_reg.SRMASKEn;
    mask.VTLBIDEn = mask_reg.VTLBIDEn;
    mask.PACMEEn = mask_reg.PACMEEn;
    mask.EnFPM = mask_reg.EnFPM;
    mask.GCSEn = mask_reg.GCSEn;
    mask.EnIDCP128 = mask_reg.EnIDCP128;
    mask.EnSDERR = mask_reg.EnSDERR;
    mask.TMEA = mask_reg.TMEA;
    mask.EnSNERR = mask_reg.EnSNERR;
    mask.D128En = mask_reg.D128En;
    mask.PTTWI = mask_reg.PTTWI;
    mask.SCTLR2En = mask_reg.SCTLR2En;
    mask.TCR2En = mask_reg.TCR2En;
    mask.MSCEn = mask_reg.MSCEn;
    mask.MCE2 = mask_reg.MCE2;
    mask.CMOW = mask_reg.CMOW;
    mask.VFNMI = mask_reg.VFNMI;
    mask.VINMI = mask_reg.VINMI;
    mask.TALLINT = mask_reg.TALLINT;
    mask.SMPME = mask_reg.SMPME;
    mask.FGTnXS = mask_reg.FGTnXS;
    mask.FnXS = mask_reg.FnXS;
    mask.EnASR = mask_reg.EnASR;
    mask.EnALS = mask_reg.EnALS;
    mask.EnAS0 = mask_reg.EnAS0;
    mask[37+: 27] = Zeros(27);
    mask[28+: 3] = Zeros(3);
    mask[12+: 2] = Zeros(2);
    return mask;
end;
```

Library pseudocode for srmask/EffectiveNVHCRMASK_EL2

```
// EffectiveNVHCRMASK_EL2()
// =====
// Return the effective value of NVHCRMASK_EL2.

func EffectiveNVHCRMASK_EL2() => NVHCR_EL2_Type
begin
    if !IsFeatureImplemented(FEAT_SRMASK2) then return Zeros{64}; end;
    if HaveEL(EL3) && SCR_EL3().SRMASKEn == '0' then return Zeros{64}; end;
    var mask : NVHCR_EL2_Type = Ones{64};
    let mask_reg : NVHCRMASK_EL2_Type = NVHCRMASK_EL2();

    mask.TWEDEL = mask_reg.TWEDEL;
    mask.TWEDEn = mask_reg.TWEDEn;
    mask.TID5 = mask_reg.TID5;
    mask.DCT = mask_reg.DCT;
    mask.ATA = mask_reg.ATA;
    mask.EnSCXT = mask_reg.EnSCXT;
    mask.TOCU = mask_reg.TOCU;
    mask.AMVOFFEN = mask_reg.AMVOFFEN;
    mask.TICAB = mask_reg.TICAB;
    mask.TID4 = mask_reg.TID4;
    mask.GPF = mask_reg.GPF;
    mask.FIEN = mask_reg.FIEN;
    mask.FWB = mask_reg.FWB;
    mask.NV2 = mask_reg.NV2;
    mask.AT = mask_reg.AT;
    mask.NV1 = mask_reg.NV1;
    mask.NV = mask_reg.NV;
    mask.API = mask_reg.API;
    mask.APK = mask_reg.APK;
    mask.TEA = mask_reg.TEA;
    mask.TERR = mask_reg.TERR;
    mask.TLOR = mask_reg.TLOR;
    mask.E2H = mask_reg.E2H;
    mask.ID = mask_reg.ID;
    mask.CD = mask_reg.CD;
    mask.RW = mask_reg.RW;
    mask.HCD = mask_reg.HCD;
    mask.TDZ = mask_reg.TDZ;
    mask.TGE = mask_reg.TGE;
    mask.TVM = mask_reg.TVM;
    mask.TTLB = mask_reg.TTLB;
    mask.TPU = mask_reg.TPU;
    mask.TPCP = mask_reg.TPCP;
    mask.TSW = mask_reg.TSW;
    mask.TACR = mask_reg.TACR;
    mask.TIDCP = mask_reg.TIDCP;
    mask.TSC = mask_reg.TSC;
    mask.TID3 = mask_reg.TID3;
    mask.TID2 = mask_reg.TID2;
    mask.TID1 = mask_reg.TID1;
    mask.TID0 = mask_reg.TID0;
    mask.TWE = mask_reg.TWE;
    mask.TWI = mask_reg.TWI;
    mask.DC = mask_reg.DC;
    mask.BSU = mask_reg.BSU;
    mask.FB = mask_reg.FB;
    mask.VSE = mask_reg.VSE;
    mask.VI = mask_reg.VI;
    mask.VF = mask_reg.VF;
    mask.AMO = mask_reg.AMO;
    mask.IMO = mask_reg.IMO;
    mask.FMO = mask_reg.FMO;
    mask.PTW = mask_reg.PTW;
    mask.SWIO = mask_reg.SWIO;
    mask.VM = mask_reg.VM;
    mask[61+: 3] = Zeros{3};
    mask[38+: 2] = Zeros{2};
    mask[11+: 1] = '0';
    return mask;
end;
```

Library pseudocode for srmask/EffectiveNVHCRXMASK_EL2

```
// EffectiveNVHCRXMASK_EL2()
// =====
// Return the effective value of NVHCRXMASK_EL2.

func EffectiveNVHCRXMASK_EL2() => NVHCRX_EL2_Type
begin
    if !IsFeatureImplemented(FEAT_SRMASK2) then return Zeros{64}; end;
    if HaveEL(EL3) && SCR_EL3().SRMASKE == '0' then return Zeros{64}; end;
    var mask : NVHCRX_EL2_Type = Ones{64};
    let mask_reg : NVHCRXMASK_EL2_Type = NVHCRXMASK_EL2();

    mask.FNB = mask_reg.FNB;
    mask.VTLBIDSEn = mask_reg.VTLBIDSEn;
    mask.NVnTTLBOS = mask_reg.NVnTTLBOS;
    mask.NVnTTLBIS = mask_reg.NVnTTLBIS;
    mask.NVnTTLB = mask_reg.NVnTTLB;
    mask.FDIT = mask_reg.FDIT;
    mask.NVTGE = mask_reg.NVTGE;
    mask.SRMASKE = mask_reg.SRMASKE;
    mask.VTLBIDEn = mask_reg.VTLBIDEn;
    mask.PACME = mask_reg.PACME;
    mask.EnFPM = mask_reg.EnFPM;
    mask.GCSE = mask_reg.GCSE;
    mask.EnIDCP128 = mask_reg.EnIDCP128;
    mask.EnSDERR = mask_reg.EnSDERR;
    mask.TMEA = mask_reg.TMEA;
    mask.EnSNERR = mask_reg.EnSNERR;
    mask.D128En = mask_reg.D128En;
    mask.PTTWI = mask_reg.PTTWI;
    mask.SCTLR2En = mask_reg.SCTLR2En;
    mask.TCR2En = mask_reg.TCR2En;
    mask.MSCEn = mask_reg.MSCEn;
    mask.MCE2 = mask_reg.MCE2;
    mask.CMOW = mask_reg.CMOW;
    mask.VFNMI = mask_reg.VFNMI;
    mask.VINMI = mask_reg.VINMI;
    mask.TALLINT = mask_reg.TALLINT;
    mask.SMPME = mask_reg.SMPME;
    mask.FGTnXS = mask_reg.FGTnXS;
    mask.FnXS = mask_reg.FnXS;
    mask.EnASR = mask_reg.EnASR;
    mask.EnALS = mask_reg.EnALS;
    mask.EnAS0 = mask_reg.EnAS0;
    mask[37+: 27] = Zeros{27};
    mask[28+: 3] = Zeros{3};
    mask[12+: 2] = Zeros{2};
    return mask;
end;
```

Library pseudocode for srmask/EffectiveSCTLR2MASK_EL1

```
// EffectiveSCTLR2MASK_EL1()
// =====
// Return the effective value of SCTLR2MASK_EL1.

func EffectiveSCTLR2MASK_EL1() => SCTLR2_EL1_Type
begin
    if !IsFeatureImplemented(FEAT_SRMASK) then return Zeros(64); end;
    if HaveEL(EL3) && SCR_EL3().SRMASKEn == '0' then return Zeros(64); end;
    if EL2Enabled() && (!IsHCRXEL2Enabled() || HCRX_EL2().SRMASKEn == '0') then
        return Zeros(64);
    end;
    var mask : SCTLR2_EL1_Type = Ones(64);
    let mask_reg : SCTLR2MASK_EL1_Type = SCTLR2MASK_EL1();

    mask.TLBOSNIS = mask_reg.TLBOSNIS;
    mask.FDIT = mask_reg.FDIT;
    mask.BTD = mask_reg.BTD;
    mask.BTD0 = mask_reg.BTD0;
    mask.EnIA2 = mask_reg.EnIA2;
    mask.EnIB2 = mask_reg.EnIB2;
    mask.EnDA2 = mask_reg.EnDA2;
    mask.EnDB2 = mask_reg.EnDB2;
    mask.CPTM0 = mask_reg.CPTM0;
    mask.CPTM = mask_reg.CPTM;
    mask.CPTA0 = mask_reg.CPTA0;
    mask.CPTA = mask_reg.CPTA;
    mask.EnPACM0 = mask_reg.EnPACM0;
    mask.EnPACM = mask_reg.EnPACM;
    mask.EnIDCP128 = mask_reg.EnIDCP128;
    mask.EASE = mask_reg.EASE;
    mask.EnANERR = mask_reg.EnANERR;
    mask.EnADERR = mask_reg.EnADERR;
    mask.NMEA = mask_reg.NMEA;
    mask[27+: 37] = Zeros(37);
    mask[13+: 6] = Zeros(6);
    mask[0+: 2] = Zeros(2);
    return mask;
end;
```

Library pseudocode for srmask/EffectiveSCTLR2MASK_EL2

```
// EffectiveSCTLR2MASK_EL2()
// =====
// Return the effective value of SCTLR2MASK_EL2.

func EffectiveSCTLR2MASK_EL2() => SCTLR2_EL2_Type
begin
    if !IsFeatureImplemented(FEAT_SRMASK) then return Zeros(64); end;
    if HaveEL(EL3) && SCR_EL3().SRMASKEn == '0' then return Zeros(64); end;
    var mask : SCTLR2_EL2_Type = Ones(64);
    let mask_reg : SCTLR2MASK_EL2_Type = SCTLR2MASK_EL2();

    mask.TLBOSNIS = mask_reg.TLBOSNIS;
    mask.FDIT = mask_reg.FDIT;
    mask.BTD = mask_reg.BTD;
    mask.BTD0 = mask_reg.BTD0;
    mask.EnIA2 = mask_reg.EnIA2;
    mask.EnIB2 = mask_reg.EnIB2;
    mask.EnDA2 = mask_reg.EnDA2;
    mask.EnDB2 = mask_reg.EnDB2;
    mask.CPTM0 = mask_reg.CPTM0;
    mask.CPTM = mask_reg.CPTM;
    mask.CPTA0 = mask_reg.CPTA0;
    mask.CPTA = mask_reg.CPTA;
    mask.EnPACM0 = mask_reg.EnPACM0;
    mask.EnPACM = mask_reg.EnPACM;
    mask.EnIDCP128 = mask_reg.EnIDCP128;
    mask.EASE = mask_reg.EASE;
    mask.EnANERR = mask_reg.EnANERR;
    mask.EnADERR = mask_reg.EnADERR;
    mask.NMEA = mask_reg.NMEA;
    mask.EMEC = mask_reg.EMEC;
    mask[27+: 37] = Zeros(37);
    mask[13+: 6] = Zeros(6);
    mask[0+: 1] = '0';
    return mask;
end;
```

Library pseudocode for srmask/EffectiveSCTLRMASK_EL1

```
// EffectiveSCTLRMASK_EL1()
// =====
// Return the effective value of SCTLRMASK_EL1.

func EffectiveSCTLRMASK_EL1() => SCTLR_EL1_Type
begin
    if !IsFeatureImplemented(FEAT_SRMASK) then return Zeros(64); end;
    if HaveEL(EL3) && SCR_EL3().SRMASKEn == '0' then return Zeros(64); end;
    if EL2Enabled() && (!IsHCRXEL2Enabled() || HCRX_EL2().SRMASKEn == '0') then
        return Zeros(64);
    end;
    var mask : SCTLR_EL1_Type = Ones(64);
    let mask_reg : SCTLRMASK_EL1_Type = SCTLRMASK_EL1();

    mask.TIDCP = mask_reg.TIDCP;
    mask.SPINTMASK = mask_reg.SPINTMASK;
    mask.NMI = mask_reg.NMI;
    mask.EnTP2 = mask_reg.EnTP2;
    mask.TCSO = mask_reg.TCSO;
    mask.TCSO0 = mask_reg.TCSO0;
    mask.EPAN = mask_reg.EPAN;
    mask.EnALS = mask_reg.EnALS;
    mask.EnAS0 = mask_reg.EnAS0;
    mask.EnASR = mask_reg.EnASR;
    mask.TWEDel = SignExtend(4)(mask_reg.TWEDel);
    mask.TWEDen = mask_reg.TWEDen;
    mask.DSSBS = mask_reg.DSSBS;
    mask.ATA = mask_reg.ATA;
    mask.ATA0 = mask_reg.ATA0;
    mask.TCF = SignExtend(2)(mask_reg.TCF);
    mask.TCF0 = SignExtend(2)(mask_reg.TCF0);
    mask.ITFSB = mask_reg.ITFSB;
    mask.BT1 = mask_reg.BT1;
    mask.BT0 = mask_reg.BT0;
    mask.EnFPM = mask_reg.EnFPM;
    mask.MSCEn = mask_reg.MSCEn;
    mask.CMOW = mask_reg.CMOW;
    mask.EnIA = mask_reg.EnIA;
    mask.EnIB = mask_reg.EnIB;
    mask.LSMAOE = mask_reg.LSMAOE;
    mask.nTLSMD = mask_reg.nTLSMD;
    mask.EnDA = mask_reg.EnDA;
    mask.UCI = mask_reg.UCI;
    mask.EE = mask_reg.EE;
    mask.E0E = mask_reg.E0E;
    mask.SPAN = mask_reg.SPAN;
    mask.EIS = mask_reg.EIS;
    mask.IESB = mask_reg.IESB;
    mask.TSCXT = mask_reg.TSCXT;
    mask.WXN = mask_reg.WXN;
    mask.nTWE = mask_reg.nTWE;
    mask.nTWI = mask_reg.nTWI;
    mask.UCT = mask_reg.UCT;
    mask.DZE = mask_reg.DZE;
    mask.EnDB = mask_reg.EnDB;
    mask.I = mask_reg.I;
    mask.EOS = mask_reg.EOS;
    mask.EnRCTX = mask_reg.EnRCTX;
    mask.UMA = mask_reg.UMA;
    mask.SED = mask_reg.SED;
    mask.ITD = mask_reg.ITD;
    mask.nAA = mask_reg.nAA;
    mask.CP15BEN = mask_reg.CP15BEN;
    mask.SA0 = mask_reg.SA0;
    mask.SA = mask_reg.SA;
    mask.C = mask_reg.C;
    mask.A = mask_reg.A;
    mask.M = mask_reg.M;
    mask[50+: 4] = Zeros(4);
    mask[17+: 1] = '0';
    return mask;
end;
```

Library pseudocode for srmask/EffectiveSCTLRMASK_EL2

```
// EffectiveSCTLRMASK_EL2()
// =====
// Return the effective value of SCTLRMASK_EL2.

func EffectiveSCTLRMASK_EL2() => SCTLR_EL2_Type
begin
    if !IsFeatureImplemented(FEAT_SRMASK) then return Zeros(64); end;
    if HaveEL(EL3) && SCR_EL3().SRMASKEn == '0' then return Zeros(64); end;
    var mask : SCTLR_EL2_Type = Ones(64);
    let mask_reg : SCTLRMASK_EL2_Type = SCTLRMASK_EL2();

    mask.TIDCP = mask_reg.TIDCP;
    mask.SPINTMASK = mask_reg.SPINTMASK;
    mask.NMI = mask_reg.NMI;
    mask.EnTP2 = mask_reg.EnTP2;
    mask.TCSO = mask_reg.TCSO;
    mask.TCSO0 = mask_reg.TCSO0;
    mask.EPAN = mask_reg.EPAN;
    mask.EnALS = mask_reg.EnALS;
    mask.EnAS0 = mask_reg.EnAS0;
    mask.EnASR = mask_reg.EnASR;
    mask.TWEDEL = SignExtend(4)(mask_reg.TWEDEL);
    mask.TWEDEn = mask_reg.TWEDEn;
    mask.DSSBS = mask_reg.DSSBS;
    mask.ATA = mask_reg.ATA;
    mask.ATA0 = mask_reg.ATA0;
    mask.TCF = SignExtend(2)(mask_reg.TCF);
    mask.TCF0 = SignExtend(2)(mask_reg.TCF0);
    mask.ITFSB = mask_reg.ITFSB;
    mask.BT = mask_reg.BT;
    mask.BT0 = mask_reg.BT0;
    mask.EnFPM = mask_reg.EnFPM;
    mask.MSCEn = mask_reg.MSCEn;
    mask.CMOW = mask_reg.CMOW;
    mask.EnIA = mask_reg.EnIA;
    mask.EnIB = mask_reg.EnIB;
    mask.LSMAOE = mask_reg.LSMAOE;
    mask.nTlSMD = mask_reg.nTlSMD;
    mask.EnDA = mask_reg.EnDA;
    mask.UCI = mask_reg.UCI;
    mask.EE = mask_reg.EE;
    mask.E0E = mask_reg.E0E;
    mask.SPAN = mask_reg.SPAN;
    mask.EIS = mask_reg.EIS;
    mask.IESB = mask_reg.IESB;
    mask.TSCXT = mask_reg.TSCXT;
    mask.WXN = mask_reg.WXN;
    mask.nTWE = mask_reg.nTWE;
    mask.nTWI = mask_reg.nTWI;
    mask.UCT = mask_reg.UCT;
    mask.DZE = mask_reg.DZE;
    mask.EnDB = mask_reg.EnDB;
    mask.I = mask_reg.I;
    mask.EOS = mask_reg.EOS;
    mask.EnRCTX = mask_reg.EnRCTX;
    mask.SED = mask_reg.SED;
    mask.ITD = mask_reg.ITD;
    mask.nAA = mask_reg.nAA;
    mask.CP15BEN = mask_reg.CP15BEN;
    mask.SA0 = mask_reg.SA0;
    mask.SA = mask_reg.SA;
    mask.C = mask_reg.C;
    mask.A = mask_reg.A;
    mask.M = mask_reg.M;
    mask[50+: 4] = Zeros(4);
    mask[17+: 1] = '0';
    mask[9+: 1] = '0';
    return mask;
end;
```

Library pseudocode for srmask/EffectiveTCR2MASK_EL1

```
// EffectiveTCR2MASK_EL1()
// =====
// Return the effective value of TCR2MASK_EL1.

func EffectiveTCR2MASK_EL1() => TCR2_EL1_Type
begin
    if !IsFeatureImplemented(FEAT_SRMASK) then return Zeros(64); end;
    if HaveEL(EL3) && SCR_EL3().SRMASKEn == '0' then return Zeros(64); end;
    if EL2Enabled() && (!IsHCRXEL2Enabled() || HCRX_EL2().SRMASKEn == '0') then
        return Zeros(64);
    end;
    var mask : TCR2_EL1_Type = Ones(64);
    let mask_reg : TCR2MASK_EL1_Type = TCR2MASK_EL1();

    mask.FNGNA1 = mask_reg.FNGNA1;
    mask.FNGNA0 = mask_reg.FNGNA0;
    mask.FNG1 = mask_reg.FNG1;
    mask.FNG0 = mask_reg.FNG0;
    mask.A2 = mask_reg.A2;
    mask.DisCH1 = mask_reg.DisCH1;
    mask.DisCH0 = mask_reg.DisCH0;
    mask.HAFT = mask_reg.HAFT;
    mask.PTTWI = mask_reg.PTTWI;
    mask.D128 = mask_reg.D128;
    mask.AIE = mask_reg.AIE;
    mask.POE = mask_reg.POE;
    mask.E0POE = mask_reg.E0POE;
    mask.PIE = mask_reg.PIE;
    mask.PnCH = mask_reg.PnCH;
    mask[22+: 42] = Zeros(42);
    mask[19+: 1] = '0';
    mask[12+: 2] = Zeros(2);
    mask[6+: 4] = Zeros(4);
    return mask;
end;
```

Library pseudocode for srmask/EffectiveTCR2MASK_EL2

```
// EffectiveTCR2MASK_EL2()
// =====
// Return the effective value of TCR2MASK_EL2.

func EffectiveTCR2MASK_EL2() => TCR2_EL2_Type
begin
    if !IsFeatureImplemented(FEAT_SRMASK) then return Zeros(64); end;
    if HaveEL(EL3) && SCR_EL3().SRMASKEn == '0' then return Zeros(64); end;
    var mask : TCR2_EL2_Type = Ones(64);
    let mask_reg : TCR2MASK_EL2_Type = TCR2MASK_EL2();

    if !ELIsinHost(EL2) then
        mask.AMEC0 = mask_reg.AMEC0;
        mask.HAFT = mask_reg.HAFT;
        mask.PTTWI = mask_reg.PTTWI;
        mask.AIE = mask_reg.AIE;
        mask.POE = mask_reg.POE;
        mask.PIE = mask_reg.PIE;
        mask.PnCH = mask_reg.PnCH;
        mask[13+: 51] = Zeros(51);
        mask[5+: 5] = Zeros(5);
        mask[2+: 1] = '0';
    else
        mask.FNG1 = mask_reg.FNG1;
        mask.FNG0 = mask_reg.FNG0;
        mask.A2 = mask_reg.A2;
        mask.DisCH1 = mask_reg.DisCH1;
        mask.DisCH0 = mask_reg.DisCH0;
        mask.AMEC1 = mask_reg.AMEC1;
        mask.AMEC0 = mask_reg.AMEC0;
        mask.HAFT = mask_reg.HAFT;
        mask.PTTWI = mask_reg.PTTWI;
        mask.D128 = mask_reg.D128;
        mask.AIE = mask_reg.AIE;
        mask.POE = mask_reg.POE;
        mask.E0POE = mask_reg.E0POE;
        mask.PIE = mask_reg.PIE;
        mask.PnCH = mask_reg.PnCH;
        mask[19+: 45] = Zeros(45);
        mask[6+: 4] = Zeros(4);
    end;
    return mask;
end;
```

Library pseudocode for srmask/EffectiveTCRMASK_EL1

```
// EffectiveTCRMASK_EL1()
// =====
// Return the effective value of TCRMASK_EL1.

func EffectiveTCRMASK_EL1() => TCR_El1_Type
begin
    if !IsFeatureImplemented(FEAT_SRMASK) then return Zeros(64); end;
    if HaveEL(EL3) && SCR_EL3().SRMASKEn == '0' then return Zeros(64); end;
    if EL2Enabled() && (!IsHCRXEL2Enabled() || HCRX_EL2().SRMASKEn == '0') then
        return Zeros(64);
    end;
    var mask : TCR_El1_Type = Ones(64);
    let mask_reg : TCRMASK_EL1_Type = TCRMASK_EL1();

    mask.MTX1 = mask_reg.MTX1;
    mask.MTX0 = mask_reg.MTX0;
    mask.DS = mask_reg.DS;
    mask.TCMA1 = mask_reg.TCMA1;
    mask.TCMA0 = mask_reg.TCMA0;
    mask.E0PD1 = mask_reg.E0PD1;
    mask.E0PD0 = mask_reg.E0PD0;
    mask.NFD1 = mask_reg.NFD1;
    mask.NFD0 = mask_reg.NFD0;
    mask.TBID1 = mask_reg.TBID1;
    mask.TBID0 = mask_reg.TBID0;
    mask.HWU162 = mask_reg.HWU162;
    mask.HWU161 = mask_reg.HWU161;
    mask.HWU160 = mask_reg.HWU160;
    mask.HWU159 = mask_reg.HWU159;
    mask.HWU062 = mask_reg.HWU062;
    mask.HWU061 = mask_reg.HWU061;
    mask.HWU060 = mask_reg.HWU060;
    mask.HWU059 = mask_reg.HWU059;
    mask.HPD1 = mask_reg.HPD1;
    mask.HPD0 = mask_reg.HPD0;
    mask.HD = mask_reg.HD;
    mask.HA = mask_reg.HA;
    mask.TBI1 = mask_reg.TBI1;
    mask.TBI0 = mask_reg.TBI0;
    mask.AS = mask_reg.AS;
    mask.IPS = SignExtend(3)(mask_reg.IPS);
    mask.TG1 = SignExtend(2)(mask_reg.TG1);
    mask.SH1 = SignExtend(2)(mask_reg.SH1);
    mask.ORG1 = SignExtend(2)(mask_reg.ORG1);
    mask.IRG1 = SignExtend(2)(mask_reg.IRG1);
    mask.EPD1 = mask_reg.EPD1;
    mask.A1 = mask_reg.A1;
    mask.T1SZ = SignExtend(6)(mask_reg.T1SZ);
    mask.TG0 = SignExtend(2)(mask_reg.TG0);
    mask.SH0 = SignExtend(2)(mask_reg.SH0);
    mask.ORG0 = SignExtend(2)(mask_reg.ORG0);
    mask.IRG0 = SignExtend(2)(mask_reg.IRG0);
    mask.EPD0 = mask_reg.EPD0;
    mask.T0SZ = SignExtend(6)(mask_reg.T0SZ);
    mask[62+: 2] = Zeros(2);
    mask[35+: 1] = '0';
    mask[6+: 1] = '0';
    return mask;
end;
```


Library pseudocode for srmask/EffectiveTCRMASK_EL2

```
// EffectiveTCRMASK_EL2()
// =====
// Return the effective value of TCRMASK_EL2.

func EffectiveTCRMASK_EL2() => TCR_EL2_Type
begin
    if !IsFeatureImplemented(FEAT_SRMASK) then return Zeros{64}; end;
    if HaveEL(EL3) && SCR_EL3().SRMASKEn == '0' then return Zeros{64}; end;
    var mask : TCR_EL2_Type = Ones{64};
    let mask_reg : TCRMASK_EL2_Type = TCRMASK_EL2();

    if !ELIsInHost(EL2) then
        mask.MTX = mask_reg.MTX;
        mask.DS = mask_reg.DS;
        mask.TCMA = mask_reg.TCMA;
        mask.TBID = mask_reg.TBID;
        mask.HWU62 = mask_reg.HWU62;
        mask.HWU61 = mask_reg.HWU61;
        mask.HWU60 = mask_reg.HWU60;
        mask.HWU59 = mask_reg.HWU59;
        mask.HPD = mask_reg.HPD;
        mask.HD = mask_reg.HD;
        mask.HA = mask_reg.HA;
        mask.TBI = mask_reg.TBI;
        mask.PS = SignExtend{3}(mask_reg.PS);
        mask.TG0 = SignExtend{2}(mask_reg.TG0);
        mask.SH0 = SignExtend{2}(mask_reg.SH0);
        mask.ORGNO = SignExtend{2}(mask_reg.ORGNO);
        mask.IRGNO = SignExtend{2}(mask_reg.IRGNO);
        mask.T0SZ = SignExtend{6}(mask_reg.T0SZ);
        mask[34+: 30] = Zeros{30};
        mask[31+: 1] = '0';
        mask[23+: 1] = '0';
        mask[19+: 1] = '0';
        mask[6+: 2] = Zeros{2};
    else
        mask.MTX1 = mask_reg.MTX1;
        mask.MTX0 = mask_reg.MTX0;
        mask.DS = mask_reg.DS;
        mask.TCMA1 = mask_reg.TCMA1;
        mask.TCMA0 = mask_reg.TCMA0;
        mask.E0PD1 = mask_reg.E0PD1;
        mask.E0PD0 = mask_reg.E0PD0;
        mask.NFD1 = mask_reg.NFD1;
        mask.NFD0 = mask_reg.NFD0;
        mask.TBID1 = mask_reg.TBID1;
        mask.TBID0 = mask_reg.TBID0;
        mask.HWU162 = mask_reg.HWU162;
        mask.HWU161 = mask_reg.HWU161;
        mask.HWU160 = mask_reg.HWU160;
        mask.HWU159 = mask_reg.HWU159;
        mask.HWU062 = mask_reg.HWU062;
        mask.HWU061 = mask_reg.HWU061;
        mask.HWU060 = mask_reg.HWU060;
        mask.HWU059 = mask_reg.HWU059;
        mask.HPD1 = mask_reg.HPD1;
        mask.HPD0 = mask_reg.HPD0;
        mask.HD = mask_reg.HD;
        mask.HA = mask_reg.HA;
        mask.TBI1 = mask_reg.TBI1;
        mask.TBI0 = mask_reg.TBI0;
        mask.AS = mask_reg.AS;
        mask.IPS = SignExtend{3}(mask_reg.IPS);
        mask.TG1 = SignExtend{2}(mask_reg.TG1);
        mask.SH1 = SignExtend{2}(mask_reg.SH1);
        mask.ORGNO1 = SignExtend{2}(mask_reg.ORGNO1);
        mask.IRGNO1 = SignExtend{2}(mask_reg.IRGNO1);
        mask.EPD1 = mask_reg.EPD1;
        mask.A1 = mask_reg.A1;
        mask.T1SZ = SignExtend{6}(mask_reg.T1SZ);
        mask.TG0 = SignExtend{2}(mask_reg.TG0);
        mask.SH0 = SignExtend{2}(mask_reg.SH0);
        mask.ORGNO = SignExtend{2}(mask_reg.ORGNO);
        mask.IRGNO = SignExtend{2}(mask_reg.IRGNO);
        mask.EPD0 = mask_reg.EPD0;
        mask.T0SZ = SignExtend{6}(mask_reg.T0SZ);
        mask[62+: 2] = Zeros{2};
        mask[35+: 1] = '0';
        mask[6+: 1] = '0';
    end;
    return mask;
end;
```

2025-12_re|2025-09_diff_tag 2025-12-102025-11-28 16:4129:4505

Copyright © 2010-2025 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

